Data Analysis Expressions (DAX) Reference

Data Analysis Expressions (DAX) is a library of functions and operators that can be combined to build formulas and expressions in Power BI, Analysis Services, and Power Pivot in Excel data models.

Functions

REFERENCE

DAX functions

DAX statements

WHAT'S NEW

New DAX functions

Learn about DAX

OVERVIEW

DAX overview

VIDEO

Videos

TRAINING

Use DAX in Power BI Desktop

Understand

CONCEPT

DAX glossary

DAX operators

DAX queries

DAX syntax

DAX overview

Article • 10/20/2023

Data Analysis Expressions (DAX) is a formula expression language used in Analysis Services, Power BI, and Power Pivot in Excel. DAX formulas include functions, operators, and values to perform advanced calculations and queries on data in related tables and columns in tabular data models.

This article provides only a basic introduction to the most important concepts in DAX. It describes DAX as it applies to all the products that use it. Some functionality may not apply to certain products or use cases. Refer to your product's documentation describing its particular implementation of DAX.

Calculations

DAX formulas are used in measures, calculated columns, calculated tables, and row-level security.

Measures

Measures are dynamic calculation formulas where the results change depending on context. Measures are used in reporting that support combining and filtering model data by using multiple attributes such as a Power BI report or Excel PivotTable or PivotChart. Measures are created by using the DAX formula bar in the model designer.

A formula in a measure can use standard aggregation functions automatically created by using the Autosum feature, such as COUNT or SUM, or you can define your own formula by using the DAX formula bar. Named measures can be passed as an argument to other measures.

When you define a formula for a measure in the formula bar, a Tooltip feature shows a preview of what the results would be for the total in the current context, but otherwise the results are not immediately output anywhere. The reason you cannot see the (filtered) results of the calculation immediately is because the result of a measure cannot be determined without context. To evaluate a measure requires a reporting client application that can provide the context needed to retrieve the data relevant to each cell and then evaluate the expression for each cell. That client might be an Excel PivotTable or PivotChart, a Power BI report, or a table expression in a DAX query in SQL Server Management Studio (SSMS).

Regardless of the client, a separate query is run for each cell in the results. That is to say, each combination of row and column headers in a PivotTable, or each selection of slicers and filters in a Power BI report, generates a different subset of data over which the measure is calculated. For example, using this very simple measure formula:

```DAX
Total Sales = SUM([Sales Amount])
```

When a user places the TotalSales measure in a report, and then places the Product Category column from a Product table into Filters, the sum of Sales Amount is calculated and displayed for each product category.

Unlike calculated columns, the syntax for a measure includes the measure's name preceding the formula. In the example just provided, the name **Total Sales** appears preceding the formula. After you've created a measure, the name and its definition appear in the reporting client application Fields list, and depending on perspectives and roles is available to all users of the model.

To learn more, see:
Measures in Power BI Desktop
Measures in Analysis Services
Measures in Power Pivot

Calculated columns

A calculated column is a column that you add to an existing table (in the model designer) and then create a DAX formula that defines the column's values. When a calculated column contains a valid DAX formula, values are calculated for each row as soon as the formula is entered. Values are then stored in the in-memory data model. For example, in a Date table, when the formula is entered into the formula bar:

```DAX
= [Calendar Year] & " Q" & [Calendar Quarter]
```

A value for each row in the table is calculated by taking values from the Calendar Year column (in the same Date table), adding a space and the capital letter Q, and then adding the values from the Calendar Quarter column (in the same Date table). The result for each row in the calculated column is calculated immediately and appears, for example, as **2017 Q1**. Column values are only recalculated if the table or any related

table is processed (refresh) or the model is unloaded from memory and then reloaded, like when closing and reopening a Power BI Desktop file.

To learn more, see:
Calculated columns in Power BI Desktop
Calculated columns in Analysis Services
Calculated Columns in Power Pivot.

Calculated tables

A calculated table is a computed object, based on a formula expression, derived from all or part of other tables in the same model. Instead of querying and loading values into your new table's columns from a data source, a DAX formula defines the table's values.

Calculated tables can be helpful in a role-playing dimension. An example is the Date table, as OrderDate, ShipDate, or DueDate, depending on the foreign key relationship. By creating a calculated table for ShipDate explicitly, you get a standalone table that is available for queries, as fully operable as any other table. Calculated tables are also useful when configuring a filtered rowset, or a subset or superset of columns from other existing tables. This allows you to keep the original table intact while creating variations of that table to support specific scenarios.

Calculated tables support relationships with other tables. The columns in your calculated table have data types, formatting, and can belong to a data category. Calculated tables can be named, and surfaced or hidden just like any other table. Calculated tables are re-calculated if any of the tables it pulls data from are refreshed or updated.

To learn more, see:
Calculated tables in Power BI Desktop
Calculated tables in Analysis Services.

Row-level security

With row-level security, a DAX formula must evaluate to a Boolean TRUE/FALSE condition, defining which rows can be returned by the results of a query by members of a particular role. For example, for members of the Sales role, the Customers table with the following DAX formula:

DAX

```
= Customers[Country] = "USA"
```

Members of the Sales role will only be able to view data for customers in the USA, and aggregates, such as SUM are returned only for customers in the USA. Row-level security is not available in Power Pivot in Excel.

When defining row-level secuirty by using DAX formula, you are creating an allowed row set. This does not deny access to other rows; rather, they are simply not returned as part of the allowed row set. Other roles can allow access to the rows excluded by the DAX formula. If a user is a member of another role, and that role's row-level security allows access to that particular row set, the user can view data for that row.

Row-level security formulas apply to the specified rows as well as related rows. When a table has multiple relationships, filters apply security for the relationship that is active. Row-level security formulas will be intersected with other formulas defined for related tables.

To learn more, see:
Row-level security (RLS) with Power BI
Roles in Analysis Services

Queries

DAX queries can be created and run in SQL Server Management Studio (SSMS) and open-source tools like DAX Studio (daxstudio.org). Unlike DAX calculation formulas, which can only be created in tabular data models, DAX queries can also be run against Analysis Services Multidimensional models. DAX queries are often easier to write and more efficient than Multidimensional Data Expressions (MDX) queries.

A DAX query is a statement, similar to a SELECT statement in T-SQL. The most basic type of DAX query is an *evaluate* statement. For example,

DAX

```
EVALUATE
 ( FILTER ( 'DimProduct', [SafetyStockLevel] < 200 ) )
ORDER BY [EnglishProductName] ASC
```

Returns in Results a table listing only those products with a SafetyStockLevel less than 200, in ascending order by EnglishProductName.

You can create measures as part of the query. Measures exist only for the duration of the query. To learn more, see DAX queries.

Formulas

DAX formulas are essential for creating calculations in calculated columns and measures, and securing your data by using row-level security. To create formulas for calculated columns and measures, use the formula bar along the top of the model designer window or the DAX Editor. To create formulas for row-level security, use the Role Manager or Manage roles dialog box. Information in this section is meant to get you started with understanding the basics of DAX formulas.

Formula basics

DAX formulas can be very simple or quite complex. The following table shows some examples of simple formulas that could be used in a calculated column.

Formula	Definition
`= TODAY()`	Inserts today's date in every row of a calculated column.
`= 3`	Inserts the value 3 in every row of a calculated column.
`= [Column1] + [Column2]`	Adds the values in the same row of [Column1] and [Column2] and puts the results in the calculated column of the same row.

Whether the formula you create is simple or complex, you can use the following steps when building a formula:

1. Each formula must begin with an equal sign (=).
2. You can either type or select a function name, or type an expression.
3. Begin to type the first few letters of the function or name you want, and AutoComplete displays a list of available functions, tables, and columns. Press TAB to add an item from the AutoComplete list to the formula.

   You can also click the **Fx** button to display a list of available functions. To select a function from the dropdown list, use the arrow keys to highlight the item, and click **OK** to add the function to the formula.
4. Supply the arguments to the function by selecting them from a dropdown list of possible tables and columns, or by typing in values.
5. Check for syntax errors: ensure that all parentheses are closed and columns, tables and values are referenced correctly.
6. Press ENTER to accept the formula.

> **Note**
>
> In a calculated column, as soon as you enter the formula and the formula is validated, the column is populated with values. In a measure, pressing ENTER saves the measure definition with the table. If a formula is invalid, an error is displayed.

In this example, let's look at a formula in a measure named **Days in Current Quarter**:

```DAX
Days in Current Quarter = COUNTROWS( DATESBETWEEN( 'Date'[Date],
STARTOFQUARTER( LASTDATE('Date'[Date])), ENDOFQUARTER('Date'[Date])))
```

This measure is used to create a comparison ratio between an incomplete period and the previous period. The formula must take into account the proportion of the period that has elapsed, and compare it to the same proportion in the previous period. In this case, [Days Current Quarter to Date]/[Days in Current Quarter] gives the proportion elapsed in the current period.

This formula contains the following elements:

Formula element	Description
`Days in Current Quarter`	The name of the measure.
`=`	The equals sign (=) begins the formula.
`COUNTROWS`	COUNTROWS counts the number of rows in the Date table
`()`	Open and closing parenthesis specifies arguments.
`DATESBETWEEN`	The DATESBETWEEN function returns the dates between the last date for each value in the Date column in the Date table.
`'Date'`	Specifies the Date table. Tables are in single quotes.
`[Date]`	Specifies the Date column in the Date table. Columns are in brackets.
`,`	
`STARTOFQUARTER`	The STARTOFQUARTER function returns the date of the start of the quarter.
`LASTDATE`	The LASTDATE function returns the last date of the quarter.
`'Date'`	Specifies the Date table.

Formula element	Description
`[Date]`	Specifies the Date column in the Date table.
`,`	
`ENDOFQUARTER`	The ENDOFQUARTER function
`'Date'`	Specifies the Date table.
`[Date]`	Specifies the Date column in the Date table.

Using formula AutoComplete

AutoComplete helps you enter a valid formula syntax by providing you with options for each element in the formula.

- You can use formula AutoComplete in the middle of an existing formula with nested functions. The text immediately before the insertion point is used to display values in the drop-down list, and all of the text after the insertion point remains unchanged.
- AutoComplete does not add the closing parenthesis of functions or automatically match parentheses. You must make sure that each function is syntactically correct or you cannot save or use the formula.

Using multiple functions in a formula

You can nest functions, meaning that you use the results from one function as an argument of another function. You can nest up to 64 levels of functions in calculated columns. However, nesting can make it difficult to create or troubleshoot formulas. Many functions are designed to be used solely as nested functions. These functions return a table, which cannot be directly saved as a result; it must be provided as input to a table function. For example, the functions SUMX, AVERAGEX, and MINX all require a table as the first argument.

Functions

A function is a named formula within an expression. Most functions have required and optional arguments, also known as parameters, as input. When the function is executed, a value is returned. DAX includes functions you can use to perform calculations using dates and times, create conditional values, work with strings, perform lookups based on relationships, and the ability to iterate over a table to perform recursive calculations. If

you are familiar with Excel formulas, many of these functions will appear very similar; however, DAX formulas are different in the following important ways:

- A DAX function always references a complete column or a table. If you want to use only particular values from a table or column, you can add filters to the formula.
- If you need to customize calculations on a row-by-row basis, DAX provides functions that let you use the current row value or a related value as a kind of parameter, to perform calculations that vary by context. To understand how these functions work, see Context in this article.
- DAX includes many functions that return a table, rather than a value. The table is not displayed in a reporting client, but is used to provide input to other functions. For example, you can retrieve a table and then count the distinct values in it, or calculate dynamic sums across filtered tables or columns.
- DAX functions include a variety of *time intelligence* functions. These functions let you define or select date ranges, and perform dynamic calculations based on these dates or range. For example, you can compare sums across parallel periods.

Aggregation functions

Aggregation functions calculate a (scalar) value such as count, sum, average, minimum, or maximum for all rows in a column or table as defined by the expression. To learn more, see Aggregation functions.

Date and time functions

The date and time functions in DAX are similar to date and time functions in Microsoft Excel. However, DAX functions are based on a **datetime** data type starting March 1, 1900. To learn more, see Date and time functions.

Filter functions

The filter functions in DAX return specific data types, look up values in related tales, and filter by related values. The lookup functions work by using tables and relationships, like a database. The filtering functions let you manipulate data context to create dynamic calculations. To learn more, see Filter functions.

Financial functions

The financial functions in DAX are used in formulas that perform financial calculations, such as net present value and rate of return. These functions are similar to financial functions used in Microsoft Excel. To learn more, see Financial functions.

Information functions

An information function looks at the cell or row that is provided as an argument and tells you whether the value matches the expected type. For example, the ISERROR function returns TRUE if the value that you reference contains an error. To learn more, see Information functions.

Logical functions

Logical functions act upon an expression to return information about the values in the expression. For example, the TRUE function lets you know whether an expression that you are evaluating returns a TRUE value. To learn more, see Logical functions.

Mathematical and trigonometric functions

The mathematical functions in DAX are very similar to the Excel mathematical and trigonometric functions. Some minor differences exist in the numeric data types used by DAX functions. To learn more, see Math and trig functions.

Other functions

These functions perform unique actions that cannot be defined by any of the categories most other functions belong to. To learn more, see Other functions.

Relationship functions

Relationship functions in DAX allow you to return values from another related table, specify a particular relationship to use in an expression, and specify cross filtering direction. To learn more, see Relationship functions.

Statistical functions

Statistical functions calculate values related to statistical distributions and probability, such as standard deviation and number of permutations. To learn more, see Statistical functions.

Text functions

Text functions in DAX are very similar to their counterparts in Excel. You can return part of a string, search for text within a string, or concatenate string values. DAX also provides functions for controlling the formats for dates, times, and numbers. To learn more, see Text functions.

Time intelligence functions

The time intelligence functions provided in DAX let you create calculations that use built-in knowledge about calendars and dates. By using time and date ranges in combination with aggregations or calculations, you can build meaningful comparisons across comparable time periods for sales, inventory, and so on. To learn more, see Time intelligence functions (DAX).

Table manipulation functions

These functions return a table or manipulate existing tables. For example, by using ADDCOLUMNS you can add calculated columns to a specified table, or you can return a summary table over a set of groups with the SUMMARIZECOLUMNS function. To learn more, see Table manipulation functions.

Variables

You can create variables within an expression by using VAR. VAR is technically not a function, it's a keyword to store the result of an expression as a named variable. That variable can then be passed as an argument to other measure expressions. For example:

DAX

```
VAR
    TotalQty = SUM ( Sales[Quantity] )

Return

    IF (
        TotalQty > 1000,
        TotalQty * 0.95,
        TotalQty * 1.25
        )
```

In this example, TotalQty can be passed as a named variable to other expressions. Variables can be of any scalar data type, including tables. Using variables in your DAX

formulas can be incredibly powerful.

Data types

You can import data into a model from many different data sources that might support different data types. When you import data into a model, the data is converted to one of the tabular model data types. When the model data is used in a calculation, the data is then converted to a DAX data type for the duration and output of the calculation. When you create a DAX formula, the terms used in the formula will automatically determine the value data type returned.

DAX supports the following data types:

Data type in model	Data type in DAX	Description
Whole Number	A 64 bit (eight-bytes) integer value [1, 2]	Numbers that have no decimal places. Integers can be positive or negative numbers, but must be whole numbers between -9,223,372,036,854,775,808 (-2^63) and 9,223,372,036,854,775,807 (2^63-1).
Decimal Number	A 64 bit (eight-bytes) real number [1, 2]	Real numbers are numbers that can have decimal places. Real numbers cover a wide range of values: Negative values from -1.79E +308 through -2.23E -308 Zero Positive values from 2.23E -308 through 1.79E + 308 However, the number of significant digits is limited to 17 decimal digits.
Boolean	Boolean	Either a True or False value.
Text	String	A Unicode character data string. Can be strings, numbers or dates represented in a text format.
Date	Date/time	Dates and times in an accepted date-time representation. Valid dates are all dates after March 1, 1900.
Currency	Currency	Currency data type allows values between -922,337,203,685,477.5808 to 922,337,203,685,477.5807 with four decimal digits of fixed precision.
N/A	Blank	A blank is a data type in DAX that represents and replaces SQL nulls. You can create a blank by using the BLANK function, and

Data type in model	Data type in DAX	Description
		test for blanks by using the logical function, ISBLANK.

Tabular data models also include the *Table* data type as the input or output to many DAX functions. For example, the FILTER function takes a table as input and outputs another table that contains only the rows that meet the filter conditions. By combining table functions with aggregation functions, you can perform complex calculations over dynamically defined data sets.

While data types are typically automatically set, it is important to understand data types and how they apply, in-particular, to DAX formulas. Errors in formulas or unexpected results, for example, are often caused by using a particular operator that cannot be used with a data type specified in an argument. For example, the formula, `= 1 & 2`, returns a string result of 12. The formula, `= "1" + "2"`, however, returns an integer result of 3.

Context

Context is an important concept to understand when creating DAX formulas. Context is what enables you to perform dynamic analysis, as the results of a formula change to reflect the current row or cell selection and also any related data. Understanding context and using context effectively are critical for building high-performing, dynamic analyses, and for troubleshooting problems in formulas.

Formulas in tabular models can be evaluated in a different context, depending on other design elements:

- Filters applied in a PivotTable or report
- Filters defined within a formula
- Relationships specified by using special functions within a formula

There are different types of context: *row context*, *query context*, and *filter context*.

Row context

Row context can be thought of as "the current row". If you create a formula in a calculated column, the row context for that formula includes the values from all columns in the current row. If the table is related to another table, the content also includes all the values from the other table that are related to the current row.

For example, suppose you create a calculated column, `= [Freight] + [Tax]`, that adds together values from two columns, Freight and Tax, from the same table. This formula

automatically gets only the values from the current row in the specified columns.

Row context also follows any relationships that have been defined between tables, including relationships defined within a calculated column by using DAX formulas, to determine which rows in related tables are associated with the current row.

For example, the following formula uses the RELATED function to fetch a tax value from a related table, based on the region that the order was shipped to. The tax value is determined by using the value for region in the current table, looking up the region in the related table, and then getting the tax rate for that region from the related table.

```
DAX
= [Freight] + RELATED('Region'[TaxRate])
```

This formula gets the tax rate for the current region from the Region table and adds it to the value of the Freight column. In DAX formulas, you do not need to know or specify the specific relationship that connects the tables.

Multiple row context

DAX includes functions that iterate calculations over a table. These functions can have multiple current rows, each with its own row context. In essence, these functions let you create formulas that perform operations recursively over an inner and outer loop.

For example, suppose your model contains a **Products** table and a **Sales** table. Users might want to go through the entire sales table, which is full of transactions involving multiple products, and find the largest quantity ordered for each product in any one transaction.

With DAX you can build a single formula that returns the correct value, and the results are automatically updated any time a user adds data to the tables.

```
DAX
= MAXX(FILTER(Sales,[ProdKey] = EARLIER([ProdKey])),Sales[OrderQty])
```

For a detailed example of this formula, see EARLIER.

To summarize, the EARLIER function stores the row context from the operation that preceded the current operation. At all times, the function stores in memory two sets of context: one set of context represents the current row for the inner loop of the formula, and another set of context represents the current row for the outer loop of the formula.

DAX automatically feeds values between the two loops so that you can create complex aggregates.

Query context

Query context refers to the subset of data that is implicitly retrieved for a formula. For example, when a user places a measure or field into a report, the engine examines row and column headers, slicers, and report filters to determine the context. The necessary queries are then run against model data to get the correct subset of data, make the calculations defined by the formula, and then populate values in the report.

Because context changes depending on where you place the formula, the results of the formula can also change. For example, suppose you create a formula that sums the values in the **Profit** column of the **Sales** table: `= SUM('Sales'[Profit])`. If you use this formula in a calculated column within the **Sales** table, the results for the formula will be the same for the entire table, because the query context for the formula is always the entire data set of the **Sales** table. Results will have profit for all regions, all products, all years, and so on.

However, users typically don't want to see the same result hundreds of times, but instead want to get the profit for a particular year, a particular country, a particular product, or some combination of these, and then get a grand total.

In a report, context is changed by filtering, adding or removing fields, and using slicers. For each change, the query context in which the measure is evaluated. Therefore, the same formula, used in a measure, is evaluated in a different *query context* for each cell.

Filter context

Filter context is the set of values allowed in each column, or in the values retrieved from a related table. Filters can be applied to the column in the designer, or in the presentation layer (reports and PivotTables). Filters can also be defined explicitly by filter expressions within the formula.

Filter context is added when you specify filter constraints on the set of values allowed in a column or table, by using arguments to a formula. Filter context applies on top of other contexts, such as row context or query context.

In tabular models, there are many ways to create filter context. Within the context of clients that can consume the model, such as Power BI reports, users can create filters on the fly by adding slicers or report filters on the row and column headings. You can also specify filter expressions directly within the formula, to specify related values, to filter

tables that are used as inputs, or to dynamically get context for the values that are used in calculations. You can also completely clear or selectively clear the filters on particular columns. This is very useful when creating formulas that calculate grand totals.

To learn more about how to create filters within formulas, see the FILTER Function (DAX). For an example of how filters can be cleared to create grand totals, see the ALL Function (DAX).

For examples of how to selectively clear and apply filters within formulas, see ALLEXCEPT.

Determining context in formulas

When you create a DAX formula, the formula is first tested for valid syntax, and then tested to make sure the names of the columns and tables included in the formula can be found in the current context. If any column or table specified by the formula cannot be found, an error is returned.

Context during validation (and recalculation operations) is determined as described in the preceding sections, by using the available tables in the model, any relationships between the tables, and any filters that have been applied.

For example, if you have just imported some data into a new table and it is not related to any other tables (and you have not applied any filters), the *current context* is the entire set of columns in the table. If the table is linked by relationships to other tables, the current context includes the related tables. If you add a column from the table to a report that has Slicers and maybe some report filters, the context for the formula is the subset of data in each cell of the report.

Context is a powerful concept that can also make it difficult to troubleshoot formulas. We recommend that you begin with simple formulas and relationships to see how context works. The following section provides some examples of how formulas use different types of context to dynamically return results.

Operators

The DAX language uses four different types of calculation operators in formulas:

- Comparison operators to compare values and return a logical TRUE\FALSE value.
- Arithmetic operators to perform arithmetic calculations that return numeric values.
- Text concatenation operators to join two or more text strings.
- Logical operators that combine two or more expressions to return a single result.

For detailed information about operators used in DAX formulas, see DAX operators.

Working with tables and columns

Tables in tabular data models look like Excel tables, but are different in the way they work with data and with formulas:

- Formulas work only with tables and columns, not with individual cells, range references, or arrays.
- Formulas can use relationships to get values from related tables. The values that are retrieved are always related to the current row value.
- You cannot have irregular or "ragged" data like you can in an Excel worksheet. Each row in a table must contain the same number of columns. However, you can have empty values in some columns. Excel data tables and tabular model data tables are not interchangeable.
- Because a data type is set for each column, each value in that column must be of the same type.

Referring to tables and columns in formulas

You can refer to any table and column by using its name. For example, the following formula illustrates how to refer to columns from two tables by using the *fully qualified* name:

DAX

```
= SUM('New Sales'[Amount]) + SUM('Past Sales'[Amount])
```

When a formula is evaluated, the model designer first checks for general syntax, and then checks the names of columns and tables that you provide against possible columns and tables in the current context. If the name is ambiguous or if the column or table cannot be found, you will get an error on your formula (an #ERROR string instead of a data value in cells where the error occurs). To learn more about naming requirements for tables, columns, and other objects, see Naming Requirements in DAX syntax.

Table relationships

By creating relationships between tables, you gain the ability for related values in other tables to be used in calculations. For example, you can use a calculated column to

determine all the shipping records related to the current reseller, and then sum the shipping costs for each. In many cases, however, a relationship might not be necessary. You can use the LOOKUPVALUE function in a formula to return the value in *result_columnName* for the row that meets criteria specified in the *search_column* and *search_value* arguments.

Many DAX functions require that a relationship exist between the tables, or among multiple tables, in order to locate the columns that you have referenced and return results that make sense. Other functions will attempt to identify the relationship; however, for best results you should always create a relationship where possible. Tabular data models support multiple relationships among tables. To avoid confusion or incorrect results, only one relationship at a time is designated as the active relationship, but you can change the active relationship as necessary to traverse different connections in the data in calculations. USERELATIONSHIP function can be used to specify one or more relationships to be used in a specific calculation.

It's important to observe these formula design rules when using relationships:

- When tables are connected by a relationship, you must ensure the two columns used as keys have values that match. Referential integrity is not enforced, therefore it is possible to have non-matching values in a key column and still create a relationship. If this happens, you should be aware that blank values or non-matching values might affect the results of formulas.

- When you link tables in your model by using relationships, you enlarge the scope, or *context*, in which your formulas are evaluated. Changes in context resulting from the addition of new tables, new relationships, or from changes in the active relationship can cause your results to change in ways that you might not anticipate. To learn more, see Context in this article.

Process and refresh

Process and *recalculation* are two separate but related operations. You should thoroughly understand these concepts when designing a model that contains complex formulas, large amounts of data, or data that is obtained from external data sources.

Process (refresh) is updating the data in a model with new data from an external data source.

Recalculation is the process of updating the results of formulas to reflect any changes to the formulas themselves and to reflect changes in the underlying data. Recalculation can affect performance in the following ways:

- The values in a calculated column are computed and stored in the model. To update the values in the calculated column, you must process the model using one of three processing commands – Process Full, Process Data, or Process Recalc. The result of the formula must always be recalculated for the entire column, whenever you change the formula.

- The values calculated by measures are dynamically evaluated whenever a user adds the measure to a PivotTable or open a report; as the user modifies the context, values returned by the measure change. The results of the measure always reflect the latest in the in-memory cache.

Processing and recalculation have no effect on row-level security formulas unless the result of a recalculation returns a different value, thus making the row queryable or not queryable by role members.

Updates

DAX is constantly being improved. New and updated functions are released with the next available update, which is usually monthly. Services are updated first, followed by installed applications like Power BI Desktop, Excel, SQL Server Management Studio (SSMS), and Analysis Services project extension for Visual Studio (SSDT). SQL Server Analysis Services is updated with the next cumulative update. New functions are first announced and described in the DAX function reference coinciding with Power BI Desktop updates.

Not all functions are supported in earlier versions of SQL Server Analysis Services and Excel.

Troubleshooting

If you get an error when defining a formula, the formula might contain either a *syntactic error*, *semantic error*, or *calculation error*.

Syntactic errors are the easiest to resolve. They typically involve a missing parenthesis or comma.

The other type of error occurs when the syntax is correct, but the value or a column referenced does not make sense in the context of the formula. Such semantic and calculation errors might be caused by any of the following problems:

- The formula refers to a non-existing column, table, or function.

- The formula appears to be correct, but when the data engine fetches the data, it finds a type mismatch and raises an error.
- The formula passes an incorrect number or type of arguments to a function.
- The formula refers to a different column that has an error, and therefore its values are invalid.
- The formula refers to a column that has not been processed, meaning it has metadata but no actual data to use for calculations.

In the first four cases, DAX flags the entire column that contains the invalid formula. In the last case, DAX grays out the column to indicate that the column is in an unprocessed state.

Apps and tools

Power BI Desktop

Power BI Desktop is a free data modeling and reporting application. The model designer includes a DAX editor for creating DAX calculation formulas.

Power Pivot in Excel

The Power Pivot in Excel models designer includes a DAX editor for creating DAX calculation formulas.

Visual Studio

Visual Studio with Analysis Services projects extension (VSIX) is used to create Analysis Services model projects. Tabular model designer, installed with the projects extension

includes a DAX editor.

SQL Server Management Studio

SQL Server Management Studio (SSMS) is an essential tool for working with Analysis Services. SSMS includes a DAX query editor for querying both tabular and multidimensional models.

DAX Studio

DAX Studio is an open-source client tool for creating and running DAX queries against Analysis Services, Power BI Desktop, and Power Pivot in Excel models.

Tabular Editor

Tabular Editor is an open-source tool that provides an intuitive, hierarchical view of every object in tabular model metadata. Tabular Editor includes a DAX Editor with syntax highlighting, which provides an easy way to edit measures, calculated column, and calculated table expressions.

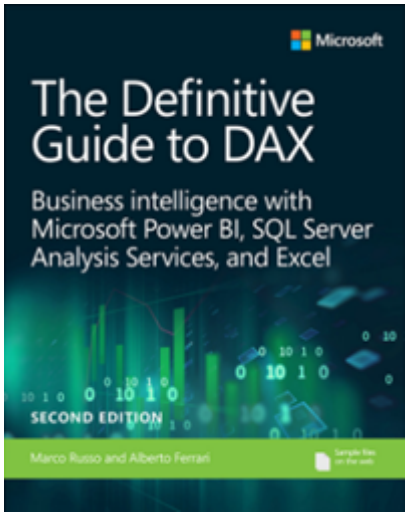
Learning resources

When learning DAX, it's best to use the application you'll be using to create your data models. Analysis Services, Power BI Desktop, and Power Pivot in Excel all have articles and tutorials that include lessons on creating measures, calculated columns, and row-filters by using DAX. Here are some additional resources:

Videos

Use DAX in Power BI Desktop learning path.

The Definitive Guide to DAX by Alberto Ferrari and Marco Russo (Microsoft Press). Now in its second edition, this extensive guide provides basics to innovative high-performance techniques for beginning data modelers and BI professionals.



Community

DAX has a vibrant community always willing to share their expertise. Microsoft Power BI Community has a special discussion forum just for DAX, DAX Commands and Tips.

Feedback

Was this page helpful? Yes No

Videos

Article • 10/20/2023

Whether you're using Power BI Desktop, Power Pivot in Excel, or Analysis Services, learning Data Analysis Expressions (DAX) is essential to creating effective data models. Here are some videos to help you get started using this powerful expression language.

DAX 101

In this DAX 101 video, Microsoft Partner, Alberto Ferrari introduces essential concepts in DAX. With practical and clear examples, you will learn about measures, calculated columns, and basic data modeling expressions with DAX.
https://www.youtube-nocookie.com/embed/klQAZLr5vxA

Advanced DAX

In this advanced DAX video, Microsoft Partner, Alberto Ferrari describes DAX theory, filter and row context, and other essential concepts in DAX.
https://www.youtube-nocookie.com/embed/6ncHnWMEdic

Practical DAX for Power BI

In this video, Microsoft Principal Program Manager, Phil Seamark, walks through some best practices for practical uses of DAX in Power BI.
https://www.youtube-nocookie.com/embed/1fGfqzS37qs

Feedback

Was this page helpful? Yes No

Write DAX queries with Copilot

Article • 12/10/2024

You can use Copilot to write and explain Data Analysis Expressions (DAX) queries in DAX query view of a Power BI semantic model.

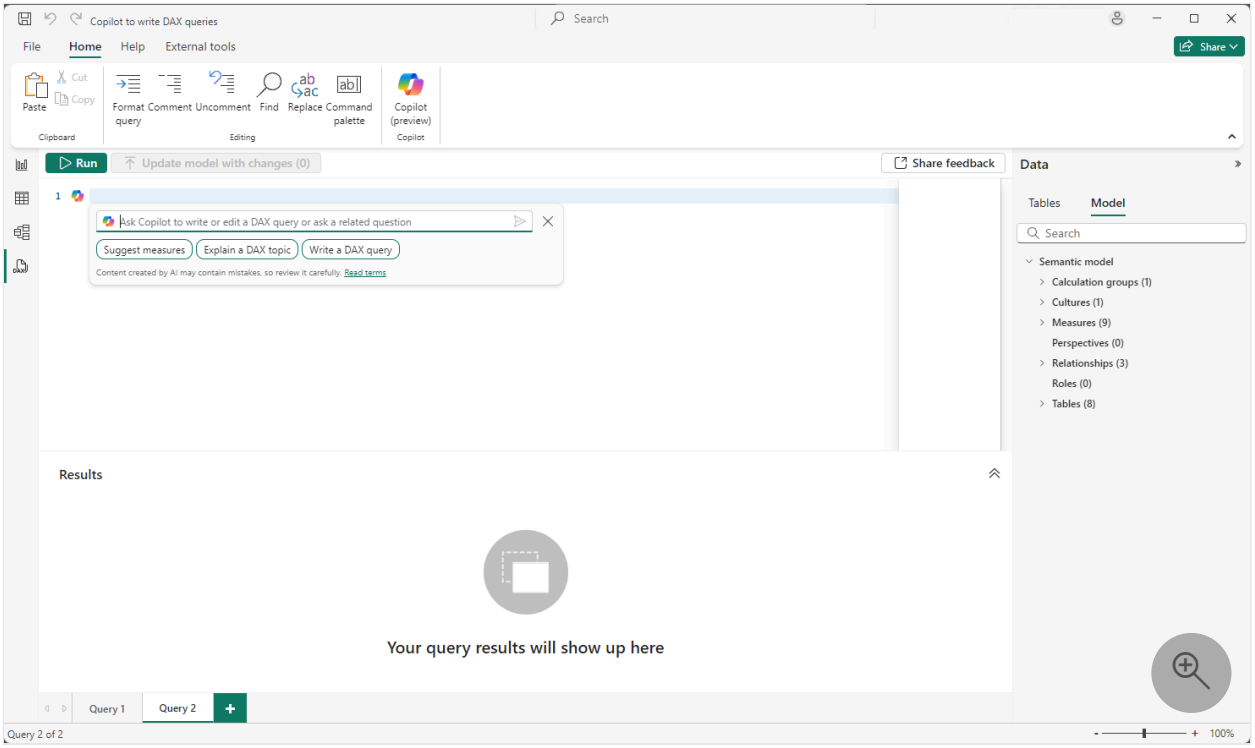
Enable Copilot

Check to make sure you meet the Fabric Copilot requirements.

Power BI Desktop

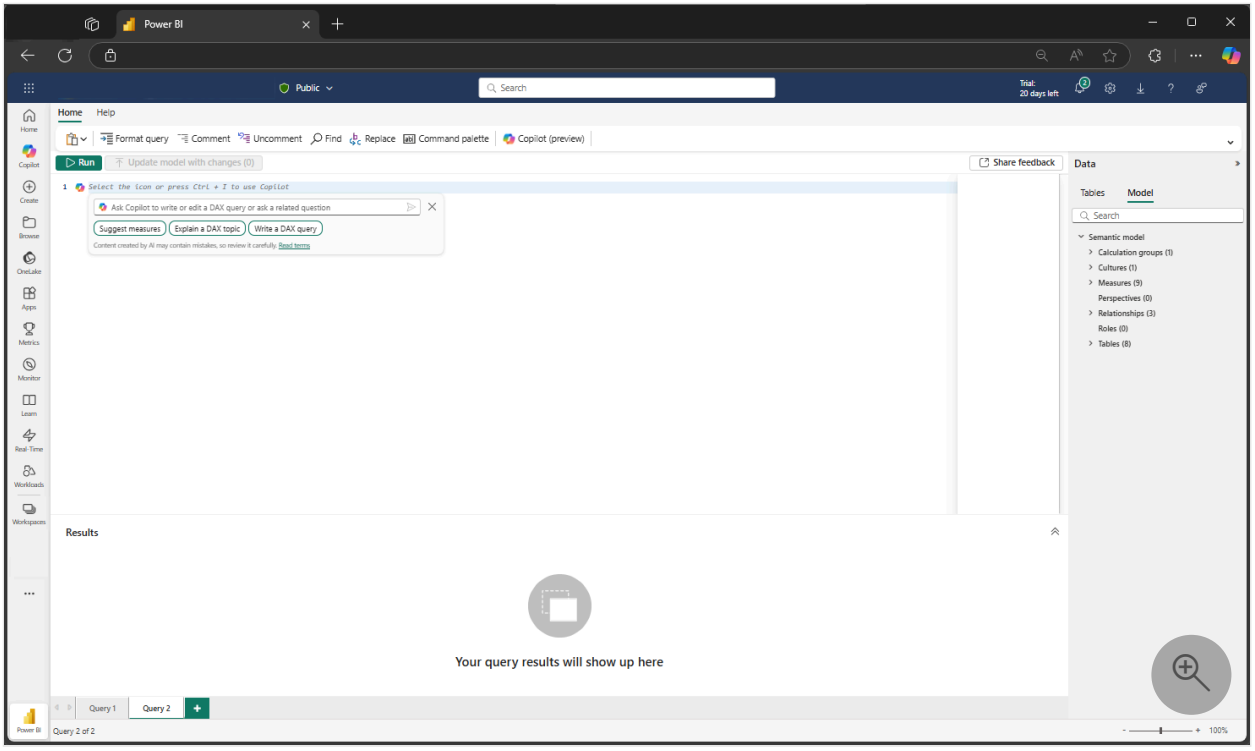
In Power BI Desktop, in the **Preview features** section of the **Options**, enable **DAX query view with Copilot**.

1. Open Power BI Desktop.
2. Open an existing Power BI report or connect to published semantic model as live edit or live connection. Alternatively, you can add data and create a new semantic model and report.
3. Go to **DAX query view**, the fourth view available on the left side of Power BI Desktop.
4. Create a new query tab.
5. Open Copilot by clicking on the **Copilot** button or using `CTRL + I`.



Power BI service or workspace

1. Choose a published semantic model in a workspace.
2. From the context menu choose **Write DAX queries**. If **Write DAX queries** is greyed out, you may need to enable **Users can edit data models in the Power BI service (preview)** in the **Power BI General** section of **Workspace settings**.
3. Create a new query tab.
4. Open Copilot by clicking on the Copilot button or using `CTRL + I`.



> **Note**
>
> DirectQuery and Direct Lake model authors can also use DAX query view. View the data in your tables whenever you want.

Use Copilot

Let's look at what Copilot can help you with in DAX query view:

- **Natural language to DAX query**. Just ask for what you want from your semantic model and have the DAX query generated by Copilot. You can also select an existing DAX query and have Copilot modify it.
- **DAX query to natural language**. Ask Copilot to explain what any DAX query is doing.
- **Understand DAX concepts**. Maybe you heard about a certain DAX function or need to find the right DAX function for the job. Copilot can help you here too.

To use Copilot to create a DAX query, type in a prompt to describe what DAX query you would like generated and select **Send** or press `Enter`. Alternatively, you can click the **Write DAX query** inspire button. To run what is returned, select **Run** or press `F5` to see the results of the DAX query. To keep the query and exit Copilot, select **Keep query** to add it to query tab.

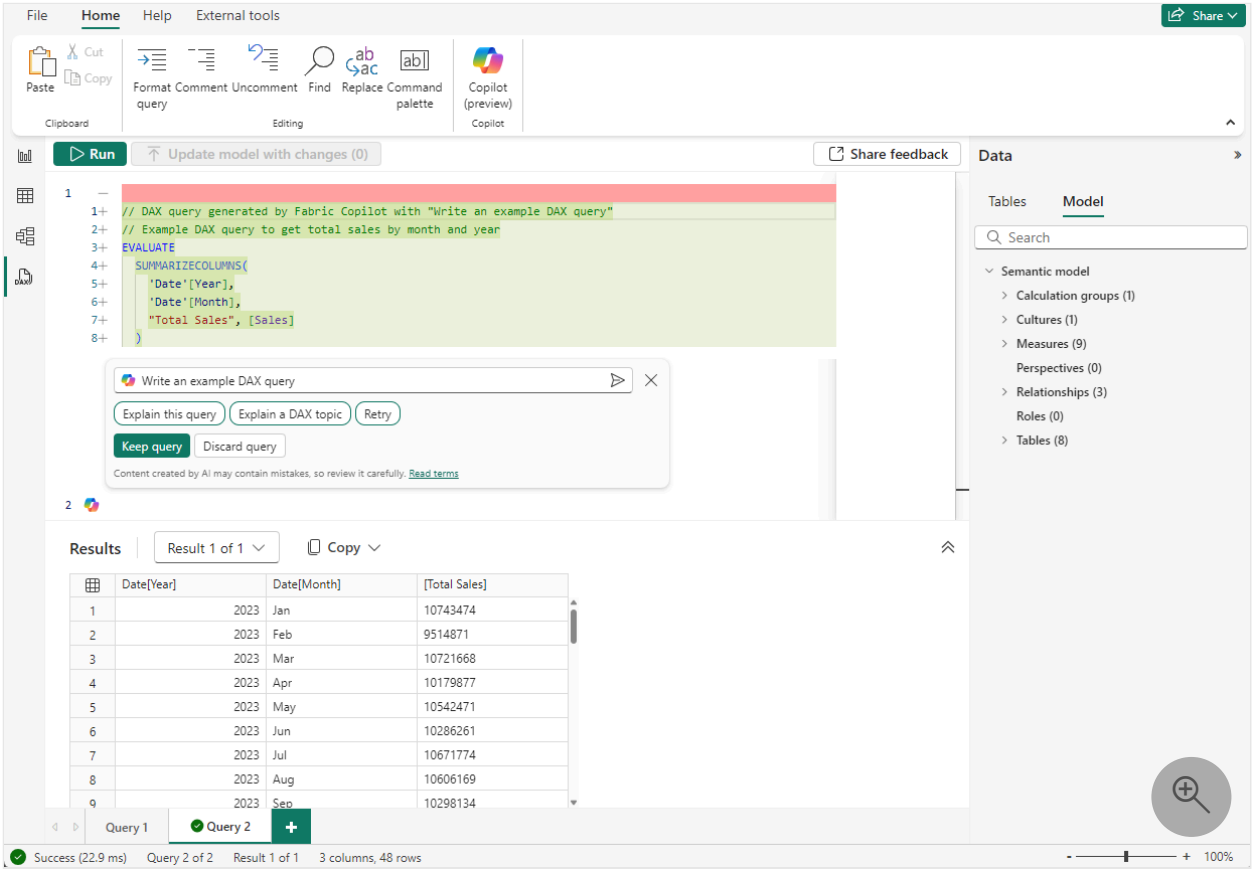
To use Copilot to explain a DAX query, select the DAX query and click the **Explain this query** inspire button. A short explanation is shown with the option to **show more** to see a longer explanation.

To use Copilot to explain a DAX topic, click the **Explain a DAX topic** inspire button, or type in a prompt to ask for an explanation of a DAX function or topic and select **Send** or press `Enter`. A short explanation is shown with the option to **show more** to see a longer explanation.

Features of the inline Copilot

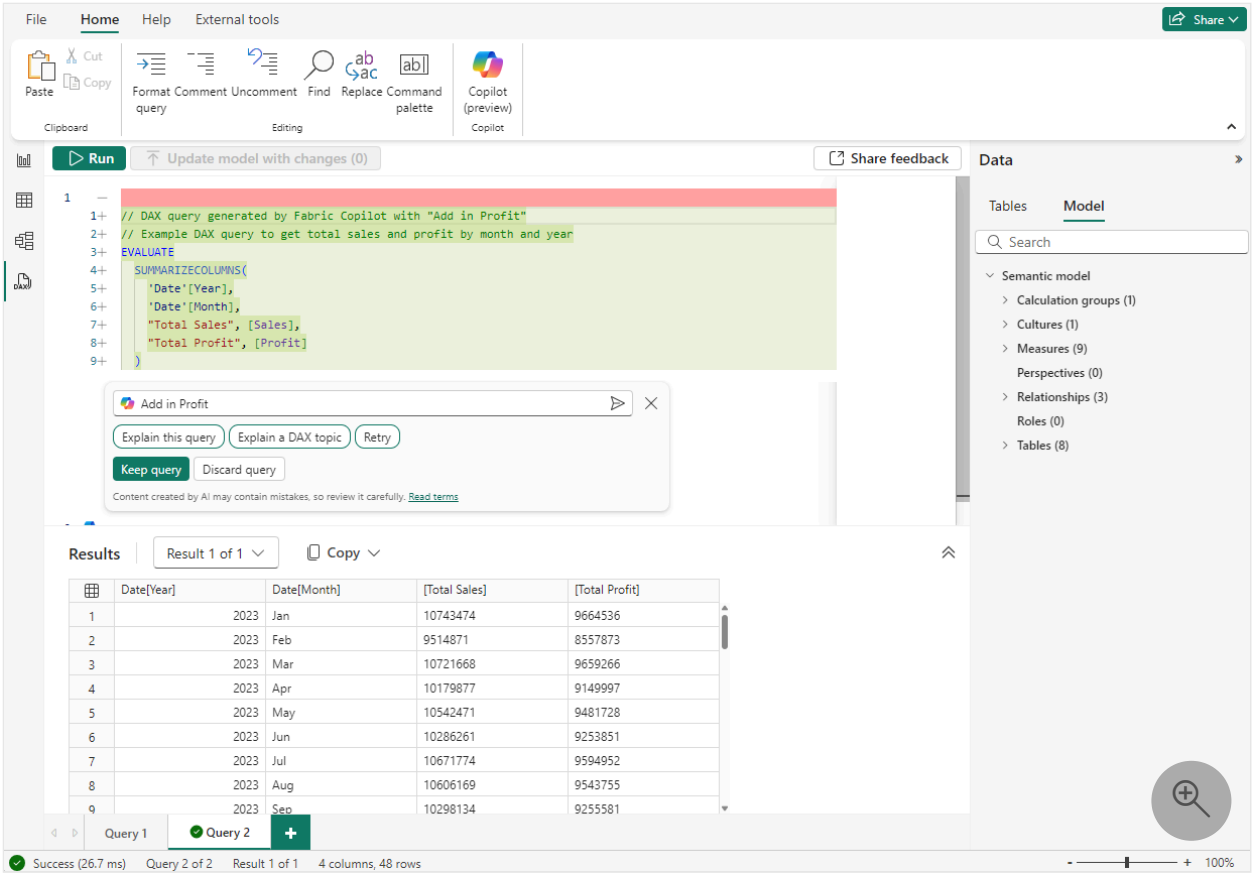
1. Run the DAX query before you keep it

You can **Run** the DAX query then decide to **Keep** or **Discard** the DAX query.



2. Build the DAX query conversationally

After the initial DAX query is generated, you can simply adjust by typing in additional user prompts.



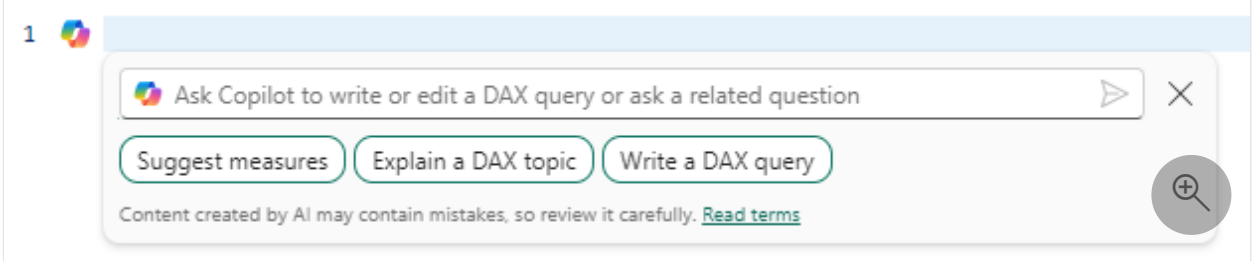
3. Syntax checks on the generated DAX query

The generated DAX query syntax is checked, and the prompt automatically retried once. If the retry is also invalid, the generated DAX query is returned with a note that there is an issue, giving you the option to rephrase your request or fix the generated DAX query:

"This query contains errors. Rephrase your request and try again or try fixing it yourself."

4. Inspire buttons to get you started with Copilot

Type in a prompt or select any of these buttons to see quickly what you can do with Copilot.



Sample prompts

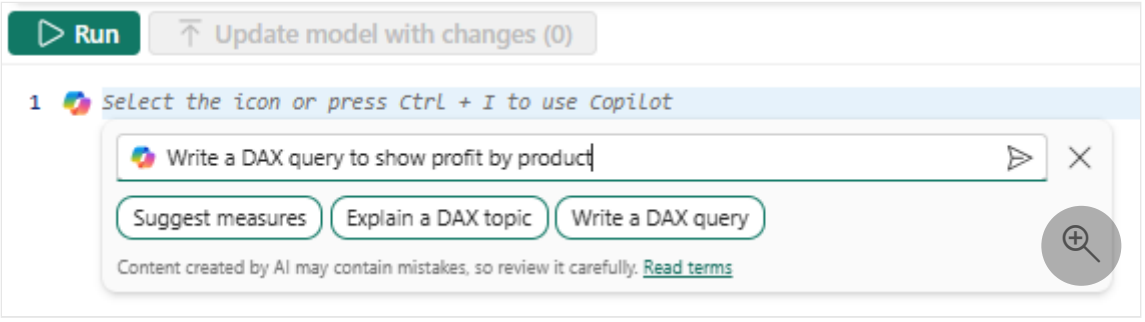
These sample prompts can illustrate how to use Copilot to write DAX queries.

- Write a DAX query
- Add a column with the diff view
- Explain a DAX query
- Explain a DAX topic

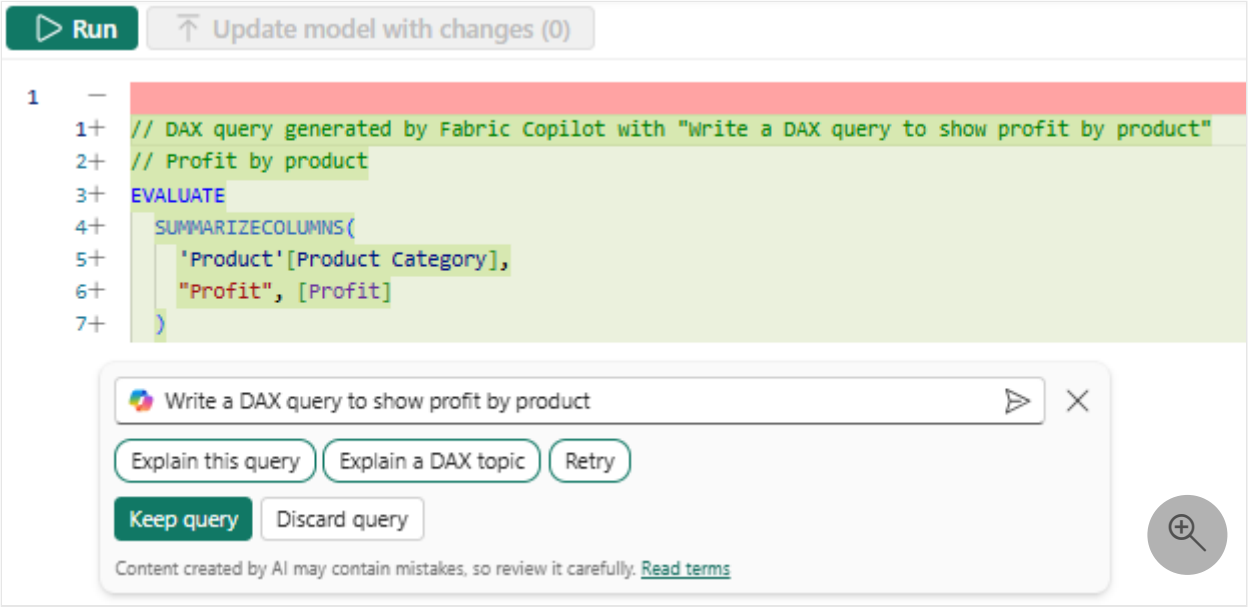
Write a DAX query

In a blank query tab, invoke Copilot with `CTRL+I` and enter this request:

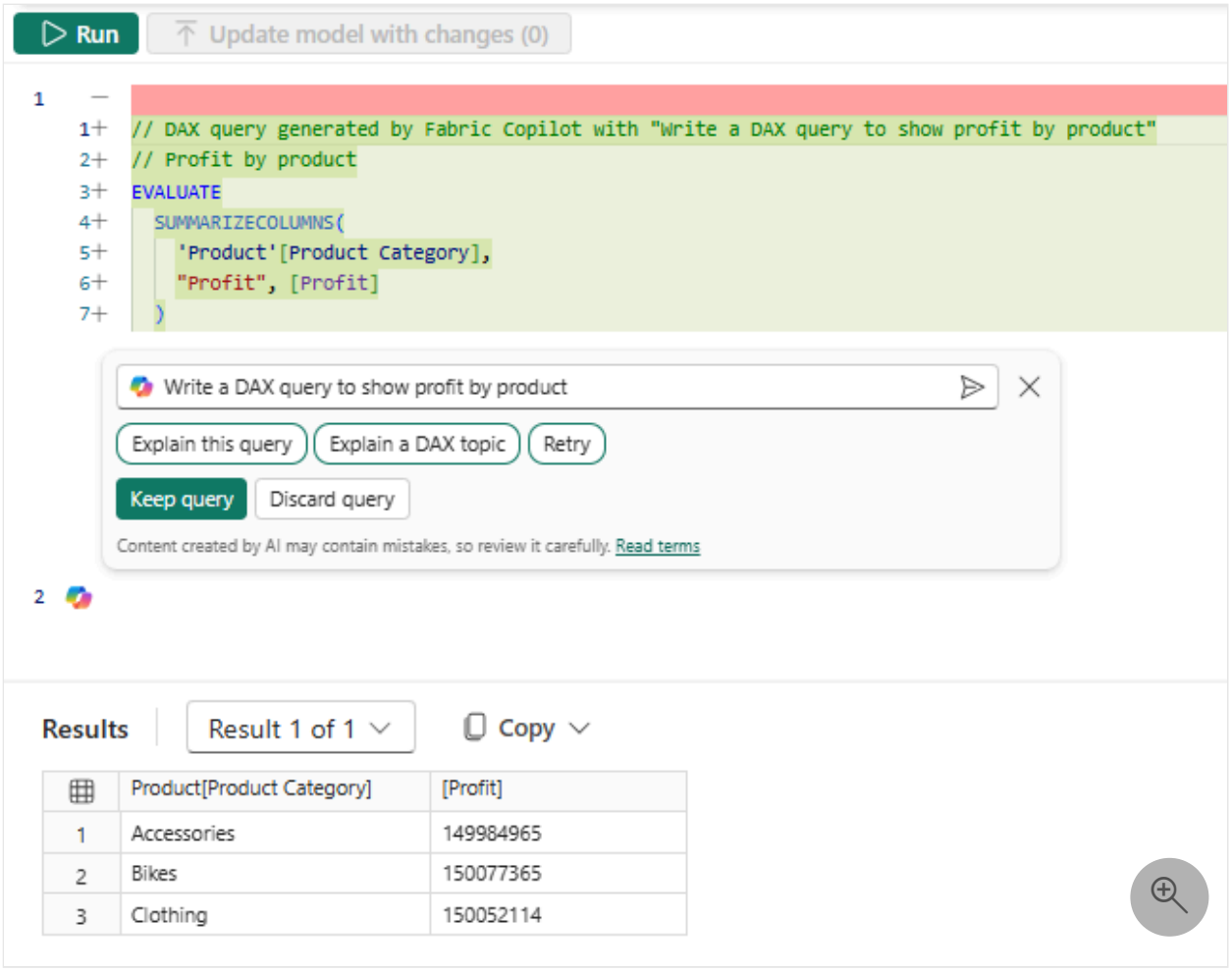
"Write a DAX query to show profit by product"



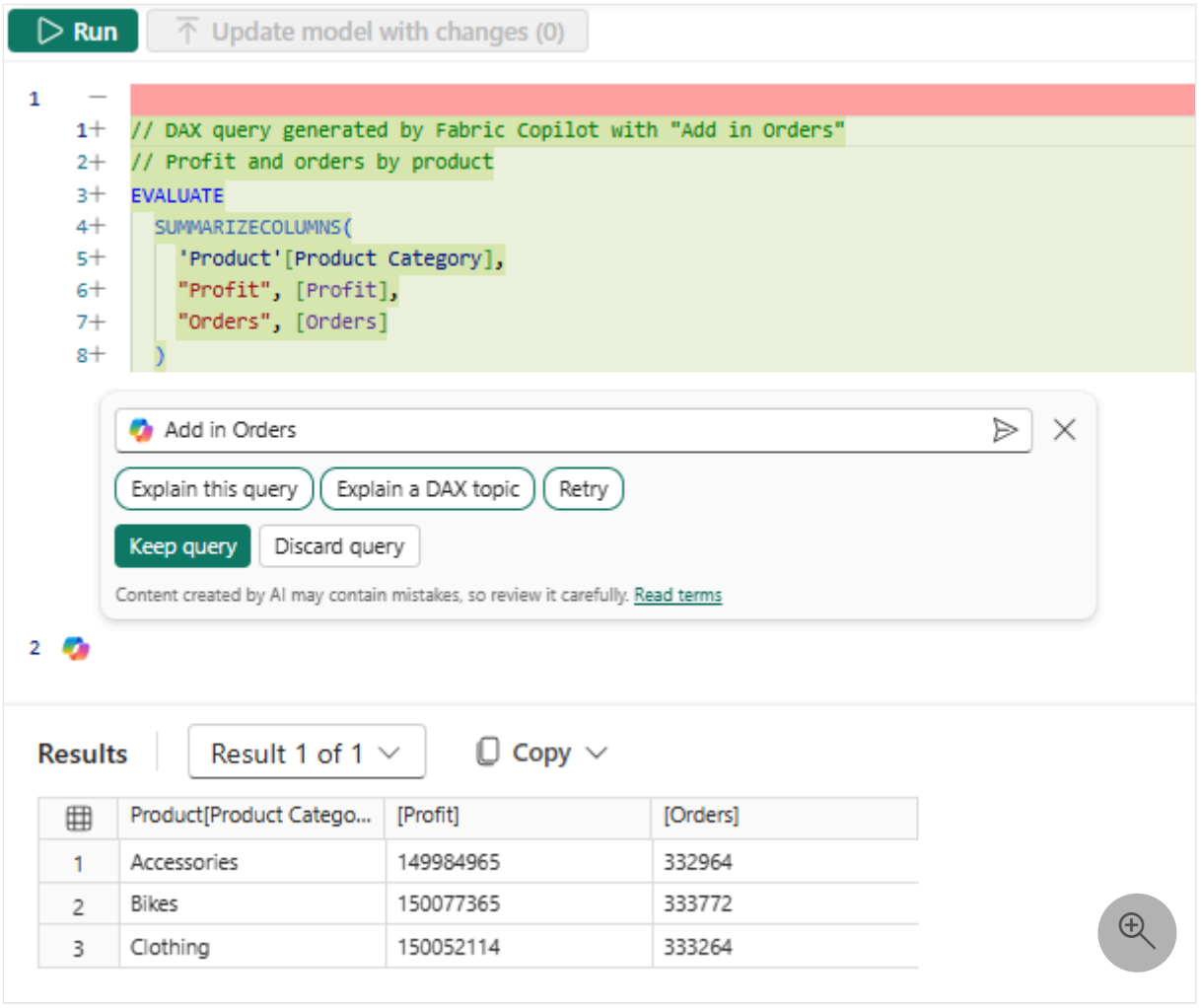
After a few moments I get a DAX query.



I can select **run** to see the results before I keep the query.



I can conversationally make updates to my initial request. Type in "Add in Orders" and hit `enter` and Copilot adds in the additional measure. I can again **run** the query to see the results.

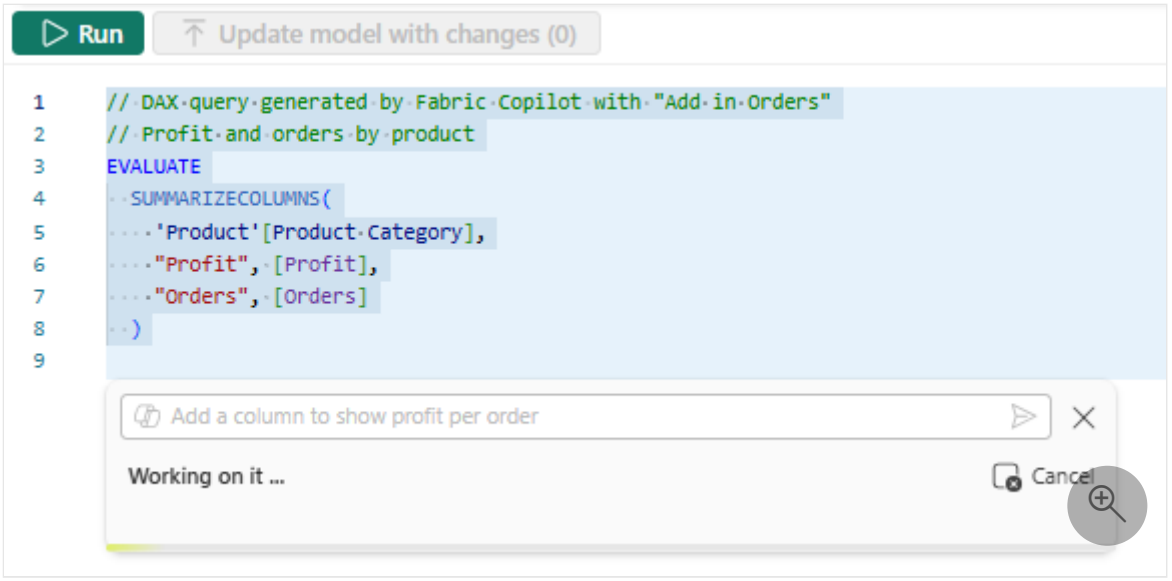


When I select **keep query**, the query is added to the query tab.

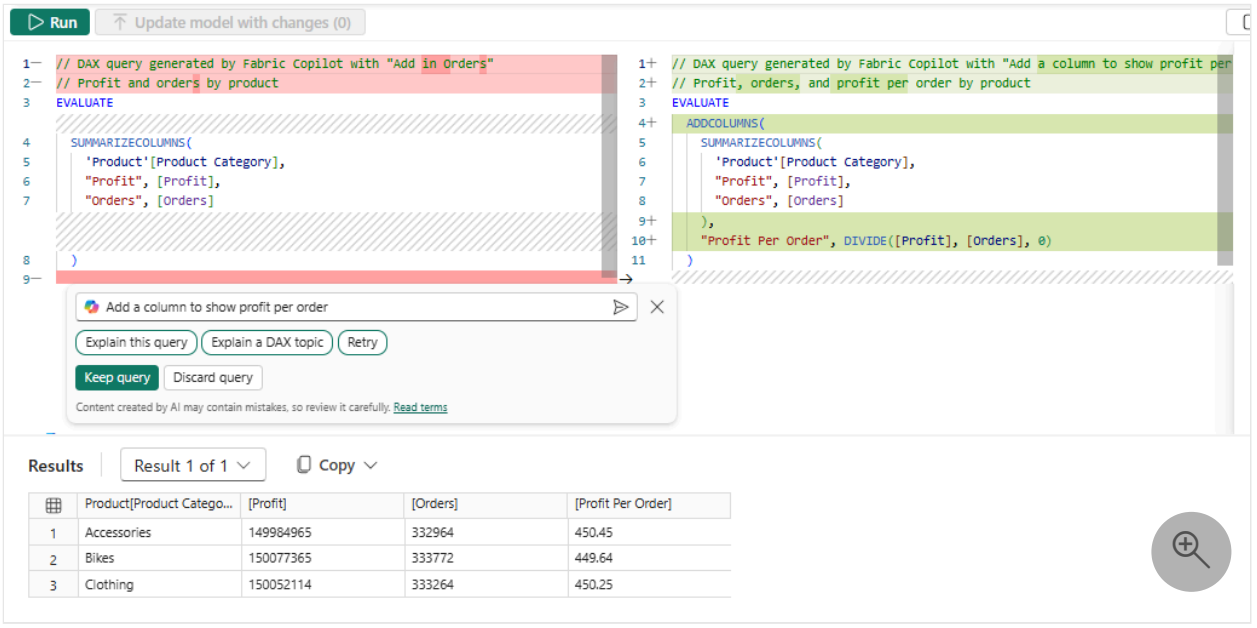
Add a column with the diff view

Let's now add a column to the existing DAX query. First I select the DAX query, then invoke Copilot. This time the selection is showing fitted to the text and the background shading highlights what is being sent with the request to Copilot. Type in this request, and hit `enter`:

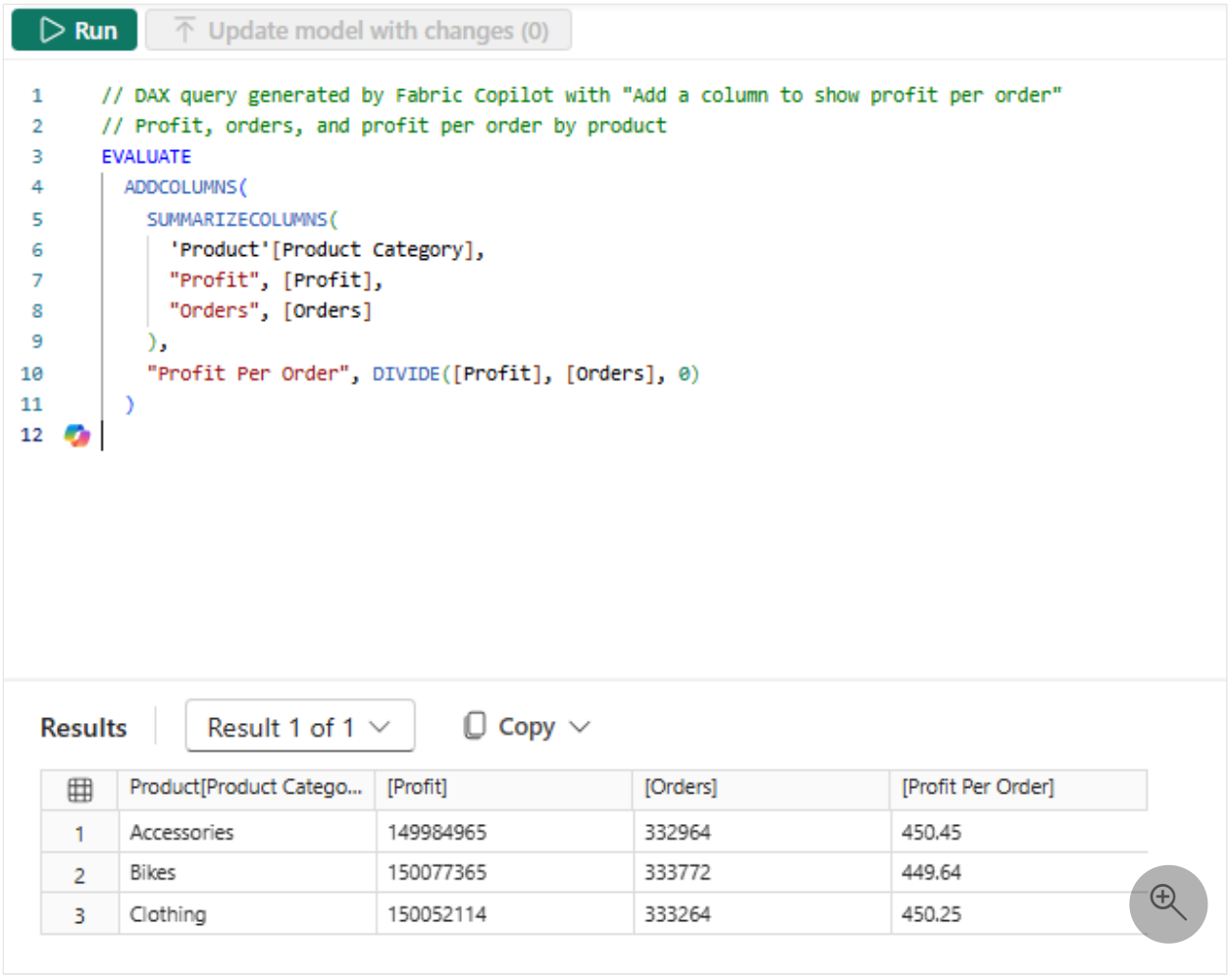
"Add a column to show profit per order"



Now the diff styling in the DAX query editor shows exactly what Copilot has changed.

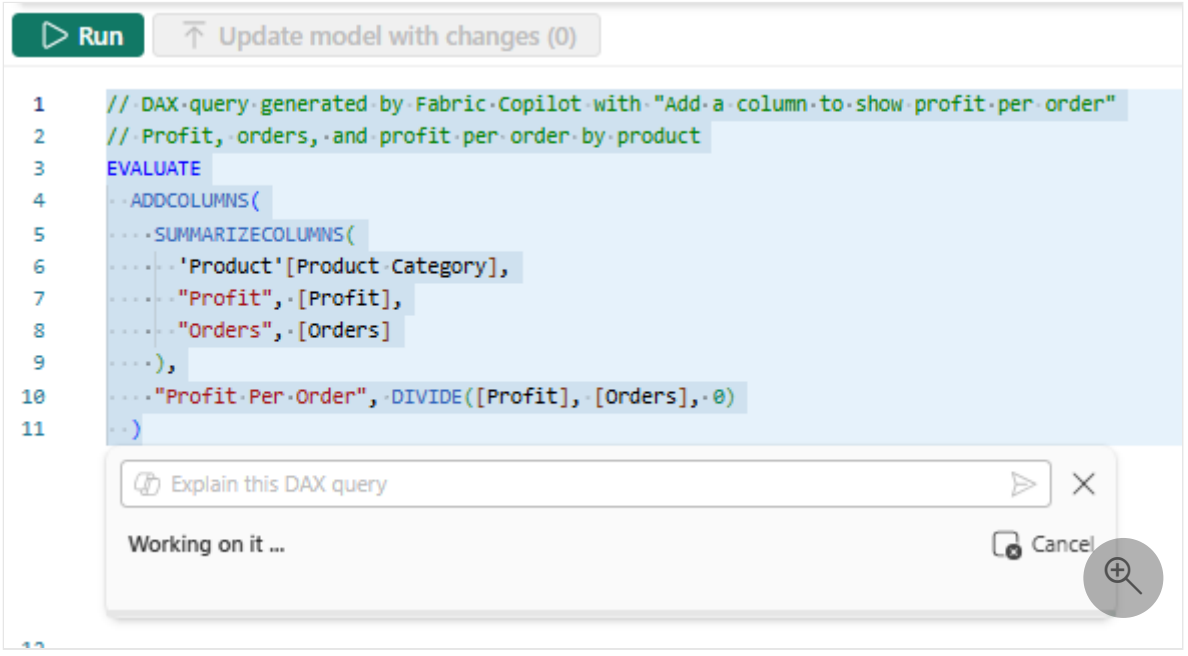


I select **run** to see the results, and **keep query** to add it to the query editor.

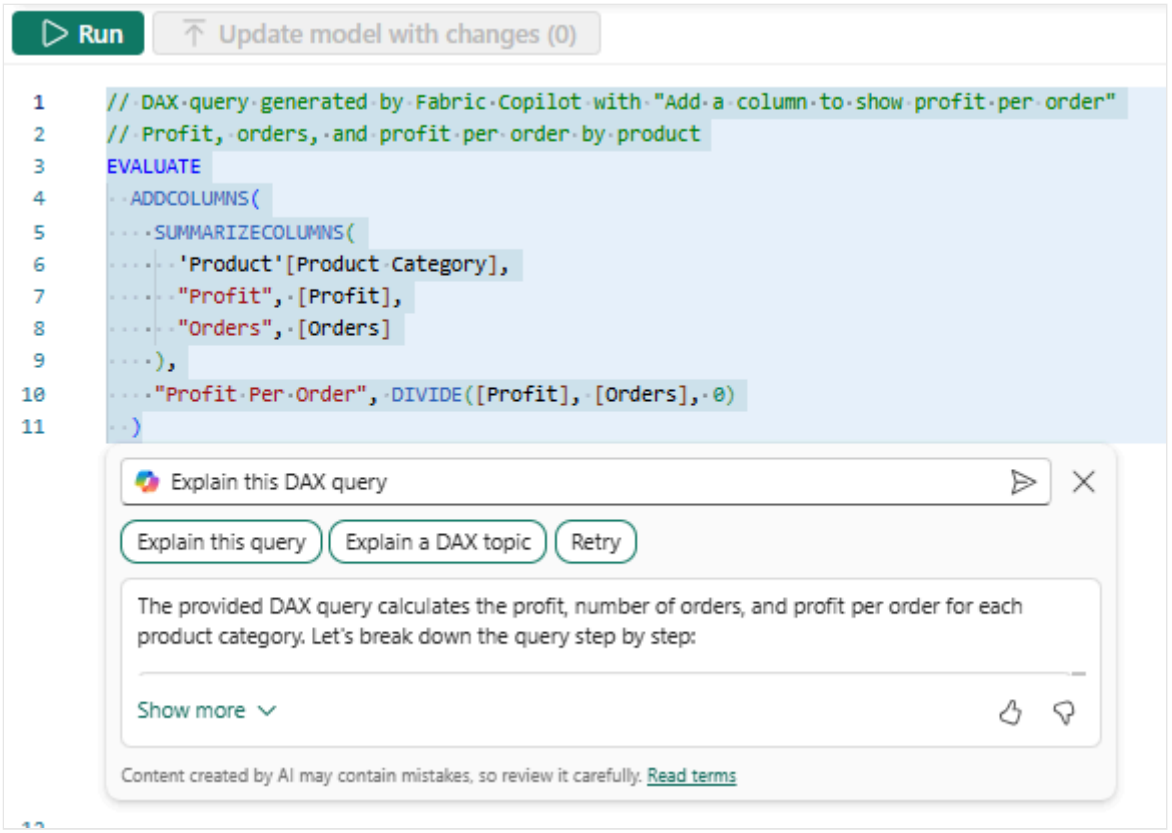


Explain a DAX query

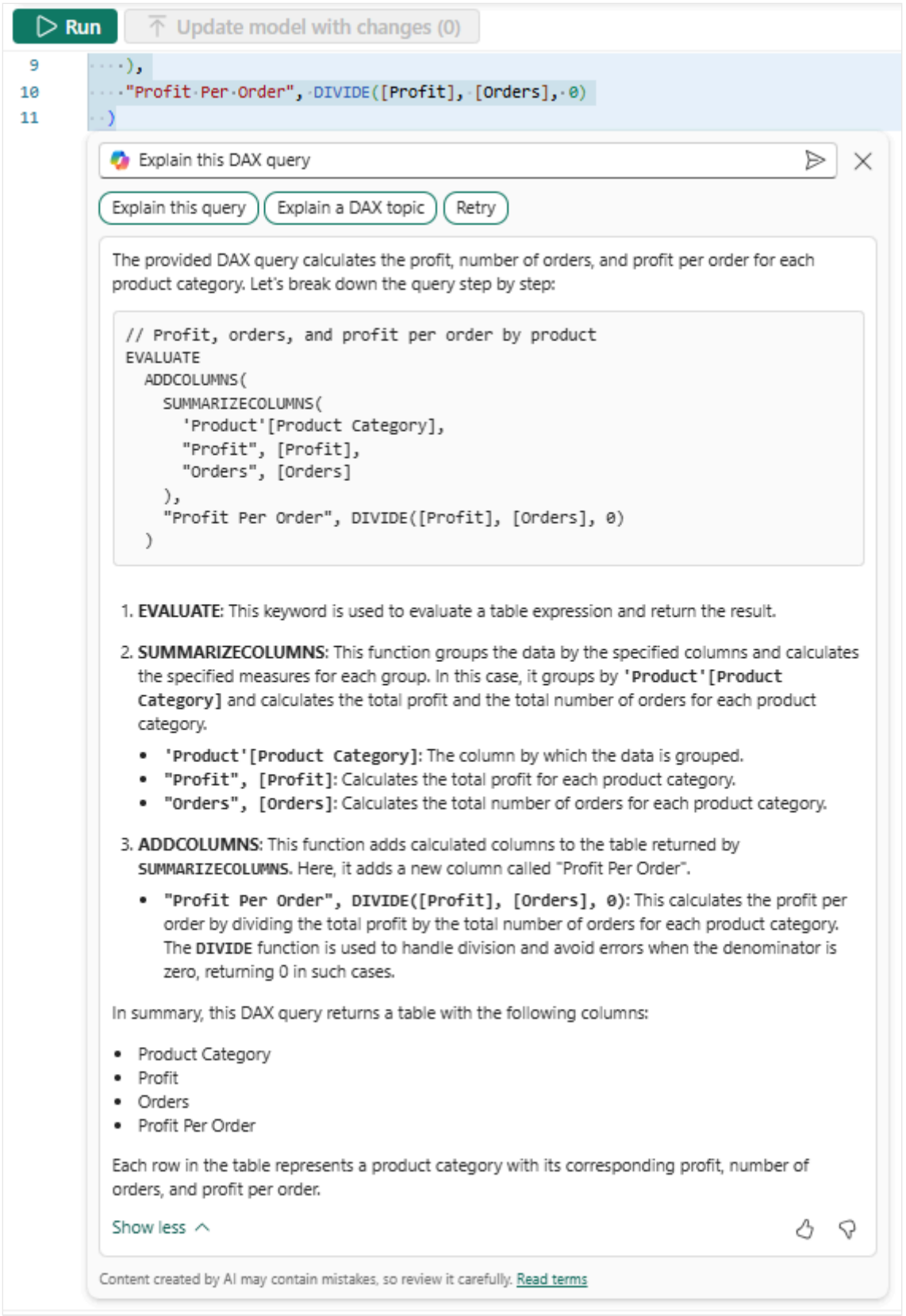
Copilot can provide a detailed explanation of a DAX query. Again, select the entire DAX query and invoke Copilot. This time select the **Explain this query** button.



Now, instead of writing DAX queries, it's responding in the inline Copilot.



Copilot generates a few lines. I select **Show more** to see the full explanation.

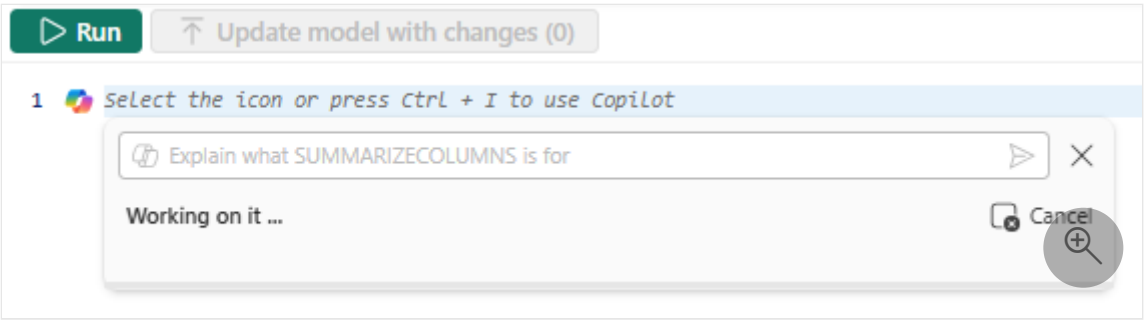


Now I understand each part and the DAX functions used.

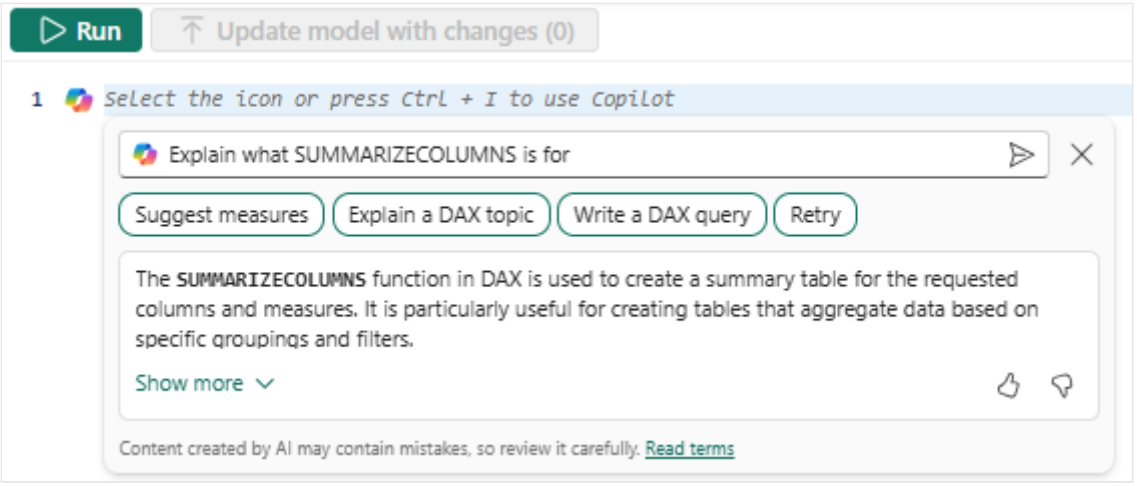
Explain a DAX topic

Copilot can also explain DAX functions with more detail. Let's ask about SUMMARIZECOLUMNS. In a new query tab, invoke Copilot, and sent the following request:

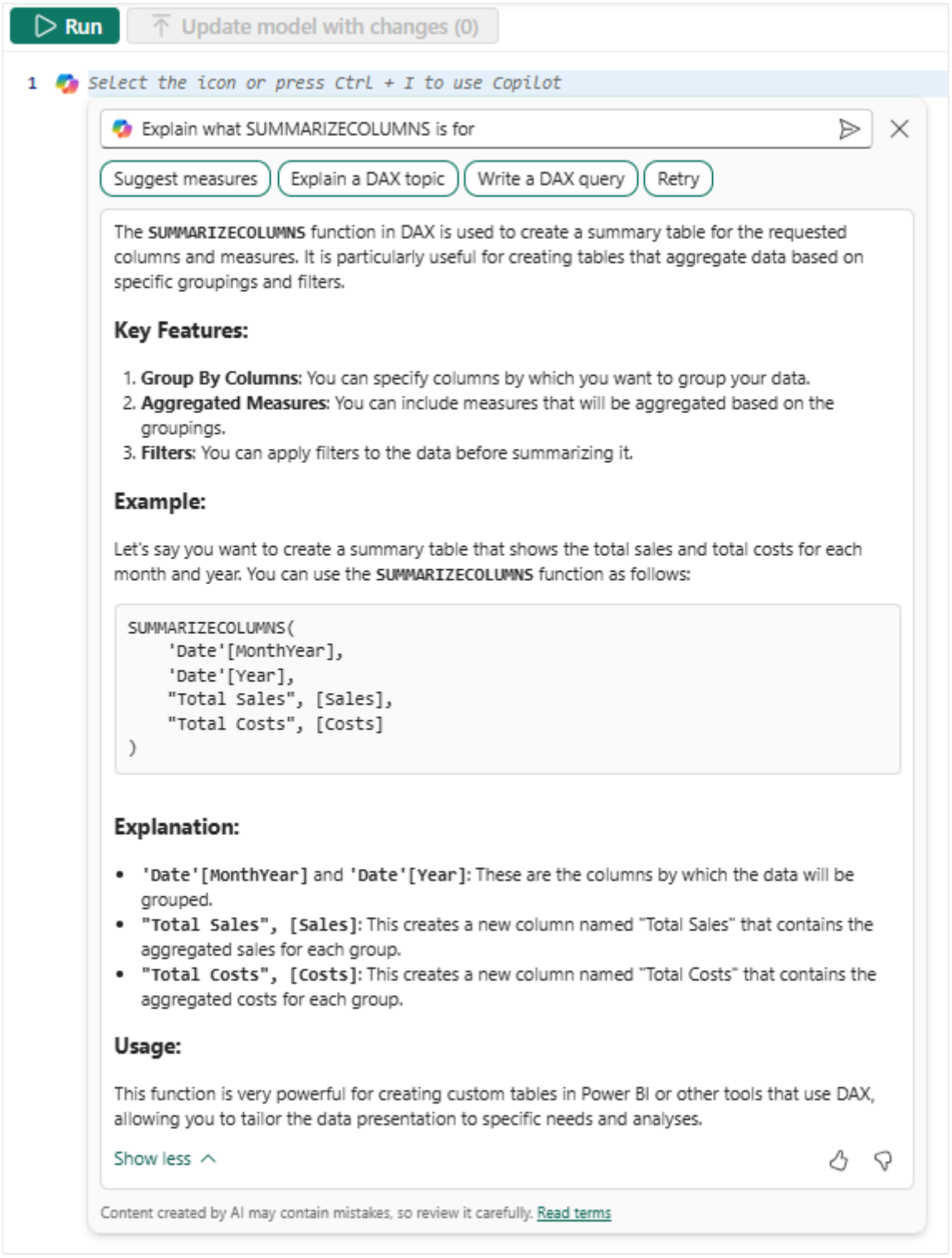
"Explain what SUMMARIZECOLUMNS is for"



Again, Copilot generates brief description:



I can select **Show more** to see a more detailed explanation with an example.



The example also uses columns in my model.

Fabric Copilot to help write DAX queries: Responsible AI FAQ

What is Fabric Copilot to help write DAX queries?

An inline Copilot found in Power BI DAX query view, available in the DAX query view of Power BI Desktop or write DAX queries in Power BI workspace, for model authors and

consumers to request help creating DAX queries. The help can include explaining a DAX query and DAX topics, in addition to creating DAX queries themselves.

What can Fabric Copilot to help write DAX queries do?

The user request can be returned by Copilot as a DAX query the user can see, then run to see the data returned in the results grid. Additional requests can be made, and a new DAX query can be returned, creating a DAX query conversationally. If the request is about explaining a DAX query or topic, a natural language response is generated in the inline Copilot. DAX topics can include what a DAX function does or which DAX function is good for returning data in a specific way.

What is Fabric Copilot to help write DAX queries' intended use?

Copilot's intended use is to help create DAX queries on a specific semantic model the user is currently working with in Power BI.

How was Fabric Copilot to help write DAX queries evaluated? What metrics are used to measure performance?

DAX queries from multiple Power BI semantic models using a variety of user requests, categorized by data availability in the semantic model and/or if additional calculations were needed to create the correct response, and then graded for accuracy and performance by members of the product team. This included a semantic model adhering to best practices. Explanations from multiple DAX queries and DAX function topics, and then graded for accuracy and readability by members of the product team.

What are the limitations of Fabric Copilot to help write DAX queries? How can users minimize the impact of Fabric Copilot to help write DAX queries' limitations when using the system?

- To use Copilot to help write DAX queries, you need to select a workspace with a Fabric capacity.
- Copilot uses the model metadata (tables, columns, measures), so DAX query responses will answer only in context of the current semantic model.

- Copilot uses the model metadata (tables, columns, measures), so providing rich metadata that is descriptive and understandable will yield better results.
- Copilot may use min/max to get sample values in import storage mode, but cannot use other data in the tables of the semantic model.
- Copilot may not be available or use reduced version of the model metadata (tables, columns, measures) when the number of model metadata items exceeds 500 tables, 1,000 columns or 3,000 measures in a table, 2,000 relationships, 10,000 total columns, 5,000 total measures, 5,000 characters in a single DAX expression, or 256 characters in a name.
- Copilot may use the description property on model metadata, but is limited to only the first 200 characters.
- DAX functions added recently may not be known to the Copilot, and explanations may be incorrect.
- Copilot requests include all text in the current query window, and if text is selected the selection information it is noted as selected, but all text in the query window is included.

What operational factors and settings allow for effective and responsible use of Fabric Copilot to help write DAX queries?

- Operational factors and settings include the current workload on a Fabric capacity and network speed.
- Copilot to help write DAX queries is contained in the privacy, security, and responsible use of Copilot in Fabric.

How do I provide feedback on Fabric Copilot to help write DAX queries?

- Use the thumbs up and thumbs down icons in the inline Copilot to provide feedback.
- Submit feedback using the Power BI support.

Learn more with these resources

- Overview of Copilot in Fabric and Power BI
- DAX query view
- DAX queries

Feedback

Was this page helpful? Yes No

DAX sample model

Article • 12/13/2023

The **Adventure Works DW 2020** Power BI Desktop sample model is designed to support your DAX learning. The model is based on the Adventure Works data warehouse sample for AdventureWorksDW2017—however, the data has been modified to suit the objectives of the sample model.

The sample model does not contain any DAX formulas. It does however support hundreds or even thousands of potential calculation formulas and queries. Some function examples, like those in CALCULATE, DATESBETWEEN, DATESIN PERIOD, IF, and LOOKUPVALUE can be added to the sample model without modification. We're working on including more examples in other function reference articles that work with the sample model.

Scenario

The Adventure Works company represents a bicycle manufacturer that sells bicycles and accessories to global markets. The company has their data warehouse data stored in an Azure SQL Database.

Model structure

The model has seven tables:

Expand table

Table	Description
Customer	Describes customers and their geographic location. Customers purchase products online (Internet sales).
Date	There are three relationships between the **Date** and **Sales** tables, for order date, ship date, and due date. The order date relationship is active. The company's reports

Table	Description
	sales using a fiscal year that commences on July 1 of each year. The table is marked as a date table using the **Date** column.
Product	Stores finished products only.
Reseller	Describes resellers and their geographic location. Reseller on sell products to their customers.
Sales	Stores rows at sales order line grain. All financial values are in US dollars (USD). The earliest order date is July 1, 2017, and the latest order date is June 15, 2020.
Sales Order	Describes sales order and order line numbers, and also the sales channel, which is either **Reseller** or **Internet**. This table has a one-to-one relationship with the **Sales** table.
Sales Territory	Sales territories are organized into groups (North America, Europe, and Pacific), countries, and regions. Only the United States sells products at the region level.

Download sample

Download the Power BI Desktop sample model file here.

Related content

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Understanding ORDERBY, PARTITIONBY, and MATCHBY functions

Article • 07/26/2023

The ORDERBY, PARTITIONBY, and MATCHBY functions in DAX are special functions that can only be used along with DAX Window functions: INDEX, OFFSET, WINDOW, RANK, ROWNUMBER.

Understanding ORDERBY, PARTITIONBY, and MATCHBY is critical to successfully using the Window functions. The examples provided here use OFFSET, but are similarly applicable to the other Window functions.

Scenario

Let's start with an example that doesn't use Window functions at all. Shown below is a table that returns total sales, per color, per calendar year. There are multiple ways to define this table, but since we're interested in understanding what happens in DAX, we'll use a calculated table. Here's the table expression:

DAX

```
BasicTable =
    SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
    )
```

You'll see this calculated table expression uses SUMMARIZECOLUMNS to calculate the SUM of the SalesAmount column in the FactInternetSales table, by the Color column from the DimProduct table, and the CalendarYear column from the DimDate table. Here's The result:

Expand table

Color	CalendarYear	CurrentYearSales
"Black"	2017	393885
"Black"	2018	1818835

Color	CalendarYear	CurrentYearSales
"Black"	2019	3981638
"Black"	2020	2644054
"Blue"	2019	994448
"Blue"	2020	1284648
"Multi"	2019	48622
"Multi"	2020	57849
"NA"	2019	207822
"NA"	2020	227295
"Red"	2017	2961198
"Red"	2018	3686935
"Red"	2019	900175
"Red"	2020	176022
"Silver"	2017	326399
"Silver"	2018	750026
"Silver"	2019	2165176
"Silver"	2020	1871788
"White"	2019	2517
"White"	2020	2589
"Yellow"	2018	163071
"Yellow"	2019	2072083
"Yellow"	2020	2621602

Now, let’s imagine we're trying to solve the business question of calculating the difference in sales, year-over-year for each color. Effectively, we need a way to find sales for the same color in the previous year and subtract that from the sales in the current year, in context. For example, for the combination [Red, 2019] we're looking for sales for [Red, 2018]. Once we have that, we can then subtract it from the current sales and return the required value.

Using OFFSET

OFFSET is perfect for the typical *compare with previous* types of calculations required to answer the business question described above, as it allows us to do a relative movement. Our first attempt might be:

DAX

```
1stAttempt =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation
            ),
            [CurrentYearSales]
        )
    )
```

A lot is happening with this expression. We used ADDCOLUMNS to expand the table from before with a column called PreviousColorSales. The contents of that column are set to the CurrentYearSales, which is SUM(FactInternetSales[SalesAmount]), for the previous Color (retrieved using OFFSET).

The result is:

Expand table

Color	CalendarYear	CurrentYearSales	PreviousColorSales
"Black"	2017	393885	
"Black"	2018	1818835	393885
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
"Blue"	2019	994448	2644054

Color	CalendarYear	CurrentYearSales	PreviousColorSales
"Blue"	2020	1284648	994448
"Multi"	2019	48622	1284648
"Multi"	2020	57849	48622
"NA"	2019	207822	57849
"NA"	2020	227295	207822
"Red"	2017	2961198	227295
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	176022
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	1871788
"White"	2020	2589	2517
"Yellow"	2018	163071	2589
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

This is one step closer to our goal, but if we look closely it doesn't match exactly what we're after. For example, for [Silver, 2017] the PreviousColorSales is set to [Red, 2020].

Adding ORDERBY

That definition above is equivalent to:

DAX

```
1stAttemptWithORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
```

```
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([Color], ASC, [CalendarYear], ASC,
[CurrentYearSales], ASC)
            ),
            [CurrentYearSales]
        )
    )
```

In this case, the call to OFFSET uses ORDERBY to order the table by Color and CalendarYear in ascending order, which determines what is considered the previous row that's returned.

The reason these two results are equivalent is because ORDERBY automatically contains all columns from the relation that aren't in PARTITIONBY. Since PARTITIONBY wasn't specified, ORDERBY is set to Color, CalendarYear, and CurrentYearSales. However, since the Color and CalendarYear pairs in the relation are unique, adding CurrentYearSales doesn't change the result. In fact, even if we were to only specify Color in ORDERBY, the results are the same since CalendarYear would be automatically added. This is because the function will add as many columns as needed to ORDERBY in order to ensure each row can be uniquely identified by the ORDERBY and PARTITIONBY columns:

DAX

```
1stAttemptWithORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS(
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([Color])
```

```
            ),
            [CurrentYearSales]
        )
    )
```

Adding PARTITIONBY

Now, to *almost* get the result we're after we can use PARTITIONBY, as shown in the following calculated table expression:

DAX

```
UsingPARTITIONBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Notice that specifying ORDERBY is optional here because ORDERBY automatically contains all the columns from the relation that aren't specified in PARTITIONBY. So, the following expression returns the same results because ORDERBY is set to CalendarYear and CurrentYearSales automatically:

DAX

```
UsingPARTITIONBYWithoutORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
```

```
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

> **Note**
>
> While ORDERBY is set to CalendarYear and CurrentYearSales automatically, no guarantee is given as to what order in which they'll be added. If CurrentYearSales is added before CalendarYear, the resulting order isn't inline with what's expected. **Be explicit when specifying ORDERBY and PARTITIONBY to avoid confusion and unexpected results**.

Both expressions return the result we're after:

Expand table

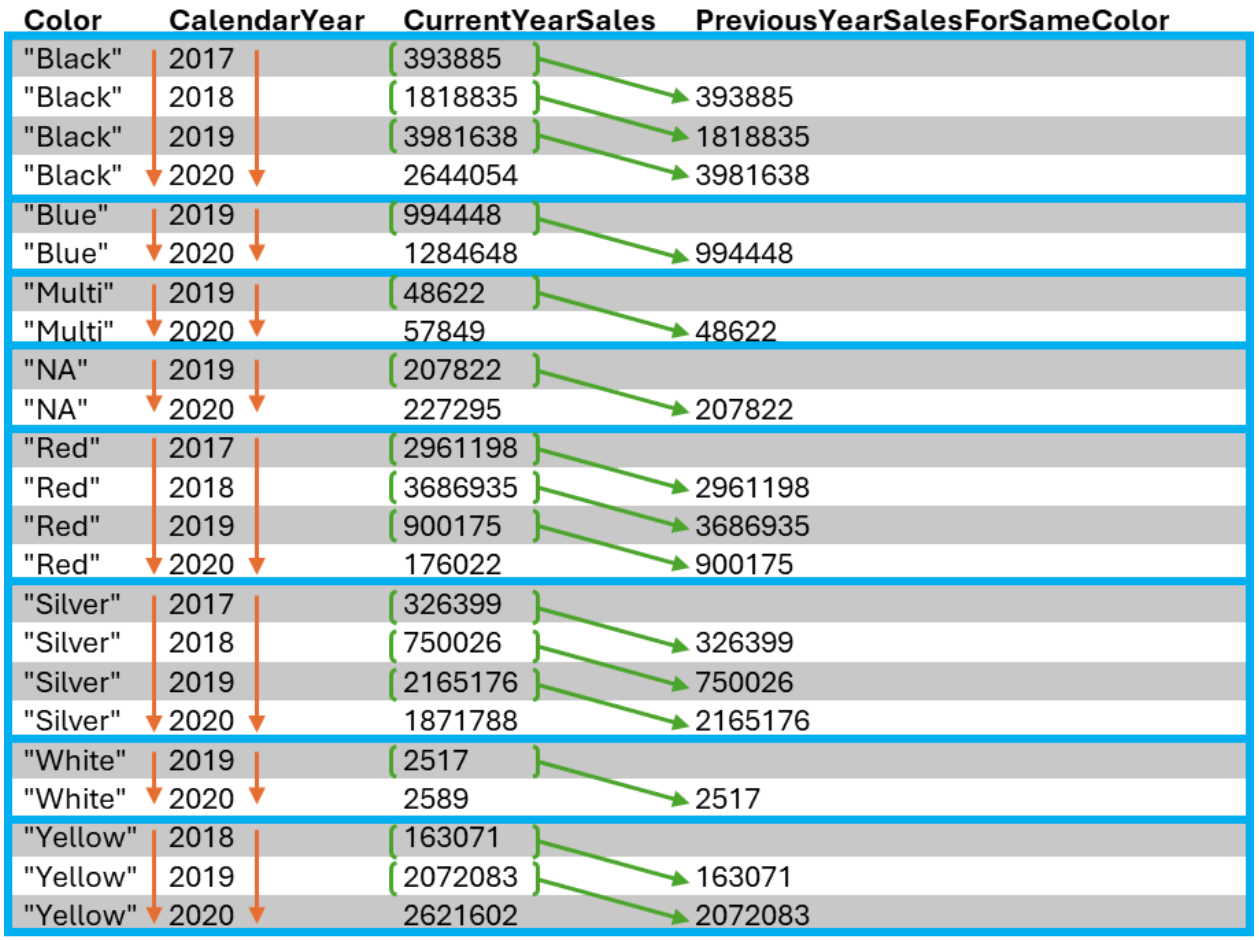
Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
"Black"	2017	393885	
"Black"	2018	1818835	393885
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
"Blue"	2019	994448	
"Blue"	2020	1284648	994448
"Multi"	2019	48622	
"Multi"	2020	57849	48622
"NA"	2019	207822	
"NA"	2020	227295	207822

Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
"Red"	2017	2961198	
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	
"White"	2020	2589	2517
"Yellow"	2018	163071	
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

As you see in this table, the PreviousYearSalesForSameColor column shows the sales for the previous year for the same color. For [Red, 2020], it returns the sales for [Red, 2019], and so on. If there's no previous year, for example in the case of [Red, 2017], no value is returned.

You can think of PARTITIONBY as a way to divide the table into parts in which to execute the OFFSET calculation. In the example above, the table is divided into as many parts as there are colors, one for each color. Then, within each part, the OFFSET is calculated, sorted by CalendarYear.

Visually, what's happening is this:



Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
"Black"	2017	393885	
"Black"	2018	1818835	393885
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
"Blue"	2019	994448	
"Blue"	2020	1284648	994448
"Multi"	2019	48622	
"Multi"	2020	57849	48622
"NA"	2019	207822	
"NA"	2020	227295	207822
"Red"	2017	2961198	
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	
"White"	2020	2589	2517
"Yellow"	2018	163071	
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

First, the call to PARTITIONBY results in the table getting divided into parts, one for each Color. This is represented by the light blue boxes in the table image. Next, ORDERBY makes sure that each part is sorted by CalendarYear (represented by the orange arrows). Finally, within each sorted part, for each row, OFFSET finds the row above it and returns that value in the PreviousYearSalesForSameColor column. Since for every first row in each part there is no previous row in that same part, the result in that row for the PreviousYearSalesForSameColor column is empty.

To achieve the final result, we simply have to subtract CurrentYearSales from the previous year sales for the same color returned by the call to OFFSET. Since we're not interested in showing the previous year sales for the same color, but only in the current year sales and the year over year difference. Here's the final calculated table expression:

DAX

```
FinalResult =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
```

```
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

And here's the result of that expression:

Expand table

Color	CalendarYear	CurrentYearSales	YoYSalesForSameColor
"Black"	2017	393885	393885
"Black"	2018	1818835	1424950
"Black"	2019	3981638	2162803
"Black"	2020	2644054	-1337584
"Blue"	2019	994448	994448
"Blue"	2020	1284648	290200
"Multi"	2019	48622	48622
"Multi"	2020	57849	9227
"NA"	2019	207822	207822
"NA"	2020	227295	19473
"Red"	2017	2961198	2961198
"Red"	2018	3686935	725737
"Red"	2019	900175	-2786760
"Red"	2020	176022	-724153
"Silver"	2017	326399	326399
"Silver"	2018	750026	423627

Color	CalendarYear	CurrentYearSales	YoYSalesForSameColor
"Silver"	2019	2165176	1415150
"Silver"	2020	1871788	-293388
"White"	2019	2517	2517
"White"	2020	2589	72
"Yellow"	2018	163071	163071
"Yellow"	2019	2072083	1909012
"Yellow"	2020	2621602	549519

Using MATCHBY

You might have noticed we didn't specify MATCHBY at all. In this case, it isn't necessary. The columns in ORDERBY and PARTITIONBY (for as far as they were specified in the examples above) are sufficient to uniquely identify each row. Since we didn't specify MATCHBY, the columns specified in ORDERBY and PARTITIONBY are used to uniquely identify each row so they can be compared to enable OFFSET to give a meaningful result. If the columns in ORDERBY and PARTITIONBY can't uniquely identify each row, additional columns can be added to the ORDERBY clause if those extra columns allow each row to be uniquely identified. If that's not possible, an error is returned. In this last case, specifying MATCHBY may help to resolve the error.

If MATCHBY is specified, the columns in MATCHBY and PARTITIONBY are used to uniquely identify each row. If that's not possible, an error is returned. Even if MATCHBY isn't required, consider explicitly specifying MATCHBY to avoid any confusion.

Continuing from the examples above, here's the last expression:

DAX

```
FinalResult =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
```

```
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

If we want to be explicit about how rows should be uniquely identified, we can specify MATCHBY as shown in the following equivalent expression:

DAX

```
FinalResultWithExplicitMATCHBYOnColorAndCalendarYear =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color]),
                MATCHBY ([Color], [CalendarYear])
            ),
            [CurrentYearSales]
        )
    )
```

Since MATCHBY is specified, both the columns specified in MATCHBY as well as in PARTITIONBY are used to uniquely identify rows. Since Color is specified in both MATCHBY and PARTITIONBY, the following expression is equivalent to the previous expression:

DAX

```
FinalResultWithExplicitMATCHBYOnCalendarYear =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color]),
                MATCHBY ([CalendarYear])
            ),
            [CurrentYearSales]
        )
    )
```

Since specifying MATCHBY isn't necessary in the examples we've looked at so far, let's look at a slightly different example that does require MATCHBY. In this case, we have a list of order lines. Each row represents an order line for an order. An order can have multiple order lines and order line 1 appears on many orders. In addition, for each order line we have a ProductKey and a SalesAmount. A sample of the relevant columns on the table looks like this:

Expand table

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount
SO51900	1	528	4.99
SO51948	1	528	5.99
SO52043	1	528	4.99
SO52045	1	528	4.99
SO52094	1	528	4.99
SO52175	1	528	4.99
SO52190	1	528	4.99

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount
SO52232	1	528	4.99
SO52234	1	528	4.99
SO52234	2	529	3.99

Notice SalesOrderNumber and SalesOrderLineNumber are both required to uniquely identify rows.

For each order, we want to return the previous sales amount of the same product (represented by the ProductKey) ordered by the SalesAmount in descending order. The following expression won't work because there are potentially multiple rows in vRelation as it's passed into OFFSET:

DAX

```
ThisExpressionFailsBecauseMATCHBYIsMissing =
    ADDCOLUMNS (
        FactInternetSales,
        "Previous Sales Amount",
            SELECTCOLUMNS (
                OFFSET (
                    -1,
                    FactInternetSales,
                    ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                    PARTITIONBY ( FactInternetSales[ProductKey] )
                ),
                FactInternetSales[SalesAmount]
            )
    )
```

This expression returns an error: "OFFSET's Relation parameter may have duplicate rows, which isn't allowed."

In order to make this expression work, MATCHBY must be specified and must include all columns that uniquely define a row. MATCHBY is required here because the relation, FactInternetSales, doesn't contain any explicit keys or unique columns. However, the columns SalesOrderNumber and SalesOrderLineNumber together form a *composite key*, where their existence together is unique in the relation and can therefore uniquely identify each row. Just specifying SalesOrderNumber or SalesOrderLineNumber isn't enough as both columns contain repeating values. The following expression solves the problem:

DAX

```
ThisExpressionWorksBecauseOfMATCHBY =
    ADDCOLUMNS (
        FactInternetSales,
        "Previous Sales Amount",
            SELECTCOLUMNS (
                OFFSET (
                    -1,
                    FactInternetSales,
                    ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                    PARTITIONBY ( FactInternetSales[ProductKey] ),
                    MATCHBY ( FactInternetSales[SalesOrderNumber],
                                FactInternetSales[SalesOrderLineNumber] )
                ),
                FactInternetSales[SalesAmount]
            )
    )
```

And this expression does indeed return the results we're after:

Expand table

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount	Previous Sales Amount
SO51900	1	528	5.99	
SO51948	1	528	4.99	5.99
SO52043	1	528	4.99	4.99
SO52045	1	528	4.99	4.99
SO52094	1	528	4.99	4.99
SO52175	1	528	4.99	4.99
SO52190	1	528	4.99	4.99
SO52232	1	528	4.99	4.99
SO52234	1	528	4.99	4.99
SO52234	2	529	3.99	

Related content

ORDERBY PARTITIONBY MATCHBY INDEX OFFSET WINDOW RANK ROWNUMBER

Feedback

Was this page helpful? Yes No

Appropriate use of error functions

Article • 09/20/2022

As a data modeler, when you write a DAX expression that might raise an evaluation-time error, you can consider using two helpful DAX functions.

- The ISERROR function, which takes a single expression and returns TRUE if that expression results in error.
- The IFERROR function, which takes two expressions. Should the first expression result in error, the value for the second expression is returned. It is in fact a more optimized implementation of nesting the ISERROR function inside an IF function.

However, while these functions can be helpful and can contribute to writing easy-to-understand expressions, they can also significantly degrade the performance of calculations. It can happen because these functions increase the number of storage engine scans required.

Most evaluation-time errors are due to unexpected BLANKs or zero values, or invalid data type conversion.

Recommendations

It's better to avoid using the ISERROR and IFERROR functions. Instead, apply defensive strategies when developing the model and writing expressions. Strategies can include:

- **Ensuring quality data is loaded into the model:** Use Power Query transformations to remove or substitute invalid or missing values, and to set correct data types. A Power Query transformation can also be used to filter rows when errors, like invalid data conversion, occur.

  Data quality can also be controlled by setting the model column **Is Nullable** property to Off, which will fail the data refresh should BLANKs be encountered. If this failure occurs, data loaded as a result of a successful refresh will remain in the tables.

- **Using the IF function:** The IF function logical test expression can determine whether an error result would occur. Note, like the ISERROR and IFERROR functions, this function can result in additional storage engine scans, but will likely perform better than them as no error needs to be raised.

- **Using error-tolerant functions:** Some DAX functions will test and compensate for error conditions. These functions allow you to enter an alternate result that would

be returned instead. The DIVIDE function is one such example. For additional guidance about this function, read the DAX: DIVIDE function vs divide operator (/) article.

Example

The following measure expression tests whether an error would be raised. It returns BLANK in this instance (which is the case when you do not provide the IF function with a value-if-false expression).

```
DAX

Profit Margin
= IF(ISERROR([Profit] / [Sales]))
```

This next version of the measure expression has been improved by using the IFERROR function in place of the IF and ISERROR functions.

```
DAX

Profit Margin
= IFERROR([Profit] / [Sales], BLANK())
```

However, this final version of the measure expression achieves the same outcome, yet more efficiently and elegantly.

```
DAX

Profit Margin
= DIVIDE([Profit], [Sales])
```

See also

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

Avoid converting BLANKs to values

Article • 09/20/2022

As a data modeler, when writing measure expressions you might come across cases where a meaningful value can't be returned. In these instances, you may be tempted to return a value—like zero—instead. It's suggested you carefully determine whether this design is efficient and practical.

Consider the following measure definition that explicitly converts BLANK results to zero.

DAX

```
Sales (No Blank) =
IF(
    ISBLANK([Sales]),
    0,
    [Sales]
)
```

Consider another measure definition that also converts BLANK results to zero.

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales], 0)
```

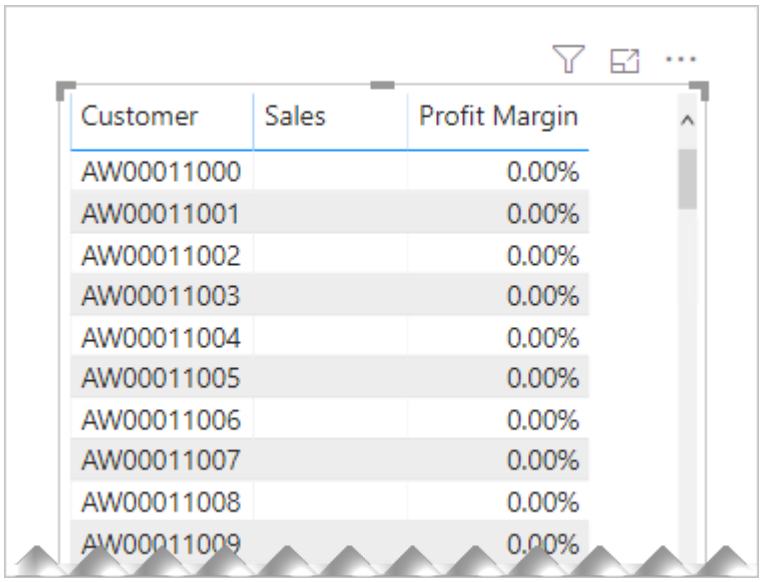
The DIVIDE function divides the **Profit** measure by the **Sales** measure. Should the result be zero or BLANK, the third argument—the alternate result (which is optional)—is returned. In this example, because zero is passed as the alternate result, the measure is guaranteed to always return a value.

These measure designs are inefficient and lead to poor report designs.

When they're added to a report visual, Power BI attempts to retrieve all groupings within the filter context. The evaluation and retrieval of large query results often leads to slow report rendering. Each example measure effectively turns a sparse calculation into a dense one, forcing Power BI to use more memory than necessary.

Also, too many groupings often overwhelm your report users.

Let's see what happens when the **Profit Margin** measure is added to a table visual, grouping by customer.



Customer	Sales	Profit Margin
AW00011000		0.00%
AW00011001		0.00%
AW00011002		0.00%
AW00011003		0.00%
AW00011004		0.00%
AW00011005		0.00%
AW00011006		0.00%
AW00011007		0.00%
AW00011008		0.00%
AW00011009		0.00%

The table visual displays an overwhelming number of rows. (There are in fact 18,484 customers in the model, and so the table attempts to display all of them.) Notice that the customers in view haven't achieved any sales. Yet, because the **Profit Margin** measure always returns a value, they are displayed.

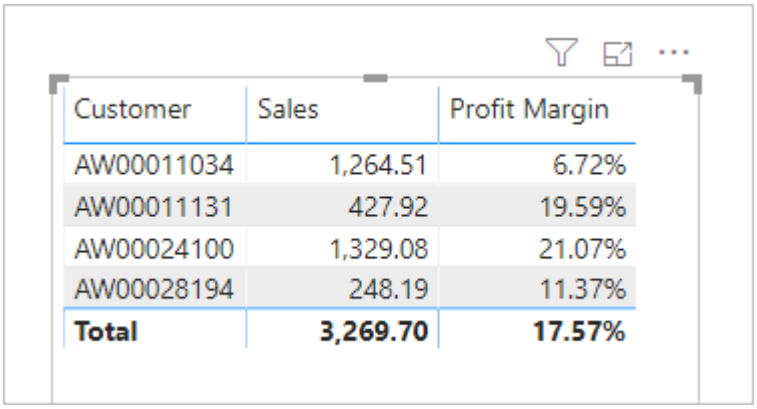
> **Note**
>
> When there are too many data points to display in a visual, Power BI may use data reduction strategies to remove or summarize large query results. For more information, see **Data point limits and strategies by visual type**.

Let's see what happens when the **Profit Margin** measure definition is improved. It now returns a value only when the **Sales** measure isn't BLANK (or zero).

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales])
```

The table visual now displays only customers who have made sales within the current filter context. The improved measure results in a more efficient and practical experience for your report users.



Customer	Sales	Profit Margin
AW00011034	1,264.51	6.72%
AW00011131	427.92	19.59%
AW00024100	1,329.08	21.07%
AW00028194	248.19	11.37%
Total	**3,269.70**	**17.57%**

> **Tip**
>
> When necessary, you can configure a visual to display all groupings (that return values or BLANK) within the filter context by enabling the **Show Items With No Data** option.

Recommendation

It's recommended that your measures return BLANK when a meaningful value cannot be returned.

This design approach is efficient, allowing Power BI to render reports faster. Also, returning BLANK is better because report visuals—by default—eliminate groupings when summarizations are BLANK.

See also

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

Avoid using FILTER as a filter argument

Article • 09/20/2022

As a data modeler, it's common you'll write DAX expressions that need to be evaluated in a modified filter context. For example, you can write a measure definition to calculate sales for "high margin products". We'll describe this calculation later in this article.

> **Note**
>
> This article is especially relevant for model calculations that apply filters to Import tables.

The CALCULATE and CALCULATETABLE DAX functions are important and useful functions. They let you write calculations that remove or add filters, or modify relationship paths. It's done by passing in filter arguments, which are either Boolean expressions, table expressions, or special filter functions. We'll only discuss Boolean and table expressions in this article.

Consider the following measure definition, which calculates red product sales by using a table expression. It will replace any filters that might be applied to the **Product** table.

DAX

```
Red Sales =
CALCULATE(
    [Sales],
    FILTER('Product', 'Product'[Color] = "Red")
)
```

The CALCULATE function accepts a table expression returned by the FILTER DAX function, which evaluates its filter expression for each row of the **Product** table. It achieves the correct result—the sales result for red products. However, it could be achieved much more efficiently by using a Boolean expression.

Here's an improved measure definition, which uses a Boolean expression instead of the table expression. The KEEPFILTERS DAX function ensures any existing filters applied to the **Color** column are preserved, and not overwritten.

DAX

```
Red Sales =
CALCULATE(
    [Sales],
```

```
    KEEPFILTERS('Product'[Color] = "Red")
)
```

It's recommended you pass filter arguments as Boolean expressions, whenever possible. It's because Import model tables are in-memory column stores. They are explicitly optimized to efficiently filter columns in this way.

There are, however, restrictions that apply to Boolean expressions when they're used as filter arguments. They:

- Cannot reference columns from multiple tables
- Cannot reference a measure
- Cannot use nested CALCULATE functions
- Cannot use functions that scan or return a table

It means that you'll need to use table expressions for more complex filter requirements.

Consider now a different measure definition. The requirement is to calculate sales, but only for months that have achieved a profit.

DAX

```
Sales for Profitable Months =
CALCULATE(
    [Sales],
    FILTER(
        VALUES('Date'[Month]),
        [Profit] > 0)
    )
)
```

In this example, the FILTER function must be used. It's because it requires evaluating the **Profit** measure to eliminate those months that didn't achieve a profit. It's not possible to use a measure in a Boolean expression when it's used as a filter argument.

Recommendations

For best performance, it's recommended you use Boolean expressions as filter arguments, whenever possible.

Therefore, the FILTER function should only be used when necessary. You can use it to perform filter complex column comparisons. These column comparisons can involve:

- Measures
- Other columns

- Using the OR DAX function, or the OR logical operator (||)

See also

- Filter functions (DAX)
- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

Column and measure references

Article • 09/20/2022

As a data modeler, your DAX expressions will refer to model columns and measures. Columns and measures are always associated with model tables, but these associations are different, so we have different recommendations on how you'll reference them in your expressions.

Columns

A column is a table-level object, and column names must be unique within a table. So it's possible that the same column name is used multiple times in your model—providing they belong to different tables. There's one more rule: a column name cannot have the same name as a measure name or hierarchy name that exists in the same table.

In general, DAX will not force using a *fully qualified* reference to a column. A fully qualified reference means that the table name precedes the column name.

Here's an example of a calculated column definition using only column name references. The **Sales** and **Cost** columns both belong to a table named **Orders**.

DAX

```
Profit = [Sales] - [Cost]
```

The same definition can be rewritten with fully qualified column references.

DAX

```
Profit = Orders[Sales] - Orders[Cost]
```

Sometimes, however, you'll be required to use fully qualified column references when Power BI detects ambiguity. When entering a formula, a red squiggly and error message will alert you. Also, some DAX functions like the LOOKUPVALUE DAX function, require the use of fully qualified columns.

It's recommended you always fully qualify your column references. The reasons are provided in the Recommendations section.

Measures

A measure is a model-level object. For this reason, measure names must be unique within the model. However, in the **Fields** pane, report authors will see each measure associated with a single model table. This association is set for cosmetic reasons, and you can configure it by setting the **Home Table** property for the measure. For more information, see Measures in Power BI Desktop (Organizing your measures).

It's possible to use a fully qualified measure in your expressions. DAX intellisense will even offer the suggestion. However, it isn't necessary, and it's not a recommended practice. If you change the home table for a measure, any expression that uses a fully qualified measure reference to it will break. You'll then need to edit each broken formula to remove (or update) the measure reference.

It's recommended you never qualify your measure references. The reasons are provided in the Recommendations section.

Recommendations

Our recommendations are simple and easy to remember:

- Always use fully qualified column references
- Never use fully qualified measure references

Here's why:

- **Formula entry**: Expressions will be accepted, as there won't be any ambiguous references to resolve. Also, you'll meet the requirement for those DAX functions that require fully qualified column references.
- **Robustness**: Expressions will continue to work, even when you change a measure home table property.
- **Readability**: Expressions will be quick and easy to understand—you'll quickly determine that it's a column or measure, based on whether it's fully qualified or not.

See also

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

Chinese coded character set (GB18030-2022) support

China's GB18030-2022 standard is the latest update to the national character set requirements. It ensures compatibility with Unicode 11.0 and mandates support for additional characters, including minority scripts and emoji. For organizations operating in or with China, compliance is not optional; it's a regulatory requirement.

Power BI can be configured to respect GB18030-2022 encoding using the `UnicodeCharacterBehavior` setting. This setting is set to `CodeUnits` by default. To ensure your model is compatible with GB18030-2022, you'll need to execute a specific XMLA command to set `UnicodeCharacterBehavior` to `CodePoints`, followed by a model refresh.

XMLA command

xmla

```
<Alter AllowCreate="true" ObjectExpansion="ObjectProperties"
xmlns="http://schemas.microsoft.com/analysisservices/2003/engine">
    <Object>
        <DatabaseID>[your database id]</DatabaseID>
    </Object>
    <ObjectDefinition>
        <Database xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:ddl2="http://schemas.microsoft.com/analysisservices/2003/engine/2"
        xmlns:ddl2_2="http://schemas.microsoft.com/analysisservices/2003/engine/2/2"

xmlns:ddl100_100="http://schemas.microsoft.com/analysisservices/2008/engine/100/100"
        xmlns:ddl200="http://schemas.microsoft.com/analysisservices/2010/engine/200"

xmlns:ddl200_200="http://schemas.microsoft.com/analysisservices/2010/engine/200/200"
        xmlns:ddl300="http://schemas.microsoft.com/analysisservices/2011/engine/300"

xmlns:ddl300_300="http://schemas.microsoft.com/analysisservices/2011/engine/300/300"
        xmlns:ddl400="http://schemas.microsoft.com/analysisservices/2012/engine/400"

xmlns:ddl400_400="http://schemas.microsoft.com/analysisservices/2012/engine/400/400"
        xmlns:ddl500="http://schemas.microsoft.com/analysisservices/2013/engine/500"

xmlns:ddl500_500="http://schemas.microsoft.com/analysisservices/2013/engine/500/500"
>
            <ID>[your model id]</ID>
            <Name>[your model name]</Name>
            <ddl200:CompatibilityLevel>[your model compatibility level]
</ddl200:CompatibilityLevel>

<ddl200_200:StorageEngineUsed>TabularMetadata</ddl200_200:StorageEngineUsed>
```

```
            <Language>1033</Language>
            <UnicodeCharacterBehavior
xmlns="http://schemas.microsoft.com/analysisservices/2025/engine/924/924">CodePoints
</UnicodeCharacterBehavior>
        </Database>
    </ObjectDefinition>
</Alter>
```

After executing this XMLA command, perform a full refresh of your model.

Example

The UnicodeCharacterBehavior influences any DAX function that determines the length of a text string, which include FIND, LEFT, LEN, MID, REPLACE, RIGHT. These functions will exhibit different behaviors when working with text strings that contain Unicode characters. Let's see the difference in action. Here's a measure that uses LEN to calculate the length of a text string:

DAX

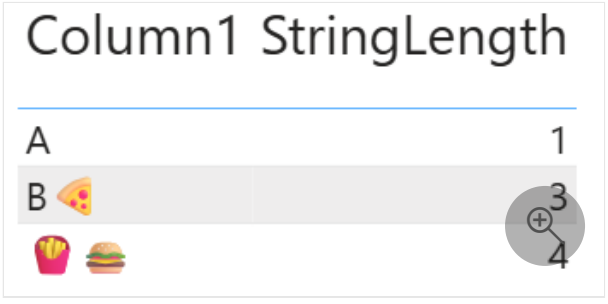
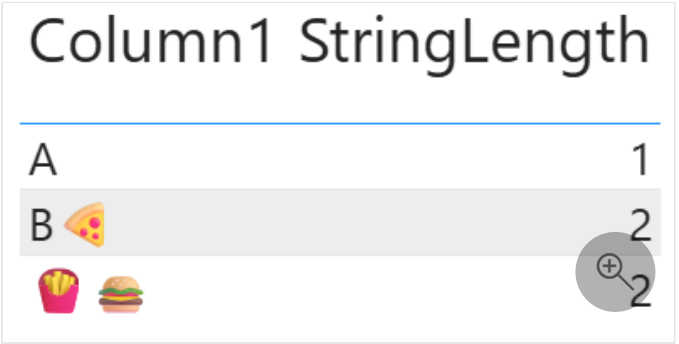
```
StringLength = LEN ( SELECTEDVALUE ( 'Table'[Column1] ) )
```

In this example, `Column1` contains three values:

- A
- B🍕
- 🍟🍔

Here's a before and after comparison of the result of StringLength on a column that contains Unicode characters:

Expand table

`UnicodeCharacterBehavior = CodeUnits` **(default)**	`UnicodeCharacterBehavior = CodePoints`
	

Notice how on the left each Unicode character has length 2, where on the right, each Unicode character has length 1.

> **Note**
>
> Changes to `UnicodeCharacterBehavior` only take hold after a model refresh.

Related content

- FIND
- LEFT
- LEN
- MID
- REPLACE
- RIGHT

Last updated on 01/20/2026

DIVIDE function vs. divide operator (/)

Article • 09/20/2022

As a data modeler, when you write a DAX expression to divide a numerator by a denominator, you can choose to use the DIVIDE function or the divide operator (/ - forward slash).

When using the DIVIDE function, you must pass in numerator and denominator expressions. Optionally, you can pass in a value that represents an *alternate result*.

DAX

```
DIVIDE(<numerator>, <denominator> [,<alternateresult>])
```

The DIVIDE function was designed to automatically handle division by zero cases. If an alternate result is not passed in, and the denominator is zero or BLANK, the function returns BLANK. When an alternate result is passed in, it's returned instead of BLANK.

The DIVIDE function is convenient because it saves your expression from having to first test the denominator value. The function is also better optimized for testing the denominator value than the IF function. The performance gain is significant since checking for division by zero is expensive. Further using DIVIDE results in a more concise and elegant expression.

Example

The following measure expression produces a safe division, but it involves using four DAX functions.

DAX

```
Profit Margin =
IF(
    OR(
        ISBLANK([Sales]),
        [Sales] == 0
    ),
    BLANK(),
    [Profit] / [Sales]
)
```

This measure expression achieves the same outcome, yet more efficiently and elegantly.

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales])
```

Recommendations

It's recommended that you use the DIVIDE function whenever the denominator is an expression that *could* return zero or BLANK.

In the case that the denominator is a constant value, we recommend that you use the divide operator. In this case, the division is guaranteed to succeed, and your expression will perform better because it will avoid unnecessary testing.

Carefully consider whether the DIVIDE function should return an alternate value. For measures, it's usually a better design that they return BLANK. Returning BLANK is better because report visuals—by default—eliminate groupings when summarizations are BLANK. It allows the visual to focus attention on groups where data exists. When necessary, in Power BI, you can configure the visual to display all groups (that return values or BLANK) within the filter context by enabling the Show items with no data option.

See also

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

Use SELECTEDVALUE instead of VALUES

Article • 12/13/2023

As a data modeler, sometimes you might need to write a DAX expression that tests whether a column is filtered by a specific value.

In earlier versions of DAX, this requirement was safely achieved by using a pattern involving three DAX functions; IF, HASONEVALUE and VALUES. The following measure definition presents an example. It calculates the sales tax amount, but only for sales made to Australian customers.

DAX

```
Australian Sales Tax =
IF(
    HASONEVALUE(Customer[Country-Region]),
    IF(
        VALUES(Customer[Country-Region]) = "Australia",
        [Sales] * 0.10
    )
)
```

In the example, the HASONEVALUE function returns TRUE only when a single value of the **Country-Region** column is visible in the current filter context. When it's TRUE, the VALUES function is compared to the literal text "Australia". When the VALUES function returns TRUE, the **Sales** measure is multiplied by 0.10 (representing 10%). If the HASONEVALUE function returns FALSE—because more than one value filters the column—the first IF function returns BLANK.

The use of the HASONEVALUE is a defensive technique. It's required because it's possible that multiple values filter the **Country-Region** column. In this case, the VALUES function returns a table of multiple rows. Comparing a table of multiple rows to a scalar value results in an error.

Recommendation

It's recommended that you use the SELECTEDVALUE function. It achieves the same outcome as the pattern described in this article, yet more efficiently and elegantly.

Using the SELECTEDVALUE function, the example measure definition is now rewritten.

DAX

```
Australian Sales Tax =
IF(
    SELECTEDVALUE(Customer[Country-Region]) = "Australia",
    [Sales] * 0.10
)
```

Tip

It's possible to pass an *alternate result* value into the SELECTEDVALUE function. The alternate result value is returned when either no filters—or multiple filters—are applied to the column.

Related content

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Use COUNTROWS instead of COUNT

Article • 12/13/2023

As a data modeler, sometimes you might need to write a DAX expression that counts table rows. The table could be a model table or an expression that returns a table.

Your requirement can be achieved in two ways. You can use the COUNT function to count column values, or you can use the COUNTROWS function to count table rows. Both functions will achieve the same result, providing that the counted column contains no BLANKs.

The following measure definition presents an example. It calculates the number of **OrderDate** column values.

DAX

```
Sales Orders =
COUNT(Sales[OrderDate])
```

Providing that the granularity of the **Sales** table is one row per sales order, and the **OrderDate** column does not contain BLANKs, then the measure will return a correct result.

However, the following measure definition is a better solution.

DAX

```
Sales Orders =
COUNTROWS(Sales)
```

There are three reasons why the second measure definition is better:

- It's more efficient, and so it will perform better.
- It doesn't consider BLANKs contained in any column of the table.
- The intention of formula is clearer, to the point of being self-describing.

Recommendation

When it's your intention to count table rows, it's recommended you always use the COUNTROWS function.

Related content

- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community
- Suggestions? Contribute ideas to improve Power BI

Feedback

Was this page helpful? Yes No

Use variables to improve your DAX formulas

Article • 10/31/2022

As a data modeler, writing and debugging some DAX calculations can be challenging. It's common that complex calculation requirements often involve writing compound or complex expressions. Compound expressions can involve the use of many nested functions, and possibly the reuse of expression logic.

Using variables in your DAX formulas can help you write more complex and efficient calculations. Variables can improve performance, reliability, readability, and reduce complexity.

In this article, we'll demonstrate the first three benefits by using an example measure for year-over-year (YoY) sales growth. (The formula for YoY sales growth is period sales, minus sales for the same period last year, divided by sales for the same period last year.)

Let's start with the following measure definition.

DAX

```
Sales YoY Growth % =
DIVIDE(
    ([Sales] - CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12,
MONTH))),
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
)
```

The measure produces the correct result, yet let's now see how it can be improved.

Improve performance

Notice that the formula repeats the expression that calculates "same period last year". This formula is inefficient, as it requires Power BI to evaluate the same expression twice. The measure definition can be made more efficient by using a variable, VAR.

The following measure definition represents an improvement. It uses an expression to assign the "same period last year" result to a variable named **SalesPriorYear**. The variable is then used twice in the RETURN expression.

DAX

```
Sales YoY Growth % =
VAR SalesPriorYear =
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
RETURN
    DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
```

The measure continues to produce the correct result, and does so in about half the query time.

Improve readability

In the previous measure definition, notice how the choice of variable name makes the RETURN expression simpler to understand. The expression is short and self-describing.

Simplify debugging

Variables can also help you debug a formula. To test an expression assigned to a variable, you temporarily rewrite the RETURN expression to output the variable.

The following measure definition returns only the **SalesPriorYear** variable. Notice how it comments-out the intended RETURN expression. This technique allows you to easily revert it back once your debugging is complete.

DAX

```
Sales YoY Growth % =
VAR SalesPriorYear =
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
RETURN
    --DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
    SalesPriorYear
```

Reduce complexity

In earlier versions of DAX, variables were not yet supported. Complex expressions that introduced new filter contexts were required to use the EARLIER or EARLIEST DAX functions to reference outer filter contexts. Unfortunately, data modelers found these functions difficult to understand and use.

Variables are always evaluated outside the filters your RETURN expression applies. For this reason, when you use a variable within a modified filter context, it achieves the same result as the EARLIEST function. The use of the EARLIER or EARLIEST functions can

therefore be avoided. It means you can now write formulas that are less complex, and that are easier to understand.

Consider the following calculated column definition added to the **Subcategory** table. It evaluates a rank for each product subcategory based on the **Subcategory Sales** column values.

DAX

```
Subcategory Sales Rank =
COUNTROWS(
    FILTER(
        Subcategory,
        EARLIER(Subcategory[Subcategory Sales]) < Subcategory[Subcategory
Sales]
    )
) + 1
```

The EARLIER function is used to refer to the **Subcategory Sales** column value *in the current row context*.

The calculated column definition can be improved by using a variable instead of the EARLIER function. The **CurrentSubcategorySales** variable stores the **Subcategory Sales** column value *in the current row context*, and the RETURN expression uses it within a modified filter context.

DAX

```
Subcategory Sales Rank =
VAR CurrentSubcategorySales = Subcategory[Subcategory Sales]
RETURN
    COUNTROWS(
        FILTER(
            Subcategory,
            CurrentSubcategorySales < Subcategory[Subcategory Sales]
        )
    ) + 1
```

See also

- VAR DAX article
- Learning path: Use DAX in Power BI Desktop
- Questions? Try asking the Power BI Community

Feedback

Was this page helpful? Yes No

Get help at Microsoft Q&A

DAX user-defined functions

DAX user-defined functions (UDFs) let you package DAX logic and reuse it like any other DAX function. UDFs introduce a new `FUNCTION` keyword, support parameters (scalar, table, and references) with optional default expressions, and include type checking helpers that make authoring safer and clearer. After you define a UDF, you can use it in a measure, calculated column, visual calculation, or even other user-defined functions. Users can centralize business rules, improve maintainability, and evolve calculations safely over time. Functions are first-class model objects you can create and manage in DAX query view and TMDL view, and they can be viewed in Model explorer under the *Functions* node.

DAX UDFs require database compatibility level 1702 or higher and the feature is generally available in Power BI Desktop and Power BI Service as of the June 2026 release.

Define and manage user-defined functions

There are several locations to define and manage functions:

- DAX query view (DQV). Define and modify functions in DQV. DQV also includes context-menu **Quick queries** (Evaluate, Define and evaluate, and Define all functions in this model) to help you test and manage UDFs fast.
- TMDL view. UDFs can also be authored and edited in TMDL. TMDL view also includes context-menu **Script TMDL to**.
- Model explorer. New functions can be created and existing functions can be modified using the formula bar. Existing functions can be viewed under the *Functions* node in Model explorer.

When defining a UDF, please follow these naming requirements:

Function names:

- Must be well-formed and unique within the model.
- Can include periods (dots) for namespacing (e.g. Microsoft.PowerBI.MyFunc). Cannot start or end with a period or have consecutive periods.
- Other than periods, names can only contain alphanumeric characters or underscores. No spaces or special characters allowed.
- Must not conflict with built-in DAX functions or reserved words (e.g. measure, function, define).

Parameter names:

- Can only contain alphanumeric characters or underscores. Periods are not allowed.
- Must not be a reserved word.

Using DAX query view

You can define, update, and evaluate user-defined functions in DAX query view. For additional information on DAX query view, see DAX query view.

General form

DAX

```
DEFINE
    /// Optional description above the function
    /// @param {ParameterType} ParameterName - ParameterDescription
    /// ...
    /// @returns Return description
    FUNCTION <FunctionName> = ( [<ParameterName> [: [<ParameterType>]
[<ParameterSubtype>] [<ParameterPassingMode>]] [= <DefaultExpression>], ...] ) =>
<FunctionBody>
```

> **Tip**
>
> Using JSDoc block tags, document your functions with descriptions, parameter names and types, and return information to make them easier to consume. Note the `///` for function descriptions. Single-line (`//`) or multi-line (`/* */`) comments will not appear in IntelliSense function descriptions.

Example: Simple tax function

DAX

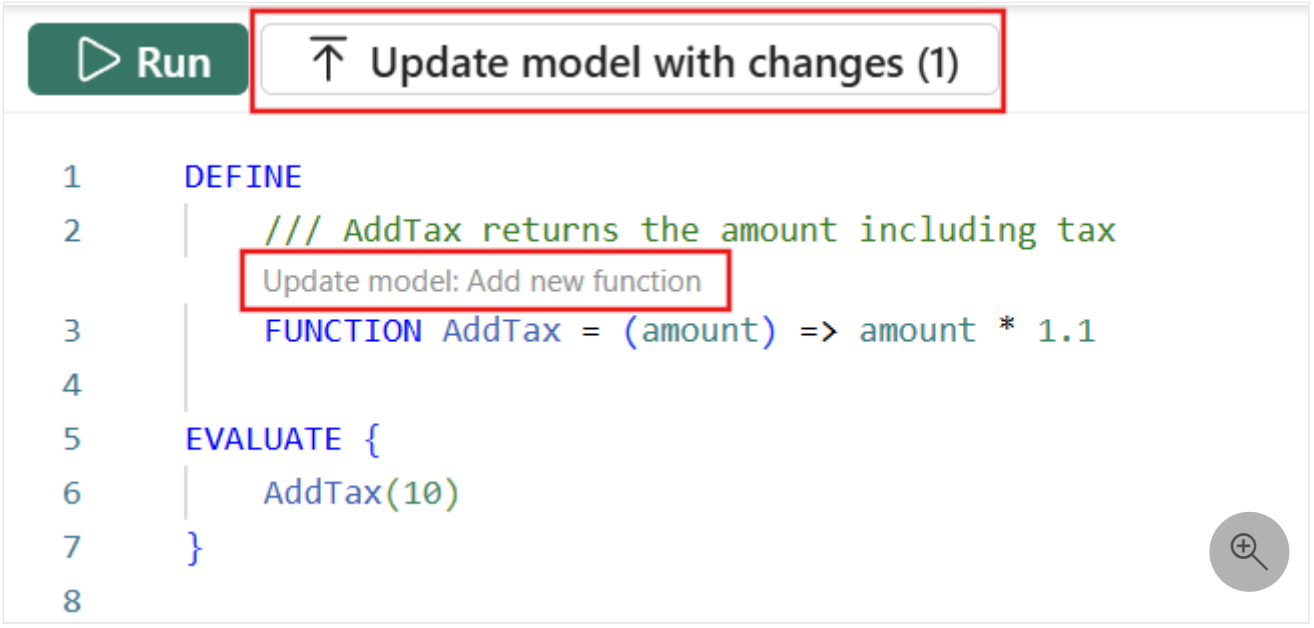
```
DEFINE
    /// AddTax takes in amount and returns amount including tax
    /// @param {NUMERIC} amount - The pre-tax value to which tax will be applied
    /// @param {NUMERIC} [taxRate] - Optional tax rate to apply, default is 0.1 (10%)
    /// @returns The amount including 10% tax
    FUNCTION AddTax =
        ( amount : NUMERIC, taxRate : NUMERIC = 0.1 ) =>
            amount * ( 1 + taxRate )

EVALUATE
```

```
{ AddTax ( 10 ) }
// Returns 11
```

Saving to the model

To save a UDF from DAX query view to the model:

- Click **Update model with changes** to save all UDFs in the query.
- Or click **Update model: Add new function** above the defined function to save a single UDF.



Using TMDL view

You can define and/or update user-defined functions in TMDL view. For additional information on TMDL view, see TMDL view.

General form

tmdl

```
createOrReplace
    /// Optional description above the function
    function <FunctionName> = ( [<ParameterName> [: <ParameterType>] [=
<DefaultExpression>], ...] ) => <FunctionBody>
```

Example: Simple tax function

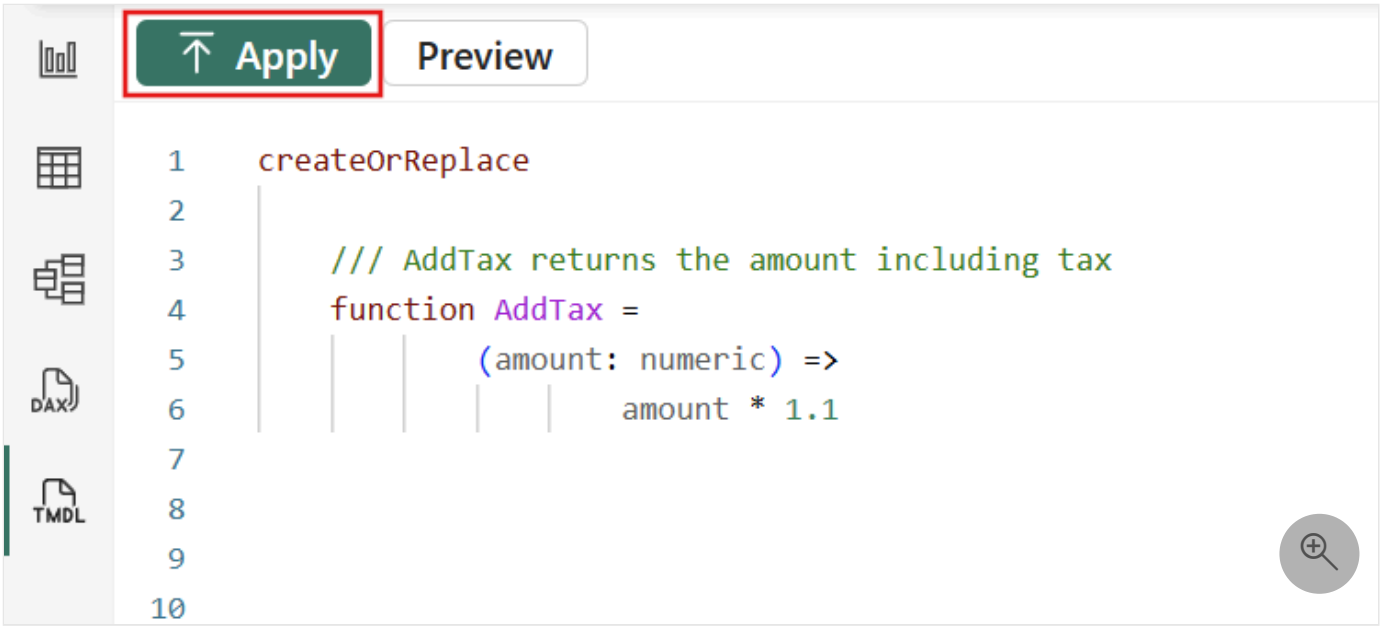
tmdl

```
createOrReplace
    /// AddTax takes in amount and returns amount including tax
```

```
function AddTax =
    (amount : NUMERIC) =>
        amount * 1.1
```

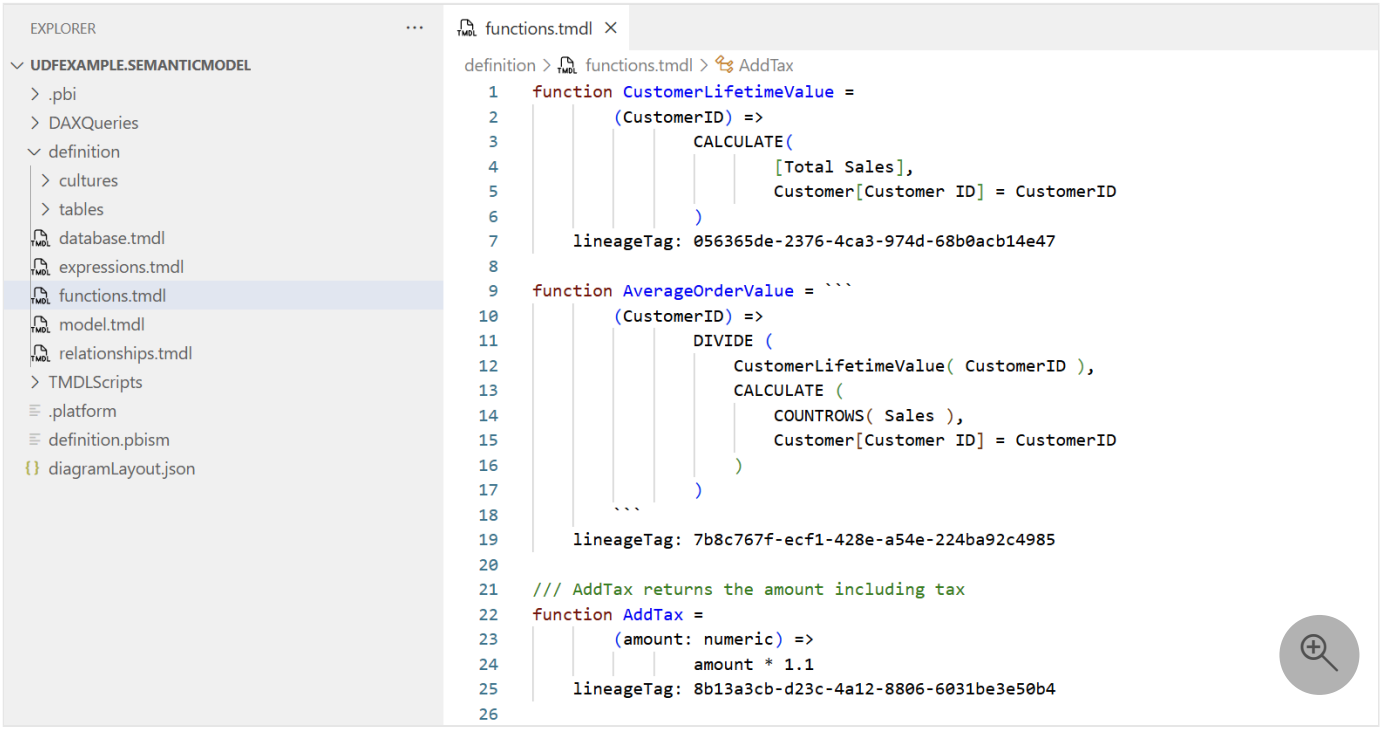
Saving to the model

Click the **Apply** button at the top of the view to save all UDFs in the script to the model.



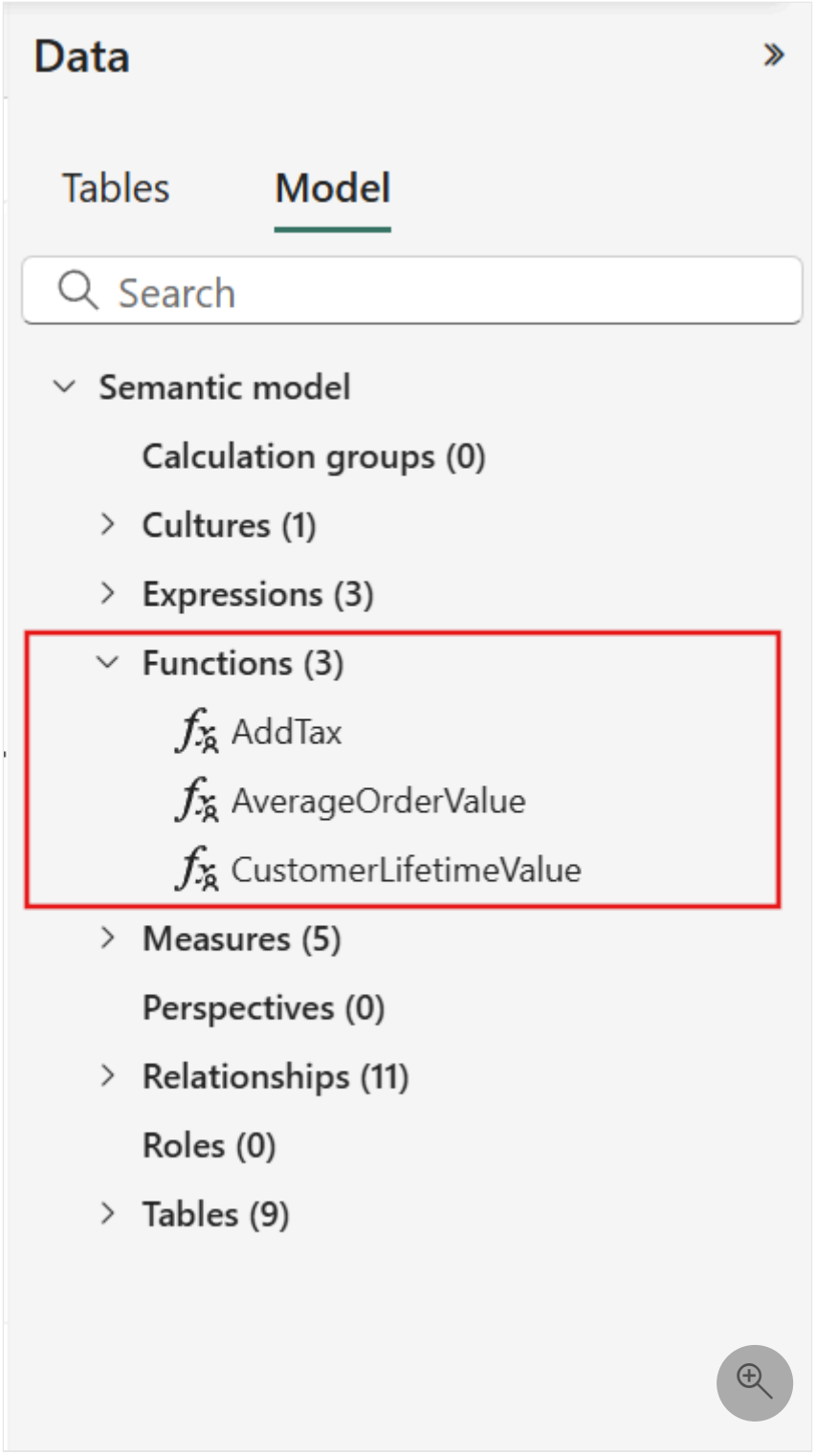
Using TMDL script in a Power BI Project

UDFs are also included in the semantic model TMDL script when using a Power BI project. They can be found in `functions.tmdl` within the *definition* folder.

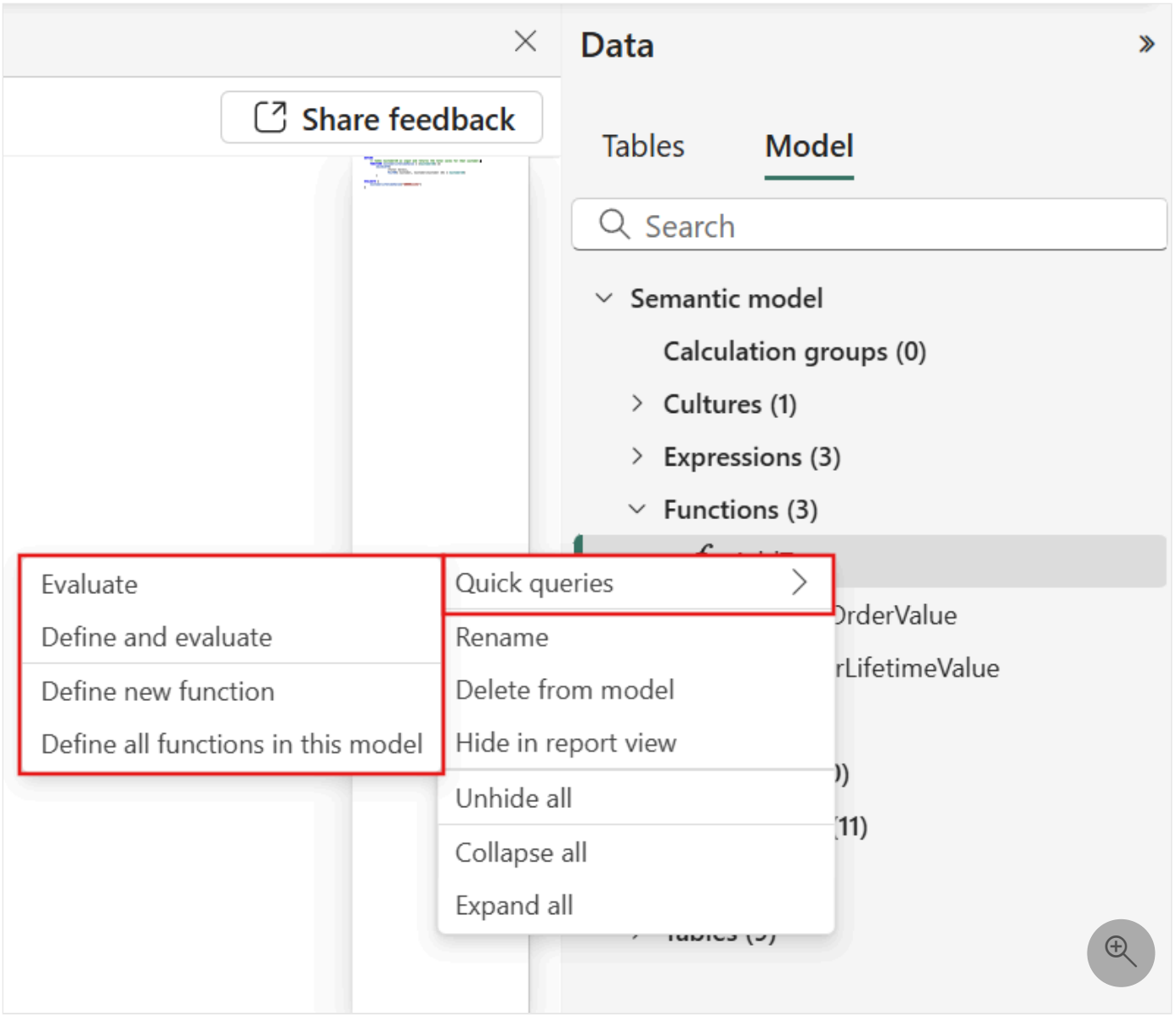


Using Model explorer

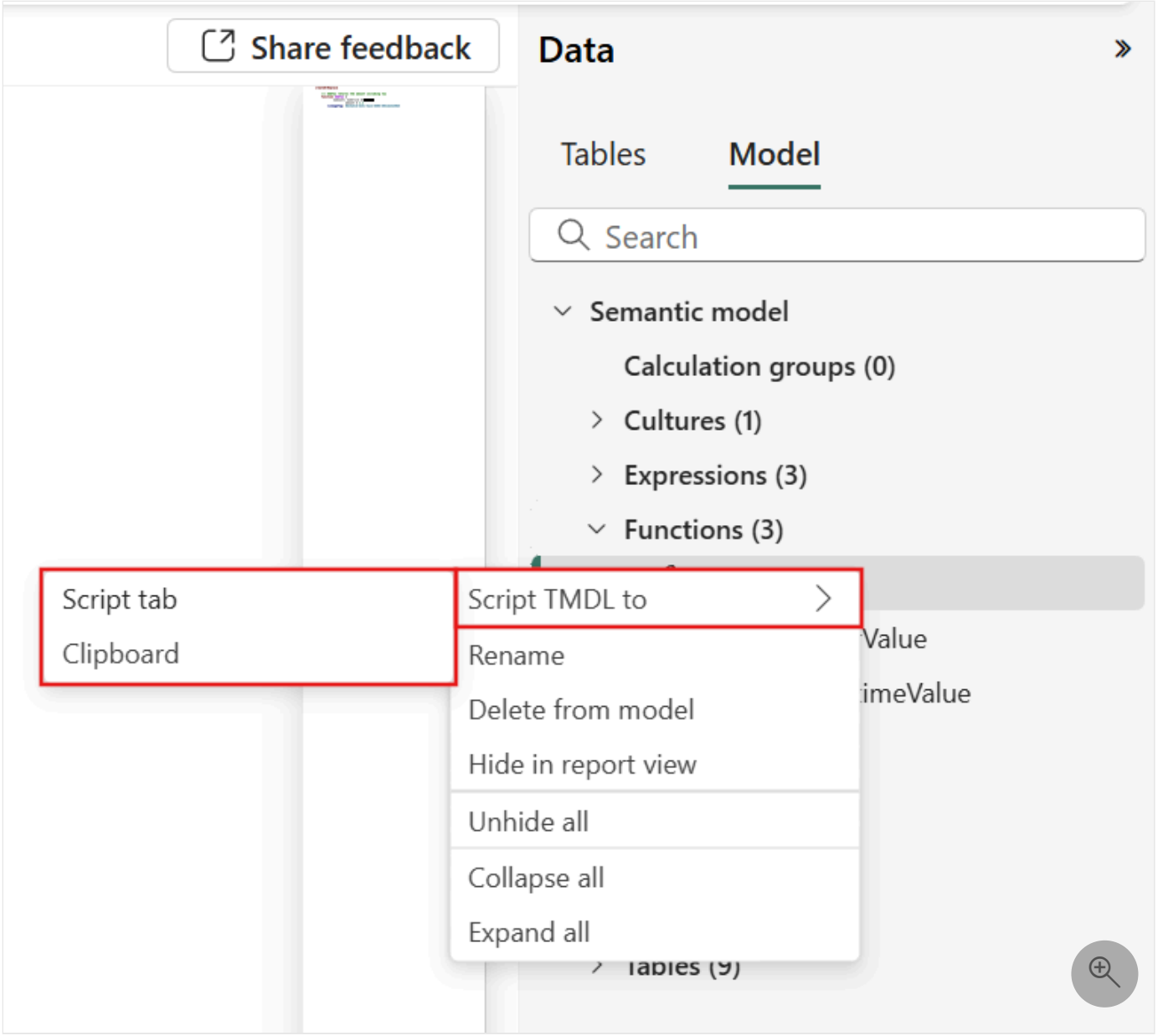
You can view all user-defined functions in the model from Model explorer under the *Functions* node. For additional information on Model explorer, see Model explorer.



In DAX query view, you can use **Quick queries** in the right-click menu of a UDF within Model explorer to easily define and evaluate functions.



In TMDL view, you can **drag and drop** functions into the canvas or use **Script TMDL to** in the right-click menu of a UDF within Model explorer to generate scripts.



Using DMVs to inspect UDFs

You can inspect UDFs in your model using Dynamic Management Views (DMVs). These views allow you to query information about functions, including UDFs.

You can use the INFO.USERDEFINEDFUNCTIONS function to inspect the UDFs in the model. This function returns full metadata and requires write permission.

DAX

```
EVALUATE INFO.USERDEFINEDFUNCTIONS()
```

Alternatively, you can use the INFO.FUNCTIONS function to return UDF names and limited metadata.

DAX

```
EVALUATE INFO.FUNCTIONS("ORIGIN", "2")
```

Using a user-defined function

Once a UDF is defined and saved to the model, you can call it from measures, calculated columns, visual calculations, and other UDFs. This works the same as calling built-in DAX functions.

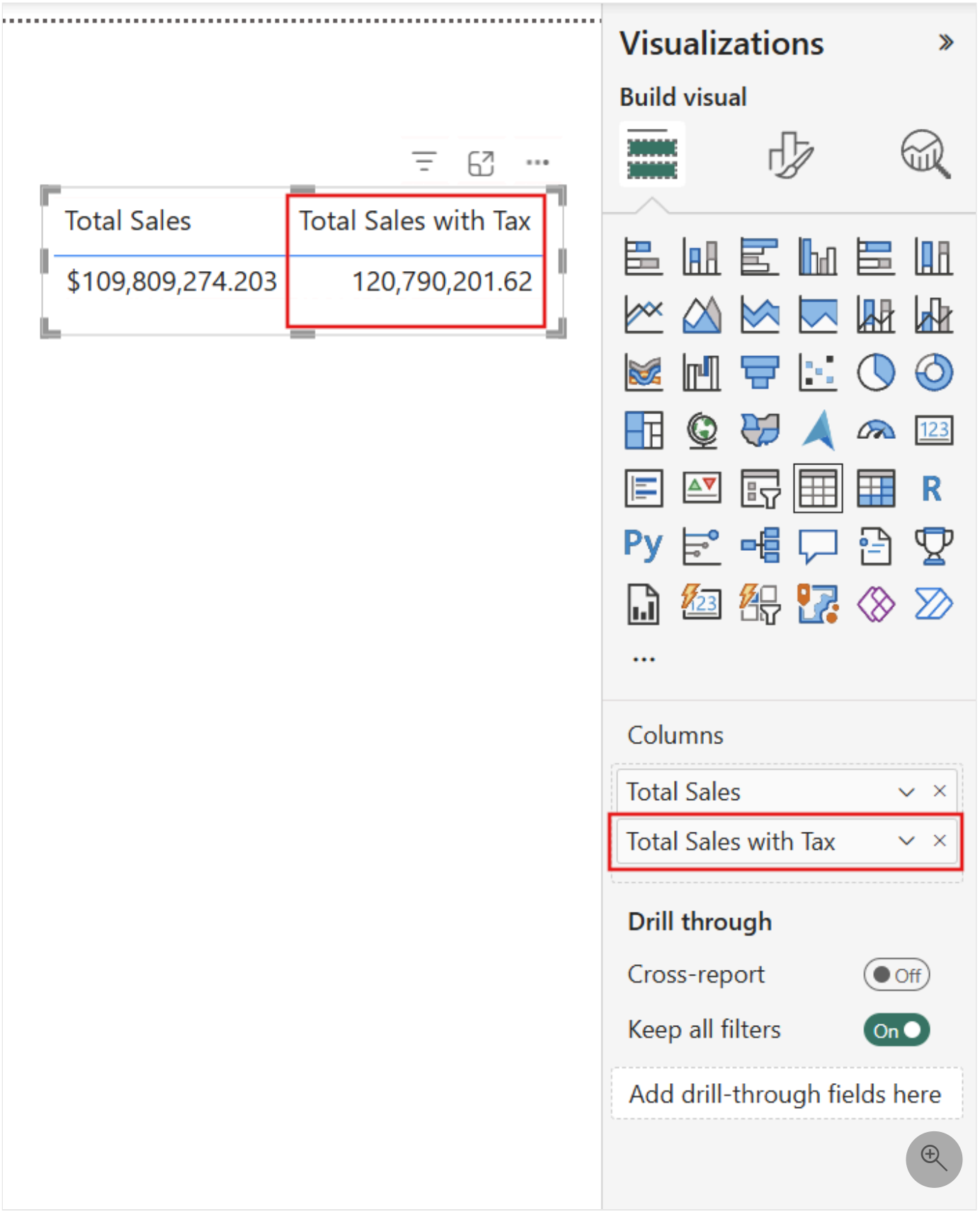
Calling a UDF in a measure

Use a UDF in a measure to apply reusable logic with full filter context.

DAX

```
Total Sales with Tax = AddTax ( [Total Sales] )
```

The example measure is shown in the table below:



Calling a UDF in a calculated column

UDFs can be used in a calculated column to apply reusable logic to every row in a table.

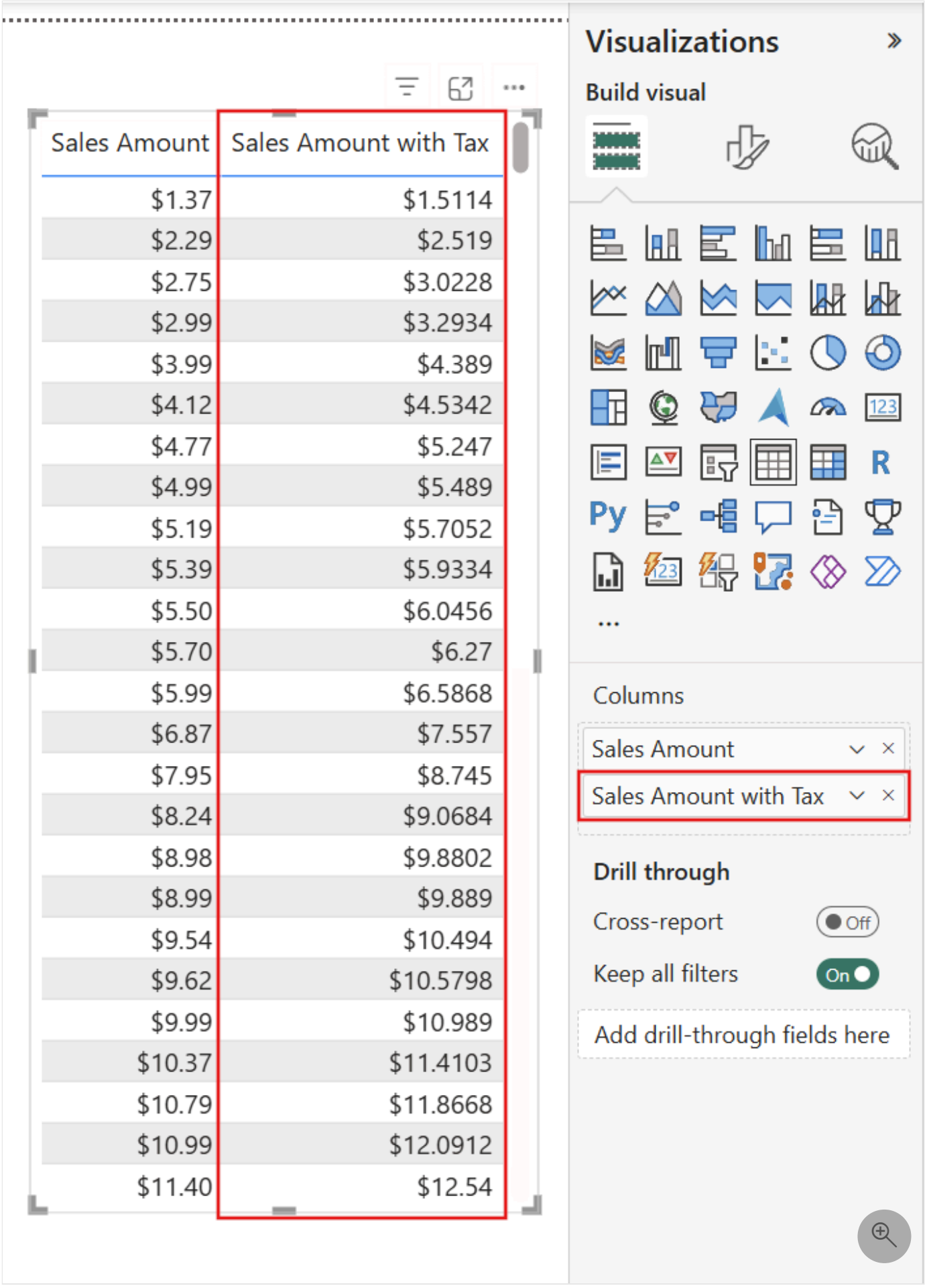
> **Note**

When using a UDF in a calculated column, ensure the function returns a scalar of a consistent type. See **Parameters** for more information. If needed, convert the result to the desired type using **CONVERT** or similar functions.

DAX

```
Sales Amount with Tax = CONVERT ( AddTax ( 'Sales'[Sales Amount] ), CURRENCY )
```

We can see this example measure used in the table below:



Calling a UDF in a visual calculation

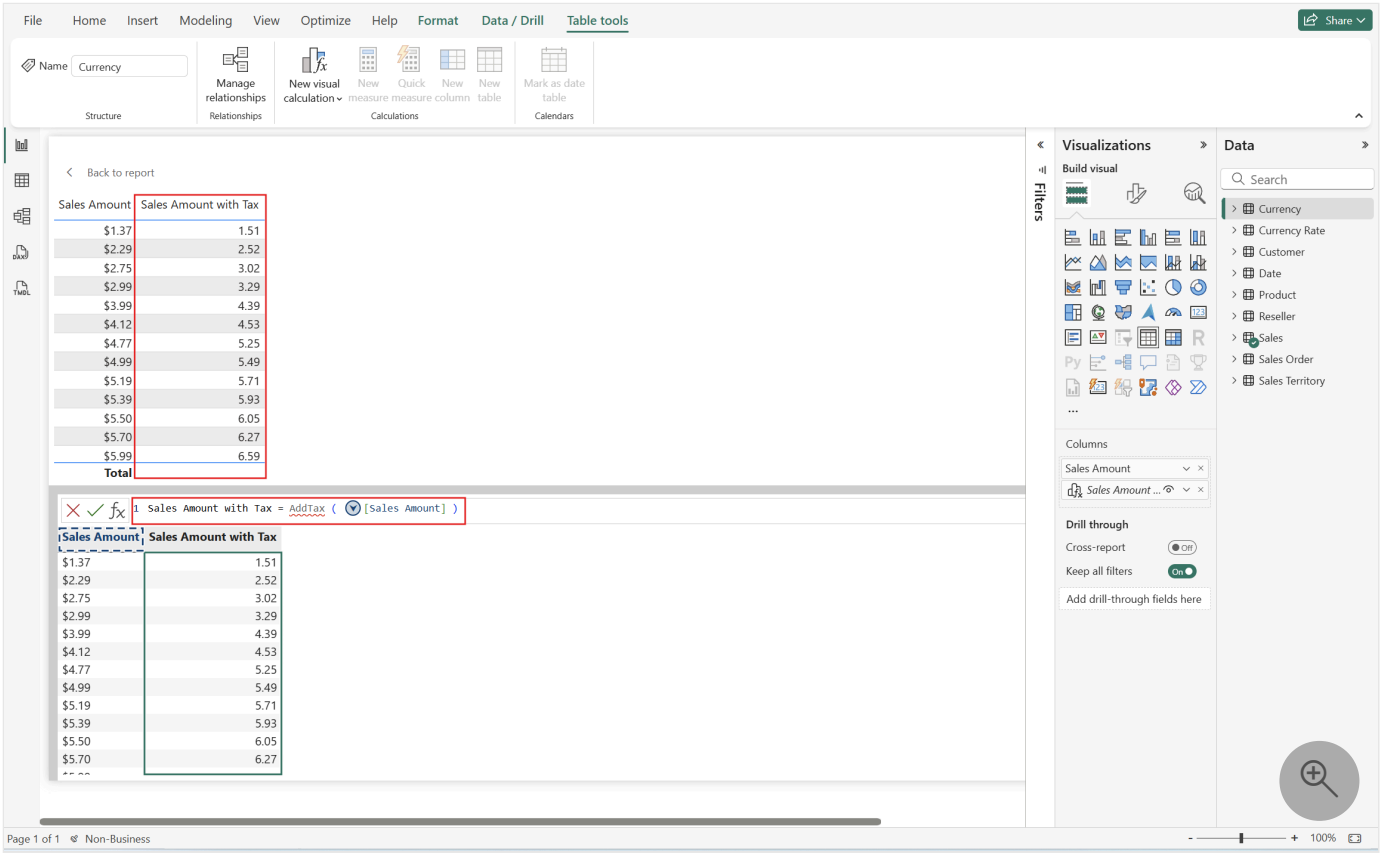
You can use UDFs in a visual calculation to apply logic directly to the visual. For additional information on visual calculations, see Visual Calculations.

> **Note**
>
> Visual calculations only operate on fields present in the visual. They cannot access model objects that aren't part of the visual, and you cannot pass model objects (such as columns or measures not in the visual) into a UDF in this context.

DAX

```
Sales Amount with Tax = AddTax ( [Sales Amount] )
```

We can see this example measure in the table below:



Calling a UDF in another UDF

You can nest UDFs by calling a function from another. In this example we define our simple `AddTax` UDF and call it in another UDF, `AddTaxAndDiscount`.

DAX

```
DEFINE
    /// AddTax takes in amount and returns amount including tax
    FUNCTION AddTax =
        ( amount : NUMERIC ) =>
            amount * 1.1

    FUNCTION AddTaxAndDiscount =
        (
            amount : NUMERIC,
            discount : NUMERIC
        ) =>
            AddTax ( amount - discount )

EVALUATE
{ AddTaxAndDiscount ( 10, 2 ) }
// Returns 8.8
```

Parameters

DAX UDFs support between 0 and 256 parameters. When you define parameters for a UDF, you can optionally specify type hints for each parameter:

- **Type**: what type of value the parameter accepts (`AnyVal`, `Scalar`, `Table`, `AnyRef`, `CalendarRef`, `ColumnRef`, `MeasureRef` or `TableRef`).
- **Subtype** (only for scalar type): the specific scalar data type (`Variant`, `Int64`, `Decimal`, `Double`, `String`, `DateTime`, `Boolean`, or `Numeric`).
- **ParameterMode**: when the argument is evaluated (`val` or `expr`).

Type hints are in the form: `[type] [subtype] [parameterMode]`

You can include all, some, or none of these type hints for each parameter to make your functions safer and more predictable at call sites. If you omit everything and just write the parameter name it behaves as `AnyVal val`, meaning the argument is evaluated immediately at call time. This is useful for simple functions.

You can also make the parameter an optional parameter by including a default expression using `= <DefaultExpression>`. This means the caller can choose to omit the argument, and if they do, the default expression will be used as the argument value.

Type

Type defines the category of argument your parameter accepts and whether it is passed as a **value** or an **expression**.

There are two type families in DAX UDF parameters: **value types** and **expression types**:

- **Value types**: This argument is **evaluated immediately** (eager evaluation) when the function is called and the resulting value is passed into the function.
  - `AnyVal`: Accepts a scalar or a table. This is the default if you omit type for a parameter.
  - `Scalar`: Accepts a scalar value (can additionally add a subtype).
  - `Table`: Accepts a table.
- **Expression types**: This argument passes an **unevaluated expression** (lazy evaluation). The function decides when and in what context to evaluate it. This is required for reference parameters and useful when you need to control filter context (e.g. inside CALCULATE). `expr` types can be references to a column, table, calendar, or measure.
  - `AnyRef`: Accepts any reference. It's the equivalent to not specifying an expression type.
  - `CalendarRef`: Accepts a reference to a calendar.
  - `ColumnRef`: Accepts a reference to a column.
  - `MeasureRef`: Accepts a reference to a measure.
  - `TableRef`: Accepts a reference to a table.

Value types (`AnyVal`, `Scalar`, `Table`) support implicit type casting. Expression types (`AnyRef`, `CalendarRef`, `ColumnRef`, `MeasureRef`, `TableRef`) do not.

Subtype

Subtype lets you define a specific `Scalar` data type. If you define a subtype, you do not need to explicitly define the parameter as a `Scalar` type, this is automatically assumed.

Subtypes are:

- `Variant`: Accepts any scalar.
- `Int64`: Accepts a whole number.
- `Decimal`: Accepts a fixed-precision decimal (such as Currency or Money).
- `Double`: Accepts a floating-point decimal.
- `String`: Accepts text.
- `DateTime`: Accepts date/time.
- `Boolean`: Accepts TRUE/FALSE.
- `Numeric`: Accepts any numerical value (`Int64`, `Decimal`, or `Double` subtypes)

ParameterMode

ParameterMode (also called **parameter passing mode** or simply **parameter mode**) controls when and where the parameter expression is evaluated. These are:

- `val` **(eager evaluation)**: The expression is evaluated once before invoking the function. The resulting value is then passed into the function. This is common for simple scalar or table inputs. This is the default if you omit parameterMode for a parameter.
- `expr` **(lazy evaluation)**: The expression is evaluated inside the function, potentially in a different context (e.g. row context or filter context) and possibly multiple times if referenced multiple times or inside iterations. This is required for reference parameters and useful when you need to control evaluation context.

The `Scalar` type can use either `val` or `expr`. Use `val` when you want the scalar evaluated once in the caller's context. Use `expr` when you want to defer evaluation and possibly apply context inside the function. See Example: Table parameter as an example.

The expression types (`AnyRef`, `ColumnRef`, etc) type must be `expr` as its references (columns, tables, measures, etc.) need to be evaluated in the function's context.

The following table summarizes the effective/allowed parameterMode:

Expand table

Type	ParameterMode not specified	ParameterMode: `val`	ParameterMode: `expr`
(Not Specified) / `AnyVal`	`val`	`val`	`expr`
`Scalar`, `Table`	`val`	`val`	`expr`
`AnyRef`	`expr`	Not allowed	`expr`
`CalendarRef`, `ColumnRef`, `MeasureRef`, `TableRef`	`expr`	Not allowed	`expr`

An `expr` parameter is evaluated lazily within the function body, so it inherits only the filter context. In contrast, a `val` parameter is evaluated eagerly before entering the function body, and therefore inherits both the row context and the filter context of the caller.

Default expression

When you assign a default expression to a parameter using `= <DefaultExpression>`, that parameter becomes optional. The caller can omit the argument, and the default expression is used as the argument value. The following behaviors apply:

- **Position**: Optional parameters can appear in any position. Required parameters can come after optional ones because callers can leave an argument empty to use the default

expression (e.g., `MyFunc(1,,3)` omits the second argument). However, the minimum number of arguments (arity) of the function is determined by the position of the rightmost required parameter. For example, if a function has three parameters and only the second is optional, callers must still supply at least three arguments.

- **Context**: Whether the filter context and row context are inherited is determined by the parameter mode of the parameter associated with the default expression.
- **Scope**: A default expression can only refer to names (columns, tables, measures, variables, functions, etc.) that are visible at the point where the UDF is defined, not where it is called. It cannot refer to another optional parameter in the UDF.
- **Type**: Type checking against the parameter's type hint is only enforced when the default expression is used. If a caller provides an explicit argument, the type hint is applied against that argument instead.

Example: Type casting

DAX

```
DEFINE
    /// returns x cast to an Int64
    FUNCTION CastToInt = (
            x : SCALAR INT64 VAL
        ) =>
        x

EVALUATE
{ CastToInt ( 3.4 ), CastToInt ( 3.5 ), CastToInt ( "5" ) }
// returns 3, 4, 5
```

This uses a `Scalar` type, `Int64` subtype, and `val` parameterMode for predictable rounding and text-to-number coercion, as well as ensuring any expressions are eagerly evaluated. You can also achieve this by just including the `Int64` subtype as seen in the example below. Non-numeric strings will result in an error.

DAX

```
DEFINE
    /// returns x as an Int64
    FUNCTION CastToInt = (
            x : INT64
        ) =>
        x

EVALUATE
```

```
{ CastToInt ( 3.4 ), CastToInt ( 3.5 ), CastToInt ( "5" ) }
// returns 3, 4, 5
```

Example: Table parameter (value vs expression)

To illustrate how UDF parameterMode affects filter context consider two functions that both count rows in the 'Sales' table. Both use `CALCULATETABLE(t, ALL('Date'))` in their bodies, but one parameter is declared as a `val` (eager evaluation) and the other as `expr` (lazy evaluation):

DAX

```
DEFINE
    /// Table val: receives a materialized table, context can't be changed
    FUNCTION CountRowsNow = (
            t : TABLE VAL
        ) =>
        COUNTROWS ( CALCULATETABLE ( t, ALL ( 'Date' ) ) )

    /// Table expr: receives an unevaluated expression, context CAN be changed
    FUNCTION CountRowsLater = (
            t : TABLE EXPR
        ) =>
        COUNTROWS ( CALCULATETABLE ( t, ALL ( 'Date' ) ) )

EVALUATE
{
    CALCULATE ( CountRowsNow ( 'Sales' ), 'Date'[Fiscal Year] = "FY2020" ),
    CALCULATE ( CountRowsLater ( 'Sales' ), 'Date'[Fiscal Year] = "FY2020" )
}
// returns 84285, 121253
```

CountRowsNow returns the count of sales for FY2020 only. The 'Sales' table is already filtered by the year before entering the function, so `ALL('Date')` inside the function has no effect.

CountRowsLater returns the count of sales for all years. The function receives an unevaluated table expression and evaluates it under `ALL('Date')`, removing the external year filter.

Type checking

Type checking in UDFs can be done with new and existing type checking functions you can call inside your function body to confirm the runtime type of passed parameters. This allows UDFs to use context control, validate parameters up front, normalize inputs before calculation.

> ⓘ Note

For `expr` parameterMode parameters, type checks occur when the parameter is referenced in the function body (not at function call time).

Available type checking functions

UDFs can use the following functions to test scalar values. Each return `TRUE`/`FALSE` depending on whether the value provided is of that type.

Expand table

Category	Functions
Numeric	ISNUMERIC, ISNUMBER
Double	ISDOUBLE
Whole number	ISINT64, ISINTEGER
Decimal	ISDECIMAL, ISCURRENCY
String	ISSTRING, ISTEXT
Boolean	ISBOOLEAN, ISLOGICAL
Date and Time	ISDATETIME

Example: Check if parameter is a string

DAX

```
DEFINE
    /// Returns the length of a string, or BLANK if not a string
    FUNCTION StringLength = (
            s
        ) =>
        IF ( ISSTRING ( s ), LEN ( s ), BLANK () )

EVALUATE
{ StringLength ( "hello" ), StringLength ( 123 ) }
// Returns: 5, BLANK
```

This prevents errors and allows you to decide how to handle non-string input into the function (in this example, returns `BLANK`).

Example: Accept multiple parameter types

DAX

```
DEFINE
    /// Helper 1: get currency name by int64 key
    FUNCTION GetCurrencyNameByKey = (
            k : INT64
        ) =>
        LOOKUPVALUE ( 'Currency'[Currency], 'Currency'[CurrencyKey], k )

    /// Helper 2: get currency name by string code
    FUNCTION GetCurrencyNameByCode = (
            code : STRING
        ) =>
        LOOKUPVALUE ( 'Currency'[Currency], 'Currency'[Code], code )

    /// Accepts key (int64) or code (string) and returns the currency name
    FUNCTION GetCurrencyName = (
            currency
        ) =>
        IF (
            ISINT64 ( currency ),
            GetCurrencyNameByKey ( currency ),
            GetCurrencyNameByCode ( currency )
        )

EVALUATE
{ GetCurrencyName ( 36 ), GetCurrencyName ( "USD" ) }
// returns "Euro", "US Dollar"
```

This example shows how to use type checking in UDFs to safely accept multiple input types and return a single, predictable result. `GetCurrencyName` takes one argument, `currency`, which can be either a whole-number currency key or a text currency code. The function checks the argument type with `ISINT64`. If the input is an integer, it calls the helper `GetCurrencyNameByKey` that looks up the currency name based on the currency key. If the input is not an integer, it calls the helper `GetCurrencyNameByCode` that looks up the currency name based on the currency code.

Useful information functions

The following information functions are useful when authoring UDFs:

- TABLEOF: Returns the full table associated with a given column, measure, or calendar.
- NAMEOF: Returns the name of a table, column, measure, or calendar as a text string

Example: A MODEX function

The following example shows a basic implementation of a MODEX function that returns the most frequently occurring value(s) of an expression evaluated over a table. It uses `TABLEOF` to

automatically resolve the correct table from the given reference.

DAX

```
DEFINE
    FUNCTION MODEX = (
            e : ANYREF
        ) =>
        VAR newTable =
            ADDCOLUMNS ( TABLEOF ( e ), "expr", e )
        VAR freqTable =
            GROUPBY ( newTable, [expr], "count", SUMX ( CURRENTGROUP (), 1 ) )
        VAR maxCount =
            MAXX ( freqTable, [count] )
        VAR topResults =
            FILTER ( freqTable, [count] = maxCount )
        RETURN
            SELECTCOLUMNS ( topResults, "expr", [expr] )

EVALUATE
MODEX ( [Total Sales] )
```

Define multiple functions at once

UDFs allow you to define several functions in a single query or script, making it easy to organize reusable logic. This is especially useful when you want to encapsulate related calculations or helper routines together. Functions can be evaluated together or individually.

DAX

```
DEFINE
    /// Multiplies two numbers
    FUNCTION Multiply = (
            a,
            b
        ) =>
        a * b

    /// Adds two numbers and 1
    FUNCTION AddOne = (
            x,
            y
        ) =>
        x + y + 1

    /// Returns a random integer between 10 and 100
    FUNCTION RandomInt = () =>
        RANDBETWEEN ( 10, 100 )

EVALUATE
```

```
{ Multiply ( 3, 5 ), AddOne ( 1, 2 ), RandomInt () }
// returns 15, 4, 98
```

Advanced example: Flexible currency conversion

To show how DAX UDFs can handle more complex logic we will look at a currency conversion scenario. This example uses type checking and nested functions to convert a given amount into a target currency using either the average or end-of-day exchange rate for a given date.

tmdl

```
createOrReplace
    function ConvertDateToDateKey =
        (
            pDate: scalar variant
        ) =>
        YEAR ( pDate ) * 10000 + MONTH ( pDate ) * 100 + DAY ( pDate )

    function ConvertToCurrency =
        (
            pCurrency:scalar variant,
            pDate: scalar variant,
            pUseAverageRate: scalar boolean,
            pAmount: scalar decimal
        ) =>
        var CurrencyKey =
            EVALUATEANDLOG (
                IF (
                    ISINT64 ( pCurrency ),
                    pCurrency,
                    CALCULATE (
                        MAX ( 'Currency'[CurrencyKey] ),
                        'Currency'[Code] == pCurrency
                    )
                )
                , "CurrencyKey"
            )

        var DateKey =
            EVALUATEANDLOG (
                SWITCH (
                    TRUE,
                    ISINT64 ( pDate ), pDate,
                    ConvertDateToDateKey ( pDate )
                )
                , "DateKey"
            )

        var ExchangeRate =
            EVALUATEANDLOG (
                IF (
```

```
                    pUseAverageRate,
                    CALCULATE (
                        MAX ( 'Currency Rate'[Average Rate] ),
                        'Currency Rate'[DateKey] == DateKey,
                        'Currency Rate'[CurrencyKey] == CurrencyKey
                    ),
                    CALCULATE (
                    MAX ( 'Currency Rate'[End Of Day Rate] ),
                        'Currency Rate'[DateKey] == DateKey,
                        'Currency Rate'[CurrencyKey] == CurrencyKey
                    )
                )
                , "ExchangeRate"
            )

        var Result =
            IF (
                ISBLANK ( pCurrency ) || ISBLANK ( pDate ) || ISBLANK ( pAmount ),
                BLANK (),
                IF (
                    ISBLANK ( ExchangeRate ) ,
                    "no exchange rate available",
                    ExchangeRate * pAmount
                )
            )

        RETURN Result
```

The `ConvertToCurrency` function accepts flexible input types for both currency and date. Users can provide either a currency key or date key directly or supply a currency code or standard date value. The function checks the type of each input and handles it accordingly: if `pCurrency` is a whole number, it is treated as a currency key; otherwise, the function assumes a currency code and attempts to resolve the corresponding key. `pDate` follows a similar pattern, if it is a whole number, it is treated as a date key; otherwise the function assumes it is a standard date value and is converted to a date key using the `ConvertDateToDateKey` helper function. If the function cannot determine a valid exchnage rate, it returns the message "no exchange rate available".

This logic can then be used to define a measure such as **Total Sales in Local Currency**.

DAX

```
Total Sales in Local Currency =
ConvertToCurrency (
    SELECTEDVALUE ( 'Currency'[Code] ),
    SELECTEDVALUE ( 'Date'[DateKey] ),
    TRUE,
    [Total Sales]
)
```

This can be optionally paired with a dynamic format string to display the result in the appropriate currency format.

DAX

```
CALCULATE (
    MAX ( 'Currency'[Format String] ),
    'Currency'[Code] == SELECTEDVALUE ( 'Currency'[Code] )
)
```

An example result can be seen in the screenshot below.

Full Date	Currency	Total Sales in Local Currency	Total Sales
2017 Aug, 01	Australian Dollar	A$2,248.65	$3,578.27
2017 Aug, 01	Canadian Dollar	C$17,621.26	$25,656.551
2017 Aug, 01	United Kingdom Pound	£5,426.09	$3,399.99
2017 Aug, 01	US Dollar	US$55,655.03	$55,655.0295
2017 Aug, 02	Australian Dollar	A$6,647.01	$10,531.53
2017 Aug, 02	Canadian Dollar	C$2,321.50	$3,374.99
2017 Aug, 02	Deutsche Mark	no exchange rate available	$3,578.27
2017 Aug, 02	French Franc	no exchange rate available	$3,374.99
2017 Aug, 02	US Dollar	US$18,269.95	$18,269.946
2017 Aug, 03	Australian Dollar	A$2,260.86	$3,578.27
2017 Aug, 03	Canadian Dollar	C$9,484.54	$13,762.0652
2017 Aug, 03	United Kingdom Pound	£5,752.85	$3,578.27
2017 Aug, 03	US Dollar	US$24,122.97	$24,122.9715
2017 Aug, 04	Australian Dollar	A$9,000.24	$14,313.08
2017 Aug, 04	Canadian Dollar	C$22,018.87	$31,958.186
2017 Aug, 04	Deutsche Mark	no exchange rate available	$3,578.27
2017 Aug, 04	French Franc	no exchange rate available	$3,578.27
2017 Aug, 04	United Kingdom Pound	£11,463.30	$7,156.54
2017 Aug, 04	US Dollar	US$10,362.04	$10,362.0379
2017 Aug, 05	Australian Dollar	A$4,939.72	$7,855.6382
2017 Aug, 05	Canadian Dollar	C$8,794.30	$12,742.9396
2017 Aug, 05	US Dollar	US$3,578.27	$3,578.27
2017 Aug, 06	Australian Dollar	A$2,247.23	$3,578.27
Total			**$109,809,274.203**

Considerations and limitations

User-defined functions have the following limitations:

General:

- Cannot author or model DAX UDFs in Service.
- Cannot hide/unhide a UDF in the model.
- Cannot put UDFs in display folders.
- No 'create function' button in the ribbon.
- Cannot combine UDFs with translations.
- UDFs are not supported in models without tables.
- Object-Level Security (OLS) does not transfer to functions or vice versa. For example, consider the following function `F` that refers to secured measure `MyMeasure`:

  DAX

  ```
  function F = () => [MyMeasure] + 42
  ```

  when the `MyMeasure` is secured using object-level security, function F is not secured automatically. If `F` runs under an identity without access to `MyMeasure`, it acts as if `MyMeasure` doesn't exist. We recommend to avoid revealing secure objects in function names and descriptions.

- Formula fix-up and dependency calculation are supported, with a known limitation around unqualified names. Because unqualified names are interpreted as measure references, they cannot be reliably fixed or included in dependency tracking when they are intended to refer to columns. An unqualified name is an object reference that does not include a table prefix.

Defining a UDF:

- Recursion or mutual recursion is not supported.
- Function overloading is not supported.
- Explicit return type is not supported.

UDF parameters:

- Parameter descriptions are not supported.

- UDFs cannot return an `enum` value. Built-in functions that accept `enum` values as their function parameters will not be able to use UDFs in that context.
- Unbound parameters of type hint `expr` are not evaluated.

IntelliSense Support:

- Although report-based measures in a live connect report can refer to UDFs declared in the source model, there is no IntelliSense support. Model-based measures in a composite model cannot refer to UDFs declared in the source model.
- Although UDFs can be used in visual calculations, the visual calculations formula bar does not have IntelliSense support for UDFs.
- TMDL View has limited IntelliSense support for UDFs.
- SQL Server Management Studio does not have IntelliSense support for UDFs.

Known bugs

The following issues are currently known and may impact functionality:

- Certain advanced scenarios involving UDFs can result in parser inconsistencies. For example, users may see red underlines or validation errors when passing columns as `expr` parameters or using unqualified column references.
- `CalendarRef`, `ColumnRef`, `MeasureRef` and `TableRef` type hints may not be accepted at all function call sites. Users can fallback to `AnyRef`.

Related content

- DAX query view
- TMDL view
- Model explorer
- INFO.USERDEFINEDFUNCTIONS

Last updated on 06/02/2026

DAX function reference

Article • 03/17/2025

The DAX function reference provides detailed information including syntax, parameters, return values, and examples for each of the over 250 functions used in Data Analysis Expression (DAX) formulas.

> **Important**
>
> Not all DAX functions are supported or included in earlier versions of Power BI Desktop, Analysis Services, and Power Pivot in Excel.

In this section

New DAX functions - These functions are new or are existing functions that have been significantly updated.

Aggregation functions - These functions calculate a (scalar) value such as count, sum, average, minimum, or maximum for all rows in a column or table as defined by the expression.

Date and time functions - These functions in DAX are similar to date and time functions in Microsoft Excel. However, DAX functions are based on the datetime data types used by Microsoft SQL Server.

Filter functions - These functions help you return specific data types, look up values in related tables, and filter by related values. Lookup functions work by using tables and relationships between them. Filtering functions let you manipulate data context to create dynamic calculations.

Financial functions - These functions are used in formulas that perform financial calculations, such as net present value and rate of return.

INFO functions - These functions return metadata about your semantic model, such as the tables, columns, relationships, and calculation DAX formulas. They can help you understand and document the model. They are based on the library of Dynamic Management Views (DMVs) in Analysis Services, which have been modified to work as DAX functions.

Information functions - These functions look at a table or column provided as an argument to another function and returns whether the value matches the expected type.

For example, the ISERROR function returns `TRUE` if the value you reference contains an error.

Logical functions - These functions return information about values in an expression. For example, the `TRUE` function lets you know whether an expression that you are evaluating returns a `TRUE` value.

Math and Trig functions - Mathematical functions in DAX are similar to Excel's mathematical and trigonometric functions. However, there are some differences in the numeric data types used by DAX functions.

Other functions - These functions perform unique actions that cannot be defined by any of the categories most other functions belong to.

Parent and Child functions - These functions help users manage data that is presented as a parent/child hierarchy in their data models.

Relationship functions - These functions are for managing and utilizing relationships between tables. For example, you can specify a particular relationship to be used in a calculation.

Statistical functions - These functions calculate values related to statistical distributions and probability, such as standard deviation and number of permutations.

Table manipulation functions - These functions return a table or manipulate existing tables.

Text functions - With these functions, you can return part of a string, search for text within a string, or concatenate string values. Additional functions are for controlling the formats for dates, times, and numbers.

Time intelligence functions - These functions help you create calculations that use built-in knowledge about calendars and dates. By using time and date ranges in combination with aggregations or calculations, you can build meaningful comparisons across comparable time periods for sales, inventory, and so on.

Related content

DAX Syntax Reference DAX Operator Reference DAX Parameter-Naming Conventions

---

Feedback

Was this page helpful? Yes No

New DAX functions

DAX is continuously being improved with new functions and functionality to support new features. New functions and updates are included in service, application, and tool updates which in most cases are monthly.

While functions and functionality are being updated all the time, only those updates that have a visible and functional change exposed to users are described in documentation. New functions and updates to existing functions within the past year are shown here.

> **Important**
>
> Not all functions are supported in all versions of Power BI Desktop, Analysis Services, and Power Pivot in Excel. New and updated functions are typically first introduced in Power BI Desktop, and then later in Analysis Services, Power Pivot in Excel, and tools.

New functions

Expand table

Function	Month	Description
TEXTCONTAINS	September, 2026	Returns `TRUE` if text in a column matches a search string according to full-text search semantics.
TEXTSIMILARITY	September, 2026	Returns a relevance score that indicates how strongly text in a column matches a search string.
TABLEOF	February, 2026	Returns a reference to the table associated with a specified column, measure, or calendar.
TOTALWTD	September, 2025	Calculates the running total of a measure to the current week in the filter context.
CLOSINGBALANCEWEEK	September, 2025	Returns the closing balance for the week in the current context.
ENDOFWEEK	September, 2025	Returns the last date of the current week in the calendar.
NEXTWEEK	September, 2025	Returns a table that contains a column of dates for the next week.

Function	Month	Description
OPENINGBALANCEWEEK	September, 2025	Returns the opening balance for the week in the current context.
PREVIOUSWEEK	September, 2025	Returns a table that contains a column of dates for the previous week.
STARTOFWEEK	September, 2025	Returns the first date of the current week in the calendar.
LOOKUPWITHTOTALS	June, 2025	Used in visual calculations only. Look up the value when filters are applied. Filters not specified would not be inferred.
LOOKUP	June, 2025	Used in visual calculations only. Look up the value when filters are applied.
FIRST	January, 2024	Used in visual calculations only. Retrieves a value in the visual matrix from the first row of an axis.
LAST	January, 2024	Used in visual calculations only. Retrieves a value in the visual matrix from the last row of an axis.
NEXT	January, 2024	Used in visual calculations only. Retrieves a value in the next row of an axis in the visual matrix.
PREVIOUS	January, 2024	Used in visual calculations only. Retrieves a value in the previous row of an axis in the visual matrix.
MATCHBY	May, 2023	Define the columns that are used to to match data and identify the current row, in a window function expression.
RANK	April, 2023	Returns the ranking for the current context within the specified partition, sorted by the specified order.
ROWNUMBER	April, 2023	Returns the unique ranking for the current context within the specified partition, sorted by the specified order.
LINEST	February, 2023	Uses the Least Squares method to calculate a straight line that best fits the given data.
LINESTX	February, 2023	Uses the Least Squares method to calculate a straight line that best fits the given data. The data result from expressions evaluated for each row in a table.

Last updated on 09/29/2026

Aggregation functions

06/12/2025

Aggregation functions calculate a (scalar) value such as count, sum, average, minimum, or maximum for all rows in a column or table as defined by the expression.

In this category

Expand table

Function	Description
APPROXIMATEDISTINCTCOUNT	Returns an *estimated* count of unique values in a column.
AVERAGE	Returns the average (arithmetic mean) of all the numbers in a column.
AVERAGEA	Returns the average (arithmetic mean) of the values in a column.
AVERAGEX	Calculates the average (arithmetic mean) of a set of expressions evaluated over a table.
COUNT	Counts the number of rows in the specified column that contain non-blank values. Does not support Boolean values.
COUNTA	Counts the number of rows in the specified column that contain non-blank values. Supports Boolean values.
COUNTAX	Counts non-blank results when evaluating the result of an expression over a table.
COUNTBLANK	Counts the number of blank cells in a column.
COUNTROWS	Counts the number of rows in the specified table, or in a table defined by an expression.
COUNTX	Counts the number of rows that contain a number or an expression that evaluates to a number, when evaluating an expression over a table.
DISTINCTCOUNT	Counts the number of distinct values in a column.
DISTINCTCOUNTNOBLANK	Counts the number of distinct values in a column.
MAX	Returns the largest numeric value in a column, or between two scalar expressions.
MAXA	Returns the largest value in a column.
MAXX	Evaluates an expression for each row of a table and returns the largest

Function	Description
	numeric value.
MIN	Returns the smallest numeric value in a column, or between two scalar expressions.
MINA	Returns the smallest value in a column, including any logical values and numbers represented as text.
MINX	Returns the smallest numeric value that results from evaluating an expression for each row of a table.
PRODUCT	Returns the product of the numbers in a column.
PRODUCTX	Returns the product of an expression evaluated for each row in a table.
SUM	Adds all the numbers in a column.
SUMX	Returns the sum of an expression evaluated for each row in a table.

APPROXIMATEDISTINCTCOUNT

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns an *estimated* count of unique values in a column.

> **Important**
>
> Support for `APPROXIMATEDISTINCTCOUNT` in Import and Direct Lake storage modes is currently in preview.

Syntax

```
APPROXIMATEDISTINCTCOUNT(<columnName>)
```

Parameters

Expand table

Term	Description
`column`	Input column.

The `column` argument can't be an expression.

Return value

The approximate number of distinct values in `column`.

Remarks

- The only argument to this function is a column. Use columns containing any type of data. When the function finds no rows to count, it returns a `BLANK`; otherwise, it returns the count of distinct values.
- In Import mode, Direct Lake on OneLake, and Direct Lake on SQL without fallback, the function uses an engine-side HyperLogLog algorithm.

- In DirectQuery mode, the function invokes a corresponding aggregation operation in the data source. This behavior is supported for Azure SQL, Azure Synapse dedicated SQL pool, BigQuery, Databricks, and Snowflake. Direct Lake on SQL uses this DirectQuery behavior when it falls back to DirectQuery mode.
- Use this function for high-cardinality columns when an approximate result is acceptable. Use DISTINCTCOUNT when you require an exact result.
- For low-cardinality columns, `APPROXIMATEDISTINCTCOUNT` might perform more slowly than `DISTINCTCOUNT`. It can also use more memory than `DISTINCTCOUNT`, especially with large models, high concurrency, or many group-by groups.
- The estimated count returned by `APPROXIMATEDISTINCTCOUNT` has an error rate of approximately 1.6% for Import and Direct Lake models.

Last updated on 09/15/2026

AVERAGE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the average (arithmetic mean) of all the numbers in a column.

Syntax

```
DAX
AVERAGE(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the numbers for which you want the average.

Return value

Returns a decimal number that represents the arithmetic mean of the numbers in the column.

Remarks

- This function takes the specified column as an argument and finds the average of the values in that column. If you want to find the average of an expression that evaluates to a set of numbers, use the AVERAGEX function instead.
- Nonnumeric values in the column are handled as follows:
  - If the column contains text, no aggregation can be performed, and the functions returns blanks.
  - If the column contains logical values or empty cells, those values are ignored.
  - Cells with the value zero are included.

- When you average cells, you must keep in mind the difference between an empty cell and a cell that contains the value 0 (zero). When a cell contains 0, it is added to the sum of numbers and the row is counted among the number of rows used as the divisor. However, when a cell contains a blank, the row is not counted.
- Whenever there are no rows to aggregate, the function returns a blank. However, if there are rows, but none of them meet the specified criteria, the function returns 0. Excel also returns a zero if no rows are found that meet the conditions.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula returns the average of the values in the column, ExtendedSalesAmount, in the table, InternetSales.

DAX

```
= AVERAGE(InternetSales[ExtendedSalesAmount])
```

Related functions

The AVERAGEX function can take as its argument an expression that is evaluated for each row in a table. This enables you to perform calculations and then take the average of the calculated values.

The AVERAGEA function takes a column as its argument, but otherwise is like the Excel function of the same name. By using the AVERAGEA function, you can calculate a mean on a column that contains empty values.

Feedback

Was this page helpful? Yes No

AVERAGEA

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the average (arithmetic mean) of the values in a column. Handles text and non-numeric values.

Syntax

DAX

```
AVERAGEA(<column>)
```

Parameters

Expand table

Term	Definition
column	A column that contains the values for which you want the average.

Return value

A decimal number.

Remarks

- The AVERAGEA function takes a column and averages the numbers in it, but also handles non-numeric data types according to the following rules:
  - Values that evaluates to TRUE count as 1.
  - Values that evaluate to FALSE count as 0 (zero).
  - Values that contain non-numeric text count as 0 (zero).
  - Empty text ("") counts as 0 (zero).
- If you do not want to include logical values and text representations of numbers in a reference as part of the calculation, use the AVERAGE function.

- Whenever there are no rows to aggregate, the function returns a blank. However, if there are rows, but none of them meet the specified criteria, the function returns 0. Microsoft Excel also returns a zero if no rows are found that meet the conditions.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns the average of non-blank cells in the referenced column, given the following table. If you used the AVERAGE function, the mean would be 21/2; with the AVERAGEA function, the result is 22/5.

Expand table

Transaction ID	Amount	Result
0000123	1	Counts as 1
0000124	20	Counts as 20
0000125	n/a	Counts as 0
0000126		Counts as 0
0000126	TRUE	Counts as 1

DAX

```
= AVERAGEA([Amount])
```

Related content

AVERAGE function
AVERAGEX function
Statistical functions

Feedback

Was this page helpful? Yes No

AVERAGEX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Calculates the average (arithmetic mean) of a set of expressions evaluated over a table.

Syntax

```DAX
AVERAGEX(<table>,<expression>)
```

Parameters

Expand table

Term	Definition
table	Name of a table, or an expression that specifies the table over which the aggregation can be performed.
expression	An expression with a scalar result, which will be evaluated for each row of the table in the first argument.

Return value

A decimal number.

Remarks

- The AVERAGEX function enables you to evaluate expressions for each row of a table, and then take the resulting set of values and calculate its arithmetic mean. Therefore, the function takes a table as its first argument, and an expression as the second argument.
- In all other respects, AVERAGEX follows the same rules as AVERAGE. You cannot include non-numeric or null cells. Both the table and expression arguments are required.

- When there are no rows to aggregate, the function returns a blank. When there are rows, but none of them meet the specified criteria, then the function returns 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example calculates the average freight and tax on each order in the InternetSales table, by first summing Freight plus TaxAmt in each row, and then averaging those sums.

DAX

```
= AVERAGEX(InternetSales, InternetSales[Freight]+ InternetSales[TaxAmt])
```

If you use multiple operations in the expression used as the second argument, you must use parentheses to control the order of calculations. For more information, see DAX Syntax Reference.

Related content

AVERAGE function
AVERAGEA function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of rows in the specified column that contain non-blank values.

Syntax

DAX

```
COUNT(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the values to be counted.

Return value

A whole number.

Remarks

- The only argument allowed to this function is a column. The COUNT function counts rows that contain the following kinds of values:
  - Numbers
  - Dates
  - Strings
- When the function finds no rows to count, it returns a blank.
- Blank values are skipped. TRUE/FALSE values are not supported.
- If you want to evaluate a column of TRUE/FALSE values, use the COUNTA function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- For best practices when using COUNT, see Use COUNTROWS instead of COUNT.

Example

The following example shows how to count the number of values in the column, ShipDate.

```
DAX
= COUNT([ShipDate])
```

To count logical values or text, use the COUNTA or COUNTAX functions.

Related content

COUNTA function
COUNTAX function
COUNTX function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNTA

Article • 09/06/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of rows in the specified column that contain non-blank values.

Syntax

DAX

```
COUNTA(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the values to be counted.

Return value

A whole number.

Remarks

- When the function does not find any rows to count, the function returns a blank.
- Unlike COUNT, COUNTA supports Boolean data type.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns the number of all rows in the `Reseller` table that have any kind of value in the column that stores phone numbers.

DAX

```
= COUNTA(Reseller[Phone])
```

Related content

COUNT function
COUNTAX function
COUNTX function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNTAX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

The COUNTAX function counts non-blank results when evaluating the result of an expression over a table. That is, it works just like the COUNTA function, but is used to iterate through the rows in a table and count rows where the specified expressions results in a non-blank result.

Syntax

```
DAX

COUNTAX(<table>,<expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.

Return value

A whole number.

Remarks

- Like the COUNTA function, the COUNTAX function counts cells containing any type of information, including other expressions. For example, if the column contains an expression that evaluates to an empty string, the COUNTAX function treats that result as non-blank. Usually the COUNTAX function does not count empty cells but in this case the cell contains a formula, so it is counted.
- Whenever the function finds no rows to aggregate, the function returns a blank.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example counts the number of nonblank rows in the column, Phone, using the table that results from filtering the Reseller table on [Status] = **Active**.

DAX

```
= COUNTAX(FILTER('Reseller',[Status]="Active"),[Phone])
```

Related content

COUNT function
COUNTA function
COUNTX function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNTBLANK

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of blank cells in a column.

Syntax

DAX

```
COUNTBLANK(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the blank cells to be counted.

Return value

A whole number. If no rows are found that meet the condition, blanks are returned.

Remarks

- The only argument allowed to this function is a column. You can use columns containing any type of data, but only blank cells are counted. Cells that have the value zero (0) are not counted, as zero is considered a numeric value and not a blank.
- Whenever there are no rows to aggregate, the function returns a blank. However, if there are rows, but none of them meet the specified criteria, the function returns 0. Microsoft Excel also returns a zero if no rows are found that meet the conditions.
- In other words, if the COUNTBLANK function finds no blanks, the result will be zero, but if there are no rows to check, the result will be blank.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to count the number of rows in the table Reseller that have blank values for BankName.

```
DAX

= COUNTBLANK(Reseller[BankName])
```

To count logical values or text, use the COUNTA or COUNTAX functions.

Related content

COUNT function
COUNTA function
COUNTAX function
COUNTX function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNTROWS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

The COUNTROWS function counts the number of rows in the specified table, or in a table defined by an expression.

Syntax

DAX

```
COUNTROWS([<table>])
```

Parameters

Expand table

Term	Definition
table	(Optional) The name of the table that contains the rows to be counted, or an expression that returns a table. When not provided, the default value is the home table of the current expression.

Return value

A whole number.

Remarks

- This function can be used to count the number of rows in a base table, but more often is used to count the number of rows that result from filtering a table, or applying context to a table.
- When the table argument contains no rows, the function returns BLANK.
- To learn more about best practices when using COUNT and COUNTROWS, see Use COUNTROWS instead of COUNT in DAX.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following example shows how to count the number of rows in the table Orders. The expected result is 52761.

```
DAX

= COUNTROWS('Orders')
```

Example 2

The following example demonstrates how to use COUNTROWS with a row context. In this scenario, there are two sets of data that are related by order number. The table Reseller contains one row for each reseller; the table ResellerSales contains multiple rows for each order, each row containing one order for a particular reseller. The tables are connected by a relationship on the column, ResellerKey.

The formula gets the value of ResellerKey and then counts the number of rows in the related table that have the same reseller ID. The result is output in the column, **CalculatedColumn1**.

```
DAX

= COUNTROWS(RELATEDTABLE(ResellerSales))
```

The following table shows a portion of the expected results:

Expand table

ResellerKey	CalculatedColumn1
1	73
2	70
3	394

Related content

COUNT function
COUNTA function
COUNTAX function
COUNTX function
Statistical functions

Feedback

Was this page helpful? Yes No

COUNTX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of rows that contain a non-blank value or an expression that evaluates to a non-blank value, when evaluating an expression over a table.

Syntax

```DAX
COUNTX(<table>,<expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows to be counted.
expression	An expression that returns the set of values that contains the values you want to count.

Return value

An integer.

Remarks

- The COUNTX function takes two arguments. The first argument must always be a table, or any expression that returns a table. The second argument is the column or expression that is searched by COUNTX.
- The COUNTX function counts only values, dates, or strings. If the function finds no rows to count, it returns a blank.
- If you want to count logical values, use the COUNTAX function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following formula returns a count of all rows in the Product table that have a list price.

```
DAX

= COUNTX(Product,[ListPrice])
```

Example 2

The following formula illustrates how to pass a filtered table to COUNTX for the first argument. The formula uses a filter expression to get only the rows in the Product table that meet the condition, ProductSubCategory = "Caps", and then counts the rows in the resulting table that have a list price. The FILTER expression applies to the table Products but uses a value that you look up in the related table, ProductSubCategory.

```
DAX

=
COUNTX(FILTER(Product,RELATED(ProductSubcategory[EnglishProductSubcategoryNa
me])="Caps"), Product[ListPrice])
```

Related content

COUNT function
COUNTA function
COUNTAX function
Statistical functions

Feedback

Was this page helpful? Yes No

DISTINCTCOUNT

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of distinct values in a column.

Syntax

DAX

```
DISTINCTCOUNT(<column>)
```

Parameters

Expand table

Term	Description
`column`	The column that contains the values to be counted

Return value

The number of distinct values in `column`.

Remarks

- The only argument allowed to this function is a column. You can use columns containing any type of data. When the function finds no rows to count, it returns a BLANK, otherwise it returns the count of distinct values.
- DISTINCTCOUNT function counts the BLANK value. To skip the BLANK value, use the DISTINCTCOUNTNOBLANK function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to count the number of distinct sales orders in the column ResellerSales_USD[SalesOrderNumber].

DAX

```
= DISTINCTCOUNT(ResellerSales_USD[SalesOrderNumber])
```

Using the above measure in a table with calendar year in the side and product category on top returns the following results:

Expand table

Row Labels	Accessories	Bikes	Clothing	Components	-	Grand Total
2005	135	345	242	205		366
2006	356	850	644	702		1015
2007	531	1234	963	1138		1521
2008	293	724	561	601		894
					1	1
Grand Total	**1315**	**3153**	**2410**	**2646**	**1**	**3797**

Understanding distinct count totals

Distinct count totals are not additive. The Grand Total is not the sum of the values in each category.

In the table above, you might expect the Grand Total for 2005 (366) to equal the sum of Accessories (135) + Bikes (345) + Clothing (242) + Components (205) = 927. However, the actual Grand Total is 366, which is much lower.

This happens because the same order can appear in multiple categories. For example, if order #1001 contains both a bike and an accessory, that order is counted once in the Bikes column and once in the Accessories column. But when calculating the Grand Total for the row, order #1001 is only counted once because it's still just one distinct order.

This is the correct and expected behavior of distinct counts:

- **Category values**: Count distinct orders within that specific category only.
- **Row totals**: Count distinct orders across all categories for that year - each order is counted only once, regardless of how many categories it appears in.
- **Grand Total**: Count distinct orders across the entire dataset.

If you need totals that add up, consider using COUNT or COUNTROWS instead. However, be aware that these functions count rows, not distinct values, and will give different results.

Related content

- COUNT function
- COUNTA function
- COUNTAX function
- COUNTX function
- Statistical functions

Last updated on 01/22/2026

DISTINCTCOUNTNOBLANK

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Counts the number of distinct values in a column.

Syntax

```
DAX

DISTINCTCOUNTNOBLANK (<column>)
```

Parameters

Expand table

Term	Description
column	The column that contains the values to be counted

Return value

The number of distinct values in *column*.

Remarks

- Unlike DISTINCTCOUNT function, DISTINCTCOUNTNOBLANK does not count the BLANK value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to count the number of distinct sales orders in the column ResellerSales_USD[SalesOrderNumber].

DAX

```
= DISTINCTCOUNT(ResellerSales_USD[SalesOrderNumber])
```

DAX query

DAX

```
EVALUATE
    ROW(
        "DistinctCountNoBlank", DISTINCTCOUNTNOBLANK(DimProduct[EndDate]),
        "DistinctCount", DISTINCTCOUNT(DimProduct[EndDate])
    )
```

Expand table

[DistinctCountNoBlank]	[DistinctCount]
2	3

Related content

DISTINCTCOUNT

Feedback

Was this page helpful? Yes No

MAX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the largest value in a column, or between two scalar expressions.

Syntax

DAX

```
MAX(<column>)
```

DAX

```
MAX(<expression1>, <expression2>)
```

Parameters

Expand table

Term	Definition
column	The column in which you want to find the largest value.
expression	Any DAX expression which returns a single value.

Return value

The largest value.

Remarks

- When comparing two expressions, blank is treated as 0 when comparing. That is, Max(1, Blank()) returns 1, and Max(-1, Blank()) returns 0. If both arguments are blank, MAX returns a blank. If either expression returns a value which is not allowed, MAX returns an error.

- TRUE/FALSE values are not supported. If you want to evaluate a column of TRUE/FALSE values, use the MAXA function.

Example 1

The following example returns the largest value found in the ExtendedAmount column of the InternetSales table.

```
DAX

= MAX(InternetSales[ExtendedAmount])
```

Example 2

The following example returns the largest value between the result of two expressions.

```
DAX

= Max([TotalSales], [TotalPurchases])
```

Related content

MAXA function
MAXX function
Statistical functions

Feedback

Was this page helpful? Yes No

MAXA

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the largest value in a column.

Syntax

```DAX
MAXA(<column>)
```

Parameters

Expand table

Term	Definition
column	The column in which you want to find the largest value.

Return value

The largest value.

Remarks

- The MAXA function takes as argument a column, and looks for the largest value among the following types of values:
  - Numbers
  - Dates
- Logical values, such as TRUE and FALSE. Rows that evaluate to TRUE count as 1; rows that evaluate to FALSE count as 0 (zero).
- Empty cells are ignored. If the column contains no values that can be used, MAXA returns 0 (zero).
- If you want to compare text values, use the MAX function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following example returns the greatest value from a calculated column, named **ResellerMargin**, that computes the difference between list price and reseller price.

```
DAX

= MAXA([ResellerMargin])
```

Example 2

The following example returns the largest value from a column that contains dates and times. Therefore, this formula gets the most recent transaction date.

```
DAX

= MAXA([TransactionDate])
```

Related content

MAX function
MAXX function
Statistical functions

Feedback

Was this page helpful? Yes No

MAXX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the highest value that results from evaluating an expression for each row of a table.

Syntax

DAX

```
MAXX(<table>,<expression>,[<variant>])
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.
variant	(Optional) If TRUE, and if there are variant or mixed value types, the highest value based on ORDER BY DESC is returned.

Return value

The highest value.

Remarks

- The **table** argument to the MAXX function can be a table name or an expression that evaluates to a table. The second argument indicates the expression to be evaluated for each row of the table.
- Of the values to evaluate, only the following are counted:
  - Numbers

    - Texts
    - Dates

- Blank values are skipped. TRUE/FALSE values are not supported.
- If the expression has variant or mixed value types such as text and number, then by default MAXX considers only numbers. If `<variant> = TRUE`, the maximum value is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following formula uses an expression as the second argument to calculate the total amount of taxes and shipping for each order in the InternetSales table. The expected result is 375.7184.

DAX

```
= MAXX(InternetSales, InternetSales[TaxAmt]+ InternetSales[Freight])
```

Example 2

The following formula first filters the InternetSales table by using a FILTER expression to return a subset of orders for a specific sales region defined as, [SalesTerritory] = 5. The MAXX function then evaluates the expression used as the second argument for each row of the filtered table and returns the highest amount for taxes and shipping for just those orders. The expected result is 250.3724.

DAX

```
= MAXX(FILTER(InternetSales,[SalesTerritoryCode]="5"),
InternetSales[TaxAmt]+ InternetSales[Freight])
```

Related content

MAX function
MAXA function
Statistical functions

Feedback

Was this page helpful? Yes No

MIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the smallest value in a column, or between two scalar expressions.

Syntax

DAX

```
MIN(<column>)
```

DAX

```
MIN(<expression1>, <expression2>)
```

Parameters

Expand table

Term	Definition
column	The column in which you want to find the smallest value.
expression	Any DAX expression which returns a single value.

Return value

The smallest value.

Remarks

- The MIN function takes a column or two expressions as an argument, and returns the smallest value. The following types of values in the columns are counted:
  - Numbers
  - Texts
  - Dates

    - Blanks

- When comparing expressions, blank is treated as 0 when comparing. That is, Min(1,Blank()) returns 0, and Min(-1, Blank()) returns -1. If both arguments are blank, MIN returns a blank. If either expression returns a value which is not allowed, MIN returns an error.

- TRUE/FALSE values are not supported. If you want to evaluate a column of TRUE/FALSE values, use the MINA function.

Example 1

The following example returns the smallest value from the calculated column, ResellerMargin.

```
DAX
```

```
= MIN([ResellerMargin])
```

Example 2

The following example returns the smallest value from a column that contains dates and times, TransactionDate. This formula therefore returns the date that is earliest.

```
DAX
```

```
= MIN([TransactionDate])
```

Example 3

The following example returns the smallest value from the result of two scalar expressions.

```
DAX
```

```
= Min([TotalSales], [TotalPurchases])
```

Related content

MINA function
MINX function
Statistical functions

Feedback

Was this page helpful? Yes No

MINA

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the smallest value in a column.

Syntax

```
DAX
MINA(<column>)
```

Parameters

Expand table

Term	Definition
column	The column for which you want to find the minimum value.

Return value

The smallest value.

Remarks

- The MINA function takes as argument a column that contains numbers, and determines the smallest value as follows:
  - If the column contains no values, MINA returns 0 (zero).
  - Rows in the column that evaluates to logical values, such as TRUE and FALSE are treated as 1 if TRUE and 0 (zero) if FALSE.
  - Empty cells are ignored.
- If you want to compare text values, use the MIN function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following expression returns the minimum freight charge from the table, InternetSales.

```DAX
= MINA(InternetSales[Freight])
```

Example 2

The following expression returns the minimum value in the column, PostalCode. Because the data type of the column is text, the function does not find any values, and the formula returns zero (0).

```DAX
= MINA([PostalCode])
```

Related content

MIN function
MINX function
Statistical functions

Feedback

Was this page helpful? Yes No

MINX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the lowest value that results from evaluating an expression for each row of a table.

Syntax

```
DAX
MINX(<table>, < expression>,[<variant>])
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.
variant	(Optional) If TRUE, and if there are variant or mixed value types, the lowest value based on ORDER BY ASC is returned.

Return value

The lowest value.

Remarks

- The MINX function takes as its first argument a table or an expression that returns a table. The second argument contains the expression that is evaluated for each row of the table.
- Blank values are skipped. TRUE/FALSE values are not supported.

- If the expression has variant or mixed value types such as text and number, then by default MINX considers only numbers. If `<variant> = TRUE`, the minimum value is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following example filters the InternetSales table and returns only rows for a specific sales territory. The formula then finds the minimum value in the Freight column.

DAX

```
= MINX( FILTER(InternetSales, [SalesTerritoryKey] = 5),[Freight])
```

Example 2

The following example uses the same filtered table as in the previous example, but instead of merely looking up values in the column for each row of the filtered table, the function calculates the sum of two columns, Freight and TaxAmt, and returns the lowest value resulting from that calculation.

DAX

```
= MINX( FILTER(InternetSales, InternetSales[SalesTerritoryKey] = 5),
InternetSales[Freight] + InternetSales[TaxAmt])
```

In the first example, the names of the columns are unqualified. In the second example, the column names are fully qualified.

Related content

MIN function
MINA function
Statistical functions

Feedback

Was this page helpful? Yes No

PRODUCT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the product of the numbers in a column.

Syntax

DAX

```
PRODUCT(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the numbers for which the product is to be computed.

Return value

A decimal number.

Remarks

- To return the product of an expression evaluated for each row in a table, use PRODUCTX function.
- Only the numbers in the column are counted. Blanks, logical values, and text are ignored. For example,

  `PRODUCT( Table[Column] )` is equivalent to `PRODUCTX( Table, Table[Column] )`.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the product of the AdjustedRates column in an Annuity table:

```DAX
= PRODUCT( Annuity[AdjustedRates] )
```

Related content

PRODUCTX

Feedback

Was this page helpful? Yes No

PRODUCTX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the product of an expression evaluated for each row in a table.

Syntax

DAX

```
PRODUCTX(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.

Return value

A decimal number.

Remarks

- To return the product of the numbers in a column, use PRODUCT.
- The PRODUCTX function takes as its first argument a table, or an expression that returns a table. The second argument is a column that contains the numbers for which you want to compute the product, or an expression that evaluates to a column.
- Only the numbers in the column are counted. Blanks, logical values, and text are ignored.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the future value of an investment:

```
DAX
= [PresentValue] * PRODUCTX( AnnuityPeriods, 1+[FixedInterestRate] )
```

Related content

PRODUCT

Feedback

Was this page helpful? Yes No

SUM

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Adds all the numbers in a column.

Syntax

```
DAX
SUM(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the numbers to sum.

Return value

A decimal number.

Remarks

If you want to filter the values that you are summing, you can use the SUMX function and specify an expression to sum over.

Example

The following example adds all the numbers that are contained in the column, Amt, from the table, Sales.

```
DAX
```

```
= SUM(Sales[Amt])
```

Related content

SUMX

Feedback

Was this page helpful? Yes No

SUMX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the sum of an expression evaluated for each row in a table.

Syntax

```
DAX
SUMX(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.

Return value

A decimal number.

Remarks

- The SUMX function takes as its first argument a table, or an expression that returns a table. The second argument is a column that contains the numbers you want to sum, or an expression that evaluates to a column.
- The SUMX is an iterator function.
- Only the numbers in the column are counted. Blanks, logical values, and text are ignored.
- For more complex examples of SUMX in formulas, see ALL and CALCULATETABLE.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example first filters the table, InternetSales, on the expression, 'InternetSales[SalesTerritoryID] = 5`, and then returns the sum of all values in the Freight column. In other words, the expression returns the sum of freight charges for only the specified sales area.

DAX

```
= SUMX(FILTER(InternetSales, InternetSales[SalesTerritoryID]=5),[Freight])
```

If you do not need to filter the column, use the SUM function. The SUM function is similar to the Excel function of the same name, except that it takes a column as a reference.

Related content

SUM
Statistical functions

Feedback

Was this page helpful? Yes No

Date and time functions

Article • 10/20/2023

These functions help you create calculations based on dates and time. Many of the functions in DAX are similar to the Excel date and time functions. However, DAX functions use a **datetime** data type, and can take values from a column as an argument.

In this category

Function	Description
CALENDAR	Returns a table with a single column named "Date" that contains a contiguous set of dates.
CALENDARAUTO	Returns a table with a single column named "Date" that contains a contiguous set of dates.
DATE	Returns the specified date in datetime format.
DATEDIFF	Returns the number of interval boundaries between two dates.
DATEVALUE	Converts a date in the form of text to a date in datetime format.
DAY	Returns the day of the month, a number from 1 to 31.
EDATE	Returns the date that is the indicated number of months before or after the start date.
EOMONTH	Returns the date in datetime format of the last day of the month, before or after a specified number of months.
HOUR	Returns the hour as a number from 0 (12:00 A.M.) to 23 (11:00 P.M.).
MINUTE	Returns the minute as a number from 0 to 59, given a date and time value.
MONTH	Returns the month as a number from 1 (January) to 12 (December).
NETWORKDAYS	Returns the number of whole workdays between two dates.
NOW	Returns the current date and time in datetime format.
QUARTER	Returns the quarter as a number from 1 to 4.
SECOND	Returns the seconds of a time value, as a number from 0 to 59.
TIME	Converts hours, minutes, and seconds given as numbers to a time in datetime format.

Function	Description
TIMEVALUE	Converts a time in text format to a time in datetime format.
TODAY	Returns the current date.
UTCNOW	Returns the current UTC date and time
UTCTODAY	Returns the current UTC date.
WEEKDAY	Returns a number from 1 to 7 identifying the day of the week of a date.
WEEKNUM	Returns the week number for the given date and year according to the return_type value.
YEAR	Returns the year of a date as a four digit integer in the range 1900-9999.
YEARFRAC	Calculates the fraction of the year represented by the number of whole days between two dates.

Feedback

Was this page helpful? Yes No

CALENDAR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with a single column named "Date" that contains a contiguous set of dates. The range of dates is from the specified start date to the specified end date, inclusive of those two dates.

Syntax

DAX

```
CALENDAR(<start_date>, <end_date>)
```

Parameters

Expand table

Term	Definition
start_date	Any DAX expression that returns a datetime value.
end_date	Any DAX expression that returns a datetime value.

Return value

Returns a table with a single column named "Date" containing a contiguous set of dates. The range of dates is from the specified start date to the specified end date, inclusive of those two dates.

Remarks

- An error is returned if start_date is greater than end_date.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

The following formula returns a table with dates between January 1st, 2015 and December 31st, 2021.

```
DAX

= CALENDAR (DATE (2015, 1, 1), DATE (2021, 12, 31))
```

For a data model which includes actual sales data and future sales forecasts, the following expression returns a date table covering the range of dates in both the Sales and Forecast tables.

```
DAX

= CALENDAR (MINX (Sales, [Date]), MAXX (Forecast, [Date]))
```

Feedback

Was this page helpful? Yes No

CALENDARAUTO

06/30/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns a table with a single column named "Date" that contains a contiguous set of dates. The range of dates is calculated automatically based on data in the model.

Syntax

```DAX
CALENDARAUTO([fiscal_year_end_month])
```

Parameters

Expand table

Term	Definition
`fiscal_year_end_month`	Any DAX expression that returns an integer from 1 to 12. If omitted, defaults to the value specified in the calendar table template for the current user, if present; otherwise, defaults to 12.

Return value

Returns a table with a single column named "Date" that contains a contiguous set of dates. The range of dates is calculated automatically based on data in the model.

Remarks

- The date range is calculated as follows:
  - The earliest date in the model which is not in a calculated column or calculated table is taken as the MinDate.

    - The latest date in the model which is not in a calculated column or calculated table is taken as the MaxDate.
    - The date range returned is dates between the beginning of the fiscal year associated with MinDate and the end of the fiscal year associated with MaxDate.

- An error is returned if the model does not contain any datetime values which are not in calculated columns or calculated tables.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In this example, the MinDate and MaxDate in the data model are July 1, 2018 and June 30, 2019. This model contains two calculated tables:

- **CalendarAuto**. Defined as `CALENDARAUTO()`. Returns all dates between January 1, 2018 and December 31, 2019.
- **CalendarAuto3**. Defined as `CALENDARAUTO(3)`. Returns all dates between April 1, 2018 and March 31, 2020. As in this example `fiscal_year_end_month` is 3, the first years starts on April 1st and ends on March 31st. As a result, the range is determined by selecting the first day of the fiscal year on or before the MinDate's year (April 1, 2018) and the last day of the fiscal year after or within the MaxDate's year (March 31, 2020).

Below are measure definitions and their return values on the example model:

DAX

```
MinDate = MIN('Date'[Date]) //returns July 1, 2018
MaxDate = MAX('Date'[Date]) //returns June 30, 2019
MinCalendarAuto = MIN('CalendarAuto'[Date]) //returns January 1, 2018
MaxCalendarAuto = MAX('CalendarAuto'[Date]) //returns December 31, 2019
MinCalendarAuto3 = MIN('CalendarAuto3'[Date]) //returns April 1, 2018
MaxCalendarAuto3 = MAX('CalendarAuto3'[Date]) //returns March 31, 2020
```

DATE

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the specified date in `datetime` format.

Syntax

DAX

```
DATE(<year>, <month>, <day>)
```

Parameters

Expand table

Term	Definition
`year`	A number representing the year. The value of the `year` argument can include one to four digits. The `year` argument is interpreted according to the date system used by your computer. Dates beginning with March 1, 1900 are supported. If you enter a number that has decimal places, the number is rounded. For values greater than 9999 or less than zero (negative values), the function returns a `#VALUE!` error. If the `year` value is between 0 and 49, the value is added to 2000 to produce the final value. If it is between 50 and 99, the value is added to 1900 to produce the final value. See the examples below. **Note:** You should use four digits for the `year` argument whenever possible to prevent unwanted results. For example, using 07 returns 2007 as the year value.
`month`	A number representing the month or a calculation according to the following rules: Negative integers are not supported. Valid values are 1-12. If `month` is a number from 1 to 12, then it represents a month of the year. 1 represents January, 2 represents February, and so on until 12 that represents December. If you enter an integer larger than 12, the following computation occurs: the date is calculated by adding the value of `month` to the `year`. For example, if you have DATE(2008, 18, 1), the function

Term	Definition
	returns a datetime value equivalent to June 1st of 2009, because 18 months are added to the beginning of 2008 yielding a value of June 2009. See examples below.
`day`	A number representing the day or a calculation according to the following rules: Negative integers are not supported. Valid values are 1-31. If `day` is a number from 1 to the last day of the given month then it represents a day of the month. If you enter an integer larger than last day of the given month, the following computation occurs: the date is calculated by adding the value of `day` to `month`. For example, in the formula `DATE( 2008, 3, 32)`, the DATE function returns a `datetime` value equivalent to April 1st of 2008, because 32 days are added to the beginning of March yielding a value of April 1st. If `day` contains a decimal portion, it is rounded to the nearest integer value.

Return value

Returns the specified date (`datetime`).

Remarks

- The DATE function takes the integers that are input as arguments, and generates the corresponding date. The DATE function is most useful in situations where the year, month, and day are supplied by formulas. For example, the underlying data might contain dates in a format that is not recognized as a date, such as YYYYMMDD. You can use the DATE function in conjunction with other functions to convert the dates to a number that can be recognized as a date.
- In contrast to Microsoft Excel, which stores dates as a serial number, DAX date functions always return a `datetime` data type. However, you can use formatting to display dates as serial numbers if you want.
- Date and datetime can also be specified as a literal in the format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"`, or `dt"YYYY-MM-DD hh:mm:ss"`. When specified as a literal, using the DATE function in the expression is not necessary. To learn more, see DAX Syntax | Date and time.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Simple Date

The following formula returns the date July 8, 2009:

```
DAX
= DATE(2009,7,8)
```

Years before 100

If the value that you enter for the `year` argument is between 0 (zero) and 49 (inclusive), that value is added to 2000 to calculate the year.

The following formula returns January 2, 2008: (2000+08).

```
DAX
= DATE(08,1,2)
```

If the value that you enter for the `year` argument is between 50 and 99 (inclusive), that value is added to 1900 to calculate the year.

The following formula returns January 2, 1950: (1900+50).

```
DAX
= DATE(50,1,2)
```

Years after 100

If `year` is between 100 and 9999 (inclusive), that value is used as the year. The following formula returns January 2, 2008:

```
DAX
= DATE(2008,1,2)
```

Months

If `month` is greater than 12, `month` adds that number of months to the first month in the year specified. The following formula returns the date February 2, 2009:

```
DAX
= DATE(2008,14,2)
```

Days

If `day` is greater than the number of days in the month specified, `day` adds that number of days to the first day in the month. The following formula returns the date February 4, 2008:

```
DAX
= DATE(2008,1,35)
```

Related content

Date and time functions DAY function TODAY function

Last updated on 11/18/2025

DATEDIFF

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of interval boundaries between two dates.

Syntax

```DAX
DATEDIFF(<Date1>, <Date2>, <Interval>)
```

Parameters

Expand table

Term	Definition
`Date1`	A scalar datetime value.
`Date2`	A scalar datetime value.
`Interval`	The interval to use when comparing dates. The value can be one of the following: - SECOND - MINUTE - HOUR - DAY - WEEK - MONTH - QUARTER - YEAR

Return value

The count of interval boundaries between two dates.

Remarks

DATEDIFF returns a positive result if Date2 is larger than Date1. It returns a negative result if Date1 is larger than Date2.

Example

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

The following DAX query:

DAX

```
EVALUATE
VAR StartDate =
    DATE ( 2019, 07, 01 )
VAR EndDate =
    DATE ( 2021, 12, 31 )
RETURN
    {
        ( "Year", DATEDIFF ( StartDate, EndDate, YEAR ) ),
        ( "Quarter", DATEDIFF ( StartDate, EndDate, QUARTER ) ),
        ( "Month", DATEDIFF ( StartDate, EndDate, MONTH ) ),
        ( "Week", DATEDIFF ( StartDate, EndDate, WEEK ) ),
        ( "Day", DATEDIFF ( StartDate, EndDate, DAY ) )
    }
```

Returns the following:

Expand table

Value1	Value2
Year	2
Quarter	9
Month	29
Week	130
Day	914

Last updated on 08/04/2026

DATEVALUE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts a date in text format to a date in datetime format.

Syntax

DAX

```
DATEVALUE(date_text)
```

Parameters

Expand table

Term	Definition
date_text	Text that represents a date.

Property Value/Return value

A date in **datetime** format.

Remarks

- When converting, DATEVALUE uses the locale and date/time settings of the model to determine a date value. If the model date/time settings represent dates in the format of Month/Day/Year, then the string, "1/8/2009", is converted to a **datetime** value equivalent to January 8th of 2009. However, if the model date/time settings represent dates in the format of Day/Month/Year, the same string is converted as a **datetime** value equivalent to August 1st of 2009.
- If conversion using the locale and date/time settings of the model fails, DATEVALUE will attempt to use other date formats. In this case, some rows may be converted using one format and other rows are converted using a different format.

For example, "5/4/2018" may convert to May 4th of 2018, and "20/4/2018" may convert to April 20th.

- If the year portion of the **date_text** argument is omitted, the DATEVALUE function uses the current year from your computer's built-in clock. Time information in the **date_text** argument is ignored.
- Model locale and data/time settings are initially determined by the application and computer when the model is created.
- Date and datetime can also be specified as a literal in the format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"`, or `dt"YYYY-MM-DD hh:mm:ss"`. When specified as a literal, using the DATEVALUE function in the expression is not necessary. To learn more, see DAX Syntax | Date and time.

Example

The following example returns a different **datetime** value depending on the model locale and settings for how dates and times are presented.

- In date/time settings where the day precedes the month, the example returns a **datetime** value corresponding to January 8th of 2009.
- In date/time settings where the month precedes the day, the example returns a **datetime** value corresponding to August 1st of 2009.

DAX

```
= DATEVALUE("8/1/2009")
```

Related content

Date and time functions

Feedback

Was this page helpful? Yes No

DAY

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the day of the month, a number from 1 to 31.

Syntax

DAX

```
DAY(<date>)
```

Parameters

Expand table

Term	Definition
date	A date in **datetime** format, or a text representation of a date.

Return value

An integer number indicating the day of the month.

Remarks

- The DAY function takes as an argument the date of the day you are trying to find. Dates can be provided to the function by using another date function, by using an expression that returns a date, or by typing a date in a **datetime** format. You can also type a date in one of the accepted string formats for dates.
- Values returned by the YEAR, MONTH and DAY functions will be Gregorian values regardless of the display format for the supplied date value. For example, if the display format of the supplied date is Hijri, the returned values for the YEAR, MONTH and DAY functions will be values associated with the equivalent Gregorian date.

- When the date argument is a text representation of the date, the day function uses the locale and date/time settings of the client computer to understand the text value in order to perform the conversion. If the current date/time settings represent dates in the format of Month/Day/Year, then the string, "1/8/2009", is interpreted as a **datetime** value equivalent to January 8th of 2009, and the function returns 8. However, if the current date/time settings represent dates in the format of Day/Month/Year, the same string would be interpreted as a **datetime** value equivalent to August 1st of 2009, and the function returns 1.

Example: Getting the day from a date column

The following formula returns the day from the date in the column, [Birthdate].

DAX

```
= DAY([Birthdate])
```

Example - Getting the day from a string date

The following formulas return the day, 4, using dates that have been supplied as strings in an accepted text format.

DAX

```
= DAY("3-4-1007")
= DAY("March 4 2007")
```

Example - Using a day value as a condition

The following expression returns the day that each sales order was placed, and flags the row as a promotional sale item if the order was placed on the 10th of the month.

DAX

```
= IF( DAY([SalesDate])=10,"promotion","")
```

Related content

Date and time functions
TODAY function
DATE function

Feedback

Was this page helpful? Yes No

EDATE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the date that is the indicated number of months before or after the start date. Use EDATE to calculate maturity dates or due dates that fall on the same day of the month as the date of issue.

Syntax

```DAX
EDATE(<start_date>, <months>)
```

Parameters

Expand table

Term	Definition
start_date	A date in **datetime** or **text** format that represents the start date.
months	An integer that represents the number of months before or after **start_date**.

Return value

A date (**datetime**).

Remarks

- In contrast to Microsoft Excel, which stores dates as sequential serial numbers, DAX works with dates in a **datetime** format. Dates stored in other formats are converted implicitly.
- If **start_date** is not a valid date, EDATE returns an error. Make sure that the column reference or date that you supply as the first argument is a date.
- If **months** is not an integer, it is truncated.

- When the date argument is a text representation of the date, the EDATE function uses the locale and date time settings of the client computer to understand the text value in order to perform the conversion. If the current date time settings represent a date in the format of Month/Day/Year, then the following string "1/8/2009" is interpreted as a datetime value equivalent to January 8th of 2009. However, if the current date time settings represent a date in the format of Day/Month/Year, the same string would be interpreted as a datetime value equivalent to August 1st of 2009.
- If the requested date is past the last day of the corresponding month, then the last day of the month is returned. For example, the following functions: EDATE("2009-01-29", 1), EDATE("2009-01-30", 1), EDATE("2009-01-31", 1) return February 28th of 2009; that corresponds to one month after the start date.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns the date three months after the order date, which is stored in the column [TransactionDate].

DAX

```
= EDATE([TransactionDate],3)
```

Related content

EOMONTH function
Date and time functions

Feedback

Was this page helpful? Yes No

EOMONTH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the date in **datetime** format of the last day of the month, before or after a specified number of months. Use EOMONTH to calculate maturity dates or due dates that fall on the last day of the month.

Syntax

DAX

```
EOMONTH(<start_date>, <months>)
```

Parameters

Expand table

Term	Definition
start_date	The start date in **datetime** format, or in an accepted text representation of a date.
months	A number representing the number of months before or after the **start_date**. **Note:** If you enter a number that is not an integer, the number is rounded up or down to the nearest integer.

Return value

A date (**datetime**).

Remarks

- In contrast to Microsoft Excel, which stores dates as sequential serial numbers, DAX works with dates in a **datetime** format. The EOMONTH function can accept dates in other formats, with the following restrictions:
- If **start_date** is not a valid date, EOMONTH returns an error.

- If **start_date** is a numeric value that is not in a **datetime** format, EOMONTH will convert the number to a date. To avoid unexpected results, convert the number to a **datetime** format before using the EOMONTH function.
- If **start_date** plus months yields an invalid date, EOMONTH returns an error. Dates before March 1st of 1900 and after December 31st of 9999 are invalid.
- When the date argument is a text representation of the date, the EDATE function uses the locale and date time settings, of the client computer, to understand the text value in order to perform the conversion. If current date time settings represent a date in the format of Month/Day/Year, then the following string "1/8/2009" is interpreted as a datetime value equivalent to January 8th of 2009. However, if the current date time settings represent a date in the format of Day/Month/Year, the same string would be interpreted as a datetime value equivalent to August 1st of 2009.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following expression returns May 31, 2008, because the **months** argument is rounded to 2.

DAX

```
= EOMONTH("March 3, 2008",1.5)
```

Related content

EDATE function
Date and time functions

Feedback

Was this page helpful? Yes No

HOUR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the hour as a number from 0 (12:00 A.M.) to 23 (11:00 P.M.).

Syntax

DAX

```
HOUR(<datetime>)
```

Parameters

Expand table

Term	Definition
datetime	A **datetime** value, such as 16:48:00 or 4:48 PM.

Return value

An integer number from 0 to 23.

Remarks

- The HOUR function takes as argument the time that contains the hour you want to find. You can supply the time by using a date/time function, an expression that returns a **datetime**, or by typing the value directly in one of the accepted time formats. Times can also be entered as any accepted text representation of a time.
- When the **datetime** argument is a text representation of the date and time, the function uses the locale and date/time settings of the client computer to understand the text value in order to perform the conversion. Most locales use the colon (:) as the time separator and any input text using colons as time separators will parse correctly. Review your locale settings to understand your results.

Example 1

The following example returns the hour from the **TransactionTime** column of a table named **Orders**.

```
DAX
= HOUR('Orders'[TransactionTime])
```

Example 2

The following example returns 15, meaning the hour corresponding to 3 PM in a 24-hour clock. The text value is automatically parsed and converted to a date/time value.

```
DAX
= HOUR("March 3, 2008 3:00 PM")
```

Related content

Date and time functions
MINUTE function
YEAR function
SECOND function

Feedback

Was this page helpful? Yes No

MINUTE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the minute as a number from 0 to 59, given a date and time value.

Syntax

```DAX
MINUTE(<datetime>)
```

Parameters

Expand table

Term	Definition
datetime	A **datetime** value or text in an accepted time format, such as 16:48:00 or 4:48 PM.

Return value

An integer number from 0 to 59.

Remarks

- In contrast to Microsoft Excel, which stores dates and times in a serial numeric format, DAX uses a **datetime** data type for dates and times. You can provide the **datetime** value to the MINUTE function by referencing a column that stores dates and times, by using a date/time function, or by using an expression that returns a date and time.
- When the **datetime** argument is a text representation of the date and time, the function uses the locale and date/time settings of the client computer to understand the text value in order to perform the conversion. Most locales use the colon (:) as the time separator and any input text using colons as time separators will parse correctly. Verify your locale settings to understand your results.

Example 1

The following example returns the minute from the value stored in the **TransactionTime** column of the **Orders** table.

```
DAX
= MINUTE(Orders[TransactionTime])
```

Example 2

The following example returns 45, which is the number of minutes in the time 1:45 PM.

```
DAX
= MINUTE("March 23, 2008 1:45 PM")
```

Related content

Date and time functions
HOUR function
YEAR function
SECOND function

Feedback

Was this page helpful? Yes No

MONTH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the month as a number from 1 (January) to 12 (December).

Syntax

```DAX
MONTH(<datetime>)
```

Parameters

Expand table

Term	Definition
date	A date in **datetime** or text format.

Return value

An integer number from 1 to 12.

Remarks

- In contrast to Microsoft Excel, which stores dates as serial numbers, DAX uses a **datetime** format when working with dates. You can enter the date used as argument to the MONTH function by typing an accepted **datetime** format, by providing a reference to a column that contains dates, or by using an expression that returns a date.
- Values returned by the YEAR, MONTH and DAY functions will be Gregorian values regardless of the display format for the supplied date value. For example, if the display format of the supplied date is Hijri, the returned values for the YEAR, MONTH and DAY functions will be values associated with the equivalent Gregorian date.

- When the date argument is a text representation of the date, the function uses the locale and date time settings of the client computer to understand the text value in order to perform the conversion. If the current date time settings represent a date in the format of Month/Day/Year, then the following string "1/8/2009" is interpreted as a datetime value equivalent to January 8th of 2009, and the function yields a result of 1. However, if the current date time settings represent a date in the format of Day/Month/Year, then the same string would be interpreted as a datetime value equivalent to August 1st of 2009, and the function yields a result of 8.
- If the text representation of the date cannot be correctly converted to a datetime value, the function returns an error.

Example 1

The following expression returns 3, which is the integer corresponding to March, the month in the **date** argument.

```
DAX

= MONTH("March 3, 2008 3:45 PM")
```

Example 2

The following expression returns the month from the date in the **TransactionDate** column of the **Orders** table.

```
DAX

= MONTH(Orders[TransactionDate])
```

Related content

Date and time functions
HOUR function
MINUTE function
YEAR function
SECOND function

Feedback

Was this page helpful? Yes No

NETWORKDAYS

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of whole workdays between two dates (inclusive). Parameters specify which and how many days are weekend days. Weekend days and days specified as holidays are not considered as workdays.

Syntax

DAX

```
NETWORKDAYS(<start_date>, <end_date>[, <weekend>, <holidays>])
```

Parameters

Expand table

Term	Definition
`start_date`	A date that represents the start date. The dates for which the difference is to be computed. The start_date can be earlier than, the same as, or later than the end_date.
`end_date`	A date that represents the end date. The dates for which the difference is to be computed. The start_date can be earlier than, the same as, or later than the end_date.
`weekend`	Indicates the days of the week that are weekend days and are not included in the number of whole working days between start_date and end_date. Weekend is a weekend number that specifies when weekends occur. Weekend number values indicate the following weekend days: 1 or omitted: Saturday, Sunday 2: Sunday, Monday 3: Monday, Tuesday 4: Tuesday, Wednesday 5: Wednesday, Thursday 6: Thursday, Friday 7: Friday, Saturday 11: Sunday only 12: Monday only 13: Tuesday only

Term	Definition
	14: Wednesday only 15: Thursday only 16: Friday only 17: Saturday only
`holidays`	A column table of one or more dates that are to be excluded from the working day calendar.

Return Value

An integer number of whole workdays.

Remarks

- This DAX function is similar to Excel NETWORKDAYS.INTL and NETWORKDAYS functions.
- If start_date and end_date both are BLANK, the output value is also BLANK.
- If either start_date or end_date is BLANK, the BLANK start_date or end_date will be treated as Date(1899, 12, 30).
- Dates must be specified by using DATE function or as the result of another expression. For example, `DATE ( 2022, 5, 30 )`, specifies May 30th, 2022. Dates can also be specified as a literal in format, `(dt”2022-05-30”)`. Do not specify dates as text.

Example

The following expression:

DAX

```
= NETWORKDAYS (
    DATE ( 2022, 5, 28 ),
    DATE ( 2022, 5, 30 ),
    1,
    {
        DATE ( 2022, 5, 30 )
    }
)
```

Returns:

Expand table

[Value]
0

In this example, 0 is returned because the start date is a Saturday and the end date is a Monday. The weekend parameter specifies that the weekend is Saturday and Sunday, so those are not work days. The holiday parameter marks the 30th (the end date) as a holiday, so no working days remain.

Feedback

Was this page helpful? Yes No

NOW

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the current date and time in **datetime** format.

The NOW function is useful when you need to display the current date and time on a worksheet or calculate a value based on the current date and time, and have that value updated each time you open the worksheet.

Syntax

```DAX
NOW()
```

Return value

A date (**datetime)**.

Remarks

- The result of the NOW function changes only when the column that contains the formula is refreshed. It is not updated continuously.
- In the Power BI Service, the result of the NOW function is always in the UTC timezone.
- The TODAY function returns the same date but is not precise with regard to time; the time returned is always 12:00:00 AM and only the date is updated.

Example

The following example returns the current date and time plus 3.5 days:

```DAX
```

```
= NOW()+3.5
```

Related content

UTCNOW function
TODAY function

Feedback

Was this page helpful? Yes No

QUARTER

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the quarter as a number from 1 (January – March) to 4 (October – December).

Syntax

```
DAX
QUARTER(<date>)
```

Parameters

Expand table

Term	Definition
`date`	A date.

Return value

An integer number from 1 to 4.

Remarks

- If the input value is BLANK, the output value is also BLANK.
- This function isn't available in Power Pivot in Excel.

Example 1

The following DAX query:

```
DAX
EVALUATE { QUARTER(DATE(2019, 2, 1)), QUARTER(DATE(2018, 12, 31)) }
```

Returns:

Expand table

[Value]
1
4

Example 2

The following DAX query:

DAX

```
EVALUATE
ADDCOLUMNS(
    FILTER(
        VALUES(
            FactInternetSales[OrderDate]),
            [OrderDate] >= DATE(2008, 3, 31) && [OrderDate] <= DATE(2008, 4, 1)
        ),
    "Quarter", QUARTER([OrderDate])
)
```

Returns:

Expand table

FactInternetSales[OrderDate]	[Quarter]
3/31/2008	1
4/1/2008	2

Related content

- YEAR
- MONTH
- DAY

Last updated on 01/23/2026

SECOND

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the seconds of a time value, as a number from 0 to 59.

Syntax

DAX

```
SECOND(<time>)
```

Parameters

Expand table

Term	Definition
time	A time in **datetime** format, such as 16:48:23 or 4:48:47 PM.

Return value

An integer number from 0 to 59.

Remarks

- In contrast to Microsoft Excel, which stores dates and times as serial numbers, DAX uses a **datetime** format when working with dates and times. If the source data is not in this format, DAX implicitly converts the data. You can use formatting to display the dates and times as a serial number of you need to.
- The date/time value that you supply as an argument to the SECOND function can be entered as a text string within quotation marks (for example, "6:45 PM"). You can also provide a time value as the result of another expression, or as a reference to a column that contains times.

- If you provide a numeric value of another data type, such as 13.60, the value is interpreted as a serial number and is represented as a **datetime** data type before extracting the value for seconds. To make it easier to understand your results, you might want to represent such numbers as dates before using them in the SECOND function. For example, if you use SECOND with a column that contains a numeric value such as, **25.56**, the formula returns 24. That is because, when formatted as a date, the value 25.56 is equivalent to January 25, 1900, 1:26:24 PM.
- When the **time** argument is a text representation of a date and time, the function uses the locale and date/time settings of the client computer to understand the text value in order to perform the conversion. Most locales use the colon (:) as the time separator and any input text using colons as time separators will parse correctly. Review your locale settings to understand your results.

Example 1

The following formula returns the number of seconds in the time contained in the **TransactionTime** column of a table named **Orders**.

```
DAX

= SECOND('Orders'[TransactionTime])
```

Example 2

The following formula returns 3, which is the number of seconds in the time represented by the value, **March 3, 2008 12:00:03**.

```
DAX

= SECOND("March 3, 2008 12:00:03")
```

Related content

Date and time functions
HOUR
MINUTE
YEAR

Feedback

Was this page helpful? Yes No

TIME

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts hours, minutes, and seconds given as numbers to a time in **datetime** format.

Syntax

DAX

```
TIME(hour, minute, second)
```

Parameters

Expand table

Term	Definition
hour	**Import mode:** A number from 0 to 32767 representing the hour. Any value greater than 23 will be divided by 24 and the remainder will be treated as the hour value, represented as a fraction of a day. For example, TIME(27,0,0) = TIME(3,0,0) = 3:00:00 AM **DirectQuery mode:** A number from 0 to 23 representing the hour.
minute	**Import mode:** A number from 0 to 32767 representing the minute. Any value greater than 59 minutes will be converted to hours and minutes. Any value greater than 1440 (24 hours) does not alter the date portion - instead, it will be divided by 1440 and the remainder will be treated as the minute value, represented as a fraction of a day. For example, TIME(0,2190,0) = TIME(0,750,0) = TIME(12,30,0) = 12:30:00 PM **DirectQuery mode:** A number from 0 to 59 representing the minute.
second	**Import mode:** A number from 0 to 32767 representing the second. Any value greater than 59 will be converted to hours, minutes, and seconds. For example, TIME(0,0,2000) = TIME(0,33,20) = 12:33:20 AM **DirectQuery mode:** A number from 0 to 59 representing the second.

Return value

A time (**datetime**) ranging from 00:00:00 (12:00:00 AM) to 23:59:59 (11:59:59 PM).

Remarks

- In contrast to Microsoft Excel, which stores dates and times as serial numbers, DAX works with date and time values in a **datetime** format. Numbers in other formats are implicitly converted when you use a date/time value in a DAX function. If you need to use serial numbers, you can use formatting to change the way that the numbers are displayed.
- Time values are a portion of a date value, and in the serial number system are represented by a decimal number. Therefore, the **datetime** value 12:00 PM is equivalent to 0.5, because it is half of a day.
- You can supply the arguments to the TIME function as values that you type directly, as the result of another expression, or by a reference to a column that contains a numeric value.
- Date and datetime can also be specified as a literal in the format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"`, or `dt"YYYY-MM-DD hh:mm:ss"`. When specified as a literal, using the TIME function in the expression is not necessary. To learn more, see DAX Syntax | Date and time.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following examples both return the time, 3:00 AM:

DAX

```
= TIME(27,0,0)
```

DAX

```
= TIME(3,0,0)
```

Example 2

The following examples both return the time, 12:30 PM:

DAX

```
= TIME(0,750,0)
```

DAX

```
= TIME(12,30,0)
```

Example 3

The following example creates a time based on the values in the columns, `intHours`, `intMinutes`, `intSeconds`:

DAX

```
= TIME([intHours],[intMinutes],[intSeconds])
```

Related content

DATE
Date and time functions

Feedback

Was this page helpful? Yes No

TIMEVALUE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts a time in text format to a time in datetime format.

Syntax

DAX

```
TIMEVALUE(time_text)
```

Parameters

Expand table

Term	Definition
time_text	A text string that that represents a certain time of the day. Any date information included in the **time_text** argument is ignored.

Return value

A date (**datetime**).

Remarks

- Time values are a portion of a date value and represented by a decimal number. For example, 12:00 PM is represented as 0.5 because it is half of a day.
- When the **time_text** argument is a text representation of the date and time, the function uses the locale and date/time settings of the model to understand the text value in order to perform the conversion. Most locales use the colon (:) as the time separator, and any input text using colons as time separators will parse correctly. Review your locale settings to understand your results.

- Date and datetime can also be specified as a literal in the format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"`, or `dt"YYYY-MM-DD hh:mm:ss"`. When specified as a literal, using the TIMEVALUE function in the expression is not necessary. To learn more, see DAX Syntax | Date and time.

Example

DAX

```
= TIMEVALUE("20:45:30")
```

Related content

Date and time functions

Feedback

Was this page helpful? Yes No

TODAY

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the current date.

Syntax

```DAX
TODAY()
```

Return value

A date (`datetime`).

Remarks

- The TODAY function is useful when you need to have the current date displayed in a report, regardless of when you open it. It is also useful for calculating intervals.
- If the TODAY function does not update the date when you expect it to, you might need to change the settings that control when the report or semantic model is refreshed.
- The NOW function is similar but returns the exact time, whereas TODAY returns the time value 12:00:00 AM (midnight) for all dates.

Example

If you know that someone was born in 1963, you might use the following formula to find that person's age as of this year's birthday:

```DAX
= YEAR(TODAY())-1963
```

This formula uses the TODAY function as an argument for the YEAR function to obtain the current year, and then subtracts 1963, returning the person's age.

Related content

- Date and time functions
- NOW

Last updated on 01/22/2026

UTCNOW

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the current UTC date and time.

Syntax

```DAX
UTCNOW()
```

Return value

A `datetime`.

Remarks

The result of the UTCNOW function changes only when the formula is refreshed. It is not continuously updated.

Example

The following:

```DAX
EVALUATE { FORMAT(UTCNOW(), "General Date") }
```

Returns:

Expand table

Value
2/2/2018 4:48:08 AM

Related content

NOW function UTCTODAY function

Feedback

Was this page helpful? Yes No

UTCTODAY

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the current UTC date.

Syntax

```DAX
UTCTODAY()
```

Return value

A date.

Remarks

- UTCTODAY returns the time value 12:00:00 PM for all dates.
- The UTCNOW function is similar but returns the exact time and date.

Example

The following:

```DAX
EVALUATE { FORMAT(UTCTODAY(), "General Date") }
```

Returns:

Expand table

[Value]
2/2/2018

Related content

NOW function
UTCNOW function

Feedback

Was this page helpful? Yes No

WEEKDAY

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a number from 1 to 7 identifying the day of the week of a date. By default the day ranges from 1 (Sunday) to 7 (Saturday).

Syntax

DAX

```
WEEKDAY(<date>, <return_type>)
```

Parameters

Expand table

Term	Definition
date	A date in **datetime** format. Dates should be entered by using the DATE function, by using expressions that result in a date, or as the result of other formulas.
return_type	A number that determines the Return value: Return type: **1**, week begins on Sunday (1) and ends on Saturday (7). numbered 1 through 7. Return type: **2**, week begins on Monday (1) and ends on Sunday (7). Return type: **3**, week begins on Monday (0) and ends on Sunday (6).numbered 1 through 7.

Return value

An integer number from 1 to 7.

Remarks

- In contrast to Microsoft Excel, which stores dates as serial numbers, DAX works with dates and times in a **datetime** format. If you need to display dates as serial numbers, you can use the formatting options in Excel.
- You can also type dates in an accepted text representation of a date, but to avoid unexpected results, it is best to convert the text date to a **datetime** format first.
- When the date argument is a text representation of the date, the function uses the locale and date/time settings of the client computer to understand the text value in order to perform the conversion. If the current date/time settings represent dates in the format of Month/Day/Year, then the string, "1/8/2009", is interpreted as a **datetime** value equivalent to January 8th of 2009. However, if the current date/time settings represent dates in the format of Day/Month/Year, then the same string would be interpreted as a **datetime** value equivalent to August 1st of 2009.

Example

The following example gets the date from the [HireDate] column, adds 1, and displays the weekday corresponding to that date. Because the **return_type** argument has been omitted, the default format is used, in which 1 is Sunday and 7 is Saturday. If the result is 4, the day would be Wednesday.

DAX

```
= WEEKDAY([HireDate]+1)
```

Related content

Date and time functions
WEEKNUM function
YEARFRAC function

Feedback

Was this page helpful? Yes No

WEEKNUM

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the week number for the given date according to the **return_type** value. The week number indicates where the week falls numerically within a year.

There are two *systems* used for this function:

- **System 1** - The week containing January 1 is the first week of the year and is numbered week 1.
- **System 2** - The week containing the first Thursday of the year is the first week of the year and is numbered as week 1. This system is the methodology specified in ISO 8601, which is commonly known as the European week numbering system.

Syntax

DAX

```
WEEKNUM(<date>[, <return_type>])
```

Parameters

Expand table

Term	Definition
date	The date in **datetime** format.
return_type	(Optional) A number that determines on which day the week begins. Default is 1. See Remarks.

Return value

An integer number.

Remarks

- By default, the WEEKNUM function uses a calendar convention in which the week containing January 1 is considered to be the first week of the year. However, the ISO 8601 calendar standard, widely used in Europe, defines the first week as the one with the majority of days (four or more) falling in the new year. This means that if **return_type** is any valid value other than 21, for any years in which there are three days or less in the first week of January, the WEEKNUM function returns week numbers that are different from the ISO 8601 definition.
- For **return_type**, except for 21, the following valid values may not be supported by some DirectQuery data sources:

Expand table

return_type	Week begins on	System
1 or omitted	Sunday	1
2	Monday	1
11	Monday	1
12	Tuesday	1
13	Wednesday	1
14	Thursday	1
15	Friday	1
16	Saturday	1
17	Sunday	1
21	Monday	2

Example 1

The following example returns the week number for February 14, 2010. This calculation assumes weeks begin on Monday.

DAX

```
= WEEKNUM("Feb 14, 2010", 2)
```

Example 2

The following example returns the week number of the date stored in the column, **HireDate**, from the table, **Employees**. This calculation assumes weeks begin on Sunday.

DAX

```
= WEEKNUM('Employees'[HireDate])
```

Related content

YEARFRAC function
WEEKDAY function

Feedback

Was this page helpful? Yes No

YEAR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the year of a date as a four digit integer in the range 1900-9999.

Syntax

DAX

```
YEAR(<date>)
```

Parameters

Expand table

Term	Definition
date	A date in **datetime** or text format, containing the year you want to find.

Return value

An integer in the range 1900-9999.

Remarks

- In contrast to Microsoft Excel, which stores dates as serial numbers, DAX uses a **datetime** data type to work with dates and times.
- Dates should be entered by using the DATE function, or as results of other formulas or functions. You can also enter dates in accepted text representations of a date, such as March 3, 2007, or Mar-3-2003.
- Values returned by the YEAR, MONTH, and DAY functions will be Gregorian values regardless of the display format for the supplied date value. For example, if the display format of the supplied date uses the Hijri calendar, the returned values for

the YEAR, MONTH, and DAY functions will be values associated with the equivalent Gregorian date.

- When the date argument is a text representation of the date, the function uses the locale and date time settings of the client computer to understand the text value in order to perform the conversion. Errors may arise if the format of strings is incompatible with the current locale settings. For example, if your locale defines dates to be formatted as month/day/year, and the date is provided as day/month/year, then 25/1/2009 will not be interpreted as January 25th of 2009 but as an invalid date.

Example

The following example returns 2007.

```
DAX
= YEAR("March 2007")
```

Example - Date as result of expression

Description

The following example returns the year for today's date.

```
DAX
= YEAR(TODAY())
```

Related content

Date and time functions
HOUR function
MINUTE function
YEAR function
SECOND function

Feedback

Was this page helpful? Yes No

YEARFRAC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Calculates the fraction of the year represented by the number of whole days between two dates. Use the YEARFRAC worksheet function to identify the proportion of a whole year's benefits or obligations to assign to a specific term.

Syntax

DAX

```
YEARFRAC(<start_date>, <end_date>, <basis>)
```

Parameters

Expand table

Term	Definition
start_date	The start date in **datetime** format.
end_date	The end date in **datetime** format.
basis	(Optional) The type of day count basis to use. All arguments are truncated to integers. Basis - Description 0 - US (NASD) 30/360 (Default value) 1 - Actual/actual 2 - Actual/360 3 - Actual/365 4 - European 30/360

Return value

A decimal number. The internal data type is a signed IEEE 64-bit (8-byte) double-precision floating-point number.

Remarks

- In contrast to Microsoft Excel, which stores dates as serial numbers, DAX uses a **datetime** format to work with dates and times. If you need to view dates as serial numbers, you can use the formatting options in Excel.
- If **start_date** or **end_date** are not valid dates, YEARFRAC returns an error.
- If **basis** < 0 or if **basis** > 4, YEARFRAC returns an error.

Example 1

The following example returns the fraction of a year represented by the difference between the dates in the two columns, `TransactionDate` and `ShippingDate`:

```
DAX

= YEARFRAC(Orders[TransactionDate],Orders[ShippingDate])
```

Example 2

The following example returns the fraction of a year represented by the difference between the dates, January 1 and March 1:

```
DAX

= YEARFRAC("Jan 1 2007","Mar 1 2007")
```

Use four-digit years whenever possible, to avoid getting unexpected results. When the year is truncated, the current year is assumed. When the date is or omitted, the first date of the month is assumed.

The second argument, **basis**, has also been omitted. Therefore, the year fraction is calculated according to the US (NASD) 30/360 standard.

Related content

Date and time functions
WEEKNUM function
YEARFRAC function
WEEKDAY function

Feedback

Was this page helpful? Yes No

Filter functions

Article • 05/19/2025

The filter and value functions in DAX are some of the most complex and powerful, and differ greatly from Excel functions. The lookup functions work by using tables and relationships, like a database. The filtering functions let you manipulate data context to create dynamic calculations.

In this category

Expand table

Function	Description
ALL	Returns all the rows in a table, or all the values in a column, ignoring any filters that might have been applied.
ALLCROSSFILTERED	Clear all filters which are applied to a table.
ALLEXCEPT	Removes all context filters in the table except filters that have been applied to the specified columns.
ALLNOBLANKROW	From the parent table of a relationship, returns all rows but the blank row, or all distinct values of a column but the blank row, and disregards any context filters that might exist.
ALLSELECTED	Removes context filters from columns and rows in the current query, while retaining all other context filters or explicit filters.
CALCULATE	Evaluates an expression in a modified filter context.
CALCULATETABLE	Evaluates a table expression in a modified filter context.
EARLIER	Returns the current value of the specified column in an outer evaluation pass of the mentioned column.
EARLIEST	Returns the current value of the specified column in an outer evaluation pass of the specified column.
FILTER	Returns a table that represents a subset of another table or expression.
FIRST	Used in visual calculations only. Retrieves a value in the visual matrix from the first row of an axis.
INDEX	Returns a row at an absolute position, specified by the position parameter, within the specified partition, sorted by the specified order or on the specified axis.
KEEPFILTERS	Modifies how filters are applied while evaluating a CALCULATE or

Function	Description
	CALCULATETABLE function.
LAST	Used in visual calculations only. Retrieves a value in the visual matrix from the last row of an axis.
LOOKUP	In visual calculation mode only. Look up the value when filters applied.
LOOKUPWITHTOTALS	In visual calculation mode only. Look up the value when filters applied. Filters not specified will not be inferred.
LOOKUPVALUE	Returns the value for the row that meets all criteria specified by search conditions. The function can apply one or more search conditions.
MATCHBY	In window functions, defines the columns that are used to determine how to match data and identify the *current row*.
MOVINGAVERAGE	Returns a moving average calculated along the given axis of the visual matrix.
NEXT	Used in visual calculations only. Retrieves a value in the next row of an axis in the visual matrix.
OFFSET	Returns a single row that is positioned either before or after the *current row* within the same table, by a given offset.
ORDERBY	Defines the columns that determine the sort order within each of a window function's partitions.
PARTITIONBY	Defines the columns that are used to partition a window function's `relation` parameter.
PREVIOUS	Used in visual calculations only. Retrieves a value in the previous row of an axis in the visual matrix.
RANGE	Returns an interval of rows within the given axis, relative to the current row. A shortcut for WINDOW.
RANK	Returns the ranking of a row within the given interval.
REMOVEFILTERS	Clears filters from the specified tables or columns.
ROWNUMBER	Returns the unique ranking of a row within the given interval.
RUNNINGSUM	Returns a running sum calculated along the given axis of the visual matrix.
SELECTEDVALUE	Returns the value when the context for columnName has been filtered down to one distinct value only. Otherwise returns alternateResult.
WINDOW	Returns multiple rows which are positioned within the given interval.

ALL

Article • 10/20/2023

Returns all the rows in a table, or all the values in a column, ignoring any filters that might have been applied. This function is useful for clearing filters and creating calculations on all the rows in a table.

Syntax

DAX

```
ALL( [<table> | <column>[, <column>[, <column>[,…]]]] )
```

Parameters

Term	Definition
table	The table that you want to clear filters on.
column	The column that you want to clear filters on.

The argument to the ALL function must be either a reference to a base table or a reference to a base column. You cannot use table expressions or column expressions with the ALL function.

Return value

The table or column with filters removed.

Remarks

- This function is not used by itself, but serves as an intermediate function that can be used to change the set of results over which some other calculation is performed.
- The normal behavior for DAX expressions containing the ALL() function is that any filters applied will be ignored. However, there are some scenarios where this is not the case because of *auto-exist*, a DAX technology that optimizes filtering in order to reduce the amount of processing required for certain DAX queries. An example

where auto-exist and ALL() provide unexpected results is when filtering on two or more columns of the same table (like when using slicers), and there is a measure on that same table that uses ALL(). In this case, auto-exist will *merge* the multiple filters into one and will only filter on existing combinations of values. Because of this merge, the measure will be calculated on the existing combinations of values and the result will be based on filtered values instead of all values as expected. To learn more about auto-exist and its effect on calculations, see Microsoft MVP Alberto Ferrari's Understanding DAX Auto-Exist article on sql.bi.com.

- The following table describes how you can use the ALL and ALLEXCEPT functions in different scenarios.

Function and usage	Description
ALL()	Removes all filters everywhere. ALL() can only be used to clear filters but not to return a table.
ALL(Table)	Removes all filters from the specified table. In effect, ALL(Table) returns all of the values in the table, removing any filters from the context that otherwise might have been applied. This function is useful when you are working with many levels of grouping, and want to create a calculation that creates a ratio of an aggregated value to the total value. The first example demonstrates this scenario.
ALL (Column[, Column[, ...]])	Removes all filters from the specified columns in the table; all other filters on other columns in the table still apply. All column arguments must come from the same table. The ALL(Column) variant is useful when you want to remove the context filters for one or more specific columns and to keep all other context filters. The second and third examples demonstrate this scenario.
ALLEXCEPT(Table, Column1 [,Column2]...)	Removes all context filters in the table except filters that are applied to the specified columns. This is a convenient shortcut for situations in which you want to remove the filters on many, but not all, columns in a table.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

Calculate ratio of Category Sales to Total Sales

Assume that you want to find the amount of sales for the current cell, in your PivotTable, divided by the total sales for all resellers. To ensure that the denominator is the same

regardless of how the PivotTable user might be filtering or grouping the data, you define a formula that uses ALL to create the correct grand total.

The following table shows the results when a new measure, **All Reseller Sales Ratio**, is created using the formula shown in the code section. To see how this works, add the field, CalendarYear, to the **Row Labels** area of the PivotTable, and add the field, ProductCategoryName, to the **Column Labels** area. Then, drag the measure, **All Reseller Sales Ratio**, to the **Values** area of the Pivot Table. To view the results as percentages, use the formatting features of Excel to apply a percentage number formatting to the cells that contains the measure.

Row Labels	Accessories	Bikes	Clothing	Components	Grand Total
2005	0.02%	9.10%	0.04%	0.75%	9.91%
2006	0.11%	24.71%	0.60%	4.48%	29.90%
2007	0.36%	31.71%	1.07%	6.79%	39.93%
2008	0.20%	16.95%	0.48%	2.63%	20.26%
Grand Total	0.70%	82.47%	2.18%	14.65%	100.00%

Formula

DAX

```
= SUMX(ResellerSales_USD,
ResellerSales_USD[SalesAmount_USD])/SUMX(ALL(ResellerSales_USD),
ResellerSales_USD[SalesAmount_USD])
```

The formula is constructed as follows:

1. The numerator, `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])`, is the sum of the values in ResellerSales_USD[SalesAmount_USD] for the current cell in the PivotTable, with context filters applied on CalendarYear and ProductCategoryName.

2. For the denominator, you start by specifying a table, ResellerSales_USD, and use the ALL function to remove all context filters on the table.

3. You then use the SUMX function to sum the values in the ResellerSales_USD[SalesAmount_USD] column. In other words, you get the sum of ResellerSales_USD[SalesAmount_USD] for all resellers sales.

Example 2

Calculate Ratio of Product Sales to Total Sales Through Current Year

Assume that you want to create a table showing the percentage of sales compared over the years for each product category (ProductCategoryName). To obtain the percentage for each year over each value of ProductCategoryName, you need to divide the sum of sales for that particular year and product category by the sum of sales for the same product category over all years. In other words, you want to keep the filter on ProductCategoryName but remove the filter on the year when calculating the denominator of the percentage.

The following table shows the results when a new measure, **Reseller Sales Year**, is created using the formula shown in the code section. To see how this works, add the field, CalendarYear, to the **Row Labels** area of a PivotTable, and add the field, ProductCategoryName, to the **Column Labels** area. To view the results as percentages, use Excel's formatting features to apply a percentage number format to the cells containing the measure, **Reseller Sales Year**.

Row labels	Accessories	Bikes	Clothing	Components	Grand Total
2005	3.48%	11.03%	1.91%	5.12%	9.91%
2006	16.21%	29.96%	27.29%	30.59%	29.90%
2007	51.62%	38.45%	48.86%	46.36%	39.93%
2008	28.69%	20.56%	21.95%	17.92%	20.26%
Grand Total	100.00%	100.00%	100.00%	100.00%	100.00%

Formula

DAX

```
= SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])/CALCULATE(
SUM( ResellerSales_USD[SalesAmount_USD]), ALL(DateTime[CalendarYear]))
```

The formula is constructed as follows:

1. The numerator, `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])`, is the sum of the values in ResellerSales_USD[SalesAmount_USD] for the current cell in the pivot table, with context filters applied on the columns CalendarYear and ProductCategoryName.

2. For the denominator, you remove the existing filter on CalendarYear by using the ALL(Column) function. This calculates the sum over the remaining rows on the ResellerSales_USD table, after applying the existing context filters from the column labels. The net effect is that for the denominator the sum is calculated over the selected ProductCategoryName (the implied context filter) and for all values in Year.

Example 3

Calculate Contribution of Product Categories to Total Sales Per Year

Assume that you want to create a table that shows the percentage of sales for each product category, on a year-by-year basis. To obtain the percentage for each product category in a particular year, you need to calculate the sum of sales for that particular product category (ProductCategoryName) in year n, and then divide the resulting value by the sum of sales for the year n over all product categories. In other words, you want to keep the filter on year but remove the filter on ProductCategoryName when calculating the denominator of the percentage.

The following table shows the results when a new measure, **Reseller Sales CategoryName**, is created using the formula shown in the code section. To see how this works, add the field, CalendarYear to the **Row Labels** area of the PivotTable, and add the field, ProductCategoryName, to the **Column Labels** area. Then add the new measure to the **Values** area of the PivotTable. To view the results as percentages, use Excel's formatting features to apply a percentage number format to the cells that contain the new measure, **Reseller Sales CategoryName**.

Row Labels	Accessories	Bikes	Clothing	Components	Grand Total
2005	0.25%	91.76%	0.42%	7.57%	100.00%
2006	0.38%	82.64%	1.99%	14.99%	100.00%
2007	0.90%	79.42%	2.67%	17.01%	100.00%
2008	0.99%	83.69%	2.37%	12.96%	100.00%
Grand Total	0.70%	82.47%	2.18%	14.65%	100.00%

Formula

DAX

```
= SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])/CALCULATE(
SUM( ResellerSales_USD[SalesAmount_USD]),
```

```
ALL(ProductCategory[ProductCategoryName]))
```

The formula is constructed as follows:

1. The numerator, `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])`, is the sum of the values in ResellerSales_USD[SalesAmount_USD] for the current cell in the PivotTable, with context filters applied on the fields, CalendarYear and ProductCategoryName.

2. For the denominator, you use the function, ALL(Column), to remove the filter on ProductCategoryName and calculate the sum over the remaining rows on the ResellerSales_USD table, after applying the existing context filters from the row labels. The net effect is that, for the denominator, the sum is calculated over the selected Year (the implied context filter) and for all values of ProductCategoryName.

See also

Filter functions
ALL function
ALLEXCEPT function
FILTER function

Feedback

Was this page helpful? Yes No

ALLCROSSFILTERED

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Clear all filters which are applied to a table.

Syntax

DAX

```
ALLCROSSFILTERED(<table>)
```

Parameters

Expand table

Term	Definition
table	The table that you want to clear filters on.

Return value

N/A. See remarks.

Remarks

- ALLCROSSFILTERED can only be used to clear filters but not to return a table.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
DEFINE
MEASURE FactInternetSales[TotalQuantity1] =
    CALCULATE(SUM(FactInternetSales[OrderQuantity]),
```

```
ALLCROSSFILTERED(FactInternetSales))
MEASURE FactInternetSales[TotalQuantity2] =
    CALCULATE(SUM(FactInternetSales[OrderQuantity]), ALL(FactInternetSales))
EVALUATE
    SUMMARIZECOLUMNS(DimSalesReason[SalesReasonName],
        "TotalQuantity1", [TotalQuantity1],
        "TotalQuantity2", [TotalQuantity2])
    ORDER BY DimSalesReason[SalesReasonName]
```

Returns,

Expand table

DimSalesReason[SalesReasonName]	[TotalQuantity1]	[TotalQuantity2]
Demo Event	60398	
Magazine Advertisement	60398	
Manufacturer	60398	1818
On Promotion	60398	7390
Other	60398	3653
Price	60398	47733
Quality	60398	1551
Review	60398	1640
Sponsorship	60398	
Television Advertisement	60398	730

Note

There is a direct or indirect many-to-many relationship between FactInternetSales table and DimSalesReason table.

Feedback

Was this page helpful? Yes No

ALLEXCEPT

Article • 10/20/2023

Removes all context filters in the table except filters that have been applied to the specified columns.

Syntax

DAX

```
ALLEXCEPT(<table>,<column>[,<column>[,…]])
```

Parameters

Term	Definition
table	The table over which all context filters are removed, except filters on those columns that are specified in subsequent arguments.
column	The column for which context filters must be preserved.

The first argument to the ALLEXCEPT function must be a reference to a base table. All subsequent arguments must be references to base columns. You cannot use table expressions or column expressions with the ALLEXCEPT function.

Return value

A table with all filters removed except for the filters on the specified columns.

Remarks

- This function is not used by itself, but serves as an intermediate function that can be used to change the set of results over which some other calculation is performed.
- ALL and ALLEXCEPT can be used in different scenarios:

Function and usage	Description
ALL(Table)	Removes all filters from the specified table. In effect, ALL(Table) returns all of the values in the table, removing any filters from the context that otherwise might have been applied. This function is useful when you are working with many levels of grouping, and want to create a calculation that creates a ratio of an aggregated value to the total value.
ALL (Column[, Column[, ...]])	Removes all filters from the specified columns in the table; all other filters on other columns in the table still apply. All column arguments must come from the same table. The ALL(Column) variant is useful when you want to remove the context filters for one or more specific columns and to keep all other context filters.
ALLEXCEPT(Table, Column1 [,Column2]...)	Removes all context filters in the table except filters that are applied to the specified columns. This is a convenient shortcut for situations in which you want to remove the filters on many, but not all, columns in a table.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following measure formula sums SalesAmount_USD and uses the ALLEXCEPT function to remove any context filters on the DateTime table except if the filter has been applied to the CalendarYear column.

DAX

```
= CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]), ALLEXCEPT(DateTime,
DateTime[CalendarYear]))
```

Because the formula uses ALLEXCEPT, whenever any column but CalendarYear from the table DateTime is used to slice a visualization, the formula will remove any slicer filters, providing a value equal to the sum of SalesAmount_USD. However, if the column CalendarYear is used to slice the visualization, the results are different. Because CalendarYear is specified as the argument to ALLEXCEPT, when the data is sliced on the year, a filter will be applied on years at the row level

See also

Filter functions
ALL function
FILTER function

Feedback

Was this page helpful? Yes No

ALLNOBLANKROW

Article • 10/20/2023

From the parent table of a relationship, returns all rows but the blank row, or all distinct values of a column but the blank row, and disregards any context filters that might exist.

Syntax

DAX

```
ALLNOBLANKROW( {<table> | <column>[, <column>[, <column>[,…]]]} )
```

Parameters

Term	Definition
table	The table over which all context filters are removed.
column	A column over which all context filters are removed.

Only one parameter must be passed; the parameter is either a table or a column.

Return value

A table, when the passed parameter was a table, or a column of values, when the passed parameter was a column.

Remarks

- The ALLNOBLANKROW function only filters the blank row that a parent table, in a relationship, will show when there are one or more rows in the child table that have non-matching values to the parent column. See the example below for a thorough explanation.
- The following table summarizes the variations of ALL that are provided in DAX, and their differences:

Function and usage	Description
ALL(Column)	Removes all filters from the specified column in the table; all other filters in the table, over other columns, still apply.
ALL(Table)	Removes all filters from the specified table.
ALLEXCEPT(Table,Col1,Col2...)	Overrides all context filters in the table except over the specified columns.
ALLNOBLANK(table|column)	From the parent table of a relationship, returns all rows but the blank row, or all distinct values of a column but the blank row, and disregards any context filters that might exist

For a general description of how the ALL function works, together with step-by-step examples that use ALL(Table) and ALL(Column), see ALL function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the sample data, the ResellerSales_USD table contains one row that has no values and therefore cannot be related to any of the parent tables in the relationships within the workbook. You will use this table in a PivotTable so that you can see the blank row behavior and how to handle counts on unrelated data.

Step 1: Verify the unrelated data

Open the **Power Pivot window**, then select the ResellerSales_USD table. In the ProductKey column, filter for blank values. One row will remain. In that row, all column values should be blank except for SalesOrderLineNumber.

Step 2: Create a PivotTable

Create a new PivotTable, then drag the column, datetime.[Calendar Year], to the Row Labels pane. The following table shows the expected results:

Row Labels
2005
2006
2007
2008

Row Labels
Grand Total

Note the blank label between **2008** and **Grand Total**. This blank label represents the Unknown member, which is a special group that is created to account for any values in the child table that have no matching value in the parent table, in this example the datetime.[Calendar Year] column.

When you see this blank label in the PivotTable, you know that in some of the tables that are related to the column, datetime.[Calendar Year], there are either blank values or non-matching values. The parent table is the one that shows the blank label, but the rows that do not match are in one or more of the child tables.

The rows that get added to this blank label group are either values that do not match any value in the parent table-- for example, a date that does not exist in the datetime table-- or null values, meaning no value for date at all. In this example we have placed a blank value in all columns of the child sales table. Having more values in the parent table than in the children tables does not cause a problem.

Step 3: Count rows using ALL and ALLNOBLANK

Add the following two measures to the datetime table, to count the table rows: **Countrows ALLNOBLANK of datetime**, **Countrows ALL of datetime**. The formulas that you can use to define these measures are:

DAX

```
// Countrows ALLNOBLANK of datetime
= COUNTROWS(ALLNOBLANKROW('DateTime'))

// Countrows ALL of datetime
= COUNTROWS(ALL('DateTime'))

// Countrows ALLNOBLANKROW of ResellerSales_USD
= COUNTROWS(ALLNOBLANKROW('ResellerSales_USD'))

// Countrows ALL of ResellerSales_USD
= COUNTROWS(ALL('ResellerSales_USD'))
```

On a blank PivotTable add datetime.[Calendar Year] column to the row labels, and then add the newly created measures. The results should look like the following table:

Row Labels	Countrows ALLNOBLANK of datetime	Countrows ALL of datetime
2005	1280	1281
2006	1280	1281
2007	1280	1281
2008	1280	1281
	1280	1281
Grand Total	1280	1281

The results show a difference of 1 row in the table rows count. However, if you open the **Power Pivot window** and select the datetime table, you cannot find any blank row in the table because the special blank row mentioned here is the Unknown member.

Step 4: Verify that the count is accurate

In order to prove that the ALLNOBLANKROW does not count any truly blank rows, and only handles the special blank row on the parent table only, add the following two measures to the ResellerSales_USD table: **Countrows ALLNOBLANKROW of ResellerSales_USD**, **Countrows ALL of ResellerSales_USD**.

Create a new PivotTable, and drag the column, datetime.[Calendar Year], to the Row Labels pane. Now add the measures that you just created. The results should look like the following:

Row Labels	Countrows ALLNOBLANKROW of ResellerSales_USD	Countrows ALL of ResellerSales_USD
2005	60856	60856
2006	60856	60856
2007	60856	60856
2008	60856	60856
	60856	60856
Grand Total	60856	60856

Now the two measures have the same results. That is because the ALLNOBLANKROW function does not count truly blank rows in a table, but only handles the blank row that

is a special case generated in a parent table, when one or more of the child tables in the relationship contain non-matching values or blank values.

See also

Filter functions
ALL function
FILTER function

Feedback

Was this page helpful? Yes No

ALLSELECTED

06/06/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns all rows in a table, or all the values in a columns, ignoring any filters that might have been applied inside the query, but keeping filters that come from outside.

The ALLSELECTED function gets the context that represents all rows and columns in the query, while keeping explicit filters and contexts other than row and column filters. This function can be used to obtain visual totals in queries.

Syntax

```
DAX

ALLSELECTED([<tableName> | <columnName>[, <columnName>[, <columnName>[,…]]]] )
```

Parameters

Expand table

Term	Definition
`tableName`	The name of an existing table, using standard DAX syntax. This parameter cannot be an expression. This parameter is optional.
`columnName`	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression. This parameter is optional.

Return value

The context of the query without any column and row filters.

Remarks

- If there is one argument, the argument is either `tableName` or `columnName`. If there is more than one argument, they must be columns from the same table.
- This function is different from ALL() because it retains all filters explicitly set within the query, and it retains all context filters other than row and column filters.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to generate different levels of visual totals in a table report using DAX expressions. In the report two (2) previous filters have been applied to the Reseller Sales data; one on Sales Territory Group = *Europe* and the other on Promotion Type = *Volume Discount*. Once filters have been applied, visual totals can be calculated for the entire report, for All Years, or for All Product Categories. Also, for illustration purposes the grand total for All Reseller Sales is obtained too, removing all filters in the report. Evaluating the following DAX expression results in a table with all the information needed to build a table with Visual Totals.

DAX

```
define
measure 'Reseller Sales'[Reseller Sales Amount]=sum('Reseller Sales'[Sales
Amount])
measure 'Reseller Sales'[Reseller Grand Total]=calculate(sum('Reseller
Sales'[Sales Amount]), ALL('Reseller Sales'))
measure 'Reseller Sales'[Reseller Visual Total]=calculate(sum('Reseller
Sales'[Sales Amount]), ALLSELECTED())
measure 'Reseller Sales'[Reseller Visual Total for All of Calendar
Year]=calculate(sum('Reseller Sales'[Sales Amount]), ALLSELECTED('Date'[Calendar
Year]))
measure 'Reseller Sales'[Reseller Visual Total for All of Product Category
Name]=calculate(sum('Reseller Sales'[Sales Amount]), ALLSELECTED('Product
Category'[Product Category Name]))
evaluate
CalculateTable(
    //CT table expression
    summarize(
//summarize table expression
crossjoin(distinct('Product Category'[Product Category Name]),
distinct('Date'[Calendar Year]))
//First Group by expression
, 'Product Category'[Product Category Name]
//Second Group by expression
, 'Date'[Calendar Year]
//Summary expressions
, "Reseller Sales Amount", [Reseller Sales Amount]
, "Reseller Grand Total", [Reseller Grand Total]
, "Reseller Visual Total", [Reseller Visual Total]
, "Reseller Visual Total for All of Calendar Year", [Reseller Visual Total for All
of Calendar Year]
, "Reseller Visual Total for All of Product Category Name", [Reseller Visual Total
for All of Product Category Name]
)
//CT filters
, 'Sales Territory'[Sales Territory Group]="Europe", 'Promotion'[Promotion
```

```
Type]="Volume Discount"
)
order by [Product Category Name], [Calendar Year]
```

After executing the above expression in SQL Server Management Studio against AdventureWorks DW Tabular Model, you obtain the following results:

Expand table

[Product Category Name]	[Calendar Year]	[Reseller Sales Amount]	[Reseller Grand Total]	[Reseller Visual Total]	[Reseller Visual Total for All of Calendar Year]	[Reseller Visual Total for All of Product Category Name]
Accessories	2000		80450596.9823	877006.7987	38786.018	
Accessories	2001		80450596.9823	877006.7987	38786.018	
Accessories	2002	625.7933	80450596.9823	877006.7987	38786.018	91495.3104
Accessories	2003	26037.3132	80450596.9823	877006.7987	38786.018	572927.0136
Accessories	2004	12122.9115	80450596.9823	877006.7987	38786.018	212584.4747
Accessories	2005		80450596.9823	877006.7987	38786.018	
Accessories	2006		80450596.9823	877006.7987	38786.018	
Bikes	2000		80450596.9823	877006.7987	689287.7939	
Bikes	2001		80450596.9823	877006.7987	689287.7939	
Bikes	2002	73778.938	80450596.9823	877006.7987	689287.7939	91495.3104
Bikes	2003	439771.4136	80450596.9823	877006.7987	689287.7939	572927.0136
Bikes	2004	175737.4423	80450596.9823	877006.7987	689287.7939	212584.4747
Bikes	2005		80450596.9823	877006.7987	689287.7939	
Bikes	2006		80450596.9823	877006.7987	689287.7939	
Clothing	2000		80450596.9823	877006.7987	95090.7757	
Clothing	2001		80450596.9823	877006.7987	95090.7757	
Clothing	2002	12132.4334	80450596.9823	877006.7987	95090.7757	91495.3104
Clothing	2003	58234.2214	80450596.9823	877006.7987	95090.7757	572927.0136

[Product Category Name]	[Calendar Year]	[Reseller Sales Amount]	[Reseller Grand Total]	[Reseller Visual Total]	[Reseller Visual Total for All of Calendar Year]	[Reseller Visual Total for All of Product Category Name]
Clothing	2004	24724.1209	80450596.9823	877006.7987	95090.7757	212584.4747
Clothing	2005		80450596.9823	877006.7987	95090.7757	
Clothing	2006		80450596.9823	877006.7987	95090.7757	
Components	2000		80450596.9823	877006.7987	53842.2111	
Components	2001		80450596.9823	877006.7987	53842.2111	
Components	2002	4958.1457	80450596.9823	877006.7987	53842.2111	91495.3104
Components	2003	48884.0654	80450596.9823	877006.7987	53842.2111	572927.0136
Components	2004		80450596.9823	877006.7987	53842.2111	212584.4747
Components	2005		80450596.9823	877006.7987	53842.2111	
Components	2006		80450596.9823	877006.7987	53842.2111	

The columns in the report are:

Reseller Sales Amount The actual value of Reseller Sales for the year and product category. This value appears in a cell in the center of your report, at the intersection of year and catergory.

Reseller Visual Total for All of Calendar Year The total value for a product category across all years. This value appears at the end of a column or row for a given product category and across all years in the report.

Reseller Visual Total for All of Product Category Name The total value for a year across all product categories. This value appears at the end of a column or row for a given year and across all product categories in the report.

Reseller Visual Total The total value for all years and product categories. This value usually appears in the bottom rightmost corner of the table.

Reseller Grand Total This is the grand total for all reseller sales, before any filter has been applied; you should notice the difference with [Reseller Visual Total]. You do remember that this report includes two (2) filters, one on Product Category Group and the other in Promotion Type.

> **Note**
>
> if you have explicit filters in your expression, those filters are also applied to the expression.

CALCULATE

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Evaluates an expression in a modified filter context.

> **Note**
>
> There's also the CALCULATETABLE function. It performs exactly the same functionality, except it modifies the filter context applied to an expression that returns a *table object*.

Syntax

```
DAX
CALCULATE(<expression>[, <filter1> [, <filter2> [, …]]])
```

Parameters

Expand table

Term	Definition
`expression`	The expression to evaluate.
`filter1`, `filter2`,…	(Optional) Boolean expressions or table expressions that defines filters, or filter modifier functions.

The expression you use as the first parameter works like a measure.

Filters can be:

- Boolean filter expressions
- Table filter expressions
- Filter modification functions

When you use multiple filters, you can evaluate them by using the AND (`&&`) logical operator, which means all conditions must be `TRUE`, or by the OR (`||`) logical operator, which means either condition can be true. For more information, see DAX operators > logical operators.

Boolean filter expressions

A Boolean expression filter is an expression that evaluates to `TRUE` or `FALSE`. It must follow several rules:

- It can reference columns from a single table.
- It can't reference measures.
- It can't use a nested CALCULATE function.
- It can't use functions that scan or return a table unless you pass them as arguments to aggregation functions.
- It can contain an aggregation function that returns a scalar value. For example:

```
DAX

Total sales on the last selected date =
CALCULATE (
    SUM ( Sales[Sales Amount] ),
    'Sales'[OrderDateKey] = MAX ( 'Sales'[OrderDateKey] )
)
```

Table filter expression

A table expression filter applies a table object as a filter. It could be a reference to a model table, but more likely it's a function that returns a table object. You can use the FILTER function to apply complex filter conditions, including those that you can't define by a Boolean filter expression.

Filter modifier functions

Filter modifier functions let you do more than simply add filters. They give you extra control when modifying filter context.

Expand table

Function	Purpose
REMOVEFILTERS	Remove all filters, or filters from one or more columns of a table, or from all columns of a single table.
ALL [1], ALLEXCEPT, ALLNOBLANKROW	Remove filters from one or more columns, or from all columns of a single table.

Function	Purpose
KEEPFILTERS	Add filter without removing existing filters on the same columns.
USERELATIONSHIP	Engage an inactive relationship between related columns, in which case the active relationship automatically becomes inactive.
CROSSFILTER	Modify filter direction (from both to single, or from single to both) or disable a relationship.

[1] The ALL function and its variants behave as both filter modifiers and as functions that return table objects. If your tool supports the REMOVEFILTERS function, use it to remove filters.

Return value

The value that results from the expression.

Remarks

- When you provide filter expressions, the CALCULATE function modifies the filter context to evaluate the expression. For each filter expression, two standard outcomes exist when the filter expression isn't wrapped in the KEEPFILTERS function:
  - If the columns or tables aren't in the filter context, new filters are added to the filter context to evaluate the expression.
  - If the columns or tables are already in the filter context, the existing filters are overwritten by the new filters to evaluate the CALCULATE expression.
- Using the CALCULATE function *without filters* achieves a specific requirement. It transitions row context to filter context. This transition is required when an expression (not a model measure) that summarizes model data needs to be evaluated in row context. This scenario can happen in a calculated column formula or when an expression in an iterator function is evaluated. When you use a model measure in row context, context transition is automatic.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

The following **Sales** table measure definition produces a revenue result, but only for products that have the color blue.

DAX

```
Blue Revenue =
CALCULATE ( SUM ( Sales[Sales Amount] ), 'Product'[Color] = "Blue" )
```

Expand table

Category	Sales Amount	Blue Revenue
Accessories	$1,272,057.89	$165,406.62
Bikes	$94,620,526.21	$8,374,313.88
Clothing	$2,117,613.45	$259,488.37
Components	$11,799,076.66	$803,642.10
Total	**$109,809,274.20**	**$9,602,850.97**

The CALCULATE function evaluates the sum of the **Sales** table **Sales Amount** column in a modified filter context. A new filter is added to the **Product** table **Color** column—or, the filter overwrites any filter that's already applied to the column.

The following **Sales** table measure definition produces a ratio of sales over sales for all sales channels.

DAX

```
Revenue % Total Channel =
DIVIDE (
    SUM ( Sales[Sales Amount] ),
    CALCULATE (
        SUM ( Sales[Sales Amount] ),
        REMOVEFILTERS ( 'Sales Order'[Channel] )
    )
)
```

Expand table

Channel	Sales Amount	Revenue % Total Channel
Internet	$29,358,677.22	26.74%

Channel	Sales Amount	Revenue % Total Channel
Reseller	$80,450,596.98	73.26%
Total	**$109,809,274.20**	**100.00%**

The DIVIDE function divides an expression that sums of the **Sales** table **Sales Amount** column value (in the filter context) by the same expression in a modified filter context. It's the CALCULATE function that modifies the filter context by using the REMOVEFILTERS function, which is a filter modifier function. It removes filters from the **Sales Order** table **Channel** column.

The following **Customer** table calculated column definition classifies customers into a loyalty class. It's a very simple scenario: When the revenue produced by the customer is less than $2500, they're classified as *Low*; otherwise they're *High*.

DAX

```
Customer Segment =
IF (
    CALCULATE (
        SUM ( Sales[Sales Amount] ),
        ALLEXCEPT ( Customer, Customer[CustomerKey] )
    ) < 2500,
    "Low",
    "High"
)
```

Expand table

Customer Segment	Customer Count	Sales Amount
High	4,637	$102,670,663.09
Low	13,848	$7,138,611.11
Total	**18,485**	**$109,809,274.20**

In this example, row context is converted to the filter context. It's known as *context transition*. The ALLEXCEPT function removes filters from all **Customer** table columns except the **CustomerKey** column.

Related content

- Filter context
- Row context

- CALCULATETABLE function
- Filter functions

Last updated on 08/04/2026

CALCULATETABLE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Evaluates a table expression in a modified filter context.

> **Note**
>
> There's also the CALCULATE function. It performs exactly the same functionality, except it modifies the filter context applied to an expression that returns a *scalar value*.

Syntax

DAX

```
CALCULATETABLE(<expression>[, <filter1> [, <filter2> [, …]]])
```

Parameters

Expand table

Term	Definition
expression	The table expression to be evaluated.
filter1, filter2,...	(Optional) Boolean expressions or table expressions that defines filters, or filter modifier functions.

The expression used as the first parameter must be a model table or a function that returns a table.

Filters can be:

- Boolean filter expressions
- Table filter expressions
- Filter modification functions

When there are multiple filters, they're evaluated by using the AND logical operator. That means all conditions must be TRUE at the same time.

Boolean filter expressions

A Boolean expression filter is an expression that evaluates to TRUE or FALSE. There are several rules that they must abide by:

- They can reference only a single column.
- They cannot reference measures.
- They cannot use a nested CALCULATE function.

Beginning with the September 2021 release of Power BI Desktop, the following also apply:

- They cannot use functions that scan or return a table unless they are passed as arguments to aggregation functions.
- They *can* contain an aggregation function that returns a scalar value.

Table filter expression

A table expression filter applies a table object as a filter. It could be a reference to a model table, but more likely it's a function that returns a table object. You can use the FILTER function to apply complex filter conditions, including those that cannot be defined by a Boolean filter expression.

Filter modifier functions

Filter modifier functions allow you to do more than simply add filters. They provide you with additional control when modifying filter context.

Expand table

Function	Purpose
REMOVEFILTERS	Remove all filters, or filters from one or more columns of a table, or from all columns of a single table.
ALL [1], ALLEXCEPT, ALLNOBLANKROW	Remove filters from one or more columns, or from all columns of a single table.
KEEPFILTERS	Add filter without removing existing filters on the same columns.

Function	Purpose
USERELATIONSHIP	Engage an inactive relationship between related columns, in which case the active relationship will automatically become inactive.
CROSSFILTER	Modify filter direction (from both to single, or from single to both) or disable a relationship.

[1] The ALL function and its variants behave as both filter modifiers and as functions that return table objects. If the REMOVEFILTERS function is supported by your tool, it's better to use it to remove filters.

Return value

A table of values.

Remarks

- When filter expressions are provided, the CALCULATETABLE function modifies the filter context to evaluate the expression. For each filter expression, there are two possible standard outcomes when the filter expression is not wrapped in the KEEPFILTERS function:
  - If the columns (or tables) aren't in the filter context, then new filters will be added to the filter context to evaluate the expression.
  - If the columns (or tables) are already in the filter context, the existing filters will be overwritten by the new filters to evaluate the CALCULATETABLE expression.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example uses the CALCULATETABLE function to get the sum of Internet sales for 2006. This value is later used to calculate the ratio of Internet sales compared to all sales for the year 2006.

The following formula:

DAX

```
= SUMX(
    CALCULATETABLE(
        'InternetSales_USD',
```

```
        'DateTime'[CalendarYear] = 2006
    ),
    [SalesAmount_USD]
)
```

It results in the following table:

Expand table

Row Labels	Internet SalesAmount_USD	CalculateTable 2006 Internet Sales	Internet Sales to 2006 ratio
2005	$2,627,031.40	$5,681,440.58	0.46
2006	$5,681,440.58	$5,681,440.58	1.00
2007	$8,705,066.67	$5,681,440.58	1.53
2008	$9,041,288.80	$5,681,440.58	1.59
Grand Total	$26,054,827.45	$5,681,440.58	4.59

Related content

- Filter context
- CALCULATE function (DAX)
- Filter functions (DAX)

Feedback

Was this page helpful? Yes No

EARLIER

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns the current value of the specified column in an outer evaluation pass of the mentioned column.

EARLIER is useful for nested calculations where you want to use a certain value as an input and produce calculations based on that input. In Microsoft Excel, you can do such calculations only within the context of the current row; however, in DAX you can store the value of the input and then make calculation using data from the entire table.

EARLIER is mostly used in the context of calculated columns.

Syntax

DAX

```
EARLIER(<column>, <number>)
```

Parameters

Expand table

Term	Definition
column	A column or expression that resolves to a column.
num	(Optional) A positive number to the outer evaluation pass. The next evaluation level out is represented by 1; two levels out is represented by 2 and so on. When omitted default value is 1.

Return value

The current value of row, from **column**, at **number** of outer evaluation passes.

Exceptions

Description of errors

Remarks

- **EARLIER** succeeds if there is a row context prior to the beginning of the table scan. Otherwise it returns an error.
- The performance of **EARLIER** might be slow because theoretically, it might have to perform a number of operations that is close to the total number of rows (in the column) times the same number (depending on the syntax of the expression). For example if you have 10 rows in the column, approximately a 100 operations could be required; if you have 100 rows then close to 10,000 operations might be performed.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

> **Note**
>
> In practice, the VertiPaq in-memory analytics engine performs optimizations to reduce the actual number of calculations, but you should be cautious when creating formulas that involve recursion.

Example

To illustrate the use of EARLIER, it is necessary to build a scenario that calculates a rank value and then uses that rank value in other calculations.

The following example is based on this simple table, **ProductSubcategory**, which shows the total sales for each ProductSubcategory.

The final table, including the ranking column is shown here.

Expand table

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
18	Bib-Shorts	$156,167.88	18
26	Bike Racks	$220,720.70	14
27	Bike Stands	$35,628.69	30
28	Bottles and Cages	$59,342.43	24
5	Bottom Brackets	$48,643.47	27
6	Brakes	$62,113.16	23
19	Caps	$47,934.54	28
7	Chains	$8,847.08	35

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
29	Cleaners	$16,882.62	32
8	Cranksets	$191,522.09	15
9	Derailleurs	$64,965.33	22
30	Fenders	$41,974.10	29
10	Forks	$74,727.66	21
20	Gloves	$228,353.58	12
4	Handlebars	$163,257.06	17
11	Headsets	$57,659.99	25
31	Helmets	$451,192.31	9
32	Hydration Packs	$96,893.78	20
21	Jerseys	$699,429.78	7
33	Lights		36
34	Locks	$15,059.47	33
1	Mountain Bikes	$34,305,864.29	2
12	Mountain Frames	$4,511,170.68	4
35	Panniers		36
13	Pedals	$140,422.20	19
36	Pumps	$12,695.18	34
2	Road Bikes	$40,551,696.34	1
14	Road Frames	$3,636,398.71	5
15	Saddles	$52,526.47	26
22	Shorts	$385,707.80	10
23	Socks	$28,337.85	31
24	Tights	$189,179.37	16
37	Tires and Tubes	$224,832.81	13
3	Touring Bikes	$13,334,864.18	3
16	Touring Frames	$1,545,344.02	6
25	Vests	$240,990.04	11

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
17	Wheels	$648,240.04	8

Creating a Rank Value

One way to obtain a rank value for a given value in a row is to count the number of rows, in the same table, that have a value larger (or smaller) than the one that is being compared. This technique returns a blank or zero value for the highest value in the table, whereas equal values will have the same rank value and next value (after the equal values) will have a non consecutive rank value. See the sample below.

A new calculated column, **SubCategorySalesRanking**, is created by using the following formula.

DAX

```
= COUNTROWS(FILTER(ProductSubcategory,
EARLIER(ProductSubcategory[TotalSubcategorySales])
<ProductSubcategory[TotalSubcategorySales]))+1
```

The following steps describe the method of calculation in more detail.

1. The **EARLIER** function gets the value of *TotalSubcategorySales* for the current row in the table. In this case, because the process is starting, it is the first row in the table
2. **EARLIER**([*TotalSubcategorySales*]) evaluates to $156,167.88, the current row in the outer loop.
3. The **FILTER** function now returns a table where all rows have a value of *TotalSubcategorySales* larger than $156,167.88 (which is the current value for **EARLIER**).
4. The **COUNTROWS** function counts the rows of the filtered table and assigns that value to the new calculated column in the current row plus 1. Adding 1 is needed to prevent the top ranked value from become a Blank.
5. The calculated column formula moves to the next row and repeats steps 1 to 4. These steps are repeated until the end of the table is reached.

The **EARLIER** function will always get the value of the column prior to the current table operation. If you need to get a value from the loop before that, set the second argument to 2.

Related content

EARLIEST function
Filter functions

Feedback

Was this page helpful? Yes No

EARLIEST

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns the current value of the specified column in an outer evaluation pass of the specified column.

Syntax

```DAX
EARLIEST(<column>)
```

Parameters

Expand table

Term	Definition
column	A reference to a column.

Return value

A column with filters removed.

Remarks

- The EARLIEST function is similar to EARLIER, but lets you specify one additional level of recursion.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The current sample data does not support this scenario.

DAX

```
= EARLIEST(<column>)
```

Related content

EARLIER function
Filter functions

Feedback

Was this page helpful? Yes No

FILTER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table that represents a subset of another table or expression.

Syntax

DAX

```
FILTER(<table>,<filter>)
```

Parameters

Expand table

Term	Definition
table	The table to be filtered. The table can also be an expression that results in a table.
filter	A Boolean expression that is to be evaluated for each row of the table. For example, `[Amount] > 0` or `[Region] = "France"`

Return value

A table containing only the filtered rows.

Remarks

- You can use FILTER to reduce the number of rows in the table that you are working with, and use only specific data in calculations. FILTER is not used independently, but as a function that is embedded in other functions that require a table as an argument.
- For best practices when using FILTER, see Avoid using FILTER as a filter argument.

Use COUNTROWS instead of COUNT in DAX

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example creates a report of Internet sales outside the United States by using a measure that filters out sales in the United States, and then slicing by calendar year and product categories. To create this measure, you filter the table, Internet Sales USD, by using Sales Territory, and then use the filtered table in a SUMX function.

In this example, the expression:

DAX

```
FILTER('InternetSales_USD', RELATED('SalesTerritory'[SalesTerritoryCountry])
<>"United States")
```

Returns a table that is a subset of Internet Sales minus all rows that belong to the United States sales territory. The RELATED function is what links the Territory key in the Internet Sales table to SalesTerritoryCountry in the SalesTerritory table.

The following table demonstrates the proof of concept for the measure, NON USA Internet Sales, the formula for which is provided in the code section below. The table compares all Internet sales with non- USA Internet sales, to show that the filter expression works, by excluding United States sales from the computation.

To re-create this table, add the field, SalesTerritoryCountry, to the **Row Labels** area of a report or PivotTable.

Table 1. Comparing total sales for U.S. vs. all other regions

Expand table

Row Labels	Internet Sales	Non USA Internet Sales
Australia	$4,999,021.84	$4,999,021.84
Canada	$1,343,109.10	$1,343,109.10
France	$2,490,944.57	$2,490,944.57
Germany	$2,775,195.60	$2,775,195.60
United Kingdom	$5,057,076.55	$5,057,076.55

Row Labels	Internet Sales	Non USA Internet Sales
United States	$9,389,479.79	
Grand Total	$26,054,827.45	$16,665,347.67

The final report table shows the results when you create a PivotTable by using the measure, NON USA Internet Sales. Add the field, CalendarYear, to the **Row Labels** area of the PivotTable and add the field, ProductCategoryName, to the **Column Labels** area.

Table 2. Comparing non- U.S. sales by product categories

Expand table

Row Labels	Accessories	Bikes	Clothing	Grand Total
2005		$1,526,481.95		$1,526,481.95
2006		$3,554,744.04		$3,554,744.04
2007	$156,480.18	$5,640,106.05	$70,142.77	$5,866,729.00
2008	$228,159.45	$5,386,558.19	$102,675.04	$5,717,392.68
Grand Total	$384,639.63	$16,107,890.23	$172,817.81	$16,665,347.67

DAX

```
SUMX(FILTER('InternetSales_USD',
RELATED('SalesTerritory'[SalesTerritoryCountry])<>"United States")
     ,'InternetSales_USD'[SalesAmount_USD])
```

Related content

Filter functions
ALL function
ALLEXCEPT function

Feedback

Was this page helpful? Yes No

FIRSTNONBLANK

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns the first value in the column, **column**, filtered by the current context, where the expression is not blank.

Syntax

DAX

```
FIRSTNONBLANK(<column>,<expression>)
```

Parameters

Expand table

Term	Definition
column	A column expression.
expression	An expression evaluated for blanks for each value of **column**.

Return value

A table containing a single column and single row with the computed first value.

Remarks

- The **column** argument can be any of the following:
  - A reference to any column.
  - A table with a single column.

- A Boolean expression that defines a single-column table .
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is typically used to return the first value of a column for which the expression is not blank. For example, you could get the last value for which there were sales of a product.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

LASTNONBLANK function
Statistical functions

Feedback

Was this page helpful? Yes No

FIRSTNONBLANKVALUE

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates an expression filtered by the sorted values of a column and returns the first value of the expression that is not blank.

Syntax

```DAX
FIRSTNONBLANKVALUE(<column>, <expression>)
```

Parameters

Expand table

Term	Definition
`column`	A column or an expression that returns a single-column table.
`expression`	An expression evaluated for each value of `column`.

Return value

The first non-blank value of `expression` corresponding to the sorted values of `column`.

Remarks

- The column argument can be any of the following:
  - A reference to any column.

    - A table with a single column.
- This function is different from FIRSTNONBLANK in that the `column` is added to the filter context for the evaluation of `expression`.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query,

DAX

```
EVALUATE
SUMMARIZECOLUMNS(
  DimProduct[Class],
  "FNBV",
  FIRSTNONBLANKVALUE(
    DimDate[Date],
    SUM(FactInternetSales[SalesAmount])
   )
)
```

Returns,

Expand table

DimProduct[Class]	[FNBV]
L	699.0982
H	13778.24
M	1000.4375
	533.83

Feedback

Was this page helpful? Yes No

INDEX

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a row at an absolute position, specified by the position parameter, within the specified partition, sorted by the specified order. If the current partition can't be deduced to a single partition, multiple rows may be returned.

Syntax

DAX

```
INDEX(<position>[, <relation> or <axis>][, <orderBy>][, <blanks>][,
<partitionBy>][, <matchBy>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`position`	The absolute position (1-based) from which to obtain the data: - `position` is positive: 1 is the first row, 2 is the second row, etc. - `position` is negative: -1 is the last row, -2 is the second last row, etc. When `position` is out of the boundary, or zero, or BLANK(), INDEX will return an empty table. It can be any DAX expression that returns a scalar value.
`relation`	(Optional) A table expression from which the output is returned. If specified, all columns in `partitionBy` must come from it or a related table. If omitted: - `orderBy` must be explicitly specified. - All `orderBy` and `partitionBy` expressions must be fully qualified column names and come from a single table. - Defaults to `ALLSELECTED()` of all columns in `orderBy` and `partitionBy`.
`axis`	(Optional) An axis in the visual shape. Available in visual calculations only, and replaces `relation`.
`orderBy`	(Optional) An ORDERBY() clause containing the expressions that define how each partition is sorted. If omitted: - `relation` must be explicitly specified.

Term	Definition
	- Defaults to ordering by every column in `relation` that is not already specified in `partitionBy`.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting. This parameter is reserved for future use. Currently, the only supported value is `DEFAULT`, where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings.
`partitionBy`	(Optional) A `PARTITIONBY()` clause containing the columns that define how `relation` is partitioned. If omitted, `relation` is treated as a single partition.
`matchBy`	(Optional) A `MATCHBY()` clause containing the columns that define how to match data and identify the current row.
`reset`	(Optional) Available in visual calculations only. Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

A row at an absolute position.

Remarks

Each `partitionBy` and `matchBy` column must have a corresponding outer value to help define the "current partition" on which to operate, with the following behavior:

- If there is exactly one corresponding outer column, its value is used.
- If there is no corresponding outer column:
  - INDEX will first determine all `partitionBy` and `matchBy` columns that have no corresponding outer column.
  - For every combination of existing values for these columns in INDEX's parent context, INDEX is evaluated and a row is returned.
  - INDEX's final output is a union of these rows.
- If there is more than one corresponding outer column, an error is returned.

If `matchBy` is present, INDEX will try to use `matchBy` and `partitionBy` columns to identify the row. If `matchBy` is not present and the columns specified within `orderBy` and `partitionBy` cannot uniquely identify every row in `relation`:

- INDEX will try to find the least number of additional columns required to uniquely identify every row.
- If such columns can be found, INDEX will automatically append these new columns to `orderBy`, and each partition is sorted using this new set of OrderBy columns.
- If such columns cannot be found, an error is returned.

An empty table is returned if:

- The corresponding outer value of a PartitionBy column does not exist within `relation`.
- The `position` value refers to a position that does not exist within the partition.

If INDEX is used within a calculated column defined on the same table as `relation` and `orderBy` is omitted, an error is returned.

`reset` can be used in visual calculations only, and cannot be used in combination with `orderBy` or `partitionBy`. If `reset` is present, `axis` can be specified but `relation` cannot.

Example 1 - calculated column

The following DAX query:

```
DAX

EVALUATE INDEX(1, ALL(DimDate[CalendarYear]))
```

Returns the following table:

Expand table

DimDate[CalendarYear]
2005

Example 2 - calculated column

The following DAX query:

```
DAX
```

```
EVALUATE
SUMMARIZECOLUMNS (
    FactInternetSales[ProductKey],
    DimDate[MonthNumberOfYear],
    FILTER (
            VALUES(FactInternetSales[ProductKey]),
            [ProductKey] < 222
    ),
    "CurrentSales", SUM(FactInternetSales[SalesAmount]),
    "LastMonthSales",
    CALCULATE (
        SUM(FactInternetSales[SalesAmount]),
        INDEX(-1, ORDERBY(DimDate[MonthNumberOfYear]))
    )
)
ORDER BY [ProductKey], [MonthNumberOfYear]
```

Returns the following table:

Expand table

FactInternetSales[ProductKey]	DimDate[MonthNumberOfYear]	[CurrentSales]	[LastMonthSales]
214	1	5423.45	8047.7
214	2	4968.58	8047.7
214	3	5598.4	8047.7
214	4	5073.55	8047.7
214	5	5248.5	8047.7
214	6	7487.86	8047.7
214	7	7382.89	8047.7
214	8	6543.13	8047.7
214	9	6788.06	8047.7
214	10	6858.04	8047.7
214	11	8607.54	8047.7
214	12	8047.7	8047.7
217	1	5353.47	7767.78
217	2	4268.78	7767.78
217	3	5773.35	7767.78

FactInternetSales[ProductKey]	DimDate[MonthNumberOfYear]	[CurrentSales]	[LastMonthSales]
217	4	5738.36	7767.78
217	5	6158.24	7767.78
217	6	6998	7767.78
217	7	5563.41	7767.78
217	8	5913.31	7767.78
217	9	5913.31	7767.78
217	10	6823.05	7767.78
217	11	6683.09	7767.78
217	12	7767.78	7767.78

Example 3 - visual calculation

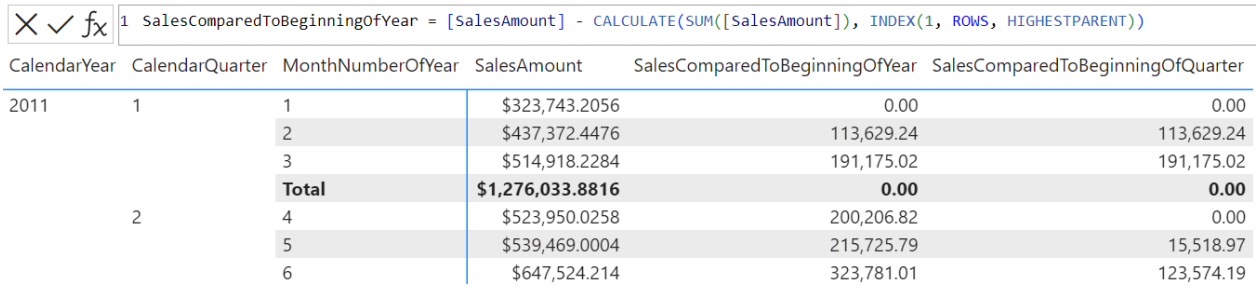
The following visual calculation DAX queries:

DAX

```
SalesComparedToBeginningOfYear = [SalesAmount] -
CALCULATE(SUM([SalesAmount]), INDEX(1, ROWS, HIGHESTPARENT))

SalesComparedToBeginningOfQuarter = [SalesAmount] -
CALCULATE(SUM([SalesAmount]), INDEX(1, , -1))
```

Enhance a table so it contains, for each month:
- the total sales amount;
- the difference to the first month of the respective year;
- and the difference to the first month of the respective quarter.

The screenshot below shows the visual matrix and the first visual calculation expression:



CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesComparedToBeginningOfYear	SalesComparedToBeginningOfQuarter
2011	1	1	$323,743.2056	0.00	0.00
		2	$437,372.4476	113,629.24	113,629.24
		3	$514,918.2284	191,175.02	191,175.02
		Total	**$1,276,033.8816**	**0.00**	**0.00**
	2	4	$523,950.0258	200,206.82	0.00
		5	$539,469.0004	215,725.79	15,518.97
		6	$647,524.214	323,781.01	123,574.19

Related content

OFFSET ORDERBY PARTITIONBY WINDOW RANK ROWNUMBER

Feedback

Was this page helpful? Yes No

KEEPFILTERS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Modifies how filters are applied while evaluating a CALCULATE or CALCULATETABLE function.

Syntax

DAX

```
KEEPFILTERS(<expression>)
```

Parameters

Expand table

Term	Definition
expression	Any expression.

Return value

A table of values.

Remarks

- You use KEEPFILTERS within the context CALCULATE and CALCULATETABLE functions, to override the standard behavior of those functions.
- By default, filter arguments in functions such as CALCULATE are used as the context for evaluating the expression, and as such filter arguments for CALCULATE replace all existing filters over the same columns. The new context effected by the filter argument for CALCULATE affects only existing filters on columns mentioned as part of the filter argument. Filters on columns other than those mentioned in the arguments of CALCULATE or other related functions remain in effect and unaltered.

- The KEEPFILTERS function allows you to modify this behavior. When you use KEEPFILTERS, any existing filters in the current context are compared with the columns in the filter arguments, and the intersection of those arguments is used as the context for evaluating the expression. The net effect over any one column is that both sets of arguments apply: both the filter arguments used in CALCULATE and the filters in the arguments of the KEEPFILTER function. In other words, whereas CALCULATE filters replace the current context, KEEPFILTERS adds filters to the current context.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example takes you through some common scenarios that demonstrate use of the KEEPFILTERS function as part of a CALCULATE or CALCULATETABLE formula.

The first three expressions obtain simple data to be used for comparisons:

- Internet Sales for the state of Washington.

- Internet Sales for the states of Washington and Oregon (both states combined).

- Internet Sales for the state of Washington and the province of British Columbia (both regions combined).

The fourth expression calculates Internet Sales for Washington and Oregon, while the filter for Washington and British Columbia is applied.

The next expression calculates Internet Sales for Washington and Oregon but uses KEEPFILTERS; the filter for Washington and British Columbia is part of the prior context.

DAX

```
EVALUATE ROW(
  "$$ in WA"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
      )
, "$$ in WA and OR"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
                  || 'Geography'[State Province Code]="OR"
      )
, "$$ in WA and BC"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
```

```
                || 'Geography'[State Province Code]="BC"
    )
, "$$ in WA and OR ??"
    , CALCULATE(
        CALCULATE('Internet Sales'[Internet Total Sales]
                 ,'Geography'[State Province Code]="WA"
                   || 'Geography'[State Province Code]="OR"
        )
        , 'Geography'[State Province Code]="WA"
          || 'Geography'[State Province Code]="BC"
    )
, "$$ in WA !!"
    , CALCULATE(
        CALCULATE('Internet Sales'[Internet Total Sales]
                , KEEPFILTERS('Geography'[State Province Code]="WA"
                           || 'Geography'[State Province Code]="OR"
                  )
        )
        , 'Geography'[State Province Code]="WA"
          || 'Geography'[State Province Code]="BC"
    )
)
```

When this expression is evaluated against the sample database AdventureWorks DW, the following results are obtained.

Expand table

Column	Value
[$$ in WA]	$ 2,467,248.34
[$$ in WA and OR]	$ 3,638,239.88
[$$ in WA and BC]	$ 4,422,588.44
[$$ in WA and OR ??]	$ 3,638,239.88
[$$ in WA !!]	$ 2,467,248.34

> **Note**
>
> The above results were formatted to a table, instead of a single row, for educational purposes.

First, examine the expression, **[$$ in WA and OR ??]**. You might wonder how this formula could return the value for sales in Washington and Oregon, since the outer CALCULATE expression includes a filter for Washington and British Columbia. The answer

is that the default behavior of CALCULATE overrides the outer filters in 'Geography'[State Province Code] and substitutes its own filter arguments, because the filters apply to the same column.

Next, examine the expression, **[$$ in WA !!]**. You might wonder how this formula could return the value for sales in Washington and nothing else, since the argument filter includes Oregon and the outer CALCULATE expression includes a filter in Washington and British Columbia. The answer is that KEEPFILTERS modifies the default behavior of CALCULATE and adds an additional filter. Because the intersection of filters is used, now the outer filter **'Geography'[State Province Code]="WA" || 'Geography'[State Province Code]="BC")** is added to the filter argument **'Geography'[State Province Code]="WA" || 'Geography'[State Province Code]="OR"**,. Because both filters apply to the same column, the resulting filter **'Geography'[State Province Code]="WA"** is the filter that is applied when evaluating the expression.

Related content

Filter functions
CALCULATE function
CALCULATETABLE function

Feedback

Was this page helpful? Yes No

LASTNONBLANK

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns the last value in the column, **column**, filtered by the current context, where the expression is not blank.

Syntax

```DAX
LASTNONBLANK(<column>,<expression>)
```

Parameters

Expand table

Term	Definition
column	A column expression.
expression	An expression evaluated for blanks for each value of **column**.

Return value

A table containing a single column and single row with the computed last value.

Remarks

- The **column** argument can be any of the following:
  - A reference to any column.
  - A table with a single column.

    - A Boolean expression that defines a single-column table
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is typically used to return the last value of a column for which the expression is not blank. For example, you could get the last value for which there were sales of a product.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

FIRSTNONBLANK function
Statistical functions

Feedback

Was this page helpful? Yes No

LASTNONBLANKVALUE

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates an expression filtered by the sorted values of a column and returns the last value of the expression that is not blank.

Syntax

```DAX
LASTNONBLANKVALUE(<column>, <expression>)
```

Parameters

Expand table

Term	Definition
`column`	A column or an expression that returns a single-column table.
`expression`	An expression evaluated for each value of `column`.

Return value

The last non-blank value of `expression` corresponding to the sorted values of `column`.

Remarks

- The column argument can be any of the following:
  - A reference to any column.

- A table with a single column.

- This function is different from LASTNONBLANK in that the `column` is added to the filter context for the evaluation of `expression`.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query,

DAX

```
EVALUATE
SUMMARIZECOLUMNS(
  DimProduct[Class],
  "LNBV",
  LASTNONBLANKVALUE(
    DimDate[Date],
    SUM(FactInternetSales[SalesAmount])
   )
)
```

Returns,

Expand table

DimProduct[Class]	[LNBV]
L	132.44
H	137.6
M	84.97
	2288.6

Feedback

Was this page helpful? Yes No

LOOKUP

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ✓ Visual calculation

Returns the value or evaluates the expression from the visual matrix using absolute navigation. Filters can be provided for any axis on the visual matrix. Any omitted filters are inferred from the context. If no single value can be determined, an error is returned.

Syntax

DAX

```
LOOKUP(<expression>, <colref>, <expression>[, <colref>, <expression>]...[,
<AssociatedColumnsBehavior>])
```

Parameters

Expand table

Term	Definition
`expression`	The expression to evaluate.
`colref`	(Optional) The column to be filtered.
`expression`	(Optional) The value to filter.
`AssociatedColumnsBehavior`	(Optional) Specifies how associated columns are handled. Allowed values are `EXPLICIT` (default) or `INFERRED`.

Return value

The value of **expression** after filters are applied.

If there isn't a match, an error is returned.

If multiple rows match the filters, an error is returned.

Remarks

The **AssociatedColumnsBehavior** parameter controls how LOOKUP handles axis columns that are functionally determined by the supplied coordinates:

- **EXPLICIT** (default): When the parameter is omitted or set to `EXPLICIT`, LOOKUP uses only the filters you explicitly provide. Axis columns not specified are inferred from the current context. This is the original LOOKUP behavior.
- **INFERRED**: When set to `INFERRED`, the engine identifies which axis columns are functionally determined by the supplied coordinates and fills them in automatically before evaluating the lookup. For example, if you filter by Year, Quarter, and Month, the engine can infer related columns such as the day or week that correspond to those coordinates.

When `INFERRED` is specified and you supply a value for a column that would have been inferred, the supplied value is validated against the inferred value. If they conflict, the call returns BLANK, similar to how `EXPLICIT` returns BLANK when you supply conflicting coordinates.

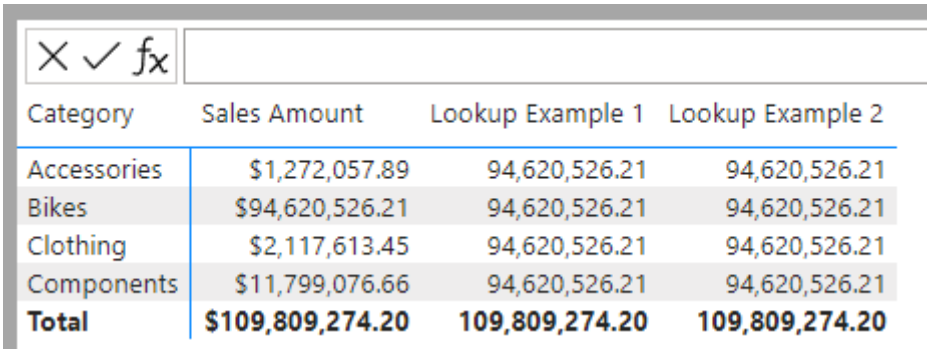
Example 1

In this example, LOOKUP retrieves the sum of sales for Bikes Category. The first argument could be a column or a scalar expression.

DAX

```
Lookup Example 1 = LOOKUP(SUM([Sales Amount]),  [Category], "Bikes")
Lookup Example 2 = LOOKUP([Sales Amount], [Category], "Bikes")
```

The screenshot below shows the matrix with two visual calculations.



Category	Sales Amount	Lookup Example 1	Lookup Example 2
Accessories	$1,272,057.89	94,620,526.21	94,620,526.21
Bikes	$94,620,526.21	94,620,526.21	94,620,526.21
Clothing	$2,117,613.45	94,620,526.21	94,620,526.21
Components	$11,799,076.66	94,620,526.21	94,620,526.21
Total	**$109,809,274.20**	**109,809,274.20**	**109,809,274.20**

Example 2

In this example, the **LookupExample2** visual calculation retrieves the sum of Sales for Fiscal Year **FY2018**. Notice that quarter filter from the row will be used since its not specified explicitly, resulting in no results being returned for **FY2019 Q1** to **FY2019 Q4** since the combination of

FY2018 and **FY2019 Q1** to **FY2019 Q4** does not exist. **LookupExample3** explicitly specifies the quarter so does not take into account the current quarter on the row.

DAX

```
LookupExample2 = LOOKUP([Sales Amount], [Fiscal Year], "FY2018")
LookupExample3 = LOOKUP([Sales Amount], [Fiscal Year], "FY2018", [Fiscal Quarter],
"FY2018 Q1")
```

Fiscal Year	Sales Amount	LookupExample2	LookupExample3
FY2018	**$23,860,891.17**	**23,860,891.17**	**6,005,207.32**
FY2018 Q1	$6,005,207.32	6,005,207.32	6,005,207.32
FY2018 Q2	$5,923,348.19	5,923,348.19	6,005,207.32
FY2018 Q3	$5,965,011.12	5,965,011.12	6,005,207.32
FY2018 Q4	$5,967,324.53	5,967,324.53	6,005,207.32
FY2019	**$34,070,108.50**	**23,860,891.17**	**6,005,207.32**
FY2019 Q1	$10,192,098.13		6,005,207.32
FY2019 Q2	$8,392,458.01		6,005,207.32
FY2019 Q3	$6,769,487.90		6,005,207.32
FY2019 Q4	$8,716,064.45		6,005,207.32
FY2020	**$51,878,274.54**	**23,860,891.17**	**6,005,207.32**
Total	**$109,809,274.20**	**23,860,891.17**	**6,005,207.32**

Example 3

This example demonstrates the difference between `EXPLICIT` and `INFERRED` behavior. Both visual calculations look up the sum of sales for January 2025 by specifying Year, Quarter, and Month Name coordinates.

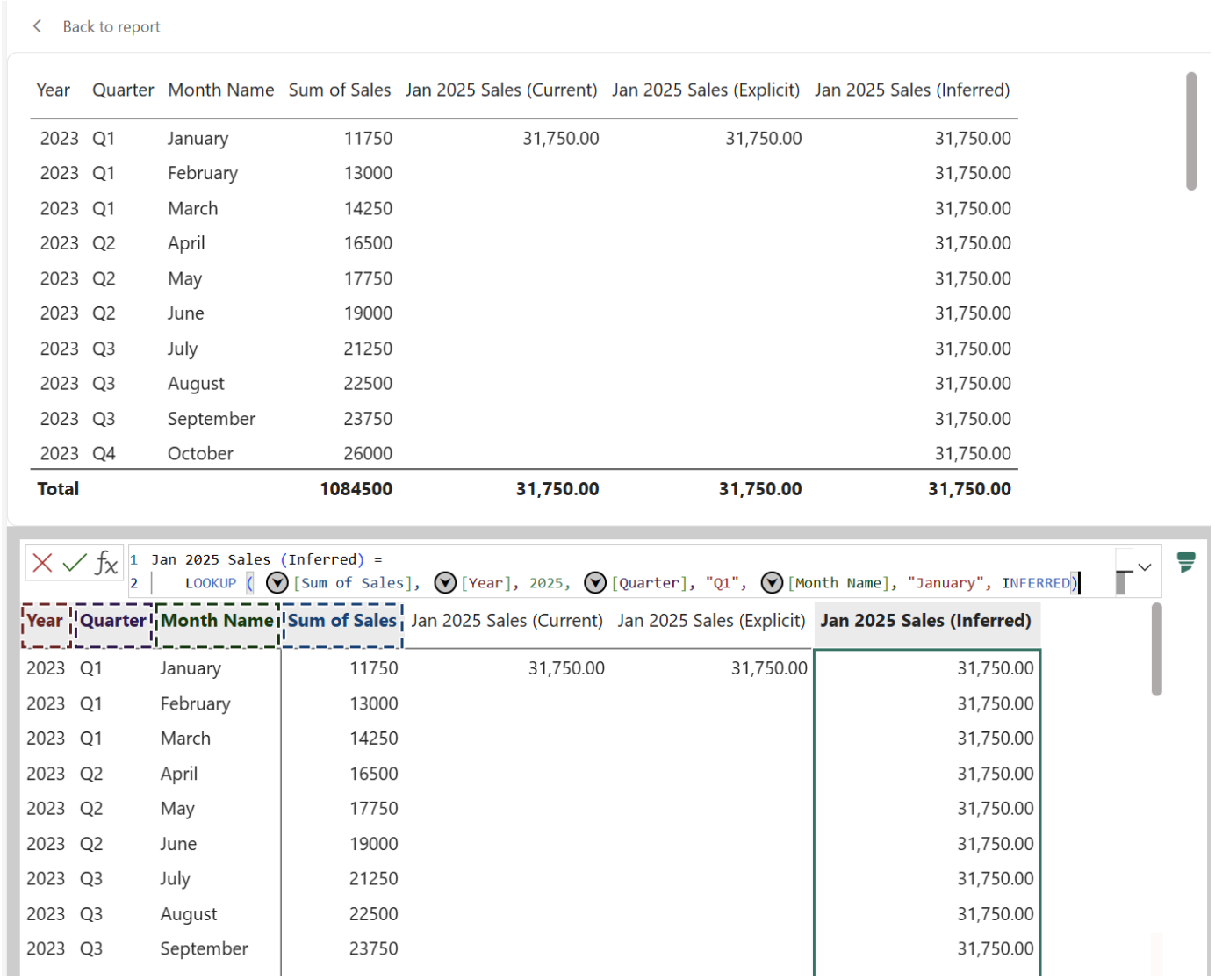
DAX

```
Jan 2025 Sales (Explicit) = LOOKUP([Sum of Sales], [Year], 2025, [Quarter], "Q1",
[Month Name], "January", EXPLICIT)
Jan 2025 Sales (Inferred) = LOOKUP([Sum of Sales], [Year], 2025, [Quarter], "Q1",
[Month Name], "January", INFERRED)
```

With `EXPLICIT`, the lookup returns a value only on the row where the current context matches the specified coordinates (the January 2025 row). On all other rows, the lookup returns BLANK

because the current row's context conflicts with the specified filters.

With `INFERRED`, the engine automatically identifies associated columns that are functionally determined by the Year, Quarter, and Month Name coordinates. Because these coordinates uniquely identify a point in time, the lookup returns the value across all rows in the matrix.



Related content

- LOOKUPWITHTOTALS

Last updated on 08/28/2026

LOOKUPVALUE

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns the value for the row that meets all criteria specified by one or more search conditions.

Syntax

DAX

```
LOOKUPVALUE (
    <result_columnName>,
    <search_columnName>,
    <search_value>
    [, <search2_columnName>, <search2_value>]…
    [, <alternateResult>]
)
```

Parameters

Expand table

Term	Definition
`result_columnName`	The name of an existing column that contains the value you want to return. It can't be an expression.
`search_columnName`	The name of an existing column. It can be in the same table as result_columnName or in a related table. It can't be an expression. You can specify multiple pairs of search_columnName and search_value.
`search_value`	The value to search for in search_columnName. You can specify multiple pairs of search_columnName and search_value.
`alternateResult`	(Optional) The value returned when the context for result_columnName is filtered down to zero or more than one distinct value. If not specified, the function returns BLANK when result_columnName is filtered down to zero values, or an error when the context for result_columnName has more than one distinct value.

Return value

The value of `result_columnName` at the row where all pairs of `search_columnName` and `search_value` have an exact match.

If no match satisfies all the search values, the function returns BLANK or `alternateResult` (if specified). In other words, the function doesn't return a lookup value if only some of the criteria match.

If multiple rows match the search values and the values in the `result_columnName` for these rows are identical, that value is returned. However, if `result_columnName` returns different values, the function returns an error or `alternateResult` (if specified).

Remarks

- If a relationship exists between the table that contains the result column and tables that contain the search columns, in most cases, using the RELATED function instead of LOOKUPVALUE is more efficient and gives better performance.
- You can specify multiple pairs of `search_columnName` and `search_value`.
- The `search_value` and `alternateResult` parameters are evaluated before the function iterates through the rows of the search table.
- Avoid using ISERROR or IFERROR to capture an error returned by LOOKUPVALUE. If some inputs to the function result in an error when a single output value can't be determined, providing an `alternateResult` parameter is the most reliable and highest-performing way to handle the error.
- The `alternateResult` parameter returns an error if specified in a Power Pivot calculated column.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

Example 1

In this example, LOOKUPVALUE searches Average Rate for the currency used to pay for the order on the day the order was placed:

DAX

```
Exchange Rate =
LOOKUPVALUE (
```

```
    'Currency Rate'[Average Rate],
    'Currency Rate'[CurrencyKey], [CurrencyKey],
    'Currency Rate'[DateKey], [OrderDateKey]
)
```

You need both the Order Date and Currency to find the Average Rate for the correct date and currency. OrderDateKey and CurrencyKey are the keys used to look up the Average Rate in the Currency Rate table.

You can use the Exchange Rate to calculate the Sales Amount in local currency with:

DAX

```
Sales Amount Local Currency =
[Sales Amount] * [Exchange Rate]
```

Example 2

In this example, the following calculated column defined in the **Sales** table uses the LOOKUPVALUE function to return channel values from the **Sales Order** table.

DAX

```
CHANNEL =
LOOKUPVALUE (
    'Sales Order'[Channel],
    'Sales Order'[SalesOrderLineKey], [SalesOrderLineKey]
)
```

However, in this case, because a relationship exists between the **Sales Order** and **Sales** tables, it's more efficient to use the RELATED function.

DAX

```
CHANNEL =
RELATED ( 'Sales Order'[Channel] )
```

Related content

- RELATED function (DAX)
- Information functions

Last updated on 08/04/2026

LOOKUPWITHTOTALS

Article • 05/19/2025

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✓ Visual calculation

Returns the value or evaluates the expression from the visual matrix using absolute navigation. Filters can be provided for any axis on the visual matrix. Any omitted filters are treated as referring to the total. If no single value can be determined, an error is returned.

Syntax

DAX

```
LOOKUPWITHTOTALS(<expression>, <colref>, <expression>[, <colref>,
<expression>]...)
```

Parameters

Expand table

Term	Definition
expression	The expression to evaluate
colref	(Optional) The column to be filtered.
expression	(Optional) The value to filter.

Return value

The value of **expression** after filters are applied.

If there isn't a match, an error is returned.

If multiple rows match the filters, an error is returned.

Example 1

The **Lookupwithtotals** visual calculation retrieves values for **FY2018** and all quarters since no value for quarter was specified as an argument. On the other hand, the **Lookup** visual calculation takes into account the current context. Therefore, the results are different and the **Lookup** visual calculation returns the **Sales Amount** for **FY2018** for each quarter. This also

explains why the results for the **Lookup** visual calculation are empty because there is no combination of **FY2019 Q1** to **FY2019 Q4** and **FY2018**.

```
DAX

LOOKUPWITHTOTALExample1 = LOOKUPWITHTOTALS([Sales Amount], [Fiscal Year],
"FY2018")

LookupExample1 = LOOKUP([Sales Amount], [Fiscal Year], "FY2018")
```

The screenshot below shows the matrix with two visual calculations.

Fiscal Year	Sales Amount	LOOKUPWITHTOTALExample1	LookupExample1
⊟ **FY2018**	**$23,860,891.17**	**23,860,891.17**	**23,860,891.17**
FY2018 Q1	$6,005,207.32	23,860,891.17	6,005,207.32
FY2018 Q2	$5,923,348.19	23,860,891.17	5,923,348.19
FY2018 Q3	$5,965,011.12	23,860,891.17	5,965,011.12
FY2018 Q4	$5,967,324.53	23,860,891.17	5,967,324.53
⊟ **FY2019**	**$34,070,108.50**	**23,860,891.17**	**23,860,891.17**
FY2019 Q1	$10,192,098.13	23,860,891.17	
FY2019 Q2	$8,392,458.01	23,860,891.17	
FY2019 Q3	$6,769,487.90	23,860,891.17	
FY2019 Q4	$8,716,064.45	23,860,891.17	
⊞ **FY2020**	**$51,878,274.54**	**23,860,891.17**	**23,860,891.17**
Total	**$109,809,274.20**	**23,860,891.17**	**23,860,891.17**

Example 2

In this example, **LookupWithTotalExample2** retrieves the sum of sales for Bikes Category. Notices that uses the total for other the dimensions since they were not specified. In other words, those dimensions are not filtered. It's useful when we want to do comparison since the result is locked to one value.

```
DAX

LookupWithTotalExample2 = lookupWithTotals([Sales Amount], [Category], "Bikes")
```

Class Category	H nount	LookupWithTotalExample2	L Sales Amount	LookupWithTotalExample2	M Sales Amount	LookupWithTotalExample2	**Total** **Sales Amount**	**LookupWithTotalExample2**
⊟ **Accessories**	**6,830.80**	**94,620,526.21**	**$43,976.94**	**94,620,526.21**	**$57,959.13**	**94,620,526.21**	**$1,272,057.89**	**94,620,526.21**
Bike Racks		94,620,526.21		94,620,526.21		94,620,526.21	**$237,096.16**	**94,620,526.21**
Bike Stands		94,620,526.21		94,620,526.21		94,620,526.21	**$39,591.00**	**94,620,526.21**
Bottles and Cages		94,620,526.21		94,620,526.21		94,620,526.21	**$64,274.79**	**94,620,526.21**
Cleaners		94,620,526.21		94,620,526.21		94,620,526.21	**$18,406.97**	**94,620,526.21**
Fenders		94,620,526.21		94,620,526.21		94,620,526.21	**$46,619.58**	**94,620,526.21**
Helmets		94,620,526.21		94,620,526.21		94,620,526.21	**$484,048.53**	**94,620,526.21**
Hydration Packs		94,620,526.21		94,620,526.21		94,620,526.21	**$105,826.42**	**94,620,526.21**
Locks		94,620,526.21		94,620,526.21		94,620,526.21	**$16,225.22**	**94,620,526.21**
Pumps		94,620,526.21		94,620,526.21		94,620,526.21	**$13,514.69**	**94,620,526.21**
Tires and Tubes	6,830.80	94,620,526.21	$43,976.94	94,620,526.21	$57,959.13	94,620,526.21	**$246,454.53**	**94,620,526.21**
⊟ **Bikes**	**4,475.80**	**94,620,526.21**	**$13,611,841.25**	**94,620,526.21**	**$17,354,209.15**	**94,620,526.21**	**$94,620,526.21**	**94,620,526.21**
Mountain Bikes	6,253.92	94,620,526.21	$1,331,657.52	94,620,526.21	$2,917,532.50	94,620,526.21	**$36,445,443.94**	**94,620,526.21**
Road Bikes	2,419.74	94,620,526.21	$9,832,806.47	94,620,526.21	$12,303,564.79	94,620,526.21	**$43,878,791.00**	**94,620,526.21**
Touring Bikes	5,802.14	94,620,526.21	$2,447,377.26	94,620,526.21	$2,133,111.87	94,620,526.21	**$14,296,291.27**	**94,620,526.21**
⊞ **Clothing**		**94,620,526.21**		**94,620,526.21**		**94,620,526.21**	**$2,117,613.45**	**94,620,526.21**
⊞ **Components**	**7,862.17**	**94,620,526.21**	**$2,285,705.71**	**94,620,526.21**	**$2,502,759.54**	**94,620,526.21**	**$11,799,076.66**	**94,620,526.21**
Total	**9,168.77**	**94,620,526.21**	**$15,941,523.91**	**94,620,526.21**	**$19,914,927.82**	**94,620,526.21**	**$109,809,274.20**	**94,620,526.21**

Related content

Lookup

MATCHBY

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

In window functions, defines the columns that are used to determine how to match data and identify the current row.

Syntax

```DAX
MATCHBY ( [<matchBy_columnName>[, matchBy_columnName [, …]]] )
```

Parameters

Expand table

Term	Definition
`matchBy_columnName`	(Optional) The name of an existing column to be used to identify current row in the window function's `relation`. RELATED() may also be used to refer to a column in a table related to `relation`.

Return value

This function does not return a value.

Remarks

This function can only be used within a window function expression.

Example

See OFFSET.

Related content

Understanding ORDERBY, PARTITIONBY, and MATCHBY functions INDEX OFFSET ORDERBY PARTITIONBY WINDOW RANK ROWNUMBER

Feedback

Was this page helpful? Yes No

MOVINGAVERAGE

07/15/2025

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ✓ Visual calculation

Returns a moving average calculated along the given axis of the visual matrix. That is, the average of the given column calculated over the last `windowSize` rows.

Syntax

DAX

```
MOVINGAVERAGE ( <column>, <windowSize>[, <includeCurrent>][, <axis>][, <orderBy>]
[, <blanks>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`column`	The column that provides the value for each element.
`windowSize`	The number of rows to include in the calculation. Must be a constant value.
`includeCurrent`	(Optional) A logical value specifying whether or not to include the current row in the range. Default value is True.
`axis`	(Optional) An axis reference, the direction along which the moving average will be calculated. If omitted, the first axis of the Visual Shape definition is used.
`orderBy`	(Optional) An ORDERBY() clause with expressions that determine how to sort each partition along the `axis`. If `orderBy` is not provided, the data is sorted by the grouping columns on the default `axis` in ascending order by default.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `axis`. The supported values are: • `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. • `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order.

Term	Definition
	• `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order.
`reset`	(Optional) Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

A scalar value, the moving average at the current element of the axis.

Remarks

This function can be used in visual calculations only.

The `includeCurrent`, `axis`, `orderBy`, `blanks` and `reset` parameters can be omitted.

If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

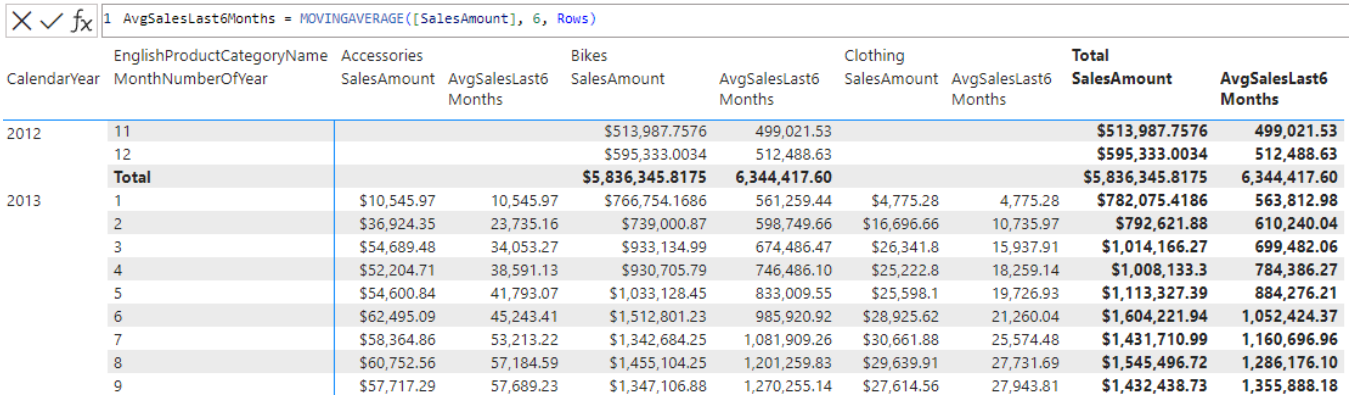
Example 1

Given a table that summarizes the total sales for each product category and calendar month, the following DAX query adds a column with the average of total sales for that category in the last 6 months:

```
DAX
```

```
AvgSalesLast6Months = MOVINGAVERAGE([SalesAmount], 6, Rows)
```

The screenshot below shows the visual matrix and the visual calculation expression:



```
1 AvgSalesLast6Months = MOVINGAVERAGE([SalesAmount], 6, Rows)
```

CalendarYear	EnglishProductCategoryName MonthNumberOfYear	Accessories SalesAmount	 AvgSalesLast6 Months	Bikes SalesAmount	 AvgSalesLast6 Months	Clothing SalesAmount	 AvgSalesLast6 Months	**Total SalesAmount**	** AvgSalesLast6 Months**
2012	11			$513,987.7576	499,021.53			**$513,987.7576**	**499,021.53**
	12			$595,333.0034	512,488.63			**$595,333.0034**	**512,488.63**
	Total			**$5,836,345.8175**	**6,344,417.60**			**$5,836,345.8175**	**6,344,417.60**
2013	1	$10,545.97	10,545.97	$766,754.1686	561,259.44	$4,775.28	4,775.28	**$782,075.4186**	**563,812.98**
	2	$36,924.35	23,735.16	$739,000.87	598,749.66	$16,696.66	10,735.97	**$792,621.88**	**610,240.04**
	3	$54,689.48	34,053.27	$933,134.99	674,486.47	$26,341.8	15,937.91	**$1,014,166.27**	**699,482.06**
	4	$52,204.71	38,591.13	$930,705.79	746,486.10	$25,222.8	18,259.14	**$1,008,133.3**	**784,386.27**
	5	$54,600.84	41,793.07	$1,033,128.45	833,009.55	$25,598.1	19,726.93	**$1,113,327.39**	**884,276.21**
	6	$62,495.09	45,243.41	$1,512,801.23	985,920.92	$28,925.62	21,260.04	**$1,604,221.94**	**1,052,424.37**
	7	$58,364.86	53,213.22	$1,342,684.25	1,081,909.26	$30,661.88	25,574.48	**$1,431,710.99**	**1,160,696.96**
	8	$60,752.56	57,184.59	$1,455,104.25	1,201,259.83	$29,639.91	27,731.69	**$1,545,496.72**	**1,286,176.10**
	9	$57,717.29	57,689.23	$1,347,106.88	1,270,255.14	$27,614.56	27,943.81	**$1,432,438.73**	**1,355,888.18**

Example 2

Given the same table, the following DAX query adds a column with the average of total sales for that category in the previous 12 months (not including the current month):

DAX

```
AvgSalesPrev12Months = MOVINGAVERAGE([SalesAmount], 12, FALSE, Rows, KEEP)
```

See also

INDEX ORDERBY PARTITIONBY RUNNINGSUM WINDOW

OFFSET

Article • 04/24/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a single row that is positioned either before or after the *current row* within the same table, by a given offset. If the current row cannot be deduced to a single row, multiple rows may be returned.

Syntax

DAX

```
OFFSET ( <delta>[, <relation> or <axis>][, <orderBy>][, <blanks>][, <partitionBy>]
[, <matchBy>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`delta`	The number of rows before (negative value) or after (positive value) the current row from which to obtain the data. It can be any DAX expression that returns a scalar value.
`relation`	(Optional) A table expression from which the output row is returned. If used in visual calculations, this parameter accepts an axis in the visual shape. If specified, all columns in `partitionBy` must come from it or a related table. If omitted: - `orderBy` must be explicitly specified. - All `orderBy` and `partitionBy` expressions must be fully qualified column names and come from a single table. - Defaults to ALLSELECTED() of all columns in `orderBy` and `partitionBy`.
`orderBy`	(Optional) An ORDERBY() clause containing the expressions that define how each partition is sorted. If omitted: - `relation` must be explicitly specified. - Defaults to ordering by every column in `relation` that is not already specified in `partitionBy`.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `relation` or `axis`. The supported values are:

Term	Definition
	• `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. • `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order. • `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order. Note, when the `blanks` parameter and blanks in the ORDERBY() function on individual expressions are both specified, `blanks` on individual orderBy expression takes priority for the relevant orderBy expression, and orderBy expressions without `blanks` being specified will honor the `blanks` parameter on the parent function.
`partitionBy`	(Optional) A PARTITIONBY() clause containing the columns that define how `relation` is partitioned. If omitted, `relation` is treated as a single partition.
`matchBy`	(Optional) A MATCHBY() clause containing the columns that define how to match data and identify the current row.
`reset`	(Optional) Available in visual calculations only. Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

One or more rows from `relation`.

Remarks

Except for columns added by DAX table functions, each column in `relation`, when `matchBy` is not present, or each column in `matchBy` and `partitionBy`, when `matchBy` is present, must have a corresponding outer value to help define the current row on which to operate, with the following behavior:

- If there is exactly one corresponding outer column, its value is used.
- If there is no corresponding outer column, then:
  - OFFSET will first determine all columns that have no corresponding outer column.

    - For every combination of existing values for these columns in OFFSET's parent context, OFFSET is evaluated and a row is returned.
    - OFFSET's final output is a union of these rows.
- If there is more than one corresponding outer column, an error is returned.

If all of `relation`'s columns were added by DAX table functions, an error is returned.

If `matchBy` is present, OFFSET will try to use `matchBy` and `partitionBy` columns to identify the row. If `matchBy` is not present and the columns specified within `orderBy` and `partitionBy` can't uniquely identify every row in `relation`, then:

- OFFSET will try to find the least number of additional columns required to uniquely identify every row.
- If such columns can be found, OFFSET will automatically append these new columns to `orderBy`, and each partition is sorted using this new set of OrderBy columns.
- If such columns cannot be found, an error is returned.

An empty table is returned if:

- The corresponding outer value of an OrderBy or PartitionBy column does not exist within `relation`.
- The `delta` value causes a shift to a row that does not exist within the partition.

If OFFSET is used within a calculated column defined on the same table as `relation`, and `orderBy` is omitted, an error is returned.

`reset` can be used in visual calculations only, and cannot be used in combination with `orderBy` or `partitionBy`. If `reset` is present, `axis` can be specified but `relation` cannot.

If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

Example 1 - calculated column

The following DAX query:

```
DAX
```

```
DEFINE
VAR vRelation = SUMMARIZECOLUMNS (
                    DimProductCategory[EnglishProductCategoryName],
                    DimDate[CalendarYear],
```

```
                    "CurrentYearSales", SUM(FactInternetSales[SalesAmount])
                )
EVALUATE
ADDCOLUMNS (
    vRelation,
    "PreviousYearSales",
    SELECTCOLUMNS(
        OFFSET (
                -1,
                vRelation,
                ORDERBY([CalendarYear]),
                PARTITIONBY([EnglishProductCategoryName])
        ),
        [CurrentYearSales]
    )
)
```

Returns a table that summarizes the total sales for each product category and calendar year, as well as the total sales for that category in the previous year.

Example 2 - measure

The following DAX query:

DAX

```
DEFINE
MEASURE DimProduct[CurrentYearSales] = SUM(FactInternetSales[SalesAmount])
MEASURE DimProduct[PreviousYearSales] =
CALCULATE(SUM(FactInternetSales[SalesAmount]), OFFSET(-1, ,
ORDERBY(DimDate[CalendarYear])))
EVALUATE
SUMMARIZECOLUMNS (
    DimDate[CalendarYear],
    "CurrentYearSales", DimProduct[CurrentYearSales],
    "PreviousYearSales", DimProduct[PreviousYearSales]
)
```

Uses OFFSET() in a measure to return a table that summarizes the total sales for each calendar year and the total sales for the previous year.

Example 3 - calculated column

The following DAX query:

DAX

```
EVALUATE
ADDCOLUMNS (
    FactInternetSales,
    "Previous Sales Amount",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                FactInternetSales,
                ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                PARTITIONBY ( FactInternetSales[ProductKey] ),
                MATCHBY( FactInternetSales[SalesOrderNumber],
FactInternetSales[SalesOrderLineNumber] )
            ),
            FactInternetSales[SalesAmount]
        )
)
```

Returns FactInternetSales table with adding a column, which indicates, for each sale, its previous sale's amount, of the same product, in descending order of sales amount, with current sale being identified by its SalesOrderNumber and SalesOrderLineNumber. Without MATCHBY, the query would return an error since there are no key columns in FactInternetSales table.

Example 4 - visual calculation

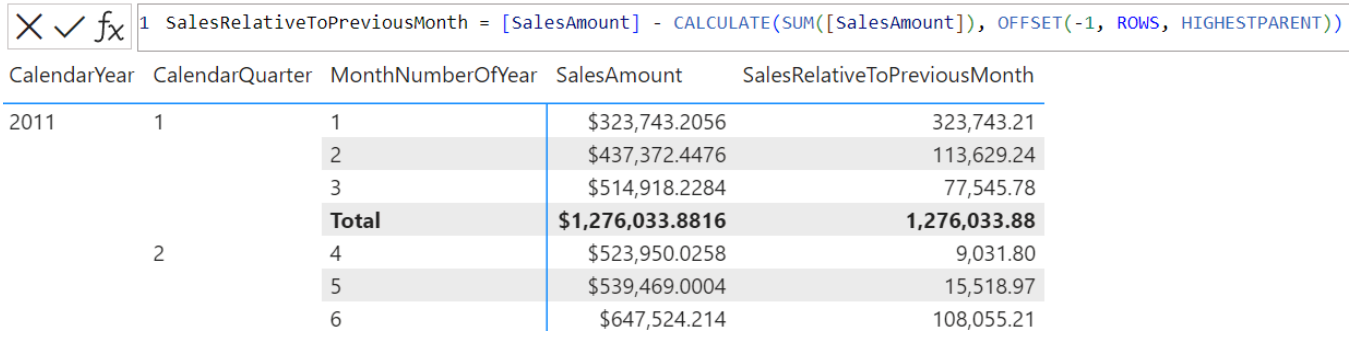
The following visual calculation DAX query:

DAX

```
SalesRelativeToPreviousMonth = [SalesAmount] - CALCULATE(SUM([SalesAmount]),
OFFSET(-1, ROWS, HIGHESTPARENT))
```

Returns the difference in total sales between each month and the previous one within the same year.

The screenshot below shows the visual matrix and the visual calculation expression:



CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRelativeToPreviousMonth
2011	1	1	$323,743.2056	323,743.21
		2	$437,372.4476	113,629.24
		3	$514,918.2284	77,545.78
		Total	**$1,276,033.8816**	**1,276,033.88**
	2	4	$523,950.0258	9,031.80
		5	$539,469.0004	15,518.97
		6	$647,524.214	108,055.21

Related content

INDEX ORDERBY PARTITIONBY MATCHBY WINDOW RANK ROWNUMBER

ORDERBY

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Defines the expressions that determine the sort order within each of a window function's partitions.

Syntax

DAX

```
ORDERBY ( [<orderBy_expression>[, <order>[, <orderBy_expression>[, <order>]] …]] )
```

Parameters

Expand table

Term	Definition
`orderBy_expression`	(Optional) Any scalar expression that will be used used to sort the data within each of a window function's partitions.
`order`	(Optional) A two-part value of the form "`OrderDirection` [`BlankHandling`]". `OrderDirection` specifies how to sort `orderBy_expression` values (i.e. ascending or descending). Valid values include: `DESC`. Alternative value: `0`(zero)/`FALSE`. Sorts in descending order of values of `orderBy_expression`. `ASC`. Alternative value: `1`/`TRUE`. Sorts in ascending order of values of `orderBy_expression`. This is the default value if `order` is omitted. `BlankHandling` part is optional. It specifies how blanks are ordered. Valid values include: `BLANKS DEFAULT`. This is the default value. The behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. `BLANKS FIRST`. Blanks are always ordered on the beginning, regardless of ascending or descending sorting order.

Term	Definition
	`BLANKS LAST`. Blanks are always ordered on the end, regardless of ascending or descending sorting order.

Return value

This function does not return a value.

Remarks

This function can only be used within a window function expression.

Example

See OFFSET.

Related content

Understanding ORDERBY, PARTITIONBY, and MATCHBY functions INDEX OFFSET PARTITIONBY MATCHBY WINDOW RANK ROWNUMBER

Feedback

Was this page helpful? Yes No

PARTITIONBY

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Defines the columns that are used to partition a window function's `relation` parameter.

Syntax

DAX

```
PARTITIONBY ( [<partitionBy_columnName>[, partitionBy_columnName [, …]]] )
```

Parameters

Expand table

Term	Definition
`partitionBy_columnName`	(Optional) The name of an existing column to be used to partition the window function's `relation`. RELATED() may also be used to refer to a column in a table related to `relation`.

Return value

This function does not return a value.

Remarks

This function can only be used within a window function expression.

Example

See OFFSET.

Related content

Understanding ORDERBY, PARTITIONBY, and MATCHBY functions INDEX OFFSET ORDERBY MATCHBY WINDOW RANK ROWNUMBER

Feedback

Was this page helpful? Yes No

RANGE

07/15/2025

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✓ Visual calculation

Returns an interval of rows within the given axis, relative to the current row. This interval will be comprised of either the last `step` rows before the current one, or the first `step` rows after the current one.

Syntax

DAX

```
RANGE ( <step>[, <includeCurrent>][, <axis>][, <orderBy>][, <blanks>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`step`	The number of rows before (negative value) or after (positive value) the current row to include in the range. Must be a constant value. - If negative, the window will contain the last -step rows before the current row. - Otherwise, the window will contain the first step rows after the current row.
`includeCurrent`	(Optional) A logical value specifying whether or not to include the current row in the range. Default value is True.
`axis`	(Optional) An axis reference, the direction along which the interval will be created. If omitted, the first axis of the Visual Shape definition is used.
`orderBy`	(Optional) An ORDERBY() clause with expressions that determine how to sort each partition along the `axis`. If `orderBy` is not provided, the data is sorted by the grouping columns on the default `axis` in ascending order by default.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `axis`. The supported values are: • `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. • `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order.

Term	Definition
	• `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order.
`reset`	(Optional) Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

An interval of data rows.

Remarks

This function can be used in visual calculations only.

The `includeCurrent`, `axis`, `orderBy`, `blanks` and `reset` parameters can be omitted.

If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

Example 1

Given a table that summarizes the total sales for each product category and month, the following DAX query adds a column with the total sales in the last 6 months:

```
DAX
```

```
TotalSalesLast6Months = CALCULATE(SUM([SalesAmount]), RANGE(-5, Rows))
```

The screenshot below shows the visual matrix and the visual calculation expression:

```
1 TotalSalesLast6Months = CALCULATE(SUM([SalesAmount]), RANGE(-5, Rows))
```

CalendarYear	EnglishProductCategoryName MonthNumberOfYear	Accessories SalesAmount	 TotalSalesLast6 Months	Bikes SalesAmount	 TotalSalesLast6 Months	Clothing SalesAmount	 TotalSalesLast6 Months	**Total SalesAmount**	**TotalSalesLast6 Months**
2012	11			$513,987.7576	2,994,129.18			**$513,987.7576**	**2,994,129.18**
	12			$595,333.0034	3,074,931.78			**$595,333.0034**	**3,074,931.78**
	Total			**$5,836,345.8175**	**12,688,835.20**			**$5,836,345.8175**	**12,688,835.20**
2013	1	$10,545.97	10,545.97	$766,754.1686	3,367,556.63	$4,775.28	4,775.28	**$782,075.4186**	**3,382,877.88**
	2	$36,924.35	47,470.32	$739,000.87	3,592,497.96	$16,696.66	21,471.94	**$792,621.88**	**3,661,440.22**
	3	$54,689.48	102,159.80	$933,134.99	4,046,918.83	$26,341.8	47,813.74	**$1,014,166.27**	**4,196,892.37**
	4	$52,204.71	154,364.51	$930,705.79	4,478,916.58	$25,222.8	73,036.54	**$1,008,133.3**	**4,706,317.63**
	5	$54,600.84	208,965.35	$1,033,128.45	4,998,057.27	$25,598.1	98,634.64	**$1,113,327.39**	**5,305,657.26**
	6	$62,495.09	271,460.44	$1,512,801.23	5,915,525.50	$28,925.62	127,560.26	**$1,604,221.94**	**6,314,546.20**
	7	$58,364.86	319,279.33	$1,342,684.25	6,491,455.58	$30,661.88	153,446.86	**$1,431,710.99**	**6,964,181.77**
	8	$60,752.56	343,107.54	$1,455,104.25	7,207,558.96	$29,639.91	166,390.11	**$1,545,496.72**	**7,717,056.61**
	9	$57,717.29	346,135.35	$1,347,106.88	7,621,530.85	$27,614.56	167,662.87	**$1,432,438.73**	**8,135,329.07**

Example 2

Given the same table, the following DAX query adds a column with the total sales in the following 12 months, not including the current month:

DAX

```
TotalSalesFollowingYear = CALCULATE(SUM([SalesAmount]), RANGE(12, FALSE, Rows, KEEP))
```

See also

INDEX ORDERBY PARTITIONBY WINDOW

RANK

Article • 04/24/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the ranking for the current context within the specified partition, sorted by the specified order. If a match cannot be found then then rank is blank.

Syntax

DAX

```
RANK ( [<ties>][, <relation> or <axis>][, <orderBy>][, <blanks>][, <partitionBy>]
[, <matchBy>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`ties`	(Optional) Defines how to handle the ranking when two or more rows are tied. If specified, the supported value is either DENSE, or SKIP. If omitted: - Defaults to SKIP
`relation`	(Optional) A table expression from which the output row is returned. If used in visual calculations, this parameter accepts an axis in the visual shape. If specified, all columns in `orderBy` and `partitionBy` must come from it. If omitted: - `orderBy` must be explicitly specified. - All `orderBy` and `partitionBy` columns must be fully qualified and come from a single table. - Defaults to ALLSELECTED() of all columns in `orderBy` and `partitionBy`.
`orderBy`	(Optional) An ORDERBY() clause containing the columns that define how each partition is sorted. If omitted: - `relation` must be explicitly specified. - Defaults to ordering by every column in `relation` that is not already specified in `partitionBy`.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `relation` or `axis`. The supported values are:

Term	Definition
	• `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. • `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order. • `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order. Note, when the `blanks` parameter and blanks in the ORDERBY() function on individual expressions are both specified, `blanks` on individual orderBy expression takes priority for the relevant orderBy expression, and orderBy expressions without `blanks` being specified will honor the `blanks` parameter on the parent function.
`partitionBy`	(Optional) A PARTITIONBY() clause containing the columns that define how `relation` is partitioned. If omitted, `relation` is treated as a single partition.
`matchBy`	(Optional) A MATCHBY() clause containing the columns that define how to match data and identify the current row.
`reset`	(Optional) Available in visual calculations only. Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

The rank number for the current context.

Remarks

- Each `orderBy`, `partitionBy`, and `matchBy` column must have a corresponding outer value to help define the current row on which to operate, with the following behavior:
  - If there is exactly one corresponding outer column, its value is used.
  - If there is no corresponding outer column, then:
    - RANK will first determine all `orderBy`, `partitionBy`, and `matchBy` columns that have no corresponding outer column.

    - For every combination of existing values for these columns in RANK parent context, RANK is evaluated and a row is returned.
    - RANK's final output is a rank number.

- If `matchBy` is present, then RANK will try to use columns in `matchBy` and `partitionBy` to idenfity the current row.
- If the columns specified within `orderBy` and `partitionBy` cannot uniquely identify every row in `relation`, then two or more rows may have the same ranking and the ranking will be determined by the ties parameter.
- RANK returns a blank value for total rows. It's recommended that you test your expression thoroughly.
- RANK does not compare to RANKX as SUM compares to SUMX.
- `reset` can be used in visual calculations only, and cannot be used in combination with `orderBy` or `partitionBy`. If `reset` is present, `axis` can be specified but `relation` cannot.
- If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

Example 1 - calculated column

The following DAX query:

DAX

```
EVALUATE
ADDCOLUMNS(
    'DimGeography',
    "Rank",
    RANK(
        DENSE,
        'DimGeography',
        ORDERBY(
            'DimGeography'[StateProvinceName], desc,
            'DimGeography'[City], asc),
        LAST,
        PARTITIONBY(
            'DimGeography'[EnglishCountryRegionName])))
ORDER BY [EnglishCountryRegionName] asc, [StateProvinceName] desc, [City] asc
```

Returns a table that ranks each geography with the same EnglishCountryRegionName, by their StateProvinceName and City. Blank `orderBy` column values are sorted on the end.

Example 2 - visual calculation

The following visual calculation DAX queries:

```
DAX

SalesRankWithinYear = RANK(DENSE, ORDERBY([SalesAmount], DESC),
PARTITIONBY([CalendarYear]))

SalesRankAllHistory = RANK(DENSE, ORDERBY([SalesAmount], DESC))
```

Create two columns that rank each month by the total sales, both within each year, and the entire history.

The screenshot below shows the visual matrix and the first visual calculation expression:

CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRankWithinYear	SalesRankAllHistory
2011	1	1	$323,743.2056	12.00	37.00
		2	$437,372.4476	11.00	34.00
		3	$514,918.2284	10.00	25.00
		Total	**$1,276,033.8816**	**4.00**	**11.00**
	2	4	$523,950.0258	9.00	24.00
		5	$539,469.0004	8.00	23.00
		6	$647,524.214	4.00	16.00
		Total	**$1,710,943.2402**	**3.00**	**7.00**
	3	7	$648,343.8132	3.00	15.00
		8	$599,874.2732	7.00	20.00
		9	$609,121.865	6.00	19.00
		Total	**$1,857,339.9514**	**2.00**	**6.00**
	4	10	$655,170.3294	2.00	14.00
		11	$716,748.7178	1.00	13.00
		12	$636,253.2642	5.00	17.00
		Total	**$2,008,172.3114**	**1.00**	**5.00**
	Total		**$6,852,489.3846**	**1.00**	**2.00**

Related content

INDEX ORDERBY PARTITIONBY WINDOW ROWNUMBER

REMOVEFILTERS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Clear filters from the specified tables or columns.

Syntax

```DAX
REMOVEFILTERS([<table> | <column>[, <column>[, <column>[,…]]]])
```

Parameters

Expand table

Term	Definition
table	The table that you want to clear filters on.
column	The column that you want to clear filters on.

Return value

N/A. See remarks.

Remarks

- REMOVEFILTERS can only be used to clear filters but not to return a table.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

DAX query

```DAX
DEFINE
MEASURE FactInternetSales[TotalSales] = SUM(FactInternetSales[SalesAmount])
MEASURE FactInternetSales[%Sales] = DIVIDE([TotalSales],
CALCULATE([TotalSales],REMOVEFILTERS()))
```

```
EVALUATE
    SUMMARIZECOLUMNS(
      ROLLUPADDISSUBTOTAL(DimProductCategory[EnglishProductCategoryName],
"IsGrandTotal"),
      "TotalSales", [TotalSales],
      "%Sales", [%Sales]
    )
ORDER BY
  [IsGrandTotal] DESC, [TotalSales] DESC
```

Returns

Expand table

DimProductCategory[EnglishProductCategoryName]	[IsGrandTotal]	[TotalSales]	[%Sales]
Row1	True	29358677.2207	1
Bikes	False	28318144.6507	0.964557920570538
Accessories	False	700759.96	0.023868921434441
Clothing	False	339772.61	0.0115731579950215

Example 2

DAX query

DAX

```
DEFINE
MEASURE FactInternetSales[TotalSales] = SUM(FactInternetSales[SalesAmount])
MEASURE FactInternetSales[%Sales] = DIVIDE([TotalSales],
CALCULATE([TotalSales],REMOVEFILTERS(DimProductSubcategory[EnglishProductSubcategory
Name])))

EVALUATE
    SUMMARIZECOLUMNS(
      DimProductCategory[EnglishProductCategoryName],
      DimProductSubcategory[EnglishProductSubcategoryName],
      "TotalSales", [TotalSales],
      "%Sales", [%Sales]
    )
ORDER BY
  DimProductCategory[EnglishProductCategoryName] ASC,
  DimProductSubcategory[EnglishProductSubcategoryName] ASC
```

Returns

Expand table

DimProductCategory [EnglishProductCategoryName]	DimProductSubcategory [EnglishProductSubcategoryName]	[TotalSales]	[%Sales]
Accessories	Bike Racks	39360	0.05616759
Accessories	Bike Stands	39591	0.05649723
Accessories	Bottles and Cages	56798.19	0.08105228
Accessories	Cleaners	7218.6	0.0103011
Accessories	Fenders	46619.58	0.06652717
Accessories	Helmets	225335.6	0.3215589
Accessories	Hydration Packs	40307.67	0.05751994
Accessories	Tires and Tubes	245529.32	0.35037578
Bikes	Mountain Bikes	9952759.564	0.35146228
Bikes	Road Bikes	14520584.04	0.51276608
Bikes	Touring Bikes	3844801.05	0.13577164
Clothing	Caps	19688.1	0.05794493
Clothing	Gloves	35020.7	0.10307099
Clothing	Jerseys	172950.68	0.5090189
Clothing	Shorts	71319.81	0.20990453
Clothing	Socks	5106.32	0.01502864
Clothing	Vests	35687	0.10503201

Feedback

Was this page helpful? Yes No

ROWNUMBER

Article • 04/24/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the unique ranking for the current context within the specified partition, sorted by the specified order. If a match cannot be found then rownumber is blank.

Syntax

DAX

```
ROWNUMBER ( [<relation> or <axis>][, <orderBy>][, <blanks>][, <partitionBy>][,
<matchBy>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`relation`	(Optional) A table expression from which the output row is returned. If used in visual calculations, this parameter accepts an axis in the visual shape. If specified, all columns in `orderBy` and `partitionBy` must come from it. If omitted: - `orderBy` must be explicitly specified. - All `orderBy` and `partitionBy` columns must be fully qualified and come from a single table. - Defaults to ALLSELECTED() of all columns in `orderBy` and `partitionBy`.
`orderBy`	(Optional) An ORDERBY() clause containing the columns that define how each partition is sorted. If omitted: - `relation` must be explicitly specified. - Defaults to ordering by every column in `relation` that is not already specified in `partitionBy`.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `relation` or `axis`. The supported values are: • `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings.

Term	Definition
	• `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order. • `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order. Note, when the `blanks` parameter and blanks in the ORDERBY() function on individual expressions are both specified, `blanks` on individual orderBy expression takes priority for the relevant orderBy expression, and orderBy expressions without `blanks` being specified will honor the `blanks` parameter on the parent function.
`partitionBy`	(Optional) A PARTITIONBY() clause containing the columns that define how `relation` is partitioned. If omitted, `relation` is treated as a single partition.
`matchBy`	(Optional) A MATCHBY() clause containing the columns that define how to match data and identify the current row.
`reset`	(Optional) Available in visual calculations only. Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

The rownumber number for the current context.

Remarks

Each `orderBy`, `partitionBy`, and `matchBy` column must have a corresponding outer value to help define the current row on which to operate, with the following behavior:

- If there is exactly one corresponding outer column, its value is used.
- If there is no corresponding outer column, then:
  - ROWNUMBER will first determine all `orderBy`, `partitionBy`, and `matchBy` columns that have no corresponding outer column.
  - For every combination of existing values for these columns in ROWNUMBER parent context, ROWNUMBER is evaluated and a row is returned.
  - ROWNUMBER's final output is a union of these rows.

- If there is more than one corresponding outer column, an error is returned.

If `matchBy` is present, then ROWNUMBER will try to use columns in `matchBy` and `partitionBy` to idenfity the current row. If the columns specified within `orderBy` and `partitionBy` cannot uniquely identify every row in `relation`, then:

- ROWNUMBER will try to find the least number of additional columns required to uniquely identify every row.
- If such columns can be found, ROWNUMBER will
  - Try to find the least number of additional columns required to uniquely identify every row.
  - Automatically append these new columns to `orderBy` clause.
  - Sort each partition using this new set of orderBy columns.
- If such columns cannot be found and the function detects a tie at runtime, an error is returned.

`reset` can be used in visual calculations only, and cannot be used in combination with `orderBy` or `partitionBy`. If `reset` is present, `axis` can be specified but `relation` cannot.

If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

Example 1 - calculated column

The following DAX query:

DAX

```
EVALUATE
ADDCOLUMNS(
    'DimGeography',
    "UniqueRank",
    ROWNUMBER(
        'DimGeography',
        ORDERBY(
            'DimGeography'[StateProvinceName], desc,
            'DimGeography'[City], asc),
        PARTITIONBY(
            'DimGeography'[EnglishCountryRegionName])))
ORDER BY [EnglishCountryRegionName] asc, [StateProvinceName] desc, [City] asc
```

Returns a table that uniquely ranks each geography with the same EnglishCountryRegionName, by their StateProvinceName and City.

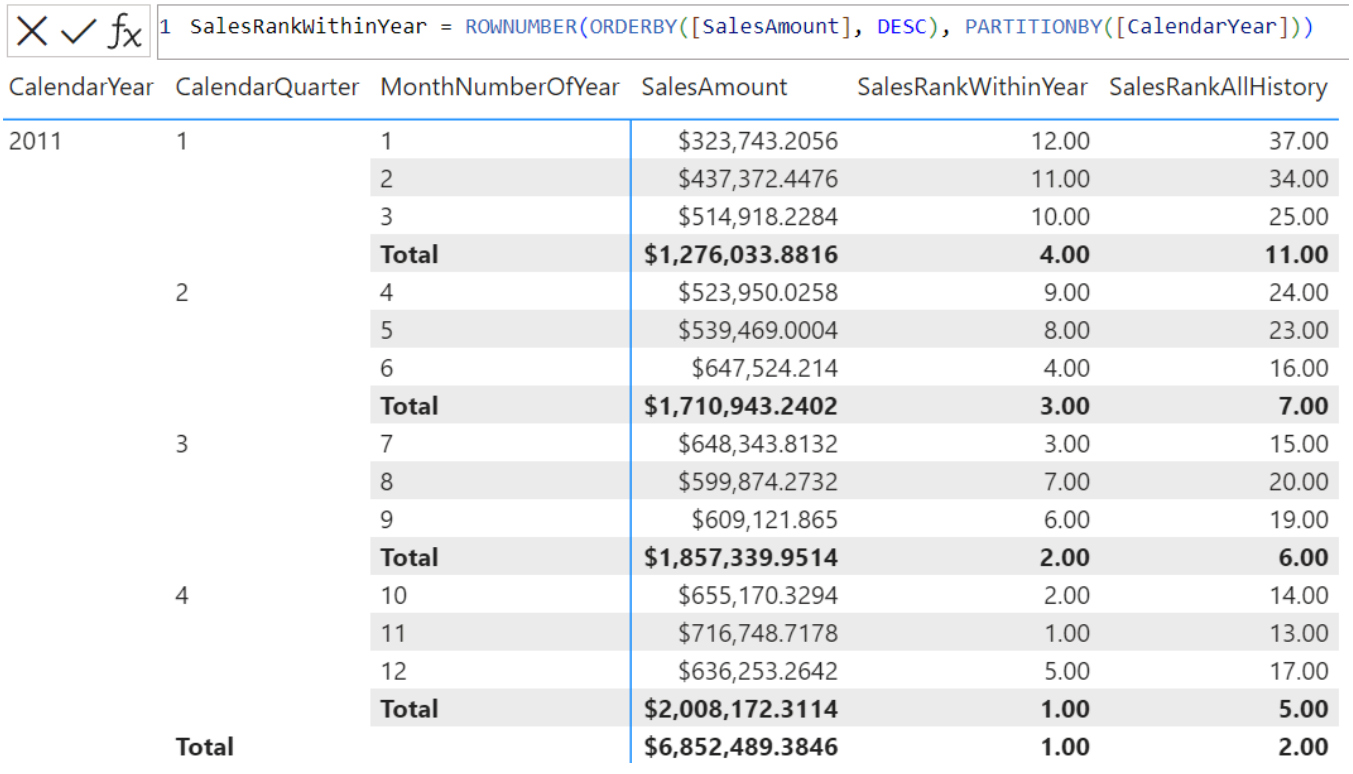
Example 2 - visual calculation

The following visual calculation DAX queries:

```DAX
SalesRankWithinYear = ROWNUMBER(ORDERBY([SalesAmount], DESC),
PARTITIONBY([CalendarYear]))

SalesRankAllHistory = ROWNUMBER(ORDERBY([SalesAmount], DESC))
```

Create two columns that uniquely rank each month by the total sales, both within each year, and the entire history.

The screenshot below shows the visual matrix and the first visual calculation expression:



CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRankWithinYear	SalesRankAllHistory
2011	1	1	$323,743.2056	12.00	37.00
		2	$437,372.4476	11.00	34.00
		3	$514,918.2284	10.00	25.00
		Total	**$1,276,033.8816**	**4.00**	**11.00**
	2	4	$523,950.0258	9.00	24.00
		5	$539,469.0004	8.00	23.00
		6	$647,524.214	4.00	16.00
		Total	**$1,710,943.2402**	**3.00**	**7.00**
	3	7	$648,343.8132	3.00	15.00
		8	$599,874.2732	7.00	20.00
		9	$609,121.865	6.00	19.00
		Total	**$1,857,339.9514**	**2.00**	**6.00**
	4	10	$655,170.3294	2.00	14.00
		11	$716,748.7178	1.00	13.00
		12	$636,253.2642	5.00	17.00
		Total	**$2,008,172.3114**	**1.00**	**5.00**
	Total		**$6,852,489.3846**	**1.00**	**2.00**

Related content

INDEX ORDERBY PARTITIONBY WINDOW RANK

RUNNINGSUM

07/15/2025

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ✓ Visual calculation

Returns a running sum calculated along the given axis of the visual matrix. That is, the sum of the given column calculated over all elements up to the current element of an axis.

Syntax

DAX

```
RUNNINGSUM ( <column>[, <axis>][, <orderBy>][, <blanks>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`column`	The column that provides the value for each element.
`axis`	(Optional) An axis reference, the direction along which the running sum will be calculated. If omitted, the first axis of the Visual Shape definition is used.
`orderBy`	(Optional) An ORDERBY() clause with expressions that determine how to sort each partition along the `axis`. If `orderBy` is not provided, the data is sorted by the grouping columns on the default `axis` in ascending order by default.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `axis`. The supported values are: • `DEFAULT` (the default value), where the behavior for numerical values is blank values are ordered between zero and negative values. The behavior for strings is blank values are ordered before all strings, including empty strings. • `FIRST`, blanks are always ordered on the beginning, regardless of ascending or descending sorting order. • `LAST`, blanks are always ordered on the end, regardless of ascending or descending sorting order.
`reset`	(Optional) Indicates if the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign:

Term	Definition
	- If zero or omitted, the calculation does not reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

A scalar value, the running sum up to the current element of the axis.

Remarks

This function can be used in visual calculations only.

The `axis`, `orderBy`, `blanks` and `reset` parameters can be omitted.

If the value of `reset` is absolute (i.e., a positive integer, `HIGHESTPARENT` or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

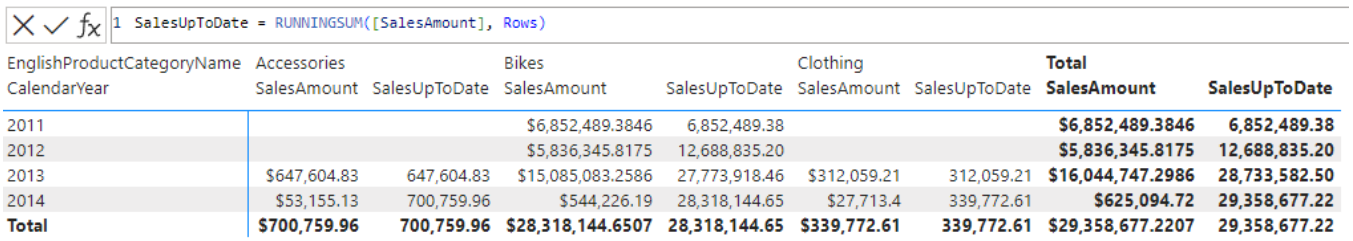
Example

Given a table that summarizes the total sales for each product category and calendar year, the following DAX query adds a column with the total sales for that category up to a given year.

DAX

```
SalesUpToDate = RUNNINGSUM([SalesAmount], Rows)
```

The screenshot below shows the visual matrix and the visual calculation expression:



EnglishProductCategoryName	Accessories		Bikes		Clothing		**Total**	
CalendarYear	SalesAmount	SalesUpToDate	SalesAmount	SalesUpToDate	SalesAmount	SalesUpToDate	**SalesAmount**	**SalesUpToDate**
2011			$6,852,489.3846	6,852,489.38			**$6,852,489.3846**	**6,852,489.38**
2012			$5,836,345.8175	12,688,835.20			**$5,836,345.8175**	**12,688,835.20**
2013	$647,604.83	647,604.83	$15,085,083.2586	27,773,918.46	$312,059.21	312,059.21	**$16,044,747.2986**	**28,733,582.50**
2014	$53,155.13	700,759.96	$544,226.19	28,318,144.65	$27,713.4	339,772.61	**$625,094.72**	**29,358,677.22**
Total	**$700,759.96**	**700,759.96**	**$28,318,144.6507**	**28,318,144.65**	**$339,772.61**	**339,772.61**	$29,358,677.2207	29,358,677.22

See also

INDEX MOVINGAVERAGE ORDERBY PARTITIONBY WINDOW

SELECTEDVALUE

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the value when the context for columnName has been filtered down to one distinct value only. Otherwise returns alternateResult.

Syntax

DAX

```
SELECTEDVALUE(<columnName>[, <alternateResult>])
```

Parameters

Expand table

Term	Definition
`columnName`	The name of an existing column, using standard DAX syntax. It cannot be an expression.
`alternateResult`	(Optional) The value returned when the context for columnName has been filtered down to zero or more than one distinct value. When not provided, the default value is BLANK().

Return value

The value when the context for columnName has been filtered down to one distinct value only. Else, alternateResult.

Remarks

- An equivalent expression for `SELECTEDVALUE(<columnName>, <alternateResult>)` is `IF(HASONEVALUE(<columnName>), VALUES(<columnName>), <alternateResult>)`.
- To learn more about best practices when using SELECTEDVALUE, see Use SELECTEDVALUE instead of VALUES in DAX.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

```DAX
DEFINE
 MEASURE DimProduct[Selected Color] = SELECTEDVALUE(DimProduct[Color], "No Single Selection")
EVALUATE
 SUMMARIZECOLUMNS
   (ROLLUPADDISSUBTOTAL(DimProduct[Color], "Is Total"),
   "Selected Color", [Selected Color])ORDER BY [Is Total] ASC,
   [Color] ASC
```

Returns the following:

Expand table

DimProduct[Color]	[Is Total]	[Selected Color]
Black	FALSE	Black
Blue	FALSE	Blue
Grey	FALSE	Grey
Multi	FALSE	Multi
NA	FALSE	NA
Red	FALSE	Red
Silver	FALSE	Silver
Silver/Black	FALSE	Silver/Black
White	FALSE	White
Yellow	FALSE	Yellow
``	TRUE	No Single Selection

Feedback

Was this page helpful? Yes No

WINDOW

Applies to: Calculated column Calculated table Measure Visual calculation

Returns multiple rows that are positioned within the given interval.

Syntax

DAX

```
WINDOW ( from[, from_type], to[, to_type][, <relation> or <axis>][, <orderBy>][,
<blanks>][, <partitionBy>][, <matchBy>][, <reset>] )
```

Parameters

Expand table

Term	Definition
`from`	Indicates where the window starts. It can be any DAX expression that returns a scalar value. The behavior depends on the `from_type` parameter: - If `from_type` is REL, the number of rows to go back (negative value) or forward (positive value) from the current row to get the first row in the window. - If `from_type` is ABS, and `from` is positive, it's the position of the start of the window from the beginning of the partition. Indexing is 1-based and 0 is interpreted as 1. For example, 0 and 1 mean the window starts from the beginning of the partition. If `from` is negative, it's the position of the start of the window from the end of the partition. -1 means the last row in the partition.
`from_type`	Modifies behavior of the `from` parameter. Possible values are ABS (absolute) and REL (relative). Default is REL.
`to`	Same as `from`, but indicates the end of the window. The last row is included in the window.
`to_type`	Same as `from_type`, but modifies the behavior of `to`.
`relation`	(Optional) A table expression from which the output rows are returned. If used in visual calculations, this parameter accepts an axis in the visual shape. If specified, all columns in `partitionBy` must come from it or a related table. If omitted: - You must specify `orderBy` explicitly. - All `orderBy` and `partitionBy` expressions must be fully qualified column names and come from a single table. - Defaults to ALLSELECTED() of all columns in `orderBy` and `partitionBy`.

Term	Definition
`orderBy`	(Optional) An ORDERBY() clause containing the expressions that define how each partition is sorted. If omitted: - You must specify `relation` explicitly. - Defaults to ordering by every column in `relation` that isn't already specified in `partitionBy`.
`blanks`	(Optional) An enumeration that defines how to handle blank values when sorting the `relation` or `axis`. The supported values are: • `DEFAULT` (the default value), where the behavior for numerical values is that blank values are ordered between zero and negative values. The behavior for strings is that blank values are ordered before all strings, including empty strings. • `FIRST`: blanks are always ordered at the beginning, regardless of ascending or descending sorting order. • `LAST`: blanks are always ordered at the end, regardless of ascending or descending sorting order. When you specify both the `blanks` parameter and blanks in the ORDERBY() function on individual expressions, `blanks` on the individual orderBy expression takes priority for the relevant orderBy expression. OrderBy expressions without `blanks` being specified honor the `blanks` parameter on the parent function.
`partitionBy`	(Optional) A PARTITIONBY() clause containing the columns that define how `relation` is partitioned. If omitted, `relation` is treated as a single partition.
`matchBy`	(Optional) A MATCHBY() clause containing the columns that define how to match data and identify the current row.
`reset`	(Optional) Available in visual calculations only. Indicates whether the calculation resets, and at which level of the visual shape's column hierarchy. Accepted values are: a field reference to a column in the current visual shape, `NONE` (default), `LOWESTPARENT`, `HIGHESTPARENT`, or an integer. The behavior depends on the integer sign: - If zero or omitted, the calculation doesn't reset. Equivalent to `NONE`. - If positive, the integer identifies the column starting from the highest, independent of grain. `HIGHESTPARENT` is equivalent to 1. - If negative, the integer identifies the column starting from the lowest, relative to the current grain. `LOWESTPARENT` is equivalent to -1.

Return value

All rows from the window.

Remarks

Except for columns added by DAX table functions, each column in `relation` (when `matchBy` isn't present), or each column in `matchBy` and `partitionBy` (when `matchBy` is present), must have a corresponding outer value to help define the current row on which to operate. If `from_type` and `to_type` both have value ABS, the following applies only to the `partitionBy` columns:

- If exactly one corresponding outer column exists, its value is used.
- If no corresponding outer column exists:
  - WINDOW first determines all columns that have no corresponding outer column.
  - For every combination of existing values for these columns in WINDOW's parent context, WINDOW is evaluated, and the corresponding rows are returned.
  - WINDOW's final output is a union of these rows.
- If more than one corresponding outer column exists, an error is returned.

If all of `relation`'s columns were added by DAX table functions, an error is returned.

If `matchBy` is present, WINDOW tries to use `matchBy` and `partitionBy` columns to identify the row. If `matchBy` isn't present and the columns specified within `orderBy` and `partitionBy` can't uniquely identify every row in `relation`:

- WINDOW tries to find the least number of additional columns required to uniquely identify every row.
- If such columns can be found, WINDOW automatically appends these new columns to `orderBy`, and each partition is sorted using this new set of orderBy columns.
- If such columns can't be found, an error is returned.

An empty table is returned if:

- The corresponding outer value of an `orderBy` or `partitionBy` column doesn't exist within `relation`.
- The whole window is outside the partition, or the beginning of the window is after its end.

If you use WINDOW within a calculated column defined on the same table as `relation`, and `orderBy` is omitted, an error is returned.

If the beginning of the window is before the first row, it's set to the first row. Similarly, if the end of the window is after the last row of the partition, it's set to the last row.

You can use `reset` in visual calculations only, and you can't use it in combination with `orderBy` or `partitionBy`. If `reset` is present, you can specify `axis` but not `relation`.

If the value of `reset` is absolute (a positive integer, `HIGHESTPARENT`, or a field reference) and the calculation is evaluated at or above the target level in the hierarchy, the calculation resets for each individual element. That is, the function is evaluated within a partition containing only that specific element.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

Example 1 - measure

The following measure:

DAX

```
3-day Average Price =
AVERAGEX (
    WINDOW (
        -2,
        REL,
        0,
        REL,
        SUMMARIZE ( ALLSELECTED ( 'Sales' ), 'Date'[Date], 'Product'[Product] ),
        ORDERBY ( 'Date'[Date] ),
        KEEP,
        PARTITIONBY ( 'Product'[Product] )
    ),
    CALCULATE ( AVERAGE ( Sales[Unit Price] ) )
)
```

Returns the 3-day average of unit prices for each product. The 3-day window consists of three days on which the product had sales, not necessarily three consecutive calendar days.

Example 2 - measure

The following measure:

DAX

```
RunningSum =
SUMX (
    WINDOW (
        1,
        ABS,
        0,
        REL,
        ALLSELECTED ( 'Date'[Fiscal Year], 'Date'[Month Number Of Year] ),
        PARTITIONBY ( 'Date'[Fiscal Year] )
```

```
    ),
    [Total Sales]
)
```

Returns the running sum for Total Sales by Month Number Of Year, restarting for every Fiscal Year:

Expand table

Year	Month Number Of Year	Sales Amount	RunningSum
FY2018	1	$1,327,675	$1,327,675
FY2018	2	$3,936,463	$5,264,138
FY2018	3	$700,873	$5,965,011
FY2018	4	$1,519,275	$7,484,286
FY2018	5	$2,960,378	$10,444,664
FY2018	6	$1,487,671	$11,932,336
FY2018	7	$1,423,357	$13,355,693
FY2018	8	$2,057,902	$15,413,595
FY2018	9	$2,523,948	$17,937,543
FY2018	10	$561,681	$18,499,224
FY2018	11	$4,764,920	$23,264,145
FY2018	12	$596,747	$23,860,891
FY2019	1	$1,847,692	$1,847,692
FY2019	2	$2,829,362	$4,677,054
FY2019	3	$2,092,434	$6,769,488
FY2019	4	$2,405,971	$9,175,459
FY2019	5	$3,459,444	$12,634,903
FY2019	6	$2,850,649	$15,485,552
FY2019	7	$2,939,691	$18,425,243
FY2019	8	$3,964,801	$22,390,045
FY2019	9	$3,287,606	$25,677,650

Year	Month Number Of Year	Sales Amount	RunningSum
FY2019	10	$2,157,287	$27,834,938
FY2019	11	$3,611,092	$31,446,030
FY2019	12	$2,624,078	$34,070,109
FY2020	1	$3,235,187	$3,235,187
FY2020	2	$4,070,046	$7,305,233
FY2020	3	$4,429,833	$11,735,066
FY2020	4	$4,002,614	$15,737,680
FY2020	5	$5,265,797	$21,003,477
FY2020	6	$3,465,241	$24,468,717
FY2020	7	$3,513,064	$27,981,781
FY2020	8	$5,247,165	$33,228,947
FY2020	9	$5,104,088	$38,333,035
FY2020	10	$3,542,150	$41,875,184
FY2020	11	$5,151,897	$47,027,081
FY2020	12	$4,851,194	$51,878,275

Example 3 - visual calculation

The following visual calculation DAX query:

DAX

```
TotalSalesRunningSumByYear =
SUMX ( WINDOW ( 0, ABS, 0, REL, ROWS, HIGHESTPARENT ), [SalesAmount] )
```

Returns the cumulative total sales by month, calculated along each year. The value 1 could be used instead of `HIGHESTPARENT`, with the same result.

The screenshot below shows the visual matrix and the visual calculation expression:

```
1 TotalSalesRunningSumByYear = SUMX(WINDOW(0, ABS, 0, REL, ROWS, HIGHESTPARENT), [SalesAmount])
```

CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	TotalSalesRunningSumByYear	TotalSalesRunningSumByQuarter
2011	1	1	$323,743.2056	323,743.21	323,743.21
		2	$437,372.4476	761,115.65	761,115.65
		3	$514,918.2284	1,276,033.88	1,276,033.88
		Total	**$1,276,033.8816**	**1,276,033.88**	**1,276,033.88**
	2	4	$523,950.0258	1,799,983.91	523,950.03
		5	$539,469.0004	2,339,452.91	1,063,419.03
		6	$647,524.214	2,986,977.12	1,710,943.24

Example 4 - visual calculation

The following visual calculation DAX query:

DAX

```
TotalSalesRunningSumByQuarter =
SUMX ( WINDOW ( 0, ABS, 0, REL,, -1 ), [SalesAmount] )
```

Returns the cumulative total sales by month, calculated along each quarter. The value `LOWESTPARENT` could be used instead of -1, with the same result.

Related content

- INDEX
- MOVINGAVERAGE
- OFFSET
- ORDERBY
- PARTITIONBY
- RANGE
- RANK
- ROWNUMBER
- RUNNINGSUM

Last updated on 08/04/2026

Financial functions

Article • 10/20/2023

Financial functions in DAX are used in formulas that perform financial calculations, such as net present value and rate of return. These functions are similar to financial functions used in Microsoft Excel.

In this category

Function	Description
ACCRINT	Returns the accrued interest for a security that pays periodic interest.
ACCRINTM	Returns the accrued interest for a security that pays interest at maturity.
AMORDEGRC	Returns the depreciation for each accounting period. Similar to AMORLINC, except a depreciation coefficient is applied depending on the life of the assets.
AMORLINC	Returns the depreciation for each accounting period.
COUPDAYBS	Returns the number of days from the beginning of a coupon period until its settlement date.
COUPDAYS	Returns the number of days in the coupon period that contains the settlement date.
COUPDAYSNC	Returns the number of days from the settlement date to the next coupon date.
COUPNCD	Returns the next coupon date after the settlement date.
COUPNUM	Returns the number of coupons payable between the settlement date and maturity date, rounded up to the nearest whole coupon.
COUPPCD	Returns the previous coupon date before the settlement date.
CUMIPMT	Returns the cumulative interest paid on a loan between start_period and end_period.
CUMPRINC	Returns the cumulative principal paid on a loan between start_period and end_period.
DB	Returns the depreciation of an asset for a specified period using the fixed-declining balance method.
DDB	Returns the depreciation of an asset for a specified period using the double-declining balance method or some other method you specify.
DISC	Returns the discount rate for a security.

Function	Description
DOLLARDE	Converts a dollar price expressed as an integer part and a fraction part, such as 1.02, into a dollar price expressed as a decimal number.
DOLLARFR	Converts a dollar price expressed as an integer part and a fraction part, such as 1.02, into a dollar price expressed as a decimal number.
DURATION	Returns the Macauley duration for an assumed par value of $100.
EFFECT	Returns the effective annual interest rate, given the nominal annual interest rate and the number of compounding periods per year.
FV	Calculates the future value of an investment based on a constant interest rate.
INTRATE	Returns the interest rate for a fully invested security.
IPMT	Returns the interest payment for a given period for an investment based on periodic, constant payments and a constant interest rate.
ISPMT	Calculates the interest paid (or received) for the specified period of a loan (or investment) with even principal payments.
MDURATION	Returns the modified Macauley duration for a security with an assumed par value of $100.
NOMINAL	Returns the nominal annual interest rate, given the effective rate and the number of compounding periods per year.
NPER	Returns the number of periods for an investment based on periodic, constant payments and a constant interest rate.
ODDFPRICE	Returns the price per $100 face value of a security having an odd (short or long) first period.
ODDFYIELD	Returns the yield of a security that has an odd (short or long) first period.
ODDLPRICE	Returns the price per $100 face value of a security having an odd (short or long) last coupon period.
ODDLYIELD	Returns the yield of a security that has an odd (short or long) last period.
PDURATION	Returns the number of periods required by an investment to reach a specified value.
PMT	Calculates the payment for a loan based on constant payments and a constant interest rate.
PPMT	Returns the payment on the principal for a given period for an investment based on periodic, constant payments and a constant interest rate.
PRICE	Returns the price per $100 face value of a security that pays periodic interest.

Function	Description
PRICEDISC	Returns the price per $100 face value of a discounted security.
PRICEMAT	Returns the price per $100 face value of a security that pays interest at maturity.
PV	Calculates the present value of a loan or an investment, based on a constant interest rate.
RATE	Returns the interest rate per period of an annuity.
RECEIVED	Returns the amount received at maturity for a fully invested security.
RRI	Returns an equivalent interest rate for the growth of an investment.
SLN	Returns the straight-line depreciation of an asset for one period.
SYD	Returns the sum-of-years' digits depreciation of an asset for a specified period.
TBILLEQ	Returns the bond-equivalent yield for a Treasury bill.
TBILLPRICE	Returns the price per $100 face value for a Treasury bill.
TBILLYIELD	Returns the yield for a Treasury bill.
VDB	Returns the depreciation of an asset for any period you specify, including partial periods, using the double-declining balance method or some other method you specify.
XIRR	Returns the internal rate of return for a schedule of cash flows that is not necessarily periodic.
XNPV	Returns the present value for a schedule of cash flows that is not necessarily periodic.
YIELD	Returns the yield on a security that pays periodic interest.
YIELDDISC	Returns the annual yield for a discounted security.
YIELDMAT	Returns the annual yield of a security that pays interest at maturity.

Feedback

Was this page helpful? Yes No

ACCRINT

Article • 10/20/2023

Returns the accrued interest for a security that pays periodic interest.

Syntax

DAX

```
ACCRINT(<issue>, <first_interest>, <settlement>, <rate>, <par>, <frequency>
[, <basis>[, <calc_method>]])
```

Parameters

Term	Definition
issue	The security's issue date.
first_interest	The security's first interest date.
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
rate	The security's annual coupon rate.
par	The security's par value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.
calc_method	(Optional) A logical value that specifies the way to calculate the total accrued interest when the date of settlement is later than the date of first_interest. If calc_method is omitted, it is assumed to be TRUE. - If calc_method evaluates to TRUE or is omitted, ACCRINT returns the total accrued interest from issue to settlement. - If calc_method evaluates to FALSE, ACCRINT returns the accrued interest from first_interest to settlement.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The accrued interest.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- ACCRINT is calculated as follows:

$$\mathrm{ACCRINT} = \mathrm{par} \times \frac{\mathrm{rate}}{\mathrm{frequency}} \times \sum_{i=1}^{\mathrm{NC}} \frac{\mathrm{A}_i}{\mathrm{NL}_i}$$

  where:
  - A_i = number of accrued days for the i^{th} quasi-coupon period within odd period.
  - NC = number of quasi-coupon periods that fit in odd period. If this number contains a fraction, raise it to the next whole number.
  - NL_i = normal length in days of the quasi-coupon period within odd period.
- issue, first_interest, and settlement are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - issue, first_interest, or settlement is not a valid date.
  - issue ≥ settlement.
  - rate ≤ 0.
  - par ≤ 0.
  - frequency is any number other than 1, 2, or 4.

- basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Data	Description
1-March-2007	Issue date
31-August-2008	First interest date
1-May-2008	Settlement date
10%	Coupon rate
1000	Par value
2	Frequency is semiannual (see above)
0	30/360 basis (see above)

Example 1

The following DAX query:

```
DAX

EVALUATE
{
  ACCRINT(DATE(2007,3,1), DATE(2008,8,31), DATE(2008,5,1), 0.1, 1000, 2, 0)
}
```

Returns the accrued interest from issue to settlement, for a security with the terms specified above.

[Value]
116.944444444444

Example 2

The following DAX query:

DAX

```
EVALUATE
{
  ACCRINT(DATE(2007,3,1), DATE(2008,8,31), DATE(2008,5,1), 0.1, 1000, 2, 0,
FALSE)
}
```

Returns the accrued interest from first_interest to settlement, for a security with the terms specified above.

[Value]
66.9444444444445

Feedback

Was this page helpful? Yes No

ACCRINTM

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the accrued interest for a security that pays interest at maturity.

Syntax

DAX

```
ACCRINTM(<issue>, <maturity>, <rate>, <par>[, <basis>])
```

Parameters

Expand table

Term	Definition
issue	The security's issue date.
maturity	The security's maturity date.
rate	The security's annual coupon rate.
par	The security's par value.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The accrued interest.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- ACCRINTM is calculated as follows:

$$\mathrm{ACCRINTM} = \mathrm{par} \times \mathrm{rate} \times \frac{\mathrm{A}}{\mathrm{D}}$$

  where:
  - A = Number of accrued days counted according to a monthly basis. For interest at maturity items, the number of days from the issue date to the maturity date is used.
  - D = Annual Year Basis.
- issue and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - issue or maturity is not a valid date.
  - issue ≥ maturity.
  - rate ≤ 0.
  - par ≤ 0.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
1-April-2008	Issue date
15-June-2008	Maturity date
10%	Percent coupon
1000	Par value
3	Actual/365 basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  ACCRINTM(DATE(2008,4,1), DATE(2008,6,15), 0.1, 1000, 3)
}
```

Returns the accrued interest for a security with the terms specified above.

Expand table

[Value]
20.5479452054795

Feedback

Was this page helpful? Yes No

AMORDEGRC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the depreciation for each accounting period. This function is provided for the French accounting system. If an asset is purchased in the middle of the accounting period, the prorated depreciation is taken into account. The function is similar to AMORLINC, except that a depreciation coefficient is applied in the calculation depending on the life of the assets.

Syntax

DAX

```
AMORDEGRC(<cost>, <date_purchased>, <first_period>, <salvage>, <period>,
<rate>[, <basis>])
```

Parameters

Expand table

Term	Definition
cost	The cost of the asset.
date_purchased	The date of the purchase of the asset.
first_period	The date of the end of the first period.
salvage	The salvage value at the end of the life of the asset.
period	The period.
rate	The rate of depreciation.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Date system
0 or omitted	360 days (NASD method)
1	Actual
3	365 days in a year
4	360 days in a year (European method)

Return Value

The depreciation for each accounting period.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- This function will return the depreciation until the last period of the life of the assets or until the cumulated value of depreciation is greater than the cost of the assets minus the salvage value.
- The depreciation coefficients are:

Expand table

Life of assets (1/rate)	Depreciation coefficient
Between 3 and 4 years	1.5
Between 5 and 6 years	2
More than 6 years	2.5

- The depreciation rate will grow to 50 percent for the period preceding the last period and will grow to 100 percent for the last period.
- period and basis are rounded to the nearest integer.
- An error is returned if:
  - cost < 0.
  - first_period or date_purchased is not a valid date.
  - date_purchased > first_period.

- salvage < 0 or salvage > cost.
- period < 0.
- rate ≤ 0.
- The life of assets is between 0 (zero) and 1, 1 and 2, 2 and 3, or 4 and 5.
- basis is any number other than 0, 1, 3, or 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
2400	Cost
19-August-2008	Date purchased
31-December-2008	End of the first period
300	Salvage value
1	Period
15%	Depreciation rate
1	Actual basis (see above)

The following DAX query:

DAX

```
EVALUATE
{
  AMORDEGRC(2400, DATE(2008,8,19), DATE(2008,12,31), 300, 1, 0.15, 1)
}
```

Returns the first period's depreciation, given the terms specified above.

Expand table

[Value]
776

Feedback

Was this page helpful? Yes No

AMORLINC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the depreciation for each accounting period. This function is provided for the French accounting system. If an asset is purchased in the middle of the accounting period, the prorated depreciation is taken into account.

Syntax

DAX

```
AMORLINC(<cost>, <date_purchased>, <first_period>, <salvage>, <period>,
<rate>[, <basis>])
```

Parameters

Expand table

Term	Definition
cost	The cost of the asset.
date_purchased	The date of the purchase of the asset.
first_period	The date of the end of the first period.
salvage	The salvage value at the end of the life of the asset.
period	The period.
rate	The rate of depreciation.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Date system
0 or omitted	360 days (NASD method)
1	Actual
3	365 days in a year
4	360 days in a year (European method)

Return Value

The depreciation for each accounting period.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- period and basis are rounded to the nearest integer.
- An error is returned if:
  - cost < 0.
  - first_period or date_purchased is not a valid date.
  - date_purchased > first_period.
  - salvage < 0 or salvage > cost.
  - period < 0.
  - rate ≤ 0.
  - basis is any number other than 0, 1, 3, or 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
2400	Cost
19-August-2008	Date purchased

Data	Description
31-December-2008	End of the first period
300	Salvage value
1	Period
15%	Depreciation rate
1	Actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  AMORLINC(2400, DATE(2008,8,19), DATE(2008,12,31), 300, 1, 0.15, 1)
}
```

Returns the first period's depreciation, given the terms specified above.

Expand table

[Value]
360

Feedback

Was this page helpful? Yes No

COUPDAYBS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of days from the beginning of a coupon period until its settlement date.

Syntax

DAX

```
COUPDAYBS(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360

Basis	Day count basis
3	Actual/365
4	European 30/360

Return Value

The number of days from the beginning of a coupon period until its settlement date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
25-Jan-11	Settlement date
15-Nov-11	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  COUPDAYBS(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Returns the number of days from the beginning of the coupon period until the settlement date, for a bond with terms above.

Expand table

[Value]
71

Feedback

Was this page helpful? Yes No

COUPDAYS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of days in the coupon period that contains the settlement date.

Syntax

DAX

```
COUPDAYS(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The number of days in the coupon period that contains the settlement date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date is January 1, 2038, 30 years after the January 1, 2008 issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
25-Jan-11	Settlement date

Data	Description
15-Nov-11	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  COUPDAYS(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Returns the number of days in the coupon period that contains the settlement date, for a bond with the terms specified above.

Expand table

[Value]
181

Feedback

Was this page helpful? Yes No

COUPDAYSNC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of days from the settlement date to the next coupon date.

Syntax

```
DAX
COUPDAYSNC(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The number of days from the settlement date to the next coupon date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
25-Jan-11	Settlement date

Data	Description
15-Nov-11	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  COUPDAYSNC(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Returns the number of days from the settlement date to the next coupon date, for a bond with the terms specified above.

Expand table

[Value]
110

Feedback

Was this page helpful? Yes No

COUPNCD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the next coupon date after the settlement date.

Syntax

DAX

```
COUPNCD(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The next coupon date after the settlement date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date is January 1, 2038, 30 years after the January 1, 2008 issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
25-Jan-11	Settlement date

Data	Description
15-Nov-11	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  COUPNCD(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Returns the next coupon date after the settlement date, for a bond with the terms specified above.

Expand table

[Value]
5/15/2011 12:00:00 AM

Feedback

Was this page helpful? Yes No

COUPNUM

Article • 10/20/2023

Returns the number of coupons payable between the settlement date and maturity date, rounded up to the nearest whole coupon.

Syntax

```
DAX

COUPNUM(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The number of coupons payable between the settlement date and maturity date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Description
25-Jan-07	Settlement date
15-Nov-08	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX
```

```
EVALUATE
{
  COUPNUM(DATE(2007,1,25), DATE(2008,11,15), 2, 1)
}
```

Returns the number of coupon payments for a bond with the terms specified above.

[Value]
4

Feedback

Was this page helpful? Yes No

COUPPCD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the previous coupon date before the settlement date.

Syntax

DAX

```
COUPPCD(<settlement>, <maturity>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The previous coupon date before the settlement date.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- frequency and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
25-Jan-11	Settlement date

Data	Description
15-Nov-11	Maturity date
2	Semiannual coupon (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  COUPPCD(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Returns the previous coupon date before the settlement date, for a bond using the terms specified above.

Expand table

[Value]
11/15/2010 12:00:00 AM

Feedback

Was this page helpful? Yes No

CUMIPMT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the cumulative interest paid on a loan between start_period and end_period.

Syntax

DAX

```
CUMIPMT(<rate>, <nper>, <pv>, <start_period>, <end_period>, <type>)
```

Parameters

Expand table

Term	Definition
rate	The interest rate.
nper	The total number of payment periods.
pv	The present value.
start_period	The first period in the calculation. Must be between 1 and end_period (inclusive).
end_period	The last period in the calculation. Must be between start_period and nper (inclusive).
type	The timing of the payment. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Type	Timing
0 (zero)	Payment at the end of the period
1	Payment at the beginning of the period

Return Value

The cumulative interest paid in the specified period.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at an annual interest rate of 10 percent, use 0.1/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.1 for rate and 4 for nper.
- start_period, end_period, and type are rounded to the nearest integer.
- An error is returned if:
  - rate ≤ 0.
  - nper < 1.
  - pv ≤ 0.
  - start_period < 1 or start_period > end_period.
  - end_period < start_period or end_period > nper.
  - type is any number other than 0 or 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
9%	Annual interest rate
30	Years of the loan
125000	Present value

Example 1

The following DAX query:

```
DAX
```

```
EVALUATE
{
  CUMIPMT(0.09/12, 30*12, 125000, 13, 24, 1)
}
```

Returns the total interest paid in the second year of payments, periods 13 through 24, assuming that the payments are made at the beginning of each month.

Expand table

[Value]
-11052.3395838718

Example 2

The following DAX query:

DAX

```
EVALUATE
{
  CUMIPMT(0.09/12, 30*12, 125000, 1, 1, 0)
}
```

Returns the interest paid in a single payment in the first month, assuming that the payment is made at the end of the month.

Expand table

[Value]
-937.5

Feedback

Was this page helpful? Yes No

CUMPRINC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the cumulative principal paid on a loan between start_period and end_period.

Syntax

DAX

```
CUMPRINC(<rate>, <nper>, <pv>, <start_period>, <end_period>, <type>)
```

Parameters

Expand table

Term	Definition
rate	The interest rate.
nper	The total number of payment periods.
pv	The present value.
start_period	The first period in the calculation. Must be between 1 and end_period (inclusive).
end_period	The last period in the calculation. Must be between start_period and nper (inclusive).
type	The timing of the payment. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Type	Timing
0 (zero)	Payment at the end of the period
1	Payment at the beginning of the period

Return Value

The cumulative principal paid in the specified period.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at an annual interest rate of 10 percent, use 0.1/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.1 for rate and 4 for nper.
- start_period, end_period, and type are rounded to the nearest integer.
- An error is returned if:
  - rate ≤ 0.
  - nper < 1.
  - pv ≤ 0.
  - start_period < 1 or start_period > end_period.
  - end_period < start_period or end_period > nper.
  - type is any number other than 0 or 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
9%	Annual interest rate
30	Term in years
125000	Present value

Example 1

The following DAX query:

```
DAX
```

```
EVALUATE
{
  CUMPRINC(0.09/12, 30*12, 125000, 13, 24, 1)
}
```

Returns the total principal paid in the second year of payments, periods 13 through 24, assuming the payments are made at the beginning of each month.

Expand table

[Value]
-927.153472378062

Example 2

The following DAX query:

DAX

```
EVALUATE
{
  CUMPRINC(0.09/12, 30*12, 125000, 1, 1, 0)
}
```

Returns the principal paid in a single payment in the first month, assuming the payment is made at the end of the month.

Expand table

[Value]
-68.2782711809784

Feedback

Was this page helpful? Yes No

DB

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the depreciation of an asset for a specified period using the fixed-declining balance method.

Syntax

DAX

```
DB(<cost>, <salvage>, <life>, <period>[, <month>])
```

Parameters

Expand table

Term	Definition
cost	The initial cost of the asset.
salvage	The value at the end of the depreciation (sometimes called the salvage value of the asset). This value can be 0.
life	The number of periods over which the asset is being depreciated (sometimes called the useful life of the asset).
period	The period for which you want to calculate the depreciation. Period must use the same units as life. Must be between 1 and life (inclusive).
month	(Optional) The number of months in the first year. If month is omitted, it is assumed to be 12.

Return Value

The depreciation over the specified period.

Remarks

- The fixed-declining balance method computes depreciation at a fixed rate. DB uses the following formulas to calculate depreciation for a period:

  $$(\text{cost} - \text{total depreciation from prior periods}) \times \text{rate}$$

  where:
  - $\text{rate} = 1 - ((\frac{\text{salvage}}{\text{cost}})^{(\frac{1}{\text{life}})})$, rounded to three decimal places

- Depreciation for the first and last periods is a special case.
  - For the first period, DB uses this formula:

    $$\frac{\text{cost} \times \text{rate} \times \text{month}}{12}$$

  - For the last period, DB uses this formula:

    $$\frac{(\text{cost} - \text{total depreciation from prior periods}) \times \text{rate} \times (12 - \text{month})}{12}$$

- period and month are rounded to the nearest integer.
- An error is returned if:
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - period < 1 or period > life.
  - month < 1 or month > 12.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Example 1

The following DAX query:

DAX

```
EVALUATE
{
  DB(1000000, 0, 6, 1, 2)
}
```

Returns an asset's depreciation in the last two months of the first year, assuming it will be worth $0 after 6 years.

Expand table

[Value]
166666.666666667

Example 2

The following calculates the total depreciation of all assets in different years over their lifetimes. Here, the first year only includes 7 months of depreciation, and the last year only includes 5 months.

DAX

```
DEFINE
VAR NumDepreciationPeriods = MAX(Asset[LifeTimeYears])+1
VAR DepreciationPeriods = GENERATESERIES(1, NumDepreciationPeriods)
EVALUATE
ADDCOLUMNS (
  DepreciationPeriods,
  "Current Period Total Depreciation",
  SUMX (
    FILTER (
      Asset,
      [Value] <= [LifetimeYears]+1
    ),
    DB([InitialCost], [SalvageValue], [LifetimeYears], [Value], 7)
  )
)
```

Feedback

Was this page helpful? Yes No

DDB

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the depreciation of an asset for a specified period using the double-declining balance method or some other method you specify.

Syntax

```
DAX

DDB(<cost>, <salvage>, <life>, <period>[, <factor>])
```

Parameters

Expand table

Term	Definition
cost	The initial cost of the asset.
salvage	The value at the end of the depreciation (sometimes called the salvage value of the asset). This value can be 0.
life	The number of periods over which the asset is being depreciated (sometimes called the useful life of the asset).
period	The period for which you want to calculate the depreciation. Period must use the same units as life. Must be between 1 and life (inclusive).
factor	(Optional) The rate at which the balance declines. If factor is omitted, it is assumed to be 2 (the double-declining balance method).

Return Value

The depreciation over the specified period.

Remarks

- The double-declining balance method computes depreciation at an accelerated rate. Depreciation is highest in the first period and decreases in successive periods. DDB uses the following formula to calculate depreciation for a period:

  $$\text{Min}((\text{cost} - \text{total depreciation from prior periods}) \times (\frac{\text{factor}}{\text{life}}), (\text{cost} - \text{salvage} - \text{total depreciation from prior periods}))$$

- Change factor if you do not want to use the double-declining balance method.
- Use the VDB function if you want to switch to the straight-line depreciation method when depreciation is greater than the declining balance calculation.
- period is rounded to the nearest integer.
- An error is returned if:
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - period < 1 or period > life.
  - factor ≤ 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Example 1

The following DAX query:

```
DAX

EVALUATE
{
  DDB(1000000, 0, 10, 5, 1.5)
}
```

Returns an asset's depreciation in the 5^{th} year, assuming it will be worth $0 after 10 years. This calculation uses a factor of 1.5.

Expand table

[Value]
78300.9375

Example 2

The following calculates the total depreciation of all assets in different years over their lifetimes. This calculation uses the default factor of 2 (the double-declining balance method).

```
DAX

DEFINE
VAR NumDepreciationPeriods = MAX(Asset[LifeTimeYears])
VAR DepreciationPeriods = GENERATESERIES(1, NumDepreciationPeriods)
EVALUATE
  ADDCOLUMNS (
  DepreciationPeriods,
  "Current Period Total Depreciation",
  SUMX (
    FILTER (
      Asset,
      [Value] <= [LifetimeYears]
    ),
    DDB([InitialCost], [SalvageValue], [LifetimeYears], [Value])
  )
)
```

Feedback

Was this page helpful? Yes No

DISC

Article • 10/20/2023

Returns the discount rate for a security.

Syntax

```
DAX
DISC(<settlement>, <maturity>, <pr>, <redemption>[, <basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
pr	The security's price per $100 face value.
redemption	The security's redemption value per $100 face value.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The discount rate.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2018, and is purchased by a buyer six months later. The issue date would be January 1, 2018, the settlement date would be July 1, 2018, and the maturity date would be January 1, 2048, 30 years after the January 1, 2018, issue date.
- DISC is calculated as follows:

$$\mathrm{DISC} = \frac{\mathrm{redemption} - \mathrm{par}}{\mathrm{redemption}} \times \frac{\mathrm{B}}{\mathrm{DSM}}$$

  where:

  - B = number of days in a year, depending on the year basis.
  - DSM = number of days between settlement and maturity.
- settlement and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - pr ≤ 0.
  - redemption ≤ 0.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Description
07/01/2018	Settlement date
01/01/2048	Maturity date

Data	Description
97.975	Price
100	Redemption value
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  DISC(DATE(2018,7,1), DATE(2048,1,1), 97.975, 100, 1)
}
```

Returns the bond discount rate, for a bond with the terms specified above.

[Value]
0.000686384169121348

Feedback

Was this page helpful? Yes No

DOLLARDE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts a dollar price expressed as an integer part and a fraction part, such as 1.02, into a dollar price expressed as a decimal number. Fractional dollar numbers are sometimes used for security prices.

Syntax

DAX

```
DOLLARDE(<fractional_dollar>, <fraction>)
```

Parameters

Expand table

Term	Definition
fractional_dollar	A number expressed as an integer part and a fraction part, separated by a decimal symbol.
fraction	The integer to use in the denominator of the fraction.

Return Value

The decimal value of *fractional_dollar*.

Remarks

- The fraction part of the value is divided by an integer that you specify. For example, if you want your price to be expressed to a precision of 1/16 of a dollar, you divide the fraction part by 16. In this case, 1.02 represents $1.125 ($1 + 2/16 = $1.125).
- fraction is rounded to the nearest integer.

- An error is returned if:
  - fraction < 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

```
DAX

EVALUATE
{
  DOLLARDE(1.02, 16)
}
```

Returns 1.125, the decimal price of the original fractional price, 1.02, read as 1 and 2/16. Since the fraction value is 16, the price has a precision of 1/16 of a dollar.

Feedback

Was this page helpful? Yes No

DOLLARFR

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Converts a dollar price expressed as a decimal number into a dollar price expressed as an integer part and a fraction part, such as 1.02. Fractional dollar numbers are sometimes used for security prices.

Syntax

DAX

```
DOLLARFR(<decimal_dollar>, <fraction>)
```

Parameters

Expand table

Term	Definition
decimal_dollar	A decimal number.
fraction	The integer to use in the denominator of the fraction.

Return Value

The fractional value of *decimal_dollar*, expressed as an integer part and a fraction part.

Remarks

- fraction is rounded to the nearest integer.
- An error is returned if:
  - fraction < 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

```
DAX

EVALUATE
{
  DOLLARFR(1.125, 16)
}
```

Returns 1.02, read as 1 and 2/16, which is the corresponding fraction price of the original decimal price, 1.125. Since the fraction value is 16, the price has a precision of 1/16 of a dollar.

Feedback

Was this page helpful? Yes No

DURATION

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the Macauley duration for an assumed par value of $100. Duration is defined as the weighted average of the present value of cash flows, and is used as a measure of a bond price's response to changes in yield.

Syntax

DAX

```
DURATION(<settlement>, <maturity>, <coupon>, <yld>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
coupon	The security's annual coupon rate.
yld	The security's annual yield.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The Macauley duration.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date is January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- frequency, and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - coupon < 0.
  - yld < 0
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
07/01/2018	Settlement date
01/01/2048	Maturity date
8.0%	Percent coupon
9.0%	Percent yield
2	Frequency is semiannual (see above)
1	Actual/actual basis (see above)

The following DAX query:

DAX

```
EVALUATE
{
  DURATION(DATE(2018,7,1), DATE(2048,1,1), 0.08, 0.09, 2, 1)
}
```

Returns the Macauley duration for a bond with the terms specified above.

Expand table

[Value]
10.9191452815919

Feedback

Was this page helpful? Yes No

EFFECT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the effective annual interest rate, given the nominal annual interest rate and the number of compounding periods per year.

Syntax

DAX

```
EFFECT(<nominal_rate>, <npery>)
```

Parameters

Expand table

Term	Definition
nominal_rate	The nominal interest rate.
npery	The number of compounding periods per year.

Return Value

The effective annual interest rate.

Remarks

- EFFECT is calculated as follows:

$$\text{EFFECT} = \left(1 + \frac{\text{nominal_rate}}{\text{npery}}\right)^{\text{npery}} - 1$$

- npery is rounded to the nearest integer.
- An error is returned if:
  - nominal_rate ≤ 0.

- npery < 1.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
5.25%	Nominal interest rate
4	Number of compounding periods per year

The following DAX query:

```
DAX

EVALUATE
{
  EFFECT(0.0525, 4)
}
```

Returns the effective interest rate using the terms specified above.

Expand table

[Value]
0.0535426673707584

Feedback

Was this page helpful? Yes No

FV

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Calculates the future value of an investment based on a constant interest rate. You can use FV with either periodic, constant payments, and/or a single lump sum payment.

Syntax

```
DAX
FV(<rate>, <nper>, <pmt>[, <pv>[, <type>]])
```

Parameters

Expand table

Term	Definition
rate	The interest rate per period.
nper	The total number of payment periods in an annuity.
pmt	The payment made each period; it cannot change over the life of the annuity. Typically, pmt contains principal and interest but no other fees or taxes.
pv	(Optional) The present value, or the lump-sum amount that a series of future payments is worth right now. If pv is omitted, it is assumed to be BLANK.
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period
1	At the beginning of the period

Note: For a more complete description of the arguments in FV and for more information on annuity functions, see the PV function.

Return Value

The future value of an investment.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at 12 percent annual interest, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.
- For all the arguments, cash you pay out, such as deposits to savings, is represented by negative numbers; cash you receive, such as dividend checks, is represented by positive numbers.
- type is rounded to the nearest integer.
- An error is returned if:
  - nper < 1
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
6%	Annual interest rate
10	Number of payments
-200	Amount of the payment
-500	Present value
1	Payment is due at the beginning of the period (0 indicates payment is due at end of period)

The following DAX query:

DAX

```
EVALUATE
{
  FV(0.06/12, 10, -200, -500, 1)
}
```

Returns the future value of an investment using the terms specified above.

Expand table

[Value]
2581.40337406012

Feedback

Was this page helpful? Yes No

INTRATE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the interest rate for a fully invested security.

Syntax

DAX

```
INTRATE(<settlement>, <maturity>, <investment>, <redemption>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
investment	The amount invested in the security.
redemption	The amount to be received at maturity.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360

Basis	Day count basis
3	Actual/365
4	European 30/360

Return Value

The interest rate.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date is January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- INTRATE is calculated as follows:

$$\mathrm{INTRATE} = \frac{\mathrm{redemption} - \mathrm{investment}}{\mathrm{investment}} \times \frac{\mathrm{B}}{\mathrm{DIM}}$$

  where:
  - B = number of days in a year, depending on the year basis.
  - DIM = number of days from settlement to maturity.
- settlement and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - investment ≤ 0.
  - redemption ≤ 0.
  - basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
2/15/2008	Settlement date
5/15/2008	Maturity date
$1,000,000	Investment
$1,014,420	Redemption value
2	Actual/360 basis

The following DAX query:

```
DAX

EVALUATE
{
  INTRATE(DATE(2008,2,15), DATE(2008,5,15), 1000000, 1014420, 2)
}
```

Returns the discount rate for a bond using the terms specified above.

Expand table

[Value]
0.05768

Feedback

Was this page helpful? Yes No

IPMT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the interest payment for a given period for an investment based on periodic, constant payments and a constant interest rate.

Syntax

DAX

```
IPMT(<rate>, <per>, <nper>, <pv>[, <fv>[, <type>]])
```

Parameters

Expand table

Term	Definition
rate	The interest rate per period.
per	The period for which you want to find the interest. Must be between 1 and nper (inclusive).
nper	The total number of payment periods in an annuity.
pv	The present value, or the lump-sum amount that a series of future payments is worth right now.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be BLANK.
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period

Set type equal to	If payments are due
1	At the beginning of the period

Return Value

The interest payment for the given period.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at 12 percent annual interest, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.
- For all the arguments, cash you pay out, such as deposits to savings, is represented by negative numbers; cash you receive, such as dividend checks, is represented by positive numbers.
- type is rounded to the nearest integer.
- An error is returned if:
  - per < 1 or per > nper
  - nper < 1
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
10.00%	Annual interest
3	Years of loan
$8,000	Present value of loan

Example 1

The following DAX query:

```
DAX

EVALUATE
{
  IPMT(0.1/12, 1, 3*12, 8000)
}
```

Returns the monthly interest due in the first month for a loan with the terms specified above.

Expand table

[Value]
-66.6666666666667

Example 2

The following DAX query:

```
DAX

EVALUATE
{
  IPMT(0.1, 3, 3, 8000)
}
```

Returns the yearly interest due in the last year for a loan with the terms specified above, where payments are made yearly.

Expand table

[Value]
-292.447129909366

Feedback

Was this page helpful? Yes No

ISPMT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Calculates the interest paid (or received) for the specified period of a loan (or investment) with even principal payments.

Syntax

```
DAX
ISPMT(<rate>, <per>, <nper>, <pv>)
```

Parameters

Expand table

Term	Definition
rate	The interest rate for the investment.
per	The period for which you want to find the interest. Must be between 0 and nper-1 (inclusive).
nper	The total number of payment periods for the investment.
pv	The present value of the investment. For a loan, pv is the loan amount.

Return Value

The interest paid (or received) for the specified period.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at an annual interest rate of 12 percent, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.

- For all the arguments, the cash you pay out, such as deposits to savings or other withdrawals, is represented by negative numbers; the cash you receive, such as dividend checks and other deposits, is represented by positive numbers.
- ISPMT counts each period beginning with zero, not one.
- Most loans use a repayment schedule with even periodic payments. The IPMT function returns the interest payment for a given period for this type of loan.
- Some loans use a repayment schedule with even principal payments. The ISPMT function returns the interest payment for a given period for this type of loan.
- An error is returned if:
  - nper = 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
$4,000	Present value
4	Number of periods
10%	Rate

To illustrate when to use ISPMT, the amortization table below uses an even-principal repayment schedule with the terms specified above. The interest charge each period is equal to the rate times the unpaid balance for the previous period. And the payment each period is equal to the even principal plus the interest for the period.

Expand table

Period	Principal Payment	Interest Payment	Total Payment	Balance
				4,000.00
1	1,000.00	400.00	1,400.00	3,000.00
2	1,000.00	300.00	1,300.00	2,000.00
3	1,000.00	200.00	1,200.00	1,000.00

Period	Principal Payment	Interest Payment	Total Payment	Balance
4	1,000.00	100.00	1,100.00	0.00

The following DAX query:

```
DAX

DEFINE
VAR NumPaymentPeriods = 4
VAR PaymentPeriods = GENERATESERIES(0, NumPaymentPeriods-1)
EVALUATE
ADDCOLUMNS (
  PaymentPeriods,
  "Interest Payment",
  ISPMT(0.1, [Value], NumPaymentPeriods, 4000)
)
```

Returns the interest paid during each period, using the even-principal repayment schedule and terms specified above. The values are negative to indicate that it is interest paid, not received.

Expand table

[Value]	[Interest Payment]
0	-400
1	-300
2	-200
3	-100

Feedback

Was this page helpful? Yes No

MDURATION

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the modified Macauley duration for a security with an assumed par value of $100.

Syntax

```
DAX
MDURATION(<settlement>, <maturity>, <coupon>, <yld>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
coupon	The security's annual coupon rate.
yld	The security's annual yield.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360

Basis	Day count basis
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The modified Macauley duration.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date is January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- Modified duration is defined as follows:

$$\text{MDURATION} = \frac{\text{DURATION}}{1 + (\frac{\text{Market yield}}{\text{Coupon payments per year}})}$$

- settlement and maturity are truncated to integers.
- frequency, and basis are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - coupon < 0.
  - yld < 0
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
1/1/2008	Settlement date
1/1/2016	Maturity date
8%	Percent coupon
9%	Percent yield
2	Frequency is semiannual (see above)
1	Actual/actual basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  MDURATION(DATE(2008,1,1), DATE(2016,1,1), 0.08, 0.09, 2, 1)
}
```

Returns the modified Macauley duration of a bond using the terms specified above.

Expand table

[Value]
5.73566981391884

Feedback

Was this page helpful? Yes No

NOMINAL

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the nominal annual interest rate, given the effective rate and the number of compounding periods per year.

Syntax

```
DAX
NOMINAL(<effect_rate>, <npery>)
```

Parameters

Expand table

Term	Definition
effect_rate	The effective interest rate.
npery	The number of compounding periods per year.

Return Value

The nominal annual interest rate.

Remarks

- The relationship between NOMINAL and EFFECT is shown in the following equation:

$$\text{EFFECT} = \left(1 + \frac{\text{nominal_rate}}{\text{npery}}\right)^{\text{npery}} - 1$$

- npery is rounded to the nearest integer.
- An error is returned if:
  - effect_rate ≤ 0.

- npery < 1.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
5.3543%	Effective interest rate
4	Number of compounding periods per year

The following DAX query:

```
DAX

EVALUATE
{
  NOMINAL(0.053543, 4)
}
```

Returns the nominal interest rate, using the terms specified above.

Expand table

[Value]
0.052500319868356

Feedback

Was this page helpful? Yes No

NPER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of periods for an investment based on periodic, constant payments and a constant interest rate.

Syntax

```
DAX
NPER(<rate>, <pmt>, <pv>[, <fv>[, <type>]])
```

Parameters

Expand table

Term	Definition
rate	The interest rate per period.
pmt	The payment made each period; it cannot change over the life of the annuity. Typically, pmt contains principal and interest but no other fees or taxes.
pv	The present value, or the lump-sum amount that a series of future payments is worth right now.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be BLANK.
type	(Optional) The number 0 or 1 and indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period
1	At the beginning of the period

Return Value

The number of periods for an investment.

Remarks

- type is rounded to the nearest integer.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
12%	Annual interest rate
-100	Payment made each period
-1000	Present value
10000	Future value
1	Payment is due at the beginning of the period (see above)

The following DAX query:

```
DAX

EVALUATE
{
  NPER(0.12/12, -100, -1000, 10000, 1)
}
```

Returns the number of periods for the investment described by the terms specified above.

Expand table

[Value]
59.6738656742946

Feedback

Was this page helpful? Yes No

ODDFPRICE

Article • 10/20/2023

Returns the price per $100 face value of a security having an odd (short or long) first period.

Syntax

```
DAX

ODDFPRICE(<settlement>, <maturity>, <issue>, <first_coupon>, <rate>, <yld>, <redemption>, <frequency>[, <basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
issue	The security's issue date.
first_coupon	The security's first coupon date.
rate	The security's interest rate.
yld	The security's annual yield.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- ODDFPRICE is calculated as follows:

  Odd short first coupon:

$$\text{ODDFPRICE} = \left[\frac{\text{redemption}}{(1+\frac{\text{yld}}{\text{frequency}})^{(N-1+\frac{\text{DSC}}{\text{E}})}}\right] + \left[\frac{100 \times \frac{\text{rate}}{\text{frequency}} \times \frac{\text{DFC}}{\text{E}}}{(1+\frac{\text{yld}}{\text{frequency}})^{(\frac{\text{DSC}}{\text{E}})}}\right] + \left[\sum_{k=2}^{N} \frac{100 \times \frac{\text{rate}}{\text{frequency}}}{(1+\frac{\text{yld}}{\text{frequency}})^{(k-1+\frac{\text{DSC}}{\text{E}})}}\right] - \left[100 \times \frac{\text{rate}}{\text{frequency}} \right.$$

where:

- A = number of days from the beginning of the coupon period to the settlement date (accrued days).
- DSC = number of days from the settlement to the next coupon date.
- DFC = number of days from the beginning of the odd first coupon to the first coupon date.
- E = number of days in the coupon period.
- N = number of coupons payable between the settlement date and the redemption date. (If this number contains a fraction, it is raised to the next whole number.)

Odd long first coupon:

$$\mathrm{ODDFPRICE} = \left[\frac{\mathrm{redemption}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(\mathrm{N}+\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\frac{100\times\frac{\mathrm{rate}}{\mathrm{frequency}}\times\left[\sum_{i=1}^{\mathrm{NC}}\frac{\mathrm{DC}_i}{\mathrm{NL}_i}\right]}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\sum_{k=1}^{\mathrm{N}}\frac{100\times\frac{\mathrm{rate}}{\mathrm{frequency}}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(k-\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] - \left[100\times\frac{\mathrm{fre}}{}\right.$$

where:

- A_i = number of days from the beginning of the i^{th}, or last, quasi-coupon period within odd period.
- DC_i = number of days from dated date (or issue date) to first quasi-coupon ($i = 1$) or number of days in quasi-coupon ($i = 2$,..., $i = \mathrm{NC}$).
- DSC = number of days from settlement to next coupon date.
- E = number of days in coupon period.
- N = number of coupons payable between the first real coupon date and redemption date. (If this number contains a fraction, it is raised to the next whole number.)
- NC = number of quasi-coupon periods that fit in odd period. (If this number contains a fraction, it is raised to the next whole number.)
- NL_i = normal length in days of the full i^{th}, or last, quasi-coupon period within odd period.
- N_q = number of whole quasi-coupon periods between settlement date and first coupon.

- settlement, maturity, issue, and first_coupon are truncated to integers.
- basis and frequency are rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, issue, or first_coupon is not a valid date.
  - maturity > first_coupon > settlement > issue is not satisfied.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Argument description
11/11/2008	Settlement date
3/1/2021	Maturity date
10/15/2008	Issue date
3/1/2009	First coupon date
7.85%	Percent coupon
6.25%	Percent yield
$100.00	Redemptive value
2	Frequency is semiannual
1	Actual/actual basis

The following DAX query:

```
DAX
```

```
EVALUATE
{
  ODDFPRICE(DATE(2008,11,11), DATE(2021,3,1), DATE(2008,10,15), DATE(2009,3,1), 0.0785, 0.0625, 100.00, 2, 1)
}
```

Returns the price per $100 face value of a security having an odd (short or long) first period, using the terms specified above.

[Value]
113.597717474079

Feedback

Was this page helpful? Yes No

ODDFYIELD

Article • 10/20/2023

Returns the yield of a security that has an odd (short or long) first period.

Syntax

DAX

```
ODDFYIELD(<settlement>, <maturity>, <issue>, <first_coupon>, <rate>, <pr>,
<redemption>, <frequency>[, <basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
issue	The security's issue date.
first_coupon	The security's first coupon date.
rate	The security's interest rate.
pr	The security's price.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual

Basis	Day count basis
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The security's yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- ODDFYIELD is calculated using an iterative method. It uses the Newton method based on the formula used for the function ODDFPRICE. The yield is changed through 100 iterations until the estimated price with the given yield is close to the price. See ODDFPRICE for the formula that ODDFYIELD uses.
- settlement, maturity, issue, and first_coupon are truncated to integers.
- basis and frequency are rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, issue, or first_coupon is not a valid date.
  - maturity > first_coupon > settlement > issue is not satisfied.
  - rate < 0.
  - pr ≤ 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Argument description
November 11, 2008	Settlement date
March 1, 2021	Maturity date
October 15, 2008	Issue date
March 1, 2009	First coupon date
5.75%	Percent coupon
84.50	Price
100	Redemptive value
2	Frequency is semiannual
0	30/360 basis

The following DAX query:

```
DAX

EVALUATE
{
  ODDFYIELD(DATE(2008,11,11), DATE(2021,3,1), DATE(2008,10,15),
DATE(2009,3,1), 0.0575, 84.50, 100, 2, 0)
}
```

Returns the yield of a security that has an odd (short or long) first period, using the terms specified above.

[Value]
0.0772455415972989

Feedback

Was this page helpful?

Yes No

ODDLPRICE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the price per $100 face value of a security having an odd (short or long) last coupon period.

Syntax

DAX

```
ODDLPRICE(<settlement>, <maturity>, <last_interest>, <rate>, <yld>,
<redemption>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
last_interest	The security's last coupon date.
rate	The security's interest rate.
yld	The security's annual yield.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement, maturity, and last_interest are truncated to integers.
- basis and frequency are rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, or last_interest is not a valid date.
  - maturity > settlement > last_interest is not satisfied.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

Expand table

Data	Argument description
February 7, 2008	Settlement date
June 15, 2008	Maturity date
October 15, 2007	Last interest date
3.75%	Percent coupon
4.05%	Percent yield
$100	Redemptive value
2	Frequency is semiannual
0	30/360 basis

DAX

```
EVALUATE
{
  ODDLPRICE(DATE(2008,2,7), DATE(2008,6,15), DATE(2007,10,15), 0.0375,
0.0405, 100, 2, 0)
}
```

Returns the price per $100 face value of a security that has an odd (short or long) last coupon period, using the terms specified above.

Expand table

[Value]
99.8782860147213

Feedback

Was this page helpful? Yes No

ODDLYIELD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the yield of a security that has an odd (short or long) last period.

Syntax

```
DAX

ODDLYIELD(<settlement>, <maturity>, <last_interest>, <rate>, <pr>, <redemption>, <frequency>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
last_interest	The security's last coupon date.
rate	The security's interest rate.
pr	The security's price.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The security's yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- ODDLYIELD is calculated as follows:

$$\text{ODDLYIELD} = \left[\frac{(\text{redemption} + ((\sum_{i=1}^{\text{NC}} \frac{\text{DC}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}})) - (\text{par} + ((\sum_{i=1}^{\text{NC}} \frac{\text{A}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}}))}{\text{par} + ((\sum_{i=1}^{\text{NC}} \frac{\text{A}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}})}\right] \times \left[\frac{\text{frequency}}{(\sum_{i=1}^{\text{NC}} \frac{\text{DSC}_i}{\text{NL}_i})}\right]$$

  where:
  - A_i = number of accrued days for the i^{th}, or last, quasi-coupon period within odd period counting forward from last interest date before redemption.
  - DC_i = number of days counted in the i^{th}, or last, quasi-coupon period as delimited by the length of the actual coupon period.
  - NC = number of quasi-coupon periods that fit in odd period; if this number contains a fraction it will be raised to the next whole number.
  - NL_i = normal length in days of the i^{th}, or last, quasi-coupon period within odd coupon period.
- settlement, maturity, last_interest are truncated to integers.
- basis and frequency are rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, last_interest is not a valid date.
  - maturity > settlement > last_interest is not satisfied.
  - rate < 0.
  - pr ≤ 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

Expand table

Data	Argument description
4/20/2008	Settlement date
6/15/2008	Maturity date
12/24/2007	Last interest date
3.75%	Percent coupon
$99.875	Price
$100	Redemption value
2	Frequency is semiannual
0	30/360 basis

DAX

```
EVALUATE
{
  ODDLYIELD(DATE(2008,4,20), DATE(2008,6,15), DATE(2007,12,24), 0.0375, 99.875, 100, 2, 0)
}
```

Returns the yield of a security that has an odd (short of long) last period, using the terms specified above.

Expand table

[Value]
0.0451922356291692

Feedback

Was this page helpful? Yes No

PDURATION

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of periods required by an investment to reach a specified value.

Syntax

```
DAX
PDURATION(<rate>, <pv>, <fv>)
```

Parameters

Expand table

Term	Definition
rate	The interest rate per period.
pv	The present value of the investment.
fv	The desired future value of the investment.

Return Value

The number of periods.

Remarks

- PDURATION uses the following equation:

$$\text{PDURATION} = \frac{log(\text{fv}) - log(\text{pv})}{log(1 + \text{rate})}$$

- An error is returned if:
  - rate ≤ 0.
  - pv ≤ 0.
  - fv ≤ 0.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following DAX query:

```
DAX

EVALUATE
{
  PDURATION(0.025, 2000, 2200)
}
```

Returns the number of years required for an investment of $2000, earning 2.5% annually, to reach $2200.

Expand table

[Value]
3.85986616262266

Example 2

The following DAX query:

```
DAX

EVALUATE
{
  PDURATION(0.025/12, 1000, 1200)
}
```

Returns the number of months required for an investment of $1000, earning 2.5% annually, to reach $1200.

Expand table

[Value]
87.6054764193714

Feedback

Was this page helpful? Yes No

PMT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Calculates the payment for a loan based on constant payments and a constant interest rate.

Syntax

DAX

```
PMT(<rate>, <nper>, <pv>[, <fv>[, <type>]])
```

Parameters

Expand table

Term	Definition
rate	The interest rate for the loan.
nper	The total number of payments for the loan.
pv	The present value, or the total amount that a series of future payments is worth now; also known as the principal.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be BLANK.
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period
1	At the beginning of the period

Note: For a more complete description of the arguments in PMT, see the PV function.

Return Value

The amount of a single loan payment.

Remarks

- The payment returned by PMT includes principal and interest but no taxes, reserve payments, or fees sometimes associated with loans.
- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at an annual interest rate of 12 percent, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.
- type is rounded to the nearest integer.
- An error is returned if:
  - nper < 1

Tip: To find the total amount paid over the duration of the loan, multiply the returned PMT value by nper.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Example 1

Expand table

Data	Description
8%	Annual interest rate
10	Number of months of payments
$10,000	Amount of loan

The following DAX query:

DAX

```
EVALUATE
{
  PMT(0.08/12, 10, 10000, 0, 1)
}
```

Returns the monthly payment amount, paid at the beginning of the month, for a loan with the terms specified above.

Expand table

[Value]
-1030.16432717797

Note: 1030.16432717797 is the payment per period. As a result, the total amount paid over the duration of the loan is approximately 1030.16 * 10 = $10,301.60. In other words, approximately $301.60 of interest is paid.

Example 2

Expand table

Data	Description
6%	Annual interest rate
18	Number of years of payments
$50,000	Amount of loan

The following DAX query:

DAX

```
EVALUATE
{
  PMT(0.06/12, 18*12, 0, 50000)
}
```

Expand table

[Value]
-129.081160867991

Returns the amount to save each month to have $50,000 at the end of 18 years, using the terms specified above.

Feedback

Was this page helpful? Yes No

PPMT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the payment on the principal for a given period for an investment based on periodic, constant payments and a constant interest rate.

Syntax

DAX

```
PPMT(<rate>, <per>, <nper>, <pv>[, <fv>[, <type>]])
```

Parameters

Expand table

Term	Definition
rate	The interest rate for the loan.
per	Specifies the period. Must be between 1 and nper (inclusive).
nper	The total number of payment periods in an annuity.
pv	The present value — the total amount that a series of future payments is worth now.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be BLANK.
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period
1	At the beginning of the period

Note: For a more complete description of the arguments in PPMT, see the PV function.

Return Value

The payment on the principal for a given period.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at an annual interest rate of 12 percent, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.
- type is rounded to the nearest integer.
- An error is returned if:
  - per < 1 or per > nper
  - nper < 1
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

Expand table

Data	Argument description
10%	Annual interest rate
2	Number of years for the loan
$2,000.00	Amount of loan

The following DAX query:

DAX

```
EVALUATE
{
  PPMT(0.1/12, 1, 2*12, 2000.00)
}
```

Returns the principal payment made in the first month for a loan with the terms specified above.

Expand table

[Value]
-75.6231860083663

Example 2

Expand table

Data	Argument description
8%	Annual interest rate
10	Number of years for the loan
$200,000.00	Amount of loan

The following DAX query:

```
DAX

EVALUATE
{
  PPMT(0.08, 10, 10, 200000.00)
}
```

Returns the principal payment made in the 10th year for a loan with the terms specified above.

Expand table

[Value]
-27598.0534624214

Feedback

Was this page helpful? Yes No

PRICE

Article • 10/20/2023

Returns the price per $100 face value of a security that pays periodic interest.

Syntax

DAX

```
PRICE(<settlement>, <maturity>, <rate>, <yld>, <redemption>, <frequency>[, <basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
rate	The security's annual coupon rate.
yld	The security's annual yield.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- basis and frequency are rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Important:

- When N > 1 (N is the number of coupons payable between the settlement date and redemption date), **PRICE** is calculated as follows:

$$\text{PRICE} = \left[\frac{\text{redemption}}{(1+\frac{\text{yld}}{\text{frequency}})^{(N-1+\frac{\text{DSC}}{\text{E}})}}\right] + \left[\sum_{k=1}^{N}\frac{100\times\frac{\text{rate}}{\text{frequency}}}{(1+\frac{\text{yld}}{\text{frequency}})^{(k-1+\frac{\text{DSC}}{\text{E}})}}\right] - \left[100\times\frac{\text{rate}}{\text{frequency}}\times\frac{\text{A}}{\text{E}}\right]$$

- When N = 1 (N is the number of coupons payable between the settlement date and redemption date), **PRICE** is calculated as follows:

$$\text{DSR} = \text{E} - \text{A}$$

$$\text{T1} = 100\times\frac{\text{rate}}{\text{frequency}} + \text{redemption}$$

$$\text{T2} = \frac{\text{yld}}{\text{frequency}}\times\frac{\text{DSR}}{\text{E}} + 1$$

$$\text{T3} = 100\times\frac{\text{rate}}{\text{frequency}}\times\frac{\text{A}}{\text{E}}$$

$$\text{PRICE} = \frac{\text{T1}}{\text{T2}} - \text{T3}$$

  where:
  - DSC = number of days from settlement to next coupon date.
  - E = number of days in coupon period in which the settlement date falls.
  - A = number of days from beginning of coupon period to settlement date.

Example

Data	Argument description
2/15/2008	Settlement date
11/15/2017	Maturity date
5.75%	Percent semiannual coupon
6.50%	Percent yield
$100	Redemption value
2	Frequency is semiannual
0	30/360 basis

The following DAX query:

```
DAX

EVALUATE
{
  PRICE(DATE(2008,2,15), DATE(2017,11,15), 0.0575, 0.065, 100, 2, 0)
}
```

Returns the bond price, for a bond using the terms specified above.

[Value]
94.6343616213221

Feedback

Was this page helpful? Yes No

PRICEDISC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the price per $100 face value of a discounted security.

Syntax

DAX

```
PRICEDISC(<settlement>, <maturity>, <discount>, <redemption>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
discount	The security's discount rate.
redemption	The security's redemption value per $100 face value.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360

Basis	Day count basis
3	Actual/365
4	European 30/360

Return Value

The price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2018, and is purchased by a buyer six months later. The issue date would be January 1, 2018, the settlement date would be July 1, 2018, and the maturity date would be January 1, 2048, 30 years after the January 1, 2018, issue date.
- PRICEDISC is calculated as follows:

$$\mathrm{PRICEDISC} = \mathrm{redemption} - \mathrm{discount} \times \mathrm{redemption} \times \frac{\mathrm{DSM}}{\mathrm{B}}$$

  where:
  - B = number of days in year, depending on year basis.
  - DSM = number of days from settlement to maturity.
- settlement and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - discount ≤ 0.
  - redemption ≤ 0.
  - basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Argument description
2/16/2008	Settlement date
3/1/2008	Maturity date
5.25%	Percent discount rate
$100	Redemption value
2	Actual/360 basis

The following DAX query:

```
DAX

EVALUATE
{
  PRICEDISC(DATE(2008,2,16), DATE(2008,3,1), 0.0525, 100, 2)
}
```

Returns the bond price per $100 face value, for a bond with the terms specified above.

Expand table

[Value]
99.7958333333333

Feedback

Was this page helpful? Yes No

PRICEMAT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the price per $100 face value of a security that pays interest at maturity.

Syntax

DAX

```
PRICEMAT(<settlement>, <maturity>, <issue>, <rate>, <yld>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
issue	The security's issue date.
rate	The security's interest rate at date of issue.
yld	The security's annual yield.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual

Basis	Day count basis
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- PRICEMAT is calculated as follows:

$$\text{PRICEMAT} = \frac{100 + (\frac{\text{DIM}}{\text{B}} \times \text{rate} \times 100)}{1 + (\frac{\text{DSM}}{\text{B}} \times \text{yld})} - (\frac{\text{A}}{\text{B}} \times \text{rate} \times 100)$$

  where:
  - B = number of days in year, depending on year basis.
  - DSM = number of days from settlement to maturity.
  - DIM = number of days from issue to maturity.
  - A = number of days from issue to settlement.
- settlement, maturity, and issue are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, or issue is not a valid date.
  - maturity > settlement > issue is not satisfied.

- rate < 0.
- yld < 0.
- basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

Expand table

Data	Description
2/15/2008	Settlement date
4/13/2008	Maturity date
11/11/2007	Issue date
6.10%	Percent semiannual coupon
6.10%	Percent yield
0	30/360 basis

DAX

```
EVALUATE
{
  PRICEMAT(DATE(2008,2,15), DATE(2008,4,13), DATE(2007,11,11), 0.061, 0.061,
0)
}
```

Returns the price per $100 face value of a security with the terms specified above.

Expand table

[Value]
99.9844988755569

Feedback

Was this page helpful? Yes No

PV

Article • 10/20/2023

Calculates the present value of a loan or an investment, based on a constant interest rate. You can use PV with either periodic, constant payments (such as a mortgage or other loan), and/or a future value that's your investment goal.

Syntax

DAX

```
PV(<rate>, <nper>, <pmt>[, <fv>[, <type>]])
```

Parameters

Term	Definition
rate	The interest rate per period. For example, if you obtain an automobile loan at a 10 percent annual interest rate and make monthly payments, your interest rate per month is 0.1/12, or 0.0083. You would enter 0.1/12, or 0.0083, into the formula as the rate.
nper	The total number of payment periods in an annuity. For example, if you get a four-year car loan and make monthly payments, your loan has 4*12 (or 48) periods. You would enter 48 into the formula for nper.
pmt	The payment made each period that cannot change over the life of the annuity. Typically, pmt includes principal and interest but no other fees or taxes. For example, the monthly payments on a $10,000, four-year car loan at 12 percent are $263.33. You would enter -263.33 into the formula as the pmt.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be BLANK. For example, if you want to save $50,000 to pay for a special project in 18 years, then $50,000 is the future value. You could then make a conservative guess at an interest rate and determine how much you must save each month.
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **type** parameter accepts the following values:

Set type equal to	If payments are due
0 or omitted	At the end of the period

Set type equal to	If payments are due
1	At the beginning of the period

Return Value

The present value of a loan or investment.

Remarks

- Make sure that you are consistent about the units you use for specifying rate and nper. If you make monthly payments on a four-year loan at 12 percent annual interest, use 0.12/12 for rate and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for rate and 4 for nper.

- The following functions apply to annuities:
  - CUMIPMT
  - CUMPRINC
  - FV
  - IPMT
  - PMT
  - PPMT
  - PV
  - RATE
  - XIRR
  - XNPV

- An annuity is a series of constant cash payments made over a continuous period. For example, a car loan or a mortgage is an annuity. For more information, see the description for each annuity function.

- In annuity functions, cash you pay out, such as a deposit to savings, is represented by a negative number; cash you receive, such as a dividend check, is represented by a positive number. For example, a $1,000 deposit to the bank would be represented by the argument -1000 if you are the depositor and by the argument 1000 if you are the bank.

- One financial argument is solved in terms of the others.

  - If rate is not 0, then:

$$\text{pv} \times (1 + \text{rate})^{\text{nper}} + \text{pmt}(1 + \text{rate} \times \text{type}) \times \left(\frac{(1 + \text{rate})^{\text{nper}} - 1}{\text{rate}} \right) + \text{fv} = 0$$

  - If rate is 0, then:

$$(\text{pmt} \times \text{nper}) + \text{pv} + \text{fv} = 0$$

- type is rounded to the nearest integer.
- An error is returned if:
  - nper < 1 or blank
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Description
$500.00	Money paid out of an insurance annuity at the end of every month.
8%	Interest rate earned on the money paid out.
20	Years the money will be paid out.

The following DAX query:

```
DAX

EVALUATE
{
  PV(0.08/12, 12*20, 500.00, 0, 0)
}
```

Returns the present value of an annuity using the terms specified above.

[Value]
-59777.1458511878

Feedback

Was this page helpful? Yes No

RATE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the interest rate per period of an annuity. RATE is calculated by iteration and can have zero or more solutions. If the successive results of RATE do not converge to within 0.0000001 after 20 iterations, an error is returned.

Syntax

DAX

```
RATE(<nper>, <pmt>, <pv>[, <fv>[, <type>[, <guess>]]])
```

Parameters

Expand table

Term	Definition
nper	The total number of payment periods in an annuity.
pmt	The payment made each period and cannot change over the life of the annuity. Typically, pmt includes principal and interest but no other fees or taxes.
pv	The present value — the total amount that a series of future payments is worth now.
fv	(Optional) The future value, or a cash balance you want to attain after the last payment is made. If fv is omitted, it is assumed to be 0 (the future value of a loan, for example, is 0).
type	(Optional) The number 0 or 1 which indicates when payments are due. If type is omitted, it is assumed to be 0. The accepted values are listed below this table.
guess	(Optional) Your guess for what the rate will be. - If omitted, it is assumed to be 10%. - If RATE does not converge, try different values for guess. RATE usually converges if guess is between 0 and 1.

The **type** parameter accepts the following values:

Expand table

Set type equal to	If payments are due
0 or omitted	At the end of the period
1	At the beginning of the period

Return Value

The interest rate per period.

Remarks

- Make sure that you are consistent about the units you use for specifying guess and nper. If you make monthly payments on a four-year loan at 12 percent annual interest, use 0.12/12 for guess and 4*12 for nper. If you make annual payments on the same loan, use 0.12 for guess and 4 for nper.
- type is rounded to the nearest integer.
- An error is returned if:
  - nper ≤ 0.
  - RATE does not converge to within 0.0000001 after 20 iterations
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
4	Years of the loan
-200	Monthly payment
8000	Amount of the loan

Example 1

The following DAX query:

```
```

```
DAX

EVALUATE
{
  RATE(4*12, -200, 8000)
}
```

Returns the monthly rate of the loan using the terms specified above.

Expand table

[Value]
0.00770147248820137

Example 2

The following DAX query:

```
DAX

EVALUATE
{
  RATE(4*12, -200, 8000) * 12
}
```

Returns the annual rate of the loan using the terms specified above.

Expand table

[Value]
0.0924176698584164

Feedback

Was this page helpful? Yes No

RECEIVED

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the amount received at maturity for a fully invested security.

Syntax

```
DAX
RECEIVED(<settlement>, <maturity>, <investment>, <discount>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
investment	The amount invested in the security.
discount	The security's discount rate.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360

Basis	Day count basis
3	Actual/365
4	European 30/360

Return Value

The amount received at maturity.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- RECEIVED is calculated as follows:

$$\text{RECEIVED} = \frac{\text{investment}}{1 - (\text{discount} \times \frac{\text{DIM}}{\text{B}})}$$

  where:
  - B = number of days in a year, depending on the year basis.
  - DIM = number of days from issue to maturity.
- settlement and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - investment ≤ 0.
  - discount ≤ 0.
  - basis < 0 or basis > 4.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

Expand table

Data	Description
15-Feb-08	Settlement (issue) date
15-May-08	Maturity date
$1,000,000.00	Investment
5.75%	Percent discount rate
2	Actual/360 basis

DAX

```
EVALUATE
{
  RECEIVED(DATE(2008,2,15), DATE(2008,5,15), 1000000.00, 0.0575, 2)
}
```

Returns the total amount to be received at maturity, for a bond with the terms specified above.

Expand table

[Value]
1014584.6544071

Feedback

Was this page helpful? Yes No

RRI

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns an equivalent interest rate for the growth of an investment.

Syntax

DAX

```
RRI(<nper>, <pv>, <fv>)
```

Parameters

Expand table

Term	Definition
nper	The number of periods for the investment.
pv	The present value of the investment.
fv	The future value of the investment.

Return Value

The equivalent interest rate.

Remarks

- RRI returns the interest rate given nper (the number of periods), pv (present value), and fv (future value), calculated by using the following equation:

$$\left(\frac{\mathrm{fv}}{\mathrm{pv}}\right)^{\left(\frac{1}{nper}\right)} - 1$$

- An error is returned if:
  - nper ≤ 0.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
$10,000	Present value
$21,000	Future value
4	Years invested

The following DAX query:

```
DAX

EVALUATE
{
  RRI(4*12, 10000, 21000)
}
```

Returns an equivalent interest rate for the growth of an investment with the terms specified above.

Expand table

[Value]
0.0155771057566627

Feedback

Was this page helpful? Yes No

SLN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the straight-line depreciation of an asset for one period.

Syntax

```
DAX
SLN(<cost>, <salvage>, <life>)
```

Parameters

Expand table

Term	Definition
cost	The initial cost of the asset.
salvage	The value at the end of the depreciation (sometimes called the salvage value of the asset).
life	The number of periods over which the asset is depreciated (sometimes called the useful life of the asset).

Return Value

The straight-line depreciation for one period.

Remarks

- An error is returned if:
  life = 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
$30,000	Cost
$7,500	Salvage value
10	Years of useful life

The following DAX query:

```
DAX

EVALUATE
{
  SLN(30000, 7500, 10)
}
```

Returns the yearly depreciation allowance using the terms specified above.

Expand table

[Value]
2250

Feedback

Was this page helpful? Yes No

SYD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the sum-of-years' digits depreciation of an asset for a specified period.

Syntax

DAX

```
SYD(<cost>, <salvage>, <life>, <per>)
```

Parameters

Expand table

Term	Definition
cost	The initial cost of the asset.
salvage	The value at the end of the depreciation (sometimes called the salvage value of the asset).
life	The number of periods over which the asset is depreciated (sometimes called the useful life of the asset).
per	The period. Must use the same units as life. Must be between 1 and life (inclusive).

Return Value

The sum-of-years' digits depreciation for the specified period.

Remarks

- SYD is calculated as follows:

$$\text{SYD} = \frac{(\text{cost} - \text{salvage}) \times (\text{life} - \text{per} + 1) \times 2}{(\text{life}) \times (\text{life} + 1)}$$

- An error is returned if:
  - life < 1.
  - per < 1 or per > life.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
$30,000.00	Initial cost
$7,500.00	Salvage value
10	Lifespan in years

Example 1

The following DAX query:

```
DAX

EVALUATE
{
  SYD(30000.00, 7500.00, 10, 1)
}
```

Returns an asset's sum-of-years' digits depreciation allowance for the first year, given the terms specified above.

Expand table

[Value]
4090.90909090909

Example 2

The following DAX query:

DAX

```
EVALUATE
{
  SYD(30000.00, 7500.00, 10, 10)
}
```

Returns an asset's sum-of-years' digits depreciation allowance for the tenth (final) year, given the terms specified above.

Expand table

[Value]
409.090909090909

Feedback

Was this page helpful? Yes No

TBILLEQ

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the bond-equivalent yield for a Treasury bill.

Syntax

DAX

```
TBILLEQ(<settlement>, <maturity>, <discount>)
```

Parameters

Expand table

Term	Definition
settlement	The Treasury bill's settlement date. The security settlement date is the date after the issue date when the Treasury bill is traded to the buyer.
maturity	The Treasury bill's maturity date. The maturity date is the date when the Treasury bill expires.
discount	The Treasury bill's discount rate.

Return Value

The Treasury Bill's bond-equivalent yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- TBILLEQ is calculated as:

$$\text{TBILLEQ} = \frac{365 \times \text{discount}}{360 - (\text{discount} \times \text{DSM})}$$

where:

- DSM is the number of days between settlement and maturity computed according to the 360 days per year basis.

- settlement and maturity are truncated to integers.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity or maturity is more than one year after settlement.
  - discount ≤ 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
3/31/2008	Settlement date
6/1/2008	Maturity date
9.14%	Percent discount rate

The following DAX query:

```
DAX

EVALUATE
{
  TBILLEQ(DATE(2008,3,31), DATE(2008,6,1), 0.0914)
}
```

Returns the bond-equivalent yield for a Treasury bill using the terms specified above.

Expand table

[Value]
0.094151493565943

Feedback

Was this page helpful? Yes No

TBILLPRICE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the price per $100 face value for a Treasury bill.

Syntax

```
DAX
TBILLPRICE(<settlement>, <maturity>, <discount>)
```

Parameters

Expand table

Term	Definition
settlement	The Treasury bill's settlement date. The security settlement date is the date after the issue date when the Treasury bill is traded to the buyer.
maturity	The Treasury bill's maturity date. The maturity date is the date when the Treasury bill expires.
discount	The Treasury bill's discount rate.

Return Value

The Treasury Bill's price per $100 face value.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- TBILLPRICE is calculated as follows:

$$\text{TBILLPRICE} = 100 \times (1 - \frac{\text{discount} \times \text{DSM}}{360})$$

where:

- DSM = number of days from settlement to maturity, excluding any maturity date that is more than one calendar year after the settlement date.

- settlement and maturity are truncated to integers.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity or maturity is more than one year after settlement.
  - discount ≤ 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
3/31/2008	Settlement date
6/1/2008	Maturity date
9.0%	Percent discount rate

The following DAX query:

```
DAX

EVALUATE
{
  TBILLPRICE(DATE(2008,3,31), DATE(2008,6,1), 0.09)
}
```

Returns the Treasury Bill's price per $100 face value, given the terms specified above.

Expand table

[Value]
98.45

Feedback

Was this page helpful? Yes No

TBILLYIELD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the yield for a Treasury bill.

Syntax

DAX

```
TBILLYIELD(<settlement>, <maturity>, <pr>)
```

Parameters

Expand table

Term	Definition
settlement	The Treasury bill's settlement date. The security settlement date is the date after the issue date when the Treasury bill is traded to the buyer.
maturity	The Treasury bill's maturity date. The maturity date is the date when the Treasury bill expires.
pr	The Treasury bill's price per $100 face value.

Return Value

The Treasury bill's yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- TBILLYIELD is calculated as follows:

$$\text{TBILLYIELD} = \frac{100 - \text{pr}}{\text{pr}} \times \frac{360}{\text{DSM}}$$

where:

- DSM = number of days from settlement to maturity, excluding any maturity date that is more than one calendar year after the settlement date.

- settlement and maturity are truncated to integers.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity or maturity is more than one year after settlement.
  - pr ≤ 0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

Expand table

Data	Description
3/31/2008	Settlement date
6/1/2008	Maturity date
$98.45	Price per $100 face value

DAX

```
EVALUATE
{
  TBILLYIELD(DATE(2008,3,31), DATE(2008,6,1), 98.45)
}
```

Returns the yield of a Treasury bill using the terms specified above.

Expand table

[Value]
0.0914169629253426

Feedback

Was this page helpful? Yes No

VDB

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the depreciation of an asset for any period you specify, including partial periods, using the double-declining balance method or some other method you specify. VDB stands for variable declining balance.

Syntax

DAX

```
VDB(<cost>, <salvage>, <life>, <start_period>, <end_period>[, <factor>[, <no_switch>]])
```

Parameters

Expand table

Term	Definition
cost	The initial cost of the asset.
salvage	The value at the end of the depreciation (sometimes called the salvage value of the asset). This value can be 0.
life	The number of periods over which the asset is being depreciated (sometimes called the useful life of the asset).
start_period	The starting period for which you want to calculate the depreciation. Start_period must use the same units as life. Must be between 1 and life (inclusive).
end_period	The ending period for which you want to calculate the depreciation. End_period must use the same units as life. Must be between start_period and life (inclusive).
factor	(Optional) The rate at which the balance declines. If factor is omitted, it is assumed to be 2 (the double-declining balance method). Change factor if you do not want to use the double-declining balance method. For a description of the double-declining balance method, see DDB.
no_switch	(Optional) A logical value specifying whether to switch to straight-line depreciation when depreciation is greater than the declining balance calculation. If omitted, it is assumed to be FALSE.

Term	Definition
	- If no_switch evaluates to TRUE, VDB does not switch to straight-line depreciation, even when the depreciation is greater than the declining balance calculation. - If no_switch evaluates to FALSE or is omitted, VDB switches to straight-line depreciation when depreciation is greater than the declining balance calculation.

Return Value

The depreciation over the specified period.

Remarks

- An error is returned if:
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - start_period < 1 or start_period > end_period.
  - end_period < start_period or end_period > life.
  - factor < 0.
  - no_switch does not evaluate to either TRUE or FALSE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Expand table

Data	Description
2400	Initial cost
300	Salvage value
10	Lifetime in years

Example 1

The following DAX query:

```
DAX
```

```
EVALUATE
{
  VDB(2400, 300, 10*365, 0, 1)
}
```

Returns an asset's first day's depreciation using a factor of 2.

Expand table

[Value]
1.31506849315068

Example 2

The following DAX query:

DAX

```
EVALUATE
{
  VDB(2400, 300, 10*12, 6, 18, 3)
}
```

Returns an asset's depreciation between the 6^{th} month and the 18^{th} month. This calculation uses a factor of 3.

Expand table

[Value]
540.185558199698

Example 3

The following DAX query:

DAX

```
EVALUATE
{
  VDB(2400, 300, 10, 0, 0.875, 1.5)
}
```

Returns an asset's depreciation in the first fiscal year that you own it, assuming that tax laws limit you to 150% depreciation of the declining balance. The asset is purchased in the middle of the first quarter of the fiscal year.

Expand table

[Value]
315

Feedback

Was this page helpful? Yes No

XIRR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the internal rate of return for a schedule of cash flows that is not necessarily periodic.

Syntax

DAX

```
XIRR(<table>, <values>, <dates>, [, <guess>[, <alternateResult>]])
```

Parameters

Expand table

Term	Definition
table	A table for which the values and dates expressions should be calculated.
values	An expression that returns the cash flow value for each row of the table.
dates	An expression that returns the cash flow date for each row of the table.
guess	(Optional) An initial guess for the internal rate of return. If omitted, the default guess of 0.1 is used.
alternateResult	(Optional) A value returned in place of an error when a solution cannot be determined.

Return value

Internal rate of return for the given inputs. If the calculation fails to return a valid result, an error or value specified as alternateResult is returned.

Remarks

- The value is calculated as the rate that satisfies the following function:

$$0 = \sum_{j=1}^{N} \frac{P_j}{(1 + \text{rate})^{\frac{d_j - d_1}{365}}}$$

Where:

- P_j is the j^{th} payment
- d_j is the j^{th} payment date
- d_1 is the first payment date

- The series of cash flow values must contain at least one positive number and one negative number.
- Avoid using ISERROR or IFERROR functions to capture an error returned by XIRR. If some inputs to the function may result in a no solution error, providing an alternateResult parameter is the most reliable and highest performing way to handle the error.
- To learn more about using the alternateResult parameter, be to check out this video.
- When the absolute value of initial payment is small, the calculation likely fails to return a valid result.
- Avoid including 0 values in payments. They will not impact the final result, and using 0 as initial payment will fail XIRR() calculation always.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula calculates the internal rate of return of the CashFlows table:

DAX

```
= XIRR( CashFlows, [Payment], [Date] )
```

Expand table

Date	Payment
1/1/2014	-10000
3/1/2014	2750

Date	Payment
10/30/2014	4250
2/15/2015	3250
4/1/2015	2750

Rate of return = 37.49%

Feedback

Was this page helpful? Yes No

XNPV

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the present value for a schedule of cash flows that is not necessarily periodic.

Syntax

DAX

```
XNPV(<table>, <values>, <dates>, <rate>)
```

Parameters

Expand table

Term	Definition
table	A table for which the values and dates expressions should be calculated.
values	An expression that returns the cash flow value for each row of the table.
dates	An expression that returns the cash flow date for each row of the table.
rate	The discount rate to apply to the cash flow for each row of the table.

Return value

Net present value.

Remarks

- The value is calculated as the following summation:

$$\sum_{j=1}^{N} \frac{P_j}{(1+\text{rate})^{\frac{d_j - d_1}{365}}}$$

  Where:
  - P_j is the j^{th} payment

- d_j is the j^{th} payment date
- d_1 is the first payment date

- The series of cash flow values must contain at least one positive number and one negative number.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following calculates the present value of the CashFlows table:

```
DAX
= XNPV( CashFlows, [Payment], [Date], 0.09 )
```

Expand table

Date	Payment
1/1/2014	-10000
3/1/2014	2750
10/30/2014	4250
2/15/2015	3250
4/1/2015	2750

Present value = 2086.65

Feedback

Was this page helpful? Yes No

YIELD

Article • 10/20/2023

Returns the yield on a security that pays periodic interest. Use YIELD to calculate bond yield.

Syntax

DAX

```
YIELD(<settlement>, <maturity>, <rate>, <pr>, <redemption>, <frequency>[,
<basis>])
```

Parameters

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
rate	The security's annual coupon rate.
pr	The security's price per $100 face value.
redemption	The security's redemption value per $100 face value.
frequency	The number of coupon payments per year. For annual payments, frequency = 1; for semiannual, frequency = 2; for quarterly, frequency = 4.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360
3	Actual/365

Basis	Day count basis
4	European 30/360

Return Value

The yield on the security.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- If there is one coupon period or less until redemption, YIELD is calculated as follows:

$$\text{YIELD} = \frac{(\frac{\text{redemption}}{100} + \frac{\text{rate}}{\text{frequency}}) - (\frac{\text{par}}{100} + (\frac{\text{A}}{\text{E}} \times \frac{\text{rate}}{\text{frequency}}))}{\frac{\text{par}}{100} + (\frac{\text{A}}{\text{E}} \times \frac{\text{rate}}{\text{frequency}})} \times \frac{\text{frequency} \times \text{E}}{\text{DSR}}$$

  where:
  - A = number of days from the beginning of the coupon period to the settlement date (accrued days).
  - DSR = number of days from the settlement date to the redemption date.
  - E = number of days in the coupon period.
- If there is more than one coupon period until redemption, YIELD is calculated through a hundred iterations. The resolution uses the Newton method, based on the formula used for the function PRICE. The yield is changed until the estimated price given the yield is close to price.
- settlement and maturity are truncated to integers.
- frequency, and basis are rounded to the nearest integer.

- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - rate < 0.
  - pr ≤ 0.
  - redemption ≤ 0.
  - frequency is any number other than 1, 2, or 4.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Data	Description
15-Feb-08	Settlement date
15-Nov-16	Maturity date
5.75%	Percent coupon
95.04287	Price
$100	Redemption value
2	Frequency is semiannual (see above)
0	30/360 basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  YIELD(DATE(2008,2,15), DATE(2016,11,15), 0.0575, 95.04287, 100, 2,0)
}
```

Returns the yield on a bond with the terms specified above.

[Value]
0.0650000068807314

Feedback

Was this page helpful? Yes No

YIELDDISC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the annual yield for a discounted security.

Syntax

DAX

```
YIELDDISC(<settlement>, <maturity>, <pr>, <redemption>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
pr	The security's price per $100 face value.
redemption	The security's redemption value per $100 face value.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual
2	Actual/360

Basis	Day count basis
3	Actual/365
4	European 30/360

Return Value

The annual yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement and maturity are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement or maturity is not a valid date.
  - settlement ≥ maturity.
  - pr ≤ 0.
  - redemption ≤ 0.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	-
16-Feb-08	Settlement date
1-Mar-08	Maturity date
99.795	Price
$100	Redemption value
2	Actual/360 basis

The following DAX query:

```
DAX

EVALUATE
{
  YIELDDISC(DATE(2008,2,16), DATE(2008,3,1), 99.795, 100, 2)
}
```

Returns the security's annual yield, given the terms specified above.

Expand table

[Value]
0.0528225719868583

Feedback

Was this page helpful? Yes No

YIELDMAT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the annual yield of a security that pays interest at maturity.

Syntax

DAX

```
YIELDMAT(<settlement>, <maturity>, <issue>, <rate>, <pr>[, <basis>])
```

Parameters

Expand table

Term	Definition
settlement	The security's settlement date. The security settlement date is the date after the issue date when the security is traded to the buyer.
maturity	The security's maturity date. The maturity date is the date when the security expires.
issue	The security's issue date.
rate	The security's interest rate at date of issue.
pr	The security's price per $100 face value.
basis	(Optional) The type of day count basis to use. If basis is omitted, it is assumed to be 0. The accepted values are listed below this table.

The **basis** parameter accepts the following values:

Expand table

Basis	Day count basis
0 or omitted	US (NASD) 30/360
1	Actual/actual

Basis	Day count basis
2	Actual/360
3	Actual/365
4	European 30/360

Return Value

The annual yield.

Remarks

- Dates are stored as sequential serial numbers so they can be used in calculations. In DAX, December 30, 1899 is day 0, and January 1, 2008 is 39448 because it is 39,448 days after December 30, 1899.
- The settlement date is the date a buyer purchases a coupon, such as a bond. The maturity date is the date when a coupon expires. For example, suppose a 30-year bond is issued on January 1, 2008, and is purchased by a buyer six months later. The issue date would be January 1, 2008, the settlement date would be July 1, 2008, and the maturity date would be January 1, 2038, which is 30 years after the January 1, 2008, issue date.
- settlement, maturity, and issue are truncated to integers.
- basis is rounded to the nearest integer.
- An error is returned if:
  - settlement, maturity, or issue is not a valid date.
  - maturity > settlement > issue is not satisfied.
  - rate < 0.
  - pr ≤ 0.
  - basis < 0 or basis > 4.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Data	Description
15-Mar-08	Settlement date
3-Nov-08	Maturity date
8-Nov-07	Issue date
6.25%	Percent semiannual coupon
100.0123	Price
0	30/360 basis (see above)

The following DAX query:

```
DAX

EVALUATE
{
  YIELDMAT(DATE(2008,3,15), DATE(2008,11,3), DATE(2007,11,8), 0.0625,
100.0123, 0)
}
```

Returns the yield for a security using the terms specified above.

Expand table

[Value]
0.0609543336915387

Feedback

Was this page helpful? Yes No

INFO functions

Data Analysis Expressions (DAX) includes a set of INFO functions based on the library of Dynamic Management Views (DMVs) in Analysis Services, which have been modified to work as DAX functions. INFO DAX functions output as a table data type. As tables they can be used with other DAX functions such as FILTER, SELECTCOLUMNS, ADDCOLUMNS, and others. This section describes INFO functions available in the DAX language.

For Power BI semantic models, just like DMVs, the INFO DAX functions require semantic model admin permissions. Some also require workspace admin permissions.

As DAX functions these INFO functions go beyond the capability of the DMVs, which use a SQL-like syntax and return a row set. As these new functions are DAX functions, they can be used like any table in a DAX query – further combined and structured in the DAX query.

DAX

```
EVALUATE
    ADDCOLUMNS(
        SELECTCOLUMNS(
            INFO.VIEW.MEASURES(),
            "Measure", [Name],
            [Description],
            "DAX Formula", [Expression],
            "State", [State]
        ),
        "Model name", "My semantic model",
        "As of date", NOW()
    )
```

INFO functions are supported on Power BI semantic models, SQL Server Analysis Services models, Azure Analysis Services models, and PowerPivot models. Errors may happen if the INFO function is for a feature not supported or available. INFO.VIEW DAX functions can be run in calculated tables, columns, measures, and DAX queries, but other INFO DAX functions can only be run in DAX queries.

INFO.VIEW DAX functions

Four of the INFO DAX functions also have INFO.VIEW DAX functions counterparts. Friendly names, such as table name instead of table ID, are used and new columns added to make it easier to use without the need to join other INFO DAX function tables. INFO.VIEW DAX functions can be used in calculations inside a semantic model. When included in a calculated table they can self-document a model for other people using it to build reports or DAX queries.

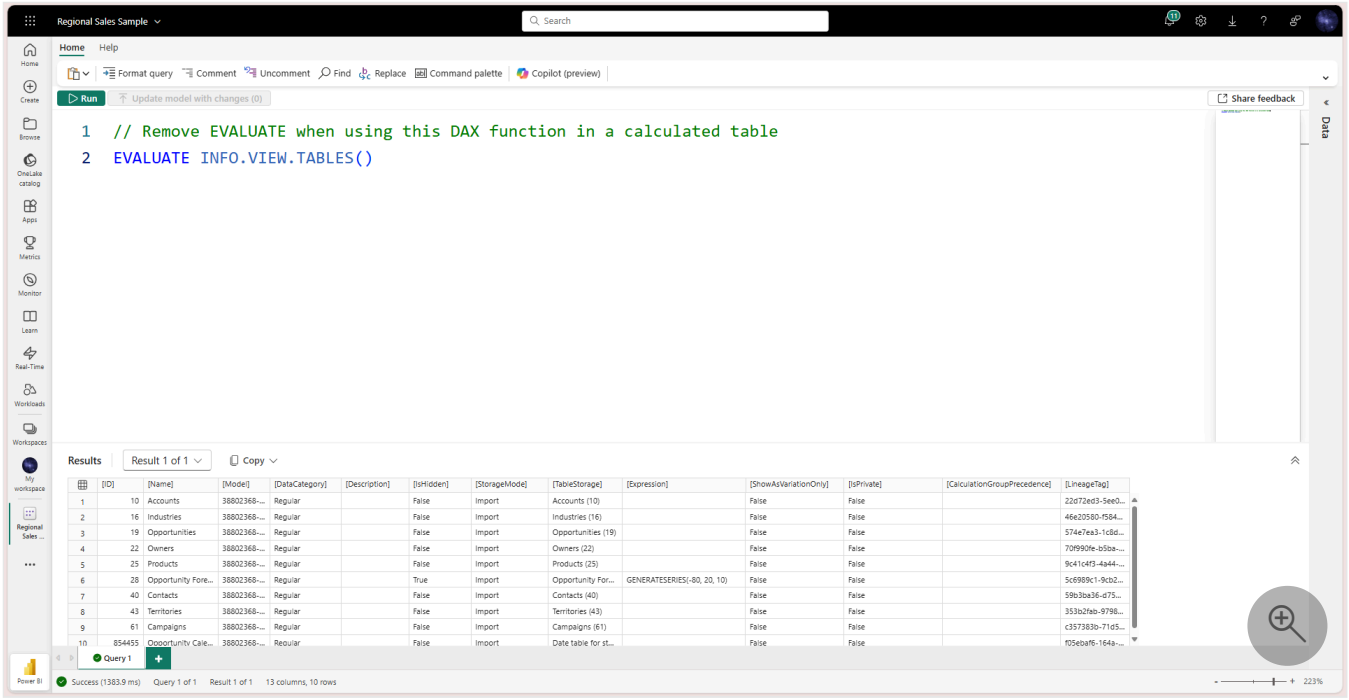
INFO.VIEW.TABLES

INFO.VIEW.TABLES contains information about the tables in the model, such as the table name, description, storage mode, and whether it is hidden or not.

DAX

```
// Remove EVALUATE when using this DAX function in a calculated table
EVALUATE INFO.VIEW.TABLES()
```

The following screenshot is an example of using this DAX function in DAX query view with the **Regional Sales Sample** available from **Learn** in the Power BI service.



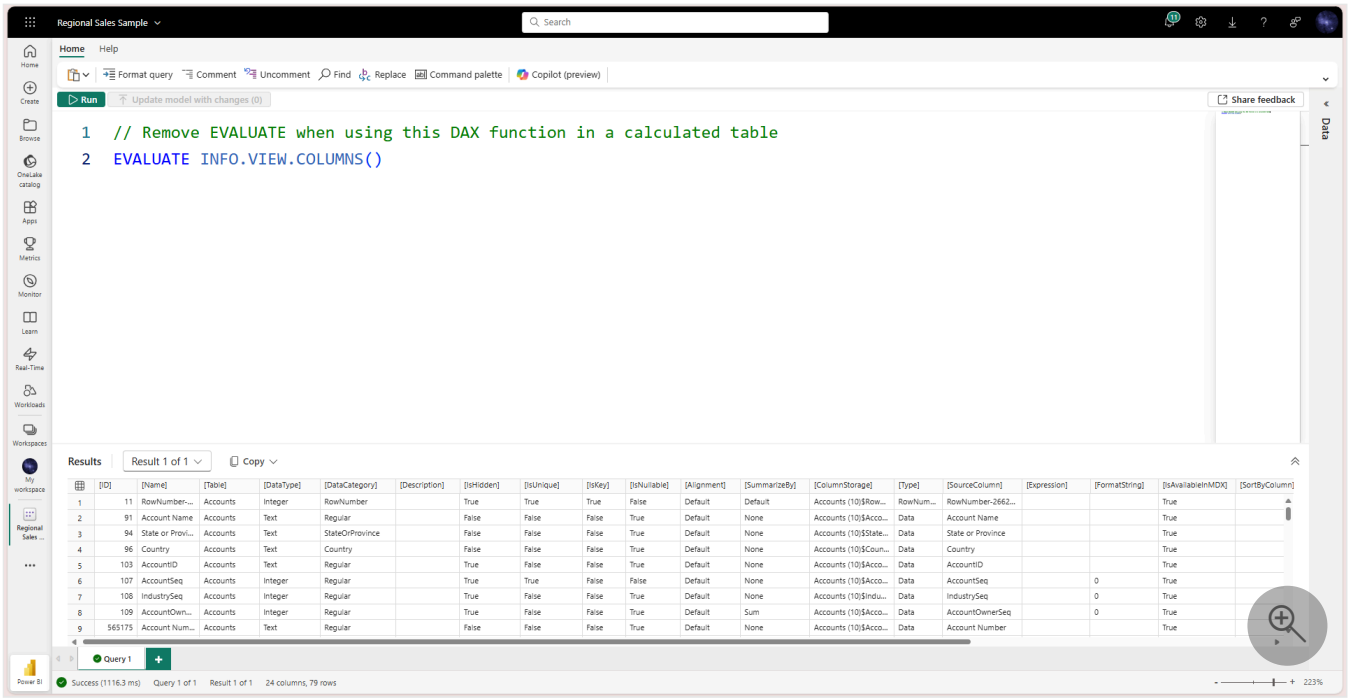
INFO.VIEW.COLUMNS

INFO.VIEW.COLUMNS contains information about the columns in a model, such as the column name, data type, and whether it is hidden or not.

DAX

```
// Remove EVALUATE when using this DAX function in a calculated table
EVALUATE INFO.VIEW.COLUMNS()
```

The following screenshot is an example of using this DAX function in DAX query view with the **Regional Sales Sample** available from **Learn** in the Power BI service.



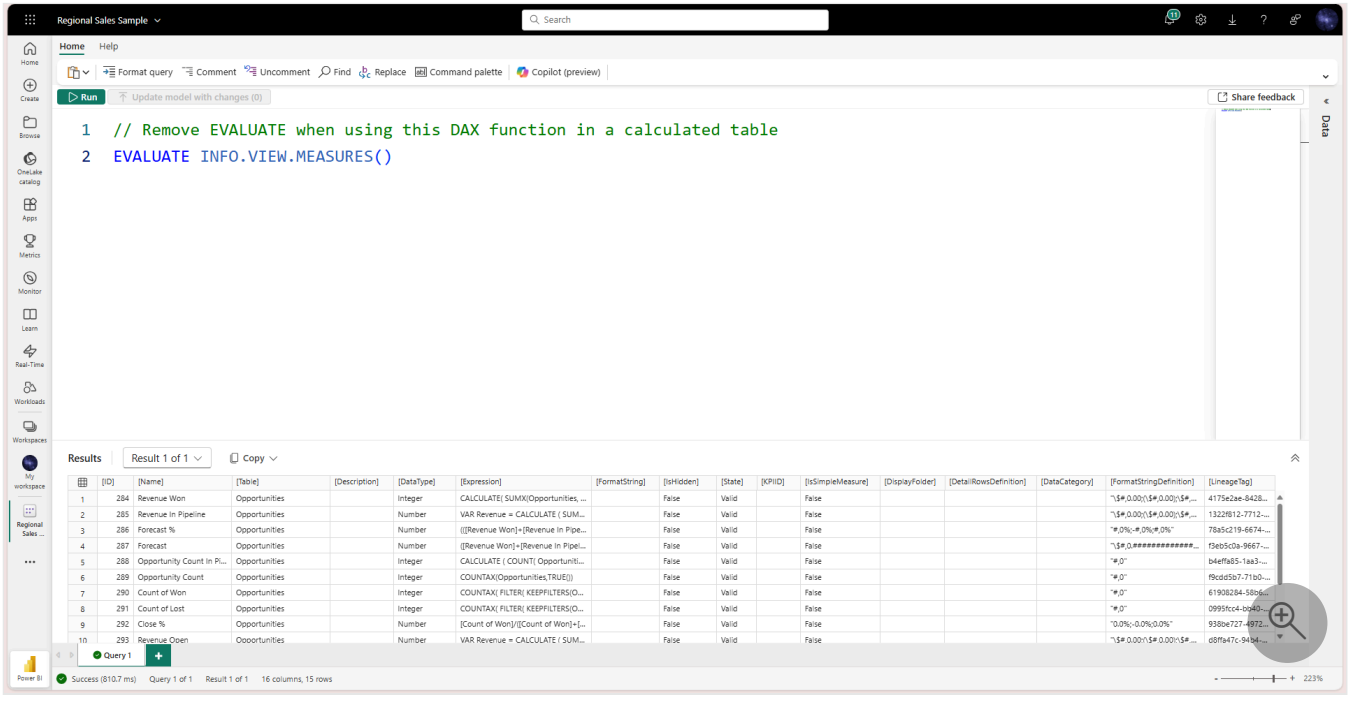
INFO.VIEW.MEASURES

INFO.VIEW.MEASURES contains information about the measures in the model, such as the measure name, expression, and format string.

DAX

```
// Remove EVALUATE when using this DAX function in a calculated table
EVALUATE INFO.VIEW.MEASURES()
```

The following screenshot is an example of using this DAX function in DAX query view with the **Regional Sales Sample** available from **Learn** in the Power BI service.



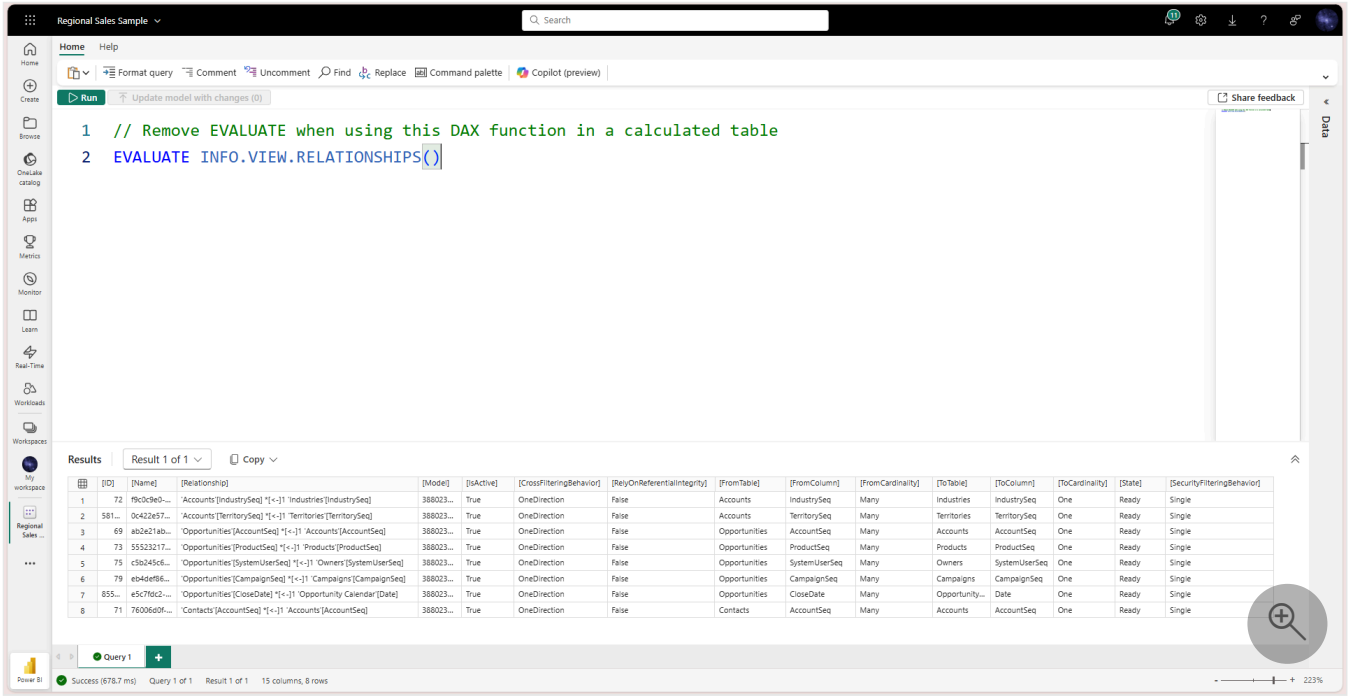
INFO.VIEW.RELATIONSHIPS

INFO.VIEW.RELATIONSHIPS contains information about the relationships in the model, such as the to and from table and columns, cardinality, and cross filter direction.

DAX

```
// Remove EVALUATE when using this DAX function in a calculated table
EVALUATE INFO.VIEW.RELATIONSHIPS()
```

The following screenshot is an example of using this DAX function in DAX query view with the **Regional Sales Sample** available from **Learn** in the Power BI service.



INFO DAX functions

Here is the list of INFO functions. Some work only on specific compat levels and for certain storage modes or other semantic model properties.

Expand table

[Function]	[Description]
INFO.VIEW.COLUMNS	Returns a list of all columns in the current model. Can use in calculations, including calculated tables.
INFO.VIEW.MEASURES	Returns a list of all measures in the current model. Can use in calculations, including calculated tables.

[Function]	[Description]
INFO.VIEW.RELATIONSHIPS	Returns a list of all relationships in the current model. Can use in calculations, including calculated tables.
INFO.VIEW.TABLES	Returns a list of all tables in the current model. Can use in calculations, including calculated tables.
INFO.ALTERNATEOFDEFINITIONS	Returns a table with information about each alternate of definition in the semantic model with metadata about alternate definitions for model objects.
INFO.ANNOTATIONS	Returns a list of all annotations in the current model with columns matching the schema rowset for annotation objects.
INFO.ATTRIBUTEHIERARCHIES	Represents the TMSCHEMA_ATTRIBUTE_HIERARCHIES DMV query function.
INFO.ATTRIBUTEHIERARCHYSTORAGES	Returns a table with information about each attribute hierarchy storage in the semantic model with metadata about the storage characteristics of attribute hierarchies.
INFO.CALCDEPENDENCY	Returns information about the calculation dependency for a DAX query.
INFO.CALCULATIONGROUPS	Returns a table with information about each calculation group in the semantic model with metadata about calculation groups and their properties.
INFO.CALCULATIONITEMS	Returns a table with information about each calculation item in the semantic model with metadata about calculation items within calculation groups.
INFO.CATALOGS	Represents the DBSCHEMA_CATALOGS DMV query function.
INFO.CHANGEDPROPERTIES	Represents the TMSCHEMA_CHANGED_PROPERTIES DMV query function.
INFO.COLUMNPARTITIONSTORAGES	Returns a table with information about each column partition storage in the

[Function]	[Description]
	semantic model with metadata about how column partitions are stored.
INFO.COLUMNPERMISSIONS	Returns a list of all column permissions in the current model with columns matching the schema rowset for column permissions objects.
INFO.COLUMNS	Returns a list of all columns in the current model with columns matching the schema rowset for column objects.
INFO.COLUMNSTORAGES	Returns a list of all column storages in the current model with columns matching the schema rowset for column storage objects.
INFO.CSDLMETADATA	Returns information about database metadata in XML format.
INFO.CULTURES	Returns a list of all cultures in the current model with columns matching the schema rowset for culture objects.
INFO.DATACOVERAGEDEFINITIONS	Returns a table with information about each data coverage definition in the semantic model with metadata about data coverage settings and definitions.
INFO.DATASOURCES	Represents the TMSCHEMA_DATASOURCES DMV query function.
INFO.DELTATABLEMETADATASTORAGES	Returns a table with information about each delta table metadata storage in the semantic model with metadata about delta table storage characteristics.
INFO.DEPENDENCIES	Returns information about the calculation dependency for a DAX query.
INFO.DETAILROWSDEFINITIONS	Returns a list of all detail rows definitions in the current model with columns matching the schema rowset for detail rows definitions objects.
INFO.DICTIONARYSTORAGES	Returns a table with information about each dictionary storage in the semantic

[Function]	[Description]
	model with metadata about dictionary storage characteristics and compression.
INFO.EXCLUDEDARTIFACTS	Represents the TMSCHEMA_EXCLUDED_ARTIFACTS DMV query function.
INFO.EXPRESSIONS	Returns a list of all expressions in the current model with columns matching the schema rowset for expressions objects.
INFO.EXTENDEDPROPERTIES	Returns a list of all extended properties in the current model with columns matching the schema rowset for extended properties objects.
INFO.FORMATSTRINGDEFINITIONS	Returns a table with information about each format string definition in the semantic model with metadata about format string definitions for measures and columns.
INFO.FUNCTIONS	Returns information about the functions that are currently available for use in the DAX programming language.
INFO.GENERALSEGMENTMAPSEGMENTMETADATASTORAGES	Returns a table with information about each general segment map segment metadata storage in the semantic model with metadata about segment map storage characteristics.
INFO.GROUPBYCOLUMNS	Returns a table with information about each group by column in the semantic model with metadata about columns used in group by operations.
INFO.HIERARCHIES	Represents the TMSCHEMA_HIERARCHIES DMV query function.
INFO.HIERARCHYSTORAGES	Returns a table with information about each hierarchy storage in the semantic model with metadata about how hierarchies are stored.
INFO.KPIS	Returns a list of all KPIS in the current model with columns matching the schema rowset for KPI objects.

[Function]	[Description]
INFO.LEVELS	Returns a list of all levels in the current model with columns matching the schema rowset for level objects.
INFO.LINGUISTICMETADATA	Represents the TMSCHEMA_LINGUISTIC_METADATA DMV query function.
INFO.MEASURES	Returns a list of all measures in the current model with columns matching the schema rowset for measure objects.
INFO.MODEL	Represents the TMSCHEMA_MODEL DMV query function.
INFO.OBJECTTRANSLATIONS	Returns a list of all object translations in the current model with columns matching the schema rowset for object translation objects.
INFO.PARQUETFILESTORAGES	Returns a table with information about each Parquet file storage in the semantic model with metadata about Parquet file storage characteristics.
INFO.PARTITIONS	Represents the TMSCHEMA_PARTITIONS DMV query function.
INFO.PARTITIONSTORAGES	Returns a list of all partition storages in the current model with columns matching the schema rowset for partition storage objects.
INFO.PERSPECTIVECOLUMNS	Returns a list of all perspective columns in the current model with columns matching the schema rowset for perspective columns objects.
INFO.PERSPECTIVEHIERARCHIES	Returns a list of all perspective hierarchies in the current model with columns matching the schema rowset for perspective hierarchies objects.
INFO.PERSPECTIVEMEASURES	Returns a list of all perspective measures in the current model with columns matching the schema rowset for perspective measures objects.

[Function]	[Description]
INFO.PERSPECTIVES	Returns a list of all perspectives in the current model with columns matching the schema rowset for perspectives objects.
INFO.PERSPECTIVETABLES	Returns a list of all perspective tables in the current model with columns matching the schema rowset for perspective tables objects.
INFO.PROPERTIES	Represents the DISCOVER_PROPERTIES DMV query function.
INFO.QUERYGROUPS	Returns a table with information about each query group in the semantic model with metadata about query groups defined in the model.
INFO.REFRESHPOLICIES	Returns a table with information about each refresh policy in the semantic model with metadata about refresh policies defined for tables.
INFO.RELATEDCOLUMNDETAILS	Returns a table with information about each related column detail in the semantic model with metadata about related column details for relationships.
INFO.RELATIONSHIPINDEXSTORAGES	Returns a table with information about each relationship index storage in the semantic model with metadata about relationship index storage characteristics.
INFO.RELATIONSHIPS	Represents the TMSCHEMA_RELATIONSHIPS DMV query function.
INFO.RELATIONSHIPSTORAGES	Returns a table with information about each relationship storage in the semantic model with metadata about how relationships are stored.
INFO.ROLEMEMBERSHIPS	Returns a list of all role memberships in the current model with columns matching the schema rowset for role memberships objects.
INFO.ROLES	Returns a list of all roles in the current model with columns matching the schema rowset for roles objects.

[Function]	[Description]
INFO.SEGMENTMAPSTORAGES	Returns a list of all segment map storages in the current model with columns matching the schema rowset for segment map storage objects.
INFO.SEGMENTSTORAGES	Returns a table with information about each segment storage in the semantic model with metadata about segment storage characteristics.
INFO.STORAGEFILES	Returns a list of all storage files in the current model with columns matching the schema rowset for storage file objects.
INFO.STORAGEFOLDERS	Returns a list of all storage folders in the current model with columns matching the schema rowset for storage folder objects.
INFO.STORAGETABLECOLUMNS	Returns statistics about the columns of in-memory tables.
INFO.STORAGETABLECOLUMNSEGMENTS	Returns information about the column segments used for storing data for in-memory tables.
INFO.STORAGETABLES	Returns statistics about in-memory tables.
INFO.TABLEPERMISSIONS	Returns a list of all table permissions in the current model with columns matching the schema rowset for table permissions objects.
INFO.TABLES	Returns a list of all tables in the current model with columns matching the schema rowset for table objects.
INFO.TABLESTORAGES	Returns a list of all table storages in the current model with columns matching the schema rowset for table storage objects.
INFO.VARIATIONS	Returns a list of all variations in the current model with columns matching the schema rowset for variations objects.

Last updated on 01/08/2026

INFO.VIEW.COLUMNS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with information about each column in the semantic model, such as name, description, and format string. This information helps you understand the model and to self-document the model when used in calculated tables.

Syntax

```DAX
INFO.VIEW.COLUMNS()
```

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	The unique ID for each column in this semantic model as an integer.
[Name]	The name of each column in this semantic model as a string.
[Table]	The table of each column in this semantic model as a string.
[DataType]	The data type of each column in this semantic model as a string.
[DataCategory]	The data category of each column in this semantic model as a string.
[Description]	The description of each column in this semantic model as a string.
[IsHidden]	The hidden state of each column in this semantic model as True or False.
[IsUnique]	The is unique of each column in this semantic model as True of False.
[IsKey]	The is key of each column in this semantic model as True or False.
[IsNullable]	The is nullable of each column in this semantic model as True or False.
[Alignment]	The alignment of each column in this semantic model as a string.
[SummarizeBy]	The summarize by of each column in this semantic model as a string.

Column	Description
[ColumnStorage]	The column storage of each column in this semantic model as a string combination of name and ID.
[Type]	The type of each column in this semantic model as a string.
[SourceColumn]	The source column of each column in this semantic model as a string.
[Expression]	The DAX formula of calculated columns.
[FormatString]	The format string of each column in this semantic model as a string.
[IsAvailableInMDX]	The is available in MDX of each column in this semantic model as True or False. Analyze in Excel pivot tables will only show columns set to True.
[SortByColumn]	The sort by column of each column in this semantic model as a string. Shows as blank when sorting by itself.
[GroupingBehavior]	The grouping behavior of each column in this semantic model as a string.
[SourceProviderType]	The source provider type of each column in this semantic model as a string.
[DisplayFolder]	The display folder of each column in this semantic model as a string. Nested folders shown with / and multiple folders separated by ;.
[AlternateOf]	The alternate of property of each column in this semantic model as a string.
[LineageTag]	The lineage tag of each column in this semantic model as a string.

Remarks

Unlike INFO.COLUMNS, this function can be used in calculated tables, columns, and measures of a semantic model, including as part of a model refresh.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.VIEW.COLUMNS()
```

This DAX query returns a table with all of the columns of this DAX function.

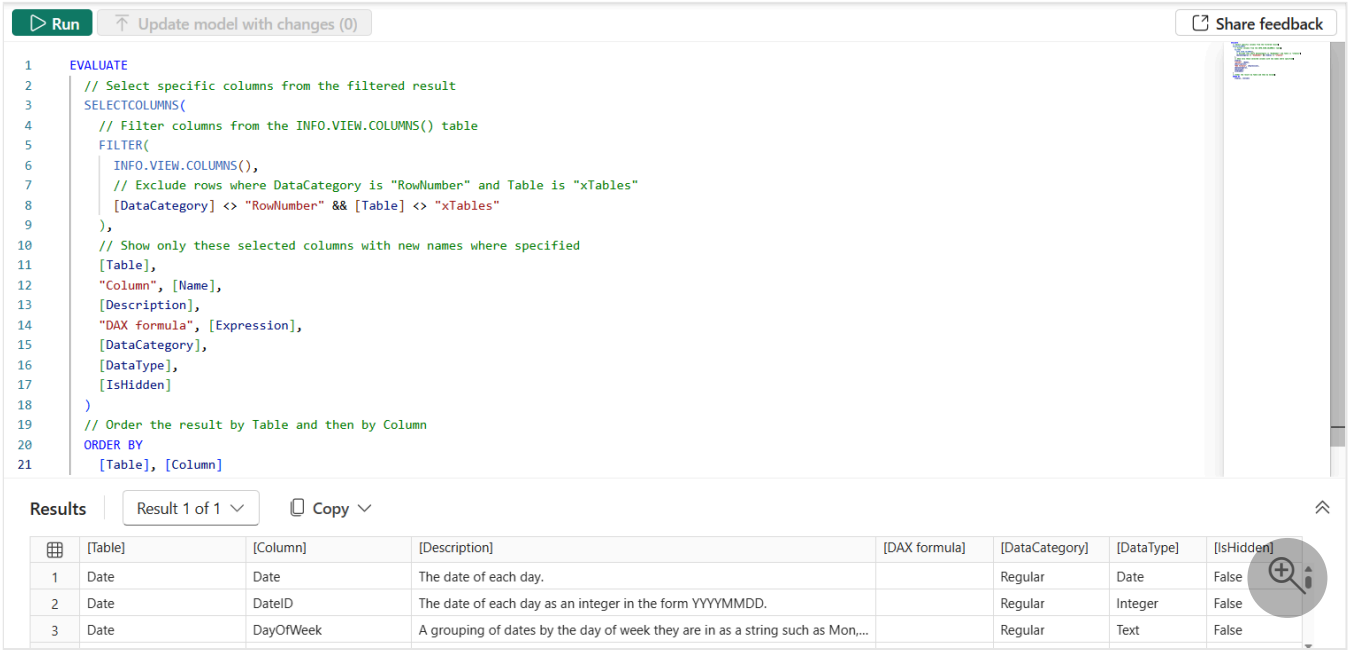
Example 2 - DAX query with SELECTCOLUMNS and FILTER

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
  // Select specific columns from the filtered result
  SELECTCOLUMNS(
    // Filter columns from the INFO.VIEW.COLUMNS() table
    FILTER(
      INFO.VIEW.COLUMNS(),
      // Exclude rows where DataCategory is "RowNumber" and Table is "xTables"
      [DataCategory] <> "RowNumber" && [Table] <> "xTables"
    ),
    // Show only these selected columns with new names where specified
    [Table],
    "Column", [Name],
    [Description],
    "DAX formula", [Expression],
    [DataCategory],
    [DataType],
    [IsHidden]
  )
  // Order the result by Table and then by Column
  ORDER BY
    [Table], [Column]
```

This DAX query returns a table with only the specified columns and rows meeting the filter condition with a DAX formula.



	[Table]	[Column]	[Description]	[DAX formula]	[DataCategory]	[DataType]	[IsHidden]
1	Date	Date	The date of each day.		Regular	Date	False
2	Date	DateID	The date of each day as an integer in the form YYYYMMDD.		Regular	Integer	False
3	Date	DayOfWeek	A grouping of dates by the day of week they are in as a string such as Mon,...		Regular	Text	False

Example 3 - calculated table with SELECTCOLUMNS and FILTER

Either of the above examples work in a calculated table when the EVALUATE and ORDER BY keywords are removed and a table name added. Here is example 2 in a calculated table:

DAX

```
Columns in this semantic model =
// Select specific columns from the filtered result
  SELECTCOLUMNS(
    // Filter columns from the INFO.VIEW.COLUMNS() table
    FILTER(
      INFO.VIEW.COLUMNS(),
      // Exclude rows where DataCategory is "RowNumber" and Table is "xTables"
      [DataCategory] <> "RowNumber" && [Table] <> "xTables"
    ),
    // Show only these selected columns with new names where specified
    [Table],
    "Column", [Name],
    [Description],
    "DAX formula", [Expression],
    [DataCategory],
    [DataType],
    [IsHidden]
  )
```

This calculated table shows the same information as the DAX query in example 2 in a table in the model itself.

Example 4 - measure

The following measure can be added to count the number of text columns in a semantic model:

DAX

```
Number of text columns =
COUNTROWS(
    FILTER(
        INFO.VIEW.COLUMNS(),
            [DataType] = "Text"
    )
)
```

This will show a scalar value with the number of text columns in my model.

This can be shown in a visual:

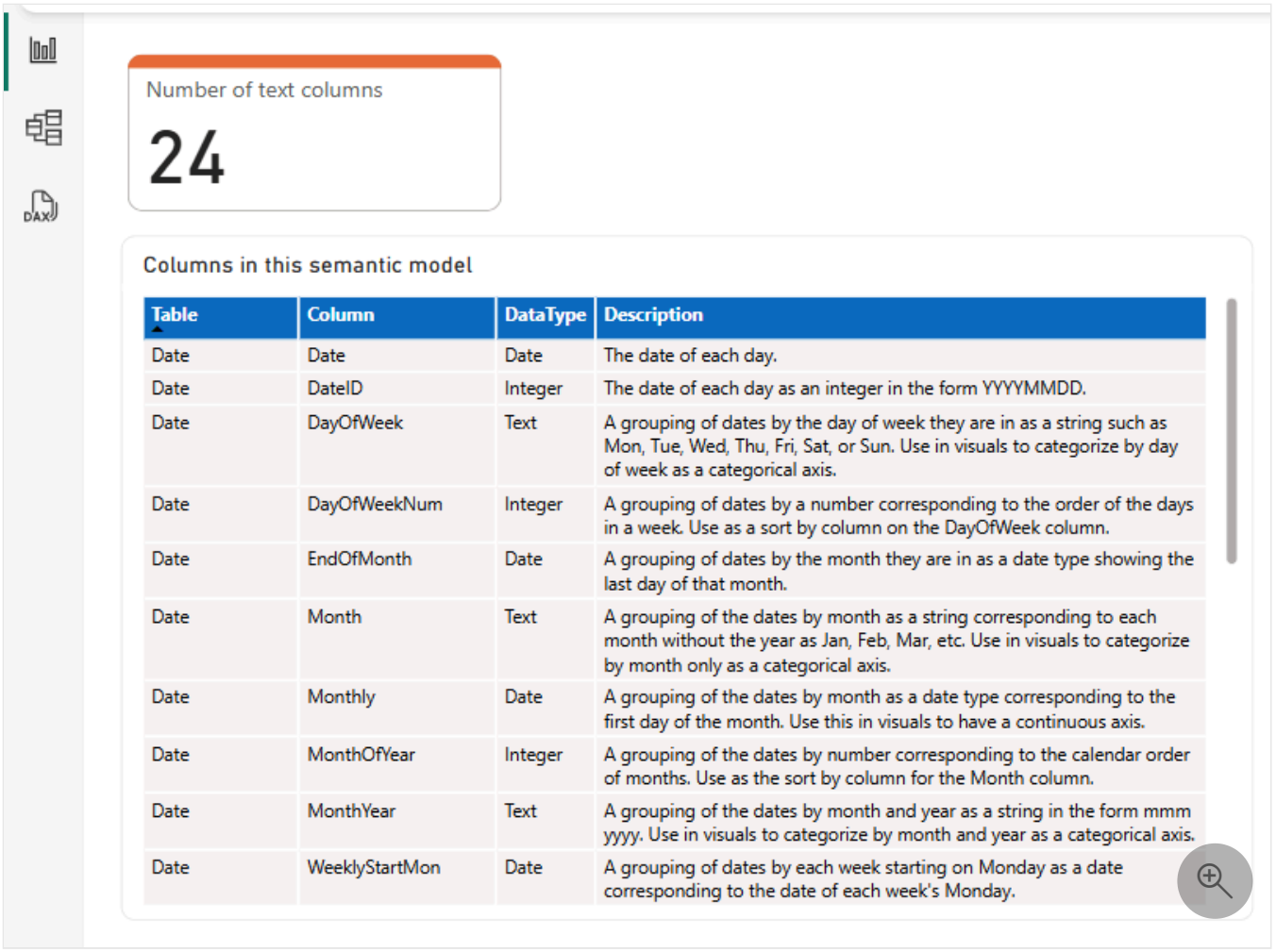


Table	Column	DataType	Description
Date	Date	Date	The date of each day.
Date	DateID	Integer	The date of each day as an integer in the form YYYYMMDD.
Date	DayOfWeek	Text	A grouping of dates by the day of week they are in as a string such as Mon, Tue, Wed, Thu, Fri, Sat, or Sun. Use in visuals to categorize by day of week as a categorical axis.
Date	DayOfWeekNum	Integer	A grouping of dates by a number corresponding to the order of the days in a week. Use as a sort by column on the DayOfWeek column.
Date	EndOfMonth	Date	A grouping of dates by the month they are in as a date type showing the last day of that month.
Date	Month	Text	A grouping of the dates by month as a string corresponding to each month without the year as Jan, Feb, Mar, etc. Use in visuals to categorize by month only as a categorical axis.
Date	Monthly	Date	A grouping of the dates by month as a date type corresponding to the first day of the month. Use this in visuals to have a continuous axis.
Date	MonthOfYear	Integer	A grouping of the dates by number corresponding to the calendar order of months. Use as the sort by column for the Month column.
Date	MonthYear	Text	A grouping of the dates by month and year as a string in the form mmm yyyy. Use in visuals to categorize by month and year as a categorical axis.
Date	WeeklyStartMon	Date	A grouping of dates by each week starting on Monday as a date corresponding to the date of each week's Monday.

Or a DAX query in DAX query view:

DAX

```
DEFINE
    MEASURE 'Columns in this semantic model'[Number of text columns] =
    COUNTROWS(
        FILTER(
            INFO.VIEW.COLUMNS(),
                [DataType] = "Text"
        )
    )

EVALUATE
    SUMMARIZECOLUMNS(
        "Number of text columns", [Number of text columns]
    )
```

See also

- INFO.VIEW.TABLES

- INFO.VIEW.MEASURES
- INFO.VIEW.RELATIONSHIPS
- INFO.TABLES
- INFO.COLUMNS

Last updated on 02/11/2026

INFO.VIEW.MEASURES

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with information about each measure in the semantic model, such as name, description, and DAX formula. This information helps you understand the model and to self-document the model when used in calculated tables.

Syntax

```DAX
INFO.VIEW.MEASURES()
```

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	The unique ID for each measure in this semantic model as an integer.
[Name]	The name of each measure in this semantic model as a string.
[Table]	The home table of each measure in this semantic model as a string.
[Description]	The description of each measure in this semantic model as a string.
[DataType]	The data type of each measure in this semantic model as a string. Measures are usually variant data type.
[Expression]	The DAX formula of each measure in this semantic model.
[FormatString]	The format string of each measure in this semantic model as a string.
[IsHidden]	The hidden state of each measure in this semantic model as True or False.
[State]	The state (such as valid or error) of each measure in this semantic model as a string.
[KPIID]	The KPI ID of each measure in this semantic model as an integer.

Column	Description
[IsSimpleMeasure]	The simple measure flag of each measure in this semantic model as True of False.
[DisplayFolder]	The display folder of each measure in this semantic model as a string. Nested folders shown with / and multiple folders separated by ;.
[DetailRowsDefinition]	The details rows definition of each measure in this semantic model.
[DataCategory]	The data category of each measure in this semantic model as a string.
[FormatStringDefinition]	The dynamic format string of each measure in this semantic model.
[LineageTag]	The lineage tag of each measure in this semantic model as a string.

Remarks

Only shows the [Expression], [DetailRowsDefinition] and [FormatStringDefinition] values when run by users with write permission on the semantic model, and not when live connected to the semantic model in Power BI Desktop. Unlike INFO.MEASURES, this function can be used in calculated tables, columns, and measures of a semantic model, including as part of a model refresh.

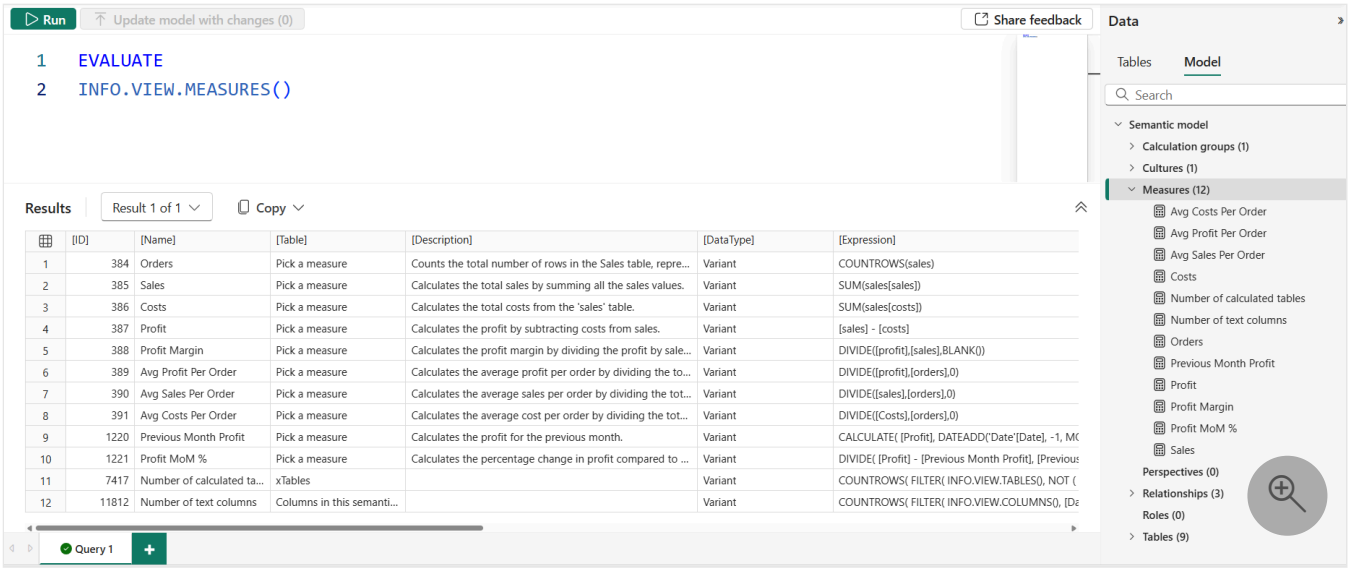
Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.VIEW.MEASURES()
```

This DAX query returns a table with all of the columns of this DAX function.



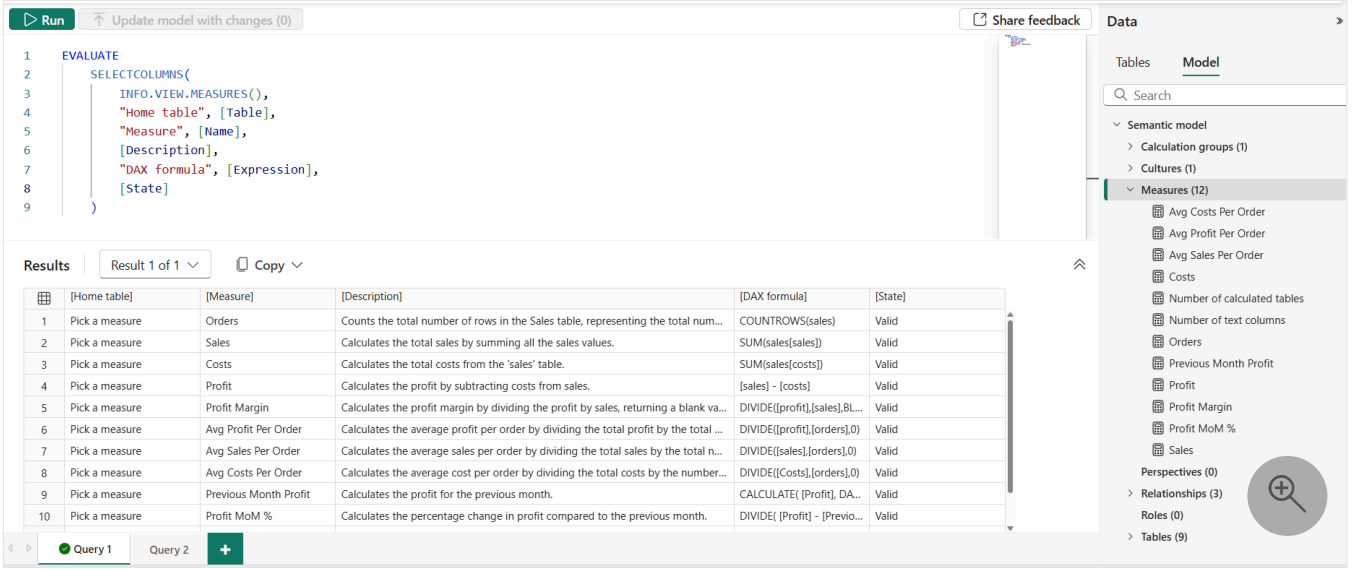
Example 2 - DAX query with SELECTCOLUMNS

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    SELECTCOLUMNS(
        INFO.VIEW.MEASURES(),
        "Home table", [Table],
        "Measure", [Name],
        [Description],
        "DAX formula", [Expression],
        [State]
    )
```

This DAX query returns a table with only the specified columns.



Example 3 - calculated table with SELECTCOLUMNS

Either of the above examples work in a calculated table when the EVALUATE keyword is removed and a table name added. Here is example 2 in a calculated table:

DAX

```
Measures in this semantic model =
SELECTCOLUMNS(
        INFO.VIEW.MEASURES(),
        "Home table", [Table],
        "Measure", [Name],
        [Description],
        "DAX formula", [Expression],
        [State]
    )
```

This calculated table shows the same information as the DAX query in example 2 in a table in the model itself.

Example 4 - measure

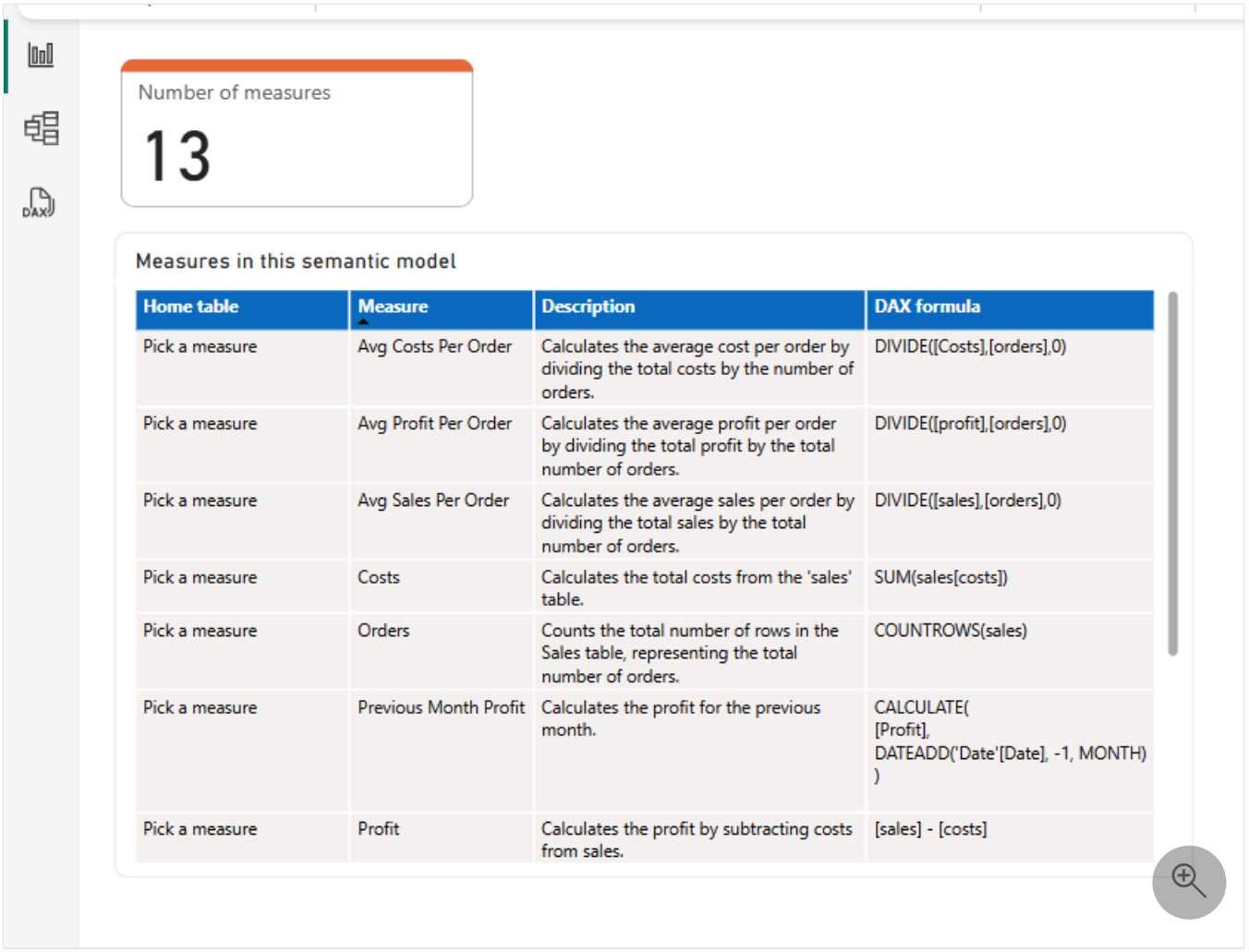
The following measure can be added to count the number of text columns in a semantic model:

DAX

```
Number of measures =
COUNTROWS( INFO.VIEW.MEASURES() )
```

This will show a scalar value with the number of measures in my model.

This can be shown in a visual:



Or a DAX query in DAX query view:

DAX

```
DEFINE
    MEASURE 'Measures in this semantic model'[Number of measures] = COUNTROWS(
INFO.VIEW.MEASURES() )

EVALUATE
    SUMMARIZECOLUMNS(
        "Number of measures", [Number of measures]
    )
```

Run | Update model with changes (0)

```
DEFINE
    MEASURE 'Measures in this semantic model'[Number of measures] = COUNTROWS( INFO.VIEW.MEASURES() )

EVALUATE
    SUMMARIZECOLUMNS(
        "Number of measures", [Number of measures]
    )
```

Results | Result 1 of 1 | Copy

	[Number of measures]
1	13

See also

- INFO.VIEW.TABLES
- INFO.VIEW.COLUMNS
- INFO.VIEW.RELATIONSHIPS
- INFO.MEASURES
- INFO.TABLES
- INFO.COLUMNS

Last updated on 02/11/2026

INFO.VIEW.RELATIONSHIPS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with information about each relationship in the semantic model, such as name, cardinality, and cross-filtering behavior. This information helps you understand the model and to self-document the model when used in calculated tables.

Syntax

DAX

```
INFO.VIEW.RELATIONSHIPS()
```

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	The unique ID for each relationship in this semantic model as an integer.
[Name]	The name of each relationship in this semantic model as a string. Might be a GUID.
[Relationship]	The descriptive relationship name of each relationship in this semantic model as a string. Includes from table and column, to table and column, with cardinality and cross filter direction.
[Model]	The relationship's semantic model ID, usually a GUID.
[IsActive]	The is active property of each relationship in this semantic model as True or False.
[CrossFilteringBehavior]	The cross-filter behavior or direction of each relationship in this semantic model as a string.
[RelyOnReferentialIntegrity]	The rely on referential integrity property of each relationship in this semantic model as a string. Also called assume referential integrity in the relationship editor, as it assumes all rows in the column in the many table have a match to a row in the one side table.

Column	Description
[FromTable]	The from table name of each relationship in this semantic model as a string.
[FromColumn]	The from column name of each relationship in this semantic model as a string.
[FromCardinality]	The from column cardinality of each relationship in this semantic model as a string.
[ToTable]	The to table name of each relationship in this semantic model as a string.
[ToColumn]	The to column name of each relationship in this semantic model as a string.
[ToCardinality]	The to column cardinality of each relationship in this semantic model as a string.
[State]	The state of each relationship in this semantic model as a string.
[SecurityFilteringBehavior]	The security filtering behavior of each relationship in this semantic model as a string. Important for row-level security roles.

Remarks

Unlike INFO.RELATIONSHIPS, this function can be used in calculated tables, columns, and measures of a semantic model, including as part of a model refresh.

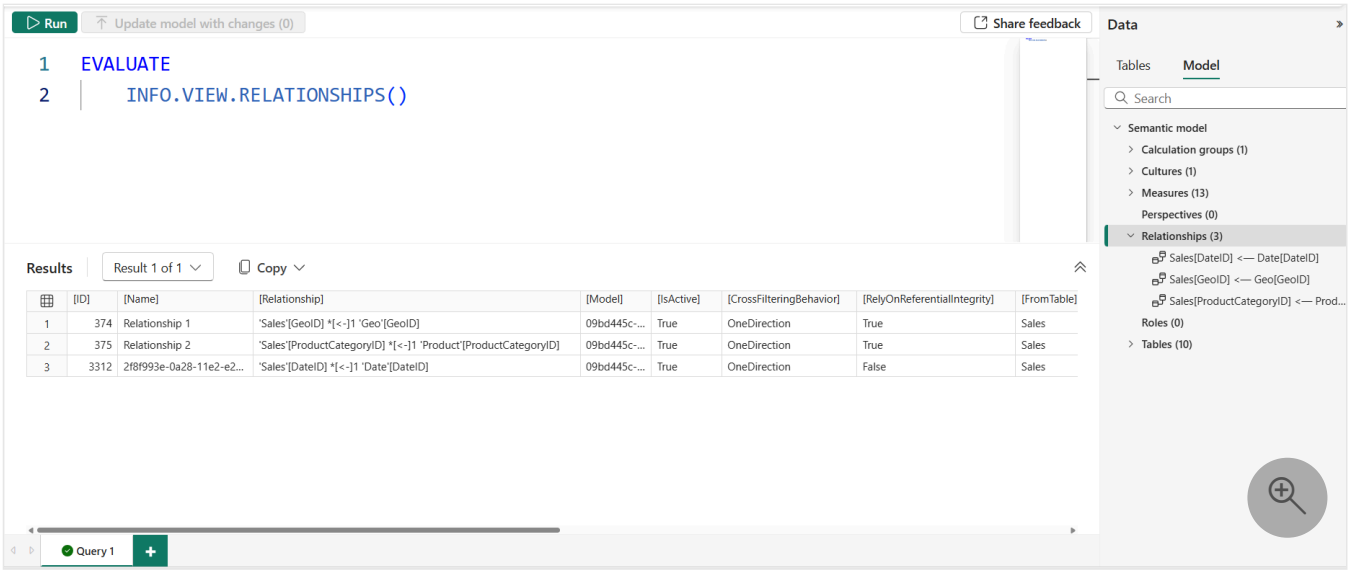
Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.VIEW.RELATIONSHIPS()
```

This DAX query returns a table with all of the columns of this DAX function.



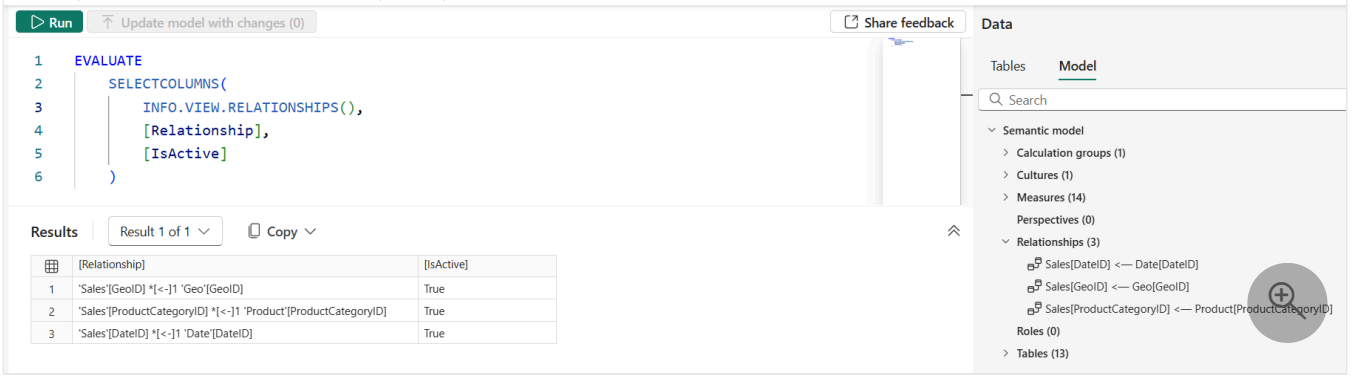
Example 2 - DAX query with SELECTCOLUMNS

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    SELECTCOLUMNS(
        INFO.VIEW.RELATIONSHIPS(),
        [Relationship],
        [IsActive]
    )
```

This DAX query returns a table with only the specified columns.



Example 3 - calculated table with SELECTCOLUMNS

Either of the above examples work in a calculated table when the EVALUATE and ORDER BY keywords are removed and a table name added. Here is example 2 in a calculated table:

DAX

```
Relationships in this semantic model =
SELECTCOLUMNS(
    INFO.VIEW.RELATIONSHIPS(),
    [Relationship],
    [IsActive]
)
```

This calculated table shows the same information as the DAX query in example 2 in a table in the model itself.

Example 4 - measure

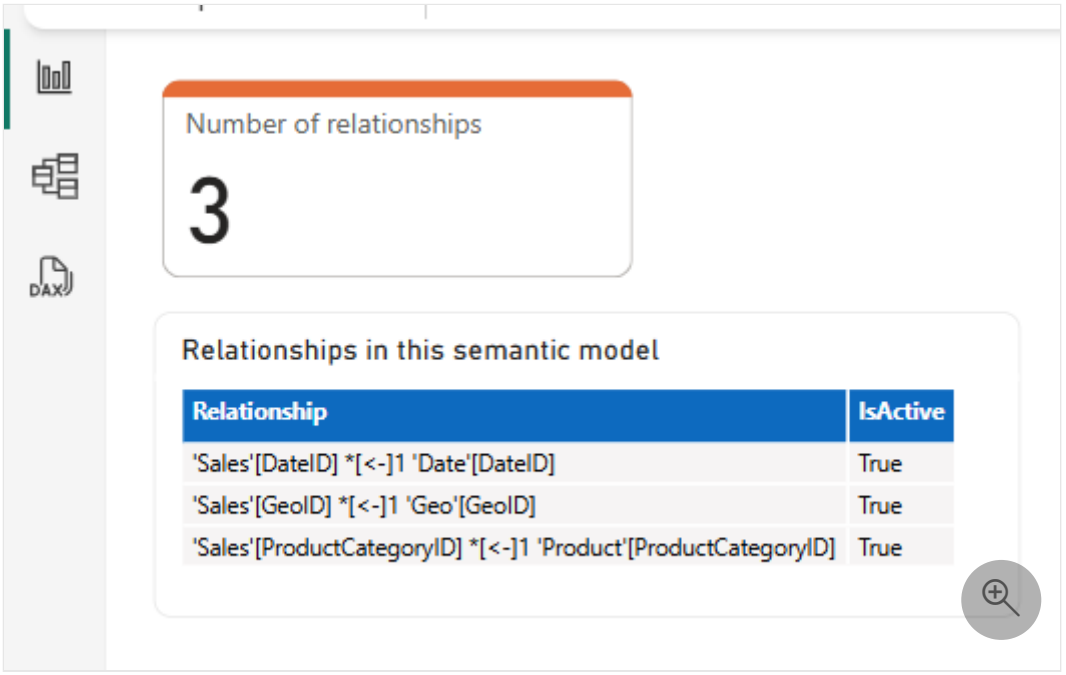
The following measure can be added to count the number of relationships in a semantic model:

DAX

```
Number of relationships =
COUNTROWS( INFO.VIEW.RELATIONSHIPS() )
```

This will show a scalar value with the number of relationships in my model.

This can be shown in a visual:

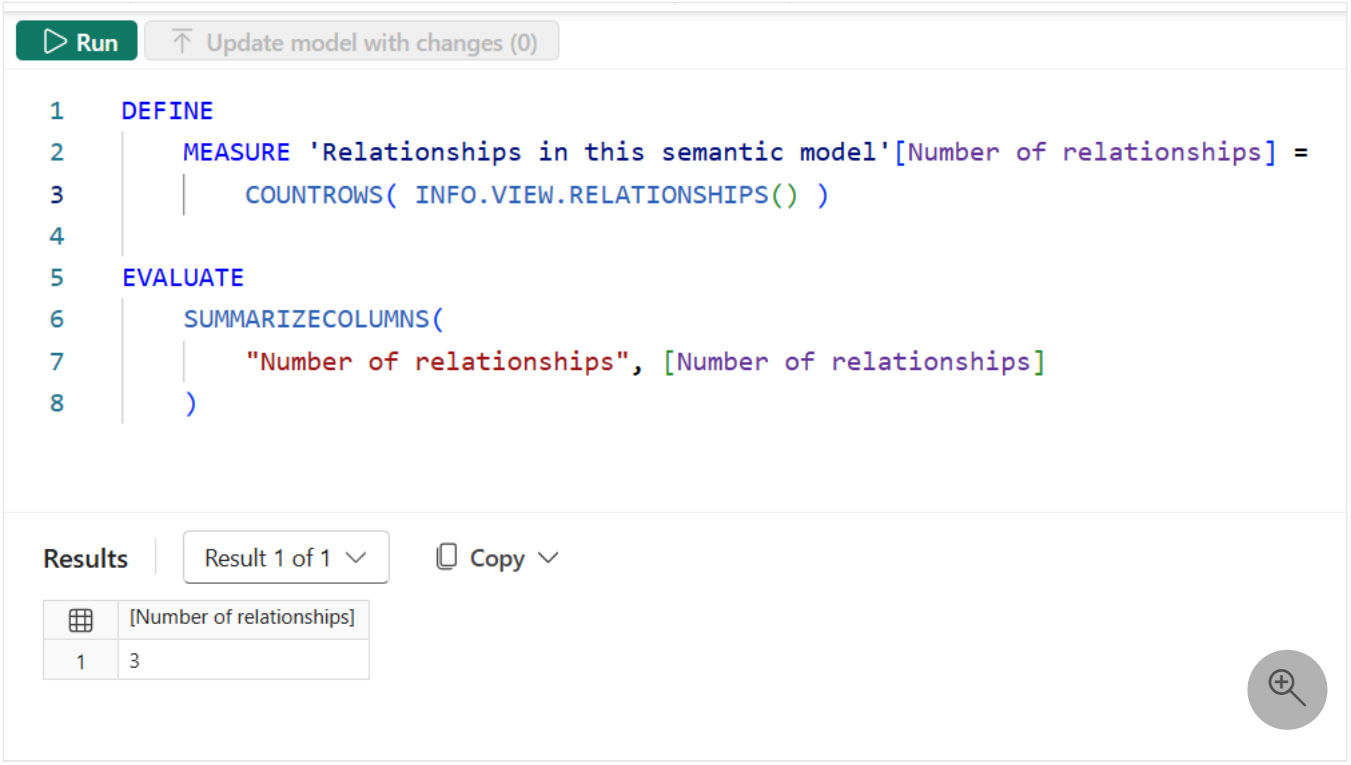


Or a DAX query in DAX query view:

DAX

```
DEFINE
    MEASURE 'Measures in this semantic model'[Number of measures] = COUNTROWS(
```

```
INFO.VIEW.MEASURES() )

EVALUATE
    SUMMARIZECOLUMNS(
        "Number of measures", [Number of measures]
    )
```



See also

- INFO.VIEW.TABLES
- INFO.VIEW.COLUMNS
- INFO.VIEW.MEASURES
- INFO.RELATIONSHIPS
- INFO.TABLES
- INFO.COLUMNS

Last updated on 02/11/2026

INFO.VIEW.TABLES

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with information about each table in the semantic model, such as table name, description, and storage mode. This information helps you understand the model and to self-document the model when used in calculated tables.

Syntax

DAX

```
INFO.VIEW.TABLES()
```

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	The unique ID for each table in this semantic model as an integer.
[Name]	The name of each table in this semantic model as a string.
[Model]	The ID of the table's semantic model as a GUID.
[DataCategory]	The data category of each table in this semantic model as a string.
[Description]	The description of each table in this semantic model as a string.
[IsHidden]	The hidden state of each table in this semantic model as TRUE or FALSE.
[StorageMode]	The storage mode of each table in this semantic model as a string.
[TableStorage]	The name and unique ID of each table in this semantic model as a string.
[Expression]	The DAX formula of each table in this semantic model as a string. Only applies to calculated tables.
[ShowAsVariationOnly]	The show as variation only state of each table in this semantic model as TRUE or FALSE.
[IsPrivate]	The private state of each table in this semantic model as TRUE or FALSE.

Column	Description
[CalculationGroupPrecedence]	The calculation group precedence of each table in this semantic model as an integer. Only applies to calculation groups.
[LineageTag]	The lineage tag of each table in this semantic model as a string.

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in calculated tables, columns, and measures of a semantic model and will update when the model is refreshed.

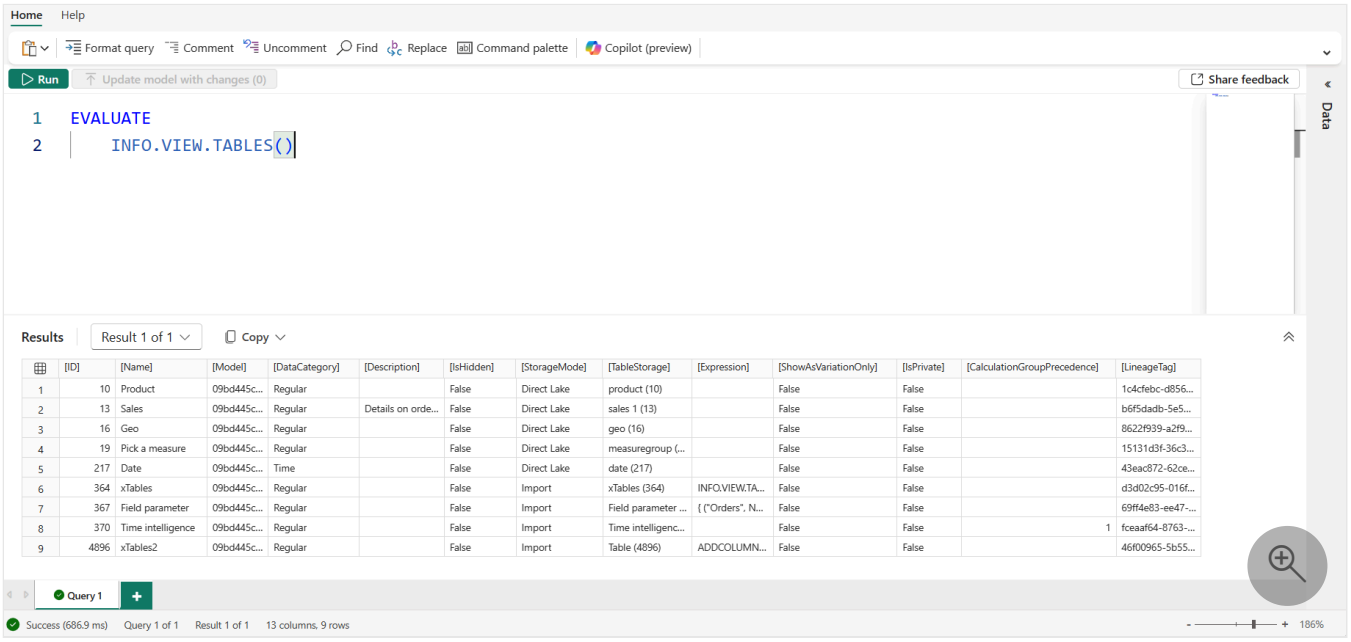
Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.VIEW.TABLES()
```

This DAX query returns a table with all of the columns of this DAX function.



	[ID]	[Name]	[Model]	[DataCategory]	[Description]	[IsHidden]	[StorageMode]	[TableStorage]	[Expression]	[ShowAsVariationOnly]	[IsPrivate]	[CalculationGroupPrecedence]	[LineageTag]
1	10	Product	09bd445c...	Regular		False	Direct Lake	product (10)		False	False		1c4cfebc-d856...
2	13	Sales	09bd445c...	Regular	Details on orde...	False	Direct Lake	sales 1 (13)		False	False		b6f5dadb-5e5...
3	16	Geo	09bd445c...	Regular		False	Direct Lake	geo (16)		False	False		8622f939-a2f9...
4	19	Pick a measure	09bd445c...	Regular		False	Direct Lake	measuregroup (...		False	False		15131d3f-36c3...
5	217	Date	09bd445c...	Time		False	Direct Lake	date (217)		False	False		43eac872-62ce...
6	364	xTables	09bd445c...	Regular		False	Import	xTables (364)	INFO.VIEW.TA...	False	False		d3d02c95-016f...
7	367	Field parameter	09bd445c...	Regular		False	Import	Field parameter ...	{ ("Orders", N...	False	False		69ff4e83-ee47-...
8	370	Time intelligence	09bd445c...	Regular		False	Import	Time intelligenc...		False	False	1	fceaaf64-8763-...
9	4896	xTables2	09bd445c...	Regular		False	Import	Table (4896)	ADDCOLUMN...	False	False		46f00965-5b55...

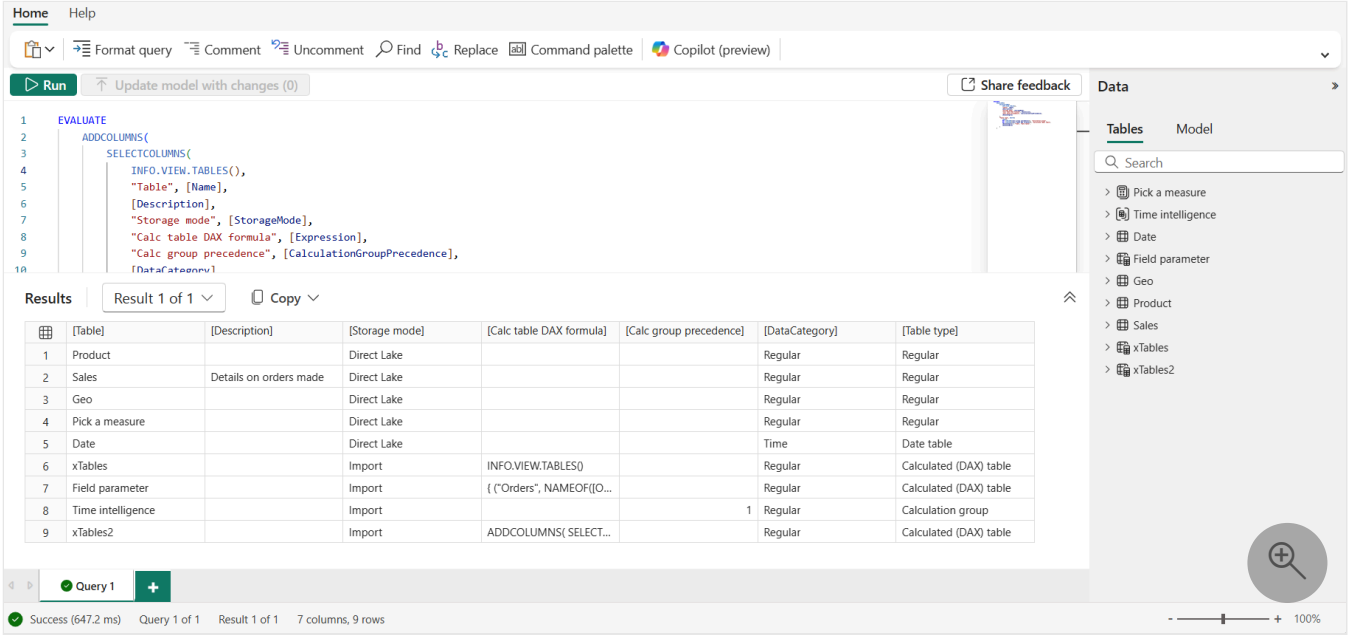
Example 2 - DAX query with SELECTCOLUMNS and ADDCOLUMNS

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    ADDCOLUMNS(
        SELECTCOLUMNS(
            INFO.VIEW.TABLES(),
            "Table", [Name],
            [Description],
            "Storage mode", [StorageMode],
            "Calc table DAX formula", [Expression],
            "Calc group precedence", [CalculationGroupPrecedence],
            [DataCategory]
        ),
        "Table type", SWITCH(
            TRUE(),
            NOT (ISBLANK([Calc group precedence])), "Calculation group",
            NOT (ISBLANK([Calc table DAX formula])), "Calculated (DAX) table",
            [DataCategory] = "Time", "Date table",
            [DataCategory]
        )
    )
```

This DAX query returns a table with only the specified columns and a new column to categorize the tables with a DAX formula.



	[Table]	[Description]	[Storage mode]	[Calc table DAX formula]	[Calc group precedence]	[DataCategory]	[Table type]
1	Product		Direct Lake			Regular	Regular
2	Sales	Details on orders made	Direct Lake			Regular	Regular
3	Geo		Direct Lake			Regular	Regular
4	Pick a measure		Direct Lake			Regular	Regular
5	Date		Direct Lake			Time	Date table
6	xTables		Import	INFO.VIEW.TABLES()		Regular	Calculated (DAX) table
7	Field parameter		Import	{ ("Orders", NAMEOF([O...		Regular	Calculated (DAX) table
8	Time intelligence		Import		1	Regular	Calculation group
9	xTables2		Import	ADDCOLUMNS(SELECT...		Regular	Calculated (DAX) table

Example 3 - calculated table with SELECTCOLUMNS and ADDCOLUMNS

Either of the above examples work in a calculated table when the EVALUATE keyword is removed and a table name added. Here is example 2 in a calculated table:

DAX

```
xTables2 =
    ADDCOLUMNS(
        SELECTCOLUMNS(
            INFO.VIEW.TABLES(),
            "Table", [Name],
            [Description],
            "Storage mode", [StorageMode],
            "Calc table DAX formula", [Expression],
            "Calc group precedence", [CalculationGroupPrecedence],
            [DataCategory]
        ),
        "Table type", SWITCH(
            TRUE(),
            NOT (ISBLANK([Calc group precedence])), "Calculation group",
            NOT (ISBLANK([Calc table DAX formula])), "Calculated (DAX) table",
            [DataCategory] = "Time", "Date table",
            [DataCategory]
        )
    )
```

This calculated table shows the same information as the DAX query in example 2 in a table in the model itself.

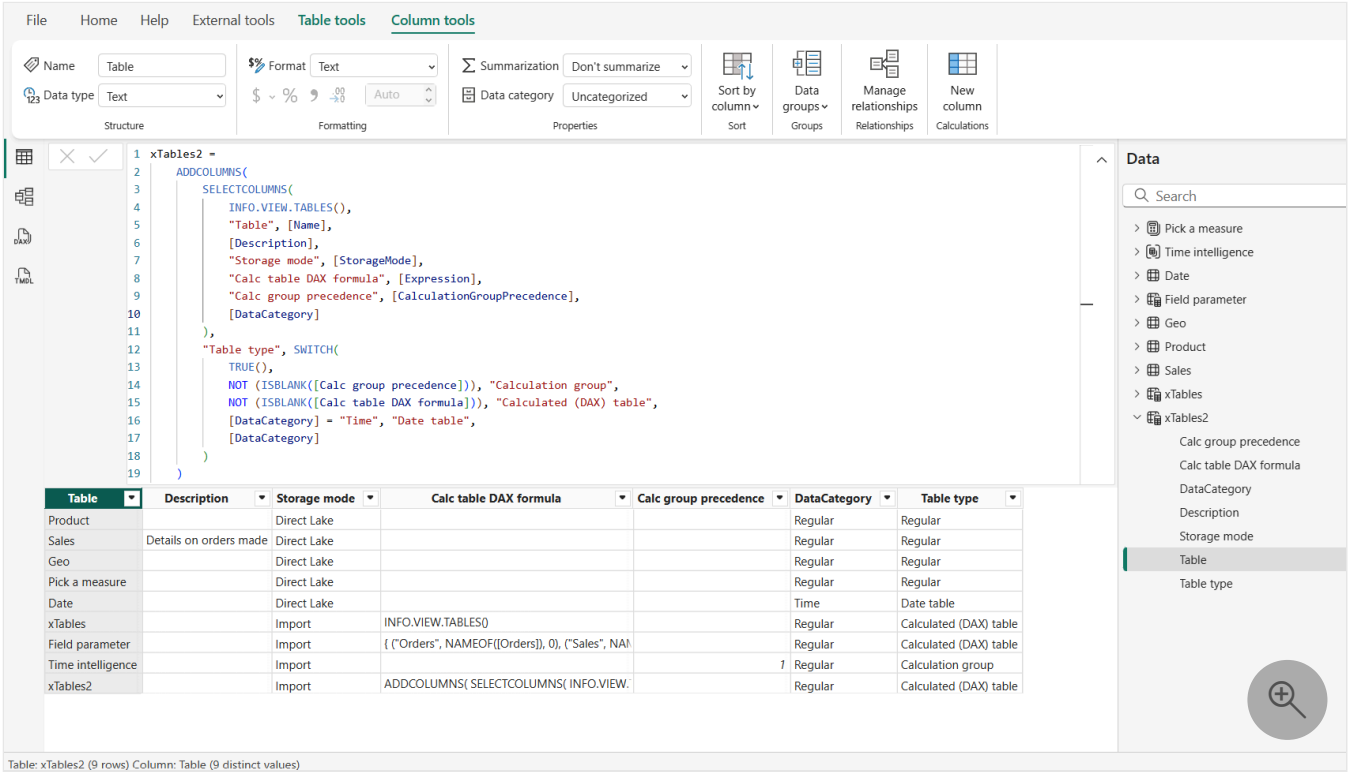


Table	Description	Storage mode	Calc table DAX formula	Calc group precedence	DataCategory	Table type
Product		Direct Lake			Regular	Regular
Sales	Details on orders made	Direct Lake			Regular	Regular
Geo		Direct Lake			Regular	Regular
Pick a measure		Direct Lake			Regular	Regular
Date		Direct Lake			Time	Date table
xTables		Import	INFO.VIEW.TABLES()		Regular	Calculated (DAX) table
Field parameter		Import	{ ("Orders", NAMEOF([Orders]), 0), ("Sales", NAI		Regular	Calculated (DAX) table
Time intelligence		Import		1	Regular	Calculation group
xTables2		Import	ADDCOLUMNS(SELECTCOLUMNS(INFO.VIEW.		Regular	Calculated (DAX) table

Example 4 - measure

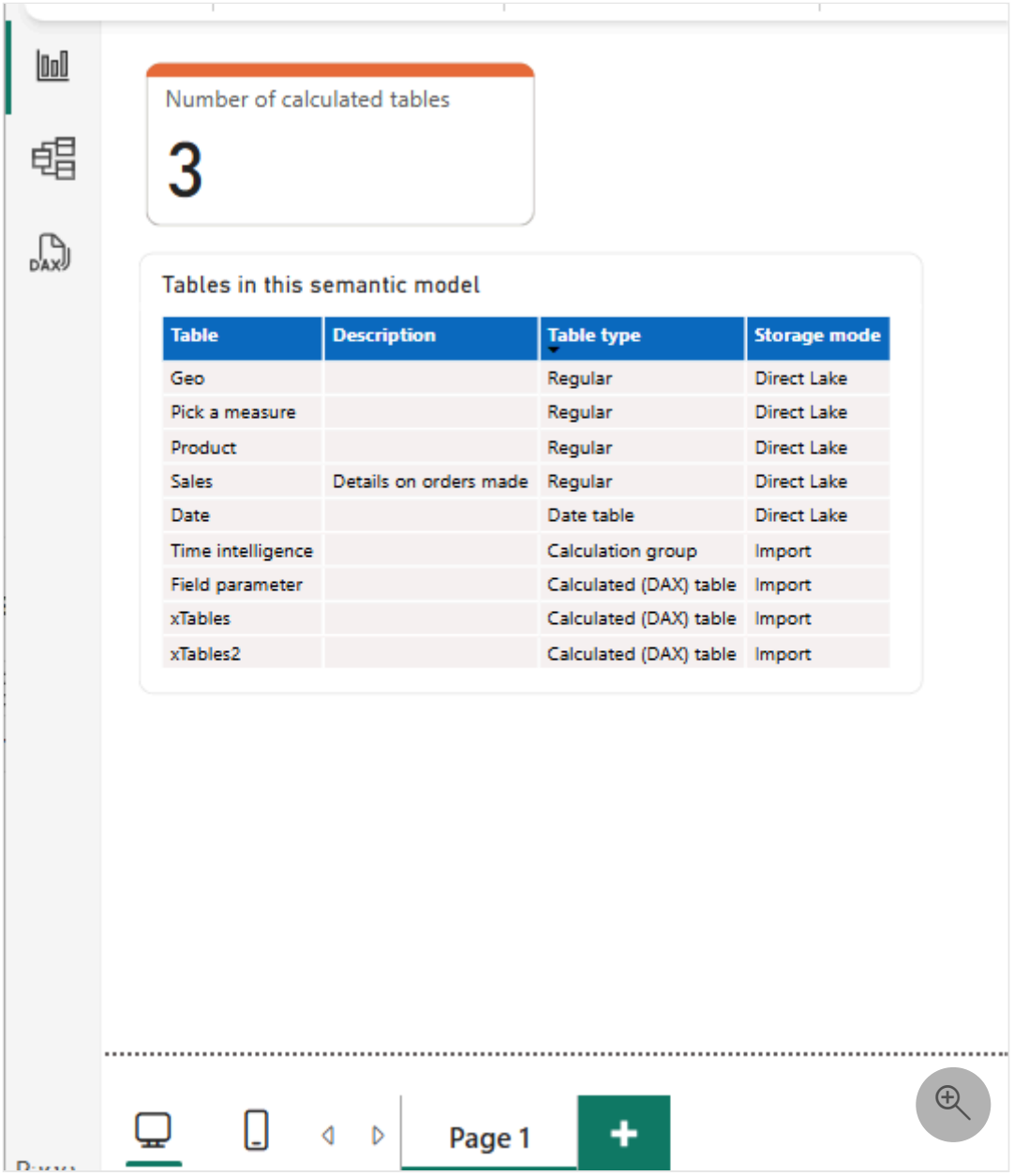
The following measure can be added to count the number of calculated columns in a semantic model:

DAX

```
Number of calculated tables =
COUNTROWS(
    FILTER(
        INFO.VIEW.TABLES(),
            NOT ( ISBLANK( [Expression] ) )
    )
)
```

This will show a scalar value with the number of calculated tables in my model.

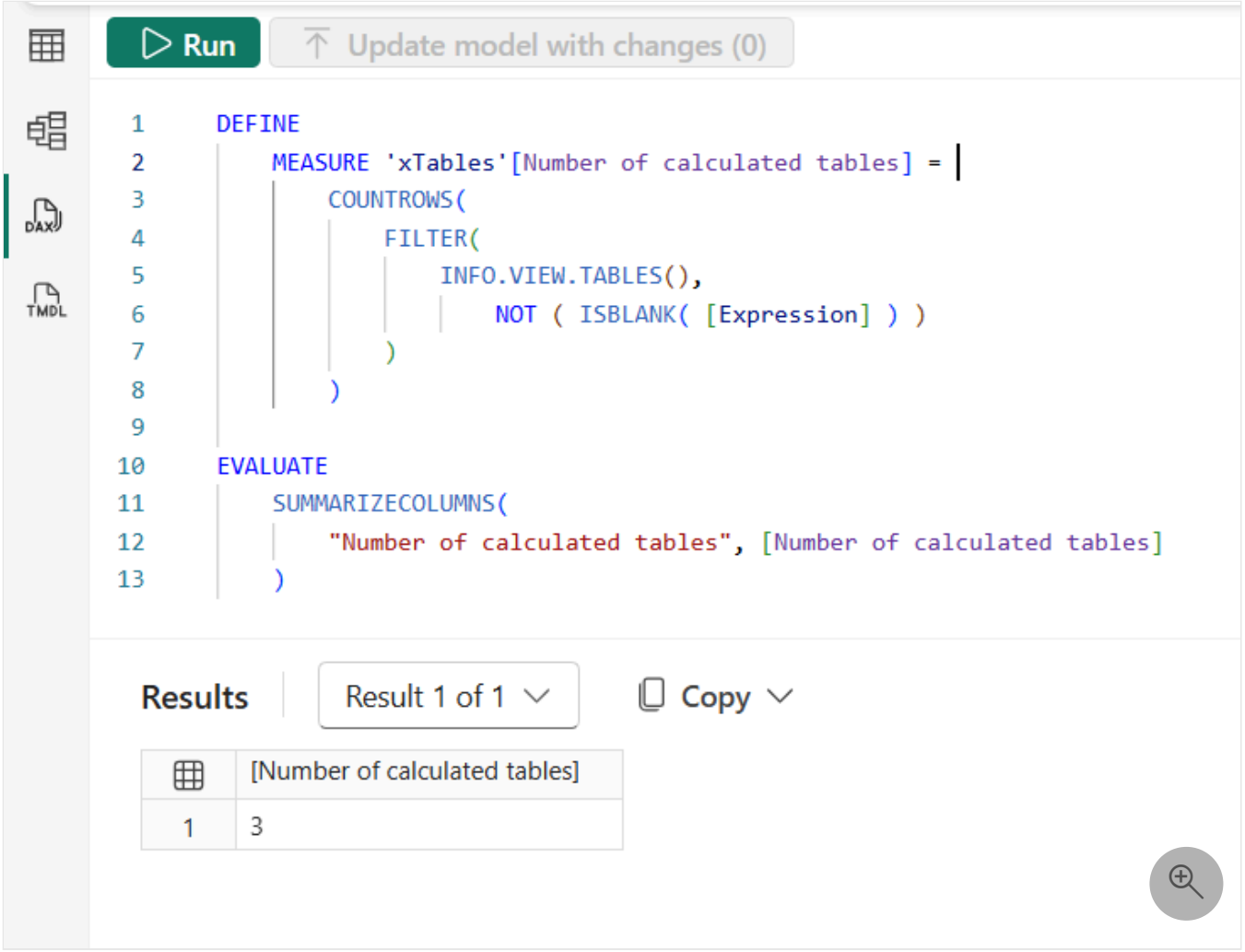
This can be shown in a visual:



Or a DAX query in DAX query view:

DAX

```
DEFINE
    MEASURE 'xTables'[Number of calculated tables] =
        COUNTROWS(
            FILTER(
                INFO.VIEW.TABLES(),
                    NOT ( ISBLANK( [Expression] ) )
            )
        )

EVALUATE
    SUMMARIZECOLUMNS(
        "Number of calculated tables", [Number of calculated tables]
    )
```



See also

INFO.VIEW.COLUMNS INFO.VIEW.MEASURES INFO.VIEW.RELATIONSHIPS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.ALTERNATEOFDEFINITIONS

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✗ Visual calculation ✓ DAX query

Returns a table with information about each alternate of definition in the semantic model. This function provides metadata about alternate definitions for model objects.

Syntax

```
DAX
INFO.ALTERNATEOFDEFINITIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for alternate of definitions in the current semantic model.

Expand table

Column	Description
[ID]	Unique identifier for the alternate of definition object.
[ColumnID]	Reference to the column object that has an alternate definition.
[BaseColumnID]	Reference to the base column that this alternate definition is derived from.
[BaseTableID]	Reference to the base table containing the base column.
[Summarization]	The summarization method applied to the alternate definition (e.g., Sum, Count, Average).

Remarks

- Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop.
- This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ALTERNATEOFDEFINITIONS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _AlternateOfDefinitions =
        INFO.ALTERNATEOFDEFINITIONS()

    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ColumnID", [ID],
            "Column Name", [ExplicitName],
            "TableID", [TableID]
        )
```

```
VAR _BaseColumns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "BaseColumnID", [ID],
        "Base Column Name", [ExplicitName]
    )

VAR _BaseTables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "BaseTableID", [ID],
        "Base Table Name", [Name]
    )

VAR _CombinedWithColumns =
    NATURALLEFTOUTERJOIN(
        _AlternateOfDefinitions,
        _Columns
    )

VAR _CombinedWithBaseColumns =
    NATURALLEFTOUTERJOIN(
        _CombinedWithColumns,
        _BaseColumns
    )

VAR _CombinedWithBaseTables =
    NATURALLEFTOUTERJOIN(
        _CombinedWithBaseColumns,
        _BaseTables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedWithBaseTables,
        "Column Name", [Column Name],
        "Base Table Name", [Base Table Name],
        "Base Column Name", [Base Column Name],
        "Summarization", [Summarization]
    )
ORDER BY [Column Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

Last updated on 01/08/2026

INFO.ANNOTATIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each annotation in the semantic model. This information helps you understand the model.

Syntax

DAX

```
INFO.ANNOTATIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[ObjectID]	A reference to the object that owns this Annotation.
[ObjectType]	A reference to the type of object that owns this Annotation.
[Name]	The name of the object, used in code, script, and queries.
[Value]	The value of the annotation.
[ModifiedTime]	The time that the object was last modified.

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ANNOTATIONS()
```

This DAX query returns a table with all of the columns of this DAX function.

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ObjectTypeEnum =
    DATATABLE(
    "ObjectType",INTEGER,
    "Object",STRING,
    {
    {0,"Null"},
    {1,"Model"},
```

```
{2,"DataSource"},
{3,"Table"},
{4,"Column"},
{5,"AttributeHierarchy"},
{6,"Partition"},
{7,"Relationship"},
{8,"Measure"},
{9,"Hierarchy"},
{10,"Level"},
{11,"Annotation"},
{12,"KPI"},
{13,"Culture"},
{14,"ObjectTranslation"},
{15,"LinguisticMetadata"},
{29,"Perspective"},
{30,"PerspectiveTable"},
{31,"PerspectiveColumn"},
{32,"PerspectiveHierarchy"},
{33,"PerspectiveMeasure"},
{34,"Role"},
{35,"RoleMembership"},
{36,"TablePermission"},
{37,"Variation"},
{38,"Set"},
{39,"PerspectiveSet"},
{40,"ExtendedProperty"},
{41,"Expression"},
{42,"ColumnPermission"},
{43,"DetailRowsDefinition"},
{44,"RelatedColumnDetails"},
{45,"GroupByColumn"},
{46,"CalculationGroup"},
{47,"CalculationItem"},
{48,"AlternateOf"},
{49,"RefreshPolicy"},
{50,"FormatStringDefinition"}
}
)

VAR _INFO =
INFO.ANNOTATIONS()

VAR _CombinedTable =
NATURALLEFTOUTERJOIN(
    _INFO,
    _ObjectTypeEnum
)

VAR _ModelObjects =
UNION(
    SELECTCOLUMNS(
        INFO.MEASURES(),
        "ObjectID", [ID],
        "Object Name", [Name]
    ),
```

```
        SELECTCOLUMNS(
            INFO.TABLES(),
            "ObjectID", [ID],
            "Object Name", [Name]
        ),
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ObjectID", [ID],
            "Object Name", [ExplicitName]
        ),
        SELECTCOLUMNS(
            INFO.EXPRESSIONS(),
            "ObjectID", [ID],
            "Object Name", [Name]
        ),
        SELECTCOLUMNS(
            INFO.MODEL(),
            "ObjectID", [ID],
            "Object Name", [Name]
        )
    )

    VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable,
        _ModelObjects
    )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable2,
            "Object type", [Object],
            "Object", [Object Name],
            "Annotation name", [Name],
            "Annotation value", [Value]
        )
    ORDER BY [Annotation name]
```

This DAX query returns a table with only the specified columns and joining to other INFO DAX functions and the enumeration table.

Enumerations

Columns returned in this INFO function give the ID of an enumeration, or lookup table. Here are the values for each enumeration in this DAX function.

ObjectType

Expand table

Number	Value
0	Null
1	Model
2	DataSource
3	Table
4	Column
5	AttributeHierarchy
6	Partition
7	Relationship
8	Measure
9	Hierarchy
10	Level
11	Annotation
12	KPI
13	Culture
14	ObjectTranslation
15	LinguisticMetadata
29	Perspective
30	PerspectiveTable
31	PerspectiveColumn
32	PerspectiveHierarchy
33	PerspectiveMeasure
34	Role
35	RoleMembership
36	TablePermission
37	Variation
38	Set

Number	Value
39	PerspectiveSet
40	ExtendedProperty
41	Expression
42	ColumnPermission
43	DetailRowsDefinition
44	RelatedColumnDetails
45	GroupByColumn
46	CalculationGroup
47	CalculationItem
48	AlternateOf
49	RefreshPolicy
50	FormatStringDefinition

This table is based on the official documentation.

To join with INFO functions use this DAX query.

DAX

```
EVALUATE
    DATATABLE(
    "ObjectType",INTEGER,
    "Object",STRING,
    {
    {0,"Null"},
    {1,"Model"},
    {2,"DataSource"},
    {3,"Table"},
    {4,"Column"},
    {5,"AttributeHierarchy"},
    {6,"Partition"},
    {7,"Relationship"},
    {8,"Measure"},
    {9,"Hierarchy"},
    {10,"Level"},
    {11,"Annotation"},
    {12,"KPI"},
    {13,"Culture"},
    {14,"ObjectTranslation"},
    {15,"LinguisticMetadata"},
```

```
    {29,"Perspective"},
    {30,"PerspectiveTable"},
    {31,"PerspectiveColumn"},
    {32,"PerspectiveHierarchy"},
    {33,"PerspectiveMeasure"},
    {34,"Role"},
    {35,"RoleMembership"},
    {36,"TablePermission"},
    {37,"Variation"},
    {38,"Set"},
    {39,"PerspectiveSet"},
    {40,"ExtendedProperty"},
    {41,"Expression"},
    {42,"ColumnPermission"},
    {43,"DetailRowsDefinition"},
    {44,"RelatedColumnDetails"},
    {45,"GroupByColumn"},
    {46,"CalculationGroup"},
    {47,"CalculationItem"},
    {48,"AlternateOf"},
    {49,"RefreshPolicy"},
    {50,"FormatStringDefinition"}
    }
    )
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.ATTRIBUTEHIERARCHIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each attribute hierarchy in the semantic model. This function provides metadata about the attribute hierarchies defined in the model.

Syntax

```DAX
INFO.ATTRIBUTEHIERARCHIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for attribute hierarchies in the current semantic model:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[ColumnID]	An ID-based reference to a Column object.
[State]	A value that provides information about the state of the AttributeHierarchy object. The possible values and their interpretation are as follows: **Ready (1)** – The Attribute Hierarchy is queryable and has up-to-date data. **CalculationNeeded (4)** – The Attribute Hierarchy does not contain any data because it was not refreshed. There is no error associated with the attribute hierarchy. **DependencyError (7)** – The column that is associated with this Attribute Hierarchy is in an error state (SemanticError, EvaluationError, or DependencyError).
[AttributeHierarchyStorageID]	An ID-based reference to an AttributeHierarchyStorage object, which can be enumerated with INFO.ATTRIBUTEHIERARCHYSTORAGES
[ModifiedTime]	The time that the object was last modified.
[RefreshedTime]	The time that the object was last refreshed.

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ATTRIBUTEHIERARCHIES()
```

This DAX query returns a table with all of the columns of this DAX function.

See also

INFO.HIERARCHIES INFO.HIERARCHYSTORAGES INFO.ATTRIBUTEHIERARCHYSTORAGES INFO.LEVELS INFO.TABLES

Last updated on 01/08/2026

INFO.ATTRIBUTEHIERARCHYSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each attribute hierarchy storage in the semantic model. This function provides metadata about the storage characteristics of attribute hierarchies.

Syntax

```DAX
INFO.ATTRIBUTEHIERARCHYSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for attribute hierarchy storages in the current semantic model:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[AttributeHierarchyID]	An ID-based reference to a AttributeHierarchy object.
[SortOrder]	The sort order used for the attribute hierarchy storage.
[OptimizationLevel]	The optimization level applied to the attribute hierarchy storage.
[MaterializationType]	The type of materialization used for the attribute hierarchy.
[ColumnPositionToData]	Mapping information from column position to data values.
[ColumnDataToPosition]	Mapping information from data values to column position.
[DistinctDataCount]	The number of distinct data values in the attribute hierarchy.
[DataVersion]	The version of the data in the attribute hierarchy storage.
[StorageFileID]	Reference to the storage file containing the attribute hierarchy data.
[SystemTableID]	Reference to the system table associated with the attribute hierarchy.
[HasStatistics]	Boolean indicating whether statistics are available for the attribute hierarchy.
[MinValue]	The minimum value in the attribute hierarchy.
[MaxValue]	The maximum value in the attribute hierarchy.
[StringValueMaxLength]	The maximum length of string values in the attribute hierarchy.

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ATTRIBUTEHIERARCHYSTORAGES()
```

See also

INFO.HIERARCHIES INFO.ATTRIBUTEHIERARCHIES INFO.HIERARCHYSTORAGES INFO.LEVELS INFO.TABLES

Last updated on 01/08/2026

INFO.CALCDEPENDENCY

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calculation dependency in the semantic model. This information helps you understand the model.

Syntax

```
DAX
INFO.CALCDEPENDENCY ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[DATABASE_NAME]	The name of the semantic model.
[OBJECT_TYPE]	The type of object.
[TABLE]	The object's table name.
[OBJECT]	The name of the object.
[EXPRESSION]	The DAX formula of the object.
[REFERENCED_OBJECT_TYPE]	The type of object this object references. The "Object" is dependent on this object.
[REFERENCED_TABLE]	The referenced object's table name.
[REFERENCED_OBJECT]	The referenced object's name.
[REFERENCED_EXPRESSION]	The referenced object's DAX formula.
[QUERY]	The query, if specified as a restriction.

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

You can also call this DAX function with INFO.DEPENDENCIES.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALCDEPENDENCY()
```

This DAX query returns a table with all of the columns of this DAX function.

Example 2 - DAX query with restriction

The following DAX query can be run in DAX query view and limits the results to the Total Sales measure:

DAX

```
EVALUATE
    INFO.CALCDEPENDENCY("Query", "EVALUATE { [Total Sales] }")
```

If a restriction has double-quotes they can be escaped with another double-quote and you can optionally use a VAR to hold the value.

DAX

```
EVALUATE
    VAR _query =
        "EVALUATE
        SELECTCOLUMNS(
            'Date',
            ""Date"", [Date]
        )"
    RETURN
        INFO.CALCDEPENDENCY(
            "Query",
            _query
        )
```

See also

INFO.DEPENDENCIES INFO.RELATEDCOLUMNDETAILS INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.CALCULATIONGROUPS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calculation group in the semantic model. This function provides metadata about calculation groups and their properties.

Syntax

```DAX
INFO.CALCULATIONGROUPS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	The unique identifier of the calculation group
[TableID]	Integer	The unique identifier of the table that contains the calculation group
[Description]	String	The description of the calculation group
[ModifiedTime]	DateTime	The date and time when the calculation group was last modified
[Precedence]	Integer	The precedence value of the calculation group for evaluation order

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALCULATIONGROUPS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _CalculationGroups =
        INFO.CALCULATIONGROUPS()

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
```

```
        _CalculationGroups,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Table Name", [Table Name],
        "Description", [Description],
        "Precedence", [Precedence],
        "Modified Time", [ModifiedTime]
    )
ORDER BY [Precedence]
```

See also

INFO.CALCULATIONITEMS INFO.MEASURES INFO.KPIS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.CALCULATIONITEMS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calculation item in the semantic model. This function provides metadata about calculation items within calculation groups.

Syntax

```DAX
INFO.CALCULATIONITEMS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	The unique identifier of the calculation item
[CalculationGroupID]	Integer	The unique identifier of the calculation group that contains this calculation item
[FormatStringDefinition]	String	The format string definition for the calculation item
[Name]	String	The name of the calculation item
[Description]	String	The description of the calculation item
[ModifiedTime]	DateTime	The date and time when the calculation item was last modified
[State]	String	The state of the calculation item
[ErrorMessage]	String	Any error message associated with the calculation item
[Expression]	String	The DAX expression for the calculation item
[Ordinal]	Integer	The ordinal position of the calculation item within its calculation group

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALCULATIONITEMS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _CalculationItems =
        INFO.CALCULATIONITEMS()

    VAR _CalculationGroups =
        SELECTCOLUMNS(
            INFO.CALCULATIONGROUPS(),
            "CalculationGroupID", [ID],
            "Calculation Group Description", [Description]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _CalculationItems,
            _CalculationGroups
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Calculation Item Name", [Name],
            "Calculation Group Description", [Calculation Group Description],
            "Expression", [Expression],
            "Ordinal", [Ordinal]
        )

    ORDER BY [Calculation Group Description], [Ordinal]
```

See also

INFO.CALCULATIONGROUPS INFO.MEASURES INFO.KPIS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.CALENDARCOLUMNGROUPS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calendar column group in the semantic model. Calendar column groups associate columns with categories within a calendar.

Syntax

```
DAX
INFO.CALENDARCOLUMNGROUPS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
ID	Unique identifier for the calendar column group
CalendarID	ID of the calendar this calendar column group belongs to
TimeUnit	The category associated with this column group (e.g. DayOfWeek)
ModifiedTime	When the calendar column group was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALENDARCOLUMNGROUPS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _CalendarColumnGroups =
    SELECTCOLUMNS(
        INFO.CALENDARCOLUMNGROUPS(),
        "GroupID", [ID],
        "CalendarID", [CalendarID],
        "Category", [TimeUnit]
    )

VAR _CalendarColumnReferences =
    SELECTCOLUMNS(
        INFO.CALENDARCOLUMNREFERENCES(),
        "GroupID", [CalendarColumnGroupID],
        "ColumnID", [ColumnID],
        "IsPrimaryColumn", [IsPrimaryColumn]
    )
```

```
VAR _Calendars =
    SELECTCOLUMNS(
        INFO.CALENDARS(),
        "CalendarID", [ID],
        "Calendar Name", [Name]
    )

VAR _Columns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "Column Name", [ExplicitName]
    )

VAR _GroupsWithReferences =
    NATURALLEFTOUTERJOIN(
        _CalendarColumnGroups,
        _CalendarColumnReferences
    )

VAR _WithCalendars =
    NATURALLEFTOUTERJOIN(
        _GroupsWithReferences,
        _Calendars
    )

VAR _WithColumns =
    NATURALLEFTOUTERJOIN(
        _WithCalendars,
        _Columns
    )

RETURN
    SELECTCOLUMNS(
        _WithColumns,
        "Calendar Name", [Calendar Name],
        "Category", [Category],
        "Is Primary Column", [IsPrimaryColumn],
        "Column Name", [Column Name]
    )
ORDER BY [Calendar Name], [Category], [Is Primary Column], [Column Name]
```

See also

- INFO.CALENDARS
- INFO.CALENDARCOLUMNREFERENCES
- INFO.COLUMNS
- INFO functions overview

Last updated on 04/13/2026

INFO.CALENDARCOLUMNREFERENCES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calendar column reference in the semantic model. Calendar column references associate columns to calendar column groups.

Syntax

```
DAX
INFO.CALENDARCOLUMNREFERENCES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
ID	Unique identifier for the calendar column reference
CalendarColumnGroupID	ID of the calendar column group this calendar column reference belongs to
ColumnID	ID of the column being referenced
IsPrimaryColumn	Indicates whether this is the primary column in the calendar column group
ModifiedTime	When the calendar column reference was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALENDARCOLUMNREFERENCES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _CalendarColumnGroups =
    SELECTCOLUMNS(
        INFO.CALENDARCOLUMNGROUPS(),
        "GroupID", [ID],
        "CalendarID", [CalendarID],
        "Category", [TimeUnit]
    )

VAR _CalendarColumnReferences =
    SELECTCOLUMNS(
        INFO.CALENDARCOLUMNREFERENCES(),
        "GroupID", [CalendarColumnGroupID],
        "ColumnID", [ColumnID],
```

```
        "IsPrimaryColumn", [IsPrimaryColumn]
    )

VAR _Calendars =
    SELECTCOLUMNS(
        INFO.CALENDARS(),
        "CalendarID", [ID],
        "Calendar Name", [Name]
    )

VAR _Columns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "Column Name", [ExplicitName]
    )

VAR _GroupsWithReferences =
    NATURALLEFTOUTERJOIN(
        _CalendarColumnGroups,
        _CalendarColumnReferences
    )

VAR _WithCalendars =
    NATURALLEFTOUTERJOIN(
        _GroupsWithReferences,
        _Calendars
    )

VAR _WithColumns =
    NATURALLEFTOUTERJOIN(
        _WithCalendars,
        _Columns
    )

RETURN
    SELECTCOLUMNS(
        _WithColumns,
        "Calendar Name", [Calendar Name],
        "Category", [Category],
        "Is Primary Column", [IsPrimaryColumn],
        "Column Name", [Column Name]
    )
ORDER BY [Calendar Name], [Category], [Is Primary Column], [Column Name]
```

See also

- INFO.CALENDARS
- INFO.CALENDARCOLUMNGROUPS
- INFO.COLUMNS
- INFO functions overview

Last updated on 04/13/2026

INFO.CALENDARS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each calendar in the semantic model. This function provides metadata about calendars defined on tables.

Syntax

```
DAX
INFO.CALENDARS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
ID	Unique identifier for the calendar
TableID	ID of the table this calendar belongs to
Name	Name of the calendar
Description	Description of the calendar
LineageTag	Lineage tag for tracking calendar lineage
SourceLineageTag	Source lineage tag
ModifiedTime	When the calendar was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CALENDARS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _Calendars =
    SELECTCOLUMNS(
        INFO.CALENDARS(),
        "CalendarID", [ID],
        "Calendar Name", [Name],
        "Description", [Description],
        "TableID", [TableID]
    )
```

```
VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _Calendars,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Table Name", [Table Name],
        "Calendar Name", [Calendar Name],
        "Description", [Description]
    )
ORDER BY [Table Name], [Calendar Name]
```

See also

- INFO.CALENDARCOLUMNGROUPS
- INFO.CALENDARCOLUMNREFERENCES
- INFO.TABLES
- INFO functions overview

Last updated on 04/13/2026

INFO.CATALOGS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each catalog in the semantic model. This function provides metadata about the catalogs available in the current context.

Syntax

```DAX
INFO.CATALOGS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[CATALOG_NAME]	String	The name of the catalog
[DESCRIPTION]	String	The description of the catalog
[ROLES]	String	The roles associated with the catalog
[DATE_MODIFIED]	DateTime	The date and time when the catalog was last modified
[COMPATIBILITY_LEVEL]	Integer	The compatibility level of the catalog
[TYPE]	Integer	The type of the catalog
[VERSION]	Integer	The version of the catalog
[DATABASE_ID]	String	The unique identifier of the database
[DATABASE_GUID]	String	The GUID of the database
[DATE_QUERIED]	DateTime	The date and time when the catalog was queried
[CURRENTLY_USED]	Boolean	Whether the catalog is currently being used
[POPULARITY]	Double	The popularity score of the catalog
[WEIGHTEDPOPULARITY]	Double	The weighted popularity score of the catalog
[CLIENTCACHEREFRESHPOLICY]	Integer	The client cache refresh policy setting
[ENCRYPTION_LEVEL]	String	The encryption level used for the catalog
[CRYPTOKEY_UPDATED]	DateTime	The date and time when the encryption key was last updated

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

```
DAX
```

```
EVALUATE
    INFO.CATALOGS()
```

See also

INFO.MODEL INFO.PROPERTIES INFO.EXPRESSIONS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.CHANGEDPROPERTIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each changed property in the semantic model. This function provides metadata about properties that have been modified in the model.

Syntax

DAX

```
INFO.CHANGEDPROPERTIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	The unique identifier of the changed property
[ObjectID]	Integer	The identifier of the object that has the changed property
[ObjectType]	Integer	The type of object that has the changed property
[Property]	String	The name of the property that was changed

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CHANGEDPROPERTIES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ChangedProperties =
        INFO.CHANGEDPROPERTIES()

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "ObjectID", [ID],
            "Object Name", [Name]
        )

    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ObjectID", [ID],
```

```
            "Object Name", [ExplicitName]
        )

    VAR _AllObjects =
        UNION(_Tables, _Columns)

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _ChangedProperties,
            _AllObjects
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Object Name", [Object Name],
            "Object Type", [ObjectType],
            "Changed Property", [Property]
        )
    ORDER BY [Object Name], [Changed Property]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.COLUMNPARTITIONSTORAGES

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✗ Visual calculation ✓ DAX query

Returns a table with information about each column partition storage in the semantic model. This function provides metadata about how column partitions are stored.

Syntax

```DAX
INFO.COLUMNPARTITIONSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	The unique identifier of the column partition storage
[ColumnStorageID]	Integer	The identifier of the column storage this partition belongs to
[PartitionStorageID]	Integer	The identifier of the partition storage
[DataVersion]	Integer	The version of the data in the partition storage
[State]	Integer	The current state of the column partition storage
[SegmentStorageID]	Integer	The identifier of the segment storage
[StorageFileID]	Integer	The identifier of the storage file

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.COLUMNPARTITIONSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ColumnPartitionStorages =
        INFO.COLUMNPARTITIONSTORAGES()

    VAR _ColumnStorages =
        SELECTCOLUMNS(
            INFO.COLUMNSTORAGES(),
            "ColumnStorageID", [ID],
            "ColumnID", [ColumnID],
```

```
        "Storage Name", [Name]
    )

VAR _Columns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "Column Name", [ExplicitName],
        "TableID", [TableID]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _CombinedWithColumnStorages =
    NATURALLEFTOUTERJOIN(
        _ColumnPartitionStorages,
        _ColumnStorages
    )

VAR _CombinedWithColumns =
    NATURALLEFTOUTERJOIN(
        _CombinedWithColumnStorages,
        _Columns
    )

VAR _CombinedWithTables =
    NATURALLEFTOUTERJOIN(
        _CombinedWithColumns,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedWithTables,
        "Table Name", [Table Name],
        "Column Name", [Column Name],
        "Storage Name", [Storage Name],
        "Data Version", [DataVersion],
        "State", [State]
    )
ORDER BY [Table Name], [Column Name]
```

See also

INFO.COLUMNSTORAGES INFO.DICTIONARYSTORAGES INFO.SEGMENTSTORAGES INFO.STORAGEFILES INFO.TABLESTORAGES

Last updated on 01/08/2026

INFO.COLUMNPERMISSIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each column permission in the semantic model. This function provides metadata about column-level security settings.

Syntax

```
DAX
INFO.COLUMNPERMISSIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	The unique identifier of the column permission
[TablePermissionID]	Integer	The identifier of the table permission this column permission belongs to
[ColumnID]	Integer	The identifier of the column
[ModifiedTime]	DateTime	The date and time when the column permission was last modified
[MetadataPermission]	Integer	The metadata permission level for the column

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.COLUMNPERMISSIONS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ColumnPermissions =
        INFO.COLUMNPERMISSIONS()

    VAR _TablePermissions =
        SELECTCOLUMNS(
            INFO.TABLEPERMISSIONS(),
            "TablePermissionID", [ID],
            "Table Permission Name", [Name],
            "RoleID", [RoleID],
            "TableID", [TableID]
```

```
    )

VAR _Roles =
    SELECTCOLUMNS(
        INFO.ROLES(),
        "RoleID", [ID],
        "Role Name", [Name]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _Columns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "Column Name", [ExplicitName]
    )

VAR _CombinedWithTablePermissions =
    NATURALLEFTOUTERJOIN(
        _ColumnPermissions,
        _TablePermissions
    )

VAR _CombinedWithRoles =
    NATURALLEFTOUTERJOIN(
        _CombinedWithTablePermissions,
        _Roles
    )

VAR _CombinedWithTables =
    NATURALLEFTOUTERJOIN(
        _CombinedWithRoles,
        _Tables
    )

VAR _CombinedWithColumns =
    NATURALLEFTOUTERJOIN(
        _CombinedWithTables,
        _Columns
    )

RETURN
    SELECTCOLUMNS(
        _CombinedWithColumns,
        "Role Name", [Role Name],
        "Table Name", [Table Name],
        "Column Name", [Column Name],
        "Metadata Permission", [MetadataPermission],
        "Modified Time", [ModifiedTime]
```

```
    )
ORDER BY [Role Name], [Table Name], [Column Name]
```

See also

INFO.ROLES INFO.ROLEMEMBERSHIPS INFO.TABLEPERMISSIONS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.COLUMNS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each column in the semantic model. This function provides metadata about all columns including their properties and characteristics.

Syntax

```DAX
INFO.COLUMNS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data type	Description
[ID]	Integer	Unique identifier for the column
[TableID]	Integer	Identifier of the table that contains this column
[ExplicitName]	String	The explicitly defined name of the column
[InferredName]	String	The inferred name of the column
[ExplicitDataType]	Integer	The explicitly defined data type of the column
[InferredDataType]	Integer	The inferred data type of the column
[DataCategory]	String	The data category classification for the column
[Description]	String	The description of the column
[IsHidden]	Boolean	Whether the column is hidden from client tools
[State]	Integer	The current state of the column
[IsUnique]	Boolean	Whether the column contains unique values
[IsKey]	Boolean	Whether the column is a key column
[IsNullable]	Boolean	Whether the column can contain null values
[Alignment]	Integer	The alignment setting for the column
[TableDetailPosition]	Integer	The position of the column in table detail views
[IsDefaultLabel]	Boolean	Whether this column is the default label
[IsDefaultImage]	Boolean	Whether this column is the default image
[SummarizeBy]	Integer	How the column should be summarized
[ColumnStorageID]	Integer	Identifier for the column's storage
[Type]	Integer	The type classification of the column
[SourceColumn]	String	The source column name
[ColumnOriginID]	Integer	Identifier for the column's origin
[Expression]	String	The DAX expression for calculated columns
[FormatString]	String	The format string for the column
[IsAvailableInMDX]	Boolean	Whether the column is available in MDX queries
[SortByColumnID]	Integer	Identifier of the column to sort by

Column	Data type	Description
[AttributeHierarchyID]	Integer	Identifier of the attribute hierarchy
[ModifiedTime]	DateTime	When the column was last modified
[StructureModifiedTime]	DateTime	When the column structure was last modified
[RefreshedTime]	DateTime	When the column data was last refreshed
[SystemFlags]	Integer	System flags for the column
[KeepUniqueRows]	Boolean	Whether to keep unique rows
[DisplayOrdinal]	Integer	The display order of the column
[ErrorMessage]	String	Any error message associated with the column
[SourceProviderType]	String	The source provider type
[DisplayFolder]	String	The display folder for organizing columns
[EncodingHint]	Integer	Encoding hint for the column
[RelatedColumnDetailsID]	Integer	Identifier for related column details
[AlternateOfID]	Integer	Identifier if this is an alternate of another column
[LineageTag]	String	The lineage tag for tracking column lineage
[SourceLineageTag]	String	The source lineage tag

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.COLUMNS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Columns =
        INFO.COLUMNS()

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _Columns,
            _Tables
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Table Name", [Table Name],
            "Column Name", [ExplicitName],
            "Data Type", [ExplicitDataType],
            "Is Hidden", [IsHidden],
            "Is Key", [IsKey]
        )
    ORDER BY [Table Name], [Column Name]
```

See also

INFO.TABLES INFO.PARTITIONS INFO.RELATIONSHIPS INFO functions overview

Last updated on 01/08/2026

INFO.COLUMNSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each column storage in the semantic model. This function provides metadata about how columns are stored and their storage characteristics.

Syntax

```DAX
INFO.COLUMNSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data type	Description
[ID]	Integer	Unique identifier for the column storage
[ColumnID]	Integer	Identifier of the column that this storage belongs to
[Name]	String	Name of the column storage
[StoragePosition]	Integer	Position of the column in storage
[DictionaryStorageID]	Integer	Identifier for the dictionary storage
[Settings]	String	Storage settings and configuration
[ColumnFlags]	Integer	Flags indicating column storage properties
[Collation]	String	Collation settings for the column
[OrderByColumn]	String	Column used for ordering
[Locale]	String	Locale information for the column
[BinaryCharacters]	String	Binary character information
[Statistics_DistinctStates]	Integer	Number of distinct states in statistics
[Statistics_MinDataID]	Integer	Minimum data ID in statistics
[Statistics_MaxDataID]	Integer	Maximum data ID in statistics
[Statistics_OriginalMinSegmentDataID]	Integer	Original minimum segment data ID
[Statistics_RLESortOrder]	Integer	RLE sort order in statistics
[Statistics_RowCount]	Integer	Row count in statistics
[Statistics_HasNulls]	Boolean	Whether statistics contain nulls
[Statistics_RLERuns]	Integer	Number of RLE runs in statistics
[Statistics_OthersRLERuns]	Integer	Number of other RLE runs
[Statistics_Usage]	Integer	Usage statistics
[Statistics_DBType]	Integer	Database type in statistics
[Statistics_XMType]	Integer	XML type in statistics

Remarks

- Typically used in DAX queries to inspect and document model metadata.

- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.COLUMNSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ColumnStorages =
        INFO.COLUMNSTORAGES()

    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ColumnID", [ID],
            "Column Name", [ExplicitName],
            "TableID", [TableID]
        )

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _CombinedWithColumns =
        NATURALLEFTOUTERJOIN(
            _ColumnStorages,
            _Columns
        )

    VAR _CombinedWithTables =
        NATURALLEFTOUTERJOIN(
            _CombinedWithColumns,
            _Tables
        )
```

```
RETURN
    SELECTCOLUMNS(
        _CombinedWithTables,
        "Table Name", [Table Name],
        "Column Name", [Column Name],
        "Storage Name", [Name],
        "Storage Position", [StoragePosition],
        "Statistics Row Count", [Statistics_RowCount]
    )
ORDER BY [Table Name], [Column Name]
```

See also

INFO.COLUMNPARTITIONSTORAGES INFO.DICTIONARYSTORAGES INFO.SEGMENTSTORAGES INFO.STORAGEFILES INFO.TABLESTORAGES

Last updated on 01/08/2026

INFO.CSDLMETADATA

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns a table with information about the CSDL metadata in the semantic model. This function provides metadata about the Conceptual Schema Definition Language representation of the model.

Syntax

```DAX
INFO.CSDLMETADATA ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for CSDL metadata in the current semantic model.

Expand table

Column	Description
[METADATA]	The complete CSDL (Conceptual Schema Definition Language) XML metadata

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CSDLMETADATA()
```

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/24/2026

INFO.CULTURES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each culture in the semantic model. This function provides metadata about the cultures and locales supported by the model.

Syntax

```DAX
INFO.CULTURES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the culture
[ModelID]	Integer	Identifier of the semantic model
[Name]	String	Name of the culture (e.g., en-US)
[LinguisticMetadataID]	Integer	Identifier for linguistic metadata associated with the culture
[ModifiedTime]	DateTime	Date and time when the culture was last modified
[StructureModifiedTime]	DateTime	Date and time when the culture structure was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.CULTURES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Cultures =
        INFO.CULTURES()

    VAR _Model =
        SELECTCOLUMNS(
            INFO.MODEL(),
            "ModelID", [ID],
            "Model Name", [Name]
        )
```

```
VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _Cultures,
        _Model
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Model Name", [Model Name],
        "Culture Name", [Name],
        "Modified Time", [ModifiedTime],
        "Structure Modified Time", [StructureModifiedTime]
    )
ORDER BY [Culture Name]
```

See also

INFO.OBJECTTRANSLATIONS INFO.LINGUISTICMETADATA INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.DATACOVERAGEDEFINITIONS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns a table with information about each data coverage definition in the semantic model. This function provides metadata about data coverage settings and definitions.

Syntax

```DAX
INFO.DATACOVERAGEDEFINITIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the data coverage definition
[PartitionID]	Identifier of the partition associated with the data coverage definition
[Description]	Description of the data coverage definition
[Expression]	DAX expression that is evaluated for the data coverage definition.
[ModifiedTime]	Timestamp of when the data coverage definition was last modified
[State]	Current state of the data coverage definition (e.g., Ready, Error)
[ErrorMessage]	Error message if the data coverage definition is in an error state

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.DATACOVERAGEDEFINITIONS()
```

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/26/2026

INFO.DATASOURCES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each data source in the semantic model. This function provides metadata about the data sources connected to the model.

Syntax

```DAX
INFO.DATASOURCES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the data source
[ModelID]	Integer	Identifier of the semantic model
[Name]	String	Name of the data source
[Description]	String	Description of the data source
[Type]	String	Type of the data source
[ConnectionString]	String	Connection string for the data source
[ImpersonationMode]	String	Impersonation mode used for the data source
[Account]	String	Account used for impersonation
[Password]	String	Password for the data source (typically masked)
[MaxConnections]	Integer	Maximum number of connections allowed
[Isolation]	String	Isolation level for the data source
[Timeout]	Integer	Timeout value for connections
[Provider]	String	Data provider for the data source
[ModifiedTime]	DateTime	Date and time when the data source was last modified
[ConnectionDetails]	String	Additional connection details
[Options]	String	Additional options for the data source
[Credential]	String	Credential information
[ContextExpression]	String	Context expression for the data source

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

```
```

DAX

```
EVALUATE
    INFO.DATASOURCES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _DataSources =
        INFO.DATASOURCES()

    VAR _Model =
        SELECTCOLUMNS(
            INFO.MODEL(),
            "ModelID", [ID],
            "Model Name", [Name]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _DataSources,
            _Model
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Model Name", [Model Name],
            "Data Source Name", [Name],
            "Data Source Type", [Type],
            "Max Connections", [MaxConnections],
            "Modified Time", [ModifiedTime]
        )
    ORDER BY [Data Source Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

Last updated on 01/08/2026

INFO.DELTATABLEMETADATASTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each delta table metadata storage in the semantic model. This function provides metadata about delta table storage characteristics.

Syntax

DAX

```
INFO.DELTATABLEMETADATASTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the delta table metadata storage
[PartitionStorageID]	Integer	Identifier of the partition storage
[DataSourceReference]	String	Reference to the data source
[TableName]	String	Name of the delta table
[RootLocation]	String	Root location of the delta table
[CurrentVersion]	Integer	Current version of the delta table
[TableObjectID]	String	Object identifier for the table
[DatamartObjectID]	String	Object identifier for the datamart
[FramedSchemaName]	String	Name of the framed schema
[FallbackReason]	String	Reason for fallback behavior
[DeltaColumnMappingMode]	String	Column mapping mode for the delta table
[DeltaLogETag]	String	ETag for the delta log
[ArtifactObjectId]	String	Object identifier for the artifact
[ArtifactSecurityVersion]	String	Security version of the artifact

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

```
DAX
EVALUATE
    INFO.DELTATABLEMETADATASTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _DeltaTableMetadataStorages =
        INFO.DELTATABLEMETADATASTORAGES()

    VAR _PartitionStorages =
        SELECTCOLUMNS(
            INFO.PARTITIONSTORAGES(),
            "PartitionStorageID", [ID],
            "Partition Storage Name", [Name]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _DeltaTableMetadataStorages,
            _PartitionStorages
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Table Name", [TableName],
            "Partition Storage Name", [Partition Storage Name],
            "Current Version", [CurrentVersion],
            "Root Location", [RootLocation],
            "Fallback Reason", [FallbackReason]
        )
    ORDER BY [Table Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.DEPENDENCIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each dependency in the semantic model. This function provides metadata about object dependencies and relationships between model objects.

Syntax

DAX

```
INFO.DEPENDENCIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[DATABASE_NAME]	String	Name of the database containing the object
[OBJECT_TYPE]	String	Type of the object (e.g., ATTRIBUTE_HIERARCHY, HIERARCHY, MEASURE)
[TABLE]	String	Name of the table containing the object
[OBJECT]	String	Name of the object
[EXPRESSION]	String	Expression associated with the object
[REFERENCED_OBJECT_TYPE]	String	Type of the referenced object
[REFERENCED_TABLE]	String	Name of the table containing the referenced object
[REFERENCED_OBJECT]	String	Name of the referenced object
[REFERENCED_EXPRESSION]	String	Expression of the referenced object
[QUERY]	String	Query context for the dependency

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.DEPENDENCIES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Dependencies =
        INFO.DEPENDENCIES()

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TABLE", [Name],
            "Table ID", [ID]
        )

    VAR _ReferencedTables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "REFERENCED_TABLE", [Name],
            "Referenced Table ID", [ID]
        )

    VAR _CombinedWithTables =
        NATURALLEFTOUTERJOIN(
            _Dependencies,
            _Tables
        )

    VAR _CombinedWithReferencedTables =
        NATURALLEFTOUTERJOIN(
            _CombinedWithTables,
            _ReferencedTables
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedWithReferencedTables,
            "Object Type", [OBJECT_TYPE],
            "Table", [TABLE],
            "Object", [OBJECT],
            "Referenced Object Type", [REFERENCED_OBJECT_TYPE],
            "Referenced Table", [REFERENCED_TABLE],
            "Referenced Object", [REFERENCED_OBJECT]
        )
    ORDER BY [TABLE], [OBJECT]
```

See also

INFO.CALCDEPENDENCY INFO.RELATEDCOLUMNDETAILS INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.DETAILROWSDEFINITIONS

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✗ Visual calculation ✓ DAX query

Returns a table with information about each detail rows definition in the semantic model. This function provides metadata about detail rows definitions for measures.

Syntax

DAX

```
INFO.DETAILROWSDEFINITIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the detail rows definition
[ObjectID]	Integer	Identifier of the object this detail rows definition belongs to
[ObjectType]	String	Type of the object (e.g., measure, table)
[Expression]	String	DAX expression for the detail rows definition
[ModifiedTime]	DateTime	Date and time when the detail rows definition was last modified
[State]	Integer	State of the detail rows definition
[ErrorMessage]	String	Error message if the detail rows definition has an error

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.DETAILROWSDEFINITIONS()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.DICTIONARYSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each dictionary storage in the semantic model. This function provides metadata about dictionary storage characteristics and compression.

Syntax

```DAX
INFO.DICTIONARYSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the dictionary storage
[ColumnStorageID]	Integer	Identifier of the column storage this dictionary belongs to
[Type]	Integer	Type of the dictionary storage
[DataType]	Integer	Data type stored in the dictionary
[DataVersion]	Integer	Version of the data format
[BaseId]	Integer	Base identifier for the dictionary
[Magnitude]	Integer	Magnitude value for the dictionary
[LastId]	Integer	Last identifier used in the dictionary
[IsNullable]	Boolean	Whether the dictionary can contain null values
[IsUnique]	Boolean	Whether all values in the dictionary are unique
[IsOperatingOn32]	Boolean	Whether the dictionary operates on 32-bit values
[DictionaryFlags]	Integer	Flags indicating dictionary characteristics
[StorageFileID]	Integer	Identifier of the storage file containing the dictionary
[Size]	Integer	Size of the dictionary in bytes

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.DICTIONARYSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _DictionaryStorages =
        INFO.DICTIONARYSTORAGES()

    VAR _ColumnStorages =
        SELECTCOLUMNS(
            INFO.COLUMNSTORAGES(),
            "ColumnStorageID", [ID],
            "Column Storage Name", [Name],
            "ColumnID", [ColumnID]
        )

    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ColumnID", [ID],
            "Column Name", [ExplicitName],
            "TableID", [TableID]
        )

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _CombinedWithColumnStorages =
        NATURALLEFTOUTERJOIN(
            _DictionaryStorages,
            _ColumnStorages
        )

    VAR _CombinedWithColumns =
        NATURALLEFTOUTERJOIN(
            _CombinedWithColumnStorages,
            _Columns
        )

    VAR _CombinedWithTables =
        NATURALLEFTOUTERJOIN(
            _CombinedWithColumns,
            _Tables
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedWithTables,
            "Table Name", [Table Name],
            "Column Name", [Column Name],
```

```
        "Dictionary Type", [Type],
        "Data Type", [DataType],
        "Is Nullable", [IsNullable],
        "Size", [Size]
    )
ORDER BY [Table Name], [Column Name]
```

See also

INFO.COLUMNSTORAGES INFO.COLUMNPARTITIONSTORAGES INFO.SEGMENTSTORAGES INFO.STORAGEFILES INFO.TABLESTORAGES

Last updated on 01/08/2026

INFO.EXCLUDEDARTIFACTS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each excluded artifact in the semantic model. This function provides metadata about artifacts that are excluded from the model.

Syntax

```DAX
INFO.EXCLUDEDARTIFACTS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for excluded artifacts in the current semantic model.

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

```dax
EVALUATE
    INFO.EXCLUDEDARTIFACTS()
```

Example 2 - DAX query with SELECTCOLUMNS

```dax
EVALUATE
    SELECTCOLUMNS(
        INFO.EXCLUDEDARTIFACTS(),
        "ObjectID", [ObjectID],
        "ObjectType", [ObjectType],
        "Reason", [Reason]
    )
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.EXPRESSIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each expression in the semantic model. This function provides metadata about expressions defined in the model.

Syntax

```DAX
INFO.EXPRESSIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the expression
[ModelID]	Integer	Identifier of the model containing the expression
[Name]	String	Name of the expression
[Description]	String	Description of the expression
[Kind]	Integer	Type of expression (e.g., calculated table, shared expression)
[Expression]	String	DAX expression text
[ModifiedTime]	DateTime	When the expression was last modified
[QueryGroupID]	Integer	Identifier of the query group if applicable
[ParameterValuesColumnID]	Integer	Column identifier for parameter values if applicable
[MAttributes]	String	Additional attributes in JSON format
[LineageTag]	String	Lineage tag for tracking purposes
[SourceLineageTag]	String	Source lineage tag for tracking purposes

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.EXPRESSIONS()
```

See also

INFO.MODEL INFO.PROPERTIES INFO.CATALOGS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.EXTENDEDPROPERTIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each extended property in the semantic model. This function provides metadata about extended properties defined for model objects.

Syntax

```DAX
INFO.EXTENDEDPROPERTIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the extended property
[ObjectID]	Integer	Identifier of the object that has this extended property
[ObjectType]	Integer	Type of object (table, column, measure, etc.)
[Name]	String	Name of the extended property
[Type]	Integer	Data type of the property value
[Value]	String	Value of the extended property
[ModifiedTime]	DateTime	When the extended property was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.EXTENDEDPROPERTIES()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.FORMATSTRINGDEFINITIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each format string definition in the semantic model. This function provides metadata about format string definitions for measures and columns.

Syntax

```
DAX
INFO.FORMATSTRINGDEFINITIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column name	Data type	Description
[ID]	Integer	Unique identifier for the format string definition
[ObjectID]	Integer	Identifier of the object (measure or column) that uses this format string
[ObjectType]	Integer	Type of object (measure, column, etc.)
[Expression]	String	DAX expression that defines the format string
[ModifiedTime]	DateTime	When the format string definition was last modified
[State]	Integer	Current state of the format string definition
[ErrorMessage]	String	Error message if the format string definition is in an error state

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.FORMATSTRINGDEFINITIONS()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.FUNCTIONS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns information about the functions currently available for use in DAX. This corresponds to the MDSCHEMA_FUNCTIONS schema rowset, but returns only DAX (and not MDX) functions by default.

Syntax

```
DAX
INFO.FUNCTIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[FUNCTION_NAME]	The name of the DAX function
[DESCRIPTION]	A description of what the function does
[PARAMETER_LIST]	The list of parameters the function accepts
[RETURN_TYPE]	The data type that the function returns
[ORIGIN]	The origin or source of the function
[INTERFACE_NAME]	The interface name for the function
[LIBRARY_NAME]	The library containing the function
[DLL_NAME]	The DLL file containing the function implementation
[HELP_FILE]	The help file associated with the function
[HELP_CONTEXT]	The help context identifier
[OBJECT]	The object information for the function
[CAPTION]	The display caption for the function
[PARAMETERINFO]	Additional parameter information
[DIRECTQUERY_PUSHABLE]	Whether the function can be pushed down to DirectQuery sources
[VISUAL_CALCULATIONS_INFO]	Information about visual calculations support

Remarks

- The result can vary by engine version and compatibility level.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.
- Possible values for [ORIGIN] are 1 (MSOLAP), 2 (UDF), 3 (RELATIONAL), 4 (SCALAR).
- If the [ORIGIN] restriction is not specified, it defaults to 3 (RELATIONAL) or 4 (SCALAR).

Example

The following DAX query can be run in DAX query view:

```
DAX
```

```
EVALUATE
    INFO.FUNCTIONS()
```

Example 2 - DAX query with filters

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    SELECTCOLUMNS(
        FILTER(
            INFO.FUNCTIONS(),
            CONTAINSSTRING([FUNCTION_NAME], "DATE")
        ),
        "Function Name", [FUNCTION_NAME],
        "Description", [DESCRIPTION],
        "Return Type", [RETURN_TYPE]
    )
ORDER BY [Function Name]
```

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/11/2026

INFO.GENERALSEGMENTMAPSEGMENTMETADATASTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each general segment map segment metadata storage in the semantic model. This function provides metadata about segment map storage characteristics.

Syntax

DAX

```
INFO.GENERALSEGMENTMAPSEGMENTMETADATASTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the general segment map segment metadata storage
[SegmentMapStorageID]	Identifier linking to the segment map storage
[RecordCount]	Number of records in the segment
[Ordinal]	Ordinal position of the segment

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.GENERALSEGMENTMAPSEGMENTMETADATASTORAGES()
```

Example 2 - DAX query with SELECTCOLUMNS

DAX

```
EVALUATE
    SELECTCOLUMNS(
        INFO.GENERALSEGMENTMAPSEGMENTMETADATASTORAGES(),
        "Segment Map Storage ID", [SegmentMapStorageID],
        "Record Count", [RecordCount],
        "Ordinal", [Ordinal]
    )
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

Last updated on 01/08/2026

INFO.GROUPBYCOLUMNS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each group by column in the semantic model. This function provides metadata about columns used in group by operations.

Syntax

DAX

```
INFO.GROUPBYCOLUMNS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the group by column
[RelatedColumnDetailsID]	Identifier linking to related column details
[GroupingColumnID]	Identifier of the column used for grouping operations
[ModifiedTime]	Timestamp of when the group by column was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.GROUPBYCOLUMNS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _GroupByColumns =
        SELECTCOLUMNS(
            INFO.GROUPBYCOLUMNS(),
            "GroupByColumnID", [ID],
            "GroupingColumnID", [GroupingColumnID],
            "RelatedColumnDetailsID", [RelatedColumnDetailsID]
        )
    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ColumnID", [ID],
            "Column Name", [ExplicitName]
        )
    VAR _CombinedTable =
```

```
    NATURALLEFTOUTERJOIN(
        _GroupByColumns,
        ADDCOLUMNS(
            _Columns,
            "GroupingColumnID", [ColumnID]
        )
    )
RETURN
    _CombinedTable
ORDER BY [Column Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.HIERARCHIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each hierarchy in the semantic model. This function provides metadata about hierarchies and their properties.

Syntax

```DAX
INFO.HIERARCHIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the hierarchy
[TableID]	Identifier of the table containing the hierarchy
[Name]	Name of the hierarchy
[Description]	Description of the hierarchy
[IsHidden]	Boolean indicating if the hierarchy is hidden from client tools
[State]	Current state of the hierarchy (e.g., Ready, Processing)
[HierarchyStorageID]	Identifier linking to the hierarchy storage information
[ModifiedTime]	Timestamp of when the hierarchy was last modified
[StructureModifiedTime]	Timestamp of when the hierarchy structure was last modified
[RefreshedTime]	Timestamp of when the hierarchy was last refreshed
[DisplayFolder]	Display folder for organizing hierarchies in client tools
[HideMembers]	Setting for hiding hierarchy members
[LineageTag]	Lineage tag for tracking data lineage
[SourceLineageTag]	Source lineage tag from the original data source

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.HIERARCHIES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Hierarchies =
        SELECTCOLUMNS(
            INFO.HIERARCHIES(),
            "HierarchyID", [ID],
            "Hierarchy Name", [Name],
            "TableID", [TableID]
        )
    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )
    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _Hierarchies,
            _Tables
        )
    RETURN
        _CombinedTable
    ORDER BY [Hierarchy Name]
```

See also

INFO.ATTRIBUTEHIERARCHIES INFO.HIERARCHYSTORAGES
INFO.ATTRIBUTEHIERARCHYSTORAGES INFO.LEVELS INFO.TABLES

Last updated on 01/08/2026

INFO.HIERARCHYSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each hierarchy storage in the semantic model. This function provides metadata about how hierarchies are stored.

Syntax

```DAX
INFO.HIERARCHYSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the hierarchy storage
[HierarchyID]	Identifier linking to the hierarchy
[Name]	Name of the hierarchy storage
[LevelDefinition]	Definition of levels within the hierarchy
[MaterializationType]	Type of materialization used for the hierarchy storage
[StructureType]	Type of structure used for the hierarchy
[SystemTableID]	Identifier of the system table associated with the hierarchy

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.HIERARCHYSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _HierarchyStorages =
        SELECTCOLUMNS(
            INFO.HIERARCHYSTORAGES(),
            "HierarchyStorageID", [ID],
            "HierarchyID", [HierarchyID],
            "Storage Name", [Name]
        )
    VAR _Hierarchies =
```

```
    SELECTCOLUMNS(
        INFO.HIERARCHIES(),
        "HierarchyID", [ID],
        "Hierarchy Name", [Name]
    )
VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _HierarchyStorages,
        _Hierarchies
    )
RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Hierarchy Name", [Hierarchy Name],
        "Storage Name", [Storage Name]
    )
ORDER BY [Hierarchy Name]
```

See also

INFO.HIERARCHIES INFO.ATTRIBUTEHIERARCHIES INFO.ATTRIBUTEHIERARCHYSTORAGES INFO.LEVELS INFO.TABLES

Last updated on 01/08/2026

INFO.KPIS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each KPI in the semantic model. This function provides metadata about Key Performance Indicators defined in the model.

Syntax

```DAX
INFO.KPIS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the KPI
[MeasureID]	Identifier of the measure associated with the KPI
[Description]	Description of the KPI
[TargetDescription]	Description of the KPI target
[TargetExpression]	DAX expression defining the KPI target value
[TargetFormatString]	Format string for displaying the target value
[StatusGraphic]	Graphic used to display the KPI status
[StatusDescription]	Description of the KPI status
[StatusExpression]	DAX expression defining the KPI status
[TrendGraphic]	Graphic used to display the KPI trend
[TrendDescription]	Description of the KPI trend
[TrendExpression]	DAX expression defining the KPI trend
[ModifiedTime]	Timestamp of when the KPI was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.KPIS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _KPIs =
        SELECTCOLUMNS(
            INFO.KPIS(),
            "KPIID", [ID],
            "MeasureID", [MeasureID],
            "KPI Description", [Description],
            "TargetExpression", [TargetExpression]
        )
    VAR _Measures =
        SELECTCOLUMNS(
            INFO.MEASURES(),
            "MeasureID", [ID],
            "Measure Name", [Name]
        )
    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _KPIs,
            _Measures
        )
    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Measure Name", [Measure Name],
            "KPI Description", [KPI Description],
            "Target Expression", [TargetExpression]
        )
    ORDER BY [Measure Name]
```

See also

INFO.CALCULATIONGROUPS INFO.CALCULATIONITEMS INFO.MEASURES INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.LEVELS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each level in the semantic model. This function provides metadata about hierarchy levels and their properties.

Syntax

DAX

```
INFO.LEVELS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the level
[HierarchyID]	Identifier of the hierarchy containing this level
[Ordinal]	Ordinal position of the level within the hierarchy
[Name]	Name of the level
[Description]	Description of the level
[ColumnID]	Identifier of the column associated with this level
[ModifiedTime]	Timestamp of when the level was last modified
[LineageTag]	Lineage tag for tracking data lineage
[SourceLineageTag]	Source lineage tag from the original data source

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.LEVELS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Levels =
        SELECTCOLUMNS(
            INFO.LEVELS(),
            "LevelID", [ID],
```

```
        "HierarchyID", [HierarchyID],
        "Level Name", [Name],
        "Ordinal", [Ordinal]
    )
VAR _Hierarchies =
    SELECTCOLUMNS(
        INFO.HIERARCHIES(),
        "HierarchyID", [ID],
        "Hierarchy Name", [Name]
    )
VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _Levels,
        _Hierarchies
    )
RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Hierarchy Name", [Hierarchy Name],
        "Level Name", [Level Name],
        "Ordinal", [Ordinal]
    )
ORDER BY [Hierarchy Name], [Ordinal]
```

See also

INFO.HIERARCHIES INFO.ATTRIBUTEHIERARCHIES INFO.HIERARCHYSTORAGES INFO.ATTRIBUTEHIERARCHYSTORAGES INFO.TABLES

Last updated on 01/08/2026

INFO.LINGUISTICMETADATA

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each linguistic metadata entry in the semantic model. This function provides metadata about linguistic metadata definitions.

Syntax

```DAX
INFO.LINGUISTICMETADATA ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the linguistic metadata entry
[CultureID]	Identifier of the culture associated with this metadata
[Content]	Content of the linguistic metadata
[ModifiedTime]	Timestamp of when the linguistic metadata was last modified
[ContentType]	Type of content stored in the linguistic metadata

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.LINGUISTICMETADATA()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _LinguisticMetadata =
        SELECTCOLUMNS(
            INFO.LINGUISTICMETADATA(),
            "LinguisticMetadataID", [ID],
            "CultureID", [CultureID],
            "ContentType", [ContentType]
        )
    VAR _Cultures =
        SELECTCOLUMNS(
            INFO.CULTURES(),
            "CultureID", [ID],
            "Culture Name", [Name]
```

```
    )
VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _LinguisticMetadata,
        _Cultures
    )
RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Culture Name", [Culture Name],
        "Content Type", [ContentType]
    )
ORDER BY [Culture Name]
```

See also

INFO.CULTURES INFO.OBJECTTRANSLATIONS INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.MEASURES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each measure in the semantic model, with columns that match the schema rowset for measure objects (for example, name, expression, and state).

Syntax

DAX

```
INFO.MEASURES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the measure
[TableID]	Identifier of the table containing the measure
[Name]	Name of the measure
[Description]	Description of the measure
[DataType]	Data type of the measure result
[Expression]	DAX expression defining the measure calculation
[FormatString]	Format string for displaying measure values
[IsHidden]	Boolean indicating if the measure is hidden from client tools
[State]	Current state of the measure (e.g., Ready, Processing, Error)
[ModifiedTime]	Timestamp of when the measure was last modified
[StructureModifiedTime]	Timestamp of when the measure structure was last modified
[KPIID]	Identifier of the KPI associated with this measure (if applicable)
[IsSimpleMeasure]	Boolean indicating if this is a simple measure
[ErrorMessage]	Error message if the measure is in an error state
[DisplayFolder]	Display folder for organizing measures in client tools
[DetailRowsDefinitionID]	Identifier of the detail rows definition for this measure
[DataCategory]	Data category classification for the measure
[LineageTag]	Lineage tag for tracking data lineage
[SourceLineageTag]	Source lineage tag from the original data source

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.MEASURES()
```

See also

INFO.CALCULATIONGROUPS INFO.CALCULATIONITEMS INFO.KPIS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.MODEL

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about the model in the semantic model. This function provides metadata about the model itself and its properties.

Syntax

```DAX
INFO.MODEL ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data Type	Description
[ID]	Integer	Unique identifier for the model
[Name]	String	Name of the model
[Description]	String	Description of the model
[StorageLocation]	Integer	Storage location indicator
[DefaultMode]	Integer	Default processing mode for the model
[DefaultDataView]	Integer	Default data view setting
[Culture]	String	Culture identifier for the model (e.g., "en-US")
[Collation]	String	Collation setting for the model
[ModifiedTime]	DateTime	Date and time when the model was last modified
[StructureModifiedTime]	DateTime	Date and time when the model structure was last modified
[Version]	Integer	Version number of the model
[DataAccessOptions]	String	JSON object containing data access configuration options
[DefaultMeasureID]	Integer	ID of the default measure, if any
[DefaultPowerBIDataSourceVersion]	Integer	Default Power BI data source version
[ForceUniqueNames]	Boolean	Whether unique names are enforced
[DiscourageImplicitMeasures]	Boolean	Whether implicit measures are discouraged
[DataSourceVariablesOverrideBehavior]	Integer	Behavior for data source variable overrides
[DataSourceDefaultMaxConnections]	Integer	Default maximum connections for data sources
[SourceQueryCulture]	String	Culture used for source queries
[MAttributes]	String	Model attributes
[DiscourageCompositeModels]	Boolean	Whether composite models are discouraged
[AutomaticAggregationOptions]	String	Options for automatic aggregations
[DisableAutoExists]	Integer	Auto exists disable setting
[VersionMarker]	String	Version marker for the model

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.MODEL()
```

See also

INFO.PROPERTIES INFO.EXPRESSIONS INFO.CATALOGS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.OBJECTTRANSLATIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each object translation in the semantic model. This function provides metadata about translations for model objects.

Syntax

DAX

```
INFO.OBJECTTRANSLATIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data Type	Description
[ID]	Integer	Unique identifier for the object translation
[CultureID]	Integer	ID of the culture this translation applies to
[ObjectID]	Integer	ID of the object being translated
[ObjectType]	Integer	Type of object being translated
[Property]	String	Property name that is being translated
[Value]	String	Translated value for the property
[ModifiedTime]	DateTime	Date and time when the translation was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

```
DAX
EVALUATE
    INFO.OBJECTTRANSLATIONS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

```
DAX
EVALUATE
    VAR _ObjectTranslations =
        INFO.OBJECTTRANSLATIONS()

    VAR _Cultures =
        SELECTCOLUMNS(
            INFO.CULTURES(),
            "CultureID", [ID],
            "Culture Name", [Name]
```

```
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _ObjectTranslations,
        _Cultures
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Culture Name", [Culture Name],
        "Object Type", [ObjectType],
        "Property", [Property],
        "Translated Value", [Value],
        "Modified Time", [ModifiedTime]
    )
ORDER BY [Culture Name], [Object Type], [Property]
```

See also

INFO.CULTURES INFO.LINGUISTICMETADATA INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.PARQUETFILESTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each Parquet file storage in the semantic model. This function provides metadata about Parquet file storage characteristics.

Syntax

```
DAX
INFO.PARQUETFILESTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data Type	Description
[ID]	Integer	Unique identifier for the Parquet file storage
[DeltaTableMetadataStorageID]	Integer	ID of the associated delta table metadata storage
[Size]	Integer	Size of the Parquet file in bytes
[Location]	String	Location path of the Parquet file
[CreatedVersion]	Integer	Version when the file was created
[DeletedVersion]	Integer	Version when the file was deleted, if applicable
[RowgroupCount]	Integer	Number of row groups in the Parquet file
[Ordinal]	Integer	Ordinal position of the file
[ETag]	String	Entity tag for file versioning and caching
[DeletionVectorFileLocation]	String	Location of the deletion vector file
[DeletionVectorBitmapOffset]	Integer	Offset for the deletion vector bitmap
[DeletionVectorBitmapSize]	Integer	Size of the deletion vector bitmap
[DeletionVectorFileETag]	String	Entity tag for the deletion vector file

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PARQUETFILESTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _ParquetFileStorages =
        INFO.PARQUETFILESTORAGES()

    VAR _DeltaTableMetadataStorages =
        SELECTCOLUMNS(
            INFO.DELTATABLEMETADATASTORAGES(),
            "DeltaTableMetadataStorageID", [ID],
            "Table Name", [TableName],
            "Root Location", [RootLocation]
        )

    VAR _CombinedTable =
        NATURALLEFTOUTERJOIN(
            _ParquetFileStorages,
            _DeltaTableMetadataStorages
        )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable,
            "Table Name", [Table Name],
            "File Location", [Location],
            "File Size", [Size],
            "Rowgroup Count", [RowgroupCount],
            "Created Version", [CreatedVersion],
            "Deleted Version", [DeletedVersion]
        )
    ORDER BY [Table Name], [Created Version]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.PARTITIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each partition in the semantic model. This function provides metadata about table partitions and their properties.

Syntax

```DAX
INFO.PARTITIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Data Type	Description
[ID]	Integer	Unique identifier for the partition
[TableID]	Integer	ID of the table this partition belongs to
[Name]	String	Name of the partition
[Description]	String	Description of the partition
[DataSourceID]	Integer	ID of the data source for this partition
[QueryDefinition]	String	Query definition used to populate the partition
[State]	Integer	Current state of the partition
[Type]	Integer	Type of partition
[PartitionStorageID]	Integer	ID of the partition storage
[Mode]	Integer	Processing mode for the partition
[DataView]	Integer	Data view setting for the partition
[ModifiedTime]	DateTime	Date and time when the partition was last modified
[RefreshedTime]	DateTime	Date and time when the partition was last refreshed
[SystemFlags]	Integer	System flags for the partition
[ErrorMessage]	String	Error message if the partition is in an error state
[RetainDataTillForceCalculate]	Boolean	Whether to retain data until force calculate
[RangeStart]	String	Start range for incremental refresh, if applicable
[RangeEnd]	String	End range for incremental refresh, if applicable
[RangeGranularity]	Integer	Granularity for range-based partitioning
[RefreshBookmark]	String	Bookmark for incremental refresh operations
[QueryGroupID]	Integer	ID of the query group this partition belongs to
[ExpressionSourceID]	Integer	ID of the expression source
[MAttributes]	String	Additional partition attributes

Remarks

- Typically used in DAX queries to inspect and document model metadata.

- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PARTITIONS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _Partitions =
        INFO.PARTITIONS()

    VAR _Tables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _DataSources =
        SELECTCOLUMNS(
            INFO.DATASOURCES(),
            "DataSourceID", [ID],
            "Data Source Name", [Name]
        )

    VAR _CombinedWithTables =
        NATURALLEFTOUTERJOIN(
            _Partitions,
            _Tables
        )

    VAR _CombinedWithDataSources =
        NATURALLEFTOUTERJOIN(
            _CombinedWithTables,
            _DataSources
        )

    RETURN
```

```
    SELECTCOLUMNS(
        _CombinedWithDataSources,
        "Table Name", [Table Name],
        "Partition Name", [Name],
        "Data Source Name", [Data Source Name],
        "Type", [Type],
        "Refreshed Time", [RefreshedTime]
    )
ORDER BY [Table Name], [Partition Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.RELATIONSHIPS INFO functions overview

---

Last updated on 01/08/2026

INFO.PARTITIONSTORAGES

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✗ Visual calculation ✓ DAX query

Returns a table with information about each partition storage in the semantic model. This function provides metadata about how partitions are stored.

Syntax

```DAX
INFO.PARTITIONSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the partition storage
[PartitionID]	Identifier of the partition
[Name]	Name of the partition storage
[StoragePosition]	Position of the storage within the partition
[SegmentMapStorageID]	Identifier linking to segment map storage
[DataObjectId]	Identifier of the data object
[StorageFolderID]	Identifier of the storage folder
[DeltaTableMetadataStorageID]	Identifier linking to delta table metadata storage

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PARTITIONSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _PartitionStorages =
        SELECTCOLUMNS(
            INFO.PARTITIONSTORAGES(),
            "PartitionStorageID", [ID],
            "PartitionID", [PartitionID],
            "Storage Name", [Name]
```

```
    )
VAR _Partitions =
    SELECTCOLUMNS(
        INFO.PARTITIONS(),
        "PartitionID", [ID],
        "Partition Name", [Name],
        "TableID", [TableID]
    )
VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )
VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _PartitionStorages,
        _Partitions
    )
VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Tables
    )
RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Table Name", [Table Name],
        "Partition Name", [Partition Name],
        "Storage Name", [Storage Name]
    )
ORDER BY [Table Name], [Partition Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.PERSPECTIVECOLUMNS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each perspective column in the semantic model. This function provides metadata about columns included in perspectives.

Syntax

```DAX
INFO.PERSPECTIVECOLUMNS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the perspective column
[PerspectiveTableID]	Identifier of the perspective table containing this column
[ColumnID]	Identifier of the column included in the perspective
[ModifiedTime]	Timestamp of when the perspective column was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PERSPECTIVECOLUMNS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _PerspectiveColumns =
        SELECTCOLUMNS(
            INFO.PERSPECTIVECOLUMNS(),
            "PerspectiveColumnID", [ID],
            "PerspectiveTableID", [PerspectiveTableID],
            "ColumnID", [ColumnID]
        )
    VAR _Columns =
        SELECTCOLUMNS(
            INFO.COLUMNS(),
            "ColumnID", [ID],
            "Column Name", [ExplicitName],
            "TableID", [TableID]
        )
```

```
VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )
VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _PerspectiveColumns,
        _Columns
    )
VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Tables
    )
RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Table Name", [Table Name],
        "Column Name", [Column Name]
    )
ORDER BY [Table Name], [Column Name]
```

See also

INFO.PERSPECTIVES INFO.PERSPECTIVEHIERARCHIES INFO.PERSPECTIVEMEASURES INFO.PERSPECTIVETABLES INFO.TABLES

Last updated on 01/08/2026

INFO.PERSPECTIVEHIERARCHIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each perspective hierarchy in the semantic model. This function provides metadata about hierarchies included in perspectives.

Syntax

```
DAX
INFO.PERSPECTIVEHIERARCHIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for perspective hierarchies in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the perspective hierarchy relationship
PerspectiveTableID	Foreign key to the perspective table containing this hierarchy
HierarchyID	Foreign key to the hierarchy included in the perspective
ModifiedTime	Date and time when the perspective hierarchy was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PERSPECTIVEHIERARCHIES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _PerspectiveHierarchies =
    SELECTCOLUMNS(
        INFO.PERSPECTIVEHIERARCHIES(),
        "PerspectiveTableID", [PerspectiveTableID],
        "HierarchyID", [HierarchyID],
        "Modified", [ModifiedTime]
    )

VAR _PerspectiveTables =
    SELECTCOLUMNS(
        INFO.PERSPECTIVETABLES(),
        "PerspectiveTableID", [ID],
        "PerspectiveID", [PerspectiveID],
        "TableID", [TableID]
```

```
    )

VAR _Hierarchies =
    SELECTCOLUMNS(
        INFO.HIERARCHIES(),
        "HierarchyID", [ID],
        "Hierarchy Name", [Name],
        "Table ID", [TableID]
    )

VAR _Perspectives =
    SELECTCOLUMNS(
        INFO.PERSPECTIVES(),
        "PerspectiveID", [ID],
        "Perspective Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _PerspectiveHierarchies,
        _PerspectiveTables
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Hierarchies
    )

VAR _CombinedTable3 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable2,
        _Perspectives
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable3,
        "Perspective Name", [Perspective Name],
        "Hierarchy Name", [Hierarchy Name],
        "Modified", [Modified]
    )
ORDER BY [Perspective Name], [Hierarchy Name]
```

See also

INFO.PERSPECTIVES INFO.PERSPECTIVECOLUMNS INFO.PERSPECTIVEMEASURES INFO.PERSPECTIVETABLES INFO.TABLES

Last updated on 01/08/2026

INFO.PERSPECTIVEMEASURES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each perspective measure in the semantic model. This function provides metadata about measures included in perspectives.

Syntax

```DAX
INFO.PERSPECTIVEMEASURES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for perspective measures in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the perspective measure relationship
PerspectiveTableID	Foreign key to the perspective table containing this measure
MeasureID	Foreign key to the measure included in the perspective
ModifiedTime	Date and time when the perspective measure was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PERSPECTIVEMEASURES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _PerspectiveMeasures =
    SELECTCOLUMNS(
        INFO.PERSPECTIVEMEASURES(),
        "PerspectiveTableID", [PerspectiveTableID],
        "MeasureID", [MeasureID],
        "Modified", [ModifiedTime]
    )

VAR _PerspectiveTables =
    SELECTCOLUMNS(
        INFO.PERSPECTIVETABLES(),
        "PerspectiveTableID", [ID],
        "PerspectiveID", [PerspectiveID],
        "TableID", [TableID]
```

```
    )

VAR _Measures =
    SELECTCOLUMNS(
        INFO.MEASURES(),
        "MeasureID", [ID],
        "Measure Name", [Name],
        "Table ID", [TableID]
    )

VAR _Perspectives =
    SELECTCOLUMNS(
        INFO.PERSPECTIVES(),
        "PerspectiveID", [ID],
        "Perspective Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _PerspectiveMeasures,
        _PerspectiveTables
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Measures
    )

VAR _CombinedTable3 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable2,
        _Perspectives
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable3,
        "Perspective Name", [Perspective Name],
        "Measure Name", [Measure Name],
        "Modified", [Modified]
    )
ORDER BY [Perspective Name], [Measure Name]
```

See also

INFO.PERSPECTIVES INFO.PERSPECTIVECOLUMNS INFO.PERSPECTIVEHIERARCHIES INFO.PERSPECTIVETABLES INFO.TABLES

Last updated on 01/08/2026

INFO.PERSPECTIVES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each perspective in the semantic model. This function provides metadata about perspectives defined in the model.

Syntax

```DAX
INFO.PERSPECTIVES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for perspectives in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the perspective
ModelID	Foreign key to the model containing this perspective
Name	Name of the perspective
Description	Description of the perspective
ModifiedTime	Date and time when the perspective was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PERSPECTIVES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _Perspectives =
    SELECTCOLUMNS(
        INFO.PERSPECTIVES(),
        "PerspectiveID", [ID],
        "Perspective Name", [Name],
        "Perspective Description", [Description],
        "Modified", [ModifiedTime]
    )

VAR _PerspectiveTables =
    SELECTCOLUMNS(
        INFO.PERSPECTIVETABLES(),
```

```
        "PerspectiveID", [PerspectiveID],
        "TableID", [TableID],
        "IncludeAll", [IncludeAll]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _Perspectives,
        _PerspectiveTables
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Perspective Name", [Perspective Name],
        "Perspective Description", [Perspective Description],
        "Table Name", [Table Name],
        "Include All", [IncludeAll],
        "Modified", [Modified]
    )
ORDER BY [Perspective Name], [Table Name]
```

See also

INFO.PERSPECTIVECOLUMNS INFO.PERSPECTIVEHIERARCHIES INFO.PERSPECTIVEMEASURES INFO.PERSPECTIVETABLES INFO.TABLES

Last updated on 01/08/2026

INFO.PERSPECTIVETABLES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each perspective table in the semantic model. This function provides metadata about tables included in perspectives.

Syntax

```DAX
INFO.PERSPECTIVETABLES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for perspective tables in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the perspective table relationship
PerspectiveID	Foreign key to the perspective containing this table
TableID	Foreign key to the table included in the perspective
IncludeAll	Boolean indicating whether all objects from the table are included in the perspective
ModifiedTime	Date and time when the perspective table was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PERSPECTIVETABLES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _PerspectiveTables =
    SELECTCOLUMNS(
        INFO.PERSPECTIVETABLES(),
        "PerspectiveID", [PerspectiveID],
        "TableID", [TableID],
        "Include All", [IncludeAll],
        "Modified", [ModifiedTime]
    )

VAR _Perspectives =
    SELECTCOLUMNS(
        INFO.PERSPECTIVES(),
```

```
        "PerspectiveID", [ID],
        "Perspective Name", [Name],
        "Perspective Description", [Description]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name],
        "Table Description", [Description]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _PerspectiveTables,
        _Perspectives
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Perspective Name", [Perspective Name],
        "Table Name", [Table Name],
        "Include All", [Include All],
        "Modified", [Modified]
    )
ORDER BY [Perspective Name], [Table Name]
```

See also

INFO.PERSPECTIVES INFO.PERSPECTIVECOLUMNS INFO.PERSPECTIVEHIERARCHIES INFO.PERSPECTIVEMEASURES INFO.TABLES

Last updated on 01/08/2026

INFO.PROPERTIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each property in the semantic model. This function provides metadata about properties defined for model objects.

Syntax

```DAX
INFO.PROPERTIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for properties in the current semantic model.

Expand table

Column	Description
PropertyName	Name of the property
PropertyDescription	Description explaining the purpose and usage of the property
PropertyType	Data type of the property value (e.g., String, Integer, Boolean)
PropertyAccessType	Access level of the property (e.g., Read, Write, ReadWrite)
IsRequired	Boolean indicating whether the property is required
Value	Current value of the property

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.PROPERTIES()
```

See also

INFO.MODEL INFO.EXPRESSIONS INFO.CATALOGS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.QUERYGROUPS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each query group in the semantic model. This function provides metadata about query groups defined in the model.

Syntax

```DAX
INFO.QUERYGROUPS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for query groups in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the query group
ModelID	Foreign key to the model containing this query group
Folder	Folder path or location for organizing the query group
Description	Description of the query group

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.QUERYGROUPS()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.REFRESHPOLICIES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each refresh policy in the semantic model. This function provides metadata about refresh policies defined for tables.

Syntax

```
DAX
INFO.REFRESHPOLICIES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for refresh policies in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the refresh policy
TableID	Foreign key to the table this refresh policy applies to
PolicyType	Type of refresh policy (e.g., Incremental, Rolling Window)
RollingWindowGranularity	Granularity for rolling window refresh (e.g., Day, Month, Year)
RollingWindowPeriods	Number of periods to keep in rolling window
IncrementalGranularity	Granularity for incremental refresh (e.g., Day, Month, Year)
IncrementalPeriods	Number of periods for incremental refresh
IncrementalPeriodsOffset	Offset periods for incremental refresh timing
PollingExpression	DAX expression used for polling to detect changes
SourceExpression	DAX expression defining the source data for refresh
Mode	Refresh mode configuration

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.REFRESHPOLICIES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _RefreshPolicies =
    SELECTCOLUMNS(
        INFO.REFRESHPOLICIES(),
        "TableID", [TableID],
        "Policy Type", [PolicyType],
        "Rolling Window Granularity", [RollingWindowGranularity],
        "Rolling Window Periods", [RollingWindowPeriods],
        "Incremental Granularity", [IncrementalGranularity],
        "Incremental Periods", [IncrementalPeriods]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name],
        "Description", [Description]
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _RefreshPolicies,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Table Name", [Table Name],
        "Description", [Description],
        "Policy Type", [Policy Type],
        "Rolling Window Granularity", [Rolling Window Granularity],
        "Rolling Window Periods", [Rolling Window Periods],
        "Incremental Granularity", [Incremental Granularity],
        "Incremental Periods", [Incremental Periods]
    )
ORDER BY [Table Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

Last updated on 01/08/2026

INFO.RELATEDCOLUMNDETAILS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each related column detail in the semantic model. This function provides metadata about related column details for relationships.

Syntax

```DAX
INFO.RELATEDCOLUMNDETAILS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for related column details in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the related column detail
ColumnID	Foreign key to the column this detail relates to
ModifiedTime	Date and time when the related column detail was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.RELATEDCOLUMNDETAILS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _RelatedColumnDetails =
    SELECTCOLUMNS(
        INFO.RELATEDCOLUMNDETAILS(),
        "ColumnID", [ColumnID],
        "Modified", [ModifiedTime]
    )

VAR _Columns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "TableID", [TableID],
        "Column Name", [Name],
        "Data Type", [DataType]
    )
```

```
VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _RelatedColumnDetails,
        _Columns
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Table Name", [Table Name],
        "Column Name", [Column Name],
        "Data Type", [Data Type],
        "Modified", [Modified]
    )
ORDER BY [Table Name], [Column Name]
```

See also

INFO.DEPENDENCIES INFO.CALCDEPENDENCY INFO.TABLES INFO.COLUMNS INFO.MEASURES

Last updated on 01/08/2026

INFO.RELATIONSHIPINDEXSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each relationship index storage in the semantic model. This function provides metadata about relationship index storage characteristics.

Syntax

```DAX
INFO.RELATIONSHIPINDEXSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for relationship index storages in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the relationship index storage
RelationshipStorageID	Foreign key to the relationship storage using this index
IndexType	Type of index used for the relationship
Flags	Index configuration flags and options
RecordCount	Number of records in the relationship index
SecondaryRecordCount	Number of secondary records in the relationship index
StorageFolderID	Foreign key to the storage folder containing the index
StorageFileID	Foreign key to the storage file containing the index
SystemTableID	Foreign key to the primary system table
SecondarySystemTableID	Foreign key to the secondary system table

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.RELATIONSHIPINDEXSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _RelationshipIndexStorages =
    SELECTCOLUMNS(
```

```
        INFO.RELATIONSHIPINDEXSTORAGES(),
        "RelationshipStorageID", [RelationshipStorageID],
        "Index Type", [IndexType],
        "Record Count", [RecordCount],
        "Secondary Record Count", [SecondaryRecordCount],
        "Flags", [Flags]
    )

VAR _RelationshipStorages =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPSTORAGES(),
        "RelationshipStorageID", [ID],
        "RelationshipID", [RelationshipID],
        "Storage Name", [Name]
    )

VAR _Relationships =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPS(),
        "RelationshipID", [ID],
        "Relationship Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _RelationshipIndexStorages,
        _RelationshipStorages
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Relationships
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Relationship Name", [Relationship Name],
        "Storage Name", [Storage Name],
        "Index Type", [Index Type],
        "Record Count", [Record Count],
        "Secondary Record Count", [Secondary Record Count]
    )
ORDER BY [Relationship Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

Last updated on 01/08/2026

INFO.RELATIONSHIPS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each relationship in the semantic model. This function provides metadata about relationships between tables.

Syntax

```DAX
INFO.RELATIONSHIPS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for relationships in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the relationship
ModelID	Foreign key to the model containing this relationship
Name	Name of the relationship
IsActive	Boolean indicating whether the relationship is active
Type	Type of relationship (e.g., OneToMany, ManyToOne)
CrossFilteringBehavior	Cross filtering behavior (e.g., OneDirection, BothDirections)
JoinOnDateBehavior	Behavior for date-based joins
RelyOnReferentialIntegrity	Boolean indicating whether to rely on referential integrity
FromTableID	Foreign key to the source table in the relationship
FromColumnID	Foreign key to the source column in the relationship
FromCardinality	Cardinality on the "from" side of the relationship
ToTableID	Foreign key to the target table in the relationship
ToColumnID	Foreign key to the target column in the relationship
ToCardinality	Cardinality on the "to" side of the relationship
State	Current state of the relationship
RelationshipStorageID	Foreign key to the relationship storage information
RelationshipStorage2ID	Foreign key to secondary relationship storage information
ModifiedTime	Date and time when the relationship was last modified
RefreshedTime	Date and time when the relationship was last refreshed
SecurityFilteringBehavior	Security filtering behavior for the relationship

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.RELATIONSHIPS()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _Relationships =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPS(),
        "Relationship Name", [Name],
        "Is Active", [IsActive],
        "Type", [Type],
        "Cross Filtering", [CrossFilteringBehavior],
        "From Table ID", [FromTableID],
        "From Column ID", [FromColumnID],
        "To Table ID", [ToTableID],
        "To Column ID", [ToColumnID]
    )

VAR _FromTables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "FromTableID", [ID],
        "From Table Name", [Name]
    )

VAR _ToTables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "ToTableID", [ID],
        "To Table Name", [Name]
    )

VAR _FromColumns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "FromColumnID", [ID],
        "From Column Name", [Name]
    )

VAR _ToColumns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
```

```
        "ToColumnID", [ID],
        "To Column Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _Relationships,
        _FromTables
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _ToTables
    )

VAR _CombinedTable3 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable2,
        _FromColumns
    )

VAR _CombinedTable4 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable3,
        _ToColumns
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable4,
        "Relationship Name", [Relationship Name],
        "From Table", [From Table Name],
        "From Column", [From Column Name],
        "To Table", [To Table Name],
        "To Column", [To Column Name],
        "Is Active", [Is Active],
        "Type", [Type],
        "Cross Filtering", [Cross Filtering]
    )
ORDER BY [Relationship Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.PARTITIONS INFO functions overview

---

Last updated on 01/08/2026

INFO.RELATIONSHIPSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each relationship storage in the semantic model. This function provides metadata about how relationships are stored.

Syntax

```DAX
INFO.RELATIONSHIPSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for relationship storages in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the relationship storage
RelationshipID	Foreign key to the relationship using this storage
Name	Name of the relationship storage
DefinitionType	Type definition for the relationship storage
Cardinality	Cardinality characteristics of the relationship storage
Flags	Storage flags and configuration options
RelationshipIndexStorageID	Foreign key to the relationship index storage

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.RELATIONSHIPSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _RelationshipStorages =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPSTORAGES(),
        "RelationshipID", [RelationshipID],
        "Storage Name", [Name],
        "Definition Type", [DefinitionType],
        "Cardinality", [Cardinality],
        "Flags", [Flags]
```

```
    )

VAR _Relationships =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPS(),
        "RelationshipID", [ID],
        "Relationship Name", [Name],
        "Is Active", [IsActive],
        "Type", [Type]
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _RelationshipStorages,
        _Relationships
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Relationship Name", [Relationship Name],
        "Storage Name", [Storage Name],
        "Is Active", [Is Active],
        "Type", [Type],
        "Definition Type", [Definition Type],
        "Cardinality", [Cardinality]
    )
ORDER BY [Relationship Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.ROLEMEMBERSHIPS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each role membership in the semantic model. This function provides metadata about role memberships and security settings.

Syntax

```DAX
INFO.ROLEMEMBERSHIPS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for role memberships in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the role membership
RoleID	Foreign key to the role containing this membership
MemberName	Name of the member (user or group)
MemberID	Unique identifier for the member
IdentityProvider	Identity provider for the member authentication
MemberType	Type of member (e.g., User, Group)
ModifiedTime	Date and time when the role membership was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ROLEMEMBERSHIPS()
```

See also

INFO.ROLES INFO.COLUMNPERMISSIONS INFO.TABLEPERMISSIONS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.ROLES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each role in the semantic model. This function provides metadata about security roles defined in the model.

Syntax

```DAX
INFO.ROLES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for roles in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the role
ModelID	Foreign key to the model containing this role
Name	Name of the role
Description	Description of the role
ModelPermission	Permission level for the role (e.g., Read, ReadRefresh, Administrator)
ModifiedTime	Date and time when the role was last modified

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.ROLES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _Roles =
    SELECTCOLUMNS(
        INFO.ROLES(),
        "RoleID", [ID],
        "Role Name", [Name],
        "Role Description", [Description],
        "Model Permission", [ModelPermission],
        "Modified", [ModifiedTime]
    )
```

```
VAR _RoleMemberships =
    SELECTCOLUMNS(
        INFO.ROLEMEMBERSHIPS(),
        "RoleID", [RoleID],
        "Member Name", [MemberName],
        "Member Type", [MemberType]
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _Roles,
        _RoleMemberships
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Role Name", [Role Name],
        "Role Description", [Role Description],
        "Model Permission", [Model Permission],
        "Member Name", [Member Name],
        "Member Type", [Member Type],
        "Modified", [Modified]
    )
ORDER BY [Role Name], [Member Name]
```

See also

INFO.ROLEMEMBERSHIPS INFO.COLUMNPERMISSIONS INFO.TABLEPERMISSIONS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.SEGMENTMAPSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each segment map storage in the semantic model. This function provides metadata about segment map storage characteristics.

Syntax

```
DAX
INFO.SEGMENTMAPSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for segment map storages in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the segment map storage
PartitionStorageID	Foreign key to the partition storage using this segment map
Type	Type of segment map storage
RecordCount	Number of records in the segment map
SegmentCount	Number of segments in the segment map
RecordsPerSegment	Average number of records per segment

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.SEGMENTMAPSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _SegmentMapStorages =
    SELECTCOLUMNS(
        INFO.SEGMENTMAPSTORAGES(),
        "PartitionStorageID", [PartitionStorageID],
        "Type", [Type],
        "Record Count", [RecordCount],
        "Segment Count", [SegmentCount],
        "Records Per Segment", [RecordsPerSegment]
    )
```

```
VAR _PartitionStorages =
    SELECTCOLUMNS(
        INFO.PARTITIONSTORAGES(),
        "PartitionStorageID", [ID],
        "PartitionID", [PartitionID],
        "Storage Name", [Name]
    )

VAR _Partitions =
    SELECTCOLUMNS(
        INFO.PARTITIONS(),
        "PartitionID", [ID],
        "TableID", [TableID],
        "Partition Name", [Name]
    )

VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _SegmentMapStorages,
        _PartitionStorages
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _Partitions
    )

VAR _CombinedTable3 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable2,
        _Tables
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable3,
        "Table Name", [Table Name],
        "Partition Name", [Partition Name],
        "Storage Type", [Type],
        "Record Count", [Record Count],
        "Segment Count", [Segment Count],
        "Records Per Segment", [Records Per Segment]
    )
ORDER BY [Table Name], [Partition Name]
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.SEGMENTSTORAGES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each segment storage in the semantic model. This function provides metadata about segment storage characteristics.

Syntax

```DAX
INFO.SEGMENTSTORAGES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for segment storages in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the segment storage
ColumnPartitionStorageID	Foreign key to the column partition storage using this segment
SegmentCount	Number of segments in the storage
StorageFileID	Foreign key to the storage file containing the segment data

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.SEGMENTSTORAGES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _SegmentStorages =
    SELECTCOLUMNS(
        INFO.SEGMENTSTORAGES(),
        "ColumnPartitionStorageID", [ColumnPartitionStorageID],
        "Segment Count", [SegmentCount],
        "StorageFileID", [StorageFileID]
    )

VAR _ColumnPartitionStorages =
    SELECTCOLUMNS(
        INFO.COLUMNPARTITIONSTORAGES(),
        "ColumnPartitionStorageID", [ID],
        "ColumnStorageID", [ColumnStorageID],
        "PartitionStorageID", [PartitionStorageID]
```

```
    )

VAR _StorageFiles =
    SELECTCOLUMNS(
        INFO.STORAGEFILES(),
        "StorageFileID", [ID],
        "File Name", [FileName]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _SegmentStorages,
        _ColumnPartitionStorages
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _StorageFiles
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Column Storage ID", [ColumnStorageID],
        "Partition Storage ID", [PartitionStorageID],
        "Segment Count", [Segment Count],
        "File Name", [File Name]
    )
ORDER BY [Column Storage ID]
```

See also

INFO.COLUMNSTORAGES INFO.COLUMNPARTITIONSTORAGES INFO.DICTIONARYSTORAGES INFO.STORAGEFILES INFO.TABLESTORAGES

Last updated on 01/08/2026

INFO.STORAGEFILES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each storage file in the semantic model. This function provides metadata about storage files and their characteristics.

Syntax

```DAX
INFO.STORAGEFILES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for storage files in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the storage file
OwnerID	Identifier of the object that owns this storage file
OwnerType	Type of the object that owns this storage file
StorageFolderID	Foreign key to the storage folder containing this file
FileName	Name of the storage file

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.STORAGEFILES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _StorageFiles =
    SELECTCOLUMNS(
        INFO.STORAGEFILES(),
        "OwnerID", [OwnerID],
        "Owner Type", [OwnerType],
        "StorageFolderID", [StorageFolderID],
        "File Name", [FileName]
    )

VAR _StorageFolders =
    SELECTCOLUMNS(
        INFO.STORAGEFOLDERS(),
```

```
        "StorageFolderID", [ID],
        "Folder Name", [Name]
    )

VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _StorageFiles,
        _StorageFolders
    )

RETURN
    SELECTCOLUMNS(
        _CombinedTable,
        "Folder Name", [Folder Name],
        "File Name", [File Name],
        "Owner Type", [Owner Type],
        "Owner ID", [OwnerID]
    )
ORDER BY [Folder Name], [File Name]
```

See also

INFO.COLUMNSTORAGES INFO.COLUMNPARTITIONSTORAGES INFO.DICTIONARYSTORAGES INFO.SEGMENTSTORAGES INFO.TABLESTORAGES

Last updated on 01/08/2026

INFO.STORAGEFOLDERS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.STORAGEFOLDERS([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	The name of the database
[OwnerID]	
[OwnerType]	
[Path]	

Remarks

Can only be run by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.STORAGEFOLDERS()
```

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/11/2026

INFO.STORAGETABLECOLUMNS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.STORAGETABLECOLUMNS([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[DATABASE_NAME]	The name of the database
[CUBE_NAME]	The name of the cube
[MEASURE_GROUP_NAME]	The name of the measure group
[DIMENSION_NAME]	The name of the dimension
[ATTRIBUTE_NAME]	The name of the attribute
[TABLE_ID]	The ID of the table
[COLUMN_ID]	The ID of the column
[COLUMN_TYPE]	The type of the column. The values are as follows: BASIC_DATA – This column contains data provided by an external data source. CALCULATED_DATA – This column contains data created by calculations. RELATIONSHIP – This column contains information about relationship between tables. HIERARCHY_POSITION_TO_DATAID –This column contains information mapping position of a value in the hierarchy to the Data ID. HIERARCHY_DATAID_TO_POSITION –This column contains information mapping a Data ID to the position of a valuein the hierarchy. UNKNOWN – The column type is not known.
[COLUMN_ENCODING]	The encoding method used on the column. The method can be one of the following: 0 – The system automatically selects a column encoding. 1 – The column uses hash encoding. 2 – The column uses value encoding.
[DATATYPE]	The type of the column data. The values are as follows: N/A – Indicates that no data type information is available. DBTYPE_EMPTY - Indicates that no value was specified. DBTYPE_I2 - Indicates a two-byte signed integer. DBTYPE_I4 - Indicates a four-byte signed integer. DBTYPE_R4 - Indicates a single-precision floating-point value. DBTYPE_R8 - Indicates a double-precision floating-point value. DBTYPE_CY - Indicates a currency value. Currency is a fixed-point number that has four digits to the right of the decimal point and that is stored in an eight-byte signed integer scaled by 10,000. DBTYPE_DATE - Indicates a date value. Date values are stored as Double, the whole part of which is th enumber of days since December 30, 1899, and the fractional part of which is the fraction of a day. DBTYPE_BSTR - Indicates anull-terminated character string (Unicode). DBTYPE_ERROR - Indicates a 32-bit error code. DBTYPE_BOOL - Indicates a Boolean value.

Column	Description
	DBTYPE_DECIMAL - Indicates an exact numeric value with a fixed precision and scale. The scale is between 0 and 28. DBTYPE_I1 - Indicates a one-byte signed integer. DBTYPE_UI1 - Indicates a one-byte unsigned integer. DBTYPE_UI2 - Indicates a two-byte unsigned integer. DBTYPE_UI4 - Indicates a four-byte unsigned integer. DBTYPE_I8 - Indicates an eight-byte signed integer. DBTYPE_UI8 - Indicates an eight-byte unsigned integer. DBTYPE_GUID - Indicates a GUID. DBTYPE_BYTES - Indicates a binary value. DBTYPE_STR - Indicates a string value. DBTYPE_WSTR - Indicates a null-terminated Unicode character string. DBTYPE_NUMERIC - Indicates an exact numeric value with a fixed precision and scale. The scale is between 0and 38. DBTYPE_DBDATE - Indicates a date value (yyyymmdd). DBTYPE_DBTIME - Indicates a time value (hhmmss). DBTYPE_DBTIMESTAMP - Indicates a date-time stamp (yyyymmddhhmmss plus a fraction in billionths)
[ISKEY]	Indicates whether the column is a key column
[ISUNIQUE]	Indicates whether the column contains unique values
[ISNULLABLE]	Indicates whether the column can contain NULL values
[ISROWNUMBER]	Indicates whether the column is a Row Number column
[DICTIONARY_SIZE]	Indicates the amount of memory that is used by the dictionary data structure associated with the column, in bytes. The dictionary data structure maps column data IDs to the actual values
[DICTIONARY_ISPAGEABLE]	Indicates the Vertipaq Data Paging (VDP) state of the dictionary, which specifies whether the dictionary is pageable. If the VDPstate of the dictionary is unknown, the default value is NULL. Otherwise, the value is a Boolean
[DICTIONARY_ISRESIDENT]	Indicates the VDP state of the dictionary, which specifies whether the dictionary is resident. If the VDP state of the dictionary is unknown, the default value is NULL. Otherwise, the value is a Boolean
[DICTIONARY_TEMPERATURE]	Indicates the scaled numeric value of the frequency of dictionary access. The value is based on the most recent access time and usage. This column has a numeric value only if the dictionary is pageable and has been loaded at least once; otherwise, the value is NULL
[DICTIONARY_LAST_ACCESSED]	Indicates the most recent time the dictionary was accessed. For non-pageable tables in the dictionary, the DICTIONARY_LAST_ACCESSED value is NULL. For pageable tables in the dictionary,

Column	Description
	DICTIONARY_LAST_ACCESSED value NULL denotes the table has not been loaded since server startup

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.STORAGETABLECOLUMNS()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.STORAGETABLECOLUMNSEGMENTS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.STORAGETABLECOLUMNSEGMENTS([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[DATABASE_NAME]	The name of the database
[CUBE_NAME]	The name of the cube
[MEASURE_GROUP_NAME]	The name of the measure group
[DIMENSION_NAME]	The name of the dimension
[TABLE_ID]	The ID of the table
[COLUMN_ID]	The ID of the column
[SEGMENT_NUMBER]	The numeric value of the segment
[TABLE_PARTITION_NUMBER]	The numeric value of the partition table
[RECORDS_COUNT]	The number of records
[ALLOCATED_SIZE]	The size of allocated data
[USED_SIZE]	The size of the data used
[COMPRESSION_TYPE]	The type of compression. This value is always "NOSPLIT" and for internal use only
[BITS_COUNT]	The count of bits required to store the Data IDs
[BOOKMARK_BITS_COUNT]	The bookmark count of BITS
[VERTIPAQ_STATE]	The state of the VertiPaq compression for this columnsegment. The value is one of the following: **COMPLETED** – The VertiPaqcompression completed successfully **TIMEBOXED** – The VertiPaqcompression was timeboxed **SKIPPED** – The VertiPaqcompression was skipped
[ISPAGEABLE]	When true, may indicate that the segment is pageable; otherwise, false. If the pagingfeature is not supported on the server, the value is NULL
[ISRESIDENT]	When true, may indicate that the segment is resident; otherwise, false. If the pagingfeature is not supported on the server, the value is NULL
[TEMPERATURE]	When the segment is pageable and is resident, may indicate the scaled numericvalue of the frequency of segment access considering the last access time andusage; otherwise, NULL
[LAST_ACCESSED]	For a pageable segment, may indicate the last accesstime of a segment if it has been paged in at least once; otherwise, NULL. Fora non-pageable segment, the value is always NULL

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.STORAGETABLECOLUMNSEGMENTS()
```

See also

INFO.TABLES INFO.COLUMNS INFO.MEASURES INFO functions overview

---

Last updated on 01/08/2026

INFO.STORAGETABLES

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```
DAX
INFO.STORAGETABLES([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[Database_Name]	The name of the database
[Cube_Name]	The name of the cube
[Measure_Group_Name]	The name of the measure group
[Dimension_Name]	The name of the dimension
[Table_ID]	The ID of the table
[Table_Partitions_Count]	The table partition count
[Hint_Table_Type]	The hint of the table type
[Rows_Count]	The row count
[RIViolation_Count]	This is a deprecated column set to 0.

Remarks

Can only be run by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.STORAGETABLES()
```

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/11/2026

INFO.TABLEPERMISSIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.TABLEPERMISSIONS([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[RoleID]	The ID of the role that provides this permission.
[TableID]	The ID of the table on which this permission applies.
[FilterExpression]	The filter expression in the permission.
[ModifiedTime]	When the permission was last modified.
[State]	The numerical repesentation of the state of the permission.
[ErrorMessage]	Any error message with the permission.
[MetadataPermission]	A numerical representation of the permission provided. 0 means no access control is enforced, while 1 means access is restricted and 2 means access is unrestricted

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.TABLEPERMISSIONS()
```

This DAX query returns a table with all of the columns of this DAX function.

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _INFO = INFO.TABLEPERMISSIONS()

    VAR MetadataPermissionEnum =
    DATATABLE(
    "MetadataPermission",INTEGER,
    "MetadataDescription",STRING,
    {
    {0,"Default behavior - no access control is enforced"},
    {1,"Access to the data is restricted"},
    {2,"Access to the data is unrestricted"}
    }
    )
    VAR StateEnum =
    DATATABLE(
        "State",INTEGER,
        "StateName",STRING,
        {
            {1,"Ready"},
            {3,"NoData"},
            {4,"CalculationNeeded"},
            {5,"SemanticError"},
            {6,"EvaluationError"},
            {7,"DependencyError"},
            {8,"Incomplete"},
            {10,"ForceCalculationNeeded"}
        }
        )
    VAR _CombinedTable =
    NATURALLEFTOUTERJOIN(
        _INFO,
        MetadataPermissionEnum
    )

    VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable,
        StateEnum
    )

    VAR _ModelRoles =
        SELECTCOLUMNS(
            INFO.ROLES(),
            "RoleID", [ID],
            "Role Name", [Name]
        )
    VAR _ModelTables =
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )

    VAR _CombinedTable3 =
```

```
    NATURALLEFTOUTERJOIN(
        _CombinedTable2,
        _ModelRoles
    )
    VAR _CombinedTable4 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable3,
        _ModelTables
    )

    RETURN

        SELECTCOLUMNS(
            _CombinedTable4,
            "Table Permission ID", [ID],
            "Filter Expression", [FilterExpression],
            "Modified Time", [ModifiedTime],
            "State Name", [StateName],
            "Error Message", [ErrorMessage],
            "Metadata permission description", [MetadataDescription],
            "Role Name", [Role Name],
            "Table Name", [Table Name]
            )

    ORDER BY [Table Permission ID]
```

This DAX query returns a table with only the specified columns and joining to other INFO DAX functions and the table permissions table.

Enumerations

Columns returned in this INFO function give the ID of an enumeration, or lookup table. Here are the values for each enumeration in this DAX function.

ObjectState

Expand table

Value	Name	Description
1	Ready	Object is refreshed, contains up-to-date data, and is queryable.
3	NoData	Object is queryable but contains no data. Refresh it to bring in data. Applies to non-calculated objects, such as DataColumns, partitions, and Tables.
4	CalculationNeeded	Object is not queryable and contains no data. It needs to be refreshed to become functional. Applies only to calculated objects, such as calculated columns, hierarchies, and calculated tables.

Value	Name	Description
5	SemanticError	Object is in an error state because of an invalid expression. It is not queryable.
6	EvaluationError	Object is in an error state because an error occurred during expression evaluation. It is not queryable.
7	DependencyError	Object is in an error state because some of its calculation dependencies are in an error state. It is not queryable.
8	Incomplete	Some parts of the object have no data. Refresh the object to add the rest of the data. The object is queryable. Applies to non-calculated objects, such as DataColumns, partitions, and tables.
10	ForceCalculationNeeded	The data is possibly outdated, but is in a queryable state. Applies only for CalculatedTable.

This table is based on the official documentation.

To join with INFO functions use this DAX query.

DAX

```
EVALUATE
    DATATABLE(
    "State",INTEGER,
    "StateName",STRING,
    {
        {1,"Ready"},
        {3,"NoData"},
        {4,"CalculationNeeded"},
        {5,"SemanticError"},
        {6,"EvaluationError"},
        {7,"DependencyError"},
        {8,"Incomplete"},
        {10,"ForceCalculationNeeded"}
    }
    )
```

See also

INFO.ROLES INFO.ROLEMEMBERSHIPS INFO.COLUMNPERMISSIONS INFO.TABLES INFO.COLUMNS

Last updated on 01/08/2026

INFO.TABLES

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each table in the semantic model, with columns that match the schema rowset for table objects (for example, name, description, and visibility).

Syntax

DAX

```
INFO.TABLES ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table whose columns match the schema rowset for table objects in the current semantic model.

Expand table

Column	Description
ID	Unique identifier for the table
ModelID	Foreign key to the model containing this table
Name	Name of the table
DataCategory	Data category classification for the table
Description	Description of the table
IsHidden	Boolean indicating whether the table is hidden from client applications
TableStorageID	Foreign key to the table storage information
ModifiedTime	Date and time when the table was last modified
StructureModifiedTime	Date and time when the table structure was last modified
SystemFlags	System flags for internal table management
ShowAsVariationsOnly	Boolean indicating whether the table should only show variations
IsPrivate	Boolean indicating whether the table is private
DefaultDetailRowsDefinitionID	Foreign key to the default detail rows definition
AlternateSourcePrecedence	Precedence order for alternate data sources
RefreshPolicyID	Foreign key to the refresh policy for incremental refresh
CalculationGroupID	Foreign key to the calculation group if this table is a calculation group
ExcludeFromModelRefresh	Boolean indicating whether to exclude this table from model refresh
LineageTag	Lineage tag for tracking table lineage
SourceLineageTag	Source lineage tag from the original data source
SystemManaged	Boolean indicating whether the table is system-managed

Remarks

- Useful for documentation and governance scenarios.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.TABLES()
```

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
VAR _Tables =
    SELECTCOLUMNS(
        INFO.TABLES(),
        "TableID", [ID],
        "Table Name", [Name],
        "Description", [Description],
        "Is Hidden", [IsHidden],
        "Calculation Group ID", [CalculationGroupID],
        "Refresh Policy ID", [RefreshPolicyID]
    )

VAR _CalculationGroups =
    SELECTCOLUMNS(
        INFO.CALCULATIONGROUPS(),
        "CalculationGroupID", [ID],
        "Calculation Group Name", [Name]
    )

VAR _RefreshPolicies =
    SELECTCOLUMNS(
        INFO.REFRESHPOLICIES(),
        "RefreshPolicyID", [ID],
        "Policy Type", [PolicyType],
        "Incremental Periods", [IncrementalPeriods]
    )

VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _Tables,
        _CalculationGroups
    )

VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _RefreshPolicies
    )
```

```
RETURN
    SELECTCOLUMNS(
        _CombinedTable2,
        "Table Name", [Table Name],
        "Description", [Description],
        "Is Hidden", [Is Hidden],
        "Calculation Group", [Calculation Group Name],
        "Policy Type", [Policy Type],
        "Incremental Periods", [Incremental Periods]
    )
ORDER BY [Table Name]
```

See also

INFO.COLUMNS INFO.PARTITIONS INFO.RELATIONSHIPS INFO functions overview

---

Last updated on 01/08/2026

INFO.TABLESTORAGES

Applies to: ✗ Calculated column ✗ Calculated table ✗ Measure ✗ Visual calculation ✓ DAX query

Returns a table with information about all table storage in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.TABLESTORAGES([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[TableID]	The ID of the table.
[Name]	The name of each table in this semantic model as a string followed by the table ID in paranthesis.
[Version]	
[Settings]	This column is for internal use only.
[RIViolationCount]	This is a deprecated column set to 0.
[StorageFolderID]	The ID of the storage folder that contains the table storage.

Remarks

Can only be ran by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.TABLESTORAGES()
```

This DAX query returns a table with all of the columns of this DAX function.

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _INFO =
    INFO.TABLESTORAGES()

    VAR _ModelTables =
```

```
        SELECTCOLUMNS(
            INFO.TABLES(),
            "TableID", [ID],
            "Table Name", [Name]
        )
    VAR _ModelStorageFolders =
        SELECTCOLUMNS(
            INFO.STORAGEFOLDERS(),
            "StorageFolderID", [ID],
            "Storage Folder Path", [Path]
        )

    VAR _CombinedTable1 =
    NATURALLEFTOUTERJOIN(
        _INFO,
        _ModelTables
    )
    VAR _CombinedTable2 =
    NATURALLEFTOUTERJOIN(
        _CombinedTable1,
        _ModelStorageFolders
    )

    RETURN
        SELECTCOLUMNS(
            _CombinedTable2,
            "Table Storage ID", [ID],
            "Table Storage Name", [Name],
            "Table Storage Version", [Version],
            "Table Name", [Table Name],
            "Storage folder Path", [Storage Folder Path]
            )

    ORDER BY [Table Storage Name]
```

This DAX query returns a table with only the specified columns and joining to other INFO DAX functions and the table storages table.

See also

INFO.COLUMNSTORAGES INFO.COLUMNPARTITIONSTORAGES INFO.DICTIONARYSTORAGES INFO.SEGMENTSTORAGES INFO.STORAGEFILES

Last updated on 01/08/2026

INFO.USERDEFINEDFUNCTIONS

Applies to: ⊗ Calculated column ⊗ Calculated table ⊗ Measure ⊗ Visual calculation ✓ DAX query

Returns a table with information about each user-defined function in the semantic model, with columns that match the schema rowset for user-defined function objects (for example, name, expression, and state).

Syntax

```DAX
INFO.USERDEFINEDFUNCTIONS ( [<Restriction name>, <Restriction value>], ... )
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	Unique identifier for the user-defined function
[ModelID]	Identifier of the model containing the user-defined function
[Name]	Name of the user-defined function
[Description]	Description of the user-defined function
[Expression]	DAX expression defining the user-defined function
[IsHidden]	Boolean indicating if the user-defined function is hidden from client tools
[State]	Current state of the user-defined function (e.g., Ready, Error)
[ErrorMessage]	Error message if the user-defined function is in an error state
[ModifiedTime]	Timestamp of when the user-defined function was last modified
[StructureModifiedTime]	Timestamp of when the user-defined function structure was last modified
[LineageTag]	Lineage tag for tracking data lineage
[SourceLineageTag]	Source lineage tag from the original data source

Remarks

- Typically used in DAX queries to inspect and document model metadata.
- Permissions required depend on the host. Querying full metadata may require model admin permissions.

Example

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.USERDEFINEDFUNCTIONS()
```

See also

- INFO.FUNCTIONS
- INFO.MEASURES

Last updated on 03/18/2026

INFO.VARIATIONS

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation DAX query

Returns a table with information about all variations in the semantic model. This information helps you understand the model.

Syntax

```DAX
INFO.VARIATIONS([<Restriction name>, <Restriction value>], ...)
```

Parameters

Parameters are optional for this DAX function. When parameters are used, both must be given. More than one pair of parameters is allowed. The restriction name and value are text and entered in double-quotes.

Expand table

Term	Definition
Restriction name	Name of the restriction used to filter the results.
Restriction value	Value used to filter the results of the restriction.

Restrictions

Typically, all columns of the DAX function results can be used as a restriction. Additional restrictions may also be allowed.

Return value

A table with the following columns:

Expand table

Column	Description
[ID]	A reference to the object. IDs are usually autogenerated and should not be changed after the model is created. Data type is unsigned long (4 bytes).
[ColumnID]	A reference to the object that owns this variation.
[Name]	The name of the variation, used in code, script, and queries.
[Description]	The description of the variation.
[RelationshipID]	The ID of the relationship on which the variation is defined.
[DefaultHierarchyID]	THe ID of the default hierarchy for the variation.
[DefaultColumnID]	The ID of the default column for the variation.
[IsDefault]	A True or False indicator indicating if the variation is default.

Remarks

Can only be run by users with write permission on the semantic model and not when live connected to the semantic model in Power BI Desktop. This function can be used in DAX queries, and can't be used in calculations.

Example 1 - DAX query

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    INFO.VARIATIONS()
```

This DAX query returns a table with all of the columns of this DAX function.

Example 2 - DAX query with joins

The following DAX query can be run in DAX query view:

DAX

```
EVALUATE
    VAR _INFO =
    INFO.VARIATIONS()
```

```
VAR _ModelColumns =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "ColumnID", [ID],
        "Column Name", [ExplicitName]
    )
VAR _ModelColumns2 =
    SELECTCOLUMNS(
        INFO.COLUMNS(),
        "DefaultColumnID", [ID],
        "Default Column Name", [ExplicitName]
    )
VAR _ModelRelationships =
    SELECTCOLUMNS(
        INFO.RELATIONSHIPS(),
        "RelationshipID", [ID],
        "Relationship Name", [Name]
    )
VAR _ModelHierarchies =
    SELECTCOLUMNS(
        INFO.HIERARCHIES(),
        "DefaultHierarchyID", [ID],
        "Default Hierarchy Name", [Name]
    )


VAR _CombinedTable2 =
NATURALLEFTOUTERJOIN(
    _INFO,
    _ModelColumns
)
VAR _CombinedTable3 =
NATURALLEFTOUTERJOIN(
    _CombinedTable2,
    _ModelColumns2
)

VAR _CombinedTable4 =
NATURALLEFTOUTERJOIN(
    _CombinedTable3,
    _ModelRelationships
)

VAR _CombinedTable5 =
NATURALLEFTOUTERJOIN(
    _CombinedTable4,
    _ModelHierarchies
)

RETURN
    SELECTCOLUMNS(
        _CombinedTable5,
        "Variation name", [Name],
        "Variation description", [Description],
```

```
            "Column Name", [Column Name],
            "Default Column Name", [Default Column Name],
            "Relationship Name", [Relationship Name],
            "Default Hierarchy Name", [Default Hierarchy Name]
        )
    ORDER BY [Variation name]
```

This DAX query returns a table with only the specified columns and joining to other INFO DAX functions and the variations table.

See also

- INFO.TABLES
- INFO.COLUMNS
- INFO.MEASURES
- INFO functions overview

Last updated on 02/11/2026

Information functions

DAX information functions look at the cell or row that is provided as an argument and tells you whether the value matches the expected type. For example, the ISERROR function returns `TRUE` if the value that you reference contains an error.

In this category

Expand table

Function	Description
COLUMNSTATISTICS	Returns a table of statistics regarding every column in every table in the model.
CONTAINS	Returns `TRUE` if values for all referred columns exist, or are contained, in those columns. Otherwise, returns `FALSE`.
CONTAINSROW	Returns `TRUE` if a row of values exists or is contained in a table. Otherwise, returns `FALSE`.
CONTAINSSTRING	Returns `TRUE` if one string contains another string. Otherwise, returns `FALSE`.
CONTAINSSTRINGEXACT	Returns `TRUE` if one string contains another string. Otherwise, returns `FALSE`.
CUSTOMDATA	Returns the content of the CustomData property in the connection string.
HASONEFILTER	Returns `TRUE` if the number of directly filtered values on `columnName` is one. Otherwise, returns `FALSE`.
HASONEVALUE	Returns `TRUE` if the context for `columnName` has been filtered down to one distinct value only. Otherwise, returns `FALSE`.
ISAFTER	Returns `TRUE` if a row meets all the condition parameters that emulate the behavior of a Start At clause. Otherwise, returns `FALSE`.
ISBLANK	Returns `TRUE` if a value is blank. Otherwise, returns `FALSE`.
ISBOOLEAN	Returns `TRUE` if a value is a logical value. Otherwise, returns `FALSE`. Alias of ISLOGICAL.
ISCROSSFILTERED	Returns `TRUE` if `columnName` or another column in the same or related table is being filtered. Otherwise, returns `FALSE`.

Function	Description
ISCURRENCY	Returns `TRUE` if a value is a decimal number. Otherwise, returns `FALSE`. Alias of ISDECIMAL.
ISDATETIME	Returns `TRUE` if a value is a date / time. Otherwise, returns `FALSE`.
ISDECIMAL	Returns `TRUE` if a value is a decimal number. Otherwise, returns `FALSE`. Alias of ISCURRENCY.
ISDOUBLE	Returns `TRUE` if a value is a floating-point number. Otherwise, returns `FALSE`.
ISEMPTY	Returns `TRUE` if a table is empty. Otherwise, returns `FALSE`.
ISERROR	Returns `TRUE` if a value is an error. Otherwise, returns `FALSE`.
ISEVEN	Returns `TRUE` if a number is even. Otherwise, returns `FALSE`.
ISFILTERED	Returns `TRUE` if `columnName` is being filtered directly. Otherwise, returns `FALSE`.
ISINSCOPE	Returns `TRUE` if the specified column is the level in a hierarchy of levels. Otherwise, returns `FALSE`.
ISINT64	Returns `TRUE` if a value is a whole number. Otherwise, returns `FALSE`. Alias of ISINTEGER.
ISINTEGER	Returns `TRUE` if a value is a whole number. Otherwise, returns `FALSE`. Alias of ISINT64.
ISLOGICAL	Returns `TRUE` if a value is a logical value. Otherwise, returns `FALSE`. Alias of ISBOOLEAN.
ISNONTEXT	Returns `TRUE` if a value is not text, including blank cells. Otherwise, returns `FALSE`.
ISNUMBER	Returns `TRUE` if a value is a number. Otherwise, returns `FALSE`. Alias of ISNUMERIC.
ISNUMERIC	Returns `TRUE` if a value is a number. Otherwise, returns `FALSE`. Alias of ISNUMBER.
ISODD	Returns `TRUE` if a number is odd. Otherwise, returns `FALSE`.
ISONORAFTER	Returns `TRUE` if a row meets all the condition parameters that emulate the behavior of a Start At clause. Otherwise, returns `FALSE`.
ISSELECTEDMEASURE	Used by expressions for calculation items to determine the measure that is in context is one of those specified in a list of measures.

Function	Description
ISSTRING	Returns `TRUE` if a value is text. Otherwise, returns `FALSE`. Alias of ISTEXT.
ISSUBTOTAL	Creates another column in a SUMMARIZE expression that returns `TRUE` if the row contains subtotal values for the specified column. Otherwise, returns `FALSE`.
ISTEXT	Returns `TRUE` if a value is text. Otherwise, returns `FALSE`. Alias of ISSTRING.
NAMEOF	Returns the name of a table, column, measure, or calendar as a text string.
NONVISUAL	Marks a value filter in a SUMMARIZECOLUMNS expression as non-visual.
SELECTEDMEASURE	Used by expressions for calculation items to reference the measure that is in context.
SELECTEDMEASUREFORMATSTRING	Used by expressions for calculation items to retrieve the format string of the measure that is in context.
SELECTEDMEASURENAME	Used by expressions for calculation items to determine the measure that is in context by name.
TABLEOF	Returns a reference to the table associated with a specified column, measure, or calendar.
TEXTCONTAINS	Returns `TRUE` if text in a column matches a search string according to full-text search semantics. Otherwise, returns `FALSE`.
TEXTSIMILARITY	Returns a relevance score that indicates how strongly text in a column matches a search string.
USERCULTURE	Returns the locale for the current user.
USERNAME	Returns the domain name and username from the credentials given to the system at connection time.
USEROBJECTID	Returns the current user's Object ID or SID.
USERPRINCIPALNAME	Returns the user principal name.

Last updated on 09/29/2026

COLUMNSTATISTICS

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns a table of statistics for every column in every table in the model.

Syntax

```DAX
COLUMNSTATISTICS ()
```

Parameters

This function doesn't take any parameters.

Return value

A table of statistics. Each row of this table represents a different column in the model. Table columns include:

- `Table Name`: The current column's table.
- `Column Name`: The current column's name.
- `Min`: The minimum value found within the current column.
- `Max`: The maximum value found within the current column.
- `Cardinality`: The number of distinct values found within the current column.
- `Max Length`: The length of the longest string found within the current column (only applicable for string columns).

Remarks

- Columns in an error state and columns from query-scope calculated tables don't appear in the result table.
- If you apply a filter from the filter context to `COLUMNSTATISTICS()`, the function returns an error.
- For binary-typed columns, the Min and Max statistics have BLANK values.

Example

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

The following DAX query:

DAX

```
DEFINE
    TABLE FilteredProduct =
        FILTER ( Product, [Color] == "Blue" )
    COLUMN Customer[Location] = [State-Province] & " " & [Country-Region]

EVALUATE
COLUMNSTATISTICS ()
```

Returns a table with statistics for all columns from all tables in the model. The table also includes statistics for the query-scope calculated column, Customer[Location]. However, the table doesn't include the columns from the query-scope calculated table, FilteredProduct.

The following excerpt shows the **Customer** table rows from the result, including the calculated **Location** column:

Expand table

Table Name	Column Name	Min	Max	Cardinality	Max Length
Customer	CustomerKey	-1	29483	18485	
Customer	Customer ID	[Not Applicable]	AW00029483	18485	16
Customer	Customer	[Not Applicable]	Zoe Watson	18401	26
Customer	City	[Not Applicable]	York	270	21
Customer	State-Province	[Not Applicable]	Yveline	54	19

Table Name	Column Name	Min	Max	Cardinality	Max Length
Customer	Country-Region	[Not Applicable]	United States	7	16
Customer	Postal Code	[Not Applicable]	YO15	324	16
Customer	Location	[Not Applicable] [Not Applicable]	Yveline France	54	33

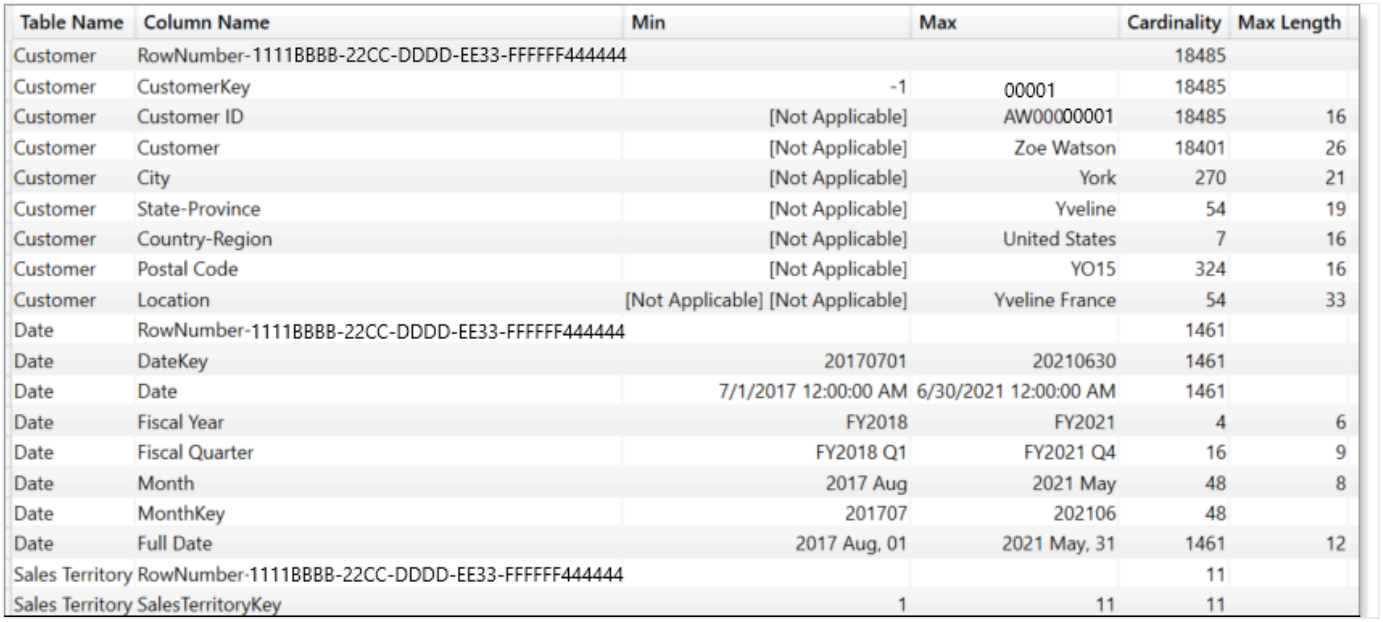


Table Name	Column Name	Min	Max	Cardinality	Max Length
Customer	RowNumber-1111BBBB-22CC-DDDD-EE33-FFFFFF444444			18485	
Customer	CustomerKey	-1	00001	18485	
Customer	Customer ID	[Not Applicable]	AW00000001	18485	16
Customer	Customer	[Not Applicable]	Zoe Watson	18401	26
Customer	City	[Not Applicable]	York	270	21
Customer	State-Province	[Not Applicable]	Yveline	54	19
Customer	Country-Region	[Not Applicable]	United States	7	16
Customer	Postal Code	[Not Applicable]	YO15	324	16
Customer	Location	[Not Applicable] [Not Applicable]	Yveline France	54	33
Date	RowNumber-1111BBBB-22CC-DDDD-EE33-FFFFFF444444			1461	
Date	DateKey	20170701	20210630	1461	
Date	Date	7/1/2017 12:00:00 AM	6/30/2021 12:00:00 AM	1461	
Date	Fiscal Year	FY2018	FY2021	4	6
Date	Fiscal Quarter	FY2018 Q1	FY2021 Q4	16	9
Date	Month	2017 Aug	2021 May	48	8
Date	MonthKey	201707	202106	48	
Date	Full Date	2017 Aug, 01	2021 May, 31	1461	12
Sales Territory	RowNumber-1111BBBB-22CC-DDDD-EE33-FFFFFF444444			11	
Sales Territory	SalesTerritoryKey	1	11	11	

Related content

- Filter context
- CALCULATETABLE function

Last updated on 08/04/2026

CONTAINS

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns true if values for all referred columns exist, or are contained, in those columns; otherwise, the function returns false.

Syntax

DAX

```
CONTAINS(<table>, <columnName>, <value>[, <columnName>, <value>]…)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data.
columnName	The name of an existing column, using standard DAX syntax. It cannot be an expression.
value	Any DAX expression that returns a single scalar value, that is to be sought in *columnName*. The expression is to be evaluated exactly once and before it is passed to the argument list.

Return value

A value of **TRUE** if each specified *value* can be found in the corresponding *columnName*, or are contained, in those columns; otherwise, the function returns **FALSE**.

Remarks

- The arguments *columnName* and *value* must come in pairs; otherwise an error is returned.

- *columnName* must belong to the specified *table*, or to a table that is related to *table*.
- If *columnName* refers to a column in a related table then it must be fully qualified; otherwise, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example creates a measure that tells you whether there were any Internet sales of product 214 and to customer 11185 at the same time.

DAX

```
= CONTAINS(InternetSales, [ProductKey], 214, [CustomerKey], 11185)
```

Feedback

Was this page helpful? Yes No

CONTAINSROW function

Applies to: Calculated column Calculated table Measure Visual calculation

Returns `TRUE` if there exists at least one row where all columns have specified values.

Syntax

```
DAX
CONTAINSROW(<Table>, <Value> [, <Value> [, …] ] )
```

Parameters

Expand table

Term	Definition
`Table`	A table to test.
`Value`	Any valid DAX expression that returns a scalar value.

Return value

`TRUE` or `FALSE`.

Remarks

- Other than syntax, the `IN` operator and CONTAINSROW function are functionally equivalent.

  ```
  DAX
  <scalarExpr> IN <tableExpr>
  ( <scalarExpr1>, <scalarExpr2>, … ) IN <tableExpr>
  ```

  - The number of scalarExprN must match the number of columns in tableExpr.
  - NOT IN isn't an operator in DAX. To perform the logical negation of the IN operator, put NOT in front of the entire expression. For example, NOT [Color] IN { "Red", "Yellow", "Blue" }.

- Unlike the = operator, the IN operator and the CONTAINSROW function perform strict comparison. For example, the BLANK value doesn't match 0.

Examples

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

Example 1

The following DAX queries:

```
DAX

EVALUATE
FILTER ( ALL ( Product[Color] ), ( [Color] ) IN { "Red", "Yellow", "Blue" } )
ORDER BY [Color]
```

and

```
DAX

EVALUATE
FILTER (
    ALL ( Product[Color] ),
    CONTAINSROW ( { "Red", "Yellow", "Blue" }, [Color] )
)
ORDER BY [Color]
```

Return the following table with a single column:

Expand table

[Color]
Blue
Red
Yellow

Example 2

The following equivalent DAX queries:

DAX

```
EVALUATE
FILTER ( ALL ( Product[Color] ), NOT [Color] IN { "Red", "Yellow", "Blue" } )
ORDER BY [Color]
```

and

DAX

```
EVALUATE
FILTER (
    ALL ( Product[Color] ),
    NOT CONTAINSROW ( { "Red", "Yellow", "Blue" }, [Color] )
)
ORDER BY [Color]
```

Return the following table with a single column:

Expand table

[Color]
Black
Grey
Multi
NA
Silver
Silver/Black
White

Related content

- IN operator
- DAX queries

Last updated on 08/04/2026

CONTAINSSTRING

Applies to: Calculated column Calculated table Measure Visual calculation

Returns `TRUE` or `FALSE` indicating whether one string contains another string.

Syntax

```DAX
CONTAINSSTRING(<within_text>, <find_text>)
```

Parameters

Expand table

Term	Definition
`within_text`	The text in which you want to search for find_text.
`find_text`	The text you want to find.

Return value

`TRUE` if find_text is a substring of within_text; otherwise `FALSE`.

Remarks

- CONTAINSSTRING is case-insensitive, kanatype-insensitive, width-insensitive and accent sensitive.
- You can use `?` and `*` wildcard characters. Use `~` to escape wildcard characters.

Example

DAX query

```DAX
EVALUATE
    ROW(
```

```
        "Case 1", CONTAINSSTRING("abcd", "bc"),
        "Case 2", CONTAINSSTRING("abcd", "BC"),
        "Case 3", CONTAINSSTRING("abcd", "a*d"),
        "Case 4", CONTAINSSTRING("abcd", "ef")
    )
```

Returns

Expand table

[Case 1]	[Case 2]	[Case 3]	[Case 4]
TRUE	TRUE	TRUE	FALSE

Related content

- TEXTCONTAINS

Last updated on 09/29/2026

CONTAINSSTRINGEXACT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns TRUE or FALSE indicating whether one string contains another string.

Syntax

```
DAX

CONTAINSSTRINGEXACT(<within_text>, <find_text>)
```

Parameters

Expand table

Term	Definition
within_text	The text in which you want to search for find_text.
find_text	The text you want to find.

Return value

TRUE if find_text is a substring of within_text; otherwise FALSE.

Remarks

CONTAINSSTRINGEXACT is case-sensitive.

Example

DAX query

```
DAX

EVALUATE
    ROW(
```

```
        "Case 1", CONTAINSSTRINGEXACT("abcd", "bc"),
        "Case 2", CONTAINSSTRINGEXACT("abcd", "BC"),
        "Case 3", CONTAINSSTRINGEXACT("abcd", "a*d"),
        "Case 4", CONTAINSSTRINGEXACT("abcd", "ef")
    )
```

Returns

Expand table

[Case 1]	[Case 2]	[Case 3]	[Case 4]
TRUE	FALSE	FALSE	FALSE

Feedback

Was this page helpful? Yes No

CUSTOMDATA

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the content of the **CustomData** property in the connection string.

Syntax

DAX

```
CUSTOMDATA()
```

Return value

The content of the **CustomData** property in the connection string.

Blank, if **CustomData** property was not defined at connection time.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX formula verifies if the CustomData property was set to "**OK**".

DAX

```
= IF(CUSTOMDATA()="OK", "Correct Custom data in connection string", "No
custom data in connection string property or unexpected value")
```

Feedback

Was this page helpful? Yes No

HASONEFILTER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns **TRUE** when the number of directly filtered values on *columnName* is one; otherwise returns **FALSE**.

Syntax

```DAX
HASONEFILTER(<columnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column, using standard DAX syntax. It cannot be an expression.

Return value

TRUE when the number of directly filtered values on *columnName* is one; otherwise returns **FALSE**.

Remarks

- This function is similar to HASONEVALUE() with the difference that HASONEVALUE() works based on cross-filters while HASONEFILTER() works by a direct filter.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to use HASONEFILTER() to return the filter for ResellerSales_USD[ProductKey]) if there is one filter, or to return BLANK if there are no filters or more than one filter on ResellerSales_USD[ProductKey]).

DAX

```
=
IF(HASONEFILTER(ResellerSales_USD[ProductKey]),FILTERS(ResellerSales_USD[Pro
ductKey]),BLANK())
```

Feedback

Was this page helpful? Yes No

HASONEVALUE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns **TRUE** when the context for *columnName* has been filtered down to one distinct value only. Otherwise is **FALSE**.

Syntax

```HTML
HASONEVALUE(<columnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column, using standard DAX syntax. It cannot be an expression.

Return value

TRUE when the context for *columnName* has been filtered down to one distinct value only. Otherwise is **FALSE**.

Remarks

- An equivalent expression for HASONEVALUE() is `COUNTROWS(VALUES(<columnName>)) = 1`.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following measure formula verifies if the context is being sliced by one value in order to estimate a percentage against a predefined scenario; in this case you want to compare Reseller Sales against sales in 2007, then you need to know if the context is filtered by single years. Also, if the comparison is meaningless you want to return BLANK.

DAX

```
=
IF(HASONEVALUE(DateTime[CalendarYear]),SUM(ResellerSales_USD[SalesAmount_USD
])/CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]),DateTime[CalendarYear]=
2007),BLANK())
```

Feedback

Was this page helpful? Yes No

ISAFTER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

A boolean function that emulates the behavior of a 'Start At' clause and returns true for a row that meets all of the condition parameters.

Based on the sort order, the first parameter is compared with the second parameter. If the sort order is ascending, the comparison to be done is first parameter greater than the second parameter. If the sort order is descending, the comparison to be done is second parameter less than the first parameter.

Syntax

DAX

```
ISAFTER(<scalar_expression>, <scalar_expression>[, sort_order [,
<scalar_expression>, <scalar_expression>[, sort_order]]…)
```

Parameters

Expand table

Term	Definition
scalar expression	Any expression that returns a scalar value like a column reference or integer or string value. Typically the first parameter is a column reference and the second parameter is a scalar value.
sort order	(optional) The order in which the column is sorted. Can be ascending (ASC) or descending (DEC). By default the sort order is ascending.

Return value

True or false.

Remarks

This function is similar to ISONORAFTER. The difference is ISAFTER returns true for values sorted strictly *after* the filter values, where ISONORAFTER returns true for values sorted *on or after* the filter values.

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Table name: 'Info'

Expand table

Country/Region	State	Count	Total
IND	JK	20	800
IND	MH	25	1000
IND	WB	10	900
USA	CA	5	500
USA	WA	10	900

The following expression:

DAX

```
FILTER (
    Info,
    ISAFTER (
        Info[Country], "IND", ASC,
        Info[State], "MH", ASC )
)
```

Returns:

Expand table

Country	State	Count	Total
IND	WB	10	900
USA	CA	5	500
USA	WA	10	900

Related content

ISONORAFTER

Feedback

Was this page helpful? Yes No

ISBLANK

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is blank, and returns TRUE or FALSE.

Syntax

DAX

```
ISBLANK(<value>)
```

Parameters

Expand table

Term	Definition
value	The value or expression you want to test.

Return value

A Boolean value of TRUE if the value is blank; otherwise FALSE.

Remarks

To learn more about best practices when working with BLANKS, see Avoid converting BLANKs to values in DAX.

Example

This formula computes the increase or decrease ratio in sales compared to the previous year. The example uses the IF function to check the value for the previous year's sales in order to avoid a divide by zero error.

DAX

```
//Sales to Previous Year Ratio

= IF( ISBLANK('CalculatedMeasures'[PreviousYearTotalSales])
   , BLANK()
   , ( 'CalculatedMeasures'[Total Sales]-
'CalculatedMeasures'[PreviousYearTotalSales] )
      /'CalculatedMeasures'[PreviousYearTotalSales])
```

Result,

Expand table

Row Labels	Total Sales	Total Sales Previous Year	Sales to Previous Year Ratio
2005	$10,209,985.08		
2006	$28,553,348.43	$10,209,985.08	179.66%
2007	$39,248,847.52	$28,553,348.43	37.46%
2008	$24,542,444.68	$39,248,847.52	-37.47%
Grand Total	$102,554,625.71		

Related content

Information functions

Feedback

Was this page helpful? Yes No

ISBOOLEAN

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a logical value, (`TRUE` or `FALSE`), and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISBOOLEAN(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value that you want to test.

Return value

`TRUE` if the value is a logical value; `FALSE` if any value other than `TRUE` OR `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISLOGICAL.

Example

The following DAX query shows the behavior of ISBOOLEAN.

```
DAX

EVALUATE
{
    IF ( ISBOOLEAN ( TRUE ), "Is Boolean type or Logical", "Is different type" ),
```

```
// RETURNS: Is Boolean type or Logical
    IF ( ISBOOLEAN ( FALSE ), "Is Boolean type or Logical", "Is different type" ),
// RETURNS: Is Boolean type or Logical
    IF ( ISBOOLEAN ( 42 ), "Is Boolean type or Logical", "Is different type" ) //
RETURNS: Is different type
}
```

Related content

- ISLOGICAL
- Information functions

ISCROSSFILTERED

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns TRUE when the specified table or column is cross-filtered.

Syntax

DAX

```
ISCROSSFILTERED(<TableNameOrColumnName>)
```

Parameters

Expand table

Term	Definition
TableNameOrColumnName	The name of an existing table or column. It cannot be an expression.

Return value

TRUE when *ColumnName* or a column of *TableName* is being cross-filtered. Otherwise returns FALSE.

Remarks

- A column or table is said to be cross-filtered when a filter is applied to *ColumnName*, any column of *TableName*, or to any column of a related table.
- A column or table is said to be filtered directly when a filter is applied to *ColumnName* or to any column of *TableName*. Therefore, the ISFILTERED function also returns TRUE when *ColumnName* or any column of *TableName* is filtered.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

ISFILTERED function
FILTERS function
HASONEFILTER function
HASONEVALUE function

Feedback

Was this page helpful? Yes No

ISCURRENCY

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a decimal number, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISCURRENCY(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a a decimal number; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This functions is an alias of ISDECIMAL.

Example

The following DAX query shows the behavior of ISCURRENCY.

```DAX
EVALUATE
{
    IF ( ISCURRENCY ( 3.1E-1 ), "Is currency", "Is not currency" ), // RETURNS: Is
```

```
not currency
    IF ( ISCURRENCY ( "42" ), "Is currency", "Is not currency" ), // RETURNS: Is
not currency
    IF ( ISCURRENCY ( 42 ), "Is currency", "Is not currency" ), // RETURNS: Is not
currency
    IF ( ISCURRENCY ( CURRENCY ( 4.2421 ) ), "Is currency", "Is not currency" ) //
RETURNS: Is currency
}
```

Related content

- ISDECIMAL
- Information functions

ISDATETIME

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a date / time, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISDATETIME(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a date / time; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query shows the behavior of ISDATETIME.

```DAX
EVALUATE
{
    IF ( ISDATETIME ( 42 ), "Is datetime", "Is not datetime" ), // RETURNS: Is not
datetime
```

```
    IF ( ISDATETIME ( "42" ), "Is datetime", "Is not datetime" ), // RETURNS: Is
not datetime
    IF ( ISDATETIME ( "2025-07-01" ), "Is datetime", "Is not datetime" ), //
RETURNS: Is not datetime
    IF ( ISDATETIME ( dt"2025-07-01" ), "Is datetime", "Is not datetime" ), //
RETURNS: Is datetime
    IF ( ISDATETIME ( DATE ( 2025, 7, 1 ) ), "Is datetime", "Is not datetime" ) //
RETURNS: Is datetime
}
```

Related content

- Information functions

ISDECIMAL

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a decimal number, and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISDECIMAL(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a decimal number; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This functions is an alias of ISCURRENCY.

Example

The following DAX query shows the behavior of ISDECIMAL.

```
DAX

EVALUATE
{
    IF ( ISDECIMAL ( 3.1E-1 ), "Is decimal", "Is not decimal" ), // RETURNS: Is
```

```
not decimal
    IF ( ISDECIMAL ( "42" ), "Is decimal", "Is not decimal" ), // RETURNS: Is not
decimal
    IF ( ISDECIMAL ( 42 ), "Is decimal", "Is not decimal" ), // RETURNS: Is not
decimal
    IF ( ISDECIMAL ( CURRENCY ( 4.2421 ) ), "Is decimal", "Is not decimal" ) //
RETURNS: Is decimal
}
```

Related content

- ISCURRENCY
- Information functions

ISDOUBLE

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a floating-point number, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISDOUBLE(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a floating-point number; otherwise `FALSE`.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query shows the behavior of ISDOUBLE.

```DAX
EVALUATE
{
    IF ( ISDOUBLE ( 4.2 ), "Is double", "Is not double" ), // RETURNS: Is double
    IF ( ISDOUBLE ( 3.1E-1 ), "Is double", "Is not double" ), // RETURNS: Is
```

```
double
    IF ( ISDOUBLE ( "42" ), "Is double", "Is not double" ), // RETURNS: Is not
double
    IF ( ISDOUBLE ( 42 ), "Is double", "Is not double" ), // RETURNS: Is not
double
}
```

Related content

Information functions

ISEMPTY

Applies to: Calculated column Calculated table Measure Visual calculation

Checks if a table is empty.

Syntax

```DAX
ISEMPTY(<tableExpression>)
```

Parameters

Expand table

Term	Definition
`tableExpression`	A table reference or a DAX expression that returns a table.

Return value

True if the table is empty (has no rows), if else, False.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

For the below table named 'Info':

Expand table

Country/Region	State	County	Total
IND	JK	20	800
IND	MH	25	1000

Country/Region	State	County	Total
IND	WB	10	900
USA	CA	5	500
USA	WA	10	900

DAX

```
EVALUATE
ROW("Any countries with count > 25?", NOT(ISEMPTY(FILTER(Info, [County]>25))))
```

Return value: `FALSE``

Last updated on 08/17/2026

ISERROR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is an error, and returns TRUE or FALSE.

Syntax

DAX

```
ISERROR(<value>)
```

Parameters

Expand table

Term	Definition
value	The value you want to test.

Return value

A Boolean value of TRUE if the value is an error; otherwise FALSE.

Remarks

- For best practices when using ISERROR, see Appropriate use of error functions.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example calculates the ratio of total Internet sales to total reseller sales. The ISERROR function is used to check for errors, such as division by zero. If there is an error a blank is returned, otherwise the ratio is returned.

DAX

```
= IF( ISERROR(
       SUM('ResellerSales_USD'[SalesAmount_USD])
       /SUM('InternetSales_USD'[SalesAmount_USD])
            )
    , BLANK()
    , SUM('ResellerSales_USD'[SalesAmount_USD])
      /SUM('InternetSales_USD'[SalesAmount_USD])
    )
```

Related content

Information functions
IFERROR function
IF function

Feedback

Was this page helpful? Yes No

ISEVEN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns TRUE if number is even, or FALSE if number is odd.

Syntax

DAX

```
ISEVEN(number)
```

Parameters

Expand table

Term	Definition
number	The value to test. If number is not an integer, it is truncated.

Return value

Returns TRUE if number is even, or FALSE if number is odd.

Remarks

- If number is nonnumeric, ISEVEN returns the #VALUE! error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

ISFILTERED

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns TRUE when the specified table or column is being filtered directly.

Syntax

```DAX
ISFILTERED(<TableNameOrColumnName>)
```

Parameters

Expand table

Term	Definition
TableNameOrColumnName	The name of an existing table or column. It cannot be an expression.

Return value

TRUE when *ColumnName* or a column of *TableName* is being filtered directly. Otherwise returns FALSE.

Remarks

- A column or table is said to be filtered directly when a filter is applied to *ColumnName* or any column of *TableName*.
- A column or table is said to be cross-filtered when a filter is applied to *ColumnName*, any column of *TableName*, or to any column of a related table. Therefore, the ISCROSSFILTERED function also returns TRUE when *ColumnName*, any column of *TableName*, or a column of a related table is filtered.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

ISCROSSFILTERED function
FILTERS function
HASONEFILTER function
HASONEVALUE function

Feedback

Was this page helpful? Yes No

ISINSCOPE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns true when the specified column is the level in a hierarchy of levels.

Syntax

DAX

```
ISINSCOPE(<columnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column, using standard DAX syntax. It cannot be an expression.

Return value

TRUE when the specified column is the level in a hierarchy of levels.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
DEFINE
MEASURE FactInternetSales[% of Parent] =
  SWITCH (TRUE(),
    ISINSCOPE(DimProduct[Subcategory]),
      DIVIDE(
        SUM(FactInternetSales[Sales Amount]),
        CALCULATE(
          SUM(FactInternetSales[Sales Amount]),
          ALLSELECTED(DimProduct[Subcategory])))
```

```
      ),
    ISINSCOPE(DimProduct[Category]),
      DIVIDE(
        SUM(FactInternetSales[Sales Amount]),
        CALCULATE(
          SUM(FactInternetSales[Sales Amount]),
          ALLSELECTED(DimProduct[Category]))
      ),
    1
  ) * 100
EVALUATE
  SUMMARIZECOLUMNS
  (
    ROLLUPADDISSUBTOTAL
    (
      DimProduct[Category], "Category Subtotal",
      DimProduct[Subcategory], "Subcategory Subtotal"
    ),
    TREATAS(
      {"Bike Racks", "Bike Stands", "Mountain Bikes", "Road Bikes", "Touring
Bikes"},
      DimProduct[Subcategory]),
    "Sales", SUM(FactInternetSales[Sales Amount]),
    "% of Parent", [% of Parent]
  )
  ORDER BY
    [Category Subtotal] DESC, [Category],
    [Subcategory Subtotal] DESC, [Subcategory]
```

Returns,

Expand table

DimProduct[Category]	DimProduct[SubCategory]	[Category Subtotal]	[Subcategory Subtotal]	[Sales]	[% of Parent]
		TRUE	TRUE	28,397,095.65	100.00
Accessories		FALSE	TRUE	78,951.00	0.28
Accessories	Bike Racks	FALSE	FALSE	39,360.00	49.85
Accessories	Bike Stands	FALSE	FALSE	39,591.00	50.15
Bikes		FALSE	TRUE	28,318,144.65	99.72
Bikes	Mountain Bikes	FALSE	FALSE	9,952,759.56	35.15
Bikes	Road Bikes	FALSE	FALSE	14,520,584.04	51.28
Bikes	Touring Bikes	FALSE	FALSE	3,844,801.05	13.58

Related content

SUMMARIZECOLUMNS function
CALCULATE function

Feedback

Was this page helpful? Yes No

ISINT64

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a whole number, and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISINT64(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a whole number; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISINTEGER.

Example

The following DAX query shows the behavior of ISINT64.

```
DAX

EVALUATE
{
    IF ( ISINT64 ( 4.2 ), "Is integer", "Is not integer" ), // RETURNS: Is not
```

```
integer
    IF ( ISINT64 ( 3.1E-1 ), "Is integer", "Is not integer" ), // RETURNS: Is not
integer
    IF ( ISINT64 ( "42" ), "Is integer", "Is not integer" ), // RETURNS: Is not
integer
    IF ( ISINT64 ( 42 ), "Is integer", "Is not integer" ) // RETURNS: Is integer
}
```

Related content

- ISINTEGER
- Information functions

ISINTEGER

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a whole number, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISINTEGER(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is a whole number; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISINT64.

Example

The following DAX query shows the behavior of ISINTEGER.

```DAX
EVALUATE
{
    IF ( ISINTEGER ( 4.2 ), "Is integer", "Is not integer" ), // RETURNS: Is not
```

```
integer
    IF ( ISINTEGER ( 3.1E-1 ), "Is integer", "Is not integer" ), // RETURNS: Is
not integer
    IF ( ISINTEGER ( "42" ), "Is integer", "Is not integer" ), // RETURNS: Is not
integer
    IF ( ISINTEGER ( 42 ), "Is integer", "Is not integer" ) // RETURNS: Is integer
}
```

Related content

- ISINTEGER
- Information functions

ISLOGICAL

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a logical value, (`TRUE` or `FALSE`), and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISLOGICAL(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value that you want to test.

Return value

`TRUE` if the value is a logical value; `FALSE` if any value other than `TRUE` OR `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISBOOLEAN.

Example

The following DAX query shows the behavior of ISLOGICAL.

```DAX
EVALUATE
{
    IF ( ISLOGICAL ( TRUE ), "Is Boolean type or Logical", "Is different type" ),
```

```
// RETURNS: Is Boolean type or Logical
    IF ( ISLOGICAL ( FALSE ), "Is Boolean type or Logical", "Is different type" ),
// RETURNS: Is Boolean type or Logical
    IF ( ISLOGICAL ( 42 ), "Is Boolean type or Logical", "Is different type" ) //
RETURNS: Is different type
}
```

Related content

- ISBOOLEAN
- Information functions

ISNONTEXT

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks if a value is not text (blank cells are not text), and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISNONTEXT(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to check.

Return value

`TRUE` if the value is not text or blank; `FALSE` if the value is text.

Remarks

- An empty string is considered text.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query shows the behavior of the ISNONTEXT function.

```
DAX

EVALUATE {
    IF( ISNONTEXT( 1 ), "Is Non-Text", "Is Text" ), // RETURNS: Is Non-Text
```

```
    IF( ISNONTEXT( BLANK ( ) ), "Is Non-Text", "Is Text" ), // RETURNS: Is Non-
Text
    IF( ISNONTEXT( ""), "Is Non-Text", "Is Text" ) // RETURNS: Is Text
}
```

Related content

Information functions

ISNUMBER

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a number, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISNUMBER(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is numeric; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISNUMERIC.

Example

The following DAX query shows the behavior of ISNUMBER.

```DAX
EVALUATE
{
    IF ( ISNUMBER ( 0 ), "Is number", "Is Not number" ), // RETURNS: Is number
```

```
    IF ( ISNUMBER ( 3.1E-1 ), "Is number", "Is Not number" ), // RETURNS: Is
number
    IF ( ISNUMBER ( "42" ), "Is number", "Is Not number" ) // RETURNS: Is Not
number
}
```

Related content

- ISNUMERIC
- Information functions

ISNUMERIC

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether a value is a number, and returns `TRUE` or `FALSE`.

Syntax

```DAX
ISNUMERIC(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to test.

Return value

`TRUE` if the value is numeric; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISNUMBER.

Example

The following DAX query shows the behavior of ISNUMERIC.

```DAX
EVALUATE
{
    IF ( ISNUMERIC ( 0 ), "Is number", "Is Not number" ), // RETURNS: Is number
```

```
    IF ( ISNUMERIC ( 3.1E-1 ), "Is number", "Is Not number" ), // RETURNS: Is
number
    IF ( ISNUMERIC ( "42" ), "Is number", "Is Not number" ) // RETURNS: Is Not
number
}
```

Related content

- ISNUMBER
- Information functions

ISODD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns TRUE if number is odd, or FALSE if number is even.

Syntax

DAX

```
ISODD(number)
```

Parameters

Expand table

Term	Definition
number	The value to test. If number is not an integer, it is truncated.

Return value

Returns TRUE if number is odd, or FALSE if number is even.

Remarks

- If number is nonnumeric, ISODD returns the #VALUE! error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

ISONORAFTER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

A boolean function that emulates the behavior of a Start At clause and returns true for a row that meets all of the condition parameters.

Based on the sort order, the first parameter is compared with the second parameter. If the sort order is ascending, the comparison to be done is first parameter greater than the second parameter. If the sort order is descending, the comparison to be done is second parameter less than the first parameter.

Syntax

DAX

```
ISONORAFTER(<scalar_expression>, <scalar_expression>[, sort_order [,
<scalar_expression>, <scalar_expression>[, sort_order]]…)
```

Parameters

Expand table

Term	Definition
scalar expression	Any expression that returns a scalar value like a column reference or integer or string value. Typically the first parameter is a column reference and the second parameter is a scalar value.
sort order	(optional) The order in which the column is sorted. Can be ascending (ASC) or descending (DESC). By default the sort order is ascending.

Return value

True or false.

Remarks

This function is similar to ISAFTER. The difference is ISONORAFTER returns true for values sorted *on or after* the filter values, where ISAFTER returns true for values sorted strictly *after* the filter values.

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

For the following table named, Info:

Expand table

Country/Region	State	Count	Total
IND	JK	20	800
IND	MH	25	1000
IND	WB	10	900
USA	CA	5	500
USA	WA	10	900

The following expression:

DAX

```
FILTER (
    Info,
    ISONORAFTER (
        Info[Country], "IND", ASC,
        Info[State], "MH", ASC )
)
```

Returns:

Expand table

Country/Region	State	Count	Total
IND	MH	25	1000
IND	WB	10	900
USA	CA	5	500

Country/Region	State	Count	Total
USA	WA	10	900

Related content

ISAFTER

Feedback

Was this page helpful? Yes No

ISSELECTEDMEASURE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Used by expressions for calculation items to determine the measure that is in context is one of those specified in a list of measures.

Syntax

DAX

```
ISSELECTEDMEASURE( M1, M2, ... )
```

Parameters

Expand table

Term	Definition
M1, M2, ...	A list of measures.

Return value

A Boolean indicating whether the measure that is currently in context is one of those specified in the list of parameters.

Remarks

- Can only be referenced in the expression for a calculation item.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following calculation item expression checks if the current measure is one of those specified in the list of parameters. If the measures are renamed, formula fixup will reflect

the name changes in the expression.

DAX

```
IF (
    ISSELECTEDMEASURE ( [Expense Ratio 1], [Expense Ratio 2] ),
    SELECTEDMEASURE (),
    DIVIDE ( SELECTEDMEASURE (), COUNTROWS ( DimDate ) )
)
```

Related content

SELECTEDMEASURE
SELECTEDMEASURENAME

Feedback

Was this page helpful? Yes No

ISSTRING

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks if a value is text, and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISSTRING(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to check.

Return value

`TRUE` if the value is text; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISTEXT.

Example

The following DAX query shows the behavior of the ISSTRING function.

```
DAX

EVALUATE
{
    IF ( ISSTRING ( "text" ), "Is Text", "Is Non-Text" ), // RETURNS: Is Text
```

```
    IF ( ISSTRING ( "" ), "Is Text", "Is Non-Text" ), // RETURNS: Is Text
    IF ( ISSTRING ( 42 ), "Is Text", "Is Non-Text" ), // RETURNS: Is Non-Text
    IF ( ISSTRING ( BLANK () ), "Is Text", "Is Non-Text" ) // RETURNS: Is Non-Text
}
```

Related content

- ISTEXT
- Information functions

ISSUBTOTAL

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Creates another column in a SUMMARIZE expression that returns True if the row contains subtotal values for the column given as argument, otherwise returns False.

Syntax

DAX

```
ISSUBTOTAL(<columnName>)
```

With SUMMARIZE,

DAX

```
SUMMARIZE(<table>, <groupBy_columnName>[, <groupBy_columnName>]…[,
ROLLUP(<groupBy_columnName>[,< groupBy_columnName>…])][, <name>,
{<expression>|ISSUBTOTAL(<columnName>)}]…)
```

Parameters

Expand table

Term	Definition
columnName	The name of any column in table of the SUMMARIZE function or any column in a related table to table.

Return value

A True value if the row contains a subtotal value for the column given as argument, otherwise returns False.

Remarks

- This function can only be used in the expression of a SUMMARIZE function.

- This function must be preceded by the name of the Boolean column.

Example

See SUMMARIZE.

Feedback

Was this page helpful? Yes No

ISTEXT

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Checks if a value is text, and returns `TRUE` or `FALSE`.

Syntax

```
DAX

ISTEXT(<value>)
```

Parameters

Expand table

Term	Definition
`value`	The value you want to check.

Return value

`TRUE` if the value is text; otherwise `FALSE`.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function is an alias of ISSTRING.

Example

The following DAX query shows the behavior of the ISTEXT function.

```
DAX

EVALUATE
{
    IF ( ISTEXT ( "text" ), "Is Text", "Is Non-Text" ), // RETURNS: Is Text
```

```
    IF ( ISTEXT ( "" ), "Is Text", "Is Non-Text" ), // RETURNS: Is Text
    IF ( ISTEXT ( 42 ), "Is Text", "Is Non-Text" ), // RETURNS: Is Non-Text
    IF ( ISTEXT ( BLANK () ), "Is Text", "Is Non-Text" ) // RETURNS: Is Non-Text
}
```

Related content

- ISSTRING
- Information functions

NAMEOF

Applies to: Calculated column · Calculated table · Measure · Visual calculation

Returns the name of a table, column, measure, or calendar as a text string. Optional parameters control which component of the name is returned and how the result is escaped.

Syntax

DAX

```
NAMEOF ( <object> [, <component> [, <escaped>]] )
```

Parameters

Expand table

Term	Definition
`object`	The table, column, measure, or calendar whose name you want to retrieve.
`component`	(Optional) An enumeration that specifies which part of the qualified name to return. If omitted, defaults to `FULL`. See Component values.
`escaped`	(Optional) An enumeration that specifies how the returned name is escaped. If omitted, defaults to `ESCAPED`. See Escaped values.

Component values

The `component` parameter accepts the following values:

Expand table

Value	Description
`TABLE`	Returns the table name. Returns an error if the object isn't associated with a table (for example, a calendar).
`COLUMN`	Returns the column name. Returns an error if the object isn't a column.
`MEASURE`	Returns the measure name. Returns an error if the object isn't a measure.
`CALENDAR`	Returns the calendar name. Returns an error if the object isn't a calendar.

Value	Description
`FULL`	(Default) Returns the fully qualified name of the object.
`SELF`	Returns the name of the object itself: the column or measure name for columns and measures, or the table/calendar name for tables and calendars.
`PARENT`	Returns the parent table name for columns and measures. Returns an error for tables and calendars.

Escaped values

The `escaped` parameter accepts the following values:

Expand table

Value	Description
`ESCAPED`	(Default) Returns the name with full DAX escaping: table names wrapped in single quotes, column and measure names wrapped in square brackets.
`UNESCAPED`	Returns the raw name without any delimiters or escape characters. Returns an error for fully qualified names that contain both a parent and child component.
`MINIMALLYESCAPED`	Returns the name with escaping applied only when the name requires it. Names that contain only simple letters, digits, and underscores are returned without delimiters. Names that contain spaces or special characters are returned with escaping.

Return value

A text string with the requested name, formatted based on the component and escaped parameters.

Remarks

- When called with only the `object` argument, NAMEOF behaves the same as in previous versions, returning a fully qualified, escaped name. Because `component` defaults to `FULL` and `escaped` defaults to `ESCAPED`, the return formats are:
  - For tables: `'TableName'`.
  - For columns: `'TableName'[ColumnName]`.
  - For measures: `'TableName'[MeasureName]`.
  - For calendars: `'CalendarName'`.
  - For variation columns: `'TableName'[ColumnName].[VariationName]`.
- Variables and dynamic expressions aren't supported as arguments to NAMEOF.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Component behavior by input type

The following table shows the result of each `component` value for different input types, using the default `ESCAPED` mode. "Error" indicates that the combination returns an error.

Expand table

Input type	`TABLE`	`COLUMN`	`MEASURE`	`CALENDAR`	`FULL`	`SELF`	`PARENT`
Column - `Sales[Sales Amount]`	`'Sales'`	`[Sales Amount]`	Error	Error	`'Sales'[Sales Amount]`	`[Sales Amount]`	`'Sales'`
Table - `Sales`	`'Sales'`	Error	Error	Error	`'Sales'`	`'Sales'`	Error
Measure - `Sales[m1]`	`'Sales'`	Error	`[m1]`	Error	`'Sales'[m1]`	`[m1]`	`'Sales'`
Calendar - `myCalendar`	Error	Error	Error	`'myCalendar'`	`'myCalendar'`	`'myCalendar'`	Error

Escaped behavior

The `escaped` parameter controls how the name produced by the `component` step is formatted. The following table uses the fully escaped name as a reference and shows the output for each `escaped` value.

Expand table

Fully escaped name	`ESCAPED`	`UNESCAPED`	`MINIMALLYESCAPED`
`'Sales'`	`'Sales'`	`Sales`	`Sales`
`'Sales'[Sales Amount]`	`'Sales'[Sales Amount]`	Error	`Sales[Sales Amount]`
`'Sales'[m1]`	`'Sales'[m1]`	Error	`Sales[m1]`
`[Amount]`	`[Amount]`	`Amount`	`Amount`
`'Sales Region'`	`'Sales Region'`	`Sales Region`	`'Sales Region'`
`'Sales Region'[Column]`	`'Sales Region'[Column]`	Error	`'Sales Region'[Column]`

Fully escaped name	ESCAPED	UNESCAPED	MINIMALLYESCAPED
[Order Quantity]	[Order Quantity]	Order Quantity	[Order Quantity]

> **Note**
>
> UNESCAPED returns an error for fully qualified names (names that include both a table and column/measure component), because the result is ambiguous without delimiters.

Special character escaping rules

The following escaping rules apply within DAX name delimiters:

- **Table names (single-quote delimited):** A literal single quote (') in a table name is escaped as two single quotes ('').
- **Column and measure names (bracket delimited):** A literal closing bracket (]) in a column or measure name is escaped as]]. An opening bracket ([) doesn't require escaping.

Expand table

Fully escaped name	ESCAPED	UNESCAPED	MINIMALLYESCAPED
'Ta''''ble' (table with ' in name)	'Ta''''ble'	Ta''ble	Ta''ble
[colu[]]mn] (column with] in name)	[colu[]]mn]	colu[]mn	[colu[]]mn]

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

Example 1

The following DAX query returns the fully qualified name of a column:

DAX

```
EVALUATE
{ NAMEOF ( 'Sales'[ORDER QUANTITY] ) }
```

Returns:

Expand table

[Value]
'Sales'[Order Quantity]

Example 2

The following DAX query returns the fully qualified name of a measure:

DAX

```
DEFINE
    MEASURE Sales[Projected Sales] =
        SUM ( 'Sales'[Sales Amount] ) * 1.06

EVALUATE
{ NAMEOF ( [Projected Sales] ) }
```

Returns:

Expand table

[Value]
'Sales'[Projected Sales]

Example 3

The following DAX query uses the `component` parameter to extract only the table name from a column reference:

DAX

```
EVALUATE
{ NAMEOF ( 'Sales'[Sales Amount], TABLE ) }
```

Returns:

Expand table

[Value]
'Sales'

Example 4

The following DAX query returns an unescaped table name:

```
DAX
EVALUATE
{ NAMEOF ( 'Sales', FULL, UNESCAPED ) }
```

Returns:

Expand table

[Value]
Sales

Example 5

The following DAX query uses the `component` and `escaped` parameters to return the minimally escaped parent table name of a column:

```
DAX
EVALUATE
{ NAMEOF ( 'Sales'[Sales Amount], PARENT, MINIMALLYESCAPED ) }
```

Returns:

Expand table

[Value]
Sales

Related content

- Information functions

Last updated on 08/04/2026

NONVISUAL

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Marks a value filter in a SUMMARIZECOLUMNS expression as non-visual. This function can only be used within a SUMMARIZECOLUMNS expression.

Syntax

```DAX
NONVISUAL(<expression>)
```

Parameters

Expand table

Term	Definition
expression	Any DAX expression that returns a single value (not a table).

Return value

A table of values.

Remarks

- Marks a value filter in SUMMARIZECOLUMNS as not affecting measure values, but only applying to group-by columns.
- This function can only be used within a SUMMARIZECOLUMNS expression. It's used as either a filterTable argument of the SUMMARIZECOLUMNS function or a groupLevelFilter argument of the ROLLUPADDISSUBTOTAL or ROLLUPISSUBTOTAL function.

Example

See SUMMARIZECOLUMNS.

Feedback

Was this page helpful? Yes No

SELECTEDMEASURE

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Used by expressions for calculation items or dynamic format strings to reference the measure that is in context.

Syntax

```DAX
SELECTEDMEASURE()
```

Parameters

None

Return value

A reference to the measure that is currently in context when the calculation item or format string is evaluated.

Remarks

- Can only be referenced in the expression for a calculation item or format string.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following calculation item expression calculates the year-to-date for whatever the measure is in context.

```DAX
CALCULATE(SELECTEDMEASURE(), DATESYTD(DimDate[Date]))
```

The following expression can be used to dynamically adjust the format string of a measure based upon whether a value is the hundreds, thousands, or millions.

DAX

```
SWITCH(
TRUE(),
SELECTEDMEASURE() < 1000,"$#,##0",            //Values less than 1000 have
no text after them
SELECTEDMEASURE() < 1000000, "$#,##0,.0 K",   //Values between 1000 and
1000000 are formatted as #.## K
"$#,##0,,.0 M"                                //Values greater than 1000000
are formatted as #.## M
)
```

Related content

SELECTEDMEASURENAME
ISSELECTEDMEASURE

Feedback

Was this page helpful? Yes No

SELECTEDMEASUREFORMATSTRING

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Used by expressions for calculation items to retrieve the format string of the measure that is in context.

Syntax

```
DAX
SELECTEDMEASUREFORMATSTRING()
```

Parameters

None

Return value

A string holding the format string of the measure that is currently in context when the calculation item is evaluated.

Remarks

- This function can only be referenced in expressions for calculation items in calculation groups. It is designed to be used by the **Format String Expression** property of calculation items.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following expression is evaluated by the Format String Expression property for a calculation item. If there is a single currency in filter context, the format string is

retrieved from the DimCurrency[FormatString] column; otherwise the format string of the measure in context is used.

```DAX
SELECTEDVALUE( DimCurrency[FormatString], SELECTEDMEASUREFORMATSTRING() )
```

Related content

SELECTEDMEASURE
ISSELECTEDMEASURE

Feedback

Was this page helpful? Yes No

SELECTEDMEASURENAME

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Used by expressions for calculation items to determine the measure that is in context by name.

Syntax

```DAX
SELECTEDMEASURENAME()
```

Parameters

None

Return value

A string value holding the name of the measure that is currently in context when the calculation item is evaluated.

Remarks

- Can only be referenced in the expression for a calculation item.
- This function is often used for debugging purposes when authoring calculation groups.

Example

The following calculation item expression checks if the current measure is Expense Ratio and conditionally applies calculation logic. Since the check is based on a string comparison, it is not subject to formula fixup and will not benefit from object renaming being automatically reflected. For a similar comparison that would benefit from formula fixup, please see the ISSLECTEDMEASURE function instead.

DAX

```
IF (
    SELECTEDMEASURENAME = "Expense Ratio",
    SELECTEDMEASURE (),
    DIVIDE ( SELECTEDMEASURE (), COUNTROWS ( DimDate ) )
)
```

Related content

SELECTEDMEASURE

ISSELECTEDMEASURE

Feedback

Was this page helpful? Yes No

TABLEOF

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns a reference to the table associated with a specified column, measure, or calendar.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

Syntax

DAX

```
TABLEOF ( <myColumnRef> )
TABLEOF ( <measureName> )
TABLEOF ( <myCalendar> )
```

Parameters

Expand table

Term	Definition
`reference`	A column, measure, or calendar reference.

Return value

A table reference.

Remarks

- The `TABLEOF` function returns a table reference, not the table data itself.
- When you pass a column name, it returns the table that contains that column.
- When you pass a measure name, it returns the table where that measure is defined.
- When you pass a calendar reference, it returns the table associated with that calendar.
- This function is useful in scenarios where you need to dynamically determine which table a column or measure belongs to.
- `TABLEOF` doesn't resolve columns from row context; it only resolves columns from the current filter context (base table).

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1 - Using TABLEOF with a column

DAX

```
EVALUATE
ROW ( "RowCount", COUNTROWS ( TABLEOF ( 'Customer'[Customer ID] ) ) )
```

Returns:

Expand table

RowCount
18485

Example 2 - Using TABLEOF with a measure

DAX

```
DEFINE
    MEASURE Sales[Projected Sales] =
        SUM ( 'Sales'[Sales Amount] ) * 1.06

EVALUATE
ROW (
    "Total Projected Sales", ROUND ( SUMX ( TABLEOF ( [Projected Sales] ), [Projected Sales] ), 2 )
)
```

Returns:

Expand table

Total Projected Sales
116397830.65

Example 3 - Using TABLEOF in a user-defined function

DAX

```
DEFINE
    FUNCTION GetTableRowCount = (
            columnRef : ANYREF
        ) =>
        COUNTROWS ( TABLEOF ( columnRef ) )

EVALUATE
ROW (
    "ResellerCount", GetTableRowCount ( 'Reseller'[Reseller ID] ),
    "CustomerCount", GetTableRowCount ( 'Customer'[Customer ID] )
)
```

Returns:

Expand table

ResellerCount	CustomerCount
702	18485

Related content

- Information functions

Last updated on 08/04/2026

TEXTCONTAINS

Applies to: Calculated column Calculated table Measure Visual calculation

Returns `TRUE` if text in a column matches a search string according to the specified full-text search mode. Otherwise, it returns `FALSE`.

Syntax

DAX

```
TEXTCONTAINS(<within_text_column>, <search_text>[, <match_mode>])
```

Parameters

Expand table

Term	Definition
`within_text_column`	A column that contains the text values to search. The column must have the String data type.
`search_text`	A text string that defines the search condition.
`match_mode`	(Optional) The matching behavior to use. Supported values are `TEXTMATCHING`, `FUZZYMATCHING`, and `PHRASEMATCHING`. The default is `TEXTMATCHING`.

Return value

`TRUE` if the text in `within_text_column` matches `search_text` according to `match_mode`; otherwise, `FALSE`.

Match modes

Expand table

Mode	Behavior
`TEXTMATCHING`	Tokenizes `search_text` and matches text that contains any of the search terms.
`FUZZYMATCHING`	Matches tokens that are similar to search tokens within a small edit distance, which

Mode	Behavior
	supports typo-tolerant matching.
`PHRASEMATCHING`	Matches analyzed tokens that appear next to each other and in the specified order. Punctuation doesn't affect the token sequence.

Remarks

- The search is case-insensitive.
- The function applies language-specific stemming and stop-word removal based on the model culture. For example, stemming allows a search for `run` to match `running`, `runs`, and `runner`. Stop-word removal ignores common filler words like `the`, `and`, `of`, and `in`.
- Per-column collation doesn't control the analyzer language. The model culture controls it. In Power BI Desktop, select **File** > **Options and settings** > **Options** > **Current file** > **Regional settings** > **Model language** to set the model language. For more information, see FORMAT.
- The supported model cultures are:
  - English (`en`)
  - French (`fr`)
  - German (`de`)
  - Spanish (`es`)
  - Portuguese (`pt`)
  - Russian (`ru`)
  - Arabic (`ar`)
  - Turkish (`tr`)
  - Danish (`da`)
  - Dutch (`nl`)
  - Finnish (`fi`)
  - Greek (`el`)
  - Hungarian (`hu`)
  - Italian (`it`)
  - Norwegian (`no`, `nb`, and `nn`)
  - Romanian (`ro`)
  - Swedish (`sv`)
  - Tamil (`ta`)
- For unsupported cultures, `TEXTCONTAINS` uses a generic lowercase tokenizer without stemming or stop-word removal.
- Accent folding isn't supported. For example, `café` doesn't match `cafe`.

- The function requires a persisted full-text index on `within_text_column`. It's supported for Import, Dual, and Direct Lake storage modes. Other storage modes return an error.
- The function is evaluated row by row. Before using it in a calculated column over a very large table, test the performance with a representative workload.
- `TEXTCONTAINS` returns a Boolean value and isn't intended for relevance ranking. Use TEXTSIMILARITY to rank matches.

Examples

Match any search term

The following query returns products whose descriptions contain a token that matches either `wireless` or `bluetooth`:

DAX

```
EVALUATE
FILTER (
    Products,
    TEXTCONTAINS ( Products[Description], "wireless bluetooth" )
)
```

Expand table

Example text	Result
`Wireless headphones with Bluetooth 5.0`	`TRUE`
`Wireless noise-cancelling headphones`	`TRUE`
`Wired headphones`	`FALSE`

Match all search terms

Combine separate calls with the `&&` operator to require all terms:

DAX

```
EVALUATE
FILTER (
    Reviews,
    TEXTCONTAINS ( Reviews[Comment], "battery" )
        && TEXTCONTAINS ( Reviews[Comment], "life" )
)
```

Expand table

Example text	Result
`Battery life is amazing`	`TRUE`
`Good life from this battery`	`TRUE`
`Battery charges fast`	`FALSE`

Find a fuzzy match

The following query uses typo-tolerant matching to find comments that contain a token similar to `headphone`:

DAX

```
EVALUATE
FILTER (
    Reviews,
    TEXTCONTAINS ( Reviews[Comment], "headphone", FUZZYMATCHING )
)
```

Expand table

Example text	Result
`These headphones are great`	`TRUE`
`Best headfone I own`	`TRUE`
`Excellent earbuds`	`FALSE`

Find a phrase match

The following query finds descriptions in which `noise` and `cancel` appear next to each other and in that order:

DAX

```
EVALUATE
FILTER (
    Products,
    TEXTCONTAINS ( Products[Description], "noise cancel", PHRASEMATCHING )
)
```

Expand table

Example text	Result
`Noise cancel headphones pro`	`TRUE`
`Cancel noise adapter`	`FALSE`

Related content

- CONTAINSSTRING
- TEXTSIMILARITY
- Information functions

Last updated on 09/29/2026

TEXTSIMILARITY

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns a floating-point relevance score that indicates how strongly text in a column matches a search string.

Syntax

DAX

```
TEXTSIMILARITY(<within_text_column>, <search_text>[, <match_mode>])
```

Parameters

Expand table

Term	Definition
`within_text_column`	A column that contains the text values to search. The column must have the String data type.
`search_text`	A text string that defines the search condition.
`match_mode`	(Optional) The matching behavior to use. Supported values are `TEXTMATCHING`, `FUZZYMATCHING`, and `PHRASEMATCHING`. The default is `TEXTMATCHING`.

Return value

A floating-point relevance score. Higher scores indicate stronger relevance, and 0 indicates no match.

Compare scores only among results from the same search against the same column. Scores aren't comparable across different search strings or columns.

Match modes

Expand table

Mode	Behavior
`TEXTMATCHING`	Tokenizes `search_text` and scores text that matches any search token.
`FUZZYMATCHING`	Scores tokens that are similar to search tokens within a small edit distance. This mode supports typo-tolerant matching.
`PHRASEMATCHING`	Scores text when the analyzed tokens appear next to each other and in the specified order. Punctuation doesn't affect the token sequence.

Remarks

- The search is case-insensitive.
- The function applies language-specific stemming and stop-word removal based on the model culture. For example, stemming allows a search for `run` to match `running`, `runs`, and `runner`. Stop-word removal ignores common filler words like `the`, `and`, `of`, and `in`.
- Per-column collation doesn't control the analyzer language. The model culture controls it. In Power BI Desktop, select **File** > **Options and settings** > **Options** > **Current file** > **Regional settings** > **Model language** to set the model language. For more information, see FORMAT.
- The supported model cultures are:
  - English (`en`)
  - French (`fr`)
  - German (`de`)
  - Spanish (`es`)
  - Portuguese (`pt`)
  - Russian (`ru`)
  - Arabic (`ar`)
  - Turkish (`tr`)
  - Danish (`da`)
  - Dutch (`nl`)
  - Finnish (`fi`)
  - Greek (`el`)
  - Hungarian (`hu`)
  - Italian (`it`)
  - Norwegian (`no`, `nb`, and `nn`)
  - Romanian (`ro`)
  - Swedish (`sv`)
  - Tamil (`ta`)

- For unsupported cultures, `TEXTSIMILARITY` uses a generic lowercase tokenizer without stemming or stop-word removal.
- Accent folding isn't supported. For example, `café` doesn't match `cafe`.
- The function requires a persisted full-text index on `within_text_column`. It's supported for Import, Dual, and Direct Lake storage modes. Other storage modes return an error.
- The function is evaluated row by row. Before using it in a calculated column over a very large table, test the performance with a representative workload.
- Use TEXTCONTAINS when you need a Boolean match result instead of a relevance score.

Examples

Return the most relevant rows

The following query returns the five reviews with the highest relevance scores for `great refrigerator`:

DAX

```
EVALUATE
TOPN (
    5,
    FactReviews,
    TEXTSIMILARITY ( FactReviews[Text], "great refrigerator" ), DESC
)
```

Expand table

Example text	Score
`This is a great refrigerator, keeps everything cold and quiet.`	3.42
`Fantastic fridge! Great cooling performance and very spacious.`	2.98
`Our old refrigerator broke and this new one works great.`	2.54
`Refrigerator is decent, not great but gets the job done.`	1.72
`The refrigerator section is large, freezer is okay.`	1.10

Filter and rank rows

The following query restricts the result to reviews that match `refrigerator`, and then ranks them by their relevance to `great refrigerator`:

DAX

```
EVALUATE
TOPN (
    5,
    FILTER (
        FactReviews,
        TEXTCONTAINS ( FactReviews[Text], "refrigerator" )
    ),
    TEXTSIMILARITY ( FactReviews[Text], "great refrigerator" ), DESC
)
```

`TEXTCONTAINS` and `TEXTSIMILARITY` execute independently. Filtering rows with `TEXTCONTAINS` doesn't change the index scope used to calculate `TEXTSIMILARITY` scores.

Expand table

Example text	Score
`This is a great refrigerator - quiet, spacious, and keeps food fresh.`	3.42
`Fantastic fridge. Cooling performance is excellent and build quality is great.`	3.01
`Our old refrigerator died, but this new one works great and runs silently.`	2.67
`The refrigerator section is huge. Would be great if the freezer were larger.`	1.94
`Solid refrigerator. Not the best, but overall a great value for the price.`	1.65

Display relevance scores

The following query adds the relevance score for each product description to the result:

DAX

```
EVALUATE
ADDCOLUMNS (
    Products,
    "Relevance score", TEXTSIMILARITY ( Products[Description], "wireless bluetooth" )
)
```

Expand table

Example text	Score
`Wireless headphones with Bluetooth`	2.35

Example text	Score
`Wired headphones`	0

Related content

- TEXTCONTAINS
- Information functions

Last updated on 09/29/2026

USERCULTURE

Article • 01/06/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the locale (language code-country code) for the current user, determined by the operating system, browser settings, or Power BI service.

Syntax

```DAX
USERCULTURE()
```

Parameters

This expression has no parameters.

Return value

Locale as a string.

Remarks

- In the Power BI service, locale is determined by **Settings** > **Language** > **Language Settings**. The default is determined by the user's browser language setting.
- When used in calculated table and calculated column expressions, the result may differ depending on whether the table is in DirectQuery or Import mode. When in DirectQuery mode, the result is determined by the language (locale) specified in Language Settings in the Power BI service. The default in Language Settings specifies locale is determined by the user's browser language setting, which means the same calculated table or column can return different results depending on the browser language settings for each user. When in Import mode, the result is statically determined during refresh and will not vary at query time. For managed refreshes, such as scheduled or interactive, locale is not based on the user's browser language setting but instead uses an invariant locale. The invariant locale,

however, can be overridden by using the XMLA endpoint to specify a custom locale.

- When combined with the Field parameters feature in Power BI, USERCULTURE can be used to reliably translate dynamic visualization titles and captions when used in measure and row-level security (RLS) object expressions within the same model. However, expressions containing USERCULTURE called from outside the model, such as queries and live-connect report measures, should not be relied upon for correctly translated titles and captions.
- USERCULTURE returns the correct user locale when used in object expressions called from within the model such as measures, row-level security (RLS), and calculation items. However, it may not return the correct user locale when used in expressions from outside the model, such as queries and live-connect report measures.
- In Live-connect reports, USERCULTURE may not return the correct user locale when called from a report measure expression.

Example

For the following expression,

DAX

```
FORMAT(TODAY(), "dddd", USERCULTURE())
```

Depending on the language setting for the current user, USERCULTURE returns the current day, for example:

Expand table

Locale	Formatted weekday
de-DE	Dienstag
en-US	Tuesday
es-ES_tradnl	martes
eu-ES	asteartea
it-IT	martedì
nl-NL	dinsdag

Locale	Formatted weekday
pl-PL	wtorek
ro-RO	marți
ru-RU	вторник
uk-UA	вівторок

Related content

Expression-based titles in Power BI USERNAME USERPRINCIPALNAME USEROBJECTID

Feedback

Was this page helpful? Yes No

USERNAME

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the domain name and username from the credentials given to the system at connection time.

Syntax

DAX

```
USERNAME()
```

Parameters

This expression has no parameters.

Return value

The username from the credentials given to the system at connection time

Example

The following formula verifies if the user login is part of the UsersTable.

DAX

```
= IF(CONTAINS(UsersTable,UsersTable[login], USERNAME()), "Allowed", BLANK())
```

Feedback

Was this page helpful? Yes No

USEROBJECTID

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the current user's Object ID from Microsoft Entra ID or security identifier (SID).

Syntax

```
DAX
USEROBJECTID()
```

Parameters

This expression has no parameters.

Return value

The current user's Object ID from Microsoft Entra ID for Power BI or Azure Analysis Services models or SID for SQL Server Analysis Services models.

Feedback

Was this page helpful? Yes No

USERPRINCIPALNAME

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the user principal name.

Syntax

```
DAX
USERPRINCIPALNAME()
```

Parameters

This expression has no parameters.

Return value

The userprincipalname at connection time.

Feedback

Was this page helpful? Yes No

Logical functions

Article • 10/20/2023

Logical functions act upon an expression to return information about the values or sets in the expression. For example, you can use the IF function to check the result of an expression and create conditional results.

In this category

Function	Description
AND	Checks whether both arguments are TRUE, and returns TRUE if both arguments are TRUE.
BITAND	Returns a bitwise 'AND' of two numbers.
BITLSHIFT	Returns a number shifted left by the specified number of bits.
BITOR	Returns a bitwise 'OR' of two numbers.
BITRSHIFT	Returns a number shifted right by the specified number of bits.
BITXOR	Returns a bitwise 'XOR' of two numbers.
COALESCE	Returns the first expression that does not evaluate to BLANK.
FALSE	Returns the logical value FALSE.
IF	Checks a condition, and returns one value when TRUE, otherwise it returns a second value.
IF.EAGER	Checks a condition, and returns one value when TRUE, otherwise it returns a second value. Uses an *eager* execution plan which always executes the branch expressions regardless of the condition expression.
IFERROR	Evaluates an expression and returns a specified value if the expression returns an error
NOT	Changes FALSE to TRUE, or TRUE to FALSE.
OR	Checks whether one of the arguments is TRUE to return TRUE.
SWITCH	Evaluates an expression against a list of values and returns one of multiple possible result expressions.
TRUE	Returns the logical value TRUE.

Feedback

Was this page helpful? Yes No

AND

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Checks whether both arguments are TRUE, and returns TRUE if both arguments are TRUE. Otherwise returns false.

Syntax

```DAX
AND(<logical1>,<logical2>)
```

Parameters

Expand table

Term	Definition
logical_1, logical_2	The logical values you want to test.

Return value

Returns true or false depending on the combination of values that you test.

Remarks

The **AND** function in DAX accepts only two (2) arguments. If you need to perform an AND operation on multiple expressions, you can create a series of calculations or, better, use the AND operator (**&&**) to join all of them in a simpler expression.

Example 1

The following formula shows the syntax of the AND function.

```DAX
```

```
= IF(AND(10 > 9, -10 < -1), "All true", "One or more false"
```

Because both conditions, passed as arguments, to the AND function are true, the formula returns "All True".

Example 2

The following sample uses the AND function with nested formulas to compare two sets of calculations at the same time. For each product category, the formula determines if the current year sales and previous year sales of the Internet channel are larger than the Reseller channel for the same periods. If both conditions are true, for each category the formula returns the value, "Internet hit".

DAX

```
= IF( AND(  SUM( 'InternetSales_USD'[SalesAmount_USD])
           >SUM('ResellerSales_USD'[SalesAmount_USD])
          , CALCULATE(SUM('InternetSales_USD'[SalesAmount_USD]),
PREVIOUSYEAR('DateTime'[DateKey] ))
           >CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
PREVIOUSYEAR('DateTime'[DateKey] ))
          )
     , "Internet Hit"
     , ""
     )
```

Returns

Expand table

Row Labels	2005	2006	2007	2008	-	Grand Total
Bib-Shorts						
Bike Racks						
Bike Stands				Internet Hit		
Bottles and Cages				Internet Hit		
Bottom Brackets						
Brakes						
Caps						

Row Labels	2005	2006	2007	2008	-	Grand Total
Chains						
Cleaners						
Cranksets						
Derailleurs						
Fenders				Internet Hit		
Forks						
Gloves						
Handlebars						
Headsets						
Helmets						
Hydration Packs						
Jerseys						
Lights						
Locks						
Mountain Bikes						
Mountain Frames						
Panniers						
Pedals						
Pumps						
Road Bikes						
Road Frames						
Saddles						
Shorts						
Socks						
Tights						
Tires and Tubes				Internet Hit		

Row Labels	2005	2006	2007	2008	-	Grand Total
Touring Bikes						
Touring Frames						
Vests						
Wheels						
Grand Total						

Related content

Logical functions

Feedback

Was this page helpful? Yes No

BITAND

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a bitwise AND of two numbers.

Syntax

```DAX
BITAND(<number>, <number>)
```

Parameters

Expand table

Term	Definition
Number	Any scalar expression that returns number. If not an integer, it is truncated.

Return value

A bitwise AND of two numbers.

Remarks

- This function supports both positive and negative numbers.

Example

The following DAX query:

```DAX
EVALUATE { BITAND(13, 11) }
```

Returns 9.

Related content

BITLSHIFT
BITRSHIFT
BITOR
BITXOR

Feedback

Was this page helpful? Yes No

BITLSHIFT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a number shifted left by the specified number of bits.

Syntax

```
DAX
BITLSHIFT(<Number>, <Shift_Amount>)
```

Parameters

Expand table

Term	Definition
Number	Any DAX expression that returns an integer expression.
Shift_Amount	Any DAX expression that returns an integer expression.

Return value

An integer value.

Remarks

- Be sure to understand the nature of bitshift operations and overflow/underflow of integers before using DAX bitshift functions.
- If Shift_Amount is negative, it will shift in the opposite direction.
- If absolute value of Shift_Amount is larger than 64, there will be no error but will result in overflow/underflow.
- There's no limit on Number, but the result may overflow/underflow.

Examples

Example 1

The following DAX query:

```
DAX

EVALUATE
    { BITLSHIFT(2, 3) }
```

Returns 16.

Example 2

The following DAX query:

```
DAX

EVALUATE
    { BITLSHIFT(128, -1) }
```

Returns 64.

Example 3

The following DAX query:

```
DAX

Define
    Measure Sales[LeftShift] = BITLSHIFT(SELECTEDVALUE(Sales[Amount]), 3)

EVALUATE
SUMMARIZECOLUMNS(
    Sales[Amount],
    "LEFTSHIFT",
    [LeftShift]
)
```

Shifts left each sales amount with 3 bits and returns the bit-shifted sales amount.

Related content

BITRSHIFT
BITAND

BITOR
BITXOR

Feedback

Was this page helpful? Yes No

BITOR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a bitwise OR of two numbers.

Syntax

```DAX
BITOR(<number>, <number>)
```

Parameters

Expand table

Term	Definition
Number	Any scalar expression that returns number. If not an integer, it is truncated.

Return value

A bitwise OR of two numbers.

Remarks

- This function supports both positive and negative numbers.

Example

The following DAX query:

```DAX
EVALUATE
    { BITOR(9, 10) }
```

Returns 11.

Related content

BITAND
BITXOR
BITLSHIFT
BITRSHIFT

Feedback

Was this page helpful? Yes No

BITRSHIFT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a number shifted right by the specified number of bits.

Syntax

```
DAX
BITRSHIFT(<Number>, <Shift_Amount>)
```

Parameters

Expand table

Term	Definition
Number	Any DAX expression that returns an integer expression.
Shift_Amount	Any DAX expression that returns an integer expression.

Return value

An integer value.

Remarks

- Be sure to understand the nature of bitshift operations and overflow/underflow of integers before using DAX bitshift functions.
- If Shift_Amount is negative, it will shift in the opposite direction.
- If absolute value of Shift_Amount is larger than 64, there will be no error but will result in overflow/underflow.
- There's no limit on Number, but the result may overflow/underflow.

Examples

Example 1

The following DAX query:

```
DAX

EVALUATE
    { BITRSHIFT(16, 3) }
```

Returns 2.

Example 2

The following DAX query:

```
DAX

EVALUATE
    { BITRSHIFT(1024, -3) }
```

Returns 8192.

Example 3

The following DAX query:

```
DAX

Define
    Measure Sales[RightShift] = BITRSHIFT(SELECTEDVALUE(Sales[Amount]), 3)

EVALUATE
SUMMARIZECOLUMNS(
    Sales[Amount],
    "RIGHTSHIFT",
    [RightShift]
)
```

Shifts right each sales amount with 3 bits and returns the bit-shifted sales amount.

Related content

BITLSHIFT
BITAND

BITOR
BITXOR

Feedback

Was this page helpful? Yes No

BITXOR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a bitwise XOR of two numbers.

Syntax

```
DAX
BITXOR(<number>, <number>)
```

Parameters

Expand table

Term	Definition
Number	Any scalar expression that returns number. If not an integer, it is truncated.

Return value

A bitwise XOR of two numbers.

Remarks

- This function supports both positive and negative numbers.

Example

The following DAX query:

```
DAX
EVALUATE { BITXOR(9, 10) }
```

Returns 3.

Related content

BITOR
BITAND
BITLSHIFT
BITRSHIFT

Feedback

Was this page helpful? Yes No

COALESCE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the first expression that does not evaluate to BLANK. If all expressions evaluate to BLANK, BLANK is returned.

Syntax

DAX

```
COALESCE(<expression>, <expression>[, <expression>]…)
```

Parameters

Expand table

Term	Definition
expression	Any DAX expression that returns a scalar expression.

Return value

A scalar value coming from one of the expressions or BLANK if all expressions evaluate to BLANK.

Remarks

Input expressions may be of different data types.

Example 1

The following DAX query:

DAX

```
EVALUATE { COALESCE(BLANK(), 10, DATE(2008, 3, 3)) }
```

Returns `10`, which is the first expression that does not evaluate to BLANK.

Example 2

The following DAX expression:

```
DAX

= COALESCE(SUM(FactInternetSales[SalesAmount]), 0)
```

Returns the sum of all values in the SalesAmount column in the FactInternetSales table, or `0`. This can be used to convert BLANK values of total sales to `0`.

Feedback

Was this page helpful? Yes No

FALSE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the logical value FALSE.

Syntax

DAX

```
FALSE()
```

Return value

Always FALSE.

Remarks

The word FALSE is also interpreted as the logical value FALSE.

Example

The formula returns the logical value FALSE when the value in the column, 'InternetSales_USD'[SalesAmount_USD], is less than or equal to 200000.

DAX

```
= IF(SUM('InternetSales_USD'[SalesAmount_USD]) >200000, TRUE(), false())
```

The following table shows the results when the example formula is used with 'ProductCategory'[ProductCategoryName] in Row Labels and 'DateTime'[CalendarYear] in Column Labels.

Expand table

Row Labels	2005	2006	2007	2008	-	Grand Total
Accessories	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
Bikes	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE
Clothing	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
Components	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
Grand Total	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE

Related content

TRUE function
NOT function
IF function

Feedback

Was this page helpful? Yes No

IF

Applies to: Calculated column Calculated table Measure Visual calculation

Checks a condition, and returns one value when it's `TRUE`, otherwise it returns a second value.

Syntax

```
DAX
IF(<logical_test>, <value_if_true>[, <value_if_false>])
```

Parameters

Expand table

Term	Definition
`logical_test`	Any value or expression that can be evaluated to `TRUE` or `FALSE`.
`value_if_true`	The value that's returned if the logical test is `TRUE`.
`value_if_false`	(Optional) The value that's returned if the logical test is `FALSE`. If omitted, BLANK is returned.

Return value

Either `value_if_true`, `value_if_false`, or `BLANK`.

Remarks

- The IF function can return a variant data type if `value_if_true` and `value_if_false` are of different data types, but the function attempts to return a single data type if both `value_if_true` and `value_if_false` are of numeric data types. In the latter case, the IF function implicitly converts data types to accommodate both values.

  For example, the formula `IF(<condition>, TRUE(), 0)` returns `TRUE` or 0, but the formula `IF(<condition>, 1.0, 0)` returns only decimal values even though `value_if_false` is of the whole number data type. To learn more about implicit data type conversion, see Data types.

- To execute the branch expressions regardless of the condition expression, use IF.EAGER instead.

Examples

The following **Product** table calculated column definitions use the IF function in different ways to classify each product based on its list price.

The first example tests whether the **List Price** column value is less than 500. When this condition is true, the function returns **Low**. Because there's no `value_if_false` value, the function returns BLANK.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
Price Group =
IF ( 'Product'[List Price] < 500, "Low" )
```

The second example uses the same test, but this time includes a `value_if_false` value. So, the formula classifies each product as either `Low` or `High`.

DAX

```
Price Group =
IF ( 'Product'[List Price] < 500, "Low", "High" )
```

The third example uses the same test, but this time nests an IF function to perform an additional test. So, the formula classifies each product as either `Low`, `Medium`, or `High`.

DAX

```
Price Group =
IF (
    'Product'[List Price] < 500,
    "Low",
    IF ( 'Product'[List Price] < 1500, "Medium", "High" )
)
```

Tip

When you need to nest multiple IF functions, the **SWITCH** function might be a better option. This function provides a more elegant way to write an expression that returns more than two possible values.

Related content

- IF.EAGER function
- SWITCH function (DAX)
- Logical functions

Last updated on 08/04/2026

IF.EAGER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Checks a condition, and returns one value when TRUE, otherwise it returns a second value. It uses an *eager* execution plan which always executes the branch expressions regardless of the condition expression.

Syntax

DAX

```
IF.EAGER(<logical_test>, <value_if_true>[, <value_if_false>])
```

Parameters

Expand table

Term	Definition
logical_test	Any value or expression that can be evaluated to TRUE or FALSE.
value_if_true	The value that's returned if the logical test is TRUE.
value_if_false	(Optional) The value that's returned if the logical test is FALSE. If omitted, BLANK is returned.

Return value

Either **value_if_true**, **value_if_false**, or BLANK.

Remarks

- The IF.EAGER function can return a variant data type if value_if_true and value_if_false are of different data types, but the function attempts to return a single data type if both **value_if_true** and **value_if_false** are of numeric data types. In the latter case, the IF.EAGER function will implicitly convert data types to accommodate both values.

For example, the formula `IF.EAGER(<condition>, TRUE(), 0)` returns TRUE or 0, but the formula `IF.EAGER(<condition>, 1.0, 0)` returns only decimal values even though **value_if_false** is of the whole number data type. To learn more about implicit data type conversion, see Data types.

- IF.EAGER has the same functional behavior as the IF function, but performance may differ due to differences in execution plans. `IF.EAGER(<logical_test>, <value_if_true>, <value_if_false>)` has the same execution plan as the following DAX expression:

  DAX

  ```
  VAR _value_if_true = <value_if_true>
  VAR _value_if_false = <value_if_false>
  RETURN
  IF (<logical_test>, _value_if_true, _value_if_false)
  ```

  Note: The two branch expressions are evaluated regardless of the condition expression.

Examples

See IF Examples.

Related content

IF function
Logical functions

Feedback

Was this page helpful? Yes No

IFERROR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Evaluates an expression and returns a specified value if the expression returns an error; otherwise returns the value of the expression itself.

Syntax

```
DAX
IFERROR(value, value_if_error)
```

Parameters

Expand table

Term	Definition
value	Any value or expression.
value_if_error	Any value or expression.

Return value

A scalar of the same type as **value**

Remarks

- You can use the IFERROR function to trap and handle errors in an expression.
- If **value** or **value_if_error** is an empty cell, IFERROR treats it as an empty string value ("").
- The IFERROR function is based on the IF function, and uses the same error messages, but has fewer arguments. The relationship between the IFERROR function and the IF function as follows:

```
IFERROR(A,B) := IF(ISERROR(A), B, A)
```

Values that are returned for A and B must be of the same data type; therefore, the column or expression used for **value** and the value returned for **value_if_error** must be the same data type.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- For best practices when using IFERROR, see Appropriate use of error functions.

Example

The following example returns 9999 if the expression 25/0 evaluates to an error. If the expression returns a value other than error, that value is passed to the invoking expression.

DAX

```
= IFERROR(25/0,9999)
```

Related content

Logical functions

Feedback

Was this page helpful? Yes No

NOT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Changes FALSE to TRUE, or TRUE to FALSE.

Syntax

```
DAX
NOT(<logical>)
```

Parameters

Expand table

Term	Definition
logical	A value or expression that can be evaluated to TRUE or FALSE.

Return value

TRUE OR FALSE.

Example

The following example retrieves values from the calculated column that was created to illustrate the IF function. For that example, the calculated column was named using the default name, **Calculated Column1**, and contains the following formula: `= IF([Orders]<300,"true","false")`

The formula checks the value in the column, [Orders], and returns "true" if the number of orders is under 300.

Now create a new calculated column, **Calculated Column2**, and type the following formula.

```
DAX
```

```
= NOT([CalculatedColumn1])
```

For each row in **Calculated Column1**, the values "true" and "false" are interpreted as the logical values TRUE or FALSE, and the NOT function returns the logical opposite of that value.

Related content

TRUE function
FALSE function
IF function

Feedback

Was this page helpful? Yes No

OR

Article • 12/13/2023

Checks whether one of the arguments is TRUE to return TRUE. The function returns FALSE if both arguments are FALSE.

Syntax

```DAX
OR(<logical1>,<logical2>)
```

Parameters

Expand table

Term	Definition
logical_1, logical_2	The logical values you want to test.

Return value

A Boolean value. The value is TRUE if any of the two arguments is TRUE; the value is FALSE if both the arguments are FALSE.

Remarks

- The **OR** function in DAX accepts only two (2) arguments. If you need to perform an OR operation on multiple expressions, you can create a series of calculations or, better, use the OR operator (||) to join all of them in a simpler expression.
- The function evaluates the arguments until the first TRUE argument, then returns TRUE.

Example

The following example shows how to use the OR function to obtain the sales people that belong to the Circle of Excellence. The Circle of Excellence recognizes those who

have achieved more than a million dollars in Touring Bikes sales or sales of over two and a half million dollars in 2007.

```DAX
IF(   OR(   CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
'ProductSubcategory'[ProductSubcategoryName]="Touring Bikes") > 1000000
         ,   CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
'DateTime'[CalendarYear]=2007) > 2500000
         )
   , "Circle of Excellence"
   , ""
   )
```

Returns

Expand table

Row Labels	2005	2006	2007	2008	-	Grand Total
Abbas, Syed E						
Alberts, Amy E						
Ansman-Wolfe, Pamela O						
Blythe, Michael G	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Campbell, David R						
Carson, Jillian	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Ito, Shu K						
Jiang, Stephen Y						
Mensa-Annan, Tete A						

Row Labels	2005	2006	2007	2008	-	Grand Total
Mitchell, Linda C	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Pak, Jae B	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Reiter, Tsvi Michael						
Saraiva, José Edvaldo	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Tsoflias, Lynn N						
Valdez, Rachel B						
Vargas, Garrett R						
Varkey Chudukatil, Ranjit R						Circle of Excellence
Grand Total	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence

Related content

Logical functions

Feedback

Was this page helpful? Yes No

SWITCH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Evaluates an expression against a list of values and returns one of multiple possible result expressions. This function can be used to avoid having multiple nested IF statements.

Syntax

DAX

```
SWITCH(<expression>, <value>, <result>[, <value>, <result>]…[, <else>])
```

Parameters

Expand table

Term	Definition
expression	Any DAX expression that returns a single scalar value where the expression is to be evaluated multiple times (for each row/context).
value	A constant value to be matched with the results of **expression**.
result	Any scalar expression to be evaluated if the results of **expression** match the corresponding **value**.
else	Any scalar expression to be evaluated if the result of **expression** doesn't match any of the **value** arguments.

Return value

If there's a match with a **value**, a scalar value from the corresponding **result** is returned. If there isn't a match with a **value**, a value from **else** is returned. If none of the **values** match and **else** isn't specified, BLANK is returned.

Remarks

- The **expression** to be evaluated can be a constant value or an expression. A common use of this function is to set the first parameter to TRUE. See examples below.
- All **result** expressions and the **else** expression must be of the same data type.
- The order of conditions matters. As soon as one **value** matches, the corresponding **result** is returned, and other subsequent **values** aren't evaluated. Make sure the most restrictive **values** to be evaluated are specified before less restrictive **values**. See examples below.

Examples

A common use of SWITCH is to compare **expression** with constant **values**. The following example creates a calculated column of month names:

DAX

```
= SWITCH (
         [Month Number Of Year],
         1, "January",
         2, "February",
         3, "March",
         4, "April",
         5, "May",
         6, "June",
         7, "July",
         8, "August",
         9, "September",
         10, "October",
         11, "November",
         12, "December",
         "Unknown month number"
        )
```

Another common use of SWITCH is to replace multiple nested IF statements. This is accomplished by setting expression to TRUE, as shown in the following example, which compares Reorder Point and Safety Stock Level on products to identify potential risks of running out of stock:

DAX

```
= SWITCH (
        TRUE,
        [Reorder Point] > [Safety Stock Level], "Good: Safety stock level
exceeded",
        [Reorder Point] = [Safety Stock Level], "Minimal: Safety stock level
met",
        [Reorder Point] < [Safety Stock Level], "At risk: Safety stock level
```

```
not met",
        ISBLANK ( [Reorder Point] ), "Incomplete: Reorder point not set",
        ISBLANK ( [Safety Stock Level] ), "Incomplete: Safety stock level
not set",
        "Unknown"
        )
```

The order of **values** matters. In the following example, the second **result** is never returned because the first value is less restrictive than the second. The result in this example is always "A" or "C", but never "B".

DAX

```
= SWITCH (
         TRUE,
         Product[Standard Cost] < 100, "A",
         Product[Standard Cost] < 10, "B",
         "C"
         )
```

The following statement returns an error because the data types in the **result** arguments are different. Keep in mind that the data types in all **result** and **else** arguments must be the same.

DAX

```
= SWITCH (
        [Class],
        "L", "Large",
        "H", 0.1
        )
```

Feedback

Was this page helpful? Yes No

TRUE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the logical value TRUE.

Syntax

```DAX
TRUE()
```

Return value

Always TRUE.

Remarks

The word TRUE is also interpreted as the logical value TRUE.

Example

The formula returns the logical value TRUE when the value in the column, 'InternetSales_USD'[SalesAmount_USD], is greater than 200000.

```DAX
= IF(SUM('InternetSales_USD'[SalesAmount_USD]) >200000, TRUE(), false())
```

The following table shows the results when the example formula is used in a report with 'ProductCategory'[ProductCategoryName] in Row Labels and 'DateTime'[CalendarYear] in Column Labels.

Expand table

Row Labels	2005	2006	2007	2008	-	Grand Total
Accessories	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
Bikes	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE
Clothing	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
Components	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
Grand Total	TRUE	TRUE	TRUE	TRUE	FALSE	TRUE

Related content

FALSE
NOT
IF

Feedback

Was this page helpful? Yes No

Math and Trig functions

Article • 10/20/2023

The mathematical functions in Data Analysis Expressions (DAX) are very similar to the Excel mathematical and trigonometric functions. This section lists the mathematical functions provided by DAX.

In this category

Function	Description
ABS	Returns the absolute value of a number.
ACOS	Returns the arccosine, or inverse cosine, of a number.
ACOSH	Returns the inverse hyperbolic cosine of a number.
ACOT	Returns the arccotangent, or inverse cotangent, of a number.
ACOTH	Returns the inverse hyperbolic cotangent of a number.
ASIN	Returns the arcsine, or inverse sine, of a number.
ASINH	Returns the inverse hyperbolic sine of a number.
ATAN	Returns the arctangent, or inverse tangent, of a number.
ATANH	Returns the inverse hyperbolic tangent of a number.
CEILING	Rounds a number up, to the nearest integer or to the nearest multiple of significance.
CONVERT	Converts an expression of one data type to another.
COS	Returns the cosine of the given angle.
COSH	Returns the hyperbolic cosine of a number.
COT	Returns the cotangent of an angle specified in radians.
COTH	Returns the hyperbolic cotangent of a hyperbolic angle.
CURRENCY	Evaluates the argument and returns the result as currency data type.
DEGREES	Converts radians into degrees.
DIVIDE	Performs division and returns alternate result or BLANK() on division by 0.
EVEN	Returns number rounded up to the nearest even integer.

Function	Description
EXP	Returns e raised to the power of a given number.
FACT	Returns the factorial of a number, equal to the series 1*2*3*...* , ending in the given number.
FLOOR	Rounds a number down, toward zero, to the nearest multiple of significance.
GCD	Returns the greatest common divisor of two or more integers.
INT	Rounds a number down to the nearest integer.
ISO.CEILING	Rounds a number up, to the nearest integer or to the nearest multiple of significance.
LCM	Returns the least common multiple of integers.
LN	Returns the natural logarithm of a number.
LOG	Returns the logarithm of a number to the base you specify.
LOG10	Returns the base-10 logarithm of a number.
MOD	Returns the remainder after a number is divided by a divisor. The result always has the same sign as the divisor.
MROUND	Returns a number rounded to the desired multiple.
ODD	Returns number rounded up to the nearest odd integer.
PI	Returns the value of Pi, 3.14159265358979, accurate to 15 digits.
POWER	Returns the result of a number raised to a power.
QUOTIENT	Performs division and returns only the integer portion of the division result.
RADIANS	Converts degrees to radians.
RAND	Returns a random number greater than or equal to 0 and less than 1, evenly distributed.
RANDBETWEEN	Returns a random number in the range between two numbers you specify.
ROUND	Rounds a number to the specified number of digits.
ROUNDDOWN	Rounds a number down, toward zero.
ROUNDUP	Rounds a number up, away from 0 (zero).
SIGN	Determines the sign of a number, the result of a calculation, or a value in a column.

Function	Description
SIN	Returns the sine of the given angle.
SINH	Returns the hyperbolic sine of a number.
SQRT	Returns the square root of a number.
SQRTPI	Returns the square root of (number * pi).
TAN	Returns the tangent of the given angle.
TANH	Returns the hyperbolic tangent of a number.
TRUNC	Truncates a number to an integer by removing the decimal, or fractional, part of the number.

Feedback

Was this page helpful? Yes No

ABS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the absolute value of a number.

Syntax

```
DAX
ABS(<number>)
```

Parameters

Expand table

Term	Definition
number	The number for which you want the absolute value.

Return value

A decimal number.

Remarks

The absolute value of a number is a decimal number, whole or decimal, without its sign. You can use the ABS function to ensure that only non-negative numbers are returned from expressions when nested in functions that require a positive number.

Example

The following example returns the absolute value of the difference between the list price and the dealer price, which you might use in a new calculated column, **DealerMarkup**.

```
DAX
```

```
= ABS([DealerPrice]-[ListPrice])
```

Related content

Math and Trig functions
SIGN function

Feedback

Was this page helpful? Yes No

ACOS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the arccosine, or inverse cosine, of a number. The arccosine is the angle whose cosine is *number*. The returned angle is given in radians in the range 0 (zero) to pi.

Syntax

DAX

```
ACOS(number)
```

Parameters

Expand table

Term	Definition
Number	The cosine of the angle you want and must be from -1 to 1.

Return value

Returns the arccosine, or inverse cosine, of a number.

Remarks

If you want to convert the result from radians to degrees, multiply it by 180/PI() or use the DEGREES function.

Example

Expand table

Formula	Description	Result
= ACOS(-0.5)	Arccosine of -0.5 in radians, 2*pi/3.	2.094395102
= ACOS(-0.5)*180/PI()	Arccosine of -0.5 in degrees.	120

Feedback

Was this page helpful? Yes No

ACOSH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse hyperbolic cosine of a number. The number must be greater than or equal to 1. The inverse hyperbolic cosine is the value whose hyperbolic cosine is *number*, so ACOSH(COSH(number)) equals number.

Syntax

DAX

```
ACOSH(number)
```

Parameters

Expand table

Term	Definition
number	Any real number equal to or greater than 1.

Return value

Returns the inverse hyperbolic cosine of a number.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= ACOSH(1)	Inverse hyperbolic cosine of 1.	0
= ACOSH(10)	Inverse hyperbolic cosine of 10.	2.993228

Feedback

Was this page helpful? Yes No

ACOT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the principal value of the arccotangent, or inverse cotangent of a number.

Syntax

DAX

```
ACOT(number)
```

Parameters

Expand table

Term	Definition
Number	The cosine of the angle you want. Must be a real number.

Return value

A single decimal value.

Feedback

Was this page helpful? Yes No

ACOTH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse hyperbolic cotangent of a number.

Syntax

DAX

```
ACOTH(number)
```

Parameters

Expand table

Term	Definition
Number	The absolute value of Number must be greater than 1.

Return value

A single decimal value.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

ASIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the arcsine, or inverse sine, of a number. The arcsine is the angle whose sine is *number*. The returned angle is given in radians in the range -pi/2 to pi/2.

Syntax

DAX

```
ASIN(number)
```

Parameters

Expand table

Term	Definition
number	The sine of the angle you want and must be from -1 to 1.

Return value

Returns the arcsine, or inverse sine, of a number.

Remarks

To express the arcsine in degrees, multiply the result by 180/PI( ) or use the DEGREES function.

Example

Expand table

Formula	Description	Result
= ASIN(-0.5)	Arcsine of -0.5 in radians, -pi/6	-0.523598776
= ASIN(-0.5)*180/PI()	Arcsine of -0.5 in degrees	-30
= DEGREES(ASIN(-0.5))	Arcsine of -0.5 in degrees	-30

Feedback

Was this page helpful? Yes No

ASINH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse hyperbolic sine of a number. The inverse hyperbolic sine is the value whose hyperbolic sine is *number*, so ASINH(SINH(number)) equals *number*.

Syntax

```DAX
ASINH(number)
```

Parameters

Expand table

Term	Definition
number	Any real number.

Return value

Returns the inverse hyperbolic sine of a number.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= ASINH(-2.5)	Inverse hyperbolic sine of -2.5	-1.647231146
= ASINH(10)	Inverse hyperbolic sine of 10	2.99822295

Feedback

Was this page helpful? Yes No

ATAN

06/12/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the arctangent, or inverse tangent, of a number. The arctangent is the angle whose tangent is `number`. The returned angle is given in radians in the range -pi/2 to pi/2.

Syntax

```DAX
ATAN(number)
```

Parameters

Expand table

Term	Definition
`number`	The tangent of the angle you want.

Return value

Returns the inverse tangent of a number.

Remarks

To express the arctangent in degrees, multiply the result by 180/PI( ) or use the DEGREES function.

Example

Expand table

Formula	Description	Result
`= ATAN(1)`	Arctangent of 1 in radians, pi/4	0.785398163

Formula	Description	Result
`= ATAN(1)*180/PI()`	Arctangent of 1 in degrees	45

ATANH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse hyperbolic tangent of a number. Number must be between -1 and 1 (excluding -1 and 1). The inverse hyperbolic tangent is the value whose hyperbolic tangent is *number*, so ATANH(TANH(number)) equals *number*.

Syntax

```DAX
ATANH(number)
```

Parameters

Expand table

Term	Definition
number	Any real number between 1 and -1.

Return value

Returns the inverse hyperbolic tangent of a number.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= ATANH(0.76159416)	Inverse hyperbolic tangent of 0.76159416	1.00000001
= ATANH(-0.1)		-0.100335348

Related content

ATAN function

Feedback

Was this page helpful? Yes No

CEILING

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number up, to the nearest integer or to the nearest multiple of significance.

Syntax

```
DAX
CEILING(<number>, <significance>)
```

Parameters

Expand table

Term	Definition
number	The number you want to round, or a reference to a column that contains numbers.
significance	The multiple of significance to which you want to round. For example, to round to the nearest integer, type 1.

Return value

A number rounded as specified.

Remarks

- There are two CEILING functions in DAX, with the following differences:
  - The CEILING function emulates the behavior of the CEILING function in Excel.
  - The ISO.CEILING function follows the ISO-defined behavior for determining the ceiling value.
- The two functions return the same value for positive numbers, but different values for negative numbers. When using a positive multiple of significance, both CEILING and ISO.CEILING round negative numbers upward (toward positive infinity). When using a negative multiple of significance, CEILING rounds negative numbers

downward (toward negative infinity), while ISO.CEILING rounds negative numbers upward (toward positive infinity).

- The return type is usually of the same type of the significant argument, with the following exceptions:
  - If the number argument type is currency, the return type is currency.
  - If the significance argument type is Boolean, the return type is integer.
  - If the significance argument type is non-numeric, the return type is real.

Example 1

The following formula returns 4.45. This might be useful if you want to avoid using smaller units in your pricing. If an existing product is priced at $4.42, you can use CEILING to round prices up to the nearest unit of five cents.

DAX

```
= CEILING(4.42,0.05)
```

Example 2

The following formula returns similar results as the previous example, but uses numeric values stored in the column, **ProductPrice**.

DAX

```
= CEILING([ProductPrice],0.05)
```

Related content

Math and Trig functions
FLOOR function
ISO.CEILING function
ROUNDUP function

Feedback

Was this page helpful? Yes No

CONVERT

Applies to: Calculated column Calculated table Measure Visual calculation

Converts an expression of one data type to another.

Syntax

```
DAX
CONVERT(<Expression>, <Datatype>)
```

Parameters

Expand table

Term	Definition
`Expression`	Any valid expression.
`Datatype`	An enumeration that includes: BOOLEAN/LOGICAL, CURRENCY/DECIMAL, DATETIME, DOUBLE, INTEGER/INT64, STRING/TEXT.

Return value

Returns the value of `Expression`, translated to `Datatype`.

Remarks

- The function returns an error when a value can't be converted to the specified data type.
- DAX calculated columns must be of a single data type. Since MEDIAN and MEDIANX functions over an integer column return mixed data types, either integer or double, the following calculated column expression returns an error:

```
DAX
MedianOrderQuantity =
MEDIAN ( [Order Quantity] )
```

- To avoid mixed data types, change the expression to always return the double data type, for example:

```
DAX
MedianOrderQuantity =
MEDIANX ( 'Sales', CONVERT ( [Order Quantity], DOUBLE ) )
```

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX query

```
DAX
EVALUATE
{ CONVERT ( DATE ( 1900, 1, 1 ), INTEGER ) }
```

Returns

Expand table

[Value]
2

Last updated on 08/04/2026

COS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the cosine of the given angle.

Syntax

DAX

```
COS(number)
```

Parameters

Expand table

Term	Definition
number	Required. The angle in radians for which you want the cosine.

Return value

Returns the cosine of the given angle.

Remarks

If the angle is in degrees, either multiply the angle by PI()/180 or use the RADIANS function to convert the angle to radians.

Example

Expand table

Formula	Description	Result
= COS(1.047)	Cosine of 1.047 radians	0.5001711

Formula	Description	Result
= COS(60*PI()/180)	Cosine of 60 degrees	0.5
= COS(RADIANS(60))	Cosine of 60 degrees	0.5

Feedback

Was this page helpful? Yes No

COSH

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the hyperbolic cosine of a number.

Syntax

```DAX
COSH(number)
```

Parameters

Expand table

Term	Definition
`number`	Required. Any real number for which you want to find the hyperbolic cosine.

Return value

The hyperbolic cosine of a number.

Remarks

- The formula for the hyperbolic cosine is:

$$\text{COSH}(z) = \frac{e^z + e^{-z}}{2}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

DAX expression	Description	Result
`= COSH(4)`	Hyperbolic cosine of 4	27.308233
`= COSH(EXP(1))`	Hyperbolic cosine of the base of the natural logarithm.	7.6101251

Feedback

Was this page helpful? Yes No

COT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the cotangent of an angle specified in radians.

Syntax

DAX

```
COT (<number>)
```

Parameters

Expand table

Term	Definition
number	The angle in radians for which you want the cotangent.

Return value

The cotangent of the given angle.

Remarks

- The absolute value of number must be less than 2^27 and cannot be 0.
- If number is outside its constraints, an error is returned.
- If number is a non-numeric value, an error is returned.

Example

The following DAX query,

DAX

```
EVALUATE { COT(30) }
```

Returns

Expand table

[Value]
-0.156119952161659

Feedback

Was this page helpful? Yes No

COTH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the hyperbolic cotangent of a hyperbolic angle.

Syntax

```
DAX

COTH (<number>)
```

Parameters

Expand table

Term	Definition
number	The hyperbolic angle in radians for which you want the hyperbolic cotangent.

Return value

The hyperbolic cotangent of the given angle.

Remarks

- The hyperbolic cotangent is an analog of the ordinary (circular) cotangent.
- The absolute value of number must be less than 2^{27} and cannot be 0.
- If number is outside its constraints, an error is returned
- If number is a non-numeric value, an error is returned.
- The following equation is used:

$$\mathrm{COTH}(N) = \frac{1}{\mathrm{TANH}(N)} = \frac{\mathrm{COSH(N)}}{\mathrm{SINH(N)}} = \frac{e^N + e^{-N}}{e^N - e^{-N}}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query,

```
DAX

EVALUATE { COTH(2) }
```

Returns

Expand table

[Value]
1.03731472072755

Feedback

Was this page helpful? Yes No

CURRENCY

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Evaluates the argument and returns the result as currency data type.

Syntax

```DAX
CURRENCY(<value>)
```

Parameters

Expand table

Term	Definition
value	Any DAX expression that returns a single scalar value where the expression is to be evaluated exactly once before all other operations.

Return value

The value of the expression evaluated and returned as a currency type value.

Remarks

- The CURRENCY function rounds up the 5th significant decimal, in value, to return the 4th decimal digit. Rounding up occurs if the 5th significant decimal is equal or larger than 5. For example, if value is 3.6666666666666 then converting to currency returns $3.6667. However, if value is 3.0123456789 then converting to currency returns $3.0123.
- If the data type of the expression is TrueFalse then CURRENCY(<TrueFalse>) will return $1.0000 for True values and $0.0000 for False values.
- If the data type of the expression is Text then CURRENCY(<Text>) will try to convert text to a number. If conversion succeeds the number will be converted to

currency, otherwise an error is returned.

- If the data type of the expression is DateTime then CURRENCY(<DateTime>) will convert the datetime value to a number and that number to currency. DateTime values have an integer part that represents the number of days between the given date and 1900-03-01 and a fraction that represents the fraction of a day (where 12 hours or noon is 0.5 day). If the value of the expression is not a proper DateTime value an error is returned.

Example

Convert number 1234.56 to currency data type.

```
DAX
= CURRENCY(1234.56)
```

Returns value 1234.56000.

Feedback

Was this page helpful? Yes No

DEGREES

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts radians into degrees.

Syntax

DAX

```
DEGREES(angle)
```

Parameters

Expand table

Term	Definition
angle	Required. The angle in radians that you want to convert.

Example

Expand table

Formula	Description	Result
= DEGREES(PI())	Degrees of pi radians	180

Feedback

Was this page helpful? Yes No

DIVIDE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Performs division and returns alternate result or BLANK() on division by 0.

Syntax

DAX

```
DIVIDE(<numerator>, <denominator> [,<alternateresult>])
```

Parameters

Expand table

Term	Definition
numerator	The dividend or number to divide.
denominator	The divisor or number to divide by.
alternateresult	(Optional) The value returned when division by zero results in an error. When not provided, the default value is BLANK().

Return value

- A decimal number.

Remarks

- Alternate result on divide by 0 must be a constant.
- For best practices when using DIVIDE, see DIVIDE function vs. divide operator (/) in DAX.

Example

The following example returns 2.5.

```
DAX
= DIVIDE(5,2)
```

Example 1

The following example returns BLANK.

```
DAX
= DIVIDE(5,0)
```

Example 2

The following example returns 1.

```
DAX
= DIVIDE(5,0,1)
```

Related content

QUOTIENT function
Math and Trig functions

Feedback

Was this page helpful? Yes No

EVEN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns number rounded up to the nearest even integer. You can use this function for processing items that come in twos. For example, a packing crate accepts rows of one or two items. The crate is full when the number of items, rounded up to the nearest two, matches the crate's capacity.

Syntax

DAX

```
EVEN(number)
```

Parameters

Expand table

Term	Definition
number	The value to round.

Return value

Returns number rounded up to the nearest even integer.

Remarks

- If number is nonnumeric, EVEN returns the #VALUE! error value.
- Regardless of the sign of number, a value is rounded up when adjusted away from zero. If number is an even integer, no rounding occurs.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= EVEN(1.5)	Rounds 1.5 to the nearest even integer	2
= EVEN(3)	Rounds 3 to the nearest even integer	4
= EVEN(2)	Rounds 2 to the nearest even integer	2
= EVEN(-1)	Rounds -1 to the nearest even integer	-2

Feedback

Was this page helpful? Yes No

EXP

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns e raised to the power of a given number. The constant e equals 2.71828182845904, the base of the natural logarithm.

Syntax

```DAX
EXP(<number>)
```

Parameters

Expand table

Term	Definition
number	The exponent applied to the base e. The constant e equals 2.71828182845904, the base of the natural logarithm.

Return value

A decimal number.

Exceptions

Remarks

- EXP is the inverse of LN, which is the natural logarithm of the given number.
- To calculate powers of bases other than e, use the exponentiation operator (^). For more information, see DAX Operator Reference.

Example

The following formula calculates e raised to the power of the number contained in the column, `[Power]`.

```
DAX
```

```
= EXP([Power])
```

Related content

Math and Trig functions
LN function
EXP function
LOG function
LOG function

Feedback

Was this page helpful? Yes No

FACT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the factorial of a number, equal to the series 1*2*3*...* , ending in the given number.

Syntax

DAX

```
FACT(<number>)
```

Parameters

Expand table

Term	Definition
number	The non-negative number for which you want to calculate the factorial.

Return value

A decimal number.

Remarks

- If the number is not an integer, it is truncated and an error is returned. If the result is too large, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula returns the factorial for the series of integers in the column, `[Values]`.

DAX

```
= FACT([Values])
```

The following table shows the expected results:

Expand table

Values	Results
0	1
1	1
2	2
3	6
4	24
5	120
170	7.257415615308E+306

Related content

Math and Trig functions
TRUNC function

Feedback

Was this page helpful? Yes No

FLOOR

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number down, toward zero, to the nearest multiple of significance.

Syntax

DAX

```
FLOOR(<number>, <significance>)
```

Parameters

Expand table

Term	Definition
`number`	The numeric value you want to round.
`significance`	The multiple to which you want to round. The arguments `number` and `significance` must either both be positive, or both be negative.

Return value

A decimal number.

Remarks

- If either argument is nonnumeric, FLOOR returns `#VALUE!` error value.
- If number and significance have different signs, FLOOR returns the `#NUM!` error value.
- Regardless of the sign of the number, a value is rounded down when adjusted away from zero. If the number is an exact multiple of significance, no rounding occurs.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula takes the values in the [Total Product Cost] column from the table, InternetSales, and rounds down to the nearest multiple of .1.

```
DAX

= FLOOR(InternetSales[Total Product Cost],.1)
```

The following table shows the expected results for some sample values:

Expand table

Values	Expected Result
10.8423	10.8
8.0373	8
2.9733	2.9

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

GCD

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the greatest common divisor of two or more integers. The greatest common divisor is the largest integer that divides both number1 and number2 without a remainder.

Syntax

DAX

```
GCD(number1, number2)
```

Parameters

Expand table

Term	Definition
`number1`, `number2`	The two integers for which you want the greatest common divisor. If a value is not an integer, it is truncated.

Return value

The greatest common divisor of two or more integers.

Remarks

- If any argument is nonnumeric, GCD returns the `#VALUE!` error value.
- If any argument is less than zero, GCD returns the `#NUM!` error value.
- One divides any value evenly.
- A prime number has only itself and one as even divisors.
- If a parameter to GCD is >=2^53, GCD returns the `#NUM!` error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

DAX expression	Description	Result
`= GCD(5, 2)`	Greatest common divisor of 5 and 2.	1
`= GCD(24, 36)`	Greatest common divisor of 24 and 36.	12
`= GCD(7, 1)`	Greatest common divisor of 7 and 1.	1

Last updated on 01/22/2026

INT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number down to the nearest integer.

Syntax

DAX

```
INT(<number>)
```

Parameters

Expand table

Term	Definition
number	The number you want to round down to an integer

Return value

A whole number.

Remarks

TRUNC and INT are similar in that both return integers. TRUNC removes the fractional part of the number. INT rounds numbers down to the nearest integer based on the value of the fractional part of the number. INT and TRUNC are different only when using negative numbers: `TRUNC(-4.3)` returns -4, but `INT(-4.3)` returns -5 because -5 is the lower number.

Example

The following expression rounds the value to 1. If you use the ROUND function, the result would be 2.

DAX

```
= INT(1.5)
```

Related content

Math and Trig functions
ROUND function
ROUNDUP function
ROUNDDOWN function
MROUND function

Feedback

Was this page helpful? Yes No

ISO.CEILING

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number up, to the nearest integer or to the nearest multiple of significance.

Syntax

DAX

```
ISO.CEILING(<number>[, <significance>])
```

Parameters

Expand table

Term	Definition
number	The number you want to round, or a reference to a column that contains numbers.
significance	(optional) The multiple of significance to which you want to round. For example, to round to the nearest integer, type 1. If the unit of significance is not specified, the number is rounded up to the nearest integer.

Return value

A number, of the same type as the *number* argument, rounded as specified.

Remarks

There are two CEILING functions in DAX, with the following differences:

- The CEILING function emulates the behavior of the CEILING function in Excel.
- The ISO.CEILING function follows the ISO-defined behavior for determining the ceiling value.

The two functions return the same value for positive numbers, but different values for negative numbers. When using a positive multiple of significance, both CEILING and

ISO.CEILING round negative numbers upward (toward positive infinity). When using a negative multiple of significance, CEILING rounds negative numbers downward (toward negative infinity), while ISO.CEILING rounds negative numbers upward (toward positive infinity).

The result type is usually the same type of the significance used as argument with the following exceptions:

- If the first argument is of currency type then the result will be currency type.
- If the optional argument is not included the result is of integer type.
- If the significance argument is of Boolean type then the result is of integer type.
- If the significance argument is non-numeric type then the result is of real type.

Example: Positive Numbers

The following formula returns 4.45. This might be useful if you want to avoid using smaller units in your pricing. If an existing product is priced at $4.42, you can use ISO.CEILING to round prices up to the nearest unit of five cents.

DAX

```
= ISO.CEILING(4.42,0.05)
```

Example: Negative Numbers

The following formula returns the ISO ceiling value of -4.40.

DAX

```
= ISO.CEILING(-4.42,0.05)
```

Related content

Math and Trig functions
FLOOR function
CEILING function
ROUNDUP function

Feedback

Was this page helpful? Yes No

LCM

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the least common multiple of integers. The least common multiple is the smallest positive integer that is a multiple of all integer arguments number1, number2, and so on. Use LCM to add fractions with different denominators.

Syntax

```
DAX
LCM(number1, number2)
```

Parameters

Expand table

Term	Definition
`number1, number2`	The two integers for which you want the least common multiple. If a value is not an integer, it is truncated.

Return value

Returns the least common multiple of integers.

Remarks

- If any argument is nonnumeric, LCM returns the `#VALUE!` error value.
- If any argument is less than zero, LCM returns the `#NUM!` error value.
- If LCM(a,b) >=2^53, LCM returns the `#NUM!` error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
`= LCM(5, 2)`	Least common multiple of 5 and 2.	10
`= LCM(24, 36)`	Least common multiple of 24 and 36.	72

Last updated on 01/22/2026

LN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the natural logarithm of a number. Natural logarithms are based on the constant e (2.71828182845904).

Syntax

DAX

```
LN(<number>)
```

Parameters

Expand table

Term	Definition
number	The positive number for which you want the natural logarithm.

Return value

A decimal number.

Remarks

LN is the inverse of the EXP function.

Example

The following example returns the natural logarithm of the number in the column, `[Values]`.

DAX

```
= LN([Values])
```

Related content

Math and Trig functions
EXP function

Feedback

Was this page helpful? Yes No

LOG

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the logarithm of a number to the base you specify.

Syntax

DAX

```
LOG(<number>,<base>)
```

Parameters

Expand table

Term	Definition
number	The positive number for which you want the logarithm.
base	The base of the logarithm. If omitted, the base is 10.

Return value

A decimal number.

Remarks

You might receive an error if the value is too large to be displayed.

The function LOG10 is similar, but always returns the common logarithm, meaning the logarithm for the base 10.

Example

The following formulas return the same result, 2.

DAX

```
= LOG(100,10)
= LOG(100)
= LOG10(100)
```

Related content

Math and Trig functions
EXP function
LOG function
LOG function

Feedback

Was this page helpful? Yes No

LOG10

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the base-10 logarithm of a number.

Syntax

DAX

```
LOG10(<number>)
```

Parameters

Expand table

Term	Definition
number	A positive number for which you want the base-10 logarithm.

Return value

A decimal number.

Remarks

The LOG function lets you change the base of the logarithm, instead of using the base 10.

Example

The following formulas return the same result, 2:

DAX

```
= LOG(100,10)
= LOG(100)
```

```
= LOG10(100)
```

Related content

Math and Trig functions
EXP function
LOG function
LOG function

Feedback

Was this page helpful? Yes No

MOD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the remainder after a number is divided by a divisor. The result always has the same sign as the divisor.

Syntax

DAX

```
MOD(<number>, <divisor>)
```

Parameters

Expand table

Term	Definition
number	The number for which you want to find the remainder after the division is performed.
divisor	The number by which you want to divide.

Return value

A whole number.

Remarks

- If the divisor is 0 (zero), MOD returns an error. You cannot divide by 0.
- The MOD function can be expressed in terms of the INT function: MOD(n, d) = n - d*INT(n/d)

Example 1

The following formula returns 1, the remainder of 3 divided by 2.

```
DAX
= MOD(3,2)
```

Example 2

The following formula returns -1, the remainder of 3 divided by 2. Note that the sign is always the same as the sign of the divisor.

```
DAX
= MOD(-3,-2)
```

Related content

Math and Trig functions
ROUND function
ROUNDUP function
ROUNDDOWN function
MROUND function
INT function

Feedback

Was this page helpful? Yes No

MROUND

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a number rounded to the desired multiple.

Syntax

DAX

```
MROUND(<number>, <multiple>)
```

Parameters

Expand table

Term	Definition
number	The number to round.
multiple	The multiple of significance to which you want to round the number.

Return value

A decimal number.

Remarks

MROUND rounds up, away from zero, if the remainder of dividing **number** by the specified **multiple** is greater than or equal to half the value of **multiple**.

Example: Decimal Places

The following expression rounds 1.3 to the nearest multiple of .2. The expected result is 1.4.

DAX

```
= MROUND(1.3,0.2)
```

Example: Negative Numbers

The following expression rounds -10 to the nearest multiple of -3. The expected result is -9.

DAX

```
= MROUND(-10,-3)
```

Example: Error

The following expression returns an error, because the numbers have different signs.

DAX

```
= MROUND(5,-2)
```

Related content

Math and Trig functions
ROUND function
ROUNDUP function
ROUNDDOWN function
MROUND function
INT function

Feedback

Was this page helpful? Yes No

ODD

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns number rounded up to the nearest odd integer.

Syntax

DAX

```
ODD(number)
```

Parameters

Expand table

Term	Definition
number	Required. The value to round.

Return value

Returns number rounded up to the nearest odd integer.

Remarks

- If number is nonnumeric, ODD returns the #VALUE! error value.
- Regardless of the sign of number, a value is rounded up when adjusted away from zero. If number is an odd integer, no rounding occurs.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= ODD(1.5)	Rounds 1.5 up to the nearest odd integer.	3
= ODD(3)	Rounds 3 up to the nearest odd integer.	3
= ODD(2)	Rounds 2 up to the nearest odd integer.	3
= ODD(-1)	Rounds -1 up to the nearest odd integer.	-1
= ODD(-2)	Rounds -2 up (away from 0) to the nearest odd integer.	-3

Feedback

Was this page helpful? Yes No

PI

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the value of Pi, 3.14159265358979, accurate to 15 digits.

Syntax

DAX

```
PI()
```

Return value

A decimal number with the value of Pi, 3.14159265358979, accurate to 15 digits.

Remarks

Pi is a mathematical constant. In DAX, Pi is represented as a real number accurate to 15 digits, the same as Excel.

Example

The following formula calculates the area of a circle given the radius in the column, `[Radius]`.

DAX

```
= PI()*([Radius]*2)
```

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

POWER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the result of a number raised to a power.

Syntax

```DAX
POWER(<number>, <power>)
```

Parameters

Expand table

Term	Definition
number	The base number, which can be any real number.
power	The exponent to which the base number is raised.

Return value

A decimal number.

Example

The following example returns 25.

```DAX
= POWER(5,2)
```

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

QUOTIENT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Performs division and returns only the integer portion of the division result. Use this function when you want to discard the remainder of division.

Syntax

DAX

```
QUOTIENT(<numerator>, <denominator>)
```

Parameters

Expand table

Term	Definition
numerator	The dividend, or number to divide.
denominator	The divisor, or number to divide by.

Return value

A whole number.

Remarks

- If either argument is non-numeric, QUOTIENT returns the **#VALUE!** error value.
- You can use a column reference instead of a literal value for either argument. However, if the column that you reference contains a 0 (zero), an error is returned for the entire column of values.

Example

The following formulas return the same result, 2.

DAX

```
= QUOTIENT(5,2)
```

DAX

```
= QUOTIENT(10/2,2)
```

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

RADIANS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts degrees to radians.

Syntax

DAX

```
RADIANS(angle)
```

Parameters

Expand table

Term	Definition
angle	Required. An angle in degrees that you want to convert.

Example

Expand table

Formula	Description	Result
= RADIANS(270)	270 degrees as radians (4.712389 or 3π/2 radians)	4.712389

Feedback

Was this page helpful? Yes No

RAND

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a random number greater than or equal to 0 and less than 1, evenly distributed. The number that is returned changes each time the cell containing this function is recalculated.

Syntax

```DAX
RAND()
```

Return value

A decimal number.

Remarks

- Recalculation depends on various factors, including whether the model is set to **Manual** or **Automatic** recalculation mode, and whether data has been refreshed.
- RAND and other volatile functions that do not have fixed values are not always recalculated. For example, execution of a query or filtering will usually not cause such functions to be re-evaluated. However, the results for these functions will be recalculated when the entire column is recalculated. These situations include refresh from an external data source or manual editing of data that causes re-evaluation of formulas that contain these functions.
- RAND is always recalculated if the function is used in the definition of a measure.
- RAND function cannot return a result of zero, to prevent errors such as division by zero.

Examples

To generate a random real number between two other numbers, use:

```
DAX

= RAND()*(b-a)+a
```

To generate a random number greater than 0 and less than 1:

```
DAX

= RAND()
```

To generate a random number greater than 0 and less than 100

```
DAX

= RAND()*100
```

To generate a random whole number greater than 0 and less than 100

```
DAX

INT(RAND()*100)
```

Related content

Math and Trig functions
Statistical functions

Feedback

Was this page helpful? Yes No

RANDBETWEEN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a random number in the range between two numbers you specify.

Syntax

DAX

```
RANDBETWEEN(<bottom>,<top>)
```

Parameters

Expand table

Term	Definition
Bottom	The smallest integer the function will return.
Top	The largest integer the function will return.

Return value

A whole number.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula returns a random number between 1 and 10.

DAX

```
= RANDBETWEEN(1,10)
```

Related content

Math and Trig functions
Statistical functions

Feedback

Was this page helpful? Yes No

ROUND

Article • 04/15/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number to the specified number of digits.

Syntax

```DAX
ROUND(<number>, <num_digits>)
```

Parameters

Expand table

Term	Definition
`number`	The number you want to round.
`num_digits`	The number of digits to which you want to round. A negative value rounds digits to the left of the decimal point; a value of zero rounds to the nearest integer.

Return value

A decimal number.

Remarks

- If `num_digits` is greater than 0 (zero), then number is rounded to the specified number of decimal places.
- If `num_digits` is 0, the number is rounded to the nearest integer.
- If `num_digits` is less than 0, the number is rounded to the left of the decimal point.
- Ties are broken by rounding half away from zero (also known as commercial rounding). | Examples | Result | | --- | --- | | `= ROUND(1.15, 1)` | 1.2 | | `= ROUND(-1.15, 1)` | -1.2 |
- Related functions

- To always round up (away from zero), use the ROUNDUP function.
- To always round down (toward zero), use the ROUNDDOWN function.
- To round a number to a specific multiple (for example, to round to the nearest multiple of 0.5), use the MROUND function.
- Use the functions TRUNC and INT to obtain the integer portion of the number.

Example 1

The following formula rounds 2.15 up, to one decimal place. The expected result is 2.2.

```
DAX

= ROUND(2.15,1)
```

Example 2

The following formula rounds 21.5 to one decimal place to the left of the decimal point. The expected result is 20.

```
DAX

= ROUND(21.5,-1)
```

Related content

Math and Trig functions ROUND ROUNDDOWN MROUND INT TRUNC

ROUNDDOWN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number down, toward zero.

Syntax

DAX

```
ROUNDDOWN(<number>, <num_digits>)
```

Parameters

Expand table

Term	Definition
number	A real number that you want rounded down.
num_digits	The number of digits to which you want to round. Negative rounds to the left of the decimal point; zero to the nearest integer.

Return value

A decimal number.

Remarks

- If **num_digits** is greater than 0 (zero), then the value in **number** is rounded down to the specified number of decimal places.
- If **num_digits** is 0, then the value in **number** is rounded down to the nearest integer.
- If **num_digits** is less than 0, then the value in **number** is rounded down to the left of the decimal point.

- ROUNDDOWN behaves like ROUND, except that it always rounds a number down. The INT function also rounds down, but with INT the result is always an integer, whereas with ROUNDDOWN you can control the precision of the result.

Example 1

The following example rounds 3.14159 down to three decimal places. The expected result is 3.141.

DAX

```
= ROUNDDOWN(3.14159,3)
```

Example 2

The following example rounds the value of 31415.92654 down to 2 decimal places to the left of the decimal. The expected result is 31400.

DAX

```
= ROUNDDOWN(31415.92654, -2)
```

Related content

Math and Trig functions
ROUND
ROUNDUP
ROUNDDOWN
MROUND
INT

Feedback

Was this page helpful? Yes No

ROUNDUP

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number up, away from 0 (zero).

Syntax

DAX

```
ROUNDUP(<number>, <num_digits>)
```

Parameters

Expand table

Term	Definition
number	A real number that you want to round up.
num_digits	The number of digits to which you want to round. A negative value for **num_digits** rounds to the left of the decimal point; if **num_digits** is zero or is omitted, **number** is rounded to the nearest integer.

Return value

A decimal number.

Remarks

- If **num_digits** is greater than 0 (zero), then the number is rounded up to the specified number of decimal places.
- If **num_digits** is 0, then number is rounded up to the nearest integer.
- If **num_digits** is less than 0, then number is rounded up to the left of the decimal point.
- ROUNDUP behaves like ROUND, except that it always rounds a number up.

Example

The following formula rounds Pi to four decimal places. The expected result is 3.1416.

```
DAX

= ROUNDUP(PI(),4)
```

Example: Decimals as Second Argument

The following formula rounds 1.3 to the nearest multiple of 0.2. The expected result is 2.

```
DAX

= ROUNDUP(1.3,0.2)
```

Example: Negative Number as Second Argument

The following formula rounds the value in the column, **FreightCost**, with the expected results shown in the following table:

```
DAX

= ROUNDUP([Values],-1)
```

When **num_digits** is less than zero, the number of places to the left of the decimal sign is increased by the value you specify.

Expand table

FreightCost	Expected Result
13.25	20
2.45	10
25.56	30
1.34	10
345.01	350

Related content

Math and Trig functions
ROUND
ROUNDDOWN
MROUND
INT

Feedback

Was this page helpful? Yes No

SIGN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Determines the sign of a number, the result of a calculation, or a value in a column. The function returns 1 if the number is positive, 0 (zero) if the number is zero, or -1 if the number is negative.

Syntax

```DAX
SIGN(<number>)
```

Parameters

Expand table

Term	Definition
number	Any real number, a column that contains numbers, or an expression that evaluates to a number.

Return value

A whole number. The possible Return values are 1, 0, and -1.

Expand table

Return value	Description
1	The number is positive
0	The number is zero
-1	The number is negative

Example

The following formula returns the sign of the result of the expression that calculates sale price minus cost.

```DAX
= SIGN( ([Sale Price] - [Cost]) )
```

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

SIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the sine of the given angle.

Syntax

```DAX
SIN(number)
```

Parameters

Expand table

Term	Definition
number	Required. The angle in radians for which you want the sine.

Return value

Returns the sine of the given angle.

Remarks

If an argument is in degrees, multiply it by PI()/180 or use the RADIANS function to convert it to radians.

Example

Expand table

Formula	Description	Result
= SIN(PI())	Sine of pi radians (0, approximately).	0.0

Formula	Description	Result
= SIN(PI()/2)	Sine of pi/2 radians.	1.0
= SIN(30*PI()/180)	Sine of 30 degrees.	0.5
= SIN(RADIANS(30))	Sine of 30 degrees.	0.5

Feedback

Was this page helpful? Yes No

SINH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the hyperbolic sine of a number.

Syntax

```
DAX
SINH(number)
```

Parameters

Expand table

Term	Definition
number	Required. Any real number.

Return value

Returns the hyperbolic sine of a number.

Remarks

- The formula for the hyperbolic sine is:

$$\text{SINH}(z) = \frac{e^z - e^{-z}}{2}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Probability of obtaining a result of less than 1.03 seconds.

DAX

```
= 2.868*SINH(0.0342\*1.03)
```

Returns, 0.1010491

Feedback

Was this page helpful? Yes No

SQRT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the square root of a number.

Syntax

```DAX
SQRT(<number>)
```

Parameters

Expand table

Term	Definition
number	The number for which you want the square root, a column that contains numbers, or an expression that evaluates to a number.

Return value

A decimal number.

Remarks

If the number is negative, the SQRT function returns an error.

Example

The following formula,

```DAX
= SQRT(25)
```

Related content

Math and Trig functions

Feedback

Was this page helpful? Yes No

SQRTPI

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the square root of (number * pi).

Syntax

```DAX
SQRTPI(number)
```

Parameters

Expand table

Term	Definition
number	Required. The number by which pi is multiplied.

Return value

Returns the square root of (number * pi).

Example

Expand table

Formula	Description	Result
= SQRTPI(1)	Square root of pi.	1.772454
= SQRTPI(2)	Square root of 2 * pi.	2.506628

Feedback

Was this page helpful? Yes No

TAN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the tangent of the given angle.

Syntax

```
DAX
TAN(number)
```

Parameters

Expand table

Term	Definition
number	Required. The angle in radians for which you want the tangent.

Return value

Returns the tangent of the given angle.

Remarks

If your argument is in degrees, multiply it by PI()/180 or use the RADIANS function to convert it to radians.

Example

Expand table

Formula	Description	Result
= TAN(0.785)	Tangent of 0.785 radians (0.99920)	0.99920

Formula	Description	Result
= TAN(45*PI()/180)	Tangent of 45 degrees (1)	1
= TAN(RADIANS(45))	Tangent of 45 degrees (1)	1

Feedback

Was this page helpful? Yes No

TANH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the hyperbolic tangent of a number.

Syntax

```
DAX
TANH(number)
```

Parameters

Expand table

Term	Definition
number	Required. Any real number.

Return value

Returns the hyperbolic tangent of a number.

Remarks

- The formula for the hyperbolic tangent is:

$$\text{TANH}(z) = \frac{\text{SINH}(z)}{\text{COSH}(z)}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= TANH(-2)	Hyperbolic tangent of -2 (-0.96403)	-0.964028
= TANH(0)	Hyperbolic tangent of 0 (0)	0
= TANH(0.5)	Hyperbolic tangent of 0.5 (0.462117)	0.462117

Feedback

Was this page helpful? Yes No

TRUNC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Truncates a number to an integer by removing the decimal, or fractional, part of the number.

Syntax

```DAX
TRUNC(<number>,<num_digits>)
```

Parameters

Expand table

Term	Definition
number	The number you want to truncate.
num_digits	A number specifying the precision of the truncation; if omitted, 0 (zero)

Return value

A whole number.

Remarks

TRUNC and INT are similar in that both return integers. TRUNC removes the fractional part of the number. INT rounds numbers down to the nearest integer based on the value of the fractional part of the number. INT and TRUNC are different only when using negative numbers: `TRUNC(-4.3)` returns -4, but `INT(-4.3)` returns -5 because -5 is the smaller number.

Example 1

The following formula returns 3, the integer part of pi.

```
DAX
= TRUNC(PI())
```

Example 2

The following formula returns -8, the integer part of -8.9.

```
DAX
= TRUNC(-8.9)
```

Related content

Math and Trig functions
ROUND
ROUNDUP
ROUNDDOWN
MROUND
INT

Feedback

Was this page helpful? Yes No

Other functions

09/16/2025

These functions perform unique actions that cannot be defined by any of the categories.

In this category

Expand table

Function	Description
BLANK	Returns a blank.
ERROR	Raises an error with an error message.
EVALUATEANDLOG	Returns the value of the first argument and logs it in a DAX Evaluation Log profiler event.
EXTERNALMEASURE	Invokes a measure defined in a remote model and returns its result.
TOCSV	Returns a table as a string in CSV format.
TOJSON	Returns a table as a string in JSON format.

BLANK

Summarize this article for me

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a blank.

Syntax

```
BLANK()
```

Return value

A blank.

Remarks

- Blanks are not equivalent to nulls. DAX uses blanks for both database nulls and for blank cells in Excel.
- Some DAX functions treat blank cells somewhat differently from Microsoft Excel. Blanks and empty strings ("") are not always equivalent, but some operations may treat them as such.

Example

The following example illustrates how you can work with blanks in formulas. The formula calculates the ratio of sales between the Resellers and the Internet channels. However, before attempting to calculate the ratio the denominator should be checked for zero values. If the denominator is zero then a blank value should be returned; otherwise, the ratio is calculated.

```
=
IF (
    SUM ( InternetSales_USD[SalesAmount_USD] ) = 0,
    BLANK (),
    SUM ( ResellerSales_USD[SalesAmount_USD] )
```

```
        / SUM ( InternetSales_USD[SalesAmount_USD] )
)
```

The table shows the expected results when this formula is used to create a table visualization. Blank cells indicate where Internet sales were zero, so the ratio could not be calculated.

Expand table

Row Labels	Accessories	Bikes	Clothing
2005		2.65	
2006		3.33	
2007	1.04	2.92	6.63
2008	0.41	1.53	2.00

In the original data source, the column evaluated by the BLANK function might have included text, empty strings, or nulls. If the original data source was a SQL Server database, nulls and empty strings are different kinds of data. However, for this operation an implicit type cast is performed and DAX treats them as the same.

Related content

- Text functions
- ISBLANK function

Last updated on 03/12/2026

ERROR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Raises an error with an error message.

Syntax

DAX

```
ERROR(<text>)
```

Parameters

Expand table

Term	Definition
`text`	A text string containing an error message.

Return value

None

Remarks

- The ERROR function can be placed in a DAX expression anywhere a scalar value is expected.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following DAX query:

DAX

```
DEFINE
MEASURE DimProduct[Measure] =
        IF(
            SELECTEDVALUE(DimProduct[Color]) = "Red",
            ERROR("red color encountered"),
            SELECTEDVALUE(DimProduct[Color])
        )
EVALUATE SUMMARIZECOLUMNS(DimProduct[Color], "Measure", [Measure])
ORDER BY [Color]
```

Fails and raises and error message containing "red color encountered".

Example 2

The following DAX query:

DAX

```
DEFINE
MEASURE DimProduct[Measure] =
        IF(
            SELECTEDVALUE(DimProduct[Color]) = "Magenta",
            ERROR("magenta color encountered"),
            SELECTEDVALUE(DimProduct[Color])
        )
EVALUATE SUMMARIZECOLUMNS(DimProduct[Color], "Measure", [Measure])
ORDER BY [Color]
```

Returns the following table:

Expand table

DimProduct[Color]	[Measure]
Black	Black
Blue	Blue
Grey	Grey
Multi	Multi
NA	NA
Red	Red
Silver	Silver

DimProduct[Color]	[Measure]
Silver\Black	Silver\Black
White	White
Yellow	Yellow

Because Magenta is not one of the product colors, the ERROR function is not executed.

Feedback

Was this page helpful? Yes No

EVALUATEANDLOG

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the value of the first argument and logs it in a DAX Evaluation Log profiler event. This function is fully functional in Power BI Desktop only. It acts as a simple passthrough function in other environments.

Syntax

```DAX
EVALUATEANDLOG(<Value>, [Label], [MaxRows])
```

Parameters

Expand table

Term	Definition
`Value`	Any scalar expression or table expression to be evaluated and logged.
`Label`	(Optional) A constant string included in both the json text and the Label column of the DAX Evaluation Log event that can be used to easily identify an instance of the function call.
`MaxRows`	(Optional) The maximum number of rows in the json text of the DAX Evaluation Log event when the first argument is a table expression. Default is 10.

Return value

The value of the first argument.

The JSON structure logged in a DAX Evaluation Log profiler event includes:

- `expression` is the text version of the first argument.
- `label` is the Label parameter when specified in the expression.
- `inputs` is a list of columns in the evaluation context that affects the values of the first argument.

- `outputs` is a list of a single column [Value] when the first argument is a scalar expression and a list of output columns when the first argument is a table expression.
- `data` is a list of input values and output values when the first argument is a scalar expression, and a list of input values and corresponding output rows when the first argument is a table expression.
- `rowCount` is the number of rows when the first argument is a table expression. Even though the number of rows in the json output is truncated by the MaxRows parameter, rowCount is the real number of rows without truncation.

Remarks

- Trace events can be captured by using SQL Server Profiler and the open-source DAX Debug Output tool.
- This function can be used with almost any sub-expression in a DAX expression, and the entire expression will still be valid.
- When the first argument is evaluated multiple times in a single query, the function generates a single DAX Evaluation Log event that contains both the input values and the corresponding output values.
- When the label parameter is specified, its value is returned in both the json output and the Label column of the DAX Evaluation Log event.
- If the first argument is a table expression, only the top MaxRows rows are shown in the DAX Evaluation Log event.
- In some cases, this function is not executed due to optimizations.
- If the DAX Evaluation Log event is greater than one million characters, it's truncated to preserve correct json structure.

Example 1

The following DAX query:

DAX

```
evaluate
SUMMARIZE(
    EVALUATEANDLOG(FILTER(Sales, [ProductKey] = 528)),
    Sales[SalesTerritoryKey],
    "sum",
```

```
    sum(Sales[Sales Amount])
)
```

Returns the following DAX Evaluation Log event:

JSON


```
{
    "expression": "FILTER(Sales, [ProductKey] = 528)",
    "inputs": [],
    "outputs": ["'Sales'[SalesOrderLineKey]", "'Sales'[ResellerKey]",
"'Sales'[CustomerKey]", "'Sales'[ProductKey]", "'Sales'[OrderDateKey]",
"'Sales'[DueDateKey]", "'Sales'[ShipDateKey]", "'Sales'[SalesTerritoryKey]",
"'Sales'[Order Quantity]", "'Sales'[Unit Price]", "'Sales'[Extended
Amount]", "'Sales'[Product Standard Cost]", "'Sales'[Total Product Cost]",
"'Sales'[Sales Amount]", "'Sales'[Unit Price Discount Pct]"],
    "data": [
        {
            "input": [],
            "rowCount": 3095,
            "output": [
                [52174001, -1, 23785, 528, 20190707, 20190717, 20190714, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52173001, -1, 26278, 528, 20190707, 20190717, 20190714, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52082001, -1, 23831, 528, 20190705, 20190715, 20190712, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52054002, -1, 11207, 528, 20190704, 20190714, 20190711, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52036001, -1, 25337, 528, 20190704, 20190714, 20190711, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51939002, -1, 23670, 528, 20190702, 20190712, 20190709, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51911002, -1, 11746, 528, 20190701, 20190711, 20190708, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51379003, -1, 13745, 528, 20190612, 20190622, 20190619, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51264002, -1, 11282, 528, 20190605, 20190615, 20190612, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51184003, -1, 11263, 528, 20190531, 20190610, 20190607, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0]
            ]
        }
    ]
}
```


Example 2

The following DAX query with a scalar argument and varying attributes:

DAX

```
evaluate
SELECTCOLUMNS(
    TOPN(5, Customer),
    [Customer],
    "Customer",
    EVALUATEANDLOG([Customer] & ", " & [Country-Region], "customerLog")
)
```

Returns the following DAX Evaluation Log event:

JSON

```
{
    "expression": "[Customer] & \", \" & [Country-Region]",
    "label": "customerLog",
    "inputs": ["'Customer'[Customer]", "'Customer'[Country-Region]"],
    "data": [
        {
            "input": ["Russell Xie", "United States"],
            "output": "Russell Xie, United States"
        },
        {
            "input": ["Savannah Baker", "United States"],
            "output": "Savannah Baker, United States"
        },
        {
            "input": ["Maurice Tang", "United States"],
            "output": "Maurice Tang, United States"
        },
        {
            "input": ["Emily Wood", "United States"],
            "output": "Emily Wood, United States"
        },
        {
            "input": ["Meghan Hernandez", "United States"],
            "output": "Meghan Hernandez, United States"
        }
    ]
}
```

Related content

TOCSV TOJSON

Feedback

Was this page helpful? Yes No

EXTERNALMEASURE

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Invokes a measure defined in a remote model and returns its result with the specified `datatype`.

Syntax

```DAX
EXTERNALMEASURE(<measurename>, <datatype>, <connection>)
```

Parameters

Expand table

Term	Definition
`measurename`	Name of the measure as defined in the remote model.
`datatype`	An enumeration that includes: BOOLEAN/LOGICAL, CURRENCY/DECIMAL, DATETIME, DOUBLE, INTEGER/INT64, STRING/TEXT, VARIANT.
`connection`	The name of the connection to the remote model.

Return value

Result of the remote measure with the datatype specified in `datatype`.

Remarks

- You can only use this function in composite models that have a remote model connection.

Example

If the remote model connection is called **DirectQuery to AS - Adventure Works DW 2020** and the remote model defines a measure called **Total Sales** you can invoke that measure and return its result as **currency** using:

```DAX
```

```
Total Sales Remote =
EXTERNALMEASURE (
    "Total Sales",
    CURRENCY,
    "DirectQuery to AS - Adventure Works DW 2020"
)
```

Related content

- Composite models

Last updated on 08/04/2026

TOCSV

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table as a string in CSV format.

Syntax

```DAX
TOCSV(<Table>, [MaxRows], [Delimiter], [IncludeHeaders])
```

Parameters

Expand table

Term	Definition
Table	The table to be converted to CSV.
MaxRows	(Optional) The maximum number fo rows to convert. Default is 10 rows.
Delimiter	(Optional) A column delimiter. Default is comma ",".
IncludeHeaders	(Optional) Specifies a header with column name as the first row. Default is True.

Return value

A string with CSV representation of the table.

Example

The following DAX query:

```DAX
EVALUATE
{TOCSV(DimSalesTerritory)}
```

Returns:

```
'DimSalesTerritory'[SalesTerritoryKey],'DimSalesTerritory'[SalesTerritoryAlt
ernateKey],'DimSalesTerritory'[SalesTerritoryRegion],'DimSalesTerritory'[Sal
esTerritoryCountry],'DimSalesTerritory'[SalesTerritoryGroup]
1,1,Northwest,United States,North America
2,2,Northeast,United States,North America
3,3,Central,United States,North America
4,4,Southwest,United States,North America
5,5,Southeast,United States,North America
6,6,Canada,Canada,North America
7,7,France,France,Europe
8,8,Germany,Germany,Europe
9,9,Australia,Australia,Pacific
10,10,United Kingdom,United Kingdom,Europe
```

Related content

TOJSON
EVALUATEANDLOG

Feedback

Was this page helpful? Yes No

TOJSON

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table as a string using JSON format.

Syntax

```
DAX
TOJSON(<Table>, [MaxRows])
```

Parameters

Expand table

Term	Definition
`Table`	The table to be converted to JSON.
`MaxRows`	(Optional) The maximum number fo rows to convert. Default is 10 rows.

Return value

A string with JSON representation of the table. The representation contains column names as "header", count-of-rows as "rowCount", and values as "data".

Example

The following DAX query:

```
DAX
EVALUATE
{TOJSON(DimSalesTerritory)}
```

Returns:

JSON


```
{
"header": ["'DimSalesTerritory'[SalesTerritoryKey]",
"'DimSalesTerritory'[SalesTerritoryAlternateKey]",
"'DimSalesTerritory'[SalesTerritoryRegion]",
"'DimSalesTerritory'[SalesTerritoryCountry]",
"'DimSalesTerritory'[SalesTerritoryGroup]"],<br>
"rowCount": 11,
"data": [
[1, 1, "Northwest", "United States", "North America"],
[2, 2, "Northeast", "United States", "North America"],
[3, 3, "Central", "United States", "North America"],
[4, 4, "Southwest", "United States", "North America"],
[5, 5, "Southeast", "United States", "North America"],
[6, 6, "Canada", "Canada", "North America"],
[7, 7, "France", "France", "Europe"],
[8, 8, "Germany", "Germany", "Europe"],
[9, 9, "Australia", "Australia", "Pacific"],
[10, 10, "United Kingdom", "United Kingdom", "Europe"]
]
}
```


Related content

TOCSV EVALUATEANDLOG

Feedback

Was this page helpful? Yes No

Parent and Child functions

Article • 10/20/2023

These functions manage data that is presented as parent/child hierarchies. To learn more, see Understanding functions for Parent-Child Hierarchies in DAX.

In this category

Function	Description
PATH	Returns a delimited text string with the identifiers of all the parents of the current identifier.
PATHCONTAINS	Returns TRUE if the specified *item* exists within the specified *path*.
PATHITEM	Returns the item at the specified *position* from a string resulting from evaluation of a PATH function.
PATHITEMREVERSE	Returns the item at the specified *position* from a string resulting from evaluation of a PATH function.
PATHLENGTH	Returns the number of parents to the specified item in a given PATH result, including self.

Feedback

Was this page helpful? Yes No

Understanding functions for parent-child hierarchies in DAX

Article • 10/20/2023

DAX provides five functions to help users manage data that is presented as a parent-child hierarchy in their models. With this functions a user can obtain the entire lineage of parents a row has, how many levels has the lineage to the top parent, who is the parent n-levels above the current row, who is the n-descendant from the top of the current row hierarchy and is certain parent a parent in the current row hierarchy?

Parent-child functions in DAX

The following table contains a Parent-Child hierarchy on the columns: **EmployeeKey** and **ParentEmployeeKey** that is used in all the functions examples.

EmployeeKey	ParentEmployeeKey
112	
14	112
3	14
11	3
13	3
162	3
117	162
221	162
81	162

In the above table you can see that employee 112 has no parent defined, employee 14 has employee 112 as manager (ParentEmployeeKey), employee 3 has employee 14 as manager and employees 11, 13, and 162 have employee 3 as manager. The above helps to understand that employee 112 has no manager above her/him and she/he is the top manager for all employees shown here; also, employee 3 reports to employee 14 and employees 11, 13, 162 report to 3.

The following table presents the available functions, a brief description of the function and an example of the function over the same data shown above.

PATH function - Returns a delimited text with the identifiers of all the parents to the current row, starting with the oldest or top most until current.

EmployeeKey	ParentEmployeeKey	Path
112		112
14	112	112|14
3	14	112|14|3
11	3	112|14|3|11
13	3	112|14|3|13
162	3	112|14|3|162
117	162	112|14|3|162|117
221	162	112|14|3|162|221
81	162	112|14|3|162|81

PATHLENGTH function - Returns the number of levels in a given PATH(), starting at current level until the oldest or top most parent level. In the following example column PathLength is defined as '`= PATHLENGTH([Path])`'; the example includes all data from the Path() example to help understand how this function works.

EmployeeKey	ParentEmployeeKey	Path	PathLength
112		112	1
14	112	112|14	2
3	14	112|14|3	3
11	3	112|14|3|11	4
13	3	112|14|3|13	4
162	3	112|14|3|162	4
117	162	112|14|3|162|117	5
221	162	112|14|3|162|221	5
81	162	112|14|3|162|81	5

PATHITEM function - Returns the item at the specified position from a PATH() like result, counting from left to right. In the following example column PathItem - 4th from left is

defined as '`= PATHITEM([Path], 4)`'; this example returns the EmployeKey at fourth position in the Path string from the left, using the same sample data from the Path() example.

EmployeeKey	ParentEmployeeKey	Path	PathItem - 4th from left
112		112	
14	112	112|14	
3	14	112|14|3	
11	3	112|14|3|11	11
13	3	112|14|3|13	13
162	3	112|14|3|162	162
117	162	112|14|3|162|117	162
221	162	112|14|3|162|221	162
81	162	112|14|3|162|81	162

PATHITEMREVERSE function - Returns the item at *position* from a PATH() like function result, counting backwards from right to left.
In the following example column PathItemReverse - 3rd from right is defined as '`= PATHITEMREVERSE([Path], 3)`'; this example returns the EmployeKey at third position in the Path string from the right, using the same sample data from the Path() example.

EmployeeKey	ParentEmployeeKey	Path	PathItemReverse - 3rd from right
112		112	
14	112	112|14	
3	14	112|14|3	112
11	3	112|14|3|11	14
13	3	112|14|3|13	14
162	3	112|14|3|162	14
117	162	112|14|3|162|117	3
221	162	112|14|3|162|221	3
81	162	112|14|3|162|81	3

PATHCONTAINS function - Returns **TRUE** if the specified *item* exists within the specified *path*. In the following example column PathContains - employee 162 is defined as '`=PATHCONTAINS([Path], "162")`'; this example returns **TRUE** if the given path contains employee 162. This example uses the results from the Path() example above.

EmployeeKey	ParentEmployeeKey	Path	PathContains - employee 162
112		112	FALSE
14	112	112|14	FALSE
3	14	112|14|3	FALSE
11	3	112|14|3|11	FALSE
13	3	112|14|3|13	FALSE
162	3	112|14|3|162	TRUE
117	162	112|14|3|162|117	TRUE

Feedback

Was this page helpful? Yes No

PATH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a delimited text string with the identifiers of all the parents of the current identifier, starting with the oldest and continuing until current.

Syntax

DAX

```
PATH(<ID_columnName>, <parent_columnName>)
```

Parameters

Expand table

Term	Definition
ID_columnName	The name of an existing column containing the unique identifier for rows in the table. This cannot be an expression. The data type of the value in *ID_columnName* must be text or integer, and must also be the same data type as the column referenced in *parent_columnName*.
parent_columnName	The name of an existing column containing the unique identifier for the parent of the current row. This cannot be an expression. The data type of the value in *parent_columnName* data type must be text or integer, and must be the same data type as the value in *ID_columnName*.

Return value

A delimited text string containing the identifiers of all the parents to the current identifier.

Remarks

- This function is used in tables that have some kind of internal hierarchy, to return the items that are related to the current row value. For example, in an Employees

table that contains employees, the managers of employees, and the managers of the managers, you can return the path that connects an employee to his or her manager.

- The path is not constrained to a single level of parent-child relationships; it can return related rows that are several levels up from the specified starting row.
  - The delimiter used to separate the ascendants is the vertical bar, '|'.
  - The values in *ID_columnName* and *parent_columnName* must have the same data type, text or integer.
  - Values in *parent_columnName* must be present in *ID_columnName*. That is, you cannot look up a parent if there is no value at the child level.
  - If *parent_columnName* is BLANK then PATH() returns *ID_columnName* value. In other words, if you look for the manager of an employee but the *parent_columnName* column has no data, the PATH function returns just the employee ID.
  - If *ID_columnName* has duplicates and *parent_columnName* is the same for those duplicates then PATH() returns the common *parent_columnName* value; however, if *parent_columnName* value is different for those duplicates then PATH() returns an error. In other words, if you have two listings for the same employee ID and they have the same manager ID, the PATH function returns the ID for that manager. However, if there are two identical employee IDs that have different manager IDs, the PATH function returns an error.
  - If *ID_columnName* is BLANK then PATH() returns BLANK.
  - If *ID_columnName* contains a vertical bar '|' then PATH() returns an error.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example creates a calculated column that lists all the managers for each employee.

DAX

```
= PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey])
```

Feedback

Was this page helpful? Yes No

PATHCONTAINS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns **TRUE** if the specified *item* exists within the specified *path*.

Syntax

```
DAX
PATHCONTAINS(<path>, <item>)
```

Parameters

Expand table

Term	Definition
path	A string created as the result of evaluating a PATH function.
item	A text expression to look for in the path result.

Return value

A value of **TRUE** if *item* exists in *path*; otherwise **FALSE**.

Remarks

- If *item* is an integer number it is converted to text and then the function is evaluated. If conversion fails then the function returns an error.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example creates a calculated column that takes a manager ID and checks a set of employees. If the manager ID is among the list of managers returned by the PATH function, the PATHCONTAINS function returns true; otherwise it returns false.

DAX

```
= PATHCONTAINS(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]),
"23")
```

Feedback

Was this page helpful? Yes No

PATHITEM

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the item at the specified *position* from a string resulting from evaluation of a PATH function. Positions are counted from left to right.

Syntax

DAX

```
PATHITEM(<path>, <position>[, <type>])
```

Parameters

Expand table

Term	Definition
path	A text string in the form of the results of a PATH function.
position	An integer expression with the position of the item to be returned.
type	(Optional)An enumeration that defines the data type of the result:

type enumeration

Expand table

Enumeration	Alternate Enumeration	Description
TEXT	0	Results are returned with the data type text. (default).
INTEGER	1	Results are returned as integers.

Return value

The identifier returned by the PATH function at the specified position in the list of identifiers. Items returned by the PATH function are ordered by most distant to current.

Remarks

- This function can be used to return a specific level from a hierarchy returned by a PATH function. For example, you could return just the skip-level managers for all employees.
- If you specify a number for *position* that is less than one (1) or greater than the number of elements in *path*, the PATHITEM function returns BLANK
- If *type* is not a valid enumeration element an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns the third tier manager of the current employee; it takes the employee and manager IDs as the input to a PATH function that returns a string with the hierarchy of parents to current employee. From that string PATHITEM returns the third entry as an integer.

DAX

```
= PATHITEM(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]), 3, 1)
```

Feedback

Was this page helpful? Yes No

PATHITEMREVERSE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the item at the specified *position* from a string resulting from evaluation of a PATH function. Positions are counted backwards from right to left.

Syntax

DAX

```
PATHITEMREVERSE(<path>, <position>[, <type>])
```

Parameters

Expand table

Term	Definition
path	A text string resulting from evaluation of a PATH function.
position	An integer expression with the position of the item to be returned. Position is counted backwards from right to left.
type	(Optional)An enumeration that defines the data type of the result:

type enumeration

Expand table

Enumeration	Alternate Enumeration	Description
TEXT	0	Results are returned with the data type text. (default).
INTEGER	1	Results are returned as integers.

Return value

The n-position ascendant in the given path, counting from current to the oldest.

Remarks

- This function can be used to get an individual item from a hierarchy resulting from a PATH function.
- This function reverses the standard order of the hierarchy, so that closest items are listed first, For example, if the PATh function returns a list of managers above an employee in a hierarchy, the PATHITEMREVERSE function returns the employee's immediate manager in position 2 because position 1 contains the employee's id.
- If the number specified for *position* is less than one (1) or greater than the number of elements in *path*, the PATHITEM function returns BLANK.
- If *type* is not a valid enumeration element an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example takes an employee ID column as the input to a PATH function, and reverses the list of grandparent elements that are returned. The position specified is 3 and the return type is 1; therefore, the PATHITEMREVERSE function returns an integer representing the manager two levels up from the employee.

DAX

```
= PATHITEMREVERSE(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]),
3, 1)
```

Feedback

Was this page helpful? Yes No

PATHLENGTH

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of parents to the specified item in a given PATH result, including self.

Syntax

```DAX
PATHLENGTH(<path>)
```

Parameters

Expand table

Term	Definition
path	A text expression resulting from evaluation of a PATH function.

Return value

The number of items that are parents to the specified item in a given PATH result, including the specified item.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example takes an employee ID as input to a PATH function and returns a list of the managers above that employee in the hierarchy, The PATHLENGTH function

takes that result and counts the different levels of employees and managers, including the employee you started with.

```
DAX
= PATHLENGTH(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]))
```

Feedback

Was this page helpful? Yes No

Relationship functions

Article • 10/20/2023

Functions in this category are for managing and utilizing relationships between tables.

In this category

Function	Description
CROSSFILTER	Specifies the cross-filtering direction to be used in a calculation for a relationship that exists between two columns.
RELATED	Returns a related value from another table.
RELATEDTABLE	Evaluates a table expression in a context modified by the given filters.
USERELATIONSHIP	Specifies the relationship to be used in a specific calculation as the one that exists between columnName1 and columnName2.

Feedback

Was this page helpful? Yes No

CROSSFILTER

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Specifies the cross-filtering direction to be used in a calculation for a relationship that exists between two columns.

Syntax

DAX

```
CROSSFILTER(<columnName1>, <columnName2>, <direction>)
```

Parameters

Expand table

Term	Definition
`columnName1`	The name of an existing column, using standard DAX syntax and fully qualified, that usually represents the many side of the relationship to be used; if the arguments are given in reverse order the function will swap them before using them. This argument cannot be an expression.
`columnName2`	The name of an existing column, using standard DAX syntax and fully qualified, that usually represents the one side or lookup side of the relationship to be used; if the arguments are given in reverse order the function will swap them before using them. This argument cannot be an expression.
`Direction`	The cross-filter direction to be used. Must be one of the following: `None` - No cross-filtering occurs along this relationship. `Both` - Filters on either side filters the other side. `OneWay` - Filters on the one side or the lookup side of a relationship filter the other side. This option cannot be used with a one-to-one relationship . Don't use this option on a many-to-many relationship because it is unclear which side is the lookup side; use OneWay_LeftFiltersRight or OneWay_RightFiltersLeft instead. `OneWay_LeftFiltersRight` - Filters on the side of `columnName1` filter the side of `columnName2`. This option cannot be used with a one-to-one or many-to-one

Term	Definition
	relationship. `OneWay_RightFiltersLeft` - Filters on the side of `columnName2` filter the side of `columnName1`. This option cannot be used with a one-to-one or many-to-one relationship.

Return value

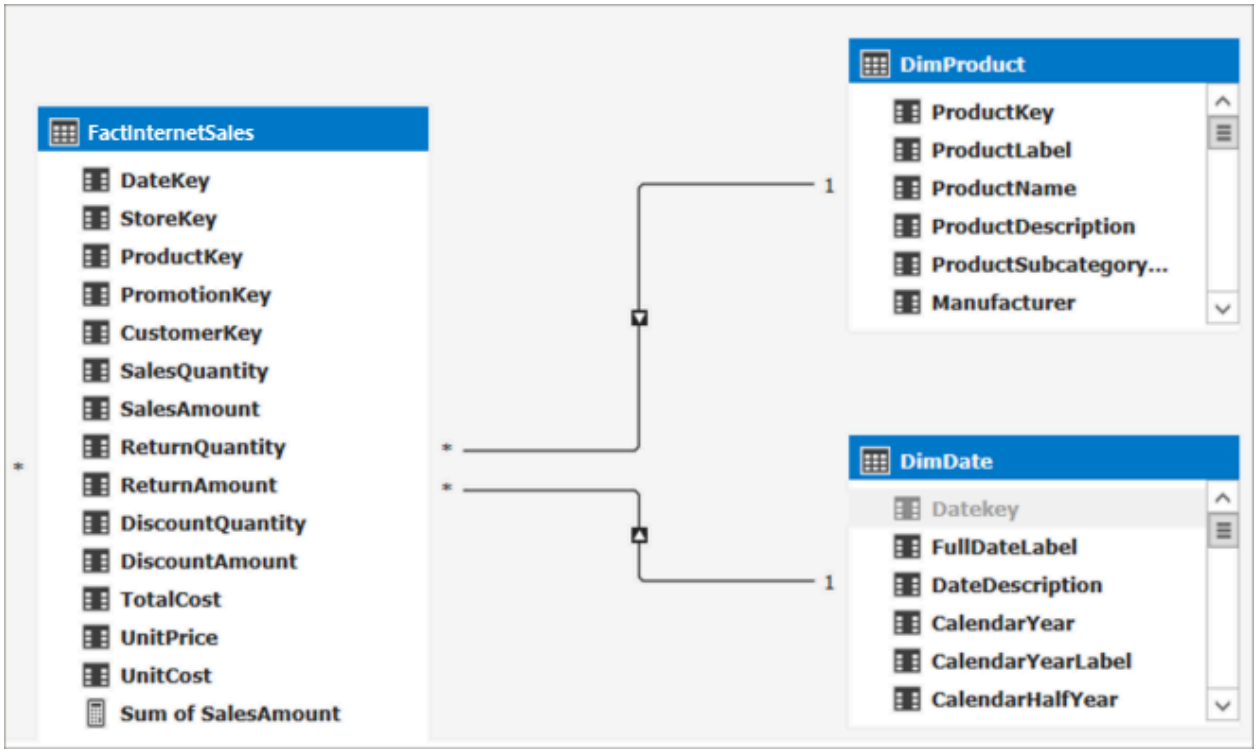
The function returns no value; the function only sets the cross-filtering direction for the indicated relationship, for the duration of the query.

Remarks

- In the case of a 1:1 relationship, there is no difference between the one and both direction.
- CROSSFILTER can only be used in functions that take a filter as an argument, for example: CALCULATE, CALCULATETABLE, CLOSINGBALANCEMONTH, CLOSINGBALANCEQUARTER, CLOSINGBALANCEYEAR, OPENINGBALANCEMONTH, OPENINGBALANCEQUARTER, OPENINGBALANCEYEAR, TOTALMTD, TOTALQTD and TOTALYTD functions.
- CROSSFILTER uses existing relationships in the model, identifying relationships by their ending point columns.
- In CROSSFILTER, the cross-filtering setting of a relationship is not important; that is, whether the relationship is set to filter one, or both directions in the model does not affect the usage of the function. CROSSFILTER will override any existing cross-filtering setting.
- An error is returned if any of the columns named as an argument is not part of a relationship or the arguments belong to different relationships.
- If CALCULATE expressions are nested, and more than one CALCULATE expression contains a CROSSFILTER function, then the innermost CROSSFILTER is the one that prevails in case of a conflict or ambiguity.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the following model diagram, both DimProduct and DimDate have a single direction relationship with FactInternetSales.



By default, we cannot get the Count of Products sold by year:

Row Labels	Sum of SalesAmount	Distinct Count of ProductKey
2005	$3,266,373.66	606
2006	$6,530,343.53	606
2007	$9,791,060.30	606
2008	$9,770,899.74	606
2009		606
2010		606
Grand Total	**$29,358,677.22**	**606**

There are two ways to get the count of products by year:

- Turn on bi-directional cross-filtering on the relationship. This will change how filters work for all data between these two tables.
- Use the CROSSFILTER function to change how the relationships work for just this measure.

When using DAX, we can use the CROSSFILTER function to change how the cross-filter direction behaves between two columns defined by a relationship. In this case, the DAX expression looks like this:

DAX

```
BiDi:= CALCULATE([Distinct Count of ProductKey],
CROSSFILTER(FactInternetSales[ProductKey], DimProduct[ProductKey] , Both))
```

By using the CROSSFILTER function in our measure expression, we get the expected results:

Row Labels	Sum of SalesAmount	Distinct Count of ProductKey	BiDi
2005	$3,266,373.66	606	25
2006	$6,530,343.53	606	56
2007	$9,791,060.30	606	133
2008	$9,770,899.74	606	102
2009		606	
2010		606	
Grand Total	**$29,358,677.22**	**606**	**606**

Feedback

Was this page helpful? Yes No

RELATED

Article • 10/20/2023

Returns a related value from another table.

Syntax

```DAX
RELATED(<column>)
```

Parameters

Term	Definition
column	The column that contains the values you want to retrieve.

Return value

A single value that is related to the current row.

Remarks

- The RELATED function requires that a relationship exists between the current table and the table with related information. You specify the column that contains the data that you want, and the function follows an existing many-to-one relationship to fetch the value from the specified column in the related table. If a relationship does not exist, you must create a relationship.
- When the RELATED function performs a lookup, it examines all values in the specified table regardless of any filters that may have been applied.
- The RELATED function needs a row context; therefore, it can only be used in calculated column expression, where the current row context is unambiguous, or as a nested function in an expression that uses a table scanning function. A table scanning function, such as SUMX, gets the value of the current row value and then scans another table for instances of that value.

- The RELATED function cannot be used to fetch a column across a limited relationship.

Example

In the following example, the measure Non USA Internet Sales is created to produce a sales report that excludes sales in the United States. In order to create the measure, the InternetSales_USD table must be filtered to exclude all sales that belong to the United States in the SalesTerritory table. The United States, as a country, appears 5 times in the SalesTerritory table; once for each of the following regions: Northwest, Northeast, Central, Southwest, and Southeast.

The first approach to filter the Internet Sales, in order to create the measure, could be to add a filter expression like the following:

DAX

```
FILTER('InternetSales_USD'
, 'InternetSales_USD'[SalesTerritoryKey]<>1 &&
'InternetSales_USD'[SalesTerritoryKey]<>2 &&
'InternetSales_USD'[SalesTerritoryKey]<>3 &&
'InternetSales_USD'[SalesTerritoryKey]<>4 &&
'InternetSales_USD'[SalesTerritoryKey]<>5)
```

However, this approach is counterintuitive, prone to typing errors, and might not work if any of the existing regions is split in the future.

A better approach would be to use the existing relationship between InternetSales_USD and SalesTerritory and explicitly state that the country must be different from the United States. To do so, create a filter expression like the following:

DAX

```
FILTER( 'InternetSales_USD',
RELATED('SalesTerritory'[SalesTerritoryCountry])<>"United States")
```

This expression uses the RELATED function to lookup the country value in the SalesTerritory table, starting with the value of the key column, SalesTerritoryKey, in the InternetSales_USD table. The result of the lookup is used by the filter function to determine if the InternetSales_USD row is filtered or not.

Note

If the example does not work, you might need to create a relationship between the tables.

DAX

```
= SUMX(FILTER( 'InternetSales_USD'
            ,  RELATED('SalesTerritory'[SalesTerritoryCountry])
               <>"United States"
             )
     ,'InternetSales_USD'[SalesAmount_USD])
```

The following table shows only totals for each region, to prove that the filter expression in the measure, Non USA Internet Sales, works as intended.

Row Labels	Internet Sales	Non USA Internet Sales
Australia	$4,999,021.84	$4,999,021.84
Canada	$1,343,109.10	$1,343,109.10
France	$2,490,944.57	$2,490,944.57
Germany	$2,775,195.60	$2,775,195.60
United Kingdom	$5,057,076.55	$5,057,076.55
United States	$9,389,479.79	
Grand Total	$26,054,827.45	$16,665,347.67

The following shows what that you might get if you used this measure in a report table visual:

Row Labels	Accessories	Bikes	Clothing	Grand Total
2005		$1,526,481.95		$1,526,481.95
2006		$3,554,744.04		$3,554,744.04
2007	$156,480.18	$5,640,106.05	$70,142.77	$5,866,729.00
2008	$228,159.45	$5,386,558.19	$102,675.04	$5,717,392.68
Grand Total	$384,639.63	$16,107,890.23	$172,817.81	$16,665,347.67

See also

RELATEDTABLE
Filter functions

Feedback

Was this page helpful? Yes No

RELATEDTABLE

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Evaluates a table expression in a context modified by the given filters.

Syntax

DAX

```
RELATEDTABLE(<tableName>)
```

Parameters

Expand table

Term	Definition
tableName	The name of an existing table using standard DAX syntax. It cannot be an expression.

Return value

A table of values.

Remarks

- The RELATEDTABLE function changes the context in which the data is filtered, and evaluates the expression in the new context that you specify.
- This function is a shortcut for CALCULATETABLE function with no logical expression.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example uses the RELATEDTABLE function to create a calculated column with the Internet Sales in the Product Category table:

```
DAX
= SUMX( RELATEDTABLE('InternetSales_USD')
     , [SalesAmount_USD])
```

The following table shows the results:

Expand table

Product Category Key	Product Category AlternateKey	Product Category Name	Internet Sales
1	1	Bikes	$28,318,144.65
2	2	Components	
3	3	Clothing	$339,772.61
4	4	Accessories	$700,759.96

Related content

CALCULATETABLE
Filter functions

Feedback

Was this page helpful? Yes No

USERELATIONSHIP

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Specifies the relationship to be used in a specific calculation as the one that exists between columnName1 and columnName2.

Syntax

DAX

```
USERELATIONSHIP(<columnName1>,<columnName2>)
```

Parameters

Expand table

Term	Definition
`columnName1`	The name of an existing column, using standard DAX syntax and fully qualified, that usually represents the many side of the relationship to be used; if the arguments are given in reverse order the function will swap them before using them. This argument cannot be an expression.
`columnName2`	The name of an existing column, using standard DAX syntax and fully qualified, that usually represents the one side or lookup side of the relationship to be used; if the arguments are given in reverse order the function will swap them before using them. This argument cannot be an expression.

Return value

The function returns no value; the function only enables the indicated relationship for the duration of the calculation.

Remarks

- USERELATIONSHIP can only be used in functions that take a filter as an argument, for example: CALCULATE, CALCULATETABLE, CLOSINGBALANCEWEEK, CLOSINGBALANCEMONTH, CLOSINGBALANCEQUARTER, CLOSINGBALANCEYEAR,

OPENINGBALANCEWEEK, OPENINGBALANCEMONTH, OPENINGBALANCEQUARTER, OPENINGBALANCEYEAR, TOTALWTD, TOTALMTD, TOTALQTD and TOTALYTD functions.

- USERELATIONSHIP cannot be used when row level security is defined for the table in which the measure is included. For example, `CALCULATE(SUM([SalesAmount]), USERELATIONSHIP(FactInternetSales[CustomerKey], DimCustomer[CustomerKey]))` will return an error if row level security is defined for DimCustomer.
- USERELATIONSHIP uses existing relationships in the model, identifying relationships by their ending point columns.
- In USERELATIONSHIP, the status of a relationship is not important; that is, whether the relationship is active or not does not affect the usage of the function. Even if the relationship is inactive, it will be used and overrides any other active relationships that might be present in the model but not mentioned in the function arguments.
- An error is returned if any of the columns named as an argument is not part of a relationship or the arguments belong to different relationships.
- If multiple relationships are needed to join table A to table B in a calculation, each relationship must be indicated in a different USERELATIONSHIP function.
- If CALCULATE expressions are nested, and more than one CALCULATE expression contains a USERELATIONSHIP function, then the innermost USERELATIONSHIP is the one that prevails in case of a conflict or ambiguity.
- Up to 10 USERELATIONSHIP functions can be nested; however, your expression might have a deeper level of nesting, ie. the following sample expression is nested 3 levels deep but only 2 for USERELATIONSHIP: `=CALCULATE(CALCULATE( CALCULATE( <anyExpression>, USERELATIONSHIP( t1[colA], t2[colB])), t99[colZ]=999), USERELATIONSHIP( t1[colA], t2[colA]))`.
- For 1-to-1 relationships, USERELATIONSHIP will only activate the relationship in one direction. In particular, filters will only be able to flow from *columnName2*'s table to *columnName1*'s table. If bi-directional cross-filtering is desired, two USERELATIONSHIPs with opposite directionality can be used in the same calculation. For example, `CALCULATE(..., USERELATIONSHIP(T1[K], T2[K]), USERELATIONSHIP(T2[K], T1[K]))`.

Example

The following sample shows how to override the default, active, relationship between InternetSales and DateTime tables. The default relationship exists between the OrderDate column, in the InternetSales table, and the Date column, in the DateTime table.

To calculate the sum of internet sales and allow slicing by ShippingDate instead of the traditional OrderDate, create measure, [InternetSales by ShippingDate] using the following expression:

```
DAX
```

```
= CALCULATE(SUM(InternetSales[SalesAmount]),
USERELATIONSHIP(InternetSales[ShippingDate], DateTime[Date]))
```

Relationships between InternetSales[ShippingDate] and DateTime[Date] must exist and should not be the active relationship; also, the relationship between InternetSales[OrderDate] and DateTime[Date] should exist and should be the active relationship.

Statistical functions

Article • 10/20/2023

Statistical functions calculate values related to statistical distributions and probability, such as standard deviation and number of permutations.

In this category

Function	Description
BETA.DIST	Returns the beta distribution.
BETA.INV	Returns the inverse of the beta cumulative probability density function (BETA.DIST).
CHISQ.DIST	Returns the chi-squared distribution.
CHISQ.DIST.RT	Returns the right-tailed probability of the chi-squared distribution.
CHISQ.INV	Returns the inverse of the left-tailed probability of the chi-squared distribution.
CHISQ.INV.RT	Returns the inverse of the right-tailed probability of the chi-squared distribution.
COMBIN	Returns the number of combinations for a given number of items.
COMBINA	Returns the number of combinations (with repetitions) for a given number of items.
CONFIDENCE.NORM	The confidence interval is a range of values.
CONFIDENCE.T	Returns the confidence interval for a population mean, using a Student's t distribution.
EXPON.DIST	Returns the exponential distribution.
GEOMEAN	Returns the geometric mean of the numbers in a column.
GEOMEANX	Returns the geometric mean of an expression evaluated for each row in a table.
LINEST	Uses the Least Squares method to calculate a straight line that best fits the given data.
LINESTX	Uses the Least Squares method to calculate a straight line that best fits the given data. The data result from expressions evaluated for each row in a table.

Function	Description
MEDIAN	Returns the median of numbers in a column.
MEDIANX	Returns the median number of an expression evaluated for each row in a table.
NORM.DIST	Returns the normal distribution for the specified mean and standard deviation.
NORM.INV	The inverse of the normal cumulative distribution for the specified mean and standard deviation.
NORM.S.DIST	Returns the standard normal distribution (has a mean of zero and a standard deviation of one).
NORM.S.INV	Returns the inverse of the standard normal cumulative distribution.
PERCENTILE.EXC	Returns the k-th percentile of values in a range, where k is in the range 0..1, exclusive.
PERCENTILE.INC	Returns the k-th percentile of values in a range, where k is in the range 0..1, inclusive.
PERCENTILEX.EXC	Returns the percentile number of an expression evaluated for each row in a table.
PERCENTILEX.INC	Returns the percentile number of an expression evaluated for each row in a table.
PERMUT	Returns the number of permutations for a given number of objects that can be selected from number objects.
POISSON.DIST	Returns the Poisson distribution.
RANK.EQ	Returns the ranking of a number in a list of numbers.
RANKX	Returns the ranking of a number in a list of numbers for each row in the *table* argument.
SAMPLE	Returns a sample of N rows from the specified table.
STDEV.P	Returns the standard deviation of the entire population.
STDEV.S	Returns the standard deviation of a sample population.
STDEVX.P	Returns the standard deviation of the entire population.
STDEVX.S	Returns the standard deviation of a sample population.
T.DIST	Returns the Student's left-tailed t-distribution.
T.DIST.2T	Returns the two-tailed Student's t-distribution.

Function	Description
T.DIST.RT	Returns the right-tailed Student's t-distribution.
T.INV	Returns the left-tailed inverse of the Student's t-distribution.
T.INV.2t	Returns the two-tailed inverse of the Student's t-distribution.
VAR.P	Returns the variance of the entire population.
VAR.S	Returns the variance of a sample population.
VARX.P	Returns the variance of the entire population.
VARX.S	Returns the variance of a sample population.

Feedback

Was this page helpful? Yes No

BETA.DIST

Article • 12/13/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the beta distribution. The beta distribution is commonly used to study variation in the percentage of something across samples, such as the fraction of the day people spend watching television.

Syntax

```DAX
BETA.DIST(x,alpha,beta,cumulative,[A],[B])
```

Parameters

Expand table

Term	Definition
`x`	The value between A and B at which to evaluate the function
`Alpha`	A parameter of the distribution.
`Beta`	A parameter of the distribution.
`A`	Optional. A lower bound to the interval of x.
`B`	Optional. An upper bound to the interval of x.

Return value

Returns the beta distribution.

Remarks

- If any argument is nonnumeric, BETA.DIST returns the `#VALUE!` error value.
- If any argument is not an integer, it is rounded.

- If `alpha ≤ 0` or beta ≤ 0, BETA.DIST returns the `#NUM!` error value.
- If x < A, x > B, or A = B, BETA.DIST returns the `#NUM!` error value.
- If you omit values for A and B, BETA.DIST uses the standard cumulative beta distribution, so that A = 0 and B = 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

BETA.INV

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse of the beta cumulative probability density function (BETA.DIST).

If probability = BETA.DIST(x,...TRUE), then BETA.INV(probability,...) = x. The beta distribution can be used in project planning to model probable completion times given an expected completion time and variability.

Syntax

DAX

```
BETA.INV(probability,alpha,beta,[A],[B])
```

Parameters

Expand table

Term	Definition
Probability	A probability associated with the beta distribution.
Alpha	A parameter of the distribution.
Beta	A parameter the distribution.
A	Optional. A lower bound to the interval of x.
B	Optional. An upper bound to the interval of x.

Return value

Returns the inverse of the beta cumulative probability density function (BETA.DIST).

Remarks

- If any argument is nonnumeric, BETA.INV returns the #VALUE! error value.

- If any argument is not an integer, it is rounded.
- If alpha ≤ 0 or beta ≤ 0, BETA.INV returns the #NUM! error value.
- If probability ≤ 0 or probability > 1, BETA.INV returns the #NUM! error value.
- If you omit values for A and B, BETA.INV uses the standard cumulative beta distribution, so that A = 0 and B = 1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

CHISQ.DIST

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the chi-squared distribution.

The chi-squared distribution is commonly used to study variation in the percentage of something across samples, such as the fraction of the day people spend watching television.

Syntax

DAX

```
CHISQ.DIST(<x>, <deg_freedom>, <cumulative>)
```

Parameters

Expand table

Term	Definition
x	The value at which you want to evaluate the distribution.
Deg_freedom	The number of degrees of freedom.
cumulative	A logical value that determines the form of the function. If cumulative is TRUE, CHISQ.DIST returns the cumulative distribution function; if FALSE, it returns the probability density function.

Return value

The chi-squared distribution.

Remarks

- If x or deg_freedom is nonnumeric, an error is returned.
- If deg_freedom is not an integer, it is rounded.

- If x < 0, an error is returned.
- If deg_freedom < 1 or deg_freedom > 10^10, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query,

```
DAX
EVALUATE { CHISQ.DIST(2, 2, TRUE) }
```

Returns

Expand table

[Value]
0.632120558828558

Feedback

Was this page helpful? Yes No

CHISQ.DIST.RT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the right-tailed probability of the chi-squared distribution.

The chi-squared distribution is associated with a chi-squared test. Use the chi-squared test to compare observed and expected values. For example, a genetic experiment might hypothesize that the next generation of plants will exhibit a certain set of colors. By comparing the observed results with the expected ones, you can decide whether your original hypothesis is valid.

Syntax

DAX

```
CHISQ.DIST.RT(<x>, <deg_freedom>)
```

Parameters

Expand table

Term	Definition
x	The value at which you want to evaluate the distribution.
Deg_freedom	The number of degrees of freedom.

Return value

The right-tailed probability of the chi-squared distribution.

Remarks

- If x or deg_freedom is nonnumeric, an error is returned.
- If deg_freedom is not an integer, it is rounded.

- If x < 0, an error is returned.
- If deg_freedom < 1 or deg_freedom > 10^10, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query,

```
DAX
EVALUATE { CHISQ.DIST.RT(2, 5) }
```

Returns

Expand table

[Value]
0.84914503608461

Feedback

Was this page helpful? Yes No

CHISQ.INV

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the inverse of the left-tailed probability of the chi-squared distribution.

The chi-squared distribution is commonly used to study variation in the percentage of something across samples, such as the fraction of the day people spend watching television.

Syntax

```DAX
CHISQ.INV(probability,deg_freedom)
```

Parameters

Expand table

Term	Definition
Probability	A probability associated with the chi-squared distribution.
Deg_freedom	The number of degrees of freedom.

Return value

Returns the inverse of the left-tailed probability of the chi-squared distribution.

Remarks

- If argument is nonnumeric, CHISQ.INV returns the #VALUE! error value.
- If probability < 0 or probability > 1, CHISQ.INV returns the #NUM! error value.
- If deg_freedom is not an integer, it is rounded.

- If deg_freedom < 0 or deg_freedom > 10^10, CHISQ.INV returns the #NUM! error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= CHISQ.INV(0.93,1)	Inverse of the left-tailed probability of the chi-squared distribution for 0.93, using 1 degree of freedom.	5.318520074
= CHISQ.INV(0.6,2)	Inverse of the left-tailed probability of the chi-squared distribution for 0.6, using 2 degrees of freedom.	1.832581464

Feedback

Was this page helpful? Yes No

CHISQ.INV.RT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse of the right-tailed probability of the chi-squared distribution.

If probability = CHISQ.DIST.RT(x,...), then CHISQ.INV.RT(probability,...) = x. Use this function to compare observed results with expected ones in order to decide whether your original hypothesis is valid.

Syntax

```DAX
CHISQ.INV.RT(probability,deg_freedom)
```

Parameters

Expand table

Term	Definition
Probability	A probability associated with the chi-squared distribution.
Deg_freedom	The number of degrees of freedom.

Return value

Returns the inverse of the right-tailed probability of the chi-squared distribution.

Remarks

- If either argument is nonnumeric, CHISQ.INV.RT returns the #VALUE! error value.
- If probability < 0 or probability > 1, CHISQ.INV.RT returns the #NUM! error value.
- If deg_freedom is not an integer, it is rounded.
- If deg_freedom < 1, CHISQ.INV.RT returns the #NUM! error value.

- Given a value for probability, CHISQ.INV.RT seeks that value x such that CHISQ.DIST.RT(x, deg_freedom) = probability. Thus, precision of CHISQ.INV.RT depends on precision of CHISQ.DIST.RT. CHISQ.INV.RT uses an iterative search technique. If the search has not converged after 64 iterations, the function returns the #N/A error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

COMBIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of combinations for a given number of items. Use COMBIN to determine the total possible number of groups for a given number of items.

Syntax

DAX

```
COMBIN(number, number_chosen)
```

Parameters

Expand table

Term	Definition
number	The number of items.
number_chosen	The number of items in each combination.

Return value

Returns the number of combinations for a given number of items.

Remarks

- Numeric arguments are truncated to integers.
- If either argument is nonnumeric, COMBIN returns the #VALUE! error value.
- If number < 0, number_chosen < 0, or number < number_chosen, COMBIN returns the #NUM! error value.
- A combination is any set or subset of items, regardless of their internal order. Combinations are distinct from permutations, for which the internal order is

significant.

- The number of combinations is as follows, where number = n and number_chosen = k:

$$\binom{n}{k} = \frac{P_{k,n}}{k!} = \frac{n!}{k!(n-k)!}$$

Where

$$P_{k,n} = \frac{n!}{(n-k)!}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= COMBIN(8,2)	Possible two-person teams that can be formed from 8 candidates.	28

Feedback

Was this page helpful? Yes No

COMBINA

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of combinations (with repetitions) for a given number of items.

Syntax

```
DAX
COMBINA(number, number_chosen)
```

Parameters

Expand table

Term	Definition
number	Must be greater than or equal to 0, and greater than or equal to Number_chosen. Non-integer values are truncated.
number_chosen	Must be greater than or equal to 0. Non-integer values are truncated.

Return value

Returns the number of combinations (with repetitions) for a given number of items.

Remarks

- If the value of either argument is outside of its constraints, COMBINA returns the #NUM! error value.
- If either argument is a non-numeric value, COMBINA returns the #VALUE! error value.
- The following equation is used, where N is Number and M is Number_chosen:

$$\binom{N+M-1}{N-1}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= COMBINA(4,3)	Returns the number of combinations (with repetitions) for 4 and 3.	20
= COMBINA(10,3)	Returns the number of combinations (with repetitions) for 10 and 3.	220

Feedback

Was this page helpful? Yes No

CONFIDENCE.NORM

Article • 10/20/2023

The confidence interval is a range of values. Your sample mean, x, is at the center of this range and the range is x ± CONFIDENCE.NORM. For example, if x is the sample mean of delivery times for products ordered through the mail, x ± CONFIDENCE.NORM is a range of population means. For any population mean, μ0, in this range, the probability of obtaining a sample mean further from μ0 than x is greater than alpha; for any population mean, μ0, not in this range, the probability of obtaining a sample mean further from μ0 than x is less than alpha. In other words, assume that we use x, standard_dev, and size to construct a two-tailed test at significance level alpha of the hypothesis that the population mean is μ0. Then we will not reject that hypothesis if μ0 is in the confidence interval and will reject that hypothesis if μ0 is not in the confidence interval. The confidence interval does not allow us to infer that there is probability 1 – alpha that our next package will take a delivery time that is in the confidence interval.

Syntax

DAX

```
CONFIDENCE.NORM(alpha,standard_dev,size)
```

Parameters

Term	Definition
alpha	The significance level used to compute the confidence level. The confidence level equals 100*(1 - alpha)%, or in other words, an alpha of 0.05 indicates a 95 percent confidence level.
standard_dev	The population standard deviation for the data range and is assumed to be known.
standard_dev,size	The sample size.

Return value

A range of values

Remarks

- If any argument is nonnumeric, CONFIDENCE.NORM returns the #VALUE! error value.
- If alpha ≤ 0 or alpha ≥ 1, CONFIDENCE.NORM returns the #NUM! error value.
- If standard_dev ≤ 0, CONFIDENCE.NORM returns the #NUM! error value.
- If size is not an integer, it is rounded.
- If size < 1, CONFIDENCE.NORM returns the #NUM! error value.
- If we assume alpha equals 0.05, we need to calculate the area under the standard normal curve that equals (1 - alpha), or 95 percent. This value is ± 1.96. The confidence interval is therefore:

$$\overline{x} \pm 1.96\left(\frac{\sigma}{\sqrt{n}}\right)$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

CONFIDENCE.T

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the confidence interval for a population mean, using a Student's t distribution.

Syntax

DAX

```
CONFIDENCE.T(alpha,standard_dev,size)
```

Parameters

Expand table

Term	Definition
alpha	The significance level used to compute the confidence level. The confidence level equals 100*(1 - alpha)%, or in other words, an alpha of 0.05 indicates a 95 percent confidence level.
standard_dev	The population standard deviation for the data range and is assumed to be known.
size	The sample size.

Return value

Returns the confidence interval for a population mean, using a Student's t distribution.

Remarks

- If any argument is nonnumeric, CONFIDENCE.T returns the #VALUE! error value.
- If alpha ≤ 0 or alpha ≥ 1, CONFIDENCE.T returns the #NUM! error value.
- If standard_dev ≤ 0, CONFIDENCE.T returns the #NUM! error value.

- If size is not an integer, it is rounded.
- If size equals 1, CONFIDENCE.T returns #DIV/0! error value.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Expand table

Formula	Description	Result
= CONFIDENCE.T(0.05,1,50)	Confidence interval for the mean of a population based on a sample size of 50, with a 5% significance level and a standard deviation of 1. This is based on a Student's t-distribution.	0.284196855

Feedback

Was this page helpful? Yes No

EXPON.DIST

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the exponential distribution. Use EXPON.DIST to model the time between events, such as how long an automated bank teller takes to deliver cash. For example, you can use EXPON.DIST to determine the probability that the process takes at most 1 minute.

Syntax

DAX

```
EXPON.DIST(x,lambda,cumulative)
```

Parameters

Expand table

Term	Definition
x	Required. The value of the function.
lambda	Required. The parameter value.
cumulative	Required. A logical value that indicates which form of the exponential function to provide. If cumulative is TRUE, EXPON.DIST returns the cumulative distribution function; if FALSE, it returns the probability density function.

Return value

Returns the exponential distribution.

Remarks

- If x or lambda is nonnumeric, EXPON.DIST returns the #VALUE! error value.
- If x or lambda is not an integer, it is rounded.

- If x < 0, EXPON.DIST returns the #NUM! error value.
- If lambda ≤ 0, EXPON.DIST returns the #NUM! error value.
- The equation for the probability density function is:

$$f(x;\lambda) = \lambda e^{-\lambda x}$$

- The equation for the cumulative distribution function is:

$$F(x;\lambda) = 1 - e^{-\lambda x}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

GEOMEAN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the geometric mean of the numbers in a column.

To return the geometric mean of an expression evaluated for each row in a table, use GEOMEANX function.

Syntax

DAX

```
GEOMEAN(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the numbers for which the geometric mean is to be computed.

Return value

A decimal number.

Remarks

- Only the numbers in the column are counted. Blanks, logical values, and text are ignored.
- GEOMEAN(Table[Column]) is equivalent to GEOMEANX(Table, Table[Column])
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the geometric mean of the Return column in the Investment table:

```
DAX
= GEOMEAN( Investment[Return] )
```

Related content

GEOMEANX function

Feedback

Was this page helpful? Yes No

GEOMEANX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the geometric mean of an expression evaluated for each row in a table.

To return the geometric mean of the numbers in a column, use GEOMEAN function.

Syntax

```
DAX

GEOMEANX(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.

Return value

A decimal number.

Remarks

- The GEOMEANX function takes as its first argument a table, or an expression that returns a table. The second argument is a column that contains the numbers for which you want to compute the geometric mean, or an expression that evaluates to a column.
- Only the numbers in the column are counted. Blanks, logical values, and text are ignored.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the geometric mean of the ReturnPct column in the Investments table:

```
DAX
= GEOMEANX( Investments, Investments[ReturnPct] + 1 )
```

Related content

GEOMEAN function

Feedback

Was this page helpful? Yes No

LINEST

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Uses the Least Squares method to calculate a straight line that best fits the given data, then returns a table describing the line. The equation for the line is of the form: y = **Slope1***x1 + **Slope2***x2 + ... + **Intercept**.

Syntax

```
DAX

LINEST ( <columnY>, <columnX>[, …][, <const>] )
```

Parameters

Expand table

Term	Definition
`columnY`	The column of known y-values. Must have scalar type.
`columnX`	The columns of known x-values. Must have scalar type. At least one must be provided.
`const`	(Optional) A constant `TRUE`/`FALSE` value specifying whether to force the constant **Intercept** to equal 0. If `TRUE` or omitted, the **Intercept** value is calculated normally; If `FALSE`, the **Intercept** value is set to zero.

Return value

A single-row table describing the line, plus additional statistics. These are the available columns:

- **Slope1**, **Slope2**, ..., **SlopeN**: the coefficients corresponding to each x-value;
- **Intercept**: intercept value;
- **StandardErrorSlope1**, **StandardErrorSlope2**, ..., **StandardErrorSlopeN**: the standard error values for the coefficients **Slope1**, **Slope2**, ..., **SlopeN**;
- **StandardErrorIntercept**: the standard error value for the constant **Intercept**;
- **CoefficientOfDetermination**: the coefficient of determination (r^2). Compares estimated and actual y-values, and ranges in value from 0 to 1: the higher the value, the higher the correlation in the sample;
- **StandardError**: the standard error for the y estimate;
- **FStatistic**: the F statistic, or the F-observed value. Use the F statistic to determine whether the observed relationship between the dependent and independent variables occurs by chance;
- **DegreesOfFreedom**: the degrees of freedom. Use this value to help you find F-critical values in a statistical table, and determine a confidence level for the model;
- **RegressionSumOfSquares**: the regression sum of squares;
- **ResidualSumOfSquares**: the residual sum of squares.

Remarks

`columnY` and the `columnX`'s must all belong to the same table.

Example 1

The following DAX query:

```
DAX

EVALUATE LINEST(
    'FactInternetSales'[SalesAmount],
    'FactInternetSales'[TotalProductCost]
)
```

Returns a single-row table with ten columns:

Expand table

Slope1	Intercept	StandardErrorSlope1	StandardErrorIntercept	CoefficientOfDetermination
1.67703250456677	6.34550460373026	0.000448675725548806	0.279131821917317	0.995695557281456

Expand table

StandardError	FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
60.9171030357485	13970688.6139993	60396	51843736761.658	224123120.339218

- **Slope1** and **Intercept**: the coefficients of the calculated linear model;
- **StandardErrorSlope1** and **StandardErrorIntercept**: the standard error values for the coefficients above;
- **CoefficientOfDetermination**, **StandardError**, **FStatistic**, **DegreesOfFreedom**, **RegressionSumOfSquares** and **ResidualSumOfSquares**: regression statistics about the model.

For a given internet sale, this model predicts the sale amount by the following formula:

```
SalesAmount = Slope1 * TotalProductCost + Intercept
```

Example 2

The following DAX query:

```
DAX

EVALUATE LINEST(
    'DimCustomer'[TotalSalesAmount],
    'DimCustomer'[YearlyIncome],
    'DimCustomer'[TotalChildren],
```

```
    'DimCustomer'[BirthDate]
)
```

Returns a single-row table with fourteen columns:

- Slope1
- Slope2
- Slope3
- Intercept
- StandardErrorSlope1
- StandardErrorSlope2
- StandardErrorSlope3
- StandardErrorIntercept
- CoefficientOfDetermination
- StandardError
- FStatistic
- DegreesOfFreedom
- RegressionSumOfSquares
- ResidualSumOfSquares

For a given customer, this model predicts total sales by the following formula (the birth date is automatically converted to a number):

```
TotalSalesAmount = Slope1 * YearlyIncome + Slope2 * TotalChildren + Slope3 * BirthDate +
Intercept
```

Related content

LINESTX Statistical functions

Feedback

Was this page helpful? Yes No

LINESTX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Uses the Least Squares method to calculate a straight line that best fits the given data, then returns a table describing the line. The data result from expressions evaluated for each row in a table. The equation for the line is of the form: y = **Slope1***x1 + **Slope2***x2 + ... + **Intercept**.

Syntax

```
DAX

LINESTX ( <table>, <expressionY>, <expressionX>[, …][, <const>] )
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expressions will be evaluated.
expressionY	The expression to be evaluated for each row of the table, to obtain the known y-values. Must have scalar type.
expressionX	The expressions to be evaluated for each row of the table, to obtain the known x-values. Must have scalar type. At least one must be provided.
const	(Optional) A constant TRUE/FALSE value specifying whether to force the constant **Intercept** to equal 0. If TRUE or omitted, the **Intercept** value is calculated normally; If FALSE, the **Intercept** value is set to zero.

Return value

A single-row table describing the line, plus additional statistics. These are the available columns:

- **Slope1**, **Slope2**, ..., **SlopeN**: the coefficients corresponding to each x-value;
- **Intercept**: intercept value;
- **StandardErrorSlope1**, **StandardErrorSlope2**, ..., **StandardErrorSlopeN**: the standard error values for the coefficients **Slope1**, **Slope2**, ..., **SlopeN**;
- **StandardErrorIntercept**: the standard error value for the constant **Intercept**;
- **CoefficientOfDetermination**: the coefficient of determination (r^2). Compares estimated and actual y-values, and ranges in value from 0 to 1: the higher the value, the higher the

correlation in the sample;

- **StandardError**: the standard error for the y estimate;
- **FStatistic**: the F statistic, or the F-observed value. Use the F statistic to determine whether the observed relationship between the dependent and independent variables occurs by chance;
- **DegreesOfFreedom**: the degrees of freedom. Use this value to help you find F-critical values in a statistical table, and determine a confidence level for the model;
- **RegressionSumOfSquares**: the regression sum of squares;
- **ResidualSumOfSquares**: the residual sum of squares.

Example 1

The following DAX query:

```
DAX

DEFINE VAR TotalSalesByRegion = SUMMARIZECOLUMNS(
    'Sales Territory'[Sales Territory Key],
    'Sales Territory'[Population],
    "Total Sales", SUM(Sales[Sales Amount])
)
EVALUATE LINESTX(
    'TotalSalesByRegion',
    [Total Sales],
    [Population]
)
```

Returns a single-row table with ten columns:

Expand table

Slope1	Intercept	StandardErrorSlope1	StandardErrorIntercept	CoefficientOfDetermination
6.42271517588	-410592.76216	0.24959467764561	307826.343996223	0.973535860750193

Expand table

StandardError	FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
630758.1747292	662.165707642	18	263446517001130	7161405749781.07

- **Slope1** and **Intercept**: the coefficients of the calculated linear model;
- **StandardErrorSlope1** and **StandardErrorIntercept**: the standard error values for the coefficients above;
- **CoefficientOfDetermination**, **StandardError**, **FStatistic**, **DegreesOfFreedom**, **RegressionSumOfSquares** and **ResidualSumOfSquares**: regression statistics about the model.

For a given sales territory, this model predicts total sales by the following formula:

```
Total Sales = Slope1 * Population + Intercept
```

Example 2

The following DAX query:

```DAX
DEFINE VAR TotalSalesByCustomer = SUMMARIZECOLUMNS(
    'Customer'[Customer ID],
    'Customer'[Age],
    'Customer'[NumOfChildren],
    "Total Sales", SUM(Sales[Sales Amount])
)
EVALUATE LINESTX(
    'TotalSalesByCustomer',
    [Total Sales],
    [Age],
    [NumOfChildren]
)
```

Returns a single-row table with twelve columns:

Expand table

Slope1	Slope2	Intercept	StandardErrorSlope1
69.0435458093763	33.005949841721	-871.118539339539	0.872588875481658

Expand table

StandardErrorSlope2	StandardErrorIntercept	CoefficientOfDetermination	StandardError
6.21158863903435	26.726292527427	0.984892920482022	68.5715034014342

Expand table

FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
3161.91535144391	97	29734974.9782379	456098.954637092

For a given customer, this model predicts total sales by the following formula:

```
```

```
Total Sales = Slope1 * Age + Slope2 * NumOfChildren + Intercept
```

Related content

LINEST
Statistical functions

Feedback

Was this page helpful? Yes No

MEDIAN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the median of numbers in a column.

To return the median of an expresssion evaluated for each row in a table, use MEDIANX function.

Syntax

DAX

```
MEDIAN(<column>)
```

Parameters

Expand table

Term	Definition
column	The column that contains the numbers for which the median is to be computed.

Return value

A decimal number.

Remarks

- Only the numbers in the column are counted. Blanks are ignored. Logical values, dates, and text are not supported.
- MEDIAN(Table[Column]) is equivalent to MEDIANX(Table, Table[Column]).
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the median of a column named Age in a table named Customers:

```
DAX
= MEDIAN( Customers[Age] )
```

Related content

MEDIANX function

Feedback

Was this page helpful? Yes No

MEDIANX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the median number of an expression evaluated for each row in a table.

To return the median of numbers in a column, use MEDIAN function.

Syntax

```DAX
MEDIANX(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.

Return value

A decimal number.

Remarks

- The MEDIANX function takes as its first argument a table, or an expression that returns a table. The second argument is a column that contains the numbers for which you want to compute the median, or an expression that evaluates to a column.
- Only the numbers in the column are counted.
- Logical values and text are ignored.

- MEDIANX does not ignore blanks; however, MEDIAN does ignore blanks
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following computes the median age of customers who live in the USA.

DAX

```
= MEDIANX( FILTER(Customers, RELATED( Geography[Country]="USA" ) ),
Customers[Age] )
```

Related content

MEDIAN function

Feedback

Was this page helpful? Yes No

NORM.DIST

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the normal distribution for the specified mean and standard deviation.

Syntax

```
DAX
NORM.DIST(X, Mean, Standard_dev, Cumulative)
```

Parameters

Expand table

Term	Definition
`X`	The value for which you want the distribution.
`Mean`	The arithmetic mean of the distribution.
`Standard_dev`	The standard deviation of the distribution.
`Cumulative*`	A logical value that determines the form of the function. If cumulative is `TRUE`, NORM.DIST returns the cumulative distribution function; if `FALSE`, it returns the probability density function.

Return value

The normal distribution for the specified mean and standard deviation.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
EVALUATE { NORM.DIST(42, 40, 1.5, TRUE) }
```

Returns

Expand table

[Value]
0.908788780274132

Related content

NORM.S.DIST function NORM.INV function NORM.S.INV

Feedback

Was this page helpful? Yes No

NORM.INV

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

The inverse of the normal cumulative distribution for the specified mean and standard deviation.

Syntax

DAX

```
NORM.INV(Probability, Mean, Standard_dev)
```

Parameters

Expand table

Term	Definition
`Probability`	A probability corresponding to the normal distribution.
`Mean`	The arithmetic mean of the distribution.
`Standard_dev`	The standard deviation of the distribution.

Return value

Returns the inverse of the normal cumulative distribution for the specified mean and standard deviation.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
EVALUATE { NORM.INV(0.908789, 40, 1.5) }
```

Returns

Expand table

[Value]
42.00000200956628780274132

Related content

NORM.S.INV NORM.S.DIST function NORM.DIST function

Feedback

Was this page helpful? Yes No

NORM.S.DIST

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the standard normal distribution (has a mean of zero and a standard deviation of one).

Syntax

DAX

```
NORM.S.DIST(Z, Cumulative)
```

Parameters

Expand table

Term	Definition
`Z`	The value for which you want the distribution.
`Cumulative`	Cumulative is a logical value that determines the form of the function. If cumulative is `TRUE`, NORM.S.DIST returns the cumulative distribution function; if `FALSE`, it returns the probability density function.

Return value

The standard normal distribution (has a mean of zero and a standard deviation of one.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
EVALUATE { NORM.S.DIST(1.333333, TRUE) }
```

Returns

Expand table

[Value]
0.908788725604095

Related content

NORM.INV function NORM.DIST function NORM.S.INV

Feedback

Was this page helpful? Yes No

NORM.S.INV

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the inverse of the standard normal cumulative distribution. The distribution has a mean of zero and a standard deviation of one.

Syntax

DAX

```
NORM.S.INV(Probability)
```

Parameters

Expand table

Term	Definition
`Probability`	A probability corresponding to the normal distribution.

Return value

The inverse of the standard normal cumulative distribution. The distribution has a mean of zero and a standard deviation of one.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
EVALUATE { NORM.S.INV(0.908789) }
```

Returns

Expand table

[Value]
1.33333467304411

Related content

NORM.INV NORM.S.DIST function NORM.DIST function

Feedback

Was this page helpful? Yes No

PERCENTILE.EXC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the k-th percentile of values in a range, where k is in the range 0..1, exclusive.

To return the percentile number of an expression evaluated for each row in a table, use PERCENTILEX.EXC function.

Syntax

```DAX
PERCENTILE.EXC(<column>, <k>)
```

Parameters

Expand table

Term	Definition
column	A column containing the values that define relative standing.
k	The percentile value in the range 0..1, exclusive.

Return value

The k-th percentile of values in a range, where k is in the range 0..1, exclusive.

Remarks

- If column is empty, BLANK() is returned.
- If k is zero or blank, percentile rank of 1/(n+1) returns the smallest value. If zero, it is out of range and an error is returned.
- If k is nonnumeric or outside the range 0 to 1, an error is returned.

- If k is not a multiple of 1/(n + 1), PERCENTILE.EXC will interpolate to determine the value at the k-th percentile.
- PERCENTILE.EXC will interpolate when the value for the specified percentile is between two values in the array. If it cannot interpolate for the k percentile specified, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

PERCENTILEX.EXC

Feedback

Was this page helpful? Yes No

PERCENTILE.INC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the k-th percentile of values in a range, where k is in the range 0..1, inclusive.

To return the percentile number of an expression evaluated for each row in a table, use PERCENTILEX.INC.

Syntax

```
DAX

PERCENTILE.INC(<column>, <k>)
```

Parameters

Expand table

Term	Definition
column	A column containing the values that define relative standing.
k	The percentile value in the range 0..1, inclusive.

Return value

The k-th percentile of values in a range, where k is in the range 0..1, inclusive.

Remarks

- If column is empty, BLANK() is returned.
- If k is zero or blank, percentile rank of 1/(n+1) returns the smallest value. If zero, it is out of range and an error is returned.
- If k is nonnumeric or outside the range 0 to 1, an error is returned.

- If k is not a multiple of 1/(n + 1), PERCENTILE.INC will interpolate to determine the value at the k-th percentile.
- PERCENTILE.INC will interpolate when the value for the specified percentile is between two values in the array. If it cannot interpolate for the k percentile specified, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

PERCENTILEX.INC

Feedback

Was this page helpful? Yes No

PERCENTILEX.EXC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the percentile number of an expression evaluated for each row in a table.

To return the percentile of numbers in a column, use PERCENTILE.EXC function.

Syntax

DAX

```
PERCENTILEX.EXC(<table>, <expression>, k)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.
k	The desired percentile value in the range 0 to 1 exclusive.

Return value

The percentile number of an expression evaluated for each row in a table.

Remarks

- If k is zero or blank, percentile rank of 1/(n+1) returns the smallest value. If zero, it is out of range and an error is returned.
- If k is nonnumeric or outside the range 0 to 1, an error is returned.
- If k is not a multiple of 1/(n + 1), PERCENTILEX.EXC will interpolate to determine the value at the k-th percentile.

- PERCENTILEX.EXC will interpolate when the value for the specified percentile is between two values in the array. If it cannot interpolate for the k percentile specified, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

PERCENTILE.EXC

Feedback

Was this page helpful? Yes No

PERCENTILEX.INC

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the percentile number of an expression evaluated for each row in a table.

To return the percentile of numbers in a column, use PERCENTILE.INC.

Syntax

DAX

```
PERCENTILEX.INC(<table>, <expression>;, k)
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of the table.
k	The desired percentile value in the range 0 to 1 inclusive.

Return value

The percentile number of an expression evaluated for each row in a table.

Remarks

- If k is zero or blank, percentile rank of 1/(n - 1) returns the smallest value. If zero, it is out of range and an error is returned.
- If k is nonnumeric or outside the range 0 to 1, an error is returned.
- If k is not a multiple of 1/(n - 1), PERCENTILEX.EXC will interpolate to determine the value at the k-th percentile.

- PERCENTILEX.INC will interpolate when the value for the specified percentile is between two values in the array. If it cannot interpolate for the k percentile specified, an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

PERCENTILE.INC

Feedback

Was this page helpful? Yes No

PERMUT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of permutations for a given number of objects that can be selected from number objects. A permutation is any set or subset of objects or events where internal order is significant. Permutations are different from combinations, for which the internal order is not significant. Use this function for lottery-style probability calculations.

Syntax

DAX

```
PERMUT(number, number_chosen)
```

Parameters

Expand table

Term	Definition
number	Required. An integer that describes the number of objects.
number_chosen	Required. An integer that describes the number of objects in each permutation.

Return value

Returns the number of permutations for a given number of objects that can be selected from number objects

Remarks

- Both arguments are truncated to integers.
- If number or number_chosen is nonnumeric, PERMUT returns the #VALUE! error value.

- If number ≤ 0 or if number_chosen < 0, PERMUT returns the #NUM! error value.
- If number < number_chosen, PERMUT returns the #NUM! error value.
- The equation for the number of permutations is:

$$P_{k,n} = \frac{n!}{(n-k)!}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the following formula, permutations possible for a group of 3 objects where 2 are chosen:

```
DAX
= PERMUT(3,2)
```

Result,

6

Feedback

Was this page helpful? Yes No

POISSON.DIST

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the Poisson distribution. A common application of the Poisson distribution is predicting the number of events over a specific time, such as the number of cars arriving at a toll plaza in 1 minute.

Syntax

```DAX
POISSON.DIST(x,mean,cumulative)
```

Parameters

Expand table

Term	Definition
x	Required. The number of events.
mean	Required. The expected numeric value.
cumulative	Required. A logical value that determines the form of the probability distribution returned. If cumulative is TRUE, POISSON.DIST returns the cumulative Poisson probability that the number of random events occurring will be between zero and x inclusive; if FALSE, it returns the Poisson probability mass function that the number of events occurring will be exactly x.

Return value

Returns the Poisson distribution.

Remarks

- If x is not an integer, it is rounded.
- If x or mean is nonnumeric, POISSON.DIST returns the #VALUE! error value.

- If x < 0, POISSON.DIST returns the #NUM! error value.
- If mean < 0, POISSON.DIST returns the #NUM! error value.
- POISSON.DIST is calculated as follows.
  - For cumulative = FALSE:

$$\text{POISSON} = \frac{e^{-\lambda}\lambda^{x}}{x!}$$

  - For cumulative = TRUE:

$$\text{CUMPOISSON} = \sum_{k=0}^{x} \frac{e^{-\lambda}\lambda^{x}}{k!}$$

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

RANK.EQ

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the ranking of a number in a list of numbers.

Syntax

DAX

```
RANK.EQ(<value>, <columnName>[, <order>])
```

Parameters

Expand table

Term	Definition
value	Any DAX expression that returns a single scalar value whose rank is to be found. The expression is to be evaluated exactly once, before the function is evaluated, and it's value passed to the argument list.
columnName	The name of an existing column against which ranks will be determined. It cannot be an expression or a column created using these functions: ADDCOLUMNS, ROW or SUMMARIZE.
order	(Optional) A value that specifies how to rank *number*, low to high or high to low:

order values

Expand table

value	alternate value	Description
0 (zero)	FALSE	Ranks in descending order of *columnName*. If *value* is equal to the highest number in *columnName* then **RANK.EQ** is 1.
1	TRUE	Ranks in ascending order of *columnName*. If *value* is equal to the lowest number in *columnName* then **RANK.EQ** is 1.

Return value

A number indicating the rank of *value* among the numbers in *columnName*.

Remarks

- *columnName* cannot refer to any column created using these functions: ADDCOLUMNS, ROW or SUMMARIZE.I
- If *value* is not in *columnName* or value is a blank, then *RANK.EQ* returns a blank value.
- Duplicate values of *value* receive the same rank value; the next rank value assigned will be the rank value plus the number of duplicate values. For example if five (5) values are tied with a rank of 11 then the next value will receive a rank of 16 (11 + 5).
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following example creates a calculated column that ranks the values in SalesAmount_USD, from the *InternetSales_USD* table, against all numbers in the same column.

DAX

```
= RANK.EQ(InternetSales_USD[SalesAmount_USD],
InternetSales_USD[SalesAmount_USD])
```

Example 2

The following example ranks a subset of values against a given sample. Assume that you have a table of local students with their performance in a specific national test and, also, you have the entire set of scores in that national test. The following calculated column will give you the national ranking for each of the local students.

DAX

```
= RANK.EQ(Students[Test_Score], NationalScores[Test_Score])
```

Feedback

Was this page helpful? Yes No

RANKX

Article • 10/20/2023

Returns the ranking of a number in a list of numbers for each row in the *table* argument.

Syntax

```
DAX
RANKX(<table>, <expression>[, <value>[, <order>[, <ties>]]])
```

Parameters

table
Any DAX expression that returns a table of data over which the expression is evaluated.

expression
Any DAX expression that returns a single scalar value. The expression is evaluated for each row of *table*, to generate all possible values for ranking. See the remarks section to understand the function behavior when *expression* evaluates to BLANK.

value
(Optional) Any DAX expression that returns a single scalar value whose rank is to be found. See the remarks section to understand the function's behavior when *value* is not found in the expression.

When the *value* parameter is omitted, the value of expression at the current row is used instead.

order
(Optional) A value that specifies how to rank *value*, low to high or high to low:

value	alternate value	Description
0 (zero)	FALSE	Ranks in descending order of values of expression. If value is equal to the highest number in expression then RANKX returns 1. This is the default value when order parameter is omitted.
1	TRUE	Ranks in ascending order of expression. If value is equal to the lowest number in expression then RANKX returns 1.

ties

(Optional) An enumeration that defines how to determine ranking when there are ties.

enumeration	Description
Skip	The next rank value, after a tie, is the rank value of the tie plus the count of tied values. For example if five (5) values are tied with a rank of 11 then the next value will receive a rank of 16 (11 + 5). This is the default value when *ties* parameter is omitted.
Dense	The next rank value, after a tie, is the next rank value. For example if five (5) values are tied with a rank of 11 then the next value will receive a rank of 12.

Return value

The rank number of *value* among all possible values of *expression* evaluated for all rows of *table* numbers.

Remarks

- If *expression* or *value* evaluates to BLANK it is treated as a 0 (zero) for all expressions that result in a number, or as an empty text for all text expressions.
- If *value* is not among all possible values of *expression* then RANKX temporarily adds *value* to the values from *expression* and re-evaluates RANKX to determine the proper rank of *value*.
- Optional arguments might be skipped by placing an empty comma (,) in the argument list, i.e. RANKX(Inventory, [InventoryCost],,,"Dense")
- Equality related comparisons (=, < >, >= and <=) between values with the *Decimal Number* data type can potentialy return unexpected results when using RANKX function. Incorrect results can occur because values with *Decimal Number* data type are stored as IEEE Standard 754 floating point numbers and have inherent limitations in their precision. To avoid unexpected results, change the data type to *Fixed Decimal Number* or do a forced rounding using ROUND.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following calculated column in the Products table calculates the sales ranking for each product in the Internet channel.

DAX

```
= RANKX(ALL(Products), SUMX(RELATEDTABLE(InternetSales), [SalesAmount]))
```

Feedback

Was this page helpful? Yes No

SAMPLE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a sample of N rows from the specified table.

Syntax

DAX

```
SAMPLE(<n_value>, <table>, <orderBy_expression>, [<order>[,
<orderBy_expression>, [<order>]]…])
```

Parameters

Expand table

Term	Definition
n_value	The number of rows to return. It is any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context). If a non-integer value (or expression) is entered, the result is cast as an integer.
table	Any DAX expression that returns a table of data from where to extract the 'n' sample rows.
orderBy_expression	Any scalar DAX expression where the result value is evaluated for each row of *table*.
order	(Optional) A value that specifies how to sort *orderBy_expression* values, ascending or descending: 0 (zero), sorts in descending order of values of *order_by*. 1, ranks in ascending order of *order_by*.

Return value

A table consisting of a sample of N rows of *table* or an empty table if *n_value* is 0 (zero) or less. If OrderBy arguments are provided, the sample will be stable and deterministic, returning the first row, the last row, and evenly distributed rows between them. If no ordering is specified, the sample will be random, not stable, and not deterministic.

Remarks

- If n_value is 0 (zero) or less then SAMPLE returns an empty table.
- In order to avoid duplicate values in the sample, the table provided as the second argument should be grouped by the column used for sorting.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

SAMPLECARTESIANPOINTSBYCOVER

Article • 01/07/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a sample subset from a Table that is obtained by plotting the rows as points in 2D space and removing overlapping points.

Syntax

DAX

```
SAMPLECARTESIANPOINTSBYCOVER(<Size>, <Table>, <XAxis>, <YAxis>[, <Radius>]
[, <MaxMinRatio>] [, <MaxBlankRatio>] )
```

Parameters

Expand table

Term	Definition
Size	Number of rows in the sample to return
Table	Any DAX expression that returns a table of data from where to return a sample subset from.
XAxis	The numerical XAxis column from the Table.
YAxis	The numerical YAxis column from the Table.
Radius	(Optional) The numerical Radius column from the Table.
MaxMinRatio	(Optional) When Radius is specified, MaxMinRatio has to be specified and defines the ratio between the maximum and the minimum radiuses of drawn points. See remarks for more details.
MaxBlankRatio	(Optional) When Radius is specified, MaxBlankRatio has to be specified and defines the ratio between the maximum and blank radiuses of the drawn points. See remarks for more details.

Return Value

The sample from Table consisting of Size number of rows.

Remarks

- If Size is less than or equal to 0, SAMPLECARTESIANPOINTSBYCOVER returns an empty table
- If Radius is specified, MaxMinRatio and MaxBlankRatio need to be specified and set to a value larger than 1. If not, SAMPLECARTESIANPOINTSBYCOVER returns an error
- Set MaxMinRatio to the maximum of Radius divided by the minimum of Radius. For example, if the minimum Radius is 20 and the maximum Radius is 60, the MaxMinRatio parameter should be set to 60/20 = 3.
- Set MaxBlankRatio to the maximum of Radius divided by the Radius for blank values. For example, if the maximum Radius is 60 and the Radius for blank values is 6, then MaxBlankRatio should be set to 60/6 = 10.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query:

DAX

```
EVALUATE
SAMPLECARTESIANPOINTSBYCOVER (
    1000,
    SELECTCOLUMNS ( Sales, "x", [Unit Price], "y", [Sales Amount] ),
    [x],
    [y]
)
```

Returns a 1000 row sample from unique points from the Sales table in a 2D space defined by Unit Price as the X axis and Sales Amount as the Y axis.

Feedback

Was this page helpful? Yes No

STDEV.S

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the standard deviation of a sample population.

Syntax

```
DAX
STDEV.S(<ColumnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression.

Return value

A number that represents the standard deviation of a sample population.

Exceptions

Remarks

- STDEV.S assumes that the column refers to a sample of the population. If your data represents the entire population, then compute the standard deviation by using STDEV.P.
- STDEV.S uses the following formula:

  $\sqrt{[\sum(x - \tilde{x})^2/(n-1)]}$

where $\tilde{x}$ is the average value of x for the sample population and n is the population size.

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a measure that calculates the standard deviation of the column, SalesAmount_USD, when the table InternetSales_USD is the sample population.

DAX

```
= STDEV.S(InternetSales_USD[SalesAmount_USD])
```

Feedback

Was this page helpful? Yes No

STDEV.P

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the standard deviation of the entire population.

Syntax

```DAX
STDEV.P(<ColumnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression.

Return value

A number representing the standard deviation of the entire population.

Remarks

- STDEV.P assumes that the column refers to the entire population. If your data represents a sample of the population, then compute the standard deviation by using STDEV.S.
- STDEV.P uses the following formula:

  $\sqrt{[\sum(x - \tilde{x})^2/n]}$

  where $\tilde{x}$ is the average value of x for the entire population and n is the population size.

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a measure that calculates the standard deviation of the column, SalesAmount_USD, when the table InternetSales_USD is the entire population.

DAX

```
= STDEV.P(InternetSales_USD[SalesAmount_USD])
```

Feedback

Was this page helpful? Yes No

STDEVX.S

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the standard deviation of a sample population.

Syntax

```
DAX
STDEVX.S(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).

Return value

A number with the standard deviation of a sample population.

Exceptions

Remarks

- STDEVX.S evaluates *expression* for each row of *table* and returns the standard deviation of *expression* assuming that *table* refers to a sample of the population. If *table* represents the entire population, then compute the standard deviation by using STDEVX.P.

- STDEVX.S uses the following formula:

  $\sqrt{[\sum(x - \tilde{x})^2/(n-1)]}$

  where $\tilde{x}$ is the average value of x for the entire population and n is the population size.

- Blank rows are filtered out from *columnName* and not considered in the calculations.

- An error is returned if *columnName* contains less than 2 non-blank rows.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a calculated column that estimates the standard deviation of the unit price per product for a sample population, when the formula is used in the Product table.

DAX

```
= STDEVX.S(RELATEDTABLE(InternetSales_USD), InternetSales_USD[UnitPrice_USD]
- (InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Feedback

Was this page helpful? Yes No

STDEVX.P

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the standard deviation of the entire population.

Syntax

DAX

```
STDEVX.P(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).

Return value

A number that represents the standard deviation of the entire population.

Remarks

- STDEVX.P evaluates *expression* for each row of *table* and returns the standard deviation of expression assuming that table refers to the entire population. If the data in *table* represents a sample of the population, you should compute the standard deviation by using STDEVX.S instead.
- STDEVX.P uses the following formula:

  $\sqrt{[\sum(x - \tilde{x})^2/n]}$

where $\tilde{x}$ is the average value of x for the entire population and n is the population size.

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a calculated column that calculates the standard deviation of the unit price per product, when the formula is used in the *Product* table.

DAX

```
= STDEVX.P(RELATEDTABLE(InternetSales_USD), InternetSales_USD[UnitPrice_USD]
- (InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Feedback

Was this page helpful? Yes No

T.DIST

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the Student's left-tailed t-distribution.

Syntax

DAX

```
T.DIST(X,Deg_freedom,Cumulative)
```

Parameters

Expand table

Term	Definition
`X`	The numeric value at which to evaluate the distribution.
`Deg_freedom`	An integer indicating the number of degrees of freedom.
`Cumulative`	A logical value that determines the form of the function. If cumulative is `TRUE`, T.DIST returns the cumulative distribution function; if `FALSE`, it returns the probability density function.

Return value

The Student's left-tailed t-distribution.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
EVALUATE { T.DIST(60, 1, TRUE) }
```

Returns,

Expand table

[Value]
0.994695326367377

Related content

T.DIST.2T T.DIST.RT T.INV T.INV.2t

Feedback

Was this page helpful? Yes No

T.DIST.2T

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the two-tailed Student's t-distribution.

Syntax

```
DAX
T.DIST.2T(X,Deg_freedom)
```

Parameters

Expand table

Term	Definition
`X`	The numeric value at which to evaluate the distribution.
`Deg_freedom`	An integer indicating the number of degrees of freedom.

Return value

The two-tailed Student's t-distribution.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

```
DAX
EVALUATE { T.DIST.2T(1.959999998, 60) }
```

Returns

Expand table

[Value]
0.054644929975921

Related content

T.DIST T.DIST.RT T.INV T.INV.2t

Feedback

Was this page helpful? Yes No

T.DIST.RT

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the right-tailed Student's t-distribution.

Syntax

```
DAX
T.DIST.RT(X,Deg_freedom)
```

Parameters

Expand table

Term	Definition
`X`	The numeric value at which to evaluate the distribution.
`Deg_freedom`	An integer indicating the number of degrees of freedom.

Return value

The right-tailed Student's t-distribution.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

```
DAX
EVALUATE { T.DIST.RT(1.959999998, 60) }
```

Returns

Expand table

[Value]
0.0273224649879605

Related content

T.DIST T.DIST.2T T.INV T.INV.2t

Feedback

Was this page helpful? Yes No

T.INV

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the left-tailed inverse of the Student's t-distribution.

Syntax

```
DAX
T.INV(Probability,Deg_freedom)
```

Parameters

Expand table

Term	Definition
`Probability`	The probability associated with the Student's t-distribution.
`Deg_freedom`	The number of degrees of freedom with which to characterize the distribution.

Return value

The left-tailed inverse of the Student's t-distribution.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

```
DAX
EVALUATE { T.INV(0.75, 2) }
```

Returns

Expand table

[Value]
0.816496580927726

Related content

T.INV.2T T.DIST T.DIST.2T T.DIST.RT

Feedback

Was this page helpful? Yes No

T.INV.2T

Article • 05/20/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the two-tailed inverse of the Student's t-distribution.

Syntax

```
DAX
T.INV.2T(Probability,Deg_freedom)
```

Parameters

Expand table

Term	Definition
`Probability`	The probability associated with the Student's t-distribution.
`Deg_freedom`	The number of degrees of freedom with which to characterize the distribution.

Return value

The two-tailed inverse of the Student's t-distribution.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

```
DAX
EVALUATE { T.INV.2T(0.546449, 60) }
```

Returns

Expand table

[Value]
0.606533075825759

Related content

T.INV T.DIST T.DIST.2T T.DIST.RT

Feedback

Was this page helpful? Yes No

VAR.S

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the variance of a sample population.

Syntax

```DAX
VAR.S(<columnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression.

Return value

A number with the variance of a sample population.

Remarks

- VAR.S assumes that the column refers to a sample of the population. If your data represents the entire population, then compute the variance by using VAR.P.
- VAR.S uses the following formula:

  $\sum(x - \tilde{x})^2/(n-1)$

  where $\tilde{x}$ is the average value of x for the sample population

  and n is the population size

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a measure that calculates the variance of the SalesAmount_USD column from the InternetSales_USD for a sample population.

DAX

```
= VAR.S(InternetSales_USD[SalesAmount_USD])
```

Feedback

Was this page helpful? Yes No

VAR.P

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the variance of the entire population.

Syntax

DAX

```
VAR.P(<columnName>)
```

Parameters

Expand table

Term	Definition
columnName	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression.

Return value

A number with the variance of the entire population.

Remarks

- VAR.P assumes that the column refers the entire population. If your data represents a sample of the population, then compute the variance by using VAR.S.
- VAR.P uses the following formula:

  $\sum(x - \tilde{x})^2/n$

  where $\tilde{x}$ is the average value of x for the entire population

  and n is the population size

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a measure that estimates the variance of the SalesAmount_USD column from the InternetSales_USD table, for the entire population.

DAX

```
= VAR.P(InternetSales_USD[SalesAmount_USD])
```

Feedback

Was this page helpful? Yes No

VARX.S

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the variance of a sample population.

Syntax

```DAX
VARX.S(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data.
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).

Return value

A number that represents the variance of a sample population.

Remarks

- VARX.S evaluates *expression* for each row of *table* and returns the variance of *expression*; on the assumption that *table* refers to a sample of the population. If *table* represents the entire population, then you should compute the variance by using VARX.P.
- VAR.S uses the following formula:

  $\sum(x - \tilde{x})^2/(n-1)$

where $\tilde{x}$ is the average value of x for the sample population

and n is the population size

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a calculated column that estimates the variance of the unit price per product for a sample population, when the formula is used in the Product table.

DAX

```
= VARX.S(InternetSales_USD, InternetSales_USD[UnitPrice_USD] -
(InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Feedback

Was this page helpful? Yes No

VARX.P

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the variance of the entire population.

Syntax

DAX

```
VARX.P(<table>, <expression>)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data.
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).

Return value

A number with the variance of the entire population.

Remarks

- VARX.P evaluates <expression> for each row of <table> and returns the variance of <expression> assuming that <table> refers to the entire population.. If <table> represents a sample of the population, then compute the variance by using VARX.S.
- VARX.P uses the following formula:

  $\sum(x - \tilde{x})^2/n$

where $\tilde{x}$ is the average value of x for the entire population

and n is the population size

- Blank rows are filtered out from *columnName* and not considered in the calculations.
- An error is returned if *columnName* contains less than 2 non-blank rows
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the formula for a calculated column that calculates the variance of the unit price per product, when the formula is used in the Product table

DAX

```
= VARX.P(InternetSales_USD, InternetSales_USD[UnitPrice_USD] –
(InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Feedback

Was this page helpful? Yes No

Table manipulation functions

Article • 10/20/2023

These functions return a table or manipulate existing tables.

In this category

Function	Description
ADDCOLUMNS	Adds calculated columns to the given table or table expression.
ADDMISSINGITEMS	Adds combinations of items from multiple columns to a table if they do not already exist.
CROSSJOIN	Returns a table that contains the Cartesian product of all rows from all tables in the arguments.
CURRENTGROUP	Returns a set of rows from the table argument of a GROUPBY expression.
DATATABLE	Provides a mechanism for declaring an inline set of data values.
DETAILROWS	Evaluates a Detail Rows Expression defined for a measure and returns the data.
DISTINCT column	Returns a one-column table that contains the distinct values from the specified column.
DISTINCT table	Returns a table by removing duplicate rows from another table or expression.
EXCEPT	Returns the rows of one table which do not appear in another table.
FILTERS	Returns a table of values directly applied as filters to *columnName*.
GENERATE	Returns a table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*.
GENERATEALL	Returns a table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*.
GENERATESERIES	Returns a single column table containing the values of an arithmetic series.
GROUPBY	Similar to the SUMMARIZE function, GROUPBY does not do an implicit CALCULATE for any extension columns that it adds.

Function	Description
IGNORE	Modifies SUMMARIZECOLUMNS by omitting specific expressions from the BLANK/NULL evaluation.
INTERSECT	Returns the row intersection of two tables, retaining duplicates.
NATURALINNERJOIN	Performs an inner join of a table with another table.
NATURALLEFTOUTERJOIN	Performs a join of the LeftTable with the RightTable.
ROLLUP	Modifies the behavior of SUMMARIZE by adding rollup rows to the result on columns defined by the groupBy_columnName parameter.
ROLLUPADDISSUBTOTAL	Modifies the behavior of SUMMARIZECOLUMNS by adding rollup/subtotal rows to the result based on the groupBy_columnName columns.
ROLLUPISSUBTOTAL	Pairs rollup groups with the column added by ROLLUPADDISSUBTOTAL within an ADDMISSINGITEMS expression.
ROLLUPGROUP	Modifies the behavior of SUMMARIZE and SUMMARIZECOLUMNS by adding rollup rows to the result on columns defined by the the groupBy_columnName parameter.
ROW	Returns a table with a single row containing values that result from the expressions given to each column.
SELECTCOLUMNS	Adds calculated columns to the given table or table expression.
SUBSTITUTEWITHINDEX	Returns a table which represents a left semijoin of the two tables supplied as arguments.
SUMMARIZE	Returns a summary table for the requested totals over a set of groups.
SUMMARIZECOLUMNS	Returns a summary table over a set of groups.
Table Constructor	Returns a table of one or more columns.
TOPN	Returns the top N rows of the specified table.
TREATAS	Applies the result of a table expression as filters to columns from an unrelated table.
UNION	Creates a union (join) table from a pair of tables.
VALUES	Returns a one-column table that contains the distinct values from the specified table or column.

Feedback

Was this page helpful? Yes No

ADDCOLUMNS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Adds calculated columns to the given table or table expression.

Syntax

DAX

```
ADDCOLUMNS(<table>, <name>, <expression>[, <name>, <expression>]…)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data.
name	The name given to the column, enclosed in double quotes.
expression	Any DAX expression that returns a scalar expression, evaluated for each row of *table*.

Return value

A table with all its original columns and the added ones.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns an extended version of the Product Category table that includes total sales values from the reseller channel and the internet sales.

DAX

```
ADDCOLUMNS(ProductCategory
                , "Internet Sales", SUMX(RELATEDTABLE(InternetSales_USD), InternetSales_USD[SalesAmount_USD])
                , "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD), ResellerSales_USD[SalesAmount_USD]))
```

The following table shows a preview of the data as it would be received by any function expecting to receive a table:

Expand table

ProductCategory[ProductCategoryName]	ProductCategory[ProductCategoryAlternateKey]	ProductCategory[ProductCategoryKey]	[Internet Sales]	[Res Sale
Bikes	1	1	25107749.77	6308
Components	2	2		1120
Clothing	3	3	306157.5829	1669
Accessories	4	4	640920.1338	5343

Feedback

Was this page helpful? Yes No

ADDMISSINGITEMS

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Adds rows with empty values to a table returned by SUMMARIZECOLUMNS.

Syntax

DAX

```
ADDMISSINGITEMS ( [<showAll_columnName> [, <showAll_columnName> [, … ] ] ],
<table> [, <groupBy_columnName> [, [<filterTable>] [, <groupBy_columnName>
[, [<filterTable>] [, … ] ] ] ] ] ] )
```

Parameters

Expand table

Term	Definition
showAll_columnName	(Optional) A column for which to return items with no data for the measures used. If not specified, all columns are returned.
table	A SUMMARIZECOLUMNS table.
groupBy_columnName	(Optional) A column to group by in the supplied table argument.
filterTable	(Optional) A table expression that defines which rows are returned.

Return value

A table with one or more columns.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

With SUMMARIZECOLUMNS

A table returned by SUMMARIZECOLUMNS will include only rows with values. By wrapping a SUMMARIZECOLUMNS expression within an ADDMISSINGITEMS expression, rows containing no values are also returned.

Example

Without ADDMISSINGITEMS, the following query:

DAX

```
SUMMARIZECOLUMNS(
    'Sales'[CustomerId],
    "Total Qty", SUM ( Sales[TotalQty] )
)
```

Returns,

Expand table

CustomerId	TotalQty
A	5
B	3
C	3
E	2

With ADDMISSINGITEMS, the following query:

DAX

```
EVALUATE
ADDMISSINGITEMS (
    'Sales'[CustomerId],
    SUMMARIZECOLUMNS(
        'Sales'[CustomerId],
        "Total Qty", SUM ( Sales[TotalQty] )
    ),
    'Sales'[CustomerId]
)
```

Returns,

Expand table

CustomerId	TotalQty
A	5
B	3
C	3
D	
E	2
F	

Feedback

Was this page helpful? Yes No

CROSSJOIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table that contains the Cartesian product of all rows from all tables in the arguments. The columns in the new table are all the columns in all the argument tables.

Syntax

DAX

```
CROSSJOIN(<table>, <table>[, <table>]…)
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data

Return value

A table that contains the Cartesian product of all rows from all tables in the arguments.

Remarks

- Column names from *table* arguments must all be different in all tables or an error is returned.
- The total number of rows returned by CROSSJOIN() is equal to the product of the number of rows from all tables in the arguments; also, the total number of columns in the result table is the sum of the number of columns in all tables. For example, if **TableA** has **rA** rows and **cA** columns, and **TableB** has **rB** rows and **cB** columns, and **TableC** has **rC** rows and **cC** column; then, the resulting table has **rA** × **rb** × **rC** rows and **cA** + **cB** + **cC** columns.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows the results of applying CROSSJOIN() to two tables: **Colors** and **Stationery**.

The table **Colors** contains colors and patterns:

Expand table

Color	Pattern
Red	Horizontal Stripe
Green	Vertical Stripe
Blue	Crosshatch

The table **Stationery** contains fonts and presentation:

Expand table

Font	Presentation
serif	embossed
sans-serif	engraved

The expression to generate the cross join is presented below:

```
DAX

CROSSJOIN( Colors, Stationery)
```

When the above expression is used wherever a table expression is expected, the results of the expression would be as follows:

Expand table

Color	Pattern	Font	Presentation
Red	Horizontal Stripe	serif	embossed

Color	Pattern	Font	Presentation
Green	Vertical Stripe	serif	embossed
Blue	Crosshatch	serif	embossed
Red	Horizontal Stripe	sans-serif	engraved
Green	Vertical Stripe	sans-serif	engraved
Blue	Crosshatch	sans-serif	engraved

Feedback

Was this page helpful? Yes No

CURRENTGROUP

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns a set of rows from the table argument of a GROUPBY expression that belong to the current row of the GROUPBY result.

Syntax

```DAX
CURRENTGROUP ( )
```

Parameters

None

Return value

The rows in the table argument of the GROUPBY function corresponding to one group of values of the groupBy_columnName arguments.

Remarks

- This function can only be used within a GROUPBY expression.
- This function takes no arguments and is only supported as the first argument to one of the following aggregation functions: AVERAGEX, COUNTAX, COUNTX, GEOMEANX, MAXX, MINX, PRODUCTX, STDEVX.S, STDEVX.P, SUMX, VARX.S, VARX.P.

Example

See GROUPBY.

Feedback

Was this page helpful? Yes No

DATATABLE

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

Provides a mechanism for declaring an inline set of data values.

Syntax

DAX

```
DATATABLE (ColumnName1, DataType1, ColumnName2, DataType2..., {{Value1,
Value2...}, {ValueN, ValueN+1...}...})
```

Parameters

Expand table

Term	Definition
`ColumnName`	A column name.
`DataType`	An enumeration that includes: BOOLEAN/LOGICAL, CURRENCY/DECIMAL, DATETIME, DOUBLE, INTEGER/INT64, STRING/TEXT.
`value`	A single argument using Excel syntax for a one dimensional array constant, nested to provide an array of arrays. This argument represents the set of data values that will be in the table For example, { {values in row1}, {values in row2}, {values in row3}, etc. } Where {values in row1} is a comma delimited set of constant expressions, namely a combination of constants, combined with a handful of basic functions including DATE, TIME, and BLANK, as well as a plus operator between DATE and TIME and a unary minus operator so that negative values can be expressed. The following are all valid values: 3, -5, BLANK(), "2009-04-15 02:45:21". Values may not refer to anything outside the immediate expression, and cannot refer to columns, tables, relationships, or anything else. A missing value will be treated identically to BLANK(). For example, the following are the same: {1,2,BLANK(),4} {1,2,,4}

Return value

A table declaring an inline set of values.

Remarks

- Unlike DATATABLE, Table Constructor allows any scalar expressions as input values.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

DAX

```
= DataTable("Name", STRING,
               "Region", STRING
               ,{
                        {" User1","East"},
                        {" User2","East"},
                        {" User3","West"},
                        {" User4","West"},
                        {" User4","East"}
                }
           )
```

DETAILROWS

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Evaluates a Detail Rows Expression defined for a measure and returns the data.

Syntax

DAX

```
DETAILROWS([Measure])
```

Parameters

Expand table

Term	Definition
Measure	Name of a measure.

Return value

A table with the data returned by the Detail Rows Expression. If no Detail Rows Expression is defined, the data for the table containing the measure is returned.

Feedback

Was this page helpful? Yes No

DISTINCT (column)

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a one-column table that contains the distinct values from the specified column. In other words, duplicate values are removed and only unique values are returned.

> **Note**
>
> This function cannot be used to Return values into a cell or column on a worksheet; rather, you nest the DISTINCT function within a formula, to get a list of distinct values that can be passed to another function and then counted, summed, or used for other operations.

Syntax

DAX

```
DISTINCT(<column>)
```

Parameters

Expand table

Term	Definition
column	The column from which unique values are to be returned. Or, an expression that returns a column.

Return value

A column of unique values.

Remarks

- The results of DISTINCT are affected by the current filter context. For example, if you use the formula in the following example to create a measure, the results would change whenever the table was filtered to show only a particular region or a time period.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related functions

There is another version of the DISTINCT function, DISTINCT (table), that returns a table by removing duplicate rows from another table or expression..

The VALUES function is similar to DISTINCT; it can also be used to return a list of unique values, and generally will return exactly the same results as DISTINCT. However, in some context VALUES will return one additional special value. For more information, see VALUES function.

Example

The following formula counts the number of unique customers who have generated orders over the internet channel. The table that follows illustrates the possible results when the formula is added to a report.

DAX

```
= COUNTROWS(DISTINCT(InternetSales_USD[CustomerKey]))
```

You cannot paste the list of values that DISTINCT returns directly into a column. Instead, you pass the results of the DISTINCT function to another function that counts, filters, or aggregates values by using the list. To make the example as simple as possible, here the table of distinct values has been passed to the COUNTROWS function.

Expand table

Row Labels	Accessories	Bikes	Clothing	Grand Total
2005		1013		1013
2006		2677		2677
2007	6792	4875	2867	9309

Row Labels	Accessories	Bikes	Clothing	Grand Total
2008	9435	5451	4196	11377
Grand Total	15114	9132	6852	18484

Also, note that the results are not additive. That is to say, the total number of unique customers in *2007* is not the sum of unique customers of *Accessories*, *Bikes* and *Clothing* for that year. The reason is that a customer can be counted in multiple groups.

Related content

Filter functions
FILTER function
RELATED function
VALUES function

Feedback

Was this page helpful? Yes No

DISTINCT (table)

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table by removing duplicate rows from another table or expression.

Syntax

```
DAX
DISTINCT(<table>)
```

Parameters

Expand table

Term	Definition
table	The table from which unique rows are to be returned. The table can also be an expression that results in a table.

Return value

A table containing only distinct rows.

Related functions

There is another version of the DISTINCT function, DISTINCT (column), that takes a column name as input parameter.

Example

The following query:

```
DAX
EVALUATE DISTINCT( { (1, "A"), (2, "B"), (1, "A") } )
```

Returns table:

Expand table

[Value1]	[Value2]
1	A
2	B

Related content

Filter functions
DISTINCT (column)
FILTER function
RELATED function
VALUES function

Feedback

Was this page helpful? Yes No

EXCEPT

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the rows of the first table in the expression which do not appear in the second table.

Syntax

```DAX
EXCEPT(<tableExpression1>, <tableExpression2>)
```

Parameters

Expand table

Term	Definition
`tableExpression`	Any DAX expression that returns a table.

Return value

A table that contains the rows of one table minus all the rows of another table.

Remarks

- If a row appears at all in both tables, it and its duplicates are not present in the result set. If a row appears in only tableExpression1, it and its duplicates will appear in the result set.
- The column names will match the column names in tableExpression1.
- The returned table has lineage based on the columns in tableExpression1 , regardless of the lineage of the columns in the second table. For example, if the first column of first tableExpression has lineage to the base column C1 in the model, the Except will reduce the rows based on the availability of values in the first column of second tableExpression and keep the lineage on base column C1 intact.
- The two tables must have the same number of columns.
- Columns are compared based on positioning, and data comparison with no type coercion.

- The set of rows returned depends on the order of the two expressions.
- The returned table does not include columns from tables related to tableExpression1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

States1

Expand table

State
A
B
B
B
C
D
D

States2

Expand table

State
B
C
D
D
D
E
E

State
E

Except(States1, States2)

Expand table

State
A

Except(States2, States1)

Expand table

State
E
E
E

Last updated on 08/17/2026

FILTERS

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the values that are directly applied as filters to *columnName*.

Syntax

```
DAX
FILTERS(<columnName>)
```

Parameters

Expand table

Term	Description
columnName	The name of an existing column, using standard DAX syntax. It cannot be an expression.

Return value

The values that are directly applied as filters to *columnName*.

Remarks

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example shows how to determine the number of direct filters a column has.

```
DAX
```

```
= COUNTROWS(FILTERS(ResellerSales_USD[ProductKey]))
```

This example lets you know how many direct filters on ResellerSales_USD[ProductKey] have been applied to the context where the expression is being evaluated.

Feedback

Was this page helpful? Yes No

GENERATE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*.

Syntax

DAX

```
GENERATE(<table1>, <table2>)
```

Parameters

Expand table

Term	Definition
table1	Any DAX expression that returns a table.
table2	Any DAX expression that returns a table.

Return value

A table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*

Remarks

- If the evaluation of *table2* for the current row in *table1* returns an empty table, then the result table will not contain the current row from *table1*. This is different than GENERATEALL() where the current row from *table1* will be included in the results and columns corresponding to *table2* will have null values for that row.
- All column names from *table1* and *table2* must be different or an error is returned.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the following example the user wants a summary table of the sales by Region and Product Category for the Resellers channel, like the following table:

Expand table

SalesTerritory[SalesTerritoryGroup]	ProductCategory[ProductCategoryName]	[Reseller Sales]
Europe	Accessories	$ 142,227.27
Europe	Bikes	$ 9,970,200.44
Europe	Clothing	$ 365,847.63
Europe	Components	$ 2,214,440.19
North America	Accessories	$ 379,305.15
North America	Bikes	$ 52,403,796.85
North America	Clothing	$ 1,281,193.26
North America	Components	$ 8,882,848.05
Pacific	Accessories	$ 12,769.57
Pacific	Bikes	$ 710,677.75
Pacific	Clothing	$ 22,902.38
Pacific	Components	$ 108,549.71

The following formula produces the above table:

DAX

```
GENERATE(
SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])
,SUMMARIZE(ProductCategory
```

```
, [ProductCategoryName]
, "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD),
ResellerSales_USD[SalesAmount_USD])
)
)
```

1. The first SUMMARIZE statement, `SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])`, produces a table of territory groups, where each row is a territory group, as shown below:

Expand table

SalesTerritory[SalesTerritoryGroup]
North America
Europe
Pacific
NA

2. The second SUMMARIZE statement, `SUMMARIZE(ProductCategory, [ProductCategoryName], "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD), ResellerSales_USD[SalesAmount_USD]))`, produces a table of Product Category groups with the Reseller sales for each group, as shown below:

Expand table

ProductCategory[ProductCategoryName]	[Reseller Sales]
Bikes	$ 63,084,675.04
Components	$ 11,205,837.96
Clothing	$ 1,669,943.27
Accessories	$ 534,301.99

3. However, when you take the above table and evaluate it under the context of each row from the territory groups table, you obtain different results for each territory.

Feedback

Was this page helpful? Yes No

GENERATEALL

Article • 10/20/2023

Returns a table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*.

Syntax

```
DAX

GENERATEALL(<table1>, <table2>)
```

Parameters

Term	Definition
table1	Any DAX expression that returns a table.
table2	Any DAX expression that returns a table.

Return value

A table with the Cartesian product between each row in *table1* and the table that results from evaluating *table2* in the context of the current row from *table1*

Remarks

- If the evaluation of *table2* for the current row in *table1* returns an empty table, then the current row from *table1* will be included in the results and columns corresponding to *table2* will have null values for that row. This is different than GENERATE() where the current row from *table1* will **not** be included in the results.
- All column names from *table1* and *table2* must be different or an error is returned.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the following example, the user wants a summary table of the sales by Region and Product Category for the Resellers channel, like the following table:

SalesTerritory[SalesTerritoryGroup]	ProductCategory[ProductCategoryName]	[Reseller Sales]
Europe	Accessories	$ 142,227.27
Europe	Bikes	$ 9,970,200.44
Europe	Clothing	$ 365,847.63
Europe	Components	$ 2,214,440.19
NA	Accessories	
NA	Bikes	
NA	Clothing	
NA	Components	
North America	Accessories	$ 379,305.15
North America	Bikes	$ 52,403,796.85
North America	Clothing	$ 1,281,193.26
North America	Components	$ 8,882,848.05
Pacific	Accessories	$ 12,769.57
Pacific	Bikes	$ 710,677.75
Pacific	Clothing	$ 22,902.38
Pacific	Components	$ 108,549.71

The following formula produces the above table:

DAX

```
GENERATEALL(
SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])
,SUMMARIZE(ProductCategory
, [ProductCategoryName]
, "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD),
```

```
ResellerSales_USD[SalesAmount_USD])
)
)
```

1. The first SUMMARIZE produces a table of territory groups, where each row is a territory group, like those listed below:

SalesTerritory[SalesTerritoryGroup]
North America
Europe
Pacific
NA

2. The second SUMMARIZE produces a table of Product Category groups with the Reseller sales for each group, as shown below:

ProductCategory[ProductCategoryName]	**[Reseller Sales]**
Bikes	$ 63,084,675.04
Components	$ 11,205,837.96
Clothing	$ 1,669,943.27
Accessories	$ 534,301.99

3. However, when you take the above table and evaluate the table under the context of each row from the territory groups table, you obtain different results for each territory.

Feedback

Was this page helpful? Yes No

GENERATESERIES

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a single column table containing the values of an arithmetic series, that is, a sequence of values in which each differs from the preceding by a constant quantity. The name of the column returned is Value.

Syntax

```DAX
GENERATESERIES(<startValue>, <endValue>[, <incrementValue>])
```

Parameters

Expand table

Term	Definition
`startValue`	The initial value used to generate the sequence.
`endValue`	The end value used to generate the sequence.
`incrementValue`	(Optional) The increment value of the sequence. When not provided, the default value is 1.

Return value

A single column table containing the values of an arithmetic series. The name of the column is Value.

Remarks

- When endValue is less than startValue, an empty table is returned.
- incrementValue must be a positive value.
- The sequence stops at the last value that is less than or equal to endValue.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example 1

The following DAX query:

```
DAX
EVALUATE GENERATESERIES(1, 5)
```

Returns the following table with a single column:

Expand table

[Value]
1
2
3
4
5

Example 2

The following DAX query:

```
DAX
EVALUATE GENERATESERIES(1.2, 2.4, 0.4)
```

Returns the following table with a single column:

Expand table

[Value]
1.2
1.6

[Value]
2
2.4

Example 3

The following DAX query:

```
DAX

EVALUATE GENERATESERIES(CURRENCY(10), CURRENCY(12.4), CURRENCY(0.5))
```

Returns the following table with a single column:

Expand table

[Value]
10
10.5
11
11.5
12

Feedback

Was this page helpful? Yes No

GROUPBY

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

The GROUPBY function is similar to the SUMMARIZE function. However, GROUPBY does not do an implicit CALCULATE for any extension columns that it adds. GROUPBY permits a new function, CURRENTGROUP, to be used inside aggregation functions in the extension columns that it adds. GROUPBY is used to perform multiple aggregations in a single table scan.

Syntax

DAX

```
GROUPBY (<table> [, <groupBy_columnName> [, <groupBy_columnName> [, …]]] [, <name>, <expression> [, <name>, <expression> [, …]]])
```

Parameters

Expand table

Term	Definition
table	Any DAX expression that returns a table of data.
groupBy_columnName	The name of an existing column in the table (or in a related table,) by which the data is to be grouped. This parameter cannot be an expression.
name	The name given to a new column that is being added to the list of GroupBy columns, enclosed in double quotes.
expression	One of the X aggregation functions with the first argument being CURRENTGROUP(). See With CURRENTGROUP section below for the full

Term	Definition
	list of supported X aggregation functions.

Return value

A table with the selected columns for the groupBy_columnName arguments and the extension columns designated by the name arguments.

Remarks

- The GROUPBY function does the following:

    1. Start with the specified table (and all related tables in the "to-one" direction).
    2. Create a grouping using all of the GroupBy columns (which are required to exist in the table from step #1.).
    3. Each group is one row in the result, but represents a set of rows in the original table.
    4. For each group, evaluate the extension columns being added. Unlike the SUMMARIZE function, an implied CALCULATE is not performed, and the group isn't placed into the filter context.

- Each column for which you define a name must have a corresponding expression; otherwise, an error is returned. The first argument, name, defines the name of the column in the results. The second argument, expression, defines the calculation performed to obtain the value for each row in that column.
- groupBy_columnName must be either in table or in a related table.
- Each name must be enclosed in double quotation marks.
- The function groups a selected set of rows into a set of summary rows by the values of one or more groupBy_columnName columns. One row is returned for each group.
- GROUPBY is primarily used to perform aggregations over intermediate results from DAX table expressions. For efficient aggregations over physical tables in the model, consider using SUMMARIZECOLUMNS or SUMMARIZE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

With CURRENTGROUP

CURRENTGROUP can only be used in an expression that defines an extension column within the GROUPBY function. In-effect, CURRENTGROUP returns a set of rows from the table argument of GROUPBY that belong to the current row of the GROUPBY result. The CURRENTGROUP function takes no arguments and is only supported as the first argument to one of the following aggregation functions: AVERAGEX, COUNTAX, COUNTX, GEOMEANX, MAXX, MINX, PRODUCTX, STDEVX.S, STDEVX.P, SUMX, VARX.S, VARX.P.

Example

The following example first calculates the total sales grouped by country and product category over physical tables by using the SUMMARIZECOLUMNS function. It then uses the GROUPBY function to scan the intermediate result from the first step to find the maximum sales in each country across the product categories.

DAX

```
DEFINE
VAR SalesByCountryAndCategory =
SUMMARIZECOLUMNS(
Geography[Country],
Product[Category],
"Total Sales", SUMX(Sales, Sales[Price] * Sales[Qty])
)

EVALUATE
GROUPBY(
SalesByCountryAndCategory,
Geography[Country],
"Max Sales", MAXX(CURRENTGROUP(), [Total Sales])
)
```

Related content

SUMMARIZE function
SUMMARIZECOLUMNS function

Feedback

Was this page helpful? Yes No

IGNORE

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Modifies the behavior of the SUMMARIZECOLUMNS function by omitting specific expressions from the BLANK/NULL evaluation. Rows for which all expressions not using IGNORE return BLANK/NULL will be excluded independent of whether the expressions which do use IGNORE evaluate to BLANK/NULL or not. This function can only be used within a SUMMARIZECOLUMNS expression.

Syntax

DAX

```
IGNORE(<expression>)
```

With SUMMARIZECOLUMNS,

DAX

```
SUMMARIZECOLUMNS(<groupBy_columnName>[, < groupBy_columnName >]…,
[<filterTable>]…[, <name>, IGNORE(…)]…)
```

Parameters

Expand table

Term	Definition
expression	Any DAX expression that returns a single value (not a table).

Return value

The function does not return a value.

Remarks

IGNORE can only be used as an expression argument to SUMMARIZECOLUMNS.

Example

See SUMMARIZECOLUMNS.

Feedback

Was this page helpful? Yes No

INTERSECT

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the row intersection of two tables, retaining duplicates.

Syntax

```
DAX
INTERSECT(<tableExpression1>, <tableExpression2>)
```

Parameters

Expand table

Term	Definition
`tableExpression`	Any DAX expression that returns a table.

Return value

A table that contains all the rows in tableExpression1 that are also in tableExpression2

Exceptions

Remarks

- Intersect is not commutative. In general, Intersect(T1, T2) will have a different result set than Intersect(T2, T1).
- Duplicate rows are retained. If a row appears in tableExpression1 and tableExpression2, it and all duplicates in tableExpression1 are included in the result set.
- The column names will match the column names in tableExpression1.
- The returned table has lineage based on the columns in tableExpression1 , regardless of the lineage of the columns in the second table. For example, if the first column of first tableExpression has lineage to the base column C1 in the model, the intersect will reduce

the rows based on the intersect on first column of second tableExpression and keep the lineage on base column C1 intact.

- Columns are compared based on positioning, and data comparison with no type coercion.
- The returned table does not include columns from tables related to tableExpression1.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

States1

Expand table

State
A
A
B
B
B
C
D
D

States2

Expand table

State
B
C
D
D

State
D
E

Intersect(States1, States2)

Expand table

State
B
B
B
C
D
D

Intersect(States2, States1)

Expand table

State
B
C
D
D
D

Last updated on 08/17/2026

NATURALINNERJOIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Performs an inner join of a table with another table.

Syntax

```
DAX
NATURALINNERJOIN(<LeftTable>, <RightTable>)
```

Parameters

Expand table

Term	Definition
LeftTable	A table expression defining the table on the left side of the join.
RightTable	A table expression defining the table on the right side of the join.

Return value

A table which includes only rows for which the values in the common columns specified are present in both tables. The table returned will have the common columns from the left table and other columns from both the tables.

Remarks

- Tables are joined on common columns (by name) in the two tables. If the two tables have no common column names, an error is returned.
- There is no sort order guarantee for the results.
- Columns being joined on must have the same data type in both tables.

- Only columns from the same source table (have the same lineage) are joined on. For example, Products[ProductID], WebSales[ProductdID], StoreSales[ProductdID] with many-to-one relationships between WebSales and StoreSales and the Products table based on the ProductID column, WebSales and StoreSales tables are joined on [ProductID].
- Strict comparison semantics are used during join. There is no type coercion; for example, 1 does not equal 1.0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

NATURALLEFTOUTERJOIN

Feedback

Was this page helpful? Yes No

NATURALLEFTOUTERJOIN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Performs a join of the LeftTable with the RightTable by using the Left Outer Join semantics.

Syntax

DAX

```
NATURALLEFTOUTERJOIN(<LeftTable>, <RightTable>)
```

Parameters

Expand table

Term	Definition
LeftTable	A table expression defining the table on the left side of the join.
RightTable	A table expression defining the table on the right side of the join.

Return value

A table which includes only rows from RightTable for which the values in the common columns specified are also present in LeftTable. The table returned will have the common columns from the left table and the other columns from both the tables.

Remarks

- Tables are joined on common columns (by name) in the two tables. If the two tables have no common column names, an error is returned.
- There is no sort order guarantee for the results.
- Columns being joined on must have the same data type in both tables.

- Only columns from the same source table (have the same lineage) are joined on. For example, Products[ProductID], WebSales[ProductdID], StoreSales[ProductdID] with many-to-one relationships between WebSales and StoreSales and the Products table based on the ProductID column, WebSales and StoreSales tables are joined on [ProductID].
- Strict comparison semantics are used during join. There is no type coercion; for example, 1 does not equal 1.0.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Related content

NATURALINNERJOIN

Feedback

Was this page helpful? Yes No

ROLLUP

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Modifies the behavior of the SUMMARIZE function by adding rollup rows to the result on columns defined by the groupBy_columnName parameter. This function can only be used within a SUMMARIZE expression.

Syntax

DAX

```
ROLLUP ( <groupBy_columnName> [, <groupBy_columnName> [, … ] ] )
```

With SUMMARIZE,

DAX

```
SUMMARIZE(<table>, <groupBy_columnName>[, <groupBy_columnName>]…[,
ROLLUP(<groupBy_columnName>[,< groupBy_columnName>…])][, <name>,
<expression>]…)
```

Parameters

Expand table

Term	Definition
groupBy_columnName	The qualified name of an existing column or ROLLUPGROUP function to be used to create summary groups based on the values found in it. This parameter cannot be an expression.

Return value

This function does not return a value. It only specifies the set of columns to be subtotaled.

Remarks

This function can only be used within a SUMMARIZE expression.

Example

See SUMMARIZE.

Feedback

Was this page helpful? Yes No

ROLLUPADDISSUBTOTAL

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Modifies the behavior of the SUMMARIZECOLUMNS function by adding rollup/subtotal rows to the result based on the groupBy_columnName columns. This function can only be used within a SUMMARIZECOLUMNS expression.

Syntax

DAX

```
ROLLUPADDISSUBTOTAL ( [<grandtotalFilter>], <groupBy_columnName>, <name> [,
[<groupLevelFilter>] [, <groupBy_columnName>, <name> [, [<groupLevelFilter>]
[, … ] ] ] ] )
```

Parameters

Expand table

Term	Definition
grandtotalFilter	(Optional) Filter to be applied to the grandtotal level.
groupBy_columnName	Name of an existing column used to create summary groups based on the values found in it. Cannot be an expression.
name	Name of an ISSUBTOTAL column. The values of the column are calculated using the ISSUBTOTAL function.
groupLevelFilter	(Optional) Filter to be applied to the current level.

Return value

The function does not return a value.

Remarks

None

Example

See SUMMARIZECOLUMNS.

Feedback

Was this page helpful? Yes No

ROLLUPGROUP

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Modifies the behavior of the SUMMARIZE and SUMMARIZECOLUMNS functions by adding rollup rows to the result on columns defined by the the groupBy_columnName parameter. This function can only be used within a SUMMARIZE or SUMMARIZECOLUMNS expression.

Syntax

DAX

```
ROLLUPGROUP ( <groupBy_columnName> [, <groupBy_columnName> [, … ] ] )
```

Parameters

Expand table

Term	Definition
groupBy_columnName	The qualified name of an existing column or ROLLUPGROUP function to be used to create summary groups based on the values found in it. This parameter cannot be an expression.

Return value

This function does not return a value. It marks a set of columns to be treated as a single group during subtotaling by ROLLUP or ROLLUPADDISSUBTOTAL.

Remarks

ROLLUPGROUP can only be used as a groupBy_columnName argument to ROLLUP, ROLLUPADDISSUBTOTAL, or ROLLUPISSUBTOTAL.

Example

See SUMMARIZE and SUMMARIZECOLUMNS.

Feedback

Was this page helpful? Yes No

ROLLUPISSUBTOTAL

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Pairs rollup groups with the column added by ROLLUPADDISSUBTOTAL. This function can only be used within an ADDMISSINGITEMS expression.

Syntax

DAX

```
ROLLUPISSUBTOTAL ( [<grandTotalFilter>], <groupBy_columnName>,
<isSubtotal_columnName> [, [<groupLevelFilter>] [, <groupBy_columnName>,
<isSubtotal_columnName> [, [<groupLevelFilter>] [, … ] ] ] ] )
```

Parameters

Expand table

Term	Definition
grandTotalFilter	(Optional) Filter to be applied to the grandtotal level.
groupBy_columnName	Name of an existing column used to create summary groups based on the values found in it. Cannot be an expression.
isSubtotal_columnName	Name of an ISSUBTOTAL column. The values of the column are calculated using the ISSUBTOTAL function.
groupLevelFilter	(Optional) Filter to be applied to the current level.

Return value

None

Remarks

This function can only be used within an ADDMISSINGITEMS expression.

Feedback

Was this page helpful? Yes No

ROW function

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with a single row containing values that result from the expressions given to each column.

Syntax

DAX

```
ROW(<name>, <expression>[[,<name>, <expression>]…])
```

Parameters

Expand table

Term	Definition
name	The name given to the column, enclosed in double quotes.
expression	Any DAX expression that returns a single scalar value to populate. *name*.

Return value

A single row table

Remarks

- Arguments must always come in pairs of *name* and *expression*.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns a single row table with the total sales for internet and resellers channels.

DAX

```
ROW("Internet Total Sales (USD)", SUM(InternetSales_USD[SalesAmount_USD]),
         "Resellers Total Sales (USD)",
SUM(ResellerSales_USD[SalesAmount_USD]))
```

Feedback

Was this page helpful? Yes No

SELECTCOLUMNS

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a table with selected columns from the table and new columns specified by the DAX expressions.

Syntax

```DAX
SELECTCOLUMNS(<Table>, [<Name>], <Expression>, [<Name>], …)
```

Parameters

Expand table

Term	Definition
`Table`	Any DAX expression that returns a table.
`Name`	The name given to the column, enclosed in double quotes.
`Expression`	Any expression that returns a scalar value like a column reference, integer, or string value.

Return value

A table with the same number of rows as the table specified as the first argument. The returned table has one column for each pair of `Name`, `Expression` arguments, and each expression is evaluated in the context of a row from the specified `Table` argument.

Remarks

SELECTCOLUMNS has the same signature as ADDCOLUMNS, and has the same behavior except that instead of starting with the `Table` specified, SELECTCOLUMNS starts with an empty table before adding columns.

This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

For the following table named **Customer**:

Expand table

Country	State	Count	Total
IND	JK	20	800
IND	MH	25	1000
IND	WB	10	900
USA	CA	5	500
USA	WA	10	900

DAX

```
SELECTCOLUMNS(Customer, "Country, State", [Country]&", "&[State])
```

Returns,

Expand table

Country, State
IND, JK
IND, MH
IND, WB
USA, CA
USA, WA

Feedback

Was this page helpful? Yes No

SUBSTITUTEWITHINDEX

Article • 10/20/2023

Returns a table which represents a left semijoin of the two tables supplied as arguments. The semijoin is performed by using common columns, determined by common column names and common data type . The columns being joined on are replaced with a single column in the returned table which is of type integer and contains an index. The index is a reference into the right join table given a specified sort order.

Columns in the right/second table supplied which do not exist in the left/first table supplied are not included in the returned table and are not used to join on.

The index starts at 0 (0-based) and is incremented by one for each additional row in the right/second join table supplied. The index is based on the sort order specified for the right/second join table.

Syntax

DAX

```
SUBSTITUTEWITHINDEX(<table>, <indexColumnName>, <indexColumnsTable>,
[<orderBy_expression>, [<order>][, <orderBy_expression>, [<order>]]…])
```

Parameters

Term	Definition
table	A table to be filtered by performing a left semijoin with the table specified as the third argument (indexColumnsTable). This is the table on the left side of the left semijoin so the table returned includes the same columns as this table except that all common columns of the two tables will be replaced with a single index column in the table returned.
indexColumnName	A string which specifies the name of the index column which is replacing all the common columns in the two tables supplied as arguments to this function.
indexColumnsTable	The second table for the left semijoin. This is the table on the right side of the left semijoin. Only values present in this table will be returned by the function. Also, the columns of this table (based on column names) will be replaced with a single index column in the table returned by this function.

Term	Definition
orderBy_expression	Any DAX expression where the result value is used to specify the desired sort order of the indexColumnsTable table for generating correct index values. The sort order specified for the indexColumnsTable table defines the index of each row in the table and that index is used in the table returned to represent combinations of values in the indexColumnsTable as they appear in the table supplied as the first argument to this function.
order	(Optional) A value that specifies how to sort orderBy_expression values, ascending or descending: Value: **Desc**. Alternative value: **0**(zero)/**FALSE**. Sorts in descending order of values of orderBy_expression. This is the default value when order parameter is omitted. Value: **ASC**. Alternative value: **1/TRUE**. Ranks in ascending order of orderBy_expression.

Return value

A table which includes only those values present in the indexColumnsTable table and which has an index column instead of all columns present (by name) in the indexColumnsTable table.

Remarks

- This function does not guarantee any result sort order.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Feedback

Was this page helpful? Yes No

SUMMARIZE

Article • 10/20/2023

Returns a summary table for the requested totals over a set of groups.

Syntax

```
DAX

SUMMARIZE (<table>, <groupBy_columnName>[, <groupBy_columnName>]…[, <name>, <expression>]…)
```

Parameters

Term	Definition
table	Any DAX expression that returns a table of data.
groupBy_ColumnName	(Optional) The qualified name of an existing column used to create summary groups based on the values found in it. This parameter cannot be an expression.
name	The name given to a total or summarize column, enclosed in double quotes.
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).

Return value

A table with the selected columns for the *groupBy_columnName* arguments and the summarized columns designed by the name arguments.

Remarks

- Each column for which you define a name must have a corresponding expression; otherwise, an error is returned. The first argument, name, defines the name of the column in the results. The second argument, expression, defines the calculation performed to obtain the value for each row in that column.
- groupBy_columnName must be either in *table* or in a related table to *table*.
- Each name must be enclosed in double quotation marks.
- The function groups a selected set of rows into a set of summary rows by the values of one or more groupBy_columnName columns. One row is returned for each group.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following example returns a summary of the reseller sales grouped around the calendar year and the product category name, this result table allows you to do analysis over the reseller sales by year and product category.

```
DAX

SUMMARIZE(ResellerSales_USD
      , DateTime[CalendarYear]
      , ProductCategory[ProductCategoryName]
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
      )
```

The following table shows a preview of the data as it would be received by any function expecting to receive a table:

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Components	2008052.706	39.9266
2005	Components	574256.9865	0
2006	Components	3428213.05	948.7674
2007	Components	5195315.216	4226.0444
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546

With ROLLUP

The addition of the ROLLUP syntax modifies the behavior of the SUMMARIZE function by adding rollup rows to the result on the groupBy_columnName columns. ROLLUP can only be used within a SUMMARIZE expression.

Example

The following example adds rollup rows to the Group-By columns of the SUMMARIZE function call:

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName])
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
)
```

Returns the following table,

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Components	2008052.706	39.9266
2005	Components	574256.9865	0
2006	Components	3428213.05	948.7674
2007	Components	5195315.216	4226.0444

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546
2008		15496115.89	41224.3038
2005		7582953.67	4326.4144
2006		22871907.85	184419.1335
2007		30543780.84	297538.0745
		76494758.25	527507.9262

With ROLLUPGROUP

The addition of ROLLUPGROUP inside a ROLLUP syntax can be used to prevent partial subtotals in rollup rows. ROLLUPGROUP can only be used within a ROLLUP, ROLLUPADDISSUBTOTAL, or ROLLUPISSUBTOTAL expression.

Example

The following example shows only the grand total of all years and categories without the subtotal of each year with all categories:

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP(ROLLUPGROUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName]))
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
)
```

Returns the following table,

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Components	2008052.706	39.9266
2005	Components	574256.9865	0
2006	Components	3428213.05	948.7674
2007	Components	5195315.216	4226.0444
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546
		76494758.25	527507.9262

With ISSUBTOTAL

With ISSUBTOTAL, you can create another column in the SUMMARIZE expression that returns True if the row contains subtotal values for the column given as argument to ISSUBTOTAL, otherwise returns False. ISSUBTOTAL can only be used within a SUMMARIZE expression.

Example

The following sample generates an ISSUBTOTAL column for each of the ROLLUP columns in the given SUMMARIZE function call:

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName])
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
      , "Is Sub Total for DateTimeCalendarYear", ISSUBTOTAL(DateTime[CalendarYear])
      , "Is Sub Total for ProductCategoryName", ISSUBTOTAL(ProductCategory[ProductCategoryName])
)
```

Returns the following table,

[Is Sub Total for DateTimeCalendarYear]	[Is Sub Total for ProductCategoryName]	DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
FALSE	FALSE				
FALSE	FALSE	2008	Bikes	12968255.42	36167.6592
FALSE	FALSE	2005	Bikes	6958251.043	4231.1621
FALSE	FALSE	2006	Bikes	18901351.08	178175.8399
FALSE	FALSE	2007	Bikes	24256817.5	276065.992
FALSE	FALSE	2008	Components	2008052.706	39.9266
FALSE	FALSE	2005	Components	574256.9865	0
FALSE	FALSE	2006	Components	3428213.05	948.7674
FALSE	FALSE	2007	Components	5195315.216	4226.0444
FALSE	FALSE	2008	Clothing	366507.844	4151.1235
FALSE	FALSE	2005	Clothing	31851.1628	90.9593
FALSE	FALSE	2006	Clothing	455730.9729	4233.039
FALSE	FALSE	2007	Clothing	815853.2868	12489.3835
FALSE	FALSE	2008	Accessories	153299.924	865.5945
FALSE	FALSE	2005	Accessories	18594.4782	4.293

[Is Sub Total for DateTimeCalendarYear]	[Is Sub Total for ProductCategoryName]	DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
FALSE	FALSE	2006	Accessories	86612.7463	1061.4872
FALSE	FALSE	2007	Accessories	275794.8403	4756.6546
FALSE	TRUE				
FALSE	TRUE	2008		15496115.89	41224.3038
FALSE	TRUE	2005		7582953.67	4326.4144
FALSE	TRUE	2006		22871907.85	184419.1335
FALSE	TRUE	2007		30543780.84	297538.0745
TRUE	TRUE			76494758.25	527507.9262

See also

SUMMARIZECOLUMNS

Feedback

Was this page helpful? Yes No

SUMMARIZECOLUMNS

Article • 07/09/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ❌ Visual calculation

Returns a summary table over a set of groups.

Syntax

DAX

```
SUMMARIZECOLUMNS( <groupBy_columnName> [, < groupBy_columnName >]…,
[<filterTable>]…[, <name>, <expression>]…)
```

Parameters

Expand table

Term	Definition
groupBy_columnName	A fully qualified column reference (Table[Column]) to a base table for which the distinct values are included in the returned table. Each groupBy_columnName column is cross-joined (different tables) or auto-existed (same table) with the subsequent specified columns.
filterTable	A table expression which is added to the filter context of all columns specified as groupBy_columnName arguments. The values present in the filter table are used to filter before cross-join/auto-exist is performed.
name	A string representing the column name to use for the subsequent expression specified.
expression	Any DAX expression that returns a single value (not a table).

Return value

A table which includes combinations of values from the supplied columns based on the grouping specified. Only rows for which at least one of the supplied expressions return a non-blank value are included in the table returned. If all expressions evaluate to BLANK/NULL for a row, that row is not included in the table returned.

Remarks

- This function does not guarantee any sort order for the results.
- A column cannot be specified more than once in the groupBy_columnName parameter. For example, the following formula is invalid.

  `SUMMARIZECOLUMNS( Sales[StoreId], Sales[StoreId] )`

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Filter context

Consider the following query:

DAX

```
SUMMARIZECOLUMNS (
    'Sales Territory'[Category],
    FILTER('Customer', 'Customer' [First Name] = "Alicia")
)
```

In this query, without a measure the groupBy columns do not contain any columns from the FILTER expression (for example, from Customer table). The filter is not applied to the groupBy columns. The Sales Territory and Customer tables may be indirectly related through the Reseller sales fact table. Since they're not directly related, the filter expression is a no-op and the groupBy columns are not impacted.

However, with this query:

DAX

```
SUMMARIZECOLUMNS (
    'Sales Territory'[Category], 'Customer' [Education],
    FILTER('Customer', 'Customer'[First Name] = "Alicia")
)
```

The groupBy columns contain a column which is impacted by the filter and that filter is applied to the groupBy results.

With IGNORE

The IGNORE syntax can be used to modify the behavior of the SUMMARIZECOLUMNS function by omitting specific expressions from the BLANK/NULL evaluation. Rows for which all expressions not using IGNORE return BLANK/NULL will be excluded independent of whether the expressions which do use IGNORE evaluate to BLANK/NULL or not. IGNORE can only be used within a SUMMARIZECOLUMNS expression.

Example

DAX

```
SUMMARIZECOLUMNS(
    Sales[CustomerId], "Total Qty",
    IGNORE( SUM( Sales[Qty] ) ),
    "BlankIfTotalQtyIsNot3", IF( SUM( Sales[Qty] )=3, 3 )
)
```

This rolls up the Sales[CustomerId] column, creating a subtotal for all customers in the given grouping. Without IGNORE, the result is:

Expand table

CustomerId	Total Qty	BlankIfTotalQtyIsNot3
A	5	
B	3	3
C	3	3

With IGNORE,

Expand table

CustomerId	Total Qty	BlankIfTotalQtyIsNot3
B	3	3
C	3	3

All expression ignored,

DAX

```
SUMMARIZECOLUMNS(
    Sales[CustomerId], "Blank",
    IGNORE( BLANK() ), "BlankIfTotalQtyIsNot5",
```

```
    IGNORE( IF( SUM( Sales[Qty] )=5, 5 ) )
)
```

Even though both expressions return blank for some rows, they're included since there are no unignored expressions which return blank.

Expand table

CustomerId	Blank	BlankIfTotalQtyIsNot5
A		5
B		
C		

With NONVISUAL

The NONVISUAL function marks a value filter in SUMMARIZECOLUMNS function as not affecting measure values, but only applying to groupBy columns. NONVISUAL can only be used within a SUMMARIZECOLUMNS expression.

Example

DAX

```
DEFINE
MEASURE FactInternetSales[Sales] = SUM(FactInternetSales[Sales Amount])
EVALUATE
SUMMARIZECOLUMNS
(
    DimDate[CalendarYear],
    NONVISUAL(TREATAS({2007, 2008}, DimDate[CalendarYear])),
    "Sales", [Sales],
    "Visual Total Sales", CALCULATE([Sales],
ALLSELECTED(DimDate[CalendarYear]))
)
ORDER BY [CalendarYear]
```

Returns the result where [Visual Total Sales] is the total across all years:

Expand table

DimDate[CalendarYear]	[Sales]	[Visual Total Sales]
2007	9,791,060.30	29,358,677.22
2008	9,770,899.74	29,358,677.22

In contrast, the same query *without* the NONVISUAL function:

DAX

```
DEFINE
MEASURE FactInternetSales[Sales] = SUM(FactInternetSales[Sales Amount])
EVALUATE
SUMMARIZECOLUMNS
(
    DimDate[CalendarYear],
    TREATAS({2007, 2008}, DimDate[CalendarYear]),
    "Sales", [Sales],
    "Visual Total Sales", CALCULATE([Sales],
ALLSELECTED(DimDate[CalendarYear]))
)
ORDER BY [CalendarYear]
```

Returns the result where [Visual Total Sales] is the total across the two selected years:

Expand table

DimDate[CalendarYear]	[Sales]	[Visual Total Sales]
2007	9,791,060.30	19,561,960.04
2008	9,770,899.74	19,561,960.04

With ROLLUPADDISSUBTOTAL

The addition of the ROLLUPADDISSUBTOTAL syntax modifies the behavior of the SUMMARIZECOLUMNS function by adding rollup/subtotal rows to the result based on the groupBy_columnName columns. ROLLUPADDISSUBTOTAL can only be used within a SUMMARIZECOLUMNS expression.

Example with single subtotal

DAX

```
DEFINE
VAR vCategoryFilter =
```

```
  TREATAS({"Accessories", "Clothing"}, Product[Category])
VAR vSubcategoryFilter =
  TREATAS({"Bike Racks", "Mountain Bikes"}, Product[Subcategory])
EVALUATE
  SUMMARIZECOLUMNS
  (
    ROLLUPADDISSUBTOTAL
    (
      Product[Category], "IsCategorySubtotal", vCategoryFilter,
      Product[Subcategory], "IsSubcategorySubtotal", vSubcategoryFilter
    ),
    "Total Qty", SUM(Sales[Qty])
  )
  ORDER BY
  [IsCategorySubtotal] DESC, [Category],
  [IsSubcategorySubtotal] DESC, [Subcategory]
```

Returns the following table,

Expand table

Category	Subcategory	IsCategorySubtotal	IsSubcategorySubtotal	Total Qty
		True	True	60398
Accessories		False	True	36092
Accessories	Bike Racks	False	False	328
Bikes	Mountain Bikes	False	False	4970
Clothing		False	True	9101

Example with multiple subtotals

DAX

```
SUMMARIZECOLUMNS (
    Regions[State], ROLLUPADDISSUBTOTAL ( Sales[CustomerId],
"IsCustomerSubtotal" ),
    ROLLUPADDISSUBTOTAL ( Sales[Date], "IsDateSubtotal"), "Total Qty", SUM(
Sales[Qty] )
)
```

Sales is grouped by state, by customer, by date, with subtotals for 1. Sales by state, by date 2. Sales by State, by Customer 3. Rolled up on both customer and date leading to sales by state.

Returns the following table,

Expand table

CustomerID	IsCustomerSubtotal	State	Total Qty	Date	IsDateSubtotal
A	FALSE	WA	5	7/10/2014	
B	FALSE	WA	1	7/10/2014	
B	FALSE	WA	2	7/11/2014	
C	FALSE	OR	2	7/10/2014	
C	FALSE	OR	1	7/11/2014	
	TRUE	WA	6	7/10/2014	
	TRUE	WA	2	7/11/2014	
	TRUE	OR	2	7/10/2014	
	TRUE	OR	1	7/11/2014	
A	FALSE	WA	5		TRUE
B	FALSE	WA	3		TRUE
C	FALSE	OR	3		TRUE
	TRUE	WA	8		TRUE
	TRUE	OR	3		TRUE

With ROLLUPGROUP

Like with the SUMMARIZE function, ROLLUPGROUP can be used together with ROLLUPADDISSUBTOTAL to specify which summary groups/granularities (subtotals) to include, reducing the number of subtotal rows returned. ROLLUPGROUP can only be used within a SUMMARIZECOLUMNS or SUMMARIZE expression.

Example with multiple subtotals

DAX

```
SUMMARIZECOLUMNS(
    ROLLUPADDISSUBTOTAL( Sales[CustomerId], "IsCustomerSubtotal" ),
    ROLLUPADDISSUBTOTAL(ROLLUPGROUP(Regions[City], Regions[State]),
```

```
"IsCityStateSubtotal"),"Total Qty", SUM( Sales[Qty] )
)
```

Still grouped by City and State, but rolled together when reporting a subtotal returns the following table,

Expand table

State	CustomerId	IsCustomerSubtotal	Total Qty	City	IsCityStateSubtotal
WA	A	FALSE	2	Bellevue	FALSE
WA	B	FALSE	2	Bellevue	FALSE
WA	A	FALSE	3	Redmond	FALSE
WA	B	FALSE	1	Redmond	FALSE
OR	C	FALSE	3	Portland	FALSE
WA		TRUE	4	Bellevue	FALSE
WA		TRUE	4	Redmond	FALSE
OR		TRUE	3	Portland	FALSE
	A	FALSE	5		FALSE
	B	FALSE	3		TRUE
	C	FALSE	3		TRUE
		TRUE	11		TRUE

Contextual SummarizeColumns

Background

Until February 2023, SUMMARIZECOLUMNS did not support evaluation within a context transition at all. In products released before that month, this limitation made SUMMARIZECOLUMNS not useful in most of the measures – it was not possible to call a measure SUMMARIZECOLUMNS in any case of context transition, including other SUMMARIZECOLUMNS statements.

From February 2023, the context transition was supported in a few scenarios, but not in all the conditions. The supported and restricted cases are as follows:

Expand table

SummarizeColumns Type	External Filter with single column	External Filter with more than one column	External GroupBy Columns
SummarizeColumns with GroupBy only	OK	OK	OK
SummarizeColumns with Filters/Measures	OK	ERROR	ERROR

From June 2024, we are enabling contextual SummarizeColumns which allows SummarizeColumns to be evaluated in any context transition, SummarizeColumns in measure is now fully supported:

Expand table

SummarizeColumns Type	External Filter with single column	External Filter with more than one column	External GroupBy Columns
SummarizeColumns with GroupBy only	OK	OK	OK
SummarizeColumns with Filters/Measures	OK	OK	OK

However, this update also includes changes to the behavior of SummarizeColumns, which may alter the results of existing expressions:

SelfValue semantics for external filters

We are introducing a semantic concept named SelfValue, which alters how filters from external tables interact with GroupBy columns in SummarizeColumns. This change disallows filters from a different table to affect the GroupBy columns, even if the tables are related through a filter-by relationship. An example illustrating the impact of this change involves the following expression:

```
CalculateTable(
  SummarizeColumns(
      'Reseller Sales'[ResellerKey],
      'Reseller Sales'[ProductKey]
  ),
```

```
  Treatas({(229)}, 'Product'[Product Key])
)
```

Before this update, the TreatAs filter would apply to the GroupBy operation within SummarizeColumns, leveraging the relationship between 'Product'[Product Key] and 'Reseller Sales'[ProductKey]. Consequently, the query results would only include rows where 'Reseller Sales'[ProductKey] equals 229. However, after the update, GroupBy columns within SummarizeColumns will no longer be filtered by columns from external tables, even if a relationship exists between them. Therefore, in the example above, the GroupBy column 'Reseller Sales'[ProductKey] will not be filtered by the 'Product'[ProductKey] column. As a result, the query will include rows where 'Reseller Sales'[ProductKey] is not equal to 229.

If you prefer to retain the previous behavior, you can rewrite the expression using AddColumns or SelectColumns instead of SummarizeColumns, as shown below:

```
CalculateTable(
    Filter(
        SelectColumns(
            'Reseller Sales',
            "ResellerKey",
            [ResellerKey],
            "ProductKey",
            [ProductKey]
        ),
        And(Not IsBlank([ResellerKey]),  Not IsBlank([ProductKey]))
    ),
    Treatas({(229)}, 'Product'[Product Key])
)
```

This rewritten expression preserves the original semantics where the GroupBy operation is not affected by the SelfValue restriction introduced by the update.

Row validation for groupby columns fully covered by Treatas

Prior to this update, within a SummarizeColumns function, if all GroupBy columns from a specific table were fully covered by a single Treatas filter from that same table, as shown below:

```
```

```
SummarizeColumns(
  Geography[Country],
  Geography[State],
  Treatas(
      {("United States", "Alberta")},
      Geography[Country],
      Geography[State]
  )
)
```

The result of the above query would include whatever rows were specified in the Treatas filter, regardless of whether they were valid or not. For instance, the result would be a single-row table ("United States", "Alberta"), even if no such row with [Country] = "United States" and [State] = "Alberta" existed in the 'Geography' table.

This issue was known and has been addressed by the update. After the update, such invalid rows will be filtered out, and only valid rows from the GroupBy table will be returned. Therefore, the result for the query above would be empty, as there are no valid rows matching the specified [Country] and [State] values in the 'Geography' table.

Disallow mixed Keepfilters/overriddefilters on same table/cluster

The recent update has introduced a temporary restriction that triggers an error message stating:

```
"SummarizeColumns filters with keepfilters behavior and overridefilters
behavior are mixed within one cluster, which is not allowed. Consider adding
keepfilters() to all filters of summarizecolumns."
```

This error occurs when both normal filters (which override existing filters) and filters with KeepFilters specified are present within the same table/cluster. For example:

```
Evaluate CalculateTable(
  SummarizeColumns(
      Product[Color],
      KeepFilters(
          TreatAs(
              {( "Washington")}
              , Geography[State]
          )
      ),
```

```
        TreatAs(
            {("United States"), ("Canada")}
            , Geography[Country]
        )
    )
    ,TreatAs({("Alberta")}, Geography[State])
    ,TreatAs({("Canada")}, Geography[Country])
)
```

In the above expression, there are two filters on the 'Geography' table: one with KeepFilters specified and one without. These filters overlap with external filters on different columns. Currently, this configuration is not allowed because internally, the two filters are clustered into one, and the system cannot determine the correct filter overriding behavior for the clustered filter overall in such cases.

Please note that this restriction is temporary. We are actively developing solutions to remove this limitation in future updates. If you encounter this error, we advise adjusting the filters within SummarizeColumns by adding or removing KeepFilters as necessary to ensure consistent overriding behavior on each table.

Related content

SUMMARIZE

Feedback

Was this page helpful? Yes No

Table constructor

Article • 10/20/2023

Returns a table of one or more columns.

Syntax

DAX

```
{ <scalarExpr1>, <scalarExpr2>, … }
{ ( <scalarExpr1>, <scalarExpr2>, … ), ( <scalarExpr1>, <scalarExpr2>, … ),
… }
```

Parameters

Term	Definition
scalarExprN	Any DAX expression that returns a scalar value.

Return value

A table of one or more columns. When there is only one column, the name of the column is Value. When there are N columns where N > 1, the names of the columns from left to right are Value1, Value2, ..., ValueN.

Remarks

- The first syntax returns a table of a single column. The second syntax returns a table of one or more columns.
- The number of scalar expressions must be the same for all rows.
- When the data types of the values for a column are different in different rows, all values are converted to a common data type.

Example 1

The following DAX queries:

```
DAX
EVALUATE { 1, 2, 3 }
```

and

```
DAX
EVALUATE { (1), (2), (3) }
```

Return the following table of a single column:

[Value]
1
2
3

Example 2

The following DAX query:

```
DAX
EVALUATE
    {
        (1.5, DATE(2017, 1, 1), CURRENCY(199.99), "A"),
        (2.5, DATE(2017, 1, 2), CURRENCY(249.99), "B"),
        (3.5, DATE(2017, 1, 3), CURRENCY(299.99), "C")
    }
```

Returns,

[Value1]	[Value2]	[Value3]	[Value4]
1.5	1/1/2017	199.99	A
2.5	1/2/2017	249.99	B
3.5	1/3/2017	299.99	C

Example 3

The following DAX query:

```
DAX

EVALUATE { 1, DATE(2017, 1, 1), TRUE, "A" }
```

Returns the following table of a single column of String data type:

[Value]
1
1/1/2017
TRUE
A

Feedback

Was this page helpful? Yes No

TOPN

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the top N rows of the specified table.

Syntax

DAX

```
TOPN(<N_Value>, <Table>, <OrderBy_Expression>, [<Order>[,
<OrderBy_Expression>, [<Order>]]…])
```

Parameters

Expand table

Parameter	Definition
N_Value	The number of rows to return. Any DAX expression that returns a scalar value, where the expression is to be evaluated multiple times (for each row/context). See Remarks to better understand when the number of rows returned could be larger than *n_value*.
Table	Any DAX expression that returns a table of data from where to extract the top 'n' rows. See Remarks to better understand when an empty table is returned.
OrderBy_Expression	(Optional) Any DAX expression where the result value is used to sort the table and evaluated for each row of *table*.
Order	(Optional) A value that specifies how to sort *OrderBy_Expression* values: - **0** (zero) or **FALSE**. Sorts in descending order of values of *Order*. Default when *Order* parameter is omitted. - **1** or **TRUE**. Ranks in ascending order of *OrderBy*.

Return value

A table with the top N rows of *Table* or an empty table if *N_Value* is 0 (zero) or less. Rows are not sorted in any particular order.

Remarks

- If there is a tie, in *Order_By* values, at the N-th row of the table, then all tied rows are returned. Then, when there are ties at the N-th row the function might return more than n rows.
- If N_Value is 0 (zero) or less, TOPN returns an empty table.
- TOPN does not guarantee any sort order for the results.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following measure formula returns the top 10 sold products by sales amount.

DAX

```
= SUMX(
        TOPN(
            10,
            SUMMARIZE(
                    InternetSales,
                    InternetSales[ProductKey],
                    "TotalSales", SUM(InternetSales[SalesAmount])
            ),
            [TotalSales], DESC
        ),
        [TotalSales]
)
```

Feedback

Was this page helpful? Yes No

TREATAS

Article • 12/13/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Applies the result of a table expression as filters to columns from an unrelated table.

Syntax

```
DAX
TREATAS(table_expression, <column>[, <column>[, <column>[,…]]]} )
```

Parameters

Expand table

Term	Definition
`table_expression`	An expression that results in a table.
`column`	One or more existing columns. It cannot be an expression.

Return value

A table that contains all the rows in column(s) that are also in table_expression.

Remarks

- The number of columns specified must match the number of columns in the table expression and be in the same order.
- If a value returned in the table expression does not exist in the column, it is ignored. For example, TREATAS({"Red", "Green", "Yellow"}, DimProduct[Color]) sets a filter on column DimProduct[Color] with three values "Red", "Green", and "Yellow". If "Yellow" does not exist in DimProduct[Color], the effective filter values would be "Red" and "Green".

- Best for use when a relationship does not exist between the tables. If you have multiple relationships between the tables involved, consider using USERELATIONSHIP instead.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

In the following example, the model contains two unrelated product tables. If a user applies a filter to DimProduct1[ProductCategory] selecting Bikes, Seats, Tires, the same filter, Bikes, Seats, Tires is applied to DimProduct2[ProductCategory].

DAX

```
CALCULATE(
    SUM(Sales[Amount]),
    TREATAS(
        VALUES(DimProduct1[ProductCategory]),
        DimProduct2[ProductCategory]
    )
)
```

Related content

INTERSECT FILTER USERELATIONSHIP

Feedback

Was this page helpful? Yes No

UNION

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Creates a union (join) table from a pair of tables.

Syntax

DAX

```
UNION(<table_expression1>, <table_expression2> [,<table_expression>]…)
```

Parameters

Expand table

Term	Definition
table_expression	Any DAX expression that returns a table.

Return value

A table that contains all the rows from each of the two table expressions.

Remarks

- The two tables must have the same number of columns.
- Columns are combined by position in their respective tables.
- The column names in the return table will match the column names in table_expression1.
- Duplicate rows are retained.
- The returned table has lineage where possible. For example, if the first column of each table_expression has lineage to the same base column C1 in the model, the first column in the UNION result will have lineage to C1. However, if combined

columns have lineage to different base columns, or if there is an extension column, the resulting column in UNION will have no lineage.

- When data types differ, the resulting data type is determined based on the rules for data type coercion.
- The returned table will not contain columns from related tables.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following expression creates a union by combining the USAInventory table and the INDInventory table into a single table:

```
DAX

UNION(UsaInventory, IndInventory)
```

USAInventory

Expand table

Country/Region	State	Count	Total
USA	CA	5	500
USA	WA	10	900

INDInventory

Expand table

Country/Region	State	Count	Total
IND	JK	20	800
IND	MH	25	1000
IND	WB	10	900

Return table,

Expand table

Country/Region	State	Count	Total
USA	CA	5	500
USA	WA	10	900
IND	JK	20	800
IND	MH	25	1000
IND	WB	10	900

Feedback

Was this page helpful? Yes No

VALUES

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

When the input parameter is a column name, returns a one-column table that contains the distinct values from the specified column. Duplicate values are removed and only unique values are returned. A BLANK value can be added. When the input parameter is a table name, returns the rows from the specified table. Duplicate rows are preserved. A BLANK row can be added.

> **Note**
>
> This function cannot be used to Return values into a cell or column on a worksheet; rather, you use it as an intermediate function, nested in a formula, to get a list of distinct values that can be counted or used to filter or sum other values.

Syntax

```DAX
VALUES(<TableNameOrColumnName>)
```

Parameters

Expand table

Term	Definition
TableName or ColumnName	A column from which unique values are to be returned, or a table from which rows are to be returned.

Return value

When the input parameter is a column name, a single column table. When the input parameter is a table name, a table of the same columns is returned.

Remarks

- When you use the VALUES function in a context that has been filtered, the unique values returned by VALUES are affected by the filter. For example, if you filter by Region, and return a list of the values for City, the list will include only those cities in the regions permitted by the filter. To return all of the cities, regardless of existing filters, you must use the ALL function to remove filters from the table. The second example demonstrates use of ALL with VALUES.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- For best practices when using VALUES, see Use SELECTEDVALUE instead of VALUES.

Related functions

In most scenarios, when the argument is a column name, the results of the VALUES function are identical to those of the **DISTINCT** function. Both functions remove duplicates and return a list of the possible values in the specified column. However, the VALUES function can also return a blank value. This blank value is useful in cases where you are looking up distinct values from a related table, but a value used in the relationship is missing from one table. In database terminology, this is termed a violation of referential integrity. Such mismatches in data can occur when one table is being updated and the related table is not.

When the argument is a table name, the result of the VALUES function returns all rows in the specified table plus a blank row, if there is a violation of referential integrity. The DISTINCT function removes duplicate rows and returns unique rows in the specified table.

> **Note**
>
> The DISTINCT function allows a column name or any valid table expression to be its argument but the VALUES function only accepts a column name or a table name as the argument.

The following table summarizes the mismatch between data that can occur in two related tables when referential integrity is not preserved.

Expand table

MyOrders table	MySales table
June 1	June 1 sales
June 2	June 2 sales
(no order dates have been entered)	June 3 sales

If you use the DISTINCT function to return a list of dates, only two dates would be returned. However, if you use the VALUES function, the function returns the two dates plus an additional blank member. Also, any row from the MySales table that does not have a matching date in the MyOrders table will be "matched" to this unknown member.

Example

The following formula counts the number of unique invoices (sales orders), and produces the following results when used in a report that includes the Product Category Names:

DAX

```
= COUNTROWS(VALUES('InternetSales_USD'[SalesOrderNumber]))
```

Returns

Expand table

Row Labels	Count Invoices
Accessories	18,208
Bikes	15,205
Clothing	7,461
Grand Total	27,659

Related content

FILTER function
COUNTROWS function
Filter functions

Feedback

Was this page helpful? Yes No

Text functions

Article • 10/20/2023

Data Analysis Expressions (DAX) includes a set of text functions based on the library of string functions in Excel, but which have been modified to work with tables and columns in tabular models. This section describes text functions available in the DAX language.

In this category

Function	Description
COMBINEVALUES	Joins two or more text strings into one text string.
CONCATENATE	Joins two text strings into one text string.
CONCATENATEX	Concatenates the result of an expression evaluated for each row in a table.
EXACT	Compares two text strings and returns TRUE if they are exactly the same, FALSE otherwise.
FIND	Returns the starting position of one text string within another text string.
FIXED	Rounds a number to the specified number of decimals and returns the result as text.
FORMAT	Converts a value to text according to the specified format.
LEFT	Returns the specified number of characters from the start of a text string.
LEN	Returns the number of characters in a text string.
LOWER	Converts all letters in a text string to lowercase.
MID	Returns a string of characters from the middle of a text string, given a starting position and length.
REPLACE	REPLACE replaces part of a text string, based on the number of characters you specify, with a different text string.
REPT	Repeats text a given number of times.
RIGHT	RIGHT returns the last character or characters in a text string, based on the number of characters you specify.
SEARCH	Returns the number of the character at which a specific character or text string is first found, reading left to right.
SUBSTITUTE	Replaces existing text with new text in a text string.

Function	Description
TRIM	Removes all spaces from text except for single spaces between words.
UNICHAR	Returns the Unicode character referenced by the numeric value.
UNICODE	Returns the numeric code corresponding to the first character of the text string.
UPPER	Converts a text string to all uppercase letters.
VALUE	Converts a text string that represents a number to a number.

Feedback

Was this page helpful? Yes No

COMBINEVALUES

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Joins two or more text strings into one text string. The primary purpose of this function is to support multi-column relationships in DirectQuery models. See **Remarks** for details.

Syntax

DAX

```
COMBINEVALUES(<delimiter>, <expression>, <expression>[, <expression>]…)
```

Parameters

Expand table

Term	Definition
delimiter	A separator to use during concatenation. Must be a constant value.
expression	A DAX expression whose value will be be joined into a single text string.

Return value

A concatenated string.

Remarks

- The COMBINEVALUES function assumes, but does not validate, that when the input values are different, the output strings are also different. Based on this assumption, when COMBINEVALUES is used to create calculated columns in order to build a relationship that joins multiple columns from two DirectQuery tables, an optimized join condition is generated at query time. For example, if users want to create a relationship between Table1(Column1, Column2) and Table2(Column1, Column2), they can create two calculated columns, one on each table, as:

```DAX
Table1[CalcColumn] = COMBINEVALUES(",", Table1[Column1],
Table1[Column2])
```

and

```DAX
Table2[CalcColumn] = COMBINEVALUES(",", Table2[Column1],
Table2[Column2])
```

And then create a relationship between `Table1[CalcColumn]` and `Table2[CalcColumn]`. Unlike other DAX functions and operators, which are translated literally to the corresponding SQL operators and functions, the above relationship generates a SQL join predicate as:

```DAX
(Table1.Column1 = Table2.Column1 OR Table1.Column1 IS NULL AND
Table2.Column1 IS NULL)
```

and

```DAX
(Table1.Column2 = Table2.Column2 OR Table1.Column2 IS NULL AND
Table2.Column2 IS NULL)
```

- The join predicate can potentially deliver much better query performance than one that involves complex SQL operators and functions.
- The COMBINEVALUES function relies on users to choose the appropriate delimiter to ensure that unique combinations of input values produce distinct output strings but it does not validate that the assumption is true. For example, if users choose `"| "` as the delimiter, but one row in Table1 has `Table1[Column1] = "| "` and `Table2[Column2] = " "`, while one row in Table2 has `Table2[Column1] = " "` and `Table2[Column2] = "| "`, the two concatenated outputs will be the same `"|| "`, which seem to indicate that the two rows are a match in the join operation. The two rows are not joined together if both tables are from the same DirectQuery source although they are joined together if both tables are imported.

Example

The following DAX query:

```
DAX

EVALUATE
DISTINCT (
    SELECTCOLUMNS ( Date, "Month", COMBINEVALUES ( ", ", [MonthName],
[CalendarYear] ) )
)
```

Returns the following single column table:

Expand table

[Month]
January, 2020
February, 2020
March, 2020
April, 2020
May, 2020
June, 2020
July, 2020
August, 2020
September, 2020
October, 2020
November, 2020
December, 2020
January, 2021
January, 2021
February, 2021
March, 2021
April, 2021

[Month]
May, 2021
June, 2021
July, 2021
August, 2021
September, 2021
October, 2021
November, 2021
December, 2021

Feedback

Was this page helpful? Yes No

CONCATENATE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Joins two text strings into one text string.

Syntax

```DAX
CONCATENATE(<text1>, <text2>)
```

Parameters

Expand table

Term	Definition
text1	The first text string to be joined into a single text string. The string can include text or numbers. You can also use column references.
text2	The second text string to be joined into a single text string. The string can include text or numbers. You can also use column references.

Return value

A concatenated string.

Remarks

- The CONCATENATE function joins two text strings into one text string. The joined items can be text, numbers, Boolean values represented as text, or a combination of those items. You can also use a column reference if the column contains appropriate values.
- The CONCATENATE function in DAX accepts only two arguments, whereas the Excel CONCATENATE function accepts up to 255 arguments. If you need to

concatenate multiple columns, you can create a series of calculations or use the concatenation operator (&) to join all of them in a simpler expression.

- If you want to use text strings directly, rather than using a column reference, you must enclose each string in double quotation marks.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example: Concatenation of Literals

The sample formula creates a new string value by combining two string values that you provide as arguments.

DAX

```
= CONCATENATE("Hello ", "World")
```

Example: Concatenation of strings in columns

The sample formula returns the customer's full name as listed in a phone book. Note how a nested function is used as the second argument. This is one way to concatenate multiple strings when you have more than two values that you want to use as arguments.

DAX

```
= CONCATENATE(Customer[LastName], CONCATENATE(", ", Customer[FirstName]))
```

Example: Conditional concatenation of strings in columns

The sample formula creates a new calculated column in the Customer table with the full customer name as a combination of first name, middle initial, and last name. If there is no middle name, the last name comes directly after the first name. If there is a middle name, only the first letter of the middle name is used and the initial letter is followed by a period.

DAX

```
= CONCATENATE( [FirstName]&" ", CONCATENATE( IF( LEN([MiddleName])>1,
LEFT([MiddleName],1)&" ", ""), [LastName]))
```

This formula uses nested CONCATENATE and IF functions, together with the ampersand (**&**) operator, to conditionally concatenate three string values and add spaces as separators.

Example: Concatenation of columns with different data types

The following example demonstrates how to concatenate values in columns that have different data types. If the value that you are concatenating is numeric, the value will be implicitly converted to text. If both values are numeric, both values will be cast to text and concatenated as if they were strings.

Expand table

Product description	Product abbreviation (column 1 of composite key)	Product number (column 2 of composite key)	New generated key column
Mountain bike	MTN	40	MTN40
Mountain bike	MTN	42	MTN42

DAX

```
= CONCATENATE('Products'[Product abbreviation],'Products'[Product number])
```

The CONCATENATE function in DAX accepts only two arguments, whereas the Excel CONCATENATE function accepts up to 255 arguments. If you need to add more arguments, you can use the ampersand (&) operator. For example, the following formula produces the results, MTN-40 and MTN-42.

DAX

```
= [Product abbreviation] & "-" & [Product number]
```

Related content

CONCATENATEX

Feedback

Was this page helpful? Yes No

CONCATENATEX

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Concatenates the result of an expression evaluated for each row in a table.

Syntax

DAX

```
CONCATENATEX(<table>, <expression>[, <delimiter> [, <orderBy_expression> [, <order>]]...])
```

Parameters

Expand table

Term	Definition
table	The table containing the rows for which the expression will be evaluated.
expression	The expression to be evaluated for each row of *table*.
delimiter	(Optional) A separator to use during concatenation.
orderBy_expression	(Optional) Any DAX expression where the result value is used to sort the concatenated values in the output string. It is evaluated for each row of *table*.
order	(Optional) A value that specifies how to sort *orderBy_expression* values, ascending or descending.

The optional **order** parameter accepts the following values:

Expand table

Value	Alternate Values	Description
0 (zero)	FALSE, DESC	Sorts in descending order of values of *orderBy_expression*. This is the default value when the *order* parameter is omitted.

Value	Alternate Values	Description
1	TRUE, ASC	Sorts in ascending order of values of *orderBy_expression*.

Return value

A concatenated string.

Remarks

- This function takes as its first argument a table or an expression that returns a table. The second argument is a column that contains the values you want to concatenate, or an expression that returns a value.
- Concatenated values are not necessarily sorted in any particular order, unless *orderBy_expression* is specified.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

Employees table

Expand table

FirstName	LastName
Alan	Brewer
Michael	Blythe

The following formula:

```
DAX
= CONCATENATEX(Employees, [FirstName] & " " & [LastName], ",")
```

Returns:
"Alan Brewer, Michael Blythe"

Feedback

Was this page helpful? Yes No

EXACT

Applies to: Calculated column Calculated table Measure Visual calculation

Compares two text strings and returns `TRUE` if they're exactly the same, otherwise returns `FALSE`. EXACT is case-sensitive but ignores formatting differences.

Syntax

DAX

```
EXACT(<text1>,<text2>)
```

Parameters

Expand table

Term	Definition
`text1`	The first text string or column that contains text.
`text2`	The second text string or column that contains text.

Return value

True or False. (Boolean)

Example

The following formula in a calculated column in the Product table checks the value of Product for the current row against the value of Model for the current row. It returns True if they're the same, and False if they're different.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
=
EXACT ( [Product], [Model] )
```

Related content

- Text functions

Last updated on 08/04/2026

FIND

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the starting position of one text string within another text string. FIND is case-sensitive.

Syntax

DAX

```
FIND(<find_text>, <within_text>[, [<start_num>][, <NotFoundValue>]])
```

Parameters

Expand table

Term	Definition
`find_text`	The text you want to find. Use double quotes (empty text) to match the first character in `within_text`.
`within_text`	The text containing the text you want to find.
`start_num`	(optional) The character at which to start the search; if omitted, `start_num` = 1. The first character in `within_text` is character number 1.
`NotFoundValue`	(optional, but strongly recommended) The value to return when the operation doesn't find a matching substring, typically 0, -1, or BLANK(). If you don't specify it, the function returns an error.

Return value

Number that shows the starting point of the text string you want to find.

Remarks

- Whereas Microsoft Excel has multiple versions of the FIND function to accommodate single-byte character set (SBCS) and double-byte character set (DBCS) languages, DAX uses Unicode and counts each character the same way; therefore, you don't need to use a different version depending on the character type.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- FIND doesn't support wildcards. To use wildcards, use SEARCH.
- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.

Example

The following DAX query finds the position of the first letter of "Bike", in the string that contains the reseller name. If not found, FIND returns Blank.

FIND is case-sensitive. In this example, if you used "bike" in the `find_text` argument, the query returns no results. Use SEARCH for case-insensitive.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
EVALUATE
CALCULATETABLE (
    ADDCOLUMNS (
        TOPN ( 10, SUMMARIZE ( 'Reseller', [Reseller], [Business Type] ) ),
        "Position of Bike", FIND ( "Bike", 'Reseller'[Reseller], 1, BLANK () )
    ),
    'Reseller'[Business Type]
        IN { "Specialty Bike Shop", "Value Added Reseller", "Warehouse" }
)
```

Returns,

Expand table

Reseller	Business Type	Position of Bike
Volume Bike Sellers	Warehouse	8
Mass Market Bikes	Value Added Reseller	13
Twin Cycles	Value Added Reseller	
Rich Department Store	Warehouse	
Rental Gallery	Specialty Bike Shop	

Reseller	Business Type	Position of Bike
Budget Toy Store	Warehouse	
Global Sports Outlet	Warehouse	
Online Bike Catalog	Warehouse	8
Helmets and Cycles	Value Added Reseller	
Jumbo Bikes	Specialty Bike Shop	7

Related content

- SEARCH
- Text functions

Last updated on 08/04/2026

FIXED

Applies to: Calculated column Calculated table Measure Visual calculation

Rounds a number to the specified number of decimals and returns the result as text. You can specify that the result be returned with or without commas.

Syntax

```
DAX
FIXED(<number>, <decimals>, <no_commas>)
```

Parameters

Expand table

Term	Definition
`number`	The number you want to round and convert to text, or a column containing a number.
`decimals`	(optional) The number of digits to the right of the decimal point; if omitted, 2.
`no_commas`	(optional) A logical value: if 1, don't display commas in the returned text; if 0 or omitted, display commas in the returned text.

Return value

A number represented as text.

Remarks

- If the value used for the `decimals` parameter is negative, `number` is rounded to the left of the decimal point.
- If you omit `decimals`, it's assumed to be 2.
- If `no_commas` is 0 or omitted, then the returned text includes commas as usual.
- The major difference between formatting a cell containing a number by using a command and formatting a number directly with the FIXED function is that FIXED converts its result to

text. A number formatted with a command from the formatting menu is still a number.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following formula used in a calculated column gets the numeric value for the current row in Product[List Price] and returns it as text with 2 decimal places and no commas.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
=
FIXED ( [List Price], 2, 1 )
```

Related content

- CEILING
- FLOOR
- ISO.CEILING
- MROUND
- ROUND
- ROUNDDOWN
- ROUNDUP

Last updated on 06/29/2026

FORMAT

Article • 10/20/2023

Converts a value to text according to the specified format.

Syntax

DAX

```
FORMAT(<value>, <format_string>[, <locale_name>])
```

Parameters

Term	Definition
value	A value or expression that evaluates to a single value.
format_string	A string with the formatting template.
locale_name	(Optional) Name of the locale to be used by the function. Possible values are strings accepted by the Windows API function LocaleNameToLCID().

Return value

A string containing **value** formatted as defined by **format_string**.

> **Note**
>
> If **value** is BLANK, the function returns an empty string.
>
> If **format_string** is BLANK, the value is formatted with a "General Number" or "General Date" format (according to **value** data type).

Remarks

- Predefined format strings use the model culture property when formatting the result. By default, the model culture property is set according to the user locale of the computer. For new Power BI Desktop models, the model culture property can be changed in Options > Regional Settings > Model language. For Analysis

Services, model culture is set according to the Language property initially defined by the instance.

- The format strings supported as an argument to the DAX FORMAT function are based on the format strings used by Visual Basic (OLE Automation), not on the format strings used by the .NET Framework. Therefore, you might get unexpected results or an error if the argument doesn't match any defined format strings. For example, "p" as an abbreviation for "Percent" isn't supported. Strings that you provide as an argument to the FORMAT function that aren't included in the list of predefined format strings are handled as part of a custom format string, or as a string literal.
- To learn more specifying a locale with FORMAT, check out this video.
- Using FORMAT changes a measure result to a text data type. If the measure result is originally of numeric data type, then with FORMAT, the measure can't be used on visuals where the values section requires a numeric data type, like with charts. In Power BI, you can alternatively use **Dynamic format strings for measures** specify a conditional format string that maintains the numeric data type of the measure.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Examples

Format strings

DAX

```
= FORMAT( 12345.67, "General Number")
= FORMAT( 12345.67, "Currency")
= FORMAT( 12345.67, "Fixed")
= FORMAT( 12345.67, "Standard")
= FORMAT( 12345.67, "Percent")
= FORMAT( 12345.67, "Scientific")
```

Returns:

12345.67 "General Number" displays the number with no formatting.

$12,345.67 "Currency" displays the number with your currency locale formatting. The sample here shows the default United States currency formatting.

12345.67 "Fixed" displays at least one digit to the left of the decimal separator and two digits to the right of the decimal separator.

12,345.67 "Standard" displays at least one digit to the left of the decimal separator and two digits to the right of the decimal separator, and includes thousand separators. The sample here shows the default United States number formatting.

1,234,567.00 % "Percent" displays the number as a percentage (multiplied by 100) with formatting and the percent sign at the right of the number separated by a single space.

1.23E+04 "Scientific" displays the number in scientific notation with two decimal digits.

Datetime with optional locale_name

DAX

```
= FORMAT( dt"2020-12-15T12:30:59", BLANK(), "en-US" )
= FORMAT( dt"2020-12-15T12:30:59", BLANK(), "en-GB" )
= FORMAT( dt"2020-12-15T12:30:59", "mm/dd/yyyy", "en-GB" )
```

Returns:

12/15/2020 12:30:59 PM Where month precedes day and time is 12-hour format.

15/12/2020 12:30:59 Where day precedes month and time is 24-hour format.

12/15/2020 12:30:59 Where month precedes day and time is 24-hour format. Because a non-locale dependent format string is specified, the locale is not applied and the non-locale format is returned.

Predefined numeric formats

The following predefined numeric formats can be specified in the **format_string** argument:

Format	Description
`"General Number"`	Displays number with no thousand separators.
`"Currency"`	Displays number with thousand separators, if appropriate; displays two digits to the right of the decimal separator. Output is based on system locale settings.
`"Fixed"`	Displays at least one digit to the left and two digits to the right of the decimal separator.

Format	Description
`"Standard"`	Displays number with thousand separators, at least one digit to the left and two digits to the right of the decimal separator.
`"Percent"`	Displays number multiplied by 100 with a percent sign (%) appended immediately to the right; always displays two digits to the right of the decimal separator.
`"Scientific"`	Uses standard scientific notation, providing two significant digits.
`"Yes/No"`	Displays No if number is 0; otherwise, displays Yes.
`"True/False"`	Displays False if number is 0; otherwise, displays True.
`"On/Off"`	Displays Off if number is 0; otherwise, displays On.

Custom numeric formats

A custom format expression for numbers can have from one to three sections separated by semicolons. If the format string argument contains one of the named numeric formats, only one section is allowed.

If you use	The result is
One section only	The format expression applies to all values.
Two sections	The first section applies to positive values and zeros, the second to negative values.
Three sections	The first section applies to positive values, the second to negative values, and the third to zeros.

DAX

```
"$#,##0;($#,##0)"
```

If you include semicolons with nothing between them, the missing section is defined using the format of the positive value. For example, the following format displays positive and negative values using the format in the first section and displays "Zero" if the value is zero.

DAX

```
"$#,##0"
```

If you include semicolons with nothing between them, the missing section is shown using the format of the positive value.

Custom numeric format characters

The following custom numeric format characters can be specified in the **format_string** argument:

Character	Description
None	Display the number with no formatting.
(**0**)	Digit placeholder. Display a digit or a zero. If the expression has a digit in the position where the 0 appears in the format string, display it; otherwise, display a zero in that position.If the number has fewer digits than there are zeros (on either side of the decimal) in the format expression, display leading or trailing zeros. If the number has more digits to the right of the decimal separator than there are zeros to the right of the decimal separator in the format expression, round the number to as many decimal places as there are zeros. If the number has more digits to the left of the decimal separator than there are zeros to the left of the decimal separator in the format expression, display the extra digits without modification.
(**#**)	Digit placeholder. Display a digit or nothing. If the expression has a digit in the position where the # appears in the format string, display it; otherwise, display nothing in that position. This symbol works like the 0 digit placeholder, except that leading and trailing zeros aren't displayed if the number has the same or fewer digits than there are # characters on either side of the decimal separator in the format expression.
(**.**)	Decimal placeholder. In some locales, a comma is used as the decimal separator. The decimal placeholder determines how many digits are displayed to the left and right of the decimal separator. If the format expression contains only number signs to the left of this symbol, numbers smaller than 1 begin with a decimal separator. To display a leading zero displayed with fractional numbers, use 0 as the first digit placeholder to the left of the decimal separator. The actual character used as a decimal placeholder in the formatted output depends on the Number Format recognized by your system.
(**%**)	Percentage placeholder. The expression is multiplied by 100. The percent character (%) is inserted in the position where it appears in the format string.
(**,**)	Thousand separator. In some locales, a period is used as a thousand separator. The thousand separator separates thousands from hundreds within a number that has four or more places to the left of the decimal separator. Standard use of the thousand separator is specified if the format contains a thousand separator

Character	Description
	surrounded by digit placeholders (**0** or #). Two adjacent thousand separators or a thousand separator immediately to the left of the decimal separator (whether or not a decimal is specified) means "scale the number by dividing it by 1000, rounding as needed." For example, you can use the format string "##0,," to represent 100 million as 100. Numbers smaller than 1 million are displayed as 0. Two adjacent thousand separators in any position other than immediately to the left of the decimal separator are treated simply as specifying the use of a thousand separator. The actual character used as the thousand separator in the formatted output depends on the Number Format recognized by your system.
(:)	Time separator. In some locales, other characters may be used to represent the time separator. The time separator separates hours, minutes, and seconds when time values are formatted. The actual character used as the time separator in formatted output is determined by your system settings.
(/)	Date separator. In some locales, other characters may be used to represent the date separator. The date separator separates the day, month, and year when date values are formatted. The actual character used as the date separator in formatted output is determined by your system settings.
(**E- E+ e- e+**)	Scientific format. If the format expression contains at least one digit placeholder (**0** or #) to the right of E-, E+, e-, or e+, the number is displayed in scientific format and E or e is inserted between the number and its exponent. The number of digit placeholders to the right determines the number of digits in the exponent. Use E- or e- to place a minus sign next to negative exponents. Use E+ or e+ to place a minus sign next to negative exponents and a plus sign next to positive exponents.
- + $ ( )	Display a literal character. To display a character other than one of those listed, precede it with a backslash (`\`) or enclose it in double quotation marks (" ").
(\)	Display the next character in the format string. To display a character that has special meaning as a literal character, precede it with a backslash (`\`). The backslash itself isn't displayed. Using a backslash is the same as enclosing the next character in double quotation marks. To display a backslash, use two backslashes (`\\`). Examples of characters that can't be displayed as literal characters are the date-formatting and time-formatting characters (a, c, d, h, m, n, p, q, s, t, w, y, /, and :), the numeric-formatting characters (#, 0, %, E, e, comma, and period), and the string-formatting characters (@, &, <, >, and !).
("ABC")	Display the string inside the double quotation marks (" ").

Predefined date/time formats

The following predefined date/time formats can be specified in the **format_string** argument. When using formats other than these, they are interpreted as a custom date/time format:

Format	Description
`"General Date"`	Displays a date and/or time. For example, 3/12/2008 11:07:31 AM. Date display is determined by your application's current culture value.
`"Long Date"` or `"Medium Date"`	Displays a date according to your current culture's long date format. For example, Wednesday, March 12, 2008.
`"Short Date"`	Displays a date using your current culture's short date format. For example, 3/12/2008.
`"Long Time"` or	Displays a time using your current culture's long time format; typically includes hours, minutes, seconds. For example, 11:07:31 AM.
`"Medium Time"`	Displays a time in 12 hour format. For example, 11:07 AM.
`"Short Time"`	Displays a time in 24 hour format. For example, 11:07.

Custom date/time formats

The following format characters can be specified in the **format_string** to create custom date/time formats:

Character	Description
(:)	Time separator. In some locales, other characters may be used to represent the time separator. The time separator separates hours, minutes, and seconds when time values are formatted. The actual character used as the time separator in formatted output is determined by your system settings.
(/)	Date separator. In some locales, other characters may be used to represent the date separator. The date separator separates the day, month, and year when date values are formatted. The actual character used as the date separator in formatted output is determined by your system settings.
(\)	Backslash. Displays the next character as a literal character. So, it's not interpreted as a formatting character.
(")	Double quote. Text enclosed within double quotes is displayed. So, it's not interpreted as formatting characters.
c	Display the date as `ddddd` and display the time as `ttttt`, in that order. Display only date information if there is no fractional part to the date serial number; display only time information if there is no integer portion.
d	Display the day as a number without a leading zero (1-31).
dd	Display the day as a number with a leading zero (01-31).

Character	Description
ddd	Display the day as an abbreviation (Sun-Sat). Localized.
dddd	Display the day as a full name (Sunday-Saturday). Localized.
ddddd	Display the date as a complete date (including day, month, and year), formatted according to your system's short date format setting. The default short date format is `mm/dd/yyyy`.
dddddd	Display a date serial number as a complete date (including day, month, and year) formatted according to the long date setting recognized by your system. The default long date format is `dddd, mmmm d, yyyy`.
w	Display the day of the week as a number (1 for Sunday through 7 for Saturday).
ww	Display the week of the year as a number (1-54).
m	Display the month as a number without a leading zero (1-12). If `m` immediately follows `h` or `hh`, minute rather than the month is displayed.
mm	Display the month as a number with a leading zero (01-12). If `mm` immediately follows `h` or `hh`, minute rather than the month is displayed.
mmm	Display the month as an abbreviation (Jan-Dec). Localized.
mmmm	Display the month as a full month name (January-December). Localized.
q	Display the quarter of the year as a number (1-4).
y	Display the day of the year as a number (1-366).
yy	Display the year as a 2-digit number (00-99).
yyyy	Display the year as a 4-digit number (100-9999).
h	Display the hour as a number without a leading zero (0-23).
hh	Display the hour as a number with a leading zero (00-23).
n	Display the minute as a number without a leading zero (0-59).
nn	Display the minute as a number with a leading zero (00-59).
s	Display the second as a number without a leading zero (0-59).
ss	Display the second as a number with a leading zero (00-59).
ttttt	Display a time as a complete time (including hour, minute, and second), formatted using the time separator defined by the time format recognized by your system. A leading zero is displayed if the leading zero option is selected and the time is before 10:00 A.M. or P.M. The default time format is `h:mm:ss`.

Character	Description
AM/PM	Use the 12-hour clock and display an uppercase AM with any hour before noon; display an uppercase PM with any hour between noon and 11:59 P.M.
am/pm	Use the 12-hour clock and display a lowercase AM with any hour before noon; display a lowercase PM with any hour between noon and 11:59 P.M.
A/P	Use the 12-hour clock and display an uppercase A with any hour before noon; display an uppercase P with any hour between noon and 11:59 P.M.
a/p	Use the 12-hour clock and display a lowercase A with any hour before noon; display a lowercase P with any hour between noon and 11:59 P.M.
AMPM	Use the 12-hour clock and display the AM string literal as defined by your system with any hour before noon; display the PM string literal as defined by your system with any hour between noon and 11:59 P.M. AMPM can be either uppercase or lowercase, but the case of the string displayed matches the string as defined by your system settings. The default format is AM/PM. If your system is set to 24-hour clock, the string is typical set to an empty string.

Date/time formatting uses the current user locale to format the string. For example, consider the date June 25, 2020. When it's formatted using format string "m/d/yyyy" it will be:

- User locale is United States of America (en-US): "6/25/2020"
- User locale is Germany (de-DE): "6.25.2020"

Custom date/time format examples

The following examples use the date/time Thursday, June 25, 2020, at 1:23:45 PM. Germany (de-DE) uses a 24-hour system. There's no equivalent of AM/PM.

Format	Result (en-US)	Result (de-DE)
`"c"`	06/25/2020 13:23:45	25.06.2020 13:23:45
`"d"`	25	25
`"dd"`	25	25
`"ddd"`	Thu	Do
`"dddd"`	Thursday	Donnerstag
`"ddddd"`	06/25/2020	25.06.2020
`"dddddd"`	Thursday, June 25, 2020	Donnerstag, 25. Juni 2020

Format	Result (en-US)	Result (de-DE)
`"w"`	5	5
`"ww"`	26	26
`"m"`	6	6
`"mm"`	06	06
`"mmm"`	Jun	Jun
`"mmmm"`	June	Juni
`"q"`	2	2
`"y"`	177	177
`"yy"`	20	20
`"yyyy"`	2020	2020
`"""Year"" yyyy"`	Year 2020	Year 2020
`"yyyy \Qq"`	2020 Q2	2020 Q2
`"dd/mm/yyyy"`	25/06/2020	25.06.2020
`"mm/dd/yyyy"`	06/25/2020	06.25.2020
`"h:nn:ss"`	13:23:45	13:23:45
`"h:nn:ss AMPM"`	1:23:45 PM	1:23:45
`"hh:nn:ss"`	13:23:45	13:23:45
`"hh:nn:ss AMPM"`	01:23:45 PM	01:23:45
`"ttttt"`	13:23:45	13:23:45
`"ttttt AMPM"`	13:23:45 PM	13:23:45
`"mm/dd/yyyy hh:nn:ss AMPM"`	06/25/2020 01:23:45 PM	6.25.2020 01:23:45

Feedback

Was this page helpful? Yes No

LEFT

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the specified number of characters from the start of a text string.

Syntax

```
DAX
LEFT(<text>, <num_chars>)
```

Parameters

Expand table

Term	Definition
`text`	The text string containing the characters you want to extract, or a reference to a column that contains text.
`num_chars`	(optional) The number of characters you want LEFT to extract; if omitted, 1.

Return value

A text string.

Remarks

- Whereas Microsoft Excel contains different functions for working with text in single-byte and double-byte character languages, DAX works with Unicode and stores all characters as the same length; therefore, a single function is enough.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.

Example

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

The following example returns the first five characters of the company city in the column [City] and the first five letters of the reseller key in the column [ResellerKey] and concatenates them, to create an identifier.

DAX

```
=
CONCATENATE (
    LEFT ( 'Reseller'[City], 5 ),
    LEFT ( 'Reseller'[ResellerKey], 5 )
)
```

If the `num_chars` argument is larger than the number of characters available, the function returns the maximum characters available and doesn't raise an error. For example, the column [ResellerKey] contains numbers such as 5, 24, and 312, so the result also has variable length.

Related content

- Text functions

Last updated on 08/04/2026

LEN

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number of characters in a text string.

Syntax

```
DAX
LEN(<text>)
```

Parameters

Expand table

Term	Definition
`text`	The text whose length you want to find, or a column that contains text. Spaces count as characters.

Return value

A whole number indicating the number of characters in the text string.

Remarks

- Whereas Microsoft Excel has different functions for working with single-byte and double-byte character languages, DAX uses Unicode and stores all characters with the same length.
- If you use LEN with a column that contains non-text values, such as dates or Booleans, the function implicitly casts the value to text, using the current column format.
- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.

Example

The following formula sums the lengths of addresses in the columns, [AddressLine1] and [AddressLine2].

DAX

```
= LEN([AddressLine1])+LEN([AddressLin2])
```

Last updated on 01/20/2026

LOWER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts all letters in a text string to lowercase.

Syntax

```
DAX
LOWER(<text>)
```

Parameters

Expand table

Term	Definition
text	The text you want to convert to lowercase, or a reference to a column that contains text.

Return value

Text in lowercase.

Remarks

Characters that are not letters are not changed. For example, the formula `= LOWER("123ABC")` returns **123abc**.

Example

The following formula gets each row in the column, [ProductCode], and converts the value to all lowercase. Numbers in the column are not affected.

```
DAX
```

```
= LOWER('New Products'[ProductCode])
```

Related content

Text functions

Feedback

Was this page helpful? Yes No

MID

Applies to: Calculated column Calculated table Measure Visual calculation

Returns a string of characters from the middle of a text string, given a starting position and length.

Syntax

```
DAX
MID(<text>, <start_num>, <num_chars>)
```

Parameters

Expand table

Term	Definition
`text`	The text string from which you want to extract the characters, or a column that contains text.
`start_num`	The position of the first character you want to extract. Positions start at 1.
`num_chars`	The number of characters to return.

Return value

A string of text of the specified length.

Remarks

- Whereas Microsoft Excel has different functions for working with single-byte and double-byte characters languages, DAX uses Unicode and stores all characters with the same length.
- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.

Examples

The following expression,

DAX

```
MID("abcde",2,3))
```

Returns `"bcd"`.

The following expression,

DAX

```
MID('Reseller'[ResellerName],1,5))
```

Returns the same result as `LEFT([ResellerName],5)`. Both expressions return the first 5 letters of column, `[ResellerName]`.

Related content

Text functions

Last updated on 01/20/2026

REPLACE

Applies to: Calculated column Calculated table Measure Visual calculation

REPLACE replaces part of a text string, based on the number of characters you specify, with a different text string.

Syntax

```
DAX
REPLACE(<old_text>, <start_num>, <num_chars>, <new_text>)
```

Parameters

Expand table

Term	Definition
`old_text`	The string of text that contains the characters you want to replace, or a reference to a column that contains text.
`start_num`	The position of the character in `old_text` that you want to replace with `new_text`.
`num_chars`	The number of characters that you want to replace. **Warning:** If the argument, `num_chars`, is a blank or references a column that evaluates to a blank, the string for `new_text` is inserted at the position, `start_num`, without replacing any characters. This is the same behavior as in Excel.
`new_text`	The replacement text for the specified characters in `old_text`.

Return value

A text string.

Remarks

- Whereas Microsoft Excel has different functions for use with single-byte and double-byte character languages, DAX uses Unicode and therefore stores all characters as the same length.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.
- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.

Example

The following formula creates a new calculated column that replaces the first two characters of the product code in column, [ProductCode], with a new two-letter code, OB.

DAX

```
= REPLACE('New Products'[Product Code],1,2,"OB")
```

Related content

Text functions SUBSTITUTE function

Last updated on 01/20/2026

REPT

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Repeats text a given number of times. Use REPT to fill a cell with a number of instances of a text string.

Syntax

```
DAX

REPT(<text>, <num_times>)
```

Parameters

Expand table

Term	Definition
text	The text you want to repeat.
num_times	A positive number specifying the number of times to repeat text.

Return value

A string containing the changes.

Remarks

- If **number_times** is 0 (zero), REPT returns a blank.
- If **number_times** is not an integer, it is truncated.
- The result of the REPT function cannot be longer than 32,767 characters, or REPT returns an error.

Example: Repeating Literal Strings

The following example returns the string, 85, repeated three times.

```
DAX
= REPT("85",3)
```

Example: Repeating Column Values

The following example returns the string in the column, [MyText], repeated for the number of times in the column, [MyNumber]. Because the formula extends for the entire column, the resulting string depends on the text and number value in each row.

```
DAX
= REPT([MyText],[MyNumber])
```

Expand table

MyText	MyNumber	CalculatedColumn1
Text	2	TextText
Number	0	
85	3	858585

Related content

Text functions

Feedback

Was this page helpful? Yes No

RIGHT

Applies to: Calculated column Calculated table Measure Visual calculation

RIGHT returns the last character or characters in a text string, based on the number of characters you specify.

Syntax

```
DAX
RIGHT(<text>, <num_chars>)
```

Parameters

Expand table

Term	Definition
`text`	The text string that contains the characters you want to extract, or a reference to a column that contains text.
`num_chars`	(optional) The number of characters you want RIGHT to extract; is omitted, 1. You can also use a reference to a column that contains numbers.

If the column reference does not contain text, it is implicitly cast as text.

Return value

A text string containing the specified right-most characters.

Remarks

- This function returns different results depending on the UnicodeCharacterBehavior setting of your model.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example: Returning a Fixed Number of Characters

The following formula returns the last two digits of the product code in the New Products table.

DAX

```
= RIGHT('New Products'[ProductCode],2)
```

Example: Using a Column Reference to Specify Character Count

The following formula returns a variable number of digits from the product code in the New Products table, depending on the number in the column, MyCount. If there is no value in the column, MyCount, or the value is a blank, RIGHT also returns a blank.

DAX

```
= RIGHT('New Products'[ProductCode],[MyCount])
```

Related content

Text functions LEFT MID

Last updated on 01/20/2026

SEARCH

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Returns the number of the character at which a specific character or text string is first found, reading left to right. Search is case-insensitive, kanatype-insensitive, width-insensitive, and accent sensitive.

Syntax

DAX

```
SEARCH(<find_text>, <within_text>[, [<start_num>][, <NotFoundValue>]])
```

Parameters

Expand table

Term	Definition
`find_text`	The text that you want to find. You can use wildcard characters — the question mark (?) and asterisk (*) — in `find_text`. A question mark matches any single character; an asterisk matches any sequence of characters. If you want to find an actual question mark or asterisk, type a tilde (~) before the character.
`within_text`	The text in which you want to search for `find_text`, or a column containing text.
`start_num`	(optional) The character position in `within_text` at which you want to start searching. If omitted, 1.
`NotFoundValue`	(optional, but strongly recommended) The value to return when the operation doesn't find a matching substring, typically 0, -1, or BLANK(). If you don't specify it, the function returns an error.

Return value

The number of the starting position of the first text string from the first character of the second text string.

Remarks

- The search function is case insensitive. Searching for "N" finds the first occurrence of 'N' or 'n'.
- The search function is kanatype-insensitive, width-insensitive. Searching for "ヵ" finds the first occurrence of 「か」 (hiragana), 「カ」 (katakana), or 「ヵ」 (half-width katakana).
- The search function is accent sensitive. Searching for "á" finds the first occurrence of 'á' but no occurrences of 'a', 'à', or the capitalized versions 'A', 'Á'.
- You can use the SEARCH function to determine the location of a character or text string within another text string, and then use the MID function to return the text, or use the REPLACE function to change the text.
- If the `find_text` can't be found in `within_text`, the formula returns an error. This behavior is like Excel, which returns #VALUE if the substring isn't found. Nulls in `within_text` are interpreted as an empty string in this context.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following DAX query finds the position of the first letter of "cycle", in the string that contains the reseller name. If not found, SEARCH returns Blank.

SEARCH is case-insensitive. In this example, if you use "cycle" or "Cycle" in the `find_text` argument, the query returns results for either case. Use FIND for case-sensitive.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
EVALUATE
CALCULATETABLE (
    ADDCOLUMNS (
        TOPN ( 10, SUMMARIZE ( 'Reseller', [Reseller], [Business Type] ) ),
        "Position of cycle", SEARCH ( "cycle", 'Reseller'[Reseller], 1, BLANK () )
    ),
    'Reseller'[Business Type]
        IN { "Specialty Bike Shop", "Value Added Reseller", "Warehouse" }
)
```

Returns,

Expand table

Reseller	Business Type	Position of cycle
Volume Bike Sellers	Warehouse	
Mass Market Bikes	Value Added Reseller	
Twin Cycles	Value Added Reseller	6
Rich Department Store	Warehouse	
Rental Gallery	Specialty Bike Shop	
Budget Toy Store	Warehouse	
Global Sports Outlet	Warehouse	
Online Bike Catalog	Warehouse	
Helmets and Cycles	Value Added Reseller	13
Jumbo Bikes	Specialty Bike Shop	

Related content

- FIND
- REPLACE
- Text functions

Last updated on 08/04/2026

SUBSTITUTE

Article • 04/25/2024

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ✅ Visual calculation

Replaces existing text with new text in a text string.

Syntax

DAX

```
SUBSTITUTE(<text>, <old_text>, <new_text>, <instance_num>)
```

Parameters

Expand table

Term	Definition
text	The text in which you want to substitute characters, or a reference to a column containing text.
old_text	The existing text that you want to replace.
new_text	The text you want to replace **old_text** with.
instance_num	(optional) The occurrence of **old_text** you want to replace. If omitted, every instance of **old_text** is replaced

Return value

A string of text.

Remarks

- Use the SUBSTITUTE function when you want to replace specific text in a text string; use the REPLACE function when you want to replace any text of variable length that occurs in a specific location in a text string.

- The SUBSTITUTE function is case-sensitive. If case does not match between **text** and **old_text**, SUBSTITUTE will not replace the text.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example: Substitution within a String

The following formula creates a copy of the column [Product Code] that substitutes the new product code **NW** for the old product code **PA** wherever it occurs in the column.

DAX

```
= SUBSTITUTE([Product Code], "NW", "PA")
```

Related content

Text functions
REPLACE

Feedback

Was this page helpful? Yes No

TRIM

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Removes all spaces from text except for single spaces between words.

Syntax

```DAX
TRIM(<text>)
```

Parameters

Expand table

Term	Definition
text	The text from which you want spaces removed, or a column that contains text.

Return value

The string with spaces removed.

Remarks

- Use TRIM on text that you have received from another application that may have irregular spacing.
- The TRIM function was originally designed to trim the 7-bit ASCII space character (value 32) from text. In the Unicode character set, there is an additional space character called the nonbreaking space character that has a decimal value of 160. This character is commonly used in Web pages as the HTML entity, . By itself, the TRIM function does not remove this nonbreaking space character. For an example of how to trim both space characters from text, see Remove spaces and nonprinting characters from text.

Example

The following formula creates a new string that does not have trailing white space.

```
DAX
= TRIM("A column with trailing spaces.   ")
```

When you create the formula, the formula is propagated through the row just as you typed it, so that you see the original string in each formula and the results are not apparent. However, when the formula is evaluated the string is trimmed.

You can verify that the formula produces the correct result by checking the length of the calculated column created by the previous formula, as follows:

```
DAX
= LEN([Calculated Column 1])
```

Related content

Text functions

Feedback

Was this page helpful? Yes No

UNICHAR

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the Unicode character referenced by the numeric value.

Syntax

```
DAX
UNICHAR(number)
```

Parameters

Expand table

Term	Definition
number	The Unicode number that represents the character.

Return value

A character represented by the Unicode number.

Remarks

- If XML characters are not invalid, UNICHAR returns an error.
- If Unicode numbers are partial surrogates and data types are not valid, UNICHAR returns an error.
- If numbers are numeric values that fall outside the allowable range, UNICHAR returns an error.
- If number is zero (0), UNICHAR returns an error.
- The Unicode character returned can be a string of characters, for example in UTF-8 or UTF-16 codes.

Example

The following example returns the character represented by the Unicode number 66 (uppercase A).

```
DAX

= UNICHAR(65)
```

The following example returns the character represented by the Unicode number 32 (space character).

```
DAX

= UNICHAR(32)
```

The following example returns the character represented by the Unicode number 9733 (★ character).

```
DAX

= UNICHAR(9733)
```

Feedback

Was this page helpful? Yes No

UNICODE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Returns the number (code point) corresponding to the first character of the text.

Syntax

DAX

```
UNICODE( <Text> )
```

Parameters

Expand table

Term	Definition
Text	Text is the character for which you want the Unicode value.

Return value

A numeric code for the first character in a text string.

Feedback

Was this page helpful? Yes No

UPPER

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts a text string to all uppercase letters.

Syntax

```
DAX

UPPER (<text>)
```

Parameters

Expand table

Term	Definition
text	The text you want converted to uppercase, or a reference to a column that contains text.

Return value

Same text, in uppercase.

Example

The following formula converts the string in the column, [ProductCode], to all uppercase. Non-alphabetic characters are not affected.

```
DAX

= UPPER(['New Products'[Product Code])
```

Related content

Feedback

Was this page helpful? Yes No

VALUE

Article • 04/25/2024

Applies to: Calculated column Calculated table Measure Visual calculation

Converts a text string that represents a number to a number.

Syntax

```DAX
VALUE(<text>)
```

Parameters

Expand table

Term	Definition
text	The text to be converted.

Return value

The converted number in decimal data type.

Remarks

- The value passed as the **text** parameter can be in any of the constant, number, date, or time formats recognized by the application or services you are using. If **text** is not in one of these formats, an error is returned.
- You do not generally need to use the VALUE function in a formula because the engine implicitly converts text to numbers as necessary.
- You can also use column references. For example, if you have a column that contains mixed number types, VALUE can be used to convert all values to a single numeric data type. However, if you use the VALUE function with a column that

contains mixed numbers and text, the entire column is flagged with an error, because not all values in all rows can be converted to numbers.

Example

The following formula converts the typed string, "3", into the numeric value 3.

```DAX
= VALUE("3")
```

Related content

Text functions

Feedback

Was this page helpful? Yes No

Time intelligence functions

09/16/2025

Data Analysis Expressions (DAX) includes time-intelligence functions that enable you to manipulate data using time periods, including days, months, quarters, and years, and then build and compare calculations over those periods. Before using any time-intelligence functions, make sure to mark one of the tables containing date column as Date Table.

In this category

Expand table

Function	Description
CLOSINGBALANCEWEEK	Evaluates the expression at the last date of the week in the current context.
CLOSINGBALANCEMONTH	Evaluates the expression at the last date of the month in the current context.
CLOSINGBALANCEQUARTER	Evaluates the expression at the last date of the quarter in the current context.
CLOSINGBALANCEYEAR	Evaluates the expression at the last date of the year in the current context.
DATEADD	Returns a table that contains a column of dates, shifted either forward or backward in time by the specified number of intervals from the dates in the current context.
DATESBETWEEN	Returns a table that contains a column of dates that begins with a specified start date and continues until a specified end date.
DATESINPERIOD	Returns a table that contains a column of dates that begins with a specified start date and continues for the specified number and type of date intervals.
DATESWTD	Returns a table that contains a column of the dates for the week to date, in the current context.
DATESMTD	Returns a table that contains a column of the dates for the month to date, in the current context.
DATESQTD	Returns a table that contains a column of the dates for the quarter to date, in the current context.
DATESYTD	Returns a table that contains a column of the dates for the year to date, in the current context.

Function	Description
ENDOFWEEK	Returns the last date of the week in the current context for the specified column of dates.
ENDOFMONTH	Returns the last date of the month in the current context for the specified column of dates.
ENDOFQUARTER	Returns the last date of the quarter in the current context for the specified column of dates.
ENDOFYEAR	Returns the last date of the year in the current context for the specified column of dates.
FIRSTDATE	Returns the first date in the current context for the specified column of dates.
LASTDATE	Returns the last date in the current context for the specified column of dates.
NEXTDAY	Returns a table that contains a column of all dates from the next day, based on the first date specified in the dates column in the current context.
NEXTWEEK	Returns a table that contains a column of all dates from the next week, based on the first date in the dates column in the current context.
NEXTMONTH	Returns a table that contains a column of all dates from the next month, based on the first date in the dates column in the current context.
NEXTQUARTER	Returns a table that contains a column of all dates in the next quarter, based on the first date specified in the dates column, in the current context.
NEXTYEAR	Returns a table that contains a column of all dates in the next year, based on the first date in the dates column, in the current context.
OPENINGBALANCEWEEK	Evaluates the expression at the first date of the week in the current context.
OPENINGBALANCEMONTH	Evaluates the expression at the first date of the month in the current context.
OPENINGBALANCEQUARTER	Evaluates the expression at the first date of the quarter, in the current context.
OPENINGBALANCEYEAR	Evaluates the expression at the first date of the year in the current context.
PARALLELPERIOD	Returns a table that contains a column of dates that represents a period parallel to the dates in the specified dates column, in the current context, with the dates shifted a number of intervals either forward in time or back in time.

Function	Description
PREVIOUSDAY	Returns a table that contains a column of all dates representing the day that is previous to the first date in the dates column, in the current context.
PREVIOUSWEEK	Returns a table that contains a column of all dates representing the week that is previous to the first date in the dates column, in the current context.
PREVIOUSMONTH	Returns a table that contains a column of all dates from the previous month, based on the first date in the dates column, in the current context.
PREVIOUSQUARTER	Returns a table that contains a column of all dates from the previous quarter, based on the first date in the dates column, in the current context.
PREVIOUSYEAR	Returns a table that contains a column of all dates from the previous year, given the last date in the dates column, in the current context.
SAMEPERIODLASTYEAR	Returns a table that contains a column of dates shifted one year back in time from the dates in the specified dates column, in the current context.
STARTOFWEEK	Returns the first date of the week in the current context for the specified column of dates.
STARTOFMONTH	Returns the first date of the month in the current context for the specified column of dates.
STARTOFQUARTER	Returns the first date of the quarter in the current context for the specified column of dates.
STARTOFYEAR	Returns the first date of the year in the current context for the specified column of dates.
TOTALWTD	Evaluates the value of the expression for the week to date, in the current context.
TOTALMTD	Evaluates the value of the expression for the month to date, in the current context.
TOTALQTD	Evaluates the value of the expression for the dates in the quarter to date, in the current context.
TOTALYTD	Evaluates the year-to-date value of the expression in the current context.

CLOSINGBALANCEWEEK

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the last date of the week in the current context.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```
CLOSINGBALANCEWEEK(<expression>, <calendar>[,<filter>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`calendar`	A calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated at the last date of the week in the current context.

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Week End Inventory Value' of the product inventory using a fiscal calendar.

DAX

```
= CLOSINGBALANCEWEEK (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

Time intelligence functions CLOSINGBALANCEYEAR function CLOSINGBALANCEQUARTER function CLOSINGBALANCEMONTH function

CLOSINGBALANCEMONTH

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the last date of the month in the current context.

Syntax

```
CLOSINGBALANCEMONTH(<expression>, <dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Parameter	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated at the last date of the month in the current context.

Remarks

- The `dates` argument can be any of the following:

- A reference to a date/time column.
- A table expression that returns a single column of date/time values.
- A Boolean expression that defines a single-column table of date/time values.

> ⓘ Note
>
> Constraints on Boolean expressions are described in CALCULATE function.

> ⓘ Note
>
> The `filter` expression has restrictions described in CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Month End Inventory Value' of the product inventory.

DAX

```
= CLOSINGBALANCEMONTH (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Month End Inventory Value' of the product inventory using a fiscal calendar.

DAX

```
= CLOSINGBALANCEMONTH (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
```

```
    ),
    FiscalCalendar
)
```

Related content

Time intelligence functions CLOSINGBALANCEYEAR function CLOSINGBALANCEQUARTER function CLOSINGBALANCEWEEK function

CLOSINGBALANCEQUARTER

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the last date of the quarter in the current context.

Syntax

```
CLOSINGBALANCEQUARTER(<expression>,<dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated at the last date of the quarter in the current context.

Remarks

- The `dates` argument can be any of the following:

- A reference to a date/time column.
- A table expression that returns a single column of date/time values.
- A Boolean expression that defines a single-column table of date/time values.

> ⓘ Note
>
> Constraints on Boolean expressions are described in CALCULATE function.

> ⓘ Note
>
> The `filter` expression has restrictions described in CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Quarter End Inventory Value' of the product inventory.

DAX

```
= CLOSINGBALANCEQUARTER (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Quarter End Inventory Value' of the product inventory using a fiscal calendar.

DAX

```
= CLOSINGBALANCEQUARTER (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
```

```
    ),
    FiscalCalendar
)
```

Related content

Time intelligence functions CLOSINGBALANCEYEAR function CLOSINGBALANCEMONTH function CLOSINGBALANCEWEEK function

CLOSINGBALANCEYEAR

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the last date of the year in the current context.

Syntax

```
CLOSINGBALANCEYEAR(<expression>,<dates> or <calendar>[,<filter>][,
<year_end_date>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. This parameter is permitted only when the date column syntax is used.

Return value

A scalar value that represents the `expression` evaluated at the last date of the year in the current context.

Remarks

- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.

    > **Note**
    >
    > Constraints on Boolean expressions are described in CALCULATE function.

    > **Note**
    >
    > The `filter` expression has restrictions described in CALCULATE function.
- The `year_end_date` parameter is permitted only when the date column syntax is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Year End Inventory Value' of the product inventory.

DAX

```
= CLOSINGBALANCEYEAR (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Year End Inventory Value' of the product inventory using a fiscal calendar.

DAX

```
= CLOSINGBALANCEYEAR (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

Time intelligence functions CLOSINGBALANCEQUARTER function CLOSINGBALANCEMONTH function CLOSINGBALANCEWEEK function

DATEADD

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

If input is a date column, returns a table that contains a column of dates, shifted either forward or backward in time by the specified number of intervals from the dates in the current context. If the input is a calendar, the function returns dates shifted forward or backward in time by the specified number of intervals, based on the current context. The output includes the primary tagged columns as well as time-related columns.

Syntax

```
DATEADD(<dates> or <calendar>, <number_of_intervals>, <interval>[,<Extension>],[,
<Truncation>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference.
`number_of_intervals`	An integer that specifies the number of intervals to add to or subtract from the dates.
`interval`	The interval by which to shift the dates. The value for interval can be one of the following: `year`, `quarter`, `month`, `week`, `day`. The week enum is only applicable when a calendar reference is provided.
`extension`	Only applicable when a calendar reference is provided. Define behavior when the original time period has fewer dates than the resulting time period. Valid values are: EXTENDING (Default), PRECISE, ENDALIGNED.
`truncation`	Only applicable when a calendar reference is provided. Define behavior when the original time period has more dates than the resulting time period. Valid values are: BLANKS (Default), ANCHORED.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and time related columns for the shifted periods, in the current context.

Remarks

The `dates` argument can be any of the following:

- A reference to a date/time column,
- A table expression that returns a single column of date/time values,
- A Boolean expression that defines a single-column table of date/time values.

  > ⓘ **Note**
  >
  > Constraints on Boolean expressions are described in the topic, CALCULATE function.

- If the number specified for `number_of_intervals` is positive, the dates in `dates` are moved forward in time; if the number is negative, the dates in `dates` are shifted back in time.
- The `interval` parameter is an enumeration, not a set of strings; therefore values should not be enclosed in quotation marks. Also, the values: `year`, `quarter`, `month`, `day` should be spelled in full when using them.
- The result table includes only dates that exist in the `dates` column.
- If the date column syntax is used and the dates in the current context do not form a contiguous interval, the function returns an error.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example - Shifting a set of dates

The following formula calculates dates that are one year before the dates in the current context.

```
DAX
```

```
= DATEADD ( DateTime[DateKey], -1, YEAR )
```

Example for calendar based time intelligence

The following formula returns a table of dates shifted one year back from the dates in the current context.

DAX

```
DATEADD ( FiscalCalendar, -1, YEAR )
```

Special behavior when input is a date column

When the selection includes the last two days of month, DATEADD will use "extension" semantics and will include the days until the end of month. For example, when Feb 27 and 28 of 2013 are included in the selection and a month is added, DATEADD will return March 27 to 31.

This behavior only happens when last two days of month are included in the selection. If only Feb 27 is selected, it will go to March 27.

DAX

```
= DATEADD(DateTime[DateKey], 1, month)
```

Calendar based time intelligence provides more control via two optional parameters: "Extension" and "Truncation". Please see the above parameter descriptions for details.

Behavior for calendar based DateAdd when selection is at a finer grain than the shift level

When calendar reference is used and the selection is at a finer grain than the shift level, an index-based approach is taken. To illustrate this behavior, let's consider the scenario where the selection is at the date level and DATEADD() is shifting by month. Here is what DateAdd will do:

- Determine the positions of the current selection within the month.
  For example, if the current selection spans March 3–10, the positions are from the 3rd to the 10th day of the month.

- Shift the month Apply the month shift — e.g., a shift of +1 changes March to April.

- Return the same relative positions in the shifted month Retrieve the 3rd to the 10th of the new month (e.g., April 3–10).

Parameters for calendar based DateAdd when selection is at a finer grain than the shift level

When the selection granularity is **finer** than the shift unit (e.g., selecting individual dates while shifting by month), the **index-based behavior** can lead to **ambiguities**, especially across months of varying lengths. To handle these edge cases, two parameters are introduced:

Extension parameter (for small → large period shifts):

Controls how the function behaves when the destination period is **longer** than the current one. Use moving forward one month as example:

- `Precise`: Keeps the original date range strictly.
  → `Feb 25–28` → `March 25–28`

- `Extending`: Allows the window to expand toward the **end of the period** if needed.
  → `Feb 25–28` → `March 25–31`

- `EndAligned`: Aligns the end date with the end of the destination period when the selection reaches the end of its period; otherwise preserves relative positions.
  → `Feb 28` → `March 31`, while `Feb 27` → `March 27`

Truncation Parameter (for large → small period shifts)

Controls how the function behaves when the destination month is **shorter** than the current one. Use moving backward one month as example:

- `Anchored`: Anchors the result to the **last valid date** of the smaller period.
  → `March 31` → `Feb 28`

- `Blanks`: Returns **blank** when the shifted date doesn't exist.
  → `March 31` → *(blank)* (since February doesn't have 31st)

Differences in behavior between classic and calendar time intelligence

Some scenarios may yield different results when comparing classic and calendar time intelligence. For example, in a lunar year, SamePeriodLastYear will produce different results at the date granularity. In calendar-based time intelligence, shifting Feb 29 2008 back one year results in Mar 1 2007, because it is treated as the 60th day of the year. In classic time intelligence, the same shift returns Feb 28 2007. The workaround is to use DATEADD(Calendar, -<number of a year>, month). For example, if a year has 13 months in calendar, use DATEADD(Calendar, -13, month). This approach will shift by month so Feb 2008 will go to Feb 2007.

Related content

Time intelligence functions Date and time functions

Last updated on 11/17/2025

DATESBETWEEN

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of dates that begins with a specified start date and continues until a specified end date.

For calendar input, returns a table that begins with a specified start date and continues until a specified end date. The table contains all primary tagged columns and all time related columns.

This function is suited to pass as a filter to the CALCULATE function. Use it to filter an expression by a custom date range.

> **Note**
>
> If you're working with standard date intervals such as days, months, quarters, or years, use the better-suited DATESINPERIOD function.

Syntax

DAX

```
DATESBETWEEN(<dates or calendar>, <StartDate>, <EndDate>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Term	Definition
`StartDate`	A date/day expression. If you use calendar syntax, use the same data type as the primary column tagged to the Day category.
`EndDate`	A date/day expression. If you use calendar syntax, use the same data type as the primary column tagged to the Day category.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- In the most common use case, `dates` is a reference to the date column of a marked date table.
- If `StartDate` is BLANK, then `StartDate` is the earliest value in the `dates` column. For calendar, it's the first value in a column that's tagged as day.
- If `EndDate` is BLANK, then `EndDate` is the latest value in the `dates` column. For calendar, it's the last value in a column that's tagged as day.
- Dates used as the `StartDate` and `EndDate` are inclusive. So, for example, if the `StartDate` value is July 1, 2019, then that date is included in the returned table (providing the date exists in the `dates` column).
- For date column input, the returned table can only contain dates stored in the `Dates` column. So, for example, if the `Dates` column starts from July 1, 2017, and the `StartDate` value is July 1, 2016, the returned table starts from July 1, 2017.
- For calendar input, if the input date isn't found in the tagged day column, it's treated as BLANK, and the first or last value is used.
- For calendar input, use the same data type and format as the tagged day column for the start date and end date. For example, if the column uses the format YYYY-Sn-Qn-Mnn-Wnn-Dnn (e.g., "2014-S2-Q4-M11-W45-D03"), the start date and end date must follow the same format (e.g., "2015-S2-Q4-M11-W45-D03"). Otherwise, the behavior is undefined.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following **Sales** table measure definition uses the DATESBETWEEN function to produce a *life-to-date* (LTD) calculation. Life-to-date represents the accumulation of a measure over time since the very beginning of time.

The formula uses the MAX function. This function returns the latest date that's in the filter context. So, the DATESBETWEEN function returns a table of dates beginning from the earliest date until the latest date being reported.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
Customers LTD =
CALCULATE (
    DISTINCTCOUNT ( Sales[CustomerKey] ),
    DATESBETWEEN ( 'Date'[Date], BLANK (), MAX ( 'Date'[Date] ) )
)
```

Consider that the earliest date stored in the **Date** table is July 1, 2017. So, when a report filters the measure by the month of June 2020, the DATESBETWEEN function returns a date range from July 1, 2017 until June 30, 2020.

Example for calendar based time intelligence

The following **Sales** table measure definition uses the DATESBETWEEN function to produce a *life-to-date* (LTD) calculation. Life-to-date represents the accumulation of a measure over time since the very beginning of time.

The formula uses the MAX function. This function returns the max datekey that's in the filter context. So, the DATESBETWEEN function returns a table of dates beginning from the earliest date until the latest date being reported. We use DateKey as an example to show that the "Day" category can be tagged with a column that isn't date-typed.

DAX

```
Customers LTD =
CALCULATE (
    DISTINCTCOUNT ( Sales[CustomerKey] ),
```

```
    DATESBETWEEN ( FiscalCalendar, BLANK (), MAX ( 'Date'[DateKey] ) )
)
```

Related content

- Time intelligence functions (DAX)
- Date and time functions (DAX)
- DATESINPERIOD function (DAX)

Last updated on 08/04/2026

DATESINPERIOD

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of dates that begins with a specified start date and continues for the specified number and type of date intervals.

For calendar input, returns a table that begins with a specified start date and continues for the specified number and type of date intervals. The table contains all primary tagged columns and all time related columns.

This function is suited to pass as a filter to the CALCULATE function. Use it to filter an expression by standard date intervals such as days, months, quarters, or years.

Syntax

DAX

```
DATESINPERIOD(<dates or calendar>, <start_date>, <number_of_intervals>, <interval>[, <endbehavior>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains either dates or a calendar reference
`start_date`	A date expression. If calendar syntax is used, please use the same data type as the primary column tagged to the Day category.
`number_of_intervals`	An integer that specifies the number of intervals to add to, or subtract from, the dates.

Term	Definition
`interval`	The interval by which to shift the dates. The value for interval can be one of the following: `DAY`, `WEEK`, `MONTH`, `QUARTER`, and `YEAR`. You can only use `WEEK` with a calendar reference.
`endbehavior`	Only applicable when you provide a calendar reference. Optional. Controls how the end date aligns when the destination interval is longer than the source span. Valid values are: `PRECISE` (default) and `ENDALIGNED`.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- In the most common use case, `dates` is a reference to the date column of a marked date table.
- If the number specified for `number_of_intervals` is positive, dates are moved forward in time; if the number is negative, dates are shifted backward in time.
- The `interval` parameter is an enumeration. Valid values are `DAY`, `WEEK`, `MONTH`, `QUARTER`, and `YEAR`. Because it's an enumeration, values aren't passed in as strings. So don't enclose them within quotation marks.
- When `endbehavior` is provided (calendar time intelligence only), DATESINPERIOD forwards the value to DATEADD's `Extension` parameter. See Understanding endbehavior for detailed examples.
- For date column input, the returned table can only contain dates stored in the `dates` column. So, for example, if the `dates` column starts from July 1, 2017, and the `start_date` value is July 1, 2016, the returned table starts from July 1, 2017.
- For calendar input, if the input date isn't found in the tagged day column, the result is undefined.
- For calendar input, use the same data type and format as the tagged day column for the start date. For example, if the column uses the format YYYY-Sn-Qn-Mnn-Wnn-Dnn (e.g., "2014-S2-Q4-M11-W45-D03"), the start date must follow the same format (e.g., "2015-S2-Q4-M11-W45-D03"). Otherwise, the behavior is undefined.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following **Sales** table measure definition uses the DATESINPERIOD function to calculate revenue for the prior year (PY).

The formula uses the MAX function. This function returns the latest date that's in the filter context. So, the DATESINPERIOD function returns a table of dates beginning from the latest date for the last year.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
Revenue PY =
CALCULATE (
    SUM ( Sales[Sales Amount] ),
    DATESINPERIOD ( 'Date'[Date], MAX ( 'Date'[Date] ), -1, YEAR )
)
```

The report is filtered by the month of June 2020. The MAX function returns June 30, 2020. The DATESINPERIOD function then returns a date range from July 1, 2019 until June 30, 2020. It's a year of date values starting from June 30, 2020 for the last year.

Example for calendar based time intelligence

The following **Sales** table measure definition uses the DATESINPERIOD function to calculate revenue for the prior year (PY).

The formula uses the MAX function. This function returns the latest date that's in the filter context. So, the DATESINPERIOD function returns primary columns beginning from latest date for the last year. We use DateKey as an example to show that the "Day" category can be tagged with a column that isn't date-typed.

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

DAX

```
Revenue PY =
CALCULATE (
    SUM ( Sales[Sales Amount] ),
    DATESINPERIOD ( FiscalCalendar, MAX ( 'Date'[DateKey] ), -1, YEAR )
)
```

The report is filtered by the month of June 2020. The MAX function returns June 30, 2020. The DATESINPERIOD function then returns a range from July 1, 2019 until June 30, 2020. It's a year starting from June 30, 2020 for the last year.

Understanding endbehavior parameter for calendar time intelligence

Internally, DATESINPERIOD computes the ending boundary by calling DATEADD with the same interval, number of intervals, and `endbehavior` that were passed into DATESINPERIOD. When `number_of_intervals` is negative (common when rolling a window backward), the function returns the range `(endDate, startDate]`, where `endDate` is the date returned by DATEADD after it shifts the calendar context that ends on `startDate` by `<number_of_intervals>` `<interval>` using `<endbehavior>`. For positive values, the returned interval is `[startDate, endDate)`.

- `PRECISE` keeps the exact value returned by DATEADD.
- `ENDALIGNED` follows the DATEADD `EndAligned` semantics, moving the boundary to the end of the destination period when the source selection already reached its own end. This is useful when the filter context already ends on the last day of a period and you want a backward-looking window (for example, six months) to use the last day of the shifted period as its boundary.

Example with EndAligned

DAX

```
EndAlignedTest =
    CALCULATE (
        COUNTROWS ( SUMMARIZE ( Sales, 'Date'[MonthName] ) ),
        DATESINPERIOD ( 'Gregorian Calendar', MAX ( 'Date'[Date] ), -6, MONTH,
EndAligned )
    )
```

This measure counts the distinct month names in the last six months relative to the current context. With `EndAligned`, the window keeps the boundary at the end of the shifted month when the context already ends on a month boundary, preventing the additional trailing dates that

`Precise` would include (for example, `Feb 28` would otherwise shift to `Aug 28`, pulling in `Aug 29–31` and effectively expanding the span to seven months). The `EndAligned` setting keeps the count at six in that scenario.

The following comparison assumes the current filter context ends on **February 28, 2023** and the expression calls `DATESINPERIOD(..., -6, MONTH, <endbehavior>)`. The functional range is `(endDate, startDate]`, so the `endDate` returned by DATEADD is excluded while `startDate` is included.

Expand table

endbehavior	`DATEADD` endDate	DATESINPERIOD range	Month names returned by the measure
`Precise`	2022-08-28	`(2022-08-28, 2023-02-28]` → Aug 29 2022 – Feb 28 2023	Aug, Sep, Oct, Nov, Dec, Jan, Feb (7)
`EndAligned`	2022-08-31	`(2022-08-31, 2023-02-28]` → Sep 1 2022 – Feb 28 2023	Sep, Oct, Nov, Dec, Jan, Feb (6)

Differences in behavior between classic and calendar time intelligence

Internally, DATESINPERIOD uses the same logic as DATEADD to determine the end date from the start date, and then calculates the range. Some scenarios can yield different results when comparing classic and calendar time intelligence. For example, in a lunar year, DATEADD produces different results at the date granularity, so the result of DATESINPERIOD differs as well. In calendar-based time intelligence, shifting Feb 29 2008 back one year results in Mar 1 2007, because it's treated as the 60th day of the year. In classic time intelligence, the same shift returns Feb 28 2007. Because the end date differs, the output of DATESINPERIOD also differs. The workaround is to use DATEADD(Calendar, -<number of a year>, month) to calculate end date. For example, if a year has 13 months in calendar, use DATEADD(Calendar, -13, month). This approach shifts by month so Feb 2008 goes to Feb 2007. Then, write a custom datesInPeriod based on the new end date.

Related content

- Time intelligence functions (DAX)
- Date and time functions (DAX)
- DATESBETWEEN function (DAX)

Last updated on 08/04/2026

DATESWTD

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Returns a table for week to date, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
DATESWTD(<calendar>)
```

Parameters

Expand table

Term	Definition
`calendar`	A calendar reference.

Return value

A table containing all primary tagged and time-related columns

Remarks

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Week To Date Total' for Internet sales uing fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESWTD ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions DATESYTD function DATESQTD function DATESMTD function

DATESMTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of the dates for the month to date, in the current context.
For calendar input, returns a table for the month to date, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
DATESMTD(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

The `dates` argument can be any of the following:

- A reference to a date/time column.
- A table expression that returns a single column of date/time values.
- A Boolean expression that defines a single-column table of date/time values.

  > **Note**
  >
  > Constraints on Boolean expressions are described in the topic, CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Month To Date Total' for Internet Sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESMTD ( DateTime[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Monthly Running Total' for Internet sales uing fiscal calendar.

DAX

```
=
CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESMTD ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions DATESWTD function DATESQTD function DATESYTD function

DATESQTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of the dates for quarter to date, in the current context.
For calendar input, returns a table for quarter to date, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
DATESQTD(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference.

Return value

For date column input, a table containing a single column of date values. For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

The `dates` argument can be any of the following:

- A reference to a date/time column.

- A table expression that returns a single column of date/time values.
- A Boolean expression that defines a single-column table of date/time values.

  > **Note**
  >
  > Constraints on Boolean expressions are described in the topic, CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Quarterly Running Total' of Internet Sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESQTD ( DateTime[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Quarterly Running Total' for Internet sales uing fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESQTD ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions DATESWTD function DATESMTD function DATESYTD function

DATESYTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of the dates for year to date, in the current context.
For calendar input, returns a table for year to date, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
DATESYTD(<dates> or <calendar> [,<year_end_date>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference.
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. This parameter is permitted only when the date column syntax is used.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

The `dates` argument can be any of the following:

- A reference to a date/time column,
- A table expression that returns a single column of date/time values,
- A Boolean expression that defines a single-column table of date/time values.

  > ⓘ **Note**
  >
  > Constraints on Boolean expressions are described in the topic, CALCULATE function.

- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored. Depending on locale, the format might be something like "m-dd" or "dd-m".
- The `year_end_date` parameter is permitted only when the date column syntax is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Running Total' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESYTD ( DateTime[DateKey] )
)
```

The following sample formula creates a measure that calculates the 'Fiscal Year Running Total' for Internet sales, using a US Locale for the Date format.

DAX

```
= CALCULATE(
    SUM(InternetSales_USD[SalesAmount_USD]),
    DATESYTD(DateTime[DateKey],
        "6-30"
```

```
        )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Running Total' for Internet sales uing fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    DATESYTD ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions DATESWTD function DATESMTD function DATESQTD function

ENDOFWEEK

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns all primary tagged columns and all time related columns for last date of week, in the current context.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```
ENDOFWEEK(<calendar>)
```

Parameters

Expand table

Term	Definition
`calendar`	A calendar reference

Return value

A table that contains all primary tagged columns and all time related columns for end of week, in the current context.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula returns tagged primary columns that corresponds to the end of the week, for the fiscal calendar.

```
DAX
= ENDOFWEEK(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions ENDOFYEAR function ENDOFQUARTER function ENDOFMONTH function

ENDOFMONTH

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the last date of the month in the current context for the specified column of dates.
For calendar input, returns a table for the last date of the month, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
ENDOFMONTH(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table with a single row that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the end of the month, for the current context.

```
DAX

= ENDOFMONTH(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula returns tagged primary columns that corresponds to the end of the month, for the fiscal calendar.

```
DAX

= ENDOFMONTH(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions ENDOFYEAR function ENDOFQUARTER function ENDOFWEEK function

ENDOFQUARTER

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the last date of quarter in the current context for the specified column of dates.
For calendar input, returns a table for last date of quarter, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
ENDOFQUARTER(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

- A reference to a date/time column,
- A table expression that returns a single column of date/time values,
- A Boolean expression that defines a single-column table of date/time values.

- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the end of the quarter, for the current context.

```
DAX

= ENDOFQUARTER(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula returns tagged primary columns that corresponds to end of quarter, for the fiscal calendar.

```
DAX

= ENDOFQUARTER(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions ENDOFWEEK function ENDOFMONTH function ENDOFYEAR function

ENDOFYEAR

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the last date of year in the current context for the specified column of dates.
For calendar input, returns a table that contains all primary tagged columns and all time related columns for last date of year, in the current context.

Syntax

```
ENDOFYEAR(<dates> or <calendar> [,<year_end_date>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. This parameter is permitted only when the date column syntax is used.

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table that contains all primary tagged columns and all time related columns for last date of year, in the current context.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column,
  - A table expression that returns a single column of date/time values,
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored.
- The `year_end_date` parameter is permitted only when the date column syntax is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the end of the fiscal year that ends on June 30, for the current context.

```
DAX

= ENDOFYEAR(DateTime[DateKey],"06/30/2004")
```

Example for calendar based time intelligence

The following sample formula returns tagged primary columns that corresponds to end of year, for the fiscal calendar.

```
DAX

= ENDOFYEAR(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions ENDOFWEEK function ENDOFMONTH function ENDOFQUARTER function

FIRSTDATE

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the first date in the current context for the specified column of dates.

For calendar input, returns the first date in the current context based on the calendar.

Syntax

```
FIRSTDATE(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.

For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- When the current context is a single date, the date returned by the FIRSTDATE and LASTDATE functions will be equal.
- For date column input, the Return value is a table that contains a single column and single value. Therefore, this function can be used as an argument to any function that requires a table in its arguments. Also, the returned value can be used whenever a date value is required.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that obtains the first date when a sale was made in the Internet sales channel for the current context.

```
DAX

= FIRSTDATE('InternetSales_USD'[SaleDateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that obtains the first date when a sale was made in the Internet sales channel for the current context.

```
DAX

= FIRSTDATE(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions LASTDATE function FIRSTNONBLANK function

LASTDATE

09/16/2025

Applies to: Calculated column | Calculated table | Measure | Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the last date in the current context for the specified column of dates.

For calendar input, returns the last date based on the calendar.

Syntax

```
LASTDATE(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.

For calendar input, a single row table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column,
    - A table expression that returns a single column of date/time values,
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- When the current context is a single date, the date returned by the FIRSTDATE and LASTDATE functions will be equal.
- For date column input, the Return value is a table that contains a single column and single value. Therefore, this function can be used as an argument to any function that requires a table in its arguments. Also, the returned value can be used whenever a date value is required.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that obtains the last date, for the current context, when a sale was made in the Internet sales channel.

```
DAX

= LASTDATE('InternetSales_USD'[SaleDateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that obtains the last date, for the current context, when a sale was made in the Internet sales channel.

```
DAX

= LASTDATE(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions FIRSTDATE function LASTNONBLANK function

NEXTDAY

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates from the next day, based on the last date specified in the `dates` column in the current context.

For calendar input, returns the next day of the last date in the current context, based on the calendar. The output will include all primary tagged columns and all time related columns for that day.

Syntax

```
NEXTDAY(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains primary tagged columns and all time related columns for next day, in the current context.

Remarks

- For date column input, this function returns all dates from the next day to the last date in the input parameter. For example, if the last date in the `dates` argument refers to June 10, 2009; then this function returns all dates equal to June 11, 2009.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'next day sales' of Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTDAY ( 'DateTime'[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'next day sales' of Internet sales for a fiscal calendar.

DAX

```
=
CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTDAY ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions NEXTWEEK function NEXTMONTH function NEXTQUARTER function NEXTYEAR function

NEXTWEEK

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Returns a table of all dates from the next week, based on the last date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
NEXTWEEK(<calendar>)
```

Parameters

Expand table

Term	Definition
`calendar`	A calendar reference

Return value

A table contains all primary tagged columns and all time related columns.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'next week sales' for Internet sales.

```DAX
=
CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTWEEK ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions NEXTDAY function NEXTMONTH function NEXTQUARTER function NEXTYEAR function

NEXTMONTH

09/16/2025

Applies to: Calculated column | Calculated table | Measure | Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates from the next month, based on the last date in the `dates` column in the current context.

For calendar input, returns a table for all dates from the next month, based on the last date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
NEXTMONTH(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values. For calendar input, a table that contains all primary tagged columns and all time related columns for next month.

Remarks

- This function returns all dates from the next month to the last date in the input parameter. For example, if the last date in the `dates` argument refers to June 10, 2009; then this function returns all dates for the month of July, 2009.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'next month sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTMONTH ( 'DateTime'[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'next month sales' for Internet sales.

DAX

```
=
CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTMONTH ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions NEXTDAY function NEXTWEEK function NEXTQUARTER function NEXTYEAR function

NEXTQUARTER

09/16/2025

Applies to: Calculated column | Calculated table | Measure | Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates in the next quarter, based on the last date specified in the `dates` column, in the current context.

For calendar input, returns a table of all dates from the next quarter, based on the last date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
NEXTQUARTER(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table for next quarter, in the current context. The table contains all primary tagged columns and all time related columns.

Remarks

- This function returns all dates in the next quarter, based on the last date in the input parameter. For example, if the last date in the `dates` column refers to June 10, 2009, this function returns all dates for the quarter July to September, 2009.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'next quarter sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTQUARTER ( 'DateTime'[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'next quarter sales' for Internet sales.

DAX

```
=
CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTQUARTER ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions NEXTDAY function NEXTWEEK function NEXTMONTH function NEXTYEAR function

NEXTYEAR

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates in the next year, based on the last date in the `dates` column, in the current context.

For calendar input, returns a table of all dates from the next year, based on the last date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
NEXTYEAR(<dates> or <calendar>[,<year_end_date>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. This parameter is permitted only when the date column syntax is used.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- This function returns all dates in the next year, based on the last date in the input column. For example, if the last date in the `dates` column refers to the year 2007, this function returns all dates for the year 2008.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored. This parameter does not apply when calendar is used.
- The `year_end_date` parameter is permitted only when the date column syntax is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'next year sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    NEXTYEAR ( 'DateTime'[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'next year sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
```

```
    NEXTYEAR ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions NEXTDAY function NEXTQUARTER function NEXTMONTH function

OPENINGBALANCEWEEK

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the date corresponding to the end of the previous week in the current context.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```
OPENINGBALANCEWEEK(<expression>,<calendar>[,<filter>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`calendar`	A calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated at the end of the previous week in the current context.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `filter` expression has restrictions described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Week Start Inventory Value' of the product inventory in terms of fiscal calendar.

DAX

```
=
OPENINGBALANCEWEEK (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

OPENINGBALANCEYEAR function OPENINGBALANCEQUARTER function OPENINGBALANCEMONTH function Time intelligence functions CLOSINGBALANCEWEEK function

OPENINGBALANCEMONTH

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the date corresponding to the end of the previous month in the current context.

Syntax

```
OPENINGBALANCEMONTH(<expression>,<dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` at the end of the previous month in the current context.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `filter` expression has restrictions described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Month Start Inventory Value' of the product inventory.

DAX

```
= OPENINGBALANCEMONTH (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Month Start Inventory Value' of the product inventory in terms of fiscal calendar.

DAX

```
=
OPENINGBALANCEMONTH (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

OPENINGBALANCEWEEK function OPENINGBALANCEYEAR function OPENINGBALANCEQUARTER function Time intelligence functions CLOSINGBALANCEMONTH function

OPENINGBALANCEQUARTER

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the date corresponding to the end of the previous quarter in the current context.

Syntax

```
OPENINGBALANCEQUARTER(<expression>,<dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` at the end of the previous quarter in the current context.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `filter` expression has restrictions described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Quarter Start Inventory Value' of the product inventory.

DAX

```
=
OPENINGBALANCEQUARTER (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Quarter Start Inventory Value' of the product inventory in terms of fiscal calendar.

DAX

```
=
OPENINGBALANCEQUARTER (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

OPENINGBALANCEWEEK function OPENINGBALANCEYEAR function OPENINGBALANCEMONTH function Time intelligence functions CLOSINGBALANCEQUARTER function

OPENINGBALANCEYEAR

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the `expression` at the date corresponding to the end of the previous year in the current context.

Syntax

```
OPENINGBALANCEYEAR(<expression>,<dates> or <calendar>[,<filter>][,
<year_end_date>])
```

Parameters

Expand table

Term	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. Only applies when date column is used.

Return value

A scalar value that represents the `expression` evaluated at the end of the previous year in the current context.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- The `filter` expression has restrictions described in the topic, CALCULATE function.
- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored.
- The `year_end_date` parameter must not be specified when a calendar is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'Year Start Inventory Value' of the product inventory.

DAX

```
=
OPENINGBALANCEYEAR (
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    DateTime[DateKey]
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'Year Start Inventory Value' of the product inventory in terms of fiscal calendar.

DAX

```
=
OPENINGBALANCEYEAR (
```

```
    SUMX (
        ProductInventory,
        ProductInventory[UnitCost] * ProductInventory[UnitsBalance]
    ),
    FiscalCalendar
)
```

Related content

OPENINGBALANCEWEEK function OPENINGBALANCEQUARTER function OPENINGBALANCEMONTH function Time intelligence functions CLOSINGBALANCEYEAR function

PARALLELPERIOD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of dates that represents a period parallel to the dates in the specified `dates` column, in the current context, with the dates shifted a number of intervals either forward in time or back in time.

For calendar input, returns a table that represents a period parallel to the dates in the current context, based on the calendar, with the dates shifted a number of intervals either forward in time or back in time. The table contains all primary tagged columns and all time related columns.

Syntax

```
PARALLELPERIOD(<dates> or <calendar>,<number_of_intervals>,<interval>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference
`number_of_intervals`	An integer that specifies the number of intervals to add to or subtract from the dates.
`interval`	The interval by which to shift the dates. The value for interval can be one of the following: `year`, `quarter`, `month`, `week`. The week value is only applicable when calendar syntax is used.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- For date column input, this function takes the current set of dates in the column specified by `dates`, shifts the first date and the last date the specified number of intervals, and then returns all contiguous dates between the two shifted dates. If the interval is a partial range of month, quarter, or year then any partial months in the result are also filled out to complete the entire interval.

- The `dates` argument can be any of the following:
  - A reference to a date/time column,
  - A table expression that returns a single column of date/time values,
  - A Boolean expression that defines a single-column table of date/time values.

- Constraints on Boolean expressions are described in the topic, CALCULATE function.

- If the number specified for `number_of_intervals` is positive, the dates in `dates` are moved forward in time; if the number is negative, the dates in `dates` are shifted back in time.

- The `interval` parameter is an enumeration, not a set of strings; therefore values should not be enclosed in quotation marks. Also, the values: `year`, `quarter`, `month` should be spelled in full when using them.

- The result table includes only dates that appear in the values of the underlying table column.

- The PARALLELPERIOD function is similar to the DATEADD function except that PARALLELPERIOD always returns full periods at the given granularity level instead of the partial periods that DATEADD returns. For example, if you have a selection of dates that starts at June 10 and finishes at June 21 of the same year, and you want to shift that selection forward by one month then the PARALLELPERIOD function will return all dates from the next month (July 1 to July 31); however, if DATEADD is used instead, then the result will include only dates from July 10 to July 21.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the previous year sales for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PARALLELPERIOD ( DateTime[DateKey], -1, YEAR )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the previous year sales for Internet sales using fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PARALLELPERIOD ( FiscalCalendar, -1, YEAR )
)
```

Related content

Time intelligence functions Date and time functions DATEADD function

PREVIOUSDAY

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates representing the day that is previous to the first date in the `dates` column, in the current context.

For calendar input, returns a table that is previous to the first date in the calendar, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
PREVIOUSDAY(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- This function determines the first date in the input parameter, and then returns all dates corresponding to the day previous to that first date. For example, if the first date in the `dates` argument refers to June 10, 2009; this function returns all dates equal to June 9, 2009.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE function.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'previous day sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSDAY ( 'DateTime'[DateKey] )
)
```

Example for calendar

The following sample formula creates a measure that calculates the 'previous day sales' for Internet sales in terms of fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSDAY ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions PREVIOUSWEEK PREVIOUSMONTH function PREVIOUSQUARTER function PREVIOUSYEAR function

PREVIOUSWEEK

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns a table of all dates from the previous week, based on the first date in the current context. The table contains all primary tagged columns and all time related columns.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```DAX
PREVIOUSWEEK(<calendar>)
```

Parameters

Expand table

Term	Definition
`calendar`	A calendar reference

Return value

A table that contains all primary tagged columns and all time related columns.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE function.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'previous week sales' for Internet sales in terms of fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSWEEK ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions PREVIOUSDAY function PREVIOUSMONTH function PREVIOUSQUARTER function PREVIOUSYEAR function

PREVIOUSMONTH

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates from the previous month, based on the first date in the `<Dates>` column, in the current context.

For calendar input, returns a table of all dates from the previous month, based on the first date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
PREVIOUSMONTH(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- This function returns all dates from the previous month, using the first date in the column used as input. For example, if the first date in the `Dates` argument refers to June 10, 2009, this function returns all dates for the month of May, 2009.
- The `Dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'previous month sales' for sales.

DAX

```
= CALCULATE ( SUM ( 'Sales'[Sales Amount] ), PREVIOUSMONTH ( 'Date'[Date] ) )
```

Example for calendar

The following sample formula creates a measure that calculates the 'previous month sales' for Internet sales in terms of fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSMONTH ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions PREVIOUSDAY PREVIOUSWEEK PREVIOUSQUARTER PREVIOUSYEAR

PREVIOUSQUARTER

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates from the previous quarter, based on the first date in the `dates` column, in the current context.

For calendar input, returns a table of all dates from the previous quarter, based on the first date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
PREVIOUSQUARTER(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table for previous quarter. The table contains all primary tagged columns and all time related columns.

Remarks

- This function returns all dates from the previous quarter, using the first date in the input column. For example, if the first date in the `dates` argument refers to June 10, 2009, this function returns all dates for the quarter January to March, 2009.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'previous quarter sales' for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSQUARTER ( 'DateTime'[DateKey] )
)
```

Example for calendar

The following sample formula creates a measure that calculates the 'previous quarter sales' for Internet sales in terms of fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSQUARTER ( FiscalCalendar )
)
```

Related content

Time intelligence functions Date and time functions PREVIOUSDAY PREVIOUSWEEK PREVIOUSMONTH PREVIOUSYEAR

PREVIOUSYEAR

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of all dates from the previous year, based on the first date in the `dates` column, in the current context.

For calendar input, returns all dates from the previous year, based on the first date in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

DAX

```
PREVIOUSYEAR(<dates> or <calendar>[,<year_end_date>])
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31. This parameter is permitted only when the date column syntax is used.

Return value

For date column input, a table containing a single column of date values.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- This function returns all dates from the previous year based on the first date in the input parameter. For example, if the first date in the `dates` argument refers to the year 2009, then this function returns all dates for the year of 2008, up to the specified `year_end_date`.
- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is ignored.
- The `year_end_date` parameter is permitted only when the date column syntax is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the previous year sales for Internet sales.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSYEAR ( 'DateTime'[DateKey] )
)
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'previous quarter sales' for Internet sales in terms of fiscal calendar.

DAX

```
= CALCULATE (
    SUM ( InternetSales_USD[SalesAmount_USD] ),
    PREVIOUSYEAR ( FiscalCalendar )
)
```

Related content

- Time intelligence functions
- Date and time functions
- PREVIOUSDAY
- PREVIOUSWEEK
- PREVIOUSMONTH
- PREVIOUSQUARTER

Last updated on 01/22/2026

SAMEPERIODLASTYEAR

09/16/2025

Applies to: Calculated column | Calculated table | Measure | Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns a table that contains a column of dates shifted one year back in time from the dates in the specified `dates` column, in the current context.

For calendar input, returns a table that is shifted one year back in time from the dates in the current context, based on the calendar. The table contains all primary tagged columns and all time related columns.

Syntax

```
SAMEPERIODLASTYEAR(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, return a single-column table of date values.
For calendar input, return a table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column,
  - A table expression that returns a single column of date/time values,
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The dates returned are the same as the dates returned by this equivalent formula: `DATEADD(dates, -1, year)`
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the previous year sales of Reseller sales.

DAX

```
= CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]),
SAMEPERIODLASTYEAR(DateTime[DateKey]))
```

Special behavior

When the selection includes last two days of month, SAMEPERIODLASTYEAR will use "extension" semantics and will include the days till the end of month. For example, when Feb 27 and 28 of 2009 are included in the selection, SAMEPERIODLASTYEAR will return Feb 27 to 29 of 2008.

This behavior only happens when last two days of month are included in the selection. If only Feb 27 is selected, it will go to Feb 27.

DAX

```
= SAMEPERIODLASTYEAR(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the previous year sales of Reseller sales.

DAX

```
= CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]),
SAMEPERIODLASTYEAR(FiscalCalendar))
```

Differences in behavior between classic and calendar time intelligence

Some scenarios may yield different results when comparing classic and calendar time intelligence. For example, in a lunar year, SamePeriodLastYear will produce different results at the date granularity. In calendar-based time intelligence, shifting Feb 29 2008 back one year results in Mar 1 2007, because it is treated as the 60th day of the year. In classic time intelligence, the same shift returns Feb 28 2007. The workaround is to use DATEADD(Calendar, -<number of a year>, month). For example, if a year has 13 months in calendar, use DATEADD(Calendar, -13, month). This approach will shift by month so Feb 2008 will go to Feb 2007.

Related content

Time intelligence functions Date and time functions PREVIOUSYEAR PARALLELPERIOD

STARTOFWEEK

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Returns a table for the first date of week in the current context, based on the calendar. The table contains all primary tagged columns and all time related columns.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```
STARTOFWEEK(<calendar>)
```

Parameters

Expand table

Term	Definition
`calendar`	A calendar reference

Return value

A table that contains all primary tagged columns and all time related columns.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE.

- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula returns primary tagged columns for the first date of the week, for the fiscal calendar.

```
DAX

= STARTOFWEEK(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions STARTOFYEAR STARTOFQUARTER STARTOFMONTH

STARTOFMONTH

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the first date of month in the current context for the specified column of dates.
For calendar input, returns a table for first date of month, in the current context. A table that contains all primary tagged columns and all time related columns.

Syntax

```
STARTOFMONTH(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the start of the month, for the current context.

```
DAX

= STARTOFMONTH(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a table that returns tagged primary columns that corresponds to the start of the month, for the fiscal calendar.

```
DAX

= STARTOFMONTH(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions STARTOFYEAR STARTOFQUARTER STARTOFWEEK

STARTOFQUARTER

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the first date of quarter in the current context for the specified column of dates.
For calendar input, returns a table for first date of quarter, in the current context. A table that contains all primary tagged columns and all time related columns.

Syntax

```
STARTOFQUARTER(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table that contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the start of the quarter, for the current context.

```
DAX

= STARTOFQUARTER(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a table that returns tagged columns that corresponds to the start of the quarter, for the fiscal calendar.

```
DAX

= STARTOFQUARTER(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions STARTOFYEAR STARTOFMONTH STARTOFWEEK

STARTOFYEAR

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

For date column input, returns the first date of year in the current context for the specified column of dates.
For calendar input, returns a table for first date of year, in the current context. The table contains all primary tagged columns and all time related columns.

Syntax

```
STARTOFYEAR(<dates> or <calendar>)
```

Parameters

Expand table

Term	Definition
`dates or calendar`	A column that contains dates or a calendar reference.
`YearEndDate`	(Optional) A year end date value. Only applies for date column input.

Return value

For date column input, a table containing a single column and single row with a date value.
For calendar input, a table for first date of year, in the current context. The table contains all primary tagged columns and all time related columns.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `year_end_date` parameter must not be specified when a calendar is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that returns the start of the year, for the current context.

```
DAX

= STARTOFYEAR(DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a table that returns tagged columns that corresponds to the start of the year, for the fiscal calendar.

```
DAX

= STARTOFYEAR(FiscalCalendar)
```

Related content

Date and time functions Time intelligence functions STARTOFQUARTER STARTOFMONTH STARTOFWEEK

TOTALWTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the value of the `expression` for the week to date, in the current context.

> **Note**
>
> Week functions only work with calendar based time intelligence.

Syntax

```
TOTALWTD(<expression>,<calendar>[,<filter>])
```

Parameters

Expand table

Parameter	Definition
`expression`	An expression that returns a scalar value.
`calendar`	A calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated for the dates in the current week-to-date, given the dates in `dates` or `calendar`.

Remarks

- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `filter` expression has restrictions described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'week running total' or 'week running sum' for Internet sales in terms of fiscal calendar.

```
DAX
```

```
= TOTALWTD(SUM(InternetSales_USD[SalesAmount_USD]), FiscalCalendar)
```

Related content

ALL CALCULATE TOTALYTD TOTALQTD TOTALMTD

TOTALMTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the value of the `expression` for the month to date, in the current context.

Syntax

```
TOTALMTD(<expression>,<dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Parameter	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated for the dates in the current month-to-date, given the dates in `dates` or `calendar`.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column.

    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `filter` expression has restrictions described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'month running total' or 'month running sum' for Internet sales.

```
DAX

= TOTALMTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'month running total' or 'month running sum' for Internet sales in terms of fiscal calendar.

```
DAX

= TOTALMTD(SUM(InternetSales_USD[SalesAmount_USD]), FiscalCalendar)
```

Related content

ALL CALCULATE TOTALYTD TOTALQTD TOTALWTD

TOTALQTD

09/16/2025

Applies to: ✅ Calculated column ✅ Calculated table ✅ Measure ⚠️ Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the value of the `expression` for the dates in the quarter to date, in the current context.

Syntax

```
TOTALQTD(<expression>,<dates> or <calendar>[,<filter>])
```

Parameters

Expand table

Parameter	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference.
`filter`	(optional) An expression that specifies a filter to apply to the current context.

Return value

A scalar value that represents the `expression` evaluated for all dates in the current quarter to date, given the dates in `dates` or `calendar`.

Remarks

- The `dates` argument can be any of the following:

    - A reference to a date/time column.
    - A table expression that returns a single column of date/time values.
    - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `filter` expression has restrictions described in the topic, CALCULATE.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'quarter running total' or 'quarter running sum' for Internet sales.

```
DAX
```

```
= TOTALQTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'quarter running total' or 'quarter running sum' for Internet sales in terms of fiscal calendar.

```
DAX
```

```
= TOTALQTD(SUM(InternetSales_USD[SalesAmount_USD]), FiscalCalendar)
```

Related content

ALL CALCULATE TOTALYTD TOTALMTD TOTALWTD

TOTALYTD

09/16/2025

Applies to: Calculated column Calculated table Measure Visual calculation

> **Note**
>
> This function is discouraged for use in visual calculations as it likely returns meaningless results.

Evaluates the year-to-date value of the `expression` in the current context.

Syntax

```
TOTALYTD(<expression>,<dates> or <calendar>[,<filter>][,<year_end_date>])
```

Parameters

Expand table

Parameter	Definition
`expression`	An expression that returns a scalar value.
`dates or calendar`	A column that contains dates or a calendar reference
`filter`	(optional) An expression that specifies a filter to apply to the current context.
`year_end_date`	(optional) A literal string with a date that defines the year-end date. The default is December 31.

Return value

A scalar value that represents the `expression` evaluated for the current year-to-date `dates` or `calendar`.

Remarks

- The `dates` argument can be any of the following:
  - A reference to a date/time column.
  - A table expression that returns a single column of date/time values.
  - A Boolean expression that defines a single-column table of date/time values.
- Constraints on Boolean expressions are described in the topic, CALCULATE.
- The `filter` expression has restrictions described in the topic, CALCULATE.
- The `year_end_date` parameter is a string literal of a date, in the same locale as the locale of the client where the workbook was created. The year portion of the date is not required and is ignored. For example, the following formula specifies a (fiscal) year_end_date of 6/30 in an EN-US locale workbook.

  DAX

  ```
  = TOTALYTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey],
  ALL('DateTime'), "6/30")
  ```

  In this example, year_end_date can be specified as "6/30", "Jun 30", "30 June", or any string that resolves to a month/day. However, it is recommended you specify year_end_date using "month/day" (as shown) to ensure the string resolves to a date.
- The `year_end_date` parameter must not be specified when a calendar is used.
- This function is not supported for use in DirectQuery mode when used in calculated columns or row-level security (RLS) rules.

Example

The following sample formula creates a measure that calculates the 'year running total' or 'year running sum' for Internet sales.

DAX

```
= TOTALYTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Example for calendar based time intelligence

The following sample formula creates a measure that calculates the 'year running total' or 'year running sum' for Internet sales in terms of fiscal calendar.

DAX

```
= TOTALYTD(SUM(InternetSales_USD[SalesAmount_USD]), FiscalCalendar)
```

Related content

ALL CALCULATE DATESYTD TOTALMTD TOTALQTD TOTALWTD

Statements

Article • 10/20/2023

In this category

Statement	Description
DEFINE	(Keyword) Introduces one or more entity definitions that can be applied to one or more EVALUATE statements.
EVALUATE	(Keyword) Introduces a statement containing a table expression required to execute a DAX query.
MEASURE	(Keyword) Introduces a measure definition that can be used in one or more EVALUATE statements in a query.
ORDER BY	(Keyword) Introduces a statement that defines the sort order of query results returned by an EVALUATE statement.
START AT	(Keyword) Introduces a statement that defines the starting value at which the query results of an ORDER BY statement are returned.
VAR	(Keyword) Stores the result of an expression as a named variable, which can then be passed as an argument to other measure expressions.

Feedback

Was this page helpful? Yes No

DEFINE

09/16/2025

Introduces a statement with one or more entity definitions that can be applied to one or more EVALUATE statements of a DAX query.

Syntax

DAX

```
[DEFINE
    (
     (COLUMN <table name>[<column name>] = <scalar expression>) |
     (FUNCTION <function name> = ([parameter name]: [parameter type], ...) =>
<function body>) |
     (MEASURE <table name>[<measure name>] = <scalar expression>) |
     (TABLE <table name> = <virtual table definition>) |
     (VAR <var name> = <table or scalar expression>) |
    ) +
]

(EVALUATE <table expression>) +
```

Parameters

Expand table

Term	Definition
`Entity`	COLUMN[1], FUNCTION, MEASURE, TABLE[1], or VAR.
`name`	The name of a column, function, measure, table, or var definition. It cannot be an expression. The name does not have to be unique. The name exists only for the duration of the query.
`expression`	Any DAX expression that returns a table or scalar value. The expression can use any of the defined entities. If there is a need to convert a scalar expression into a table expression, wrap the expression inside a table constructor with curly braces `{}`, or use the `ROW()` function to return a single row table.
`parameter type`, `parameter name`, `function body`	See FUNCTION statement.

[1] **Caution:** Query scoped TABLE and COLUMN definitions are meant for internal use only. While you can define TABLE and COLUMN expressions for a query without syntax error, they may produce runtime errors and are not recommended.

Remarks

- A DAX query can have multiple EVALUATE statements, but can have only one DEFINE statement. Definitions in the DEFINE statement can apply to any EVALUATE statements in the query.
- At least one definition is required in a DEFINE statement.
- Measure definitions for a query override model measures of the same name.
- VAR names have unique restrictions. To learn more, see VAR - Parameters.
- To learn more about how a DEFINE statement is used, see DAX queries.
- To learn more about virtual column, see Virtual Column
- To learn more about virtual table, see Virtual Table
- To learn more about DAX user defined functions, see DAX User Defined Functions

Related content

- EVALUATE
- FUNCTION
- VAR
- MEASURE
- Virtual Column
- Virtual Table
- DAX queries

EVALUATE

Article • 12/13/2023

Introduces a statement containing a table expression required in a DAX query.

Syntax

```
DAX

EVALUATE <table>
```

Parameters

Expand table

Term	Definition
table	A table expression

Return value

The result of a table expression.

Remarks

- A DAX query can contain multiple EVALUATE statements.
- To learn more about how EVALUATE statements are used, see DAX queries.

Example

```
DAX

EVALUATE
    'Internet Sales'
```

Returns all rows and columns from the Internet Sales table, as a table.

Related content

ORDER BY
START AT
DEFINE
VAR
DAX queries

Feedback

Was this page helpful? Yes No

FUNCTION

Introduces a function definition in a DEFINE statement of a DAX query.

Syntax

DAX

```
[DEFINE
    (
      FUNCTION <function name> = ([<parameter name> [: [<type>] [<subtype>] [<passing
mode>]] [= <default expression>], ...]) => <function body>
    ) +
]

(EVALUATE <table expression>) +
```

Parameters

Expand table

Term	Definition
`function name`	The name of a function.
`parameter name`	The name of the parameter. This cannot be a reserved keyword such as `measure`.
`type`	The parameter type. Can be one of the following: `ANYVAL`, `SCALAR`, `TABLE`, `ANYREF`, `CALENDARREF`, `COLUMNREF`, `MEASUREREF`, `TABLEREF`. `ANYVAL` is an abstract type for `SCALAR` or `TABLE`. `ANYREF` is an abstract type for all references.
`subtype`	The parameter subtype. Applies only to `parameter type` = `SCALAR`. Can be one of the following: `BOOLEAN`, `DATETIME`, `DECIMAL`, `DOUBLE`, `INT64`, `NUMERIC`, `STRING`, `VARIANT`.
`passing mode`	The parameter passing mode. Can be `VAL` (eagerly evaluated) or `EXPR` (lazily evaluated).
`default expression`	A DAX expression used when the argument is omitted by the caller. Makes the parameter optional.
`function body`	A DAX expression for the function.

Return value

The calculated result of the function body.

Remarks

- To learn more about DAX User Defined Functions, see DAX User Defined Functions.
- To learn more about how FUNCTION statements are used, see DAX queries.

Related content

- Working with DAX User Defined Functions
- DEFINE
- EVALUATE
- DAX queries

Last updated on 06/02/2026

MEASURE

Article • 12/13/2023

Introduces a measure definition in a DEFINE statement of a DAX query.

Syntax

```DAX
[DEFINE
    (
      MEASURE <table name>[<measure name>] = <scalar expression>
    ) +
]

(EVALUATE <table expression>) +
```

Parameters

Expand table

Term	Definition
table name	The name of a table containing the measure.
measure name	The name of the measure. It cannot be an expression. The name does not have to be unique. The name exists only for the duration of the query.
scalar expression	A DAX expression that returns a scalar value.

Return value

The calculated result of the measure expression.

Remarks

- Measure definitions for a query override model measures of the same name for the duration of the query. They will not affect the model measure.
- The measure expression can be used with any other expression in the same query.

- To learn more about how MEASURE statements are used, see DAX queries.

Related content

DEFINE
EVALUATE
VAR
DAX queries

Feedback

Was this page helpful? Yes No

ORDER BY

Article • 12/13/2023

Introduces a statement that defines sort order of query results returned by an EVALUATE statement in a DAX query.

Syntax

DAX

```
[ORDER BY {<expression> [{ASC | DESC}]}[, …]]
```

Parameters

Expand table

Term	Definition
expression	Any DAX expression that returns a single scalar value.
ASC	(default) Ascending sort order.
DESC	Descending sort order.

Return value

The result of an EVALUATE statement in ascending (ASC) or descending (DESC) order.

Remarks

To learn more about how ORDER BY statements are used, see DAX queries.

Related content

START AT
EVALUATE
VAR
DEFINE
DAX queries

Feedback

Was this page helpful? Yes No

START AT

Article • 12/13/2023

Introduces a statement that defines the starting value at which the query results of an ORDER BY clause in an EVALUATE statement in a DAX query are returned.

Syntax

DAX

```
[START AT {<value>|<parameter>} [, …]]
```

Parameters

Expand table

Term	Definition
value	A constant value. Cannot be an expression.
parameter	The name of a parameter in an XMLA statement prefixed with an `@` character.

Remarks

- START AT arguments have a one-to-one correspondence with the columns in the ORDER BY statement. There can be as many arguments in the START AT statement as there are in the ORDER BY statement, but not more. The first argument in the START AT statement defines the starting value in column 1 of the ORDER BY columns. The second argument in the START AT statement defines the starting value in column 2 of the ORDER BY columns within the rows that meet the first value for column 1.
- To learn more about how START AT statements are used, see DAX queries.

Related content

ORDER BY
EVALUATE
VAR

Feedback

Was this page helpful? Yes No

VAR

Stores the result of an expression as a named variable, which you can then pass as an argument to other measure expressions. After DAX calculates the values for a variable expression, those values don't change, even if you reference the variable in another expression.

Syntax

DAX

```
VAR <name> = <expression>
```

Parameters

Expand table

Term	Definition
`name`	The name of the variable (identifier). Delimiters aren't supported. For example, 'varName' or [varName] results in an error. Supported character set: a-z, A-Z, 0-9. 0-9 aren't valid as first character. __ (double underscore) is allowed as a prefix to the identifier name. No other special characters are supported. Reserved keywords aren't allowed. Names of existing tables aren't allowed. Empty spaces aren't allowed.
`expression`	A DAX expression that returns a scalar or table value.

Return value

A named variable containing the result of the expression argument.

Remarks

- An expression passed as an argument to VAR can contain another VAR declaration.
- When you reference a variable:
  - Measures can't refer to variables defined outside the measure expression, but can refer to functional scope variables defined within the expression.

- Variables can refer to measures.
- Variables can refer to previously defined variables.
- You can't reference columns in table variables by using TableName[ColumnName] syntax.

- For best practices when using VAR, see Use variables to improve your DAX formulas.
- To learn more about how VAR is used within a DAX Query, see DAX queries.

Example

Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see DAX sample model.

To calculate a percentage of year-over-year growth without using a variable, you could create three separate measures. This first measure calculates Sum of Sales Amount:

DAX

```
Sum of Sales Amount =
SUM ( Sales[Sales Amount] )
```

A second measure calculates the sales amount for the previous year:

DAX

```
Sales Amount PreviousYear =
CALCULATE ( [Sum of Sales Amount], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
```

You can then create a third measure that combines the other two measures to calculate a growth percentage. Notice that Sum of SalesAmount is used in two places: first to check whether a sale exists, then again to calculate a percentage.

DAX

```
Sum of SalesAmount YoY%: =
IF (
    [Sum of Sales Amount] && [Sales Amount PreviousYear],
    DIVIDE (
        ( [Sum of Sales Amount] - [Sales Amount PreviousYear] ),
        [Sales Amount PreviousYear]
    )
)
```

By using a variable, you can create a single measure that calculates the same result:

DAX

```
YoY% =
VAR Sales =
    SUM ( Sales[Sales Amount] )
VAR SalesLastYear =
    CALCULATE ( SUM ( Sales[Sales Amount] ), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN
    IF ( Sales && SalesLastYear, DIVIDE ( Sales - SalesLastYear, SalesLastYear ) )
```

By using a variable, you can get the same outcome in a more readable way. And because the result of the expression is stored in the variable, the measure's performance improves significantly because DAX doesn't recalculate it each time it's used.

Related content

- Use variables to improve your DAX formulas
- DAX queries

Last updated on 08/04/2026

DAX glossary

09/16/2025

Analytic query

Power BI visuals query a data model by using an *analytic query*. An analytic query strives to reduce potentially large data volumes and model complexities using three distinct phases: Filter, group and summarize. An analytic query is created automatically when fields are assigned to the wells of report visuals. Report authors can control the behavior of field assignments by renaming fields, modifying the summarization technique, or disabling summarization to achieve grouping. At report design time, filters can be added to the report, a report page, or a visual. In reading view, filters can be modified in the **Filters** pane, or by interactions with slicers and other visuals (cross-filtering).

BLANK

DAX defines the absence of a value as BLANK. It's the equivalent of SQL NULL, but it doesn't behave exactly the same. It's more closely aligned to Excel and how it defines an empty cell. BLANK is evaluated as zero or an empty string when combined with other operations. For example, BLANK + 20 = 20. Always use capital letters; the plural is BLANKs, with a lowercase "s".

Calculated column

A model calculation used to add a column to a tabular model by writing a DAX formula. The formula must return a scalar value, and it's evaluated for each row in the table. A calculated column can be added to an Import or DirectQuery storage mode table.

Calculated measure

In tabular modeling, there's no such concept as a *calculated measure*. Use *measure* instead. The word *calculated* is used to describe calculated tables and calculated columns. It distinguishes them from tables and columns that originate from Power Query. Power Query doesn't have the concept of a measure.

Calculated table

A model calculation used to add a table to a tabular model by writing a DAX formula. The formula must return a table object. It results in a table that uses Import storage mode.

Calculation

A deliberate process that transforms one or more inputs into one or more results. In a tabular data model, a calculation can be a model object; either a calculated table, calculated column, or measure.

Context

Describes the environment in which a DAX formula is evaluated. There are two types of context: *Row context* and *filter context*. Row context represents the "current row", and is used to evaluate calculated column formulas and expressions used by table iterators. Filter context is used to evaluate measures, and it represents filters applied directly to model columns and filters propagated by model relationships.

DAX

Data Analysis Expressions (DAX) language is a formula language for Power Pivot in Excel, Power BI, Azure Analysis Services, and tabular modeling in SQL Server Analysis Services. You can also use DAX to add data model calculations and define row-level security (RLS) rules.

Dynamic security

When row-level security (RLS) rules are enforced by using the identity of the report user. Rules filter model tables by using the user's account name, which can be done with the USERNAME or USERPRINCIPALNAME functions. See Row-level security.

Expression

A unit of DAX logic that's evaluated and returns a result. Expressions can declare variables in which case they're assigned a sub-expression and must include a RETURN statement that outputs a final expression. Expressions are constructed by using model objects (tables, columns, or measures), functions, operators, or constants.

Field

Data model resource presented in the **Fields** pane. Fields are used to configure report filters and visuals. Fields consist of model columns, hierarchy levels, and measures.

Formula

One or more DAX expressions used to define a model calculation. Inner expressions are called sub-expressions. Plural is *formulas*.

Function

DAX functions have arguments that allow passing in parameters. Formulas can use many function calls, possibly nesting functions within other functions. In a formula, function names must be followed by parentheses. Within the parentheses, parameters are passed in.

Implicit measure

An automatically generated calculation achieved by configuring a Power BI visual to summarize column values. **Numeric** columns support the greatest range of summarization, including: Sum, Average, Minimum, Maximum, Count (Distinct), Count, Standard deviation, Variance, or Median. Columns of other data types can be summarized, too. **Text** columns can be summarized by using: First (alphabetically), Last (alphabetically), Count (Distinct), or Count. **Date** columns can be summarized by using: Earliest, Latest, Count (Distinct), or Count. **Boolean** columns can be summarized by using: Count (Distinct), or Count.

Iterator function

A DAX function that enumerates all rows of a given table and evaluate a given expression for each row. It provides flexibility and control over how model calculations summarize data.

MDX

Multidimensional Expressions (MDX) language is a formula language for SQL Server Analysis Services multidimensional models (also known as *cubes*). MDX can be used to query tabular models, however it can't define implicit measures. It can only query measures that are already defined in the model.

Measure

A calculation that achieves summarization. Measures are either *implicit* or *explicit*. An explicit measure is a calculation added to a tabular data model by writing a DAX formula. A measure formula must return a scalar value. In the **Fields** pane, explicit measures are adorned with a calculator icon. Explicit measures are required when the model is queried by using Multidimensional Expressions (MDX), as is the case when using Analyze in Excel. An explicit measure is commonly just called a measure.

Measure group

A model table that contains at least one measure, and has no hierarchies or visible columns. In the **Fields** pane, each measure group is adorned with a multi-calculator icon. Measure groups are listed together at the top of the **Fields** pane, and sorted alphabetically by name.

Model calculation

A named formula that's used to add a calculated table, calculated column, or measure to a tabular data model. Its structure is `NAME` = `FORMULA`. Most calculations are added by data modelers in Power BI Desktop, but measures can also be added to a live connection report. See Report measures.

Quick measures

A feature in Power BI Desktop that eliminates the need to write DAX formulas for commonly defined measures. Quick measures include average per category, rank, and difference from baseline.

Report measures

Also called *report-level measures*. They're added to a live connection report in Power BI Desktop by writing a DAX formula, but only for connections to Power BI models or Analysis Services tabular models.

Row-level security

Also called `RLS`. Design technique to restrict access to subsets of data for specific users. In a tabular model, it's achieved by creating model roles. Roles have rules, which are DAX expressions to filter table rows.

Scalar

In DAX, a scalar is a single value. A scalar can be of any data type: Decimal, Integer, DateTime, String, Currency, Boolean. A scalar value can be the result of an expression calculated from multiple values. For example, an aggregation function such as MAX() returns a single maximum value from a set of values from which to evaluate.

Summarization

An operation applied to the values of a column. See measure.

Time intelligence

Time intelligence relates to calculations over time, like year-to-date (YTD).

Time intelligence function

DAX includes many time intelligence functions. Each time intelligence function achieves its result by modifying the filter context for date filters. Example functions: TOTALYTD and SAMEPERIODLASTYEAR.

User-defined function

A user-created DAX function. Provides the ability to define named functions with parameters for use in any DAX expression. UDFs promote code reuse and simply model maintenance by centralizing logic. Functions are defined in DAX Query View or TMDL View and can include parameter types for validation. This is currently in preview. To learn more, see DAX user-defined functions (preview).

Value, values

Data to be visualized.

What-if parameter

A Power BI Desktop feature that provides the ability to accept user input through slicers. Each parameter creates a single-column calculated table and a measure that returns a single-selected value. The measure can be used in model calculations to respond to the user's input.

DAX operators

Article • 10/20/2023

The Data Analysis Expression (DAX) language uses operators to create expressions that compare values, perform arithmetic calculations, or work with strings.

Types of operators

There are four different types of calculation operators: arithmetic, comparison, text concatenation, and logical.

Arithmetic operators

To perform basic mathematical operations such as addition, subtraction, or multiplication; combine numbers; and produce numeric results, use the following arithmetic operators.

Arithmetic operator	Meaning	Example
+ (plus sign)	Addition	3+3
– (minus sign)	Subtraction or sign	3–1–1
* (asterisk)	Multiplication	3*3
/ (forward slash)	Division	3/3
^ (caret)	Exponentiation	16^4

> **Note**
>
> The plus sign can function both as a *binary operator* and as a *unary operator*. A binary operator requires numbers on both sides of the operator and performs addition. When you use values in a DAX formula on both sides of the binary operator, DAX tries to cast the values to numeric data types if they are not already numbers. In contrast, the unary operator can be applied to any type of argument. The plus symbol does not affect the type or value and is simply ignored, whereas the minus operator creates a negative value, if applied to a numeric value.

Comparison operators

You can compare two values with the following operators. When two values are compared by using these operators, the result is a logical value, either TRUE or FALSE.

Comparison operator	Meaning	Example
=	Equal to	[Region] = "USA"
==	Strict equal to	[Region] == "USA"
>	Greater than	[Sales Date] > "Jan 2009"
<	Less than	[Sales Date] < "Jan 1 2009"
>=	Greater than or equal to	[Amount] >= 20000
<=	Less than or equal to	[Amount] <= 100
<>	Not equal to	[Region] <> "USA"

All comparison operators except == treat BLANK as equal to number 0, empty string "", DATE(1899, 12, 30), or FALSE. As a result, [Column] = 0 will be true when the value of [Column] is either 0 or BLANK. In contrast, [Column] == 0 is true only when the value of [Column] is 0.

Text concatenation operator

Use the ampersand (**&**) to join, or concatenate, two or more text strings to produce a single piece of text.

Text operator	Meaning	Example
& (ampersand)	Connects, or concatenates, two values to produce one continuous text value	[Region] & ", " & [City]

Logical operators

Use logical operators (&&) and (||) to combine expressions to produce a single result.

Text operator	Meaning	Examples
&& (double ampersand)	Creates an AND condition between two expressions that each have a Boolean result. If both expressions return TRUE, the combination of the expressions also returns TRUE; otherwise the combination returns FALSE.	([Region] = "France") && ([BikeBuyer] = "yes"))

Text operator	Meaning	Examples
|| (double pipe symbol)	Creates an OR condition between two logical expressions. If either expression returns TRUE, the result is TRUE; only when both expressions are FALSE is the result FALSE.	(([Region] = "France") || ([BikeBuyer] = "yes"))
IN	Creates a logical OR condition between each row being compared to a table. Note: the table constructor syntax uses curly braces.	'Product'[Color] IN { "Red", "Blue", "Black" }

Operators and precedence order

In some cases, the order in which calculation is performed can affect the Return value; therefore, it is important to understand how the order is determined and how you can change the order to obtain the desired results.

Calculation order

An expression evaluates the operators and values in a specific order. All expressions always begin with an equal sign (=). The equal sign indicates that the succeeding characters constitute an expression.

Following the equal sign are the elements to be calculated (the operands), which are separated by calculation operators. Expressions are always read from left to right, but the order in which the elements are grouped can be controlled to some degree by using parentheses.

Operator precedence

If you combine several operators in a single formula, the operations are ordered according to the following table. If the operators have equal precedence value, they are ordered from left to right. For example, if an expression contains both a multiplication and division operator, they are evaluated in the order that they appear in the expression, from left to right.

Operator	Description
^	Exponentiation
–	Sign (as in –1)
* and /	Multiplication and division

Operator	Description
+ and –	Addition and subtraction
&	Connects two strings of text (concatenation)
=,==,<,>,<=,>=,<>,IN	Comparison
NOT	NOT (unary operator)

Using parentheses to control calculation order

To change the order of evaluation, you should enclose in parentheses that part of the formula that must be calculated first. For example, the following formula produces 11 because multiplication is calculated before addition. The formula multiplies 2 by 3, and then adds 5 to the result.

```
DAX

=5+2*3
```

In contrast, if you use parentheses to change the syntax, the order is changed so that 5 and 2 are added together, and the result multiplied by 3 to produce 21.

```
DAX

=(5+2)*3
```

In the following example, the parentheses around the first part of the formula force the calculation to evaluate the expression `(3 + 0.25)` first and then divide the result by the result of the expression, `(3 - 0.25)`.

```
DAX

=(3 + 0.25)/(3 - 0.25)
```

In the following example, the exponentiation operator is applied first, according to the rules of precedence for operators, and then the sign operator is applied. The result for this expression is -4.

```
DAX

=-2^2
```

To ensure that the sign operator is applied to the numeric value first, you can use parentheses to control operators, as shown in the following example. The result for this expression is 4.

```
DAX
= (-2)^2
```

Compatibility

DAX easily handles and compares various data types, much like Microsoft Excel. However, the underlying computation engine is based on SQL Server Analysis Services and provides additional advanced features of a relational data store, including richer support for date and time types. Therefore, in some cases the results of calculations or the behavior of functions may not be the same as in Excel. Moreover, DAX supports more data types than does Excel. This section describes the key differences.

Coercing data types of operands

In general, the two operands on the left and right sides of any operator should be the same data type. However, if the data types are different, DAX will convert them to a common data type to apply the operator in some cases:

1. Both operands are converted to the largest possible common data type.
2. The operator is applied, if possible.

For example, suppose you have two numbers that you want to combine. One number results from a formula, such as `=[Price] * .20`, and the result may contain many decimal places. The other number is an integer that has been provided as a string value.

In this case, DAX will convert both numbers to real numbers in a numeric format, using the largest numeric format that can store both kinds of numbers. Then DAX will apply the multiplication.

Depending on the data-type combination, type coercion may not be applied for comparison operations. For a complete list of data types supported by DAX, see Data types supported in tabular models and Data types in Power BI Desktop.

Integer, Real Number, Currency, Date/time and Blank are considered numeric for comparison purposes. Blank evaluates to zero when performing a comparison. The following data-type combinations are supported for comparison operations.

Left Side Data Type	Right Side Data Type
Numeric	Numeric
Boolean	Boolean
String	String

Other mixed data-type comparisons will return an error. For example, a formula such as ="1" > 0 returns an error stating that *DAX comparison operations do not support comparing values of type Text with values of type Integer*.

Data Types used in DAX	Data Types used in Excel
Numbers (I8, R8)	Numbers (R8)
String	String
Boolean	Boolean
DateTime	Variant
Currency	Currency

Differences in precedence order

The precedence order of operations in DAX formulas is basically the same as that used by Microsoft Excel, but some Excel operators are not supported, such as percent. Also, ranges are not supported.

Therefore, whenever you copy and paste formulas from Excel, be sure to review the formula carefully, as some operators or elements in the formulas may not be valid. When there is any doubt about the order in which operations are performed, it's recommended you use parentheses to control the order of operations and remove any ambiguity about the result.

See also

DAX syntax
DAX parameter-naming

Feedback

Was this page helpful? Yes No

DAX queries

Article • 12/13/2024

Reporting clients like Power BI and Excel execute DAX queries whenever visuals display in a report, or a field added to a table, and these DAX queries adjust when a filter is applied. The performance analyzer in Power BI Desktop can show you these DAX queries and even run them in the DAX query view.

By using DAX query view in Power BI Desktop or Write DAX queries in Power BI service, you can create and run your own DAX queries. With Microsoft Fabric, you can further increase your productivity with Copilot to write DAX queries in DAX query view of Desktop or web. In addition to Power BI tools, DAX queries can be run in Fabric notebooks using semantic link to read data from semantic models with python, and with the Execute Queries REST API, also available in Power Automate. Other tools such as SQL Server Management Studio (SSMS), Power BI Report Builder, and open-source tools like DAX Studio, also allow you to create and run DAX queries.

DAX queries return results as a table right within the tool, allowing you to quickly create and test the performance of your DAX formulas in measures or simply view the data in your semantic model. INFO and INFO.VIEW DAX functions can also get information about your semantic model, such as a listing of tables, columns, measures, and much more.

Before learning about queries, it is important you have a solid understanding of DAX basics. If you haven't already, be sure to check out DAX overview.

Keywords

DAX queries have a simple syntax comprised of just one required keyword, EVALUATE. EVALUATE is followed by a table expression, such as a DAX function or table name, that when run outputs a result table. Table expressions that output a result table include:

1. Common DAX functions that output a table, such as SUMMARIZE, SUMMARIZECOLUMNS, SELECTCOLUMNS, FILTER, UNION, TOPN, ADDCOLUMNS, DATATABLE, and many others, work with EVALUATE to output a result table.
2. Tables in the model when referenced by name work with EVALUATE to output a result table showing the data in the table. For example, **EVALUATE 'Table name'** can be ran as a DAX query.
3. Measures in the model or any DAX formula, which return a scalar value, work with EVALUATE to show the value as a result table when enclosed in curly braces. For

example, **EVALUATE {[Total Sales]}** or **EVALUATE {COUNTROWS('Sales')}** can be ran as a DAX query. These are called table constructors.

There are several optional keywords specific to DAX queries: ORDER BY, START AT, DEFINE, MEASURE, VAR, TABLE, and COLUMN.

EVALUATE (Required)

At the most basic level, a DAX query is an `EVALUATE` statement containing a table expression. At least one EVALUATE statement is required, however, a query can contain any number of EVALUATE statements.

EVALUATE Syntax

DAX

```
EVALUATE <table>
```

EVALUATE Parameters

Expand table

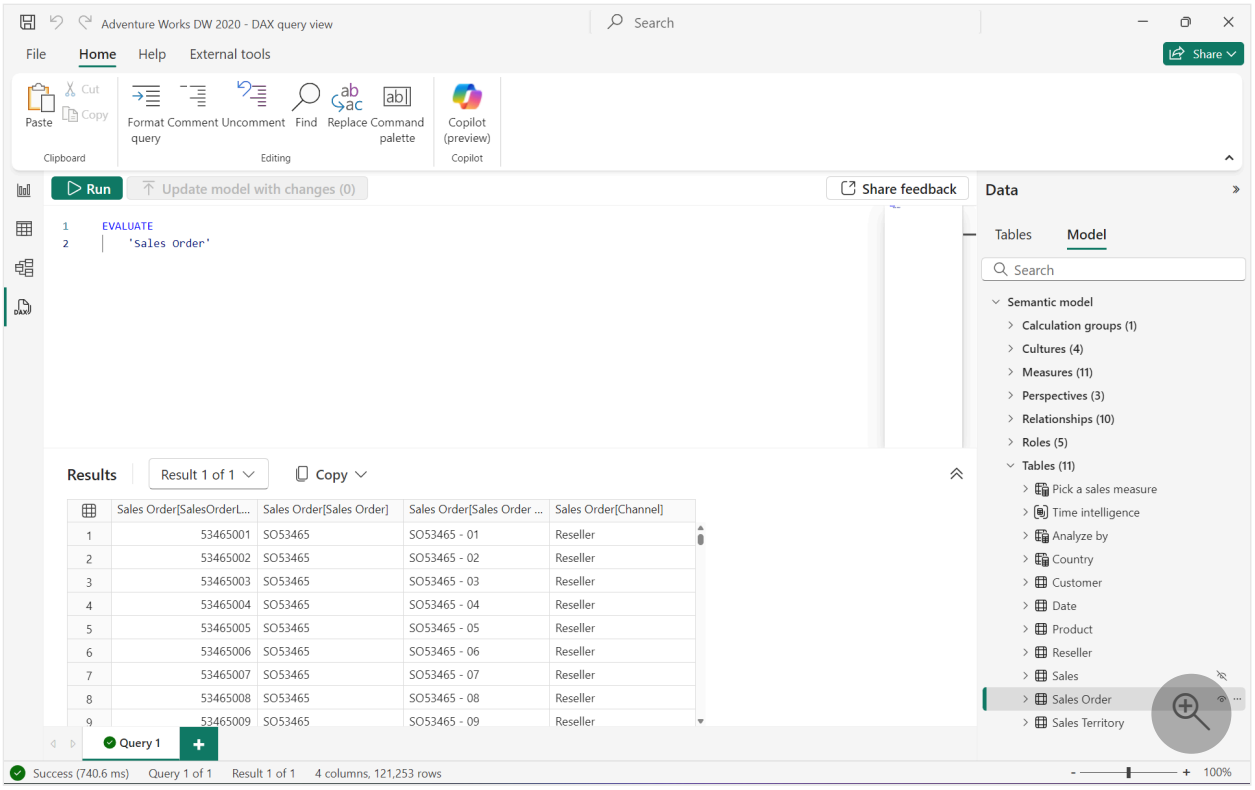
Term	Definition
`table`	A table expression.

EVALUATE Example

DAX

```
EVALUATE
    'Sales Order'
```

Returns all rows and columns from the Sales Order table, as a result table. This can be limited with the use of TOPN or FILTER, and sorted with ORDER BY.



ORDER BY (Optional)

The optional `ORDER BY` keyword defines one or more columns in the query or expressions used to sort query results. Any expression that can be evaluated for each row of the result is valid. Any column in the query itself is also valid.

Sort by column property in semantic models do not apply to DAX query results. If a column should be sorted by a different column in the model, such as in the case of Month Name, the sort by column should also be included in the DAX query to be used in the ORDER BY.

ORDER BY Syntax

DAX

```
EVALUATE <table>
[ORDER BY {<expression> [{ASC | DESC}]}[, …]]
```

ORDER BY Parameters

Expand table

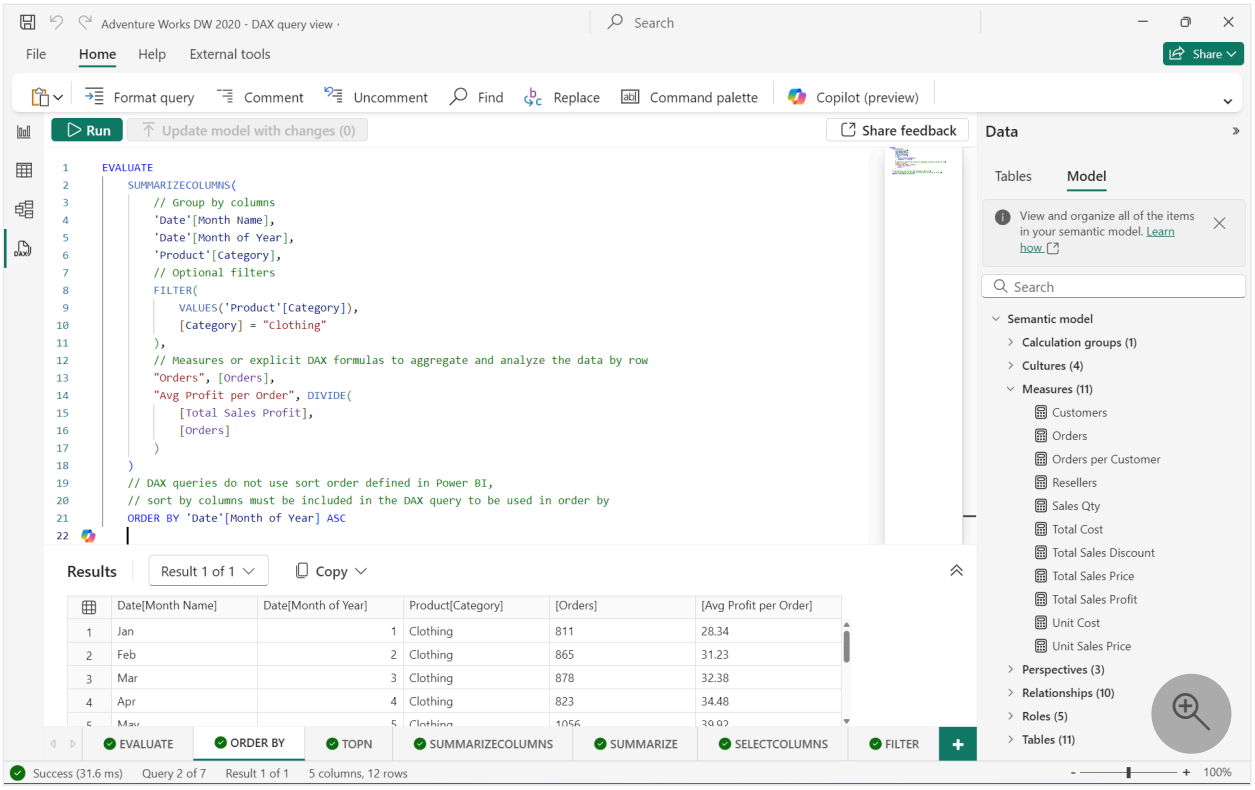
Term	Definition
`expression`	Any DAX expression that returns a single scalar value, or column included in the DAX query.
`ASC`	(default) Ascending sort order.
`DESC`	Descending sort order.

ORDER BY Example

DAX

```
EVALUATE
    SUMMARIZECOLUMNS(
        // Group by columns
        'Date'[Month Name],
        'Date'[Month of Year],
        'Product'[Category],

        // Optional filters
        FILTER(
            VALUES('Product'[Category]),
            [Category] = "Clothing"
        ),

        // Measures or explicit DAX formulas to aggregate and analyze the
data by row
        "Orders", [Orders],
        "Avg Profit per Order", DIVIDE(
            [Total Sales Profit],
            [Orders]
        )
    )

    // DAX queries do not use sort order defined in Power BI,
    // sort by columns must be included in the DAX query to be used in order
by
    ORDER BY 'Date'[Month of Year] ASC
```

Returns clothing orders and average profit per order by month, in ascending order by month, as a result table.

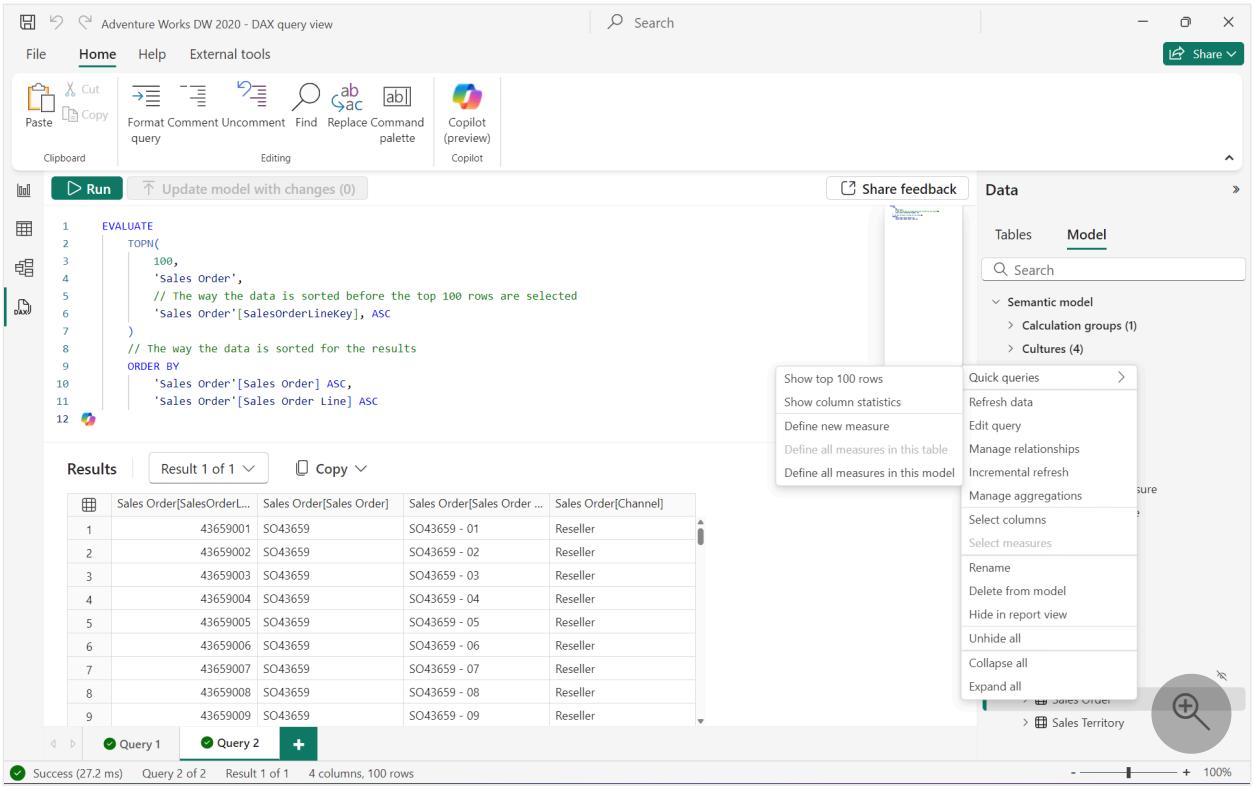


TOPN does not choose the specified number of rows to return based on the sort order specified in ORDER BY. Instead, TOPN has its own syntax to optionally specify a sort before the top 100 rows are return. ORDER BY only sorts the result table returned by TOPN.

DAX

```
EVALUATE
    TOPN(
        100,
        'Sales Order',
        // The way the data is sorted before the top 100 rows are selected
        'Sales Order'[SalesOrderLineKey], ASC
    )
    // The way the data is sorted for the results
    ORDER BY
        'Sales Order'[Sales Order] ASC,
        'Sales Order'[Sales Order Line] ASC
```

Returns the top 100 sales orders sorted by SalesOrderLienKey ascending, then sorts the results first by sales order, then by sales order line.



START AT (Optional)

The optional `START AT` keyword is used inside an `ORDER BY` clause. It defines the value at which the query results begin.

START AT Syntax

DAX

```
EVALUATE <table>
[ORDER BY {<expression> [{ASC | DESC}]}[, …]
[START AT {<value>|<parameter>} [, …]]]
```

START AT Parameters

Expand table

Term	Definition
`value`	A constant value. Cannot be an expression.
`parameter`	The name of a parameter in an XMLA statement prefixed with an `@` character.

START AT Remarks

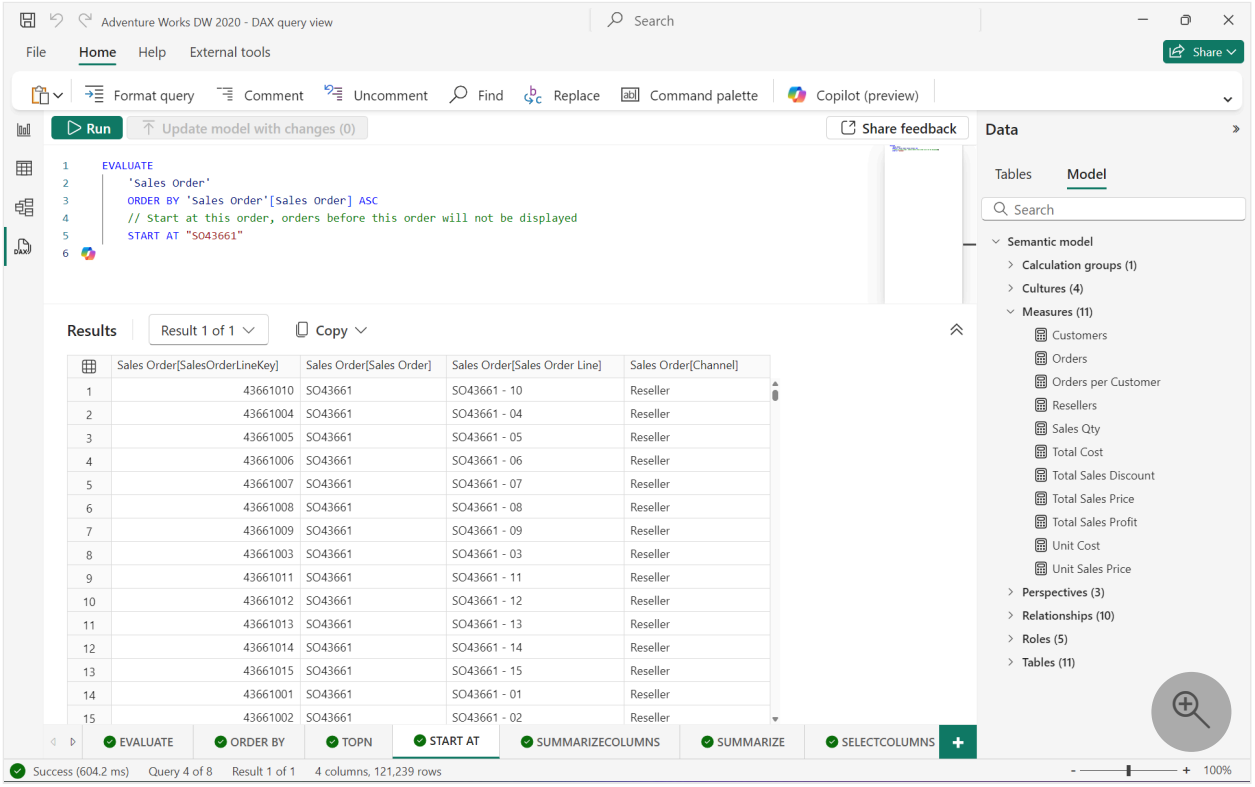
START AT arguments have a one-to-one correspondence with the columns in the ORDER BY clause. There can be as many arguments in the START AT clause as there are in the ORDER BY clause, but not more. The first argument in the START AT defines the starting value in column 1 of the ORDER BY columns. The second argument in the START AT defines the starting value in column 2 of the ORDER BY columns within the rows that meet the first value for column 1.

START AT Example

DAX

```
EVALUATE
    'Sales Order'
    ORDER BY 'Sales Order'[Sales Order] ASC
    // Start at this order, orders before this order will not be displayed
    START AT "SO43661"
```

Returns all columns from the Sales Order table, in ascending order by Sales Order, beginning at SO43661. Rows before this sales order are not included in the result table.



	Sales Order[SalesOrderLineKey]	Sales Order[Sales Order]	Sales Order[Sales Order Line]	Sales Order[Channel]
1	43661010	SO43661	SO43661 - 10	Reseller
2	43661004	SO43661	SO43661 - 04	Reseller
3	43661005	SO43661	SO43661 - 05	Reseller
4	43661006	SO43661	SO43661 - 06	Reseller
5	43661007	SO43661	SO43661 - 07	Reseller
6	43661008	SO43661	SO43661 - 08	Reseller
7	43661009	SO43661	SO43661 - 09	Reseller
8	43661003	SO43661	SO43661 - 03	Reseller
9	43661011	SO43661	SO43661 - 11	Reseller
10	43661012	SO43661	SO43661 - 12	Reseller
11	43661013	SO43661	SO43661 - 13	Reseller
12	43661014	SO43661	SO43661 - 14	Reseller
13	43661015	SO43661	SO43661 - 15	Reseller
14	43661001	SO43661	SO43661 - 01	Reseller
15	43661002	SO43661	SO43661 - 02	Reseller

DEFINE (Optional)

The optional `DEFINE` keyword introduces one or more calculated entity definitions that exist only for the duration of the query. Unlike `EVALUATE`, there can only be one `DEFINE` block with one or more definitions in a DAX query. `DEFINE` must precede the first

`EVALUATE` statement and are valid for all EVALUATE statements in the query. Definitions can be variables, measures, tables[1], and columns[1]. Definitions can reference other definitions that appear before or after the current definition. At least one definition is required if the `DEFINE` keyword is included in a query.

`DEFINE MEASURE` is a common scenario to build new measures or edit existing measures in a semantic model. When the measure already exists in the model, the DAX query will use the measure DAX formula defined in the query. This is helpful for testing measures with a DAX query before updating the model.

`DEFINE MEASURE` is also helpful to build additional analysis with DAX formulas for a specific DAX query where you may not have permission to add a model measure or it is not necessary to have it in the model.

DEFINE Syntax

DAX

```
[DEFINE
    (
     (MEASURE <table name>[<measure name>] = <scalar expression>) |
     (VAR <var name> = <table or scalar expression>) |
     (TABLE <table name> = <virtual table definition>) |
     (COLUMN <table name>[<column name>] = <scalar expression>) |
    ) +
]

(EVALUATE <table expression>) +
```

DEFINE Parameters

Expand table

Term	Definition
`Entity`	MEASURE, VAR, TABLE[1], or COLUMN[1].
`name`	The name of a measure, var, table, or column definition. It cannot be an expression. The name does not have to be unique. The name exists only for the duration of the query.
`expression`	Any DAX expression that returns a table or scalar value. The expression can use any of the defined entities. If there is a need to convert a scalar expression into a table expression, wrap the expression inside a table constructor with curly braces `{}`, or use the `ROW()` function to return a single row table.

[1] **Caution:** Query scoped TABLE and COLUMN definitions are meant for internal use only. While you can define TABLE and COLUMN expressions for a query without syntax error, they may produce runtime errors and are not recommended.

DEFINE Remarks

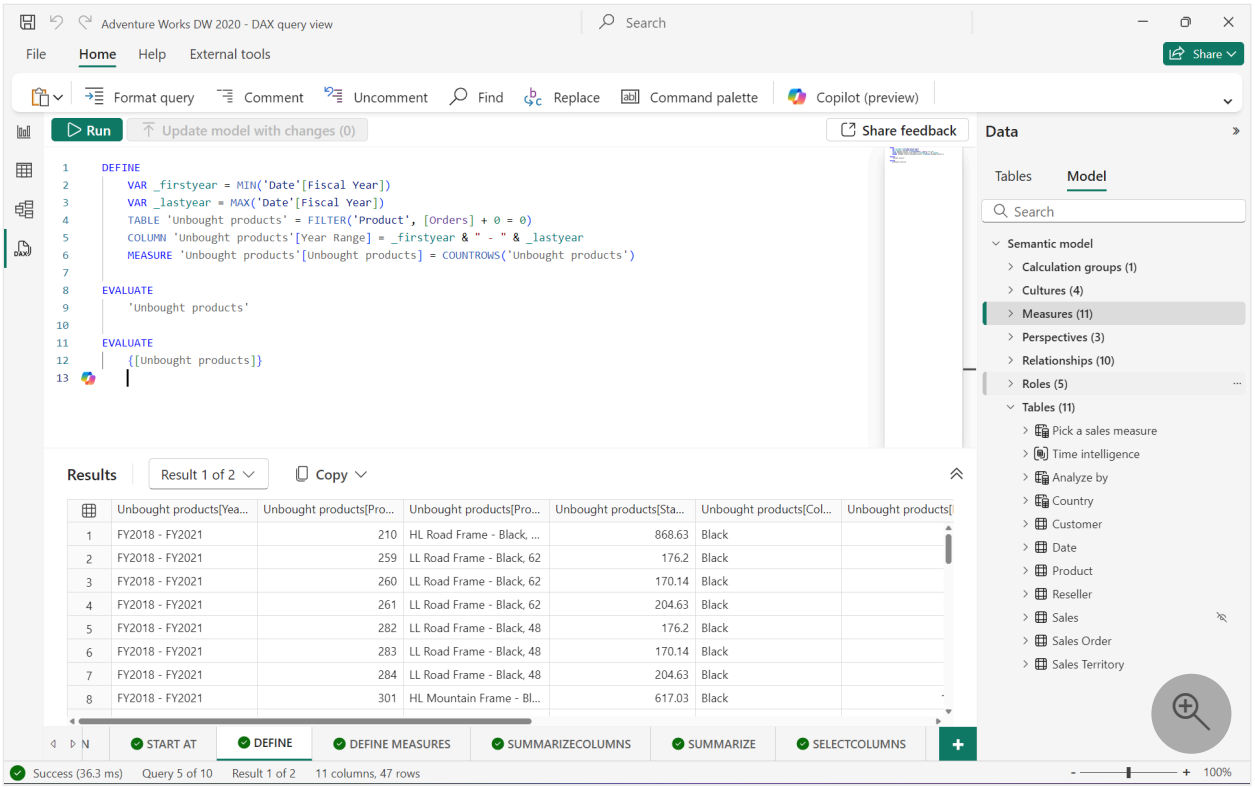
- A DAX query can have multiple EVALUATE statements, but can have only one DEFINE statement. Definitions in the DEFINE statement can apply to any EVALUATE statements in the query.
- At least one definition is required in a DEFINE statement.
- Measure definitions for a query override model measures of the same name but are only used within the query. They will not affect the model measure.
- VAR names have unique restrictions. To learn more, see VAR - Parameters.

DEFINE Example

DAX

```
DEFINE
    VAR _firstyear = MIN('Date'[Fiscal Year])
    VAR _lastyear = MAX('Date'[Fiscal Year])
    TABLE 'Unbought products' = FILTER('Product', [Orders] + 0 = 0)
    COLUMN 'Unbought products'[Year Range] = _firstyear & " - " & _lastyear
    MEASURE 'Unbought products'[Unbought products] = COUNTROWS('Unbought
products')

EVALUATE
    'Unbought products'

EVALUATE
    {[Unbought products]}
```

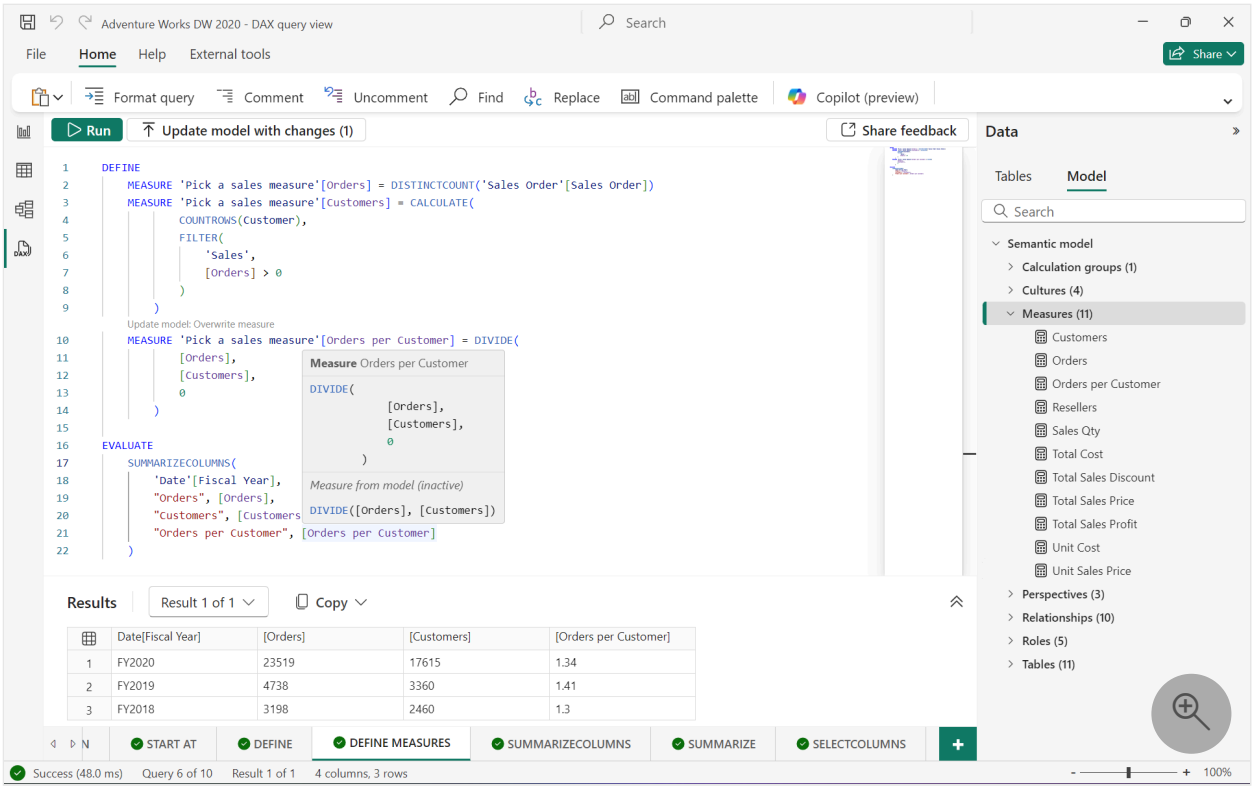
Returns the table defined in the DAX query to show unbought products with an additional defined column referencing defined variables. A measure is also defined and evaluated to count the rows of unbought products.



DAX

```
DEFINE
    MEASURE 'Pick a sales measure'[Orders] = DISTINCTCOUNT('Sales
Order'[Sales Order])
    MEASURE 'Pick a sales measure'[Customers] = CALCULATE(
            COUNTROWS(Customer),
            FILTER(
                'Sales',
                [Orders] > 0
            )
        )
    MEASURE 'Pick a sales measure'[Orders per Customer] = DIVIDE(
            [Orders],
            [Customers],
            0
        )

EVALUATE
    SUMMARIZECOLUMNS(
        'Date'[Fiscal Year],
        "Orders", [Orders],
        "Customers", [Customers],
        "Orders per Customer", [Orders per Customer]
    )
```

Returns a table evaluating three defined measures to show the results by fiscal year. All measures also exist in the model, and Orders per Customer is modified in the DAX query.



Parameters in DAX queries

A well-defined DAX query statement can be parameterized and then used over and over with just changes in the parameter values.

The Execute Method (XMLA) method has a Parameters Element (XMLA) collection element that allows parameters to be defined and assigned a value. Within the collection, each Parameter Element (XMLA) element defines the name of the parameter and a value to it.

Reference XMLA parameters by prefixing the name of the parameter with an `@` character. Any place in the syntax where a value is allowed, the value can be replaced with a parameter call. All XMLA parameters are typed as text.

> **Important**
>
> Parameters defined in the parameters section and not used in the `<STATEMENT>` element generate an error response in XMLA. Parameters used and not defined in the `<Parameters>` element generate an error response in XMLA.

Related content

DAX statements SUMMARIZECOLUMNS

Feedback

Was this page helpful? Yes No

DAX parameter-naming conventions

Article • 10/20/2023

Parameter names are standardized in DAX reference to facilitate the usage and understanding of the functions.

Parameter names

Term	Definition
expression	Any DAX expression that returns a single scalar value, where the expression is to be evaluated multiple times (for each row/context).
value	Any DAX expression that returns a single scalar value where the expression is to be evaluated exactly once before all other operations.
table	Any DAX expression that returns a table of data.
tableName	The name of an existing table using standard DAX syntax. It cannot be an expression.
columnName	The name of an existing column using standard DAX syntax, usually fully qualified. It cannot be an expression.
name	A string constant that will be used to provide the name of a new object.
order	An enumeration used to determine the sort order.
ties	An enumeration used to determine the handling of tie values.
type	An enumeration used to determine the data type for PathItem and PathItemReverse.

Prefixing parameter names or using the prefix only

Term	Definition
prefixing	Parameter names may be further qualified with a prefix that is descriptive of how the argument is used and to avoid ambiguous reading of the parameters. For example: Result_ColumnName - Refers to an existing column used to get the result values in the LOOKUPVALUE() function. Search_ColumnName - Refers to an existing column used to search for a value in the LOOKUPVALUE() function.

Term	Definition
omitting	Parameter names will be omitted if the prefix is clear enough to describe the parameter. For example, instead of having the following syntax DATE (Year_Value, Month_Value, Day_Value) it is clearer for the user to read DATE (Year, Month, Day); repeating three times the suffix value does not add anything to a better comprehension of the function and it clutters the reading unnecessarily. However, if the prefixed parameter is Year_columnName then the parameter name and the prefix will stay to make sure the user understands that the parameter requires a reference to an existing column of Years.

Feedback

Was this page helpful? Yes No

DAX syntax

Article • 10/20/2023

This article describes syntax and requirements for the DAX formula expression language.

Syntax requirements

A DAX formula always starts with an equal sign (=). After the equals sign, you can provide any expression that evaluates to a scalar, or an expression that can be converted to a scalar. These include the following:

- A scalar constant, or expression that uses a scalar operator (+,-,*,/,>=,...,&&, ...)
- References to columns or tables. The DAX language always uses tables and columns as inputs to functions, never an array or arbitrary set of values.
- Operators, constants, and values provided as part of an expression.
- The result of a function and its required arguments. Some DAX functions return a table instead of a scalar, and must be wrapped in a function that evaluates the table and returns a scalar; unless the table is a single column, single row table, then it is treated as a scalar value.

  Most DAX functions require one or more arguments, which can include tables, columns, expressions, and values. However, some functions, such as PI, do not require any arguments, but always require parentheses to indicate the null argument. For example, you must always type PI(), not PI. You can also nest functions within other functions.
- Expressions. An expression can contain any or all of the following: operators, constants, or references to columns.

For example, the following are all valid formulas.

Formula	**Result**
= 3	3
= "**Sales**"	**Sales**
= 'Sales'[Amount]	If you use this formula within the Sales table, you will get the value of the column Amount in the Sales table for the current row.

Formula	Result
= (0.03 * [Amount]) =0.03 * [Amount]	Three percent of the value in the Amount column of the current table. Although this formula can be used to calculate a percentage, the result is not shown as a percentage unless you apply formatting in the table.
= PI()	The value of the constant pi.

Formulas can behave differently depending on how they are used. You must always be aware of the context and how the data that you use in the formula is related to other data that might be used in the calculation.

Naming requirements

A data model often contains multiple tables. Together the tables and their columns comprise a database stored in the in-memory analytics engine (VertiPaq). Within that database, all tables must have unique names. The names of columns must also be unique within each table. All object names are *case-insensitive*; for example, the names **SALES** and **Sales** would represent the same table.

Each column and measure you add to an existing data model must belong to a specific table. You specify the table that contains the column either implicitly, when you create a calculated column within a table, or explicitly, when you create a measure and specify the name of the table where the measure definition should be stored.

When you use a table or column as an input to a function, you must generally *qualify* the column name. The *fully qualified* name of a column is the table name, followed by the column name in square brackets: for examples, 'U.S. Sales'[Products]. A fully qualified name is always required when you reference a column in the following contexts:

- As an argument to the function, VALUES
- As an argument to the functions, ALL or ALLEXCEPT
- In a filter argument for the functions, CALCULATE or CALCULATETABLE
- As an argument to the function, RELATEDTABLE
- As an argument to any time intelligence function

An *unqualified* column name is just the name of the column, enclosed in brackets: for example, [Sales Amount]. For example, when you are referencing a scalar value from the

same row of the current table, you can use the unqualified column name.

If the name of a table contains spaces, reserved keywords, or disallowed characters, you must enclose the table name in single quotation marks. You must also enclose table names in quotation marks if the name contains any characters outside the ANSI alphanumeric character range, regardless of whether your locale supports the character set or not. For example, if you open a workbook that contains table names written in Cyrillic characters, such as 'Таблица', the table name must be enclosed in quotation marks, even though it does not contain spaces.

> **Note**
>
> To make it easier to enter the fully qualified names of columns, use the AutoComplete feature in the formula editor.

Tables

- Table names are required whenever the column is from a different table than the current table. Table names must be unique within the database.
- Table names must be enclosed in single quotation marks if they contain spaces, other special characters or any non-English alphanumeric characters.

Measures

- Measure names must always be in brackets.
- Measure names can contain spaces.
- Each measure name must be unique within a model. Therefore, the table name is optional in front of a measure name when referencing an existing measure. However, when you create a measure you must always specify a table where the measure definition will be stored.

Columns

Column names must be unique in the context of a table; however, multiple tables can have columns with the same names (disambiguation comes with the table name).

In general, columns can be referenced without referencing the base table that they belong to, except when there might be a name conflict to resolve or with certain functions that require column names to be fully qualified.

Reserved keywords

If the name that you use for a table is the same as an Analysis Services reserved keyword, an error is raised, and you must rename the table. However, you can use keywords in object names if the object name is enclosed in brackets (for columns) or quotation marks (for tables).

> **Note**
>
> Quotation marks can be represented by several different characters, depending on the application. If you paste formulas from an external document or Web page, make sure to check the ASCII code of the character that is used for opening and closing quotes, to ensure that they are the same. Otherwise DAX may be unable to recognize the symbols as quotation marks, making the reference invalid.

Special characters

The following characters and character types are not valid in the names of tables, columns, or measures:

- Leading or trailing spaces; unless the spaces are enclosed by name delimiters, brackets, or single apostrophes.
- Control characters
- The following characters that are not valid in the names of objects:

  .,;':/*|?&%$!+=()[]{}<>

Examples of object names

The following table shows examples of some object names:

Object Types	Examples	Comment
Table name	**Sales**	If the table name does not contain spaces or other special characters, the name does not need to be enclosed in quotation marks.
Table name	**'Canada Sales'**	If the name contains spaces, tabs or other special characters, enclose the name in single quotation marks.
Fully qualified column name	**Sales[Amount]**	The table name precedes the column name, and the column name is enclosed in brackets.

Object Types	Examples	Comment
Fully qualified measure name	**Sales[Profit]**	The table name precedes the measure name, and the measure name is enclosed in brackets. In certain contexts, a fully qualified name is always required.
Unqualified column name	**[Amount]**	The unqualified name is just the column name, in brackets. Contexts where you can use the unqualified name include formulas in a calculated column within the same table, or in an aggregation function that is scanning over the same table.
Fully qualified column in table with spaces	**'Canada Sales'[Qty]**	The table name contains spaces, so it must be surrounded by single quotes.

Other restrictions

The syntax required for each function, and the type of operation it can perform, varies greatly depending on the function. In general, however, the following rules apply to all formulas and expressions:

- DAX formulas and expressions cannot modify or insert individual values in tables.
- You cannot create calculated rows by using DAX. You can create only calculated columns and measures.
- When defining calculated columns, you can nest functions to any level.
- DAX has several functions that return a table. Typically, you use the values returned by these functions as input to other functions, which require a table as input.

DAX operators and constants

The following table lists the operators that are supported by DAX. For more information about the syntax of individual operators, see DAX operators.

Operator type	Symbol and use
Parenthesis operator	() precedence order and grouping of arguments
Arithmetic operators	+ (addition) - (subtraction/ sign)

Operator type	Symbol and use
	* (multiplication) / (division) ^ (exponentiation)
Comparison operators	= (equal to) > (greater than) < (less than) >= (greater than or equal to) <= (less than or equal to) <> (not equal to)
Text concatenation operator	& (concatenation)
Logic operators	&& (and) || (or)

Data types

You do not need to cast, convert, or otherwise specify the data type of a column or value that you use in a DAX formula. When you use data in a DAX formula, DAX automatically identifies the data types in referenced columns and of the values that you type in, and performs implicit conversions where necessary to complete the specified operation.

For example, if you try to add a number to a date value, the engine will interpret the operation in the context of the function, and convert the numbers to a common data type, and then present the result in the intended format, a date.

However, there are some limitations on the values that can be successfully converted. If a value or a column has a data type that is incompatible with the current operation, DAX returns an error. Also, DAX does not provide functions that let you explicitly change, convert, or cast the data type of existing data that you have imported into a data model.

ⓘ Important

DAX does not support use of the variant data type. Therefore, when you load or import data into a data model, it's expected the data in each column is generally of a consistent data type.

Some functions return scalar values, including strings, whereas other functions work with numbers, both integers and real numbers, or dates and times. The data type required for each function is described in the section, DAX functions.

You can use tables containing multiple columns and multiple rows of data as the argument to a function. Some functions also return tables, which are stored in memory and can be used as arguments to other functions.

Date and time

DAX stores date and time values using the datetime data type used by Microsoft SQL Server. Datetime format uses a floating-point number where Date values correspond to the integer portion representing the number of days since December 30, 1899. Time values correspond to the decimal portion of a date value where Hours, minutes, and seconds are represented by decimal fractions of a day. DAX date and time functions implicitly convert arguments to datetime data type.

Note

The exact maximum DateTime value supported by DAX is December 31, 9999 00:00:00.

Date and time literal

Beginning with the August 2021 version of Power BI Desktop, DAX date and datetime values can be specified as a literal in the format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"`, or `dt"YYYY-MM-DD hh:mm:ss"`. When specified as a literal, use of DATE, TIME, DATEVALUE, TIMEVALUE functions in the expression are not necessary.

For example, the following expression uses DATE and TIME functions to filter on OrderDate:

DAX

```
EVALUATE
FILTER (
        FactInternetSales,
```

```
        [OrderDate] > (DATE(2015,1,9) + TIME(2,30,0)) &&[OrderDate] <
(DATE(2015,12,31) + TIME(11,59,59))
)
```

The same filter expression can be specified as a literal:

DAX

```
EVALUATE
FILTER (
        FactInternetSales,
        [OrderDate] > dt"2015-1-9T02:30:00" && [OrderDate] < dt"2015-12-
31T11:59:59"
)
```

> **Note**
>
> The DAX date and datetime-typed literal format is not supported in all versions of Power BI Desktop, Analysis Services, and Power Pivot in Excel. New and updated DAX functionality are typically first introduced in Power BI Desktop and then later included in Analysis Services and Power Pivot in Excel.

Feedback

Was this page helpful? Yes No